



แพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพบุคคลด้วยการจดจำใบหน้า
Online Social Media Platform and Portrait Management with Face Recognition

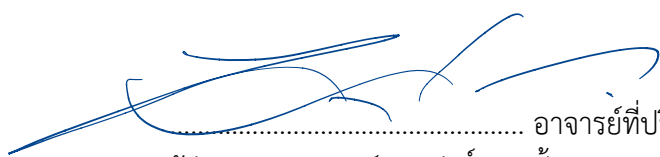
นายสุวิทย์ จันทเสน
นายณัฐชนน ภูเวียง

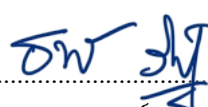
โครงการนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิทยาศาสตรบัณฑิต
สาขาวิชาคณิตศาสตร์เชิงวิทยาการคอมพิวเตอร์ ภาควิชาคณิตศาสตร์ คณะวิทยาศาสตร์ประยุกต์
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ
ปีการศึกษา 2567

ลิขสิทธิ์ของมหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

ชื่อโครงการพิเศษ (ภาษาไทย)	: สื่อสังคมแพลตฟอร์มและการจัดการรูปภาพบุคคล ด้วยการจดจำใบหน้า
ชื่อโครงการพิเศษ (ภาษาอังกฤษ)	: Social Media Platform and Portrait Management with Face Recognition
คำสำคัญ	: การประมวลผลภาพ การจดจำใบหน้า และ การจัดกลุ่มรูปภาพ
จัดทำโดย	: นายสุวิทย์ จันทเสน นายณัฐชนน ภูเวียง
สาขาวิชา	: คณิตศาสตร์เชิงวิทยาการคอมพิวเตอร์
ภาควิชา	: คณิตศาสตร์
คณะ	: วิทยาศาสตร์ประยุกต์
อาจารย์ที่ปรึกษา	: ผู้ช่วยศาสตราจารย์ ดร.ศักดิ์ชาย ตั้งประเสริฐ
ปีการศึกษา	: 2567

ภาควิชาคณิตศาสตร์ คณะวิทยาศาสตร์ประยุกต์ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้า
พระนครเหนือ อนุมัติให้โครงการนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิทยาศาสตรบัณฑิต
สาขาวิชาคณิตศาสตร์เชิงวิทยาการคอมพิวเตอร์


..... อาจารย์ที่ปรึกษา
(ผู้ช่วยศาสตราจารย์ ดร.ศักดิ์ชาย ตั้งประเสริฐ)


..... กรรมการ
(ผู้ช่วยศาสตราจารย์ ดร.ชนวัฒน์ วิเชียรไพศาล)


..... กรรมการ
(ดร.นันทพล อัมอารีรัตน์)

ลิขสิทธิ์ของภาควิชาคณิตศาสตร์ คณะวิทยาศาสตร์ประยุกต์
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ
ปีการศึกษา 2567

บทคัดย่อ

ในยุคที่ข้อมูลภาพบุคคลในสื่อสังคมออนไลน์มีจำนวนเพิ่มขึ้นอย่างต่อเนื่อง การนำเทคโนโลยีการรู้จำใบหน้า (Face Recognition) ซึ่งเป็นส่วนหนึ่งของปัญญาประดิษฐ์ (Artificial Intelligence: AI) เข้ามาช่วยในการจัดการ จัดกลุ่ม และยืนยันตัวตนบุคคลจากภาพ จึงมีความสำคัญยิ่ง โครงการฉบับนี้นำเสนอการพัฒนาแพลตฟอร์มสื่อสังคมออนไลน์ที่สามารถจัดการภาพบุคคลได้แบบครบวงจร โดยใช้กระบวนการประมวลผลภาพดิจิทัล (Digital Image Processing) ควบคู่กับการเรียนรู้ของเครื่อง (Machine Learning) เพื่อฝึก และปรับปรุงแบบจำลองการรู้จำใบหน้า

ระบบประกอบด้วยฟังก์ชันหลัก เช่น การลงทะเบียนผู้ใช้ด้วยใบหน้า การแท็กอัตโนมัติ การค้นหาภาพโดยใช้ใบหน้า การจัดกลุ่มภาพด้วยการเรียนรู้แบบไม่มีผู้สอน (Unsupervised Learning) และระบบตรวจสอบความถูกต้องของภาพด้วยการตรวจจับภาพปลอม (Deepfake Detection) ทั้งนี้ระบบได้รับการออกแบบภายใต้ Microservices Architecture และใช้เทคนิคการเรียนรู้แบบผสม (Ensemble Learning) เพื่อรวมแบบจำลองหลายประเภท เช่น FaceNet, ArcFace และ AdaFace ให้ทำงานร่วมกันอย่างมีประสิทธิภาพ จากการทดสอบพบว่าระบบสามารถจำแนกและตรวจสอบภาพบุคคลได้อย่างแม่นยำกว่าร้อยละ 98 และมีศักยภาพในการนำไปใช้ในงานจริง เช่น ระบบตรวจสอบสิทธิ์ หรือการวิเคราะห์ภาพในองค์กร

คำสำคัญ: การจดจำใบหน้า การประมวลผลภาพ และการจัดกลุ่มรูปภาพ

Abstract

As digital image data continues to grow significantly, the integration of facial recognition technology within artificial intelligence has become essential for managing, grouping, and verifying individuals in images. In the digital era, the sheer volume of personal image data on social media platforms necessitates sophisticated processing and analysis tools. This thesis presents the development of a comprehensive platform that applies digital image processing and machine learning to enhance facial recognition performance.

The system includes core functions such as user registration via facial data, automatic face tagging, image retrieval using face embeddings, image grouping through unsupervised learning, and image authenticity verification via deepfake detection. The architecture is built on a microservices model for scalability and integrates multiple facial recognition models, such as FaceNet, ArcFace, and AdaFace, using ensemble learning techniques to improve accuracy and adaptability. Experimental results demonstrate that the system achieves over 98% accuracy in personal identification and is practical for real-world scenarios like authentication systems and enterprise-level image analysis.

Keywords: Face Recognition, Image Processing and Group Image

สารบัญ

เรื่อง	หน้า
บทคัดย่อ	ก
Abstract	ข
สารบัญ	ค
สารบัญภาพ	จ
สารบัญตาราง	ฉ
กิตติกรรมประกาศ	ฅ
บทที่ 1 บทนำ	1
1.1 ความเป็นมา และความสำคัญของปัญหา	1
1.2 วัตถุประสงค์ของการศึกษา	1
1.3 ขอบเขตของการศึกษา	2
1.4 การดำเนินงาน	2
1.5 ประโยชน์ที่คาดว่าจะได้รับ	3
1.6 คำจำกัดความ	3
บทที่ 2 เอกสาร และงานวิจัยที่เกี่ยวข้อง	4
2.1 งานวิจัยที่เกี่ยวข้อง (Related Works)	4
2.2 เทคโนโลยีการรู้จำใบหน้า (Face Recognition Technology)	8
2.3 ปัญญาประดิษฐ์ และการเรียนรู้ของเครื่อง	12
2.4 รูปแบบการเรียนรู้ของเครื่อง (Learning Paradigms in Machine Learning)	15
2.5 การเรียนรู้แบบผสม (Ensemble Learning)	18
2.6 การประมวลผลภาพดิจิทัล (Digital Image Processing)	24
2.7 สถาปัตยกรรม Microservices (Microservices Architecture)	30
2.8 เทคโนโลยี Web Application (Web Application Technologies)	35
บทที่ 3 ขั้นตอนและวิธีการดำเนินงาน	42
3.1 การวิเคราะห์ปัญหาและความเป็นไปได้ของระบบ	42
3.2 การออกแบบสถาปัตยกรรมระบบ	43
3.3 การวิเคราะห์ระบบ (System Analysis)	46
3.4 แผนภาพกรณีการใช้งานของระบบ (Use Case Diagram)	47
3.5 แผนภาพขั้นตอนการทำงานของระบบ (Activity Diagram)	48
3.6 การออกแบบฐานข้อมูล (Entity Relationship Diagram)	69
3.7 การเลือกแบบจำลอง Face Detection	93
3.8 การเลือกแบบจำลอง Face Recognition	110

สารบัญ (ต่อ)

เรื่อง	หน้า
3.9 การเลือกแบบจำลอง Deepfake Detection	130
บทที่ 4 ผลการดำเนินการ	140
4.1 การประเมินผลอัลกอริทึม	140
4.2 การทำงานของอัลกอริทึม	147
4.3 ผลการพัฒนาระบบแพลตฟอร์มสื่อสังคมออนไลน์ฯ	176
4.4 ผลการทดสอบระบบ	190
บทที่ 5 สรุป และอภิปรายผล	205
5.1 สรุปผลการดำเนินงาน	205
5.2 อภิปรายผลการดำเนินงาน	206
5.3 ข้อเสนอแนะและแนวทางการพัฒนา	208
เอกสารอ้างอิง	206
ภาพผนวก ก ชุดคำสั่ง แพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพบุคคลด้วยการจดจำใบหน้า	215
ภาพผนวก ข คู่มือการติดตั้ง แพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพบุคคลด้วยการจดจำใบหน้า	263
ภาพผนวก ค คู่มือการใช้งาน แพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพบุคคลด้วยการจดจำใบหน้า	275
ภาพผนวก จ แบบฟอร์มประเมินผลการใช้งานแพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพบุคคลด้วยการจดจำใบหน้า	286
ประวัติผู้จัดทำโครงการ	341

สารบัญภาพ

ภาพที่		หน้า
2-1	โครงสร้างของแบบจำลอง FaceNet	9
2-2	แนวคิด Additive Angular Margin ของ ArcFace	10
2-3	แนวคิด Adaptive Margin ของ AdaFace	11
2-4	สรุปหลักการ Elastic Margin Loss ของ ElasticFace	12
2-5	โครงสร้างเครือข่ายประสาทเทียมเชิงลึกแบบคอนโวลูชัน (CNN)	14
2-6	แนวคิด Feature Learning สำหรับการเรียนรู้แบบมีผู้สอน	16
2-7	การทำงานพื้นฐานของ Reinforcement Learning	17
2-8	การทำงานของ Random Forest ที่ใช้เทคนิค Bagging	20
2-9	แผนภาพการทำงานของ AdaBoost	21
2-10	แผนภาพการเรียงซ้อนแบบจำลอง (Stacked Ensemble)	22
2-11	แสดงระบบ Ensemble CNN	23
2-12	กระบวนการประมวลผลภาพดิจิทัลพื้นฐานสำหรับงานวิเคราะห์ภาพ	25
2-13	การเปรียบเทียบภาพใบหน้าก่อนและหลังการปรับ Contrast ด้วย CLAHE	26
2-14	ผลลัพธ์การลดสัญญาณรบกวนด้วย Median Filter	27
2-15	การเปรียบเทียบภาพใบหน้าก่อนและหลังการจัดแนว	28
2-16	ตัวอย่างการตรวจจับขอบภาพด้วยวิธี Canny	29
2-17	เทคนิคการเพิ่มข้อมูลภาพ: ดันฉบับ, พลิกแนวนอน, หมุน และปรับความสว่าง	30
2-18	การเปรียบเทียบสถาปัตยกรรม Monolithic และ Microservices	32
2-19	แผนภาพสถาปัตยกรรม Microservices	34
2-20	สถาปัตยกรรม Client-Server และการสื่อสารผ่าน Protocol HTTP	36
2-21	ความสัมพันธ์ระหว่าง HTML CSS และ JavaScript	37
2-22	กระบวนการพิสูจน์ตัวตนด้วย JSON Web Token	39
2-23	การเปรียบเทียบการสื่อสารระหว่าง HTTP และ WebSocket	40
2-24	สถาปัตยกรรม Web Application แบบหลายส่วน	41
3-1	การวิเคราะห์จุดแข็ง จุดอ่อน โอกาส และอุปสรรคของแพลตฟอร์ม FaceSocial	43
3-2	สถาปัตยกรรมระบบ FaceSocial แบบ Microservices	45
3-3	แผนผังระบบของ FaceSocial	46
3-4	Use Case Diagram ขอ FaceSocial	48
3-5	Activity Diagram ขั้นตอนการลงทะเบียนใบหน้าในระบบ FaceSocial	49
3-6	Requirement Specification การทดสอบใบหน้า (Face Recognition Test)	52
3-7	Activity Diagram ขั้นตอนการวิเคราะห์เพศ และอายุของภาพใบหน้าในระบบ	55
3-8	Activity Diagram ขั้นตอนตรวจสอบความถูกต้องของภาพและวิดีโอในระบบ	57

สารบัญภาพ (ต่อ)

ภาพที่		หน้า
3-9	Activity Diagram ขั้นตอนการโพสต์ และแชร์ภาพในระบบ Social Media	60
3-10	Activity Diagram ขั้นตอนระบบการสนทนา (Chat System)	62
3-11	Activity Diagram ขั้นตอนดูประวัติโดยย่อผู้ใช้ (Profile)	64
3-12	Activity Diagram ขั้นตอนกล้องวงจรปิด (CCTV Monitoring)	66
3-13	แบบจำลองการออกแบบฐานข้อมูล	69
3-14	UML Class Diagram ของระบบ FaceSocial	70
3-15	แผนภาพความของสัมพันธ์โดเมนการจัดการผู้ใช้งาน	71
3-16	แผนภาพความของสัมพันธ์โดเมนการยืนยันตัวตน และความปลอดภัย	72
3-17	แผนภาพความของสัมพันธ์โดเมนการจดจำใบหน้า	73
3-18	แผนภาพความของสัมพันธ์โดเมนแพลตฟอร์มโซเชียล	74
3-19	แผนภาพความของสัมพันธ์โดเมนการเชื่อมต่อทางสังคม	75
3-20	แผนภาพความของสัมพันธ์โดเมนการส่งข้อความ	76
3-21	แผนภาพความของสัมพันธ์ โดเมนการแจ้งเตือน และกิจกรรม	77
3-22	ภาพถ่ายตัวอย่างต้นฉบับของ ชุดข้อมูลภาพ (Image Dataset)	100
3-23	แผนผังกระบวนการทำงานของสคริปต์ทดสอบการครอบตัดใบหน้า	101
3-24	การทดสอบการครอบตัดใบหน้าด้วย blazeface	102
3-25	การทดสอบการครอบตัดใบหน้าด้วย haar_frontal	103
3-26	การทดสอบการครอบตัดใบหน้าด้วย insightface	104
3-27	การทดสอบการครอบตัดใบหน้าด้วย MTCNN	105
3-28	การทดสอบการครอบตัดใบหน้าด้วย yolov9c	106
3-29	การทดสอบการครอบตัดใบหน้าด้วย yolov9e	107
3-30	การทดสอบการครอบตัดใบหน้าด้วย yolov11m	108
3-31	การทดสอบการครอบตัดใบหน้าด้วย yolov11n	109
3-32	ภาพข้อมูล Diverse Asian Facial Ages	112
3-33	การลดลงของ Loss ในระหว่างการฝึกฝน	118
3-34	ความแม่นยำในการ Validation ระหว่างการฝึกฝน	119
3-35	การปรับ Learning Rate	120
3-36	เวลาการฝึกฝน และการใช้ทรัพยากรต่อ Epoch	121
3-37	การวิเคราะห์การ Converge ของแบบจำลอง	122
3-38	ประสิทธิภาพในระหว่าง Thai Fine-tuning Phase	123
3-39	ตัวอย่างชุดข้อมูล Face-Recognition-Dataset	126
3-40	ผลการทดสอบการเปรียบเทียบประสิทธิภาพแบบจำลอง	127

สารบัญภาพ (ต่อ)

ภาพที่	หน้า
3-41 ผลการทดสอบความทนทานต่อประสิทธิภาพ	128
3-42 ตัวอย่างชุดข้อมูล CASIA-FASD ภาพใบหน้าจริง	134
3-43 ตัวอย่างชุดข้อมูล CASIA-FASD ภาพใบหน้าปลอม	134
3-44 เปรียบเทียบความสัมพันธ์ Accuracy และ EER	135
3-45 การเปรียบเทียบ Accuracy	136
3-46 การเปรียบเทียบ Equal Error Rate	137
3-47 การเปรียบเทียบประสิทธิภาพแบบ Multi-Metric	138
4-1 ตัวอย่างชุดข้อมูลภาพทดสอบ	140
4-2 ขั้นตอนการประมวลผลการตรวจจับใบหน้า	148
4-3 สถาปัตยกรรมของแบบจำลอง YOLOv11	150
4-4 เปรียบเทียบสถาปัตยกรรมระหว่าง YOLOv11 และ YOLOv9	152
4-5 ผลการทดสอบการตรวจจับใบหน้าด้วยแบบจำลอง YOLOv11m	153
4-6 ผลการทดสอบการตรวจจับใบหน้าด้วยแบบจำลอง YOLOv9	154
4-7 ขั้นตอนการประมวลผลการจดจำใบหน้า	156
4-8 สถาปัตยกรรมของแบบจำลอง FaceNet	159
4-9 สถาปัตยกรรมของแบบจำลอง ArcFace	161
4-10 สถาปัตยกรรมของแบบจำลอง AdaFace	163
4-11 ผลการทดสอบการจดจำใบหน้าด้วยแบบจำลอง FaceNet	165
4-12 ผลการทดสอบการจดจำใบหน้าด้วยแบบจำลอง ArcFace	166
4-13 ผลการทดสอบการจดจำใบหน้าด้วยแบบจำลอง AdaFace	166
4-14 ขั้นตอนการประมวลผลการตรวจสอบความมีชีวิต	169
4-15 สถาปัตยกรรมของแบบจำลอง MiniFASNetV1	172
4-16 สถาปัตยกรรมของแบบจำลอง MiniFASNetV2	173
4-17 ผลการทดสอบการตรวจสอบความมีชีวิตด้วยแบบจำลอง DeepFace Silent Face Anti-Spoofing	175
4-18 หน้าตอนรับ และอธิบายข้อมูลเกี่ยวกับ ระบบ	177
4-19 ส่วนการดูว่าแบบจำลองตัวไหน และ AI อะไรทำงานอยู่	177
4-20 หน้าการทดสอบ AI	178
4-21 หน้าทดสอบ การตรวจจับใบหน้า (Face Detection)	179
4-22 หน้าทดสอบ การตรวจจับการปลอมแปลง (Anti-Spoofing)	180
4-23 หน้าทดสอบ การวิเคราะห์อายุ และเพศ (Age and Gender Analysis)	181
4-24 หน้า Real-time CCTV	182

สารบัญภาพ (ต่อ)

ภาพที่	หน้า
4-25 หน้าเข้าสู่ระบบ	183
4-26 หน้าเข้าสู่ระบบด้วยใบหน้า	184
4-27 หน้าสมัครสมาชิก และสแกนหน้า	185
4-28 หน้า Dashboard	186
4-29 หน้า Feed	186
4-30 หน้า Chat	187
4-31 หน้าการแจ้งเตือน	187
4-32 หน้ารายชื่อเพื่อน	188
4-33 ส่วนการแสดงประวัติโดยย่อ	188
4-34 หน้าจัดการข้อมูลทั้งหมดของผู้ใช้	189
4-35 ส่วนการแสดงฐานข้อมูล	189
4-36 หน้าต่างสถานการณ์ทำงานของ Docker API AI Service	190
4-37 หน้าต่างสถานการณ์ทำงานของ Docker Frontend	190
4-38 การทำงานของ Face Detection	191
4-39 การตรวจจับการปลอมแปลง (Anti-Spoofing)	192
4-40 การวิเคราะห์อายุ และเพศ (Age and Gender Analysis)	193
4-41 การทำงานของ ระบบสแกนหน้าเพื่อสมัครสมาชิก	194
4-42 การทำงานของ ระบบเข้าสู่ระบบด้วยการสแกนหน้า	196
4-43 การค้นหาเพื่อน (Friend Search)	197
4-44 การแจ้งเตือน (Notification System)	199
4-45 Admin Dashboard	200
4-46 Chat System	200
4-47 การโพสต์ และแท็กใบหน้า (Post and Face Tagging)	202
4-48 Closed-Circuit Television (CCTV)	203
๗-1 หน้าจอการเข้าสู่ระบบเว็บไซต์ GitHub	264
๗-2 หน้าจอการสร้าง Repository	265
๗-3 หน้าจอการตั้งชื่อ Repository	266
๗-4 หน้าจอการคัดลอกโค้ดสำหรับการอัปโหลดไฟล์	267
๗-5 หน้าจอการรันคำสั่ง git init	267
๗-6 หน้าจอการรันคำสั่ง git add	268
๗-7 หน้าจอการรันคำสั่ง git commit -m "first commit"	268
๗-8 หน้าจอการรันคำสั่ง git branch -M main	269

สารบัญภาพ (ต่อ)

ภาพที่	หน้า
ข-9 หน้าจอการรันคำสั่ง git remote add origin	270
ข-10 หน้าจอการรันคำสั่ง git push -u origin main	270
ข-11 หน้าจอไฟล์ที่อัปโหลดในเว็บไซต์ GitHub	271
ข-12 หน้าต่างติดตั้ง Docker บน Windows 11	271
ข-13 หน้าต่าง Docker Desktop สามารถติดตั้งได้	272
ข-14 หน้าต่าง Docker Desktop	272
ข-15 หน้าจอโปรเจกต์ที่โคลนจากที่เก็บข้อมูล (Repository)	273
ข-16 หน้าจอการเริ่มต้นระบบแพลตฟอร์มฯ	274
ค-1 หน้าจอแรกของเว็บไซต์	276
ค-2 หน้าต่างกรอกข้อมูลการสมัครสมาชิก	277
ค-3 หน้าต่างสแกนหน้าเพื่อสมัครสมาชิก	277
ค-4 หน้าต่างข้อความสมัครสมาชิกสำเร็จ	278
ค-5 หน้าจอการกรอกข้อมูลเข้าสู่ระบบแบบทั่วไป	278
ค-6 หน้าจอการสแกนหน้าเพื่อเข้าสู่ระบบ	279
ค-7 หน้าจอข้อความเข้าสู่ระบบสำเร็จ	279
ค-8 หน้า Dashboard	280
ค-9 หน้า Feed	280
ค-10 ฟังก์ชันโพสต์ของหน้า Feed หลังจากกดโพสต์แล้ว	281
ค-11 การโพสต์ข้อความในหน้า Feed	281
ค-12 หน้าเพื่อน	282
ค-13 หน้า Chat	282
ค-14 หน้าการแจ้งเตือน	283
ค-15 ประวัติโดยย่อของผู้ใช้	283
ค-16 หน้าจอการตรวจจับใบหน้า (Face Detection)	284
ค-17 หน้าจอการตรวจจับการปลอมแปลง (Anti-Spoofing)	284
ค-18 หน้าจอการวิเคราะห์อายุและเพศ (Age and Gender Analysis)	285
ค-19 หน้าจอการตรวจจับกล้องวงจรปิดแบบเรียลไทม์ (CCTV Real-Time)	285
ง-1 หน้าแรกยังไม่ได้เข้าสู่ระบบ	288
ง-2 เข้าสู่ระบบ	289
ง-3 หน้าสมัครสมาชิก	290
ง-4 หน้าทดลองฟีเจอร์ AI	291
ง-5 หน้าเข้าสู่ระบบ	294

สารบัญภาพ (ต่อ)

ภาพที่		หน้า
ง-6	หน้า Dashboardของผู้ใช้	295
ง-7	หน้าเข้าสู่ระบบที่กรอกชื่อผู้ใช้หรือรหัสผ่านผิด	296
ง-8	หน้าเข้าสู่ระบบที่ไม่กรอกชื่อผู้ใช้หรือรหัสผ่าน	297
ง-9	หน้าต่างสแกนใบหน้าเพื่อเข้าสู่ระบบ	298
ง-10	หน้าต่างสแกนใบหน้าผู้ใช้ที่ลงทะเบียนไว้เพื่อเข้าสู่ระบบ	300
ง-11	หน้าต่างสแกนใบหน้าผู้ใช้ที่ไม่ได้ลงทะเบียนไว้	301
ง-12	หน้าต่างสแกนหน้าตรวจสอบการป้องกันภาพปลอม	302
ง-13	หน้าจอแจ้งเตือนเมื่อมีข้อมูลซ้ำกับฐานข้อมูล (Database)	304
ง-14	หน้าจอแจ้งเตือนเมื่อกรอกข้อมูลไม่ตรงกับรูปแบบที่กำหนดไว้	305
ง-15	หน้าจอแจ้งเตือนเมื่อไม่กรอกข้อมูล	306
ง-16	หน้าสแกนใบหน้าเพื่อความปลอดภัย	307
ง-17	หน้าจอแจ้งเตือนการลงทะเบียนสมาชิกสำเร็จ	309
ง-18	หน้าจอแจ้งเตือนผู้ใช้นี้มีในระบบแล้ว	310
ง-19	หน้าจอผลการตรวจจับใบหน้า	315
ง-20	หน้าจอแจ้งเตือนไม่พบใบหน้า	316
ง-21	หน้าจอการตรวจจับใบหน้าแบบเรียลไทม์	317
ง-22	หน้าจอผลการตรวจสอบภาพจริง	318
ง-23	หน้าจอผลการตรวจสอบภาพปลอมแปลง	319
ง-24	หน้าจอการตรวจจับภาพปลอมแบบเรียลไทม์	320
ง-25	หน้าจอผลการวิเคราะห์อายุและเพศ	321
ง-26	หน้าจอแจ้งเตือนในภาพที่ไม่มีใบหน้า	322
ง-27	หน้าจอกล้องวงจรปิดที่ตรวจจับและระบุตัวตนใบหน้า	323
ง-28	หน้า Dashboard	326
ง-29	หน้า Feed	327
ง-30	หน้าเพื่อน	328
ง-31	หน้า Chat	329
ง-32	หน้าการแจ้งเตือน	330
ง-33	หน้าประวัติโดยย่อ	331
ง-34	หน้าจอปุ่มสร้างโพสต์	333
ง-35	หน้าสร้างโพสต์	334
ง-36	หน้าจอการกรอกเนื้อหาเพื่อทดสอบสร้างโพสต์	335
ง-37	หน้าจอการกดปุ่มโพสต์	336

สารบัญภาพ (ต่อ)

ภาพที่		หน้า
ง-38	หน้าจอการสร้างโพสต์สำเร็จ	337
ง-39	หน้าจอการเลือกรูปภาพ	338
ง-40	หน้าจอการแท็กเพื่อน	339
ง-41	หน้าจอระบบแจ้งเตือนผู้ใช้ที่มีในระบบอัตโนมัติ	340

สารบัญตาราง

ตารางที่		หน้า
3-1	สรุป Use Case Diagram	47
3-2	Requirement Specification: การลงทะเบียนใบหน้าใหม่	50
3-3	Requirement Specification การทดสอบใบหน้า (Face Recognition Test)	53
3-4	Requirement Specification การวิเคราะห์เพศ และอายุ	56
3-5	Requirement Specification การตรวจสอบ Deepfake และ Liveness	58
3-6	Requirement Specification การโพสต์ และแชร์ภาพ	61
3-7	Requirement Specification ระบบการสนทนา (Chat System)	63
3-8	Requirement Specification ประวัติโดยย่อผู้ใช้ (Profile)	65
3-9	Requirement Specification ระบบกล้องวงจรปิด (CCTV Monitoring)	67
3-10	พจนานุกรมผู้ใช้งาน (User)	80
3-11	พจนานุกรมประวัติโดยย่อผู้ใช้งาน (User)	81
3-12	พจนานุกรมการตั้งค่าผู้ใช้งาน (User Settings)	82
3-13	พจนานุกรมเซสชันผู้ใช้งาน (User Session)	82
3-14	พจนานุกรมการเข้าสู่ระบบ (Login Attempt)	83
3-15	พจนานุกรมโทเค็นรีเซ็ตรหัสผ่าน (Password Reset Token)	84
3-16	พจนานุกรมโทเค็นยืนยันอีเมล (Email Verification Token)	84
3-17	พจนานุกรมข้อมูลใบหน้า (Face Data)	84
3-18	พจนานุกรมข้อมูลการฝึกใบหน้า (Face Training Data)	85
3-19	พจนานุกรมโพสต์ (Post)	86
3-20	พจนานุกรมแท็กใบหน้าในโพสต์ (Face Tag in Post)	86
3-21	พจนานุกรมไลค์โพสต์ (Post Like)	87
3-22	พจนานุกรมคอมเมนต์โพสต์ (Post Comment)	87
3-23	พจนานุกรมไลค์คอมเมนต์ (Comment Like)	88
3-24	พจนานุกรมการเชื่อมต่อผู้ใช้งาน (User Connection)	88
3-25	พจนานุกรมการสนทนา (Conversation)	89
3-26	พจนานุกรมผู้เข้าร่วมการสนทนา (Conversation Participant)	89
3-27	พจนานุกรมข้อความ (Message)	90
3-28	พจนานุกรมแท็กใบหน้าในข้อความ (Face Tag in Message)	91
3-29	พจนานุกรมรีแอคชันข้อความ (Message Reaction)	91
3-30	พจนานุกรมการแจ้งเตือน (Notification)	91
3-31	พจนานุกรมกิจกรรมผู้ใช้งาน (User Activity)	92
3-32	การเลือกแบบจำลอง Face Detection	94

สารบัญตาราง (ต่อ)

ตารางที่	หน้า
3-33 รายละเอียดชุดข้อมูลทดสอบ Face Detection	95
3-34 การแบ่งตามระดับความยากในการตรวจจับ	96
3-35 ผลการเปรียบเทียบประสิทธิภาพแบบจำลอง Face Detection ผลการเปรียบเทียบประสิทธิภาพแบบจำลอง Face Detection ในการครอบ	97
3-36 ตัดใบหน้า	110
3-37 รายละเอียดชุดข้อมูล Diverse Asian Facial Ages	111
3-38 ชุดข้อมูลทดสอบมาตรฐานที่ใช้ในการประเมิน	114
3-39 ชุดข้อมูลทดสอบภาพคนไทย	115
3-40 ชุดข้อมูลทดสอบสภาพแวดล้อมทำทนาย	115
3-41 ผลการประเมินความแม่นยำบนชุดข้อมูลมาตรฐาน	115
3-42 ผลการประเมินบนชุดข้อมูลภาพคนไทย	116
3-43 ประสิทธิภาพการประมวลผลของแบบจำลองต่างๆ	116
3-44 ความแกร่งต่อสภาพแวดล้อมทำทนาย	117
3-45 เปรียบเทียบกระบวนการฝึกฝนแบบจำลอง	117
3-46 สรุปเมตริกการฝึกฝนของแบบจำลองทั้งหมด	124
3-47 การประเมินแบบถ่วงน้ำหนักของแบบจำลอง	125
3-48 ประสิทธิภาพมาตรฐาน	129
3-49 ประสิทธิภาพความทนทาน	130
3-50 ผลการทดสอบประสิทธิภาพแบบจำลอง	135
4-1 ผลการเปรียบเทียบประสิทธิภาพเชิงปริมาณของแบบจำลอง Face Detection ผลการเปรียบเทียบประสิทธิภาพแบบจำลอง Face Detection ในการครอบ	142
4-2 ตัดใบหน้า	143
4-3 สรุปเมตริกการฝึกฝนของแบบจำลองทั้งหมด	144
4-4 การประเมินแบบถ่วงน้ำหนักของแบบจำลอง	144
4-5 ประสิทธิภาพมาตรฐาน	145
4-6 ประสิทธิภาพความทนทาน	145
4-7 ผลการทดสอบประสิทธิภาพแบบจำลอง Deepfake Detection	146
4-8 การเลือกอัลกอริทึมที่เหมาะสม	147
ง-1 หน้าแรกที่ยังไม่มีการเข้าสู่ระบบ	287
ง-2 เข้าสู่ระบบ	292
ง-3 เข้าสู่ระบบด้วยใบหน้า	299

สารบัญตาราง (ต่อ)

ตารางที่		หน้า
ง-4	หน้าสมัครสมาชิก	303
ง-5	หน้าสแกนหน้าเพื่อความปลอดภัย	308
ง-6	หน้า AI Testing Lab	311
ง-7	Navigation Menu	324
ง-8	สร้างโพสต์	332

กิตติกรรมประกาศ

ข้าพเจ้าขอกราบขอบพระคุณ ผู้ช่วยศาสตราจารย์ ดร.ศักดิ์ชาย ตั้งประเสริฐ อาจารย์ที่ปรึกษาโครงงานหลัก ที่ได้ให้คำแนะนำอย่างใกล้ชิด ให้ข้อเสนอแนะอันมีคุณค่ารวมถึงให้การสนับสนุนทั้งด้านวิชาการและด้านการเงินในช่วงเวลาสำคัญของการดำเนินงานวิจัย ขอขอบพระคุณ ผู้ช่วยศาสตราจารย์ ดร.ธนวัฒน์ วิเชียรไพศาล และอาจารย์ ดร.นันทพล ธีมอารีย์รัตน์ ในฐานะคณะกรรมการสอบโครงงาน ที่ได้ให้คำชี้แนะอย่างละเอียดและเป็นประโยชน์ยิ่งต่อการปรับปรุงผลงาน รวมถึงคณาจารย์ภาควิชาคณิตศาสตร์และคณะวิทยาศาสตร์ประยุกต์ ที่มีส่วนในการถ่ายทอดความรู้และทักษะที่จำเป็นสำหรับการพัฒนาผลงานวิจัยฉบับนี้ ข้าพเจ้าขอแสดงความขอบคุณอย่างสุดซึ้งต่อ บิดา มารดา และครอบครัว ที่ให้การสนับสนุนทั้งด้านกำลังใจและด้านการเงินมาโดยตลอด ซึ่งเป็นแรงผลักดันสำคัญที่ทำให้สามารถดำเนินการศึกษาและวิจัยจนแล้วเสร็จได้อย่างมีประสิทธิภาพ ขอขอบคุณเพื่อนร่วมรุ่นและผู้มีส่วนเกี่ยวข้องทุกท่าน ที่ให้ความช่วยเหลือ แบ่งปันประสบการณ์ และร่วมแรงร่วมใจตลอดช่วงระยะเวลาการศึกษา ขอขอบคุณทุกภาคส่วนที่มีส่วนสนับสนุนให้การทำโครงงานฉบับนี้สำเร็จสมบูรณ์ตามเป้าหมาย

นายสุวิทย์ จันทเสน

นายณัฐชนน ภูเวียง

บทที่ 1

บทนำ

ในยุคที่ข้อมูลภาพบุคคลในสื่อสังคมออนไลน์เพิ่มขึ้นอย่างต่อเนื่อง การนำเทคโนโลยีการรู้จำใบหน้า (Face Recognition) ซึ่งเป็นส่วนหนึ่งของปัญญาประดิษฐ์ (Artificial Intelligence: AI) มาใช้เพื่อจัดการและระบุตัวบุคคลจึงมีความสำคัญอย่างยิ่ง งานวิจัยนี้มุ่งพัฒนาแพลตฟอร์มที่สามารถจัดการภาพบุคคลได้อย่างมีประสิทธิภาพ ครอบคลุมทั้งการแท็กอัตโนมัติ ค้นหาภาพ และตรวจสอบความถูกต้องของภาพภายใต้สถาปัตยกรรม Microservices ผู้ศึกษาได้กำหนดหัวข้อย่อยของบทนี้ดังนี้

- 1.1 ความเป็นมา และความสำคัญของปัญหา
- 1.2 วัตถุประสงค์ของการศึกษา
- 1.3 ขอบเขตของการศึกษา
- 1.4 การดำเนินงาน
- 1.5 ประโยชน์ที่คาดว่าจะได้รับ
- 1.6 คำจำกัดความ

1.1 ความเป็นมา และความสำคัญของปัญหา

ในยุคที่ข้อมูลภาพบุคคลในสื่อสังคมออนไลน์มีจำนวนมากขึ้นอย่างรวดเร็ว เทคโนโลยีการรู้จำใบหน้า (Face Recognition) ซึ่งเป็นส่วนหนึ่งของปัญญาประดิษฐ์ (Artificial Intelligence: AI) ได้กลายเป็นเครื่องมือสำคัญในการระบุตัวบุคคลจากภาพถ่าย และวิดีโอ ทั้งในด้านความปลอดภัย และการจัดการข้อมูลส่วนบุคคล โครงการฉบับนี้มุ่งพัฒนาแพลตฟอร์มโปรแกรมบนเว็บไซต์ที่ใช้การประมวลผลภาพแบบเลข (Digital Image Processing) และการเรียนรู้ของเครื่อง (Machine Learning) เพื่อให้สามารถลงทะเบียน ตรวจสอบตัวตน และแท็กชื่อผู้ใช้งานได้โดยอัตโนมัติ

1.2 วัตถุประสงค์ของการศึกษา

การศึกษานี้มีวัตถุประสงค์เพื่อพัฒนาแพลตฟอร์มสื่อสังคมออนไลน์ที่สามารถจัดการรูปภาพบุคคลด้วยเทคโนโลยีการรู้จำใบหน้า (Face Recognition) และเพิ่มความปลอดภัยของข้อมูลผ่านการตรวจสอบภาพปลอม (Deepfake Detection) โดยมีรายละเอียดดังนี้

- 1.2.1 เพื่อพัฒนาแพลตฟอร์มโปรแกรมบนเว็บไซต์ที่สามารถลงทะเบียนผู้ใช้ และตรวจสอบตัวตนผ่านภาพใบหน้า
- 1.2.2 เพื่อออกแบบระบบประมวลผลภาพที่สามารถจัดกลุ่มภาพใบหน้า และแท็กผู้ใช้งานได้โดยอัตโนมัติ
- 1.2.3 เพื่อพัฒนาโมดูลที่สามารถรองรับไฟล์รูปภาพมาตรฐาน เช่น JPG และ PNG เป็นต้น สำหรับการรู้จำใบหน้า

1.2.4 เพื่อใช้เทคนิคการเรียนรู้ของเครื่อง และการเรียนรู้แบบไม่มีผู้สอนในการจำแนก และจัดกลุ่มภาพใบหน้า

1.2.5 เพื่อพัฒนาระบบภายใต้โครงสร้างสถาปัตยกรรม Microservices ที่สามารถขยาย และบำรุงรักษาได้ง่าย

1.3 ขอบเขตของการศึกษา

การศึกษาค้างนี้มีขอบเขตเฉพาะในด้านการออกแบบ พัฒนา และทดสอบแพลตฟอร์มต้นแบบสำหรับการจัดการรูปภาพบุคคลบนสื่อสังคมออนไลน์ โดยมีรายละเอียดดังนี้

1.3.1 ระบบนี้เป็นการพัฒนาเว็บแอปพลิเคชันโดยใช้ภาษา Hypertext Markup Language (HTML) and Cascading Style Sheets (CSS) และภาษา Python สำหรับฝั่งเซิร์ฟเวอร์

1.3.2 ระบบสามารถประมวลผลภาพเพื่อตรวจสอบตัวตนผู้ใช้งาน และแท็กชื่อบุคคลที่เป็นสมาชิกของแพลตฟอร์มได้อย่างแม่นยำ

1.3.3 รองรับการอัปโหลด และประมวลผลไฟล์ภาพในรูปแบบมาตรฐาน เช่น JPG และ PNG สำหรับการรู้จำใบหน้า

1.3.4 ระบบได้รับการออกแบบเป็นเว็บแอปพลิเคชันที่สามารถทำงานได้บนเว็บเบราว์เซอร์ โดยไม่ต้องติดตั้งซอฟต์แวร์เพิ่มเติม

1.3.5 ขอบเขตของการทดสอบเน้นเฉพาะการใช้งานภายในสภาพแวดล้อมจำลอง และยังไม่เปิดใช้งานในระบบออนไลน์เชิงพาณิชย์

1.4 การดำเนินงาน

การดำเนินงานในครั้งนี้นำแบ่งออกเป็นลำดับขั้นตอนอย่างเป็นระบบ โดยมุ่งเน้นการออกแบบ พัฒนา และทดสอบการทำงานของแพลตฟอร์มในสถานะใช้งานจริง เพื่อให้ระบบสามารถจัดการรูปภาพบุคคลได้อย่างมีประสิทธิภาพ ดังนี้

1.4.1 ศึกษาแนวคิดเทคโนโลยีที่เกี่ยวข้อง ได้แก่ การรู้จำใบหน้า (Face Recognition) การตรวจจับภาพปลอม (Deepfake Detection) และสถาปัตยกรรมไมโครเซอร์วิส (Microservices Architecture)

1.4.2 ออกแบบสถาปัตยกรรมระบบทั้งในส่วนของผู้ใช้ (Frontend), เซิร์ฟเวอร์หลัก (Backend) และบริการรู้จำใบหน้า (AI Services)

1.4.3 พัฒนาระบบต้นแบบโดยใช้โมเดลรู้จำใบหน้าหลายแบบ เช่น FaceNet, ArcFace และ AdaFace พร้อมปรับแต่งด้วยเทคนิค Ensemble Learning

1.4.4 ทดสอบระบบในแง่ความถูกต้อง ความเร็วในการประมวลผล และความสามารถในการจัดการข้อมูลภาพในสถานการณ์จำลองการใช้งานจริง

1.4.5 ประเมินผล และจัดทำรายงานผลการดำเนินงาน พร้อมสรุปข้อเสนอแนะสำหรับการพัฒนาระบบในอนาคต

1.5 ประโยชน์ที่คาดว่าจะได้รับ

ผลจากการศึกษาครั้งนี้คาดว่าจะก่อให้เกิดประโยชน์ในด้านต่าง ๆ ทั้งในเชิงเทคโนโลยี และการประยุกต์ใช้ในชีวิตจริง ดังนี้

1.5.1 ได้แพลตฟอร์มต้นแบบที่สามารถจัดการภาพบุคคลด้วยเทคโนโลยีการรู้จำใบหน้า (Face Recognition) อย่างครบวงจร

1.5.2 ได้ระบบตรวจสอบภาพปลอมเบื้องต้น (Deepfake Detection) ที่ช่วยเพิ่มความปลอดภัยของข้อมูลภาพในระบบ

1.5.3 ได้แนวทางการออกแบบระบบด้วยสถาปัตยกรรม Microservices ที่สามารถนำไปต่อยอดสำหรับระบบขนาดใหญ่หรือใช้งานในระดับองค์กร

1.5.4 ได้องค์ความรู้ด้านการผสมผสานการทำงานของแบบจำลอง ปัญญาประดิษฐ์ (Artificial Intelligence: AI) หลายตัวร่วมกันผ่านเทคนิค Ensemble Learning เพื่อเพิ่มประสิทธิภาพของระบบรู้จำใบหน้า.

1.6 คำจำกัดความ

ในการศึกษานี้มีการใช้คำศัพท์เฉพาะบางคำ เพื่อความชัดเจนในการทำความเข้าใจ จึงขอให้คำจำกัดความไว้ดังนี้

1.6.1 การรู้จำใบหน้า (Face Recognition) เทคโนโลยีที่ใช้ในการระบุตัวบุคคลจากภาพใบหน้าโดยอาศัยการวิเคราะห์ลักษณะทางชีวภาพของใบหน้า และเปรียบเทียบกับข้อมูลที่จัดเก็บไว้ในระบบฐานข้อมูล (Zhao et al., 2003)

1.6.2 ปัญญาประดิษฐ์ (Artificial Intelligence: AI) สาขาวิชาทางคอมพิวเตอร์ที่มุ่งให้เครื่องจักรสามารถเลียนแบบการคิด ตัดสินใจ และเรียนรู้เหมือนมนุษย์ (Russell and Norvig, 2016)

1.6.3 การตรวจจับภาพปลอม (Deepfake Detection) เทคนิคในการตรวจสอบภาพหรือวิดีโอที่ถูกปลอมแปลงด้วย AI เพื่อระบุความผิดปกติจากภาพจริง เช่น ความไม่สอดคล้องของพื้นผิวหรือการเคลื่อนไหวของใบหน้า เป็นต้น (Agarwal et al., 2019)

1.6.4 สถาปัตยกรรม Microservices แนวทางการออกแบบโปรแกรมที่แบ่งระบบออกเป็นบริการย่อย ๆ ซึ่งสามารถพัฒนา และปรับใช้แยกกันได้อย่างอิสระ (Newman, 2015)

1.6.5 การเรียนรู้แบบผสม (Ensemble Learning) เทคนิคการรวมผลลัพธ์ของแบบจำลอง Machine Learning หลายตัวเพื่อเพิ่มประสิทธิภาพการจำแนกหรือพยากรณ์ (Zhou, 2012)

บทที่ 2

เอกสาร และงานวิจัยที่เกี่ยวข้อง

ในการศึกษาครั้งนี้ ผู้วิจัยได้ศึกษาทฤษฎี แนวคิดพื้นฐาน และงานวิจัยที่เกี่ยวข้องเพื่อประกอบในการออกแบบ และพัฒนาแพลตฟอร์มสื่อสังคมออนไลน์สำหรับการจัดการรูปภาพบุคคลด้วยเทคโนโลยีการรู้จำใบหน้า ซึ่งเนื้อหาในบทนี้แบ่งออกเป็นหัวข้อสำคัญดังนี้

- 2.1 งานวิจัยที่เกี่ยวข้อง (Related Works)
- 2.2 เทคโนโลยีการรู้จำใบหน้า (Face Recognition Technology)
- 2.3 ปัญญาประดิษฐ์ และการเรียนรู้ของเครื่อง (Artificial Intelligence and Machine Learning)
- 2.4 รูปแบบการเรียนรู้ของเครื่อง (Learning Paradigms in Machine Learning)
- 2.5 การเรียนรู้แบบผสม (Ensemble Learning)
- 2.6 การประมวลผลภาพแบบเลข (Digital Image Processing)
- 2.7 สถาปัตยกรรม Microservices (Microservices Architecture)
- 2.8 เทคโนโลยีโปรแกรมบนเว็บไซต์ (Web Application Technologies)

2.1 งานวิจัยที่เกี่ยวข้อง (Related Works)

นิลธดา (2557) ได้ศึกษาการพัฒนากระบวนการจดจำใบหน้าที่ใช้ขั้นตอนวิธีการวิเคราะห์องค์ประกอบหลัก (Principal Component Analysis) ร่วมกับการวิเคราะห์ลักษณะเฉพาะของใบหน้า เช่น รูปแบบเลขฐานสองท้องถิ่น (Local Binary Patterns) เป็นต้น เพื่อเพิ่มความแม่นยำในการจำแนกบุคคล ระบบดังกล่าวได้รับการทดสอบกับชุดข้อมูลใบหน้าภายใต้สภาวะแสง และท่าทางที่จำกัด ผลการทดลองแสดงให้เห็นว่าเทคนิคนี้สามารถจดจำใบหน้าได้อย่างมีประสิทธิภาพในสภาพแวดล้อมที่ควบคุม แต่ประสิทธิภาพจะลดลงเมื่อเผชิญกับการเปลี่ยนแปลงของแสงหรือมุมกล้อง ซึ่งบ่งชี้ถึงความจำเป็นในการปรับปรุงวิธีการเพื่อความเสถียรในสถานการณ์ที่หลากหลาย

Viola and Jones (2001) ได้นำเสนอวิธีการตรวจจับใบหน้าแบบทันทีทันใด (Real-time) โดยใช้คุณลักษณะ Haar และเทคนิคการเพิ่มกำลังแบบปรับตัว (AdaBoost) ในการคัดเลือกคุณลักษณะที่สำคัญ วิธีการนี้สามารถตรวจจับใบหน้าได้อย่างรวดเร็ว และมีประสิทธิภาพสูงในยุคนั้น กลายเป็นรากฐานสำคัญของเทคโนโลยีตรวจจับใบหน้าสมัยใหม่ และถูกนำไปใช้อย่างแพร่หลายในระบบต่าง ๆ เนื่องจากมีความเร็วในการประมวลผลเหนือกว่าวิธีการก่อนหน้านี้อย่างชัดเจน

Taigman et.al, (2014) จากทีมวิจัยของ Facebook ได้พัฒนาระบบจดจำใบหน้าชื่อ DeepFace ที่ใช้โครงข่ายประสาทเทียมเชิงลึกควบคู่กับการปรับภาพใบหน้าให้ตรงตามมุมมองสามมิติก่อนการรู้จำ ระบบ DeepFace นี้ผ่านการฝึกด้วยภาพใบหน้าจำนวนหลายล้านภาพ ทำให้สามารถระบุตัวตนบุคคลจากใบหน้าได้ด้วยความแม่นยำในระดับใกล้เคียงกับมนุษย์ ผลงานนี้ถือเป็นการปิดช่องว่างระหว่างประสิทธิภาพของคอมพิวเตอร์กับมนุษย์ในการตรวจสอบใบหน้า และเป็นหมุดหมายสำคัญที่แสดงให้เห็นว่าปัญญาประดิษฐ์สามารถทำงานเทียบเคียงมนุษย์ได้ในงานจดจำใบหน้า

Schroff et.al, (2015) ได้นำเสนอแบบจำลอง (Model) FaceNet ซึ่งเป็นแนวทางใหม่ในการรู้จำใบหน้าโดยการเรียนรู้ให้ได้เวกเตอร์คุณลักษณะของใบหน้า (Face Embedding) แทนการจำแนกประเภทแบบเดิม แบบจำลองนี้ใช้หลักการการสูญเสียแบบสามส่วน (Triplet Loss) ในการฝึกเครือข่ายเพื่อจัดระยะห่างระหว่างเวกเตอร์ใบหน้าของคนเดียวกัน และคนต่างกันอย่างเหมาะสม ส่งผลให้ภาพใบหน้าของบุคคลเดียวกันมีเวกเตอร์ที่ใกล้เคียงกัน และแตกต่างอย่างชัดเจนจากบุคคลอื่น ๆ ผลการทดลองแสดงว่า FaceNet สามารถบรรลุความแม่นยำสูงเป็นอันดับต้น ๆ ในการระบุและยืนยันตัวบุคคลบนชุดข้อมูลมาตรฐาน เช่น LFW และ YouTube Faces เป็นต้น ซึ่งนับเป็นความก้าวหน้าสำคัญของวงการในขณะนั้น

Zhang et.al, (2016) ได้พัฒนาเครือข่ายประสาทแบบหลายงานสำหรับการตรวจจับ และจัดตำแหน่งใบหน้าอัตโนมัติที่เรียกว่า โครงข่ายประสาทแบบคาสเคดหลายงาน (Multitask Cascaded Convolutional Networks) โดยแบบจำลองโครงข่ายประสาทแบบคาสเคดหลายงาน ประกอบด้วยโครงข่ายคาสเคดสามชั้น ที่ทำการค้นหาใบหน้า และปรับตำแหน่งจุดสำคัญบนใบหน้า (Facial Landmarks) ไปพร้อมกัน วิธีการนี้ช่วยให้การตรวจจับใบหน้าที่มีความแม่นยำ และเสถียรมากขึ้น แม้ในกรณีที่ใบหน้าที่มีการเอียงหรือการแสดงออกทางสีหน้าที่หลากหลาย เมื่อเปรียบเทียบกับวิธีดั้งเดิม โครงข่ายประสาทแบบคาสเคดหลายงาน สามารถจัดการกับความซับซ้อนของภาพ และสภาวะแวดล้อมที่แตกต่างกันได้ดียิ่งขึ้น

Deng et.al, (2019) ได้นำเสนอ Retina Face ซึ่งเป็นวิธีการตรวจจับใบหน้าแบบยิงครั้งเดียว (Single Shot) ที่ผสมผสานการทำนายกรอบใบหน้าพร้อมกับจุดสำคัญบนใบหน้า (Landmark) ภายในแบบจำลองเดียว วิธีนี้ช่วยให้ระบบสามารถระบุตำแหน่งใบหน้าที่ได้อย่างแม่นยำ และละเอียดขึ้นบนภาพถ่ายที่มีความซับซ้อนในสภาพแวดล้อมจริง Retina Face แสดงประสิทธิภาพสูงในการตรวจจับใบหน้าบนชุดข้อมูล WIDER FACE โดยให้ความแม่นยำเหนือกว่าวิธีที่มีอยู่ก่อนหน้านี้ ขณะที่ยังคงความเร็วในการประมวลผลในระดับที่เหมาะสมสำหรับการใช้งานจริง

Guo et.al, (2021) ได้พัฒนาการกระจายตัวอย่างและการคำนวณใหม่เพื่อการตรวจจับใบหน้าที่มีประสิทธิภาพ (Sample and Computation Redistribution for Efficient Face Detection) ซึ่งเป็นแบบจำลองตรวจจับใบหน้าที่เน้นความมีประสิทธิภาพสูงทั้งด้านความแม่นยำ และความเร็ว การกระจายตัวอย่างและการคำนวณใหม่เพื่อการตรวจจับใบหน้าที่มีประสิทธิภาพ ใช้วิธีการ Sample Redistribution และ Computation Redistribution ในการฝึกแบบจำลอง เพื่อปรับสมดุลข้อมูลฝึก และการใช้ทรัพยากรการคำนวณในโครงสร้างของแบบจำลองใหม่ ผลการทดลองบนชุดข้อมูล WIDER FACE ระบุว่า การกระจายตัวอย่างและการคำนวณใหม่เพื่อการตรวจจับใบหน้าที่มีประสิทธิภาพ สามารถบรรลุความสมดุลระหว่างความแม่นยำ และความเร็วได้ดีกว่าแบบจำลองตรวจจับใบหน้าที่ก่อนหน้านี้ โดย การกระจายตัวอย่างและการคำนวณใหม่เพื่อการตรวจจับใบหน้าที่มีประสิทธิภาพ สามารถตรวจจับใบหน้าได้แม่นยำในระดับล้ำสมัย (State-of-the-art) ขณะที่ใช้เวลา และพลังประมวลผลน้อยกว่า จึงเหมาะสมสำหรับระบบที่ต้องตรวจจับใบหน้าจำนวนมากแบบทันทีทันใด

Deng et.al, (2019) ได้พัฒนาเทคนิค ArcFace ซึ่งเป็นวิธีปรับปรุงการเรียนรู้คุณลักษณะใบหน้าด้วยการเพิ่มค่ากำกับเชิงมุม (Additive Angular Margin) ในฟังก์ชันความสูญเสียของแบบจำลอง แนวคิดนี้ทำให้เวกเตอร์ใบหน้าที่ได้จากแบบจำลองมีระยะห่างระหว่างคนต่างคนที่มากขึ้น และระยะใกล้เคียงระหว่างภาพของคนคนเดียวกัน ส่งผลให้แบบจำลองสามารถแยกแยะบุคคลต่าง ๆ ได้อย่างมีประสิทธิภาพยิ่งขึ้น ArcFace สามารถสร้างผลงานได้อย่างยอดเยี่ยมบนการทดสอบชุดข้อมูลมาตรฐาน เช่น LFW และ Megaface เป็นต้น โดยมีความแม่นยำสูงเป็นอันดับต้น ๆ แนวทาง ArcFace นี้กลายเป็นมาตรฐานใหม่ที่ถูกนำมาใช้ในงานวิจัย และพัฒนาระบบจดจำใบหน้าสมัยใหม่อย่างแพร่หลาย

Pan et.al, (2007) ได้เสนอวิธีการตรวจสอบความมีชีวิตของใบหน้าโดยการกะพริบตา เพื่อป้องกันการปลอมแปลงด้วยภาพนิ่งของใบหน้า เทคนิคนี้อาศัยการตรวจจับการกะพริบตาตามธรรมชาติของมนุษย์จากวิดีโอใบหน้า โดยใบหน้าจริงจะมีการกะพริบตาเป็นช่วง ๆ อย่างสม่ำเสมอ ขณะที่ใบหน้าปลอมซึ่งเป็นภาพถ่ายจะไม่มีการกะพริบตา วิธีการดังกล่าวสามารถทำงานได้แบบทันทีทันใดด้วยกล้องเว็บแคมทั่วไป และเป็นหนึ่งในแนวทางแรก ๆ ที่นำลักษณะพฤติกรรมตามธรรมชาติมาใช้แยกแยะใบหน้าจริงกับใบหน้าปลอมอย่างมีประสิทธิภาพ

Boulkenafet et.al, (2016) ได้พัฒนาเทคนิคการตรวจจับการปลอมแปลงใบหน้าด้วยการวิเคราะห์เนื้อผิวและสี (Color Texture Analysis) โดยอาศัยคุณลักษณะเชิงพื้นผิว และสีเพื่อแยกความแตกต่างระหว่างใบหน้าจริงกับใบหน้าปลอม เช่น ภาพพิมพ์หรือภาพบนหน้าจอ เป็นต้น ผลการวิจัยแสดงให้เห็นว่าการผสมผสานข้อมูลด้านสี และพื้นผิวสามารถเพิ่มความแม่นยำในการตรวจจับการปลอมแปลงใบหน้าได้อย่างมาก เมื่อทดสอบกับชุดข้อมูลการปลอมแปลงใบหน้าหลากหลายรูปแบบ วิธีการนี้สามารถจำแนกใบหน้าที่ถูกปลอมแปลงได้อย่างแม่นยำ และมีความทนทานต่อการเปลี่ยนแปลงของสภาวะแสง และอุปกรณ์ที่ใช้ในการปลอมแปลง

Atoum et.al, (2017) ได้นำเสนอวิธีการตรวจจับใบหน้าปลอมด้วยโครงข่ายประสาทลึกสองสาย (Two-stream Convolutional Neural Network) โดยสายหนึ่งทำการสร้างแผนที่ความลึก (Depth Map) ของใบหน้าจากการวิเคราะห์ภาพใบหน้าเป็นส่วนย่อย ๆ ในขณะที่อีกสายหนึ่งวิเคราะห์คุณลักษณะภาพใบหน้าทั้งหมดแบบครบถ้วน วิธีนี้ช่วยให้ระบบสามารถตรวจจับการปลอมแปลงได้ทั้งการโจมตีด้วยรูปถ่ายนิ่ง และวิดีโอรีเพลย์อย่างมีประสิทธิภาพ ผลการทดลองชี้ให้เห็นว่าแนวทางดังกล่าวให้ความแม่นยำสูงในการจำแนกใบหน้าจริง และใบหน้าปลอมบนชุดข้อมูลมาตรฐาน และเป็นการเน้นย้ำถึงความสำคัญของการใช้ข้อมูลเชิงลึก (Depth) เพื่อรับมือกับการปลอมแปลงใบหน้า

Afchar et.al, (2018) ได้พัฒนาแบบจำลอง MesoNet ซึ่งเป็นโครงข่ายประสาทเทียมแบบ Convolution ขนาดเล็กสำหรับงานตรวจจับวิดีโอปลอมใบหน้า (Deepfake) โดยมุ่งเน้นที่ใช้คุณลักษณะระดับกลาง (Mesoscopic Features) ของภาพใบหน้าแทนการใช้โครงข่ายขนาดใหญ่ที่ซับซ้อน ผลการทดลองแสดงให้เห็นว่า MesoNet สามารถตรวจจับวิดีโอใบหน้าปลอมที่สร้างด้วยเทคนิคต่าง ๆ เช่น Deepfake และ Face2Face เป็นต้น ได้อย่างมีประสิทธิภาพ โดยให้ความแม่นยำในการจำแนกสูงกว่าร้อยละ 95 ร้อยละ 98 ในการทดสอบชุดข้อมูลมาตรฐาน การวิจัยนี้ชี้ให้เห็นว่า

สถาปัตยกรรมแบบจำลองที่เล็กลงสามารถให้ผลการตรวจจับที่ดี และเหมาะสมสำหรับการใช้งานจริง ที่มีทรัพยากรจำกัด

Li et.al, (2018) ได้เสนอวิธีการตรวจจับวิดีโอใบหน้าปลอมที่สร้างโดย AI โดยอาศัยการวิเคราะห์การกะพริบตาของตัวแบบในวิดีโอ เนื่องจากวิดีโอ Deepfake ในยุคแรกหลาย ๆ ชิ้นมักมีความผิดปกติคือ ตัวละครในวิดีโอไม่กะพริบตาหรือกะพริบน้อยอย่างไม่เป็นธรรมชาติเมื่อเทียบกับคนจริง ทีมวิจัยได้พัฒนาแบบจำลองเพื่อตรวจจับ และวัดความถี่ในการกะพริบตาจากวิดีโอใบหน้า แล้วใช้ข้อมูลนี้ในการระบุว่าวิดีโอดังกล่าวเป็นของจริงหรือถูกปลอมแปลง ผลการทดลองระบุว่าวิธีนี้สามารถตรวจจับการปลอมแปลงได้อย่างมีนัยสำคัญ โดยเฉพาะอย่างยิ่งสำหรับวิดีโอปลอมที่แบบจำลองการสร้างยังไม่สามารถเลียนแบบพฤติกรรมการกะพริบตาได้สมจริง

Rössler et.al, (2019) ได้สร้างชุดข้อมูล FaceForensics++ ซึ่งประกอบด้วยวิดีโอใบหน้าที่ถูกดัดแปลงด้วยวิธีการต่าง ๆ (ทั้งการสลับใบหน้าแบบ FaceSwap และการสร้างใบหน้าใหม่ด้วยวิธี Deepfake พร้อมทั้งระดับการบีบอัดของไฟล์ที่แตกต่างกัน เพื่อใช้เป็นมาตรฐานสำหรับการฝึก และทดสอบแบบจำลองตรวจจับใบหน้าปลอม นอกจากนี้งานวิจัยยังได้ประเมินแบบจำลองการตรวจจับหลายประเภท โดยพบว่าโครงข่ายแบบ Xception ที่ฝึกบนชุดข้อมูลดังกล่าวสามารถตรวจจับใบหน้าที่ถูกปลอมแปลงได้ด้วยความแม่นยำสูงมากในสถานะที่ข้อมูลมีความคมชัด อย่างไรก็ตาม งานวิจัยนี้ชี้ให้เห็นว่าความแม่นยำในการตรวจจับจะลดลงเมื่อวิดีโอถูกบีบอัดหรือมีความละเอียดต่ำ ซึ่งสะท้อนถึงความท้าทายที่ยังมีอยู่ในการตรวจจับ Deepfake ในสภาพแวดล้อมการใช้งานจริง

Dolhansky et.al, (2020) ได้จัดทำชุดข้อมูลขนาดใหญ่สำหรับการแข่งขัน DeepFake Detection Challenge (DFDC) ซึ่งประกอบด้วยวิดีโอที่สร้างด้วยเทคนิคการปลอมแปลงใบหน้าหลากหลายรูปแบบหลายพันรายการ เพื่อกระตุ้นการพัฒนาขั้นตอนวิธีตรวจจับวิดีโอปลอมระดับโลก ผลการรายงานระบุว่า การตรวจจับ Deepfake ยังคงเป็นเรื่องท้าทาย แม้ว่าจะมีการใช้ชุดข้อมูลขนาดใหญ่ และแบบจำลองที่ซับซ้อนก็ตาม แบบจำลองที่ชนะการแข่งขันสามารถตรวจจับวิดีโอปลอมได้ในระดับที่น่าพอใจ แต่ยังไม่สมบูรณ์แบบ โดยมีการผิดที่ตรวจไม่พบอยู่บ้าง งานนี้แสดงให้เห็นถึงความจำเป็นในการวิจัยเพิ่มเติม และความร่วมมือระหว่างชุมชนนักวิจัยเพื่อรับมือกับภัยคุกคามจากสื่อปลอมที่พัฒนาอย่างต่อเนื่อง

Dragoni et.al, (2017) ได้ศึกษาเปรียบเทียบวิวัฒนาการของสถาปัตยกรรม Microservices ตั้งแต่อดีตจนถึงปัจจุบัน โดยสรุปหลักการ และเหตุผลที่ทำให้แนวคิด Microservices ได้รับความนิยมอย่างแพร่หลาย ผู้วิจัยได้อธิบายข้อดีของสถาปัตยกรรม Microservices เช่น ความยืดหยุ่นในการปรับปรุงขยายระบบทีละส่วน (Scalability) การแยกการพัฒนาบริการ (Service) อย่างอิสระ (Independent Deployability) และความทนทานเมื่อส่วนใดส่วนหนึ่งล้มเหลว (Fault Isolation) นอกจากนี้ยังกล่าวถึงความท้าทาย ได้แก่ ความซับซ้อนในการจัดการการสื่อสารระหว่างบริการจำนวนมาก และความจำเป็นในการมีระบบบริหารจัดการที่ดีสำหรับการดูแลบริการเหล่านั้น ทั้งนี้บทความดังกล่าวช่วยให้เข้าใจถึงแนวโน้ม และแนวปฏิบัติที่ดีในการออกแบบระบบแบบ Microservices ซึ่งมีผลโดยตรงต่อการสร้างสถาปัตยกรรมระบบที่มีประสิทธิภาพ และรองรับการขยายตัวในอนาคต

2.2 เทคโนโลยีการรู้จำใบหน้า (Face Recognition Technology)

การรู้จำใบหน้าเป็นกระบวนการที่นำภาพใบหน้าที่ตรวจจับได้ และผ่านการประมวลผล โดย การนำข้อมูลทั้งหมดที่ได้จากขั้นตอนก่อนหน้า มาเปรียบเทียบกับฐานข้อมูลของใบหน้าเพื่อระบุว่า ใบหน้าที่ตรวจจับได้ตรงกับบุคคลใด แล้วจึงนำผลลัพธ์ที่ได้ ส่งไปให้ระบบหรือโปรแกรมเพื่อ ประมวลผลอื่น ๆ

2.2.1 แนวคิดทฤษฎีพื้นฐาน

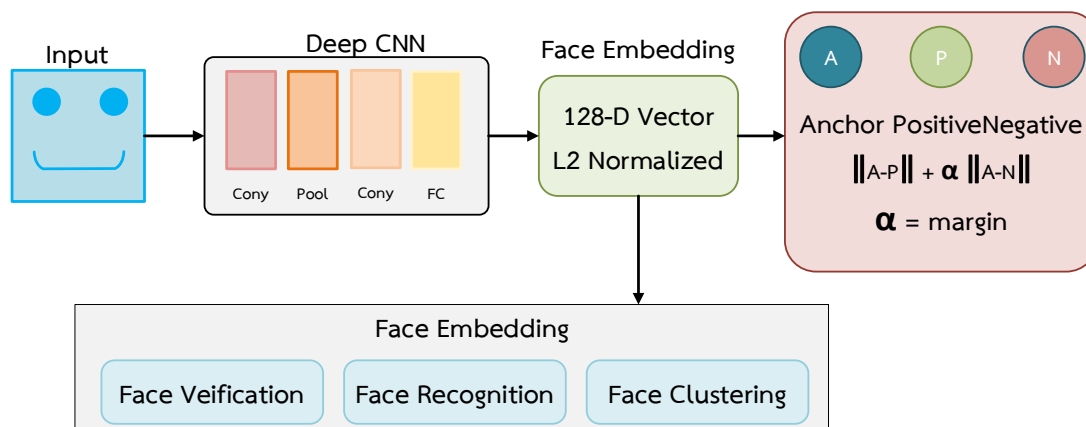
การแปลงภาพใบหน้าให้เป็นเวกเตอร์คุณลักษณะการแทนใบหน้า (Face Embedding) เพื่อ ใช้วัดความคล้ายกันระหว่างใบหน้า โดยปกติแล้วต้องการให้เวกเตอร์ของใบหน้าของคนเดียวกันอยู่ ใกล้กันมีระยะห่างภายในคลาสน้อย และของคนต่างคนอยู่ไกลกันมีระยะห่างระหว่างคลาสมาก การเรียนรู้การแทนใบหน้า (Face Embedding) สามารถทำได้หลายวิธี เช่น การใช้การสูญเสียแบบ สามส่วน (Triplet Loss) เป็นต้น

ซึ่งฝึกแบบจำลองด้วยชุดตัวอย่างสามภาพ (Anchor, Positive, Negative) โดยพยายามให้ ระยะห่างระหว่าง Anchor กับบุคคลเดียวกัน (Positive) น้อยกว่าระยะห่างระหว่าง Anchor กับ negative ซึ่งเป็นบุคคลต่างคน อย่างน้อยเท่ากับ margin ที่กำหนด นอกจากนี้ยังมีวิธีฝึกด้วย การสูญเสียแบบซอฟต์แวร์แมกซ์ (Softmax Loss) ที่มีการเพิ่ม Angular Margin เข้าไป เช่น ArcFace เป็นต้น ซึ่งจะทำให้การปรับมาตรฐาน (Normalize) เวกเตอร์คุณลักษณะ และน้ำหนักให้มีความยาว เท่ากัน คำนวณมุมระหว่างคุณลักษณะกับน้ำหนักของคลาสเป้าหมาย แล้วบวก Angular Margin ก่อนคำนวณ Softmax Cross-entropy ทำให้การแทนข้อมูล (Embedding) มีความแยกแยะได้ดีขึ้น

2.2.2 FaceNet

Facenet เป็นแบบจำลองแรก ๆ ที่นำแนวคิดการแทนใบหน้า (Face Embedding) และ การสูญเสียแบบสามส่วน (Triplet Loss) มาใช้

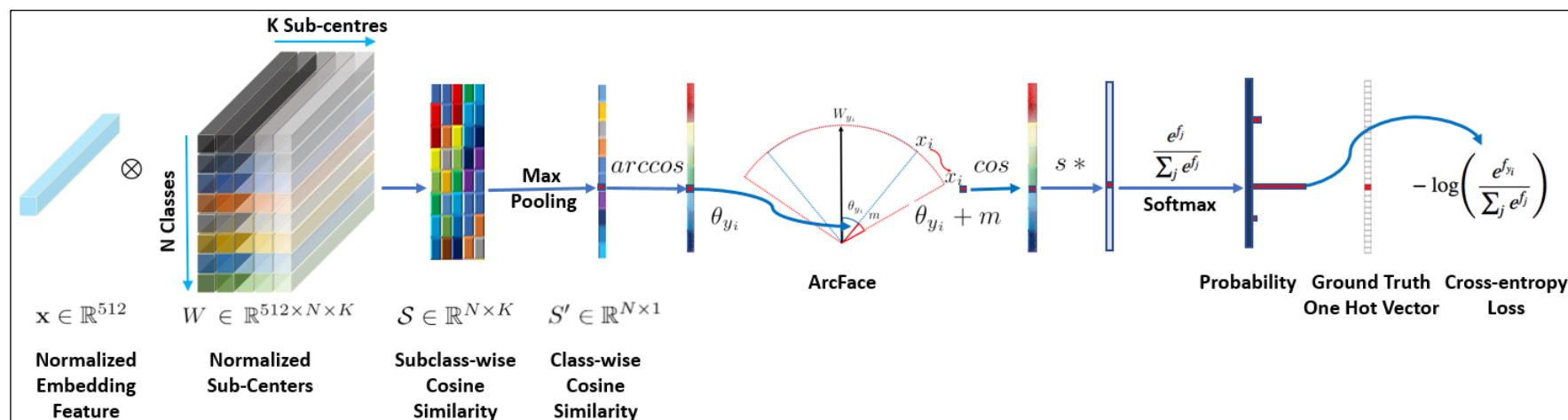
โดยใช้โครงข่ายประสาทแบบ (Convolutional Neural Network: CNN) เพื่อเรียนรู้ การแทนภาพใบหน้าลงในเวกเตอร์คุณลักษณะ 128 มิติ กระบวนการฝึก Facenet ใช้การสูญเสีย แบบสามส่วน (Triplet Loss) ซึ่งมีเป้าหมายให้เวกเตอร์ของใบหน้าบุคคลเดียวกันอยู่ใกล้กัน และ เวกเตอร์ของบุคคลต่างคนอยู่ห่างกันโดยมี Margin กำหนด Facenet แสดงประสิทธิภาพสูงมากใน การรู้จำใบหน้า เช่น บนชุดข้อมูล Labeled Faces In The Wild (LFW) ให้ความแม่นยำถึงร้อยละ 99.63 ซึ่งเป็นผลลัพธ์สูงสุดในขณะนั้น ดังภาพที่ 2-1



ภาพที่ 2-1 โครงสร้างของแบบจำลอง FaceNet (Schroff et al., 2015)

2.2.3 ArcFace

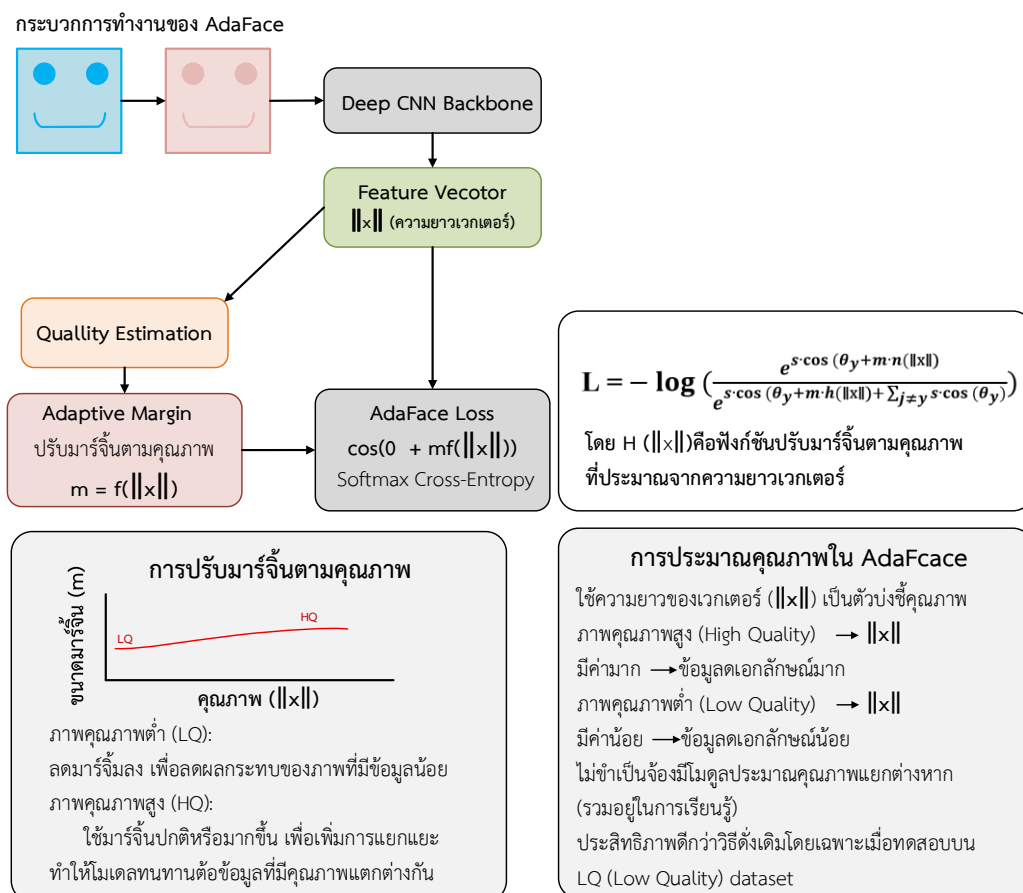
ArcFace เป็นแบบจำลองที่ใช้แนวทางการจำแนกประเภท Classification-Based โดยฝึกด้วย Softmax Loss ที่เพิ่มมาร์จินเชิงมุมแบบบวก (Additive Angular Margin) ระหว่างเวกเตอร์คุณลักษณะกับน้ำหนักของคลาสเป้าหมาย กระบวนการคือการปรับมาตรฐาน (Normalize) คุณลักษณะ และน้ำหนักให้มีขนาดเท่ากัน คำนวณค่าโคไซน์ (Cosine) ของมุมระหว่างคุณลักษณะและน้ำหนัก จากนั้นเพิ่มมาร์จินมุมต่อมุมเป้าหมาย ก่อนจะนำไปคำนวณ Softmax Cross-Entropy เหมือนปกติ การเพิ่มมาร์จินเชิงมุมนี้ทำให้ระยะห่างเชิงมุมระหว่างคลาสเพิ่มขึ้น ซึ่งช่วยให้คุณลักษณะเรียนรู้ได้มีความแตกต่างกันมากขึ้น ผลการทดลอง ArcFace แสดงให้เห็นว่าการเพิ่ม Margin ดังกล่าวช่วยปรับปรุงผลการรู้จำใบหน้าอย่างมีนัยสำคัญ เช่น บนชุดข้อมูล Labeled Faces in the Wild, ArcFace ให้ความแม่นยำสูงถึงร้อยละ 99.53 และในชุดข้อมูล CFP-FP และ AgeDB-30 ก็ได้ผลลัพธ์สูงใกล้เคียงกัน ดังภาพที่ 2.2



ภาพที่ 2-2 แนวคิด Additive Angular Margin ของ ArcFace (Deng et al., 2019)

2.2.4 AdaFace

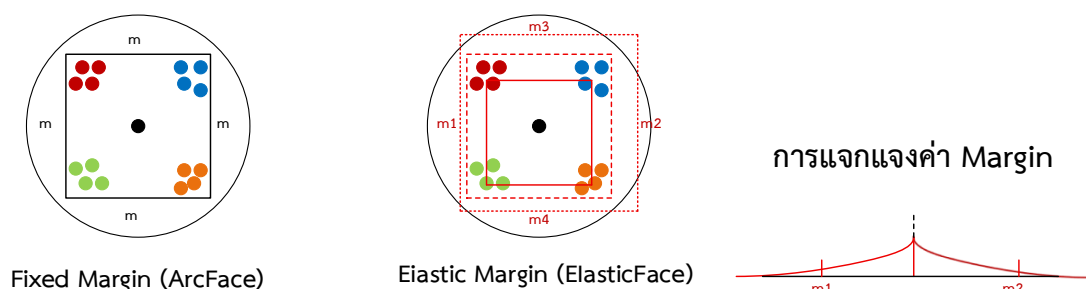
AdaFace เป็นฟังก์ชันการสูญเสียใหม่สำหรับการรู้จำใบหน้าที่เน้นการปรับมาร์จินให้เหมาะสมกับคุณภาพของภาพ หรือมาร์จินปรับตามคุณภาพ (Quality-Adaptive Margin) โดยใช้ความยาวเวกเตอร์คุณลักษณะเป็นตัวประมาณคุณภาพของภาพ แบบจำลอง AdaFace จะให้ความสำคัญกับตัวอย่างที่ยาก (Hard Samples) มากขึ้นหรือลดลงตามระดับคุณภาพภาพที่ประเมินได้ เพื่อป้องกันไม่ให้ภาพคุณภาพต่ำซึ่งมีข้อมูลตัวตนน้อย ส่งผลทำให้ประสิทธิภาพโดยรวมแย่ลง การทดลองแสดงว่า AdaFace ให้ผลดีกว่าหรือเทียบเท่ากับเทคโนโลยีล้ำสมัย (State-of-the-art) บนชุดข้อมูลภาพคุณภาพต่ำหลายชุด เช่น IJB-B, IJB-C, IJB-S และ TinyFace เป็นต้น ดังภาพที่ 2-3



ภาพที่ 2-3 แนวคิด Adaptive Margin ของ AdaFace (Kim et al., 2022)

2.2.5 ElasticFace

ElasticFace เป็นฟังก์ชันการสูญเสียใหม่ที่เสนอแนวคิดการสูญเสียแบบมาร์จิ้นยืดหยุ่น (Elastic Margin Loss) โดยไม่ใช้มาร์จิ้นตายตัวเหมือน ArcFace แต่จะสุ่มค่ามาร์จิ้นจากการแจกแจงแบบปกติ (Normal Distribution) ในแต่ละรอบการฝึกแทน วิธีนี้ทำให้เส้นแบ่งการตัดสินใจ (Decision Boundary) สามารถยืดหยุ่นเข้าออกได้ยืดหยุ่นมากขึ้นตามข้อมูลที่เรียนรู้ ช่วยให้แบบจำลองเรียนรู้การแยกคลาสได้ดีขึ้นโดยไม่แข็งตัวมากเกินไป ผลการทดลองของ ElasticFace บนหลายชุดข้อมูลแสดงว่าแบบจำลองทำได้ดีกว่าเทคโนโลยีล้ำสมัย (State-of-the-art) หลายกรณี ตัวอย่างเช่น บนชุดข้อมูล AgeDB-30, ElasticFace-Arc ให้ความแม่นยำร้อยละ 98.35 ดีกว่าผลงานก่อนหน้านี้ร้อยละ 98.32 และบน CFP-FP, ElasticFace-Arc+ ได้ร้อยละ 98.73 นอกจากนี้ ElasticFace ยังได้คะแนนสูงสุดในชุดข้อมูล IJB-B, IJB-C และ MegaFace (Refined) เป็นต้น ดังภาพที่ 2-4



หลักการ ElasticFace

- ElasticFace ใช้มาร์จิ้นแบบยืดหยุ่นที่ถูกสุ่มจากการแจกแจงแบบปกติ N
- ในแต่ละรอบการฝึก ค่ามาร์จิ้นถูกสุ่มใหม่ ทำให้ Decision boundary ยืดหยุ่น
- ข้อดี: เพิ่มความทนทานต่อความผันผวนของข้อมูล และการแยกแยะที่ดีขึ้น
- โมเดลให้ผลลัพธ์ดีกว่า State-of-the-art บนหลายชุดข้อมูล เช่น IJB-B, IJB-C

ภาพที่ 2-4 สรุปหลักการ Elastic Margin Loss ของ ElasticFace (Boutros et al., 2022)

2.3 ปัญญาประดิษฐ์ และการเรียนรู้ของเครื่อง (Artificial Intelligence and Machine Learning)

งานรู้จำใบหน้าเป็นส่วนหนึ่งของวิสัยทัศน์คอมพิวเตอร์ที่ใช้เทคนิคปัญญาประดิษฐ์และการเรียนรู้ของเครื่องเพื่อให้เครื่องคอมพิวเตอร์สามารถตรวจจับ ตระหนัก และจดจำใบหน้าของบุคคลได้ หลักการพื้นฐานคือการแปลงภาพใบหน้าเป็นคุณลักษณะหรือการแทนข้อมูล (Embedding) ทางคณิตศาสตร์ แล้วใช้วิธีการเรียนรู้ของเครื่องในการเทียบเคียงหรือจำแนกใบหน้าแต่ละภาพเป็นของบุคคลเดียวกันหรือไม่ ในปัจจุบัน โครงข่ายประสาทเทียมเชิงลึก (Deep Convolutional Neural Networks) กลายเป็นแนวทางที่ทันสมัยที่สุดสำหรับการรู้จำใบหน้า โดยสามารถให้ประสิทธิภาพใกล้เคียงหรือเทียบเท่ามนุษย์ในงานทดสอบหลายชุดข้อมูล กระบวนการทั่วไปมักประกอบด้วย การตรวจจับใบหน้าจากภาพ (Face Detection) การนำใบหน้ามาปรับตำแหน่ง (Alignment) และการส่งผ่านเข้าแบบจำลองเพื่อดึงคุณลักษณะหรือตัวแทนใบหน้าเชิงความหมาย (Feature Extraction/Embedding) ต่อไปคือการใช้ตัวแบบ เช่น Support Vector Machine หรือ Softmax ในการจำแนกหรือวัดระยะความใกล้เคียงระหว่างการแทนข้อมูล (Embedding) เหล่านี้

2.3.1 แนวคิดทฤษฎีพื้นฐาน

ในการเรียนรู้ของเครื่อง ต้องกำหนดปัญหาเป็นสมการหรือฟังก์ชัน ที่ระบบจะได้เรียนรู้จากข้อมูลตัวอย่าง (Training Data) โดยมีการป้อนข้อมูลภาพใบหน้าให้ระบบพร้อมป้ายกำกับ (Label) ที่เกี่ยวข้อง เช่น หมายเลขประจำตัวบุคคล เป็นต้น จากนั้นเทคนิคการเรียนรู้ของเครื่องจะแก้ไขพารามิเตอร์ภายในแบบจำลองเพื่อให้ผลลัพธ์ใกล้เคียงกับคำตอบที่คาดหวัง เมื่อขั้นตอนนี้เสร็จสิ้นจะได้แบบจำลองที่สามารถนำไปทดสอบกับภาพใหม่ได้ (Inference) การเรียนรู้ของเครื่องที่ใช้ในงานรู้จำใบหน้าสามารถแบ่งได้หลายประเภท เช่น การเรียนรู้แบบมีผู้กำกับ (Supervised Learning) ที่ใช้ตรรกะการจำแนกประเภท (Classification) หรือการเรียนรู้การวัดระยะ (Metric Learning) ในอวกาศการแทนข้อมูล (Embedding) เป็นต้น ในบริบทของการรู้จำใบหน้า มักเน้นการสร้างการแทนข้อมูล (Embedding) ให้ตรงกับวัตถุประสงค์การจดจำ การใช้ฟังก์ชันการสูญเสียเพื่อจัด

ระยะห่างภายในคลาสให้ใกล้กัน และภายนอกคลาสให้ห่างกัน เช่น Triplet Loss หรือ Contrastive Loss เป็นต้น เป็นสิ่งสำคัญในการจัดการปัญหาความแปรปรวนระหว่างคลาส และภายในคลาส (Inter-class/Intra-class Variation)

2.3.2 เทคนิคที่เกี่ยวข้อง

2.3.2.1 แผนภาพที่แสดงการแจกแจงความถี่ของข้อมูล(Histogram of Oriented Gradients)

แผนภาพที่แสดงการแจกแจงความถี่ของข้อมูล(Histogram of Oriented Gradients) เป็นวิธีดีที่อธิบายคุณลักษณะจากรูปภาพ โดยจะแบ่งภาพเป็นเซลล์เล็ก ๆ แล้วคำนวณทิศทางทางกรไล่ระดับภายในแต่ละเซลล์ วิธีนี้เหมาะสำหรับงานตรวจจับใบหน้า และวัตถุต่าง ๆ เนื่องจากมีความทนทานต่อการเปลี่ยนแปลงแสง และรูปร่างได้ดี งานวิจัยหลายชิ้นแสดงว่า การใช้ Histogram of Oriented Gradients ในการตรวจจับใบหน้าให้ผลแม่นยำดีกว่าเทคนิคดั้งเดิมอย่าง Haar Cascades

2.3.2.2 เครื่องเวกเตอร์สนับสนุน (Support Vector Machine)

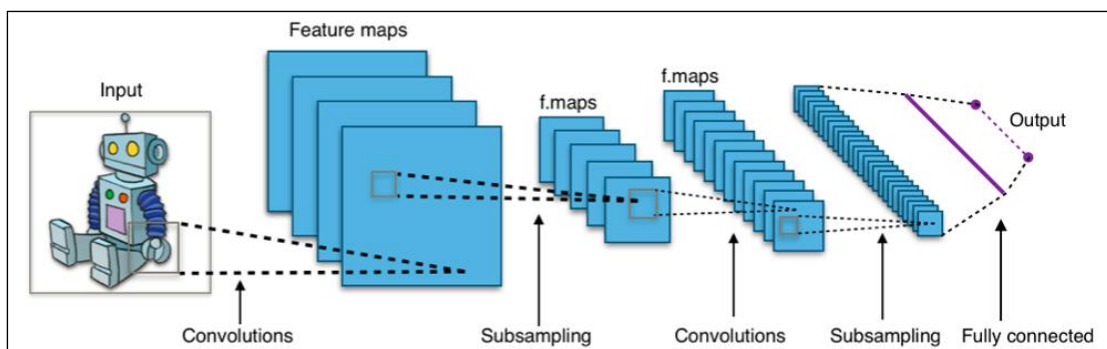
เครื่องเวกเตอร์สนับสนุน (Support Vector Machine) เป็นขั้นตอนวิธีการจำแนกประเภทที่นิยมใช้ร่วมกับ Histogram of Oriented Gradients หรือคุณลักษณะเชิงภาพอื่น ๆ โดยการหา Hyperplane เพื่อแบ่งสองคลาส เช่น ใบหน้า และไม่ใช่อใบหน้า เป็นต้น ข้อดีของ Support Vector Machine คือมีความเสถียรต่อการเปลี่ยนแปลงเล็กน้อยของข้อมูล เพราะ Hyperplane ไม่เปลี่ยนแปลงมากนักหากข้อมูลมีการคลาดเคลื่อนเล็กน้อย จึงเหมาะสำหรับใช้จำแนกใบหน้าหรือคนในระบบต่าง ๆ

2.3.2.3 การประมวลผลภาพเบื้องต้น

การประมวลผลภาพเบื้องต้นก่อนนำภาพใบหน้าลงระบบเรียนรู้ มักมีการปรับแสง และคอนทราสต์ เช่น เทคนิคการสร้างสมดุลแผนภาพที่แสดงการแจกแจงความถี่ของข้อมูลแบบปรับตัว จำกัดคอนทราสต์ (Contrast Limited Adaptive Histogram Equalization) เพื่อเพิ่มความชัดเจน และลดสัญญาณรบกวนในภาพ

2.3.2.4 เครือข่ายประสาทเทียมเชิงลึก (Convolutional Neural Network)

เครือข่ายประสาทเทียมเชิงลึก (Convolutional Neural Network) เป็นเทคนิคสำคัญในยุคปัจจุบัน โดย Convolutional Neural Network จะประกอบด้วยชั้น Convolution และชั้น Pooling หลายชั้นเพื่อเรียนรู้คุณลักษณะเชิงพื้นที่ในรูปภาพอย่างอัตโนมัติ การเรียนรู้คุณลักษณะอัตโนมัติทำให้ Convolutional Neural Network สามารถดึงรายละเอียดเชิงลึกของใบหน้าได้ดีกว่าวิธีดั้งเดิม ดังภาพที่ 2-5



ภาพที่ 2-5 โครงสร้างเครือข่ายประสาทเทียมเชิงลึกแบบคอนโวลูชัน (CNN) (Aphex34, 2015)

2.3.2.5 ฟังก์ชันการสูญเสีย (Triplet Loss)

เมื่อใช้เครือข่ายประสาทเทียมแบบคอนโวลูชัน เพื่อดึงการแทนข้อมูล (Embedding) ของภาพใบหน้า จำเป็นต้องกำหนดฟังก์ชันการสูญเสียที่ช่วยจัดการระยะห่างระหว่างการแทนข้อมูลของบุคคลเดียวกันและต่างบุคคล

การสูญเสียแบบสามส่วน เป็นหนึ่งในเทคนิคที่ได้รับความนิยม โดยจะใช้ภาพ 3 ใบ ได้แก่ ภาพหลัก (X_i) และภาพบวก (X_p) ที่เป็นใบหน้าคนเดียวกัน รวมถึงภาพลบ (X_n) ที่เป็นคนต่างกัน การสูญเสียจะกระตุ้นให้ระยะระหว่างภาพหลักกับภาพบวกต้องน้อยกว่าระยะระหว่างภาพหลักกับภาพลบอย่างน้อยค่าช่องว่าง หรือ α

$$L = \sum \max(\|f(x_a) - f(x_p)\|_2 - \|f(x_a) - f(x_n)\|_2 + \alpha, 0)$$

โดย $f(x)$ คือการแทนข้อมูล (Embedding) ของภาพ x ที่ผ่านการปรับมาตรฐาน (Normalized) เพื่อให้ $\|f(x)\| = 1$

2.3.3 แบบจำลอง (Model)

แบบจำลองหรือสถาปัตยกรรมของเครือข่ายประสาทเทียมที่ได้รับความนิยมในการรู้จำใบหน้ามีหลายแบบ โดยส่วนใหญ่เป็นการปรับปรุงเครือข่ายประสาทเทียมแบบคอนโวลูชัน (Convolutional Neural Network) ที่มีต้นกำเนิดแตกต่างกัน เช่น VGG-Face, ResNet, Inception และเครือข่ายประสาทเทียมแบบผสม (Hybrid Convolutional Neural Network/Long Short-Term Memory) สำหรับวิดีโอ เป็นต้น ตัวอย่างเด่นคือ FaceNet ซึ่งใช้เครือข่ายประสาทเทียมแบบคอนโวลูชันขนาดใหญ่ และฟังก์ชันการสูญเสียแบบสามส่วน (Triplet Loss) เพื่อฝึกให้ได้การแทนข้อมูล (Embedding) ขนาด 128 มิติ ในการวัดระยะใบหน้าแบบครบวงจร (End-to-End) ผลงานของ FaceNet ได้คะแนนความแม่นยำบนชุดทดสอบ Labeled Faces in the Wild สูงถึงร้อยละ 99.63 แบบจำลอง DeepFace ของ Facebook อีกแบบจำลองหนึ่ง ใช้วิธีจัดตำแหน่งใบหน้าแบบสองมิติและสามมิติ แล้วผ่านเครือข่ายประสาทเทียมแบบคอนโวลูชันแบบ 9 ชั้น ทำได้ความแม่นยำประมาณร้อยละ 97.35 บน Labeled Faces in the Wild ในทำนองเดียวกัน DeepID/DeepID2 ใช้เครือข่ายประสาทเทียมแบบคอนโวลูชันขนาดยาวเพื่อเพิ่มความแตกต่างระหว่างบุคคลและลดความแตกต่างภายในบุคคล จนได้ผลร้อยละ 99.15 บน Labeled Faces in the Wild

ปัจจุบันยังมีแบบจำลองใหม่ ๆ เช่น SphereFace, CosFace และ ArcFace เป็นต้น ซึ่งเพิ่มมาร์จิ้นเชิงมุม (Angular Margin) ในฟังก์ชัน Softmax เพื่อปรับปรุงความแยกคลาสและได้ผลแม่นยำสูง ArcFace ได้ประมาณร้อยละ 99.83 บน Labeled Faces in the Wild ในงานวิจัยต้นแบบ

นอกเหนือจากเครือข่ายประสาทเทียมแบบคอนโวลูชันเชิงลึก ยังมีเทคนิคอื่น ๆ ที่เสริมประสิทธิภาพ เช่น การรวมผลจากหลายกล่องหรือหลายเฟรมในวิดีโอ, การใช้การรวมกลุ่มของตัวแบบ (Ensemble) หรือการใช้สถาปัตยกรรมใหม่ ๆ เช่น เครือข่ายประสาทเทียมแบบแผนภาพ (Graph Neural Network) กับคุณลักษณะของใบหน้า เป็นต้น อย่างไรก็ตามทั้งหมดล้วนเป็นเครื่องมือที่ช่วยให้ระบบรู้จำใบหน้าในเชิงการเรียนรู้ของเครื่อง (Machine Learning) ทำงานได้แม่นยำและมีประสิทธิภาพสูงขึ้น

2.4 รูปแบบการเรียนรู้ของเครื่อง (Learning Paradigms in Machine Learning)

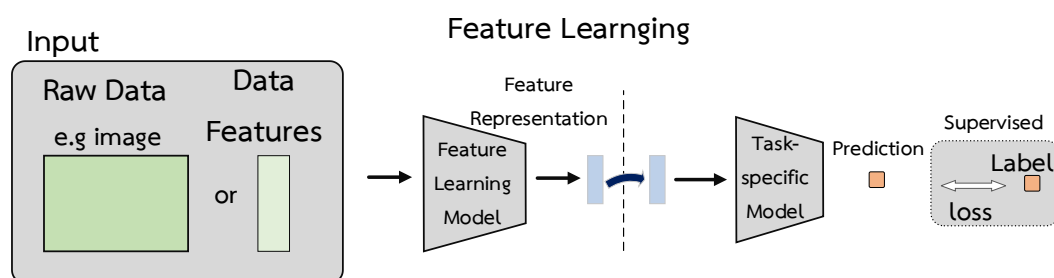
2.4.1 แนวคิดทฤษฎีพื้นฐาน

การเรียนรู้ของเครื่อง (Machine Learning) คือศาสตร์ที่ให้คอมพิวเตอร์สามารถเรียนรู้จากข้อมูลโดยไม่ต้องเขียนโปรแกรมอย่างชัดเจน โดยมีหลักการพื้นฐานคือการใช้ขั้นตอนวิธีในการวิเคราะห์ข้อมูล สร้างแบบจำลองทางคณิตศาสตร์ และนำแบบจำลองนั้นไปทำนายหรือตัดสินใจกับข้อมูลใหม่ แนวคิดหลักของการเรียนรู้ของเครื่องแบ่งออกเป็นหลายรูปแบบตามลักษณะการเรียนรู้และวัตถุประสงค์การใช้งาน โดยปัจจัยสำคัญในการเลือกรูปแบบการเรียนรู้ประกอบด้วย ลักษณะของข้อมูลที่มีซึ่งอาจมีป้ายกำกับหรือเป้าหมายของงานที่อาจเป็นการจำแนก การทำนาย หรือการหาแบบความซับซ้อนของปัญหา และ ทรัพยากรที่มี เช่น เวลาและค่าใช้จ่ายในการติดป้ายข้อมูล เป็นต้น ทฤษฎีพื้นฐานของการเรียนรู้ของเครื่องอยู่บนหลักการทางสถิติและการหาค่าเหมาะสมที่สุด (Optimization) ซึ่งมีจุดมุ่งหมายเพื่อสร้างแบบจำลองที่มีความสามารถในการขยายผล (Generalization) คือสามารถทำงานได้ดีกับข้อมูลที่ไม่เคยเห็นมาก่อน ไม่ใช่เพียงแค่การจดจำข้อมูลสำหรับฝึก (Memorizing Training Data) การออกแบบขั้นตอนวิธีการเรียนรู้มักคำนึงถึงความสมดุลระหว่างความผิดพลาดจากอคติ (Bias) และความแปรปรวน (Variance) ซึ่งเป็นที่รู้จักในชื่อการถ่วงดุลระหว่างอคติและความแปรปรวน (Bias-Variance Tradeoff) ในด้านการรู้จำใบหน้าและการตรวจจับภาพปลอม ทฤษฎีพื้นฐานเหล่านี้ถูกนำมาประยุกต์ใช้เพื่อสร้างแบบจำลองที่มีความแม่นยำสูง ทนทานต่อสภาพแวดล้อมที่หลากหลาย และสามารถต่อต้านการโจมตีได้ ขณะที่รูปแบบการเรียนรู้ต่าง ๆ ที่จะกล่าวต่อไปนี้จะแสดงให้เห็นถึงวิธีการที่แตกต่างกันในการฝึกฝนแบบจำลองสำหรับงานเฉพาะทางเหล่านี้

2.4.2 การเรียนรู้แบบมีผู้สอน (Supervised Learning)

การเรียนรู้แบบมีผู้สอนเป็นรูปแบบพื้นฐานที่สุดของการเรียนรู้ของเครื่อง ซึ่งมีลักษณะสำคัญคือใช้ชุดข้อมูลสำหรับฝึกที่มีป้ายกำกับ (Labeled Data) กำกับคำตอบซึ่งอาจเป็นป้ายกำกับคลาสหรือค่าต่อเนื่องสำหรับแต่ละตัวอย่าง เช่น ภาพใบหน้าที่ติดป้ายชื่อหรือยืนยันตัวตน เป็นต้น จากนั้นขั้นตอนวิธีจะเรียนรู้ฟังก์ชันการจับคู่ (Mapping Function) จากข้อมูลเหล่านี้เพื่อให้สามารถทำนายผลลัพธ์ซึ่งคือป้ายกำกับของข้อมูลใหม่ได้ การเรียนรู้ประเภทนี้ครอบคลุมทั้งงานประเภทการจำแนกประเภท (Classification) และการถดถอย (Regression) ตัวอย่างขั้นตอนวิธีที่นิยมได้แก่ k-Nearest

Neighbors (k-NN), Support Vector Machine (SVM) และต้นไม้ตัดสินใจ รวมถึงโครงข่ายประสาทเทียมอย่าง Convolutional Neural Network (CNN) และ Recurrent Neural Network (RNN) ที่ใช้กับข้อมูลภาพและข้อความ เป็นต้น ข้อดีของการเรียนรู้แบบมีผู้สอนคือแบบจำลองจะฝึกได้แม่นยำสูงหากมีข้อมูลและป้ายกำกับที่เพียงพอ แต่ก็ต้องใช้ต้นทุนในการติดป้ายข้อมูล ดังภาพที่ 2-6



ภาพที่ 2-6 แนวคิด Feature Learning สำหรับการเรียนรู้แบบมีผู้สอน (Fgpacini, 2022)

ในการประยุกต์ด้านการรู้จำใบหน้า (Face Recognition) แบบจำลองโครงข่ายประสาทเทียมแบบคอนโวลูชันที่ผ่านการเรียนรู้แบบมีผู้สอนมักถูกนำมาใช้สร้างคุณลักษณะของใบหน้าที่ยกชนิตได้ชัดเจน เช่น FaceNet, ArcFace เป็นต้น จากนั้นใช้ระยะห่างของการแทนข้อมูล (Embedding) ในการเปรียบเทียบหรือตรวจสอบอัตลักษณ์ นอกจากนี้ การตรวจจับภาพปลอมเชิงลึก (Deepfake) มักใช้วิธีการจำแนกประเภทแบบมีผู้สอนโดยฝึกแบบจำลองให้แยกความแตกต่างระหว่างภาพจริงและภาพปลอมด้วยโครงข่ายประสาทเชิงลึก การทำงานแบบมีผู้สอนจึงเป็นแกนหลักของระบบรู้จำใบหน้าและตรวจจับภาพปลอมเชิงลึกในงาน FaceSocial ที่ใช้ชุดข้อมูลติดป้ายประกอบกับสถาปัตยกรรม Microservices

2.4.3 การเรียนรู้แบบไม่มีผู้สอน (Unsupervised Learning)

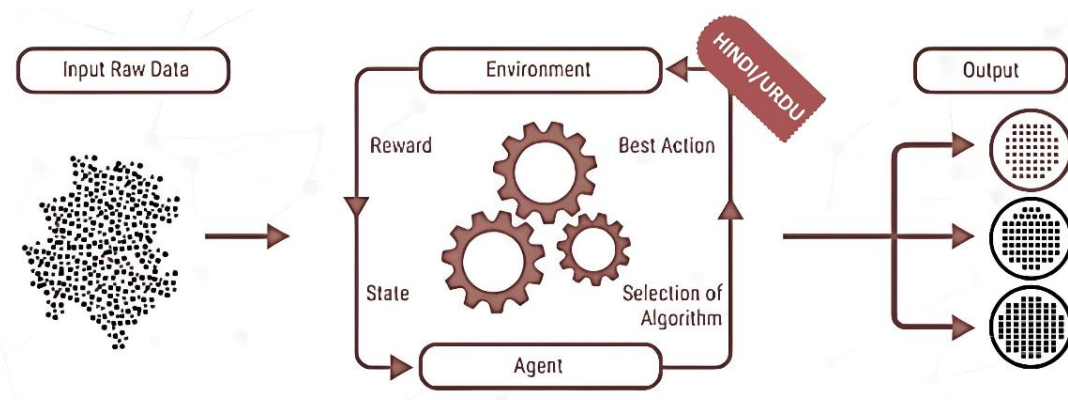
การเรียนรู้แบบไม่มีผู้สอนเป็นรูปแบบที่ระบบเรียนรู้จากชุดข้อมูลที่ไม่มีป้ายกำกับโดยตรง กล่าวคือ ขั้นตอนวิธีจะค้นหาลักษณะหรือโครงสร้างที่ซ่อนอยู่ภายในข้อมูลได้แก่รูปแบบหรือกลุ่ม โดยไม่กำหนดคำตอบล่วงหน้า เทคนิคหลักในกลุ่มนี้ประกอบด้วยการจัดกลุ่ม (Clustering) เช่น K-Means, และการลดมิติ (Dimensionality Reduction) เช่น Principal Component Analysis (PCA) และ Autoencoder เป็นต้น ตลอดจนแบบจำลองเชิงสร้างสรรค์ (Generative Models) และการตรวจจับความผิดปกติ (Outlier Detection) จุดเด่นคือช่วยแยกแยะโครงสร้างข้อมูลหรือสร้างตัวแทนคุณลักษณะที่ดีได้โดยไม่ต้องพึ่งพาข้อมูลที่ติดป้าย ทำให้เหมาะกับงานที่มีข้อมูลปริมาณมากแต่ได้ทำการติดป้ายข้อมูลไม่ครบถ้วนหรือยากจะติดป้ายอย่างละเอียด

ในงานการรู้จำใบหน้า บางครั้งอาศัยการเรียนรู้แบบไม่มีผู้สอนเพื่อจัดกลุ่มภาพใบหน้าที่ไม่รู้ชื่อ (Face Clustering) ตัวอย่างเช่น การใช้ K-Means หรือการจัดกลุ่มแบบลำดับขั้น (Hierarchical Clustering) รวมถึงการใช้ autoencoder ฝึกเรียนการแทนข้อมูลที่บีบอัดข้อมูลใบหน้าและจับกลุ่มโดยใช้ความคล้ายคลึง เป็นต้น การศึกษาโดย Schroff (2015) ยังระบุว่าเมื่อแบบจำลองที่ฝึกแบบมีผู้สอนสร้างการแทนข้อมูลของใบหน้าแล้ว งานการจัดกลุ่มสามารถทำได้ง่ายขึ้น

ด้วยเทคนิคพื้นฐานอย่าง K-Means ผลลัพธ์เหล่านี้ช่วยให้ FaceSocial สามารถจัดกลุ่มคนในการรู้จำใบหน้าได้

2.4.4 การเรียนรู้แบบเสริมแรง (Reinforcement Learning)

การเรียนรู้แบบเสริมแรง (Reinforcement Learning) เป็นรูปแบบที่แตกต่างจากทั้งการเรียนรู้แบบมีผู้สอนและแบบไม่มีผู้สอน โดยแบบจำลองจะทำหน้าที่เป็นตัวแทน (Agent) เรียนรู้การตัดสินใจจากการปฏิสัมพันธ์กับสิ่งแวดล้อม (Environment) ผ่านสัญญาณผลตอบแทน (Reward) หรือการลงโทษ (Punishment) ที่ได้รับการกระทำในแต่ละสถานะในลักษณะที่คล้ายกับการลองผิดลองถูก ระบบการเรียนรู้แบบเสริมแรงมีเป้าหมายเพื่อให้ได้นโยบาย (Policy) ที่นำไปสู่การสะสมผลตอบแทนรวมสูงสุดในระยะยาว ตัวอย่างของขั้นตอนวิธีการเรียนรู้แบบเสริมแรง ได้แก่ Q-Learning, State-Action-Reward-State-Action, และเทคนิคการไล่ระดับนโยบาย (Policy Gradient) เป็นต้น ส่วนในปัญหาเชิงลึก การเรียนรู้แบบเสริมแรงเชิงลึก (Deep Reinforcement Learning) มักผสมผสานเครือข่ายประสาทเพื่อจัดการกับสถานะหรือการกระทำที่มีมิติสูง ดังภาพที่ 2-7



ภาพที่ 2-7 การทำงานพื้นฐานของ Reinforcement Learning (Megajuce, 2019)

ในงานนำไปประยุกต์ระบบจดจำใบหน้าที่มีการใช้การเรียนรู้แบบเสริมแรงในส่วนต่าง ๆ เช่น การใช้การเรียนรู้แบบเสริมแรงเพื่อลดอคติทางเชื้อชาติในการรู้จำใบหน้า และการจัดลำดับความสำคัญของข้อมูลจากวิดีโอเพื่อยืนยันตัวตนใบหน้า (Face Verification) เป็นต้น นอกจากนี้ การผสมผสานการเรียนรู้เชิงลึก (Deep Learning) กับการเรียนรู้แบบเสริมแรงยังถูกใช้สร้างแบบจำลองที่เรียนรู้ลักษณะใบหน้าโดยอาศัยรางวัลจากการตัดสินใจของตัวแทน (Agent) ซึ่งเปิดโอกาสให้ระบบเรียนรู้วิธีการสนใจคุณสมบัติที่สำคัญของใบหน้าได้เอง แม้ในกรณีที่ไม่ได้ให้คำตอบล่วงหน้าโดยตรง ผลลัพธ์เหล่านี้สะท้อนแนวคิดของ Barto and Sutton (1998) ที่นิยามการเรียนรู้แบบเสริมแรงว่าเป็นการเรียนรู้ลักษณะตามเป้าหมายโดยการทดลอง และรับรางวัลจากสิ่งแวดล้อม

2.4.5 การเรียนรู้แบบกึ่งมีผู้สอน (Semi-supervised Learning)

การเรียนรู้แบบกึ่งมีผู้สอน (Semi-supervised Learning) เป็นสาขาย่อยที่รวมประโยชน์จากข้อมูลที่ติดป้าย และไม่ติดป้าย โดยมีแนวคิดที่ว่าวิธีนี้จะใช้ชุดข้อมูลที่ติดป้ายจำนวนจำกัดร่วมกับชุดข้อมูลไม่ติดป้ายจำนวนมากเพื่อปรับปรุงความแม่นยำของแบบจำลอง กล่าวคือ ใช้ข้อมูลติดป้ายช่วย

ควบคุมทิศทางการเรียนรู้ และเสริมด้วยข้อมูลไม่ติดป้ายเพื่อให้แบบจำลองเห็นโครงสร้างโดยรวมของข้อมูล ตัวอย่างวิธีการเรียนรู้แบบกึ่งมีผู้สอน เช่น การแพร่กระจายป้ายกำกับ (Label Propagation) การติดป้ายเทียม (Pseudo-labeling) การฝึกกร่วม (Co-training) และวิธีใช้ความสม่ำเสมอ (Consistency) ในการฝึก เช่น π -model หรือ Mean Teacher เป็นต้น การเรียนรู้แบบกึ่งมีผู้สอนได้รับความนิยมเพิ่มขึ้นเนื่องจากลดภาระการติดป้ายข้อมูลด้วยตนเอง ในงานการรู้จำใบหน้า หรือ การตรวจจับ Deepfake อาจประยุกต์ใช้โดยฝึกแบบจำลองด้วยภาพใบหน้าหรือวิดีโอที่ติดป้ายบ้าง ซึ่งเป็นการบอกชื่อบุคคลหรือระบุของปลอม เป็นต้น ควบคู่กับภาพจำนวนมากที่ไม่มีป้าย กล่าวคือ ใช้ข้อมูลไม่ติดป้ายเพื่อเรียนรู้ความเป็นโครงสร้างทั่วไปของใบหน้า ขณะที่ข้อมูลติดป้ายช่วยให้แบบจำลองมีแนวทางในการจำแนกผลลัพธ์ที่ถูกต้อง ตัวอย่างงานวิจัยที่เกี่ยวข้องมักเน้นที่การปรับใช้กับเครือข่ายประสาทเทียมสมัยใหม่ เช่น หลากหลายงานเรียนรู้แบบกึ่งมีผู้สอนร่วมกับโครงข่ายลึก และขั้นตอนวิธีการเพิ่มประสิทธิภาพ เป็นต้น

2.4.6 การเรียนรู้แบบอัตโนมัติกำกับตนเอง (Self-supervised Learning)

การเรียนรู้แบบอัตโนมัติกำกับตนเอง (Self-supervised Learning) เป็นแนวทางการเรียนรู้ที่อยู่ในลำดับเดียวกับการเรียนรู้แบบไม่มีผู้สอน โดยข้อมูลเองถูกใช้สร้างป้ายกำกับจำลอง (Pseudo-label) ให้แบบจำลองฝึกฝน เช่น การปิดบางส่วนข้อมูลแล้วให้แบบจำลองทำนายส่วนที่ปิด การจับคู่ตัวอย่างที่เกี่ยวข้องผ่านการเพิ่มข้อมูล (Augmentation) หรือการเรียนรู้แบบเปรียบเทียบ (Contrastive Learning) เป็นต้น แนวคิดคือให้แบบจำลองเรียนรู้ตัวแทน (Representation) ที่ดีจากรูปแบบภายในของข้อมูลโดยไม่ต้องพึ่งป้ายกำกับภายนอก การเรียนรู้แบบอัตโนมัติกำกับตนเองได้รับความสนใจในงานวิสัยทัศน์คอมพิวเตอร์ (Computer Vision) และการประมวลผลภาษาธรรมชาติ (Natural Language Processing) ด้วยแนวคิดที่สามารถใช้การดัดแปลงข้อมูล เช่น หมุนภาพ ตัดบางส่วน เพิ่มเสียงรบกวน แล้วให้แบบจำลองเรียนรู้ที่จะทนต่อการดัดแปลงเหล่านี้หรือสร้างข้อมูลต้นฉบับขึ้นมาใหม่ เป็นต้น เทคนิคยอดนิยม เช่น การเรียนรู้แบบเปรียบเทียบ (Contrastive Learning) ซึ่งมีเทคนิคย่อยคือ Simple Framework for Contrastive Learning of Visual Representations, Momentum Contrast for Unsupervised Visual Representation Learning และการเรียนรู้เชิงสร้างสรรค์ (Generative Learning) ซึ่งมีเทคนิคย่อยคือ AUTOENCODER และ Bidirectional Encoder Representations from Transformers ที่ใช้ในงานการประมวลผลภาษาธรรมชาติ เป็นต้น แม้ว่าจะงานหลัก ๆ จะยังเน้นภาพ และข้อความ แต่แนวคิดถูกนำมาประยุกต์กับข้อมูลใบหน้าโดยอาจฝึกแบบจำลองด้วยการสร้างงานจำลอง (Task) เช่น คาดเดามุมกล้องของภาพใบหน้า หรือสร้างเงื่อนไขภาพบางส่วนใหม่ เป็นต้น การเรียนรู้แบบอัตโนมัติกำกับตนเองช่วยลดความจำเป็นในการติดป้ายข้อมูล และได้ผลลัพธ์ในการเรียนรู้คุณลักษณะได้ดีจากข้อมูลไม่ติดป้าย (Unlabeled) จำนวนมาก

2.5 การเรียนรู้แบบผสม (Ensemble Learning)

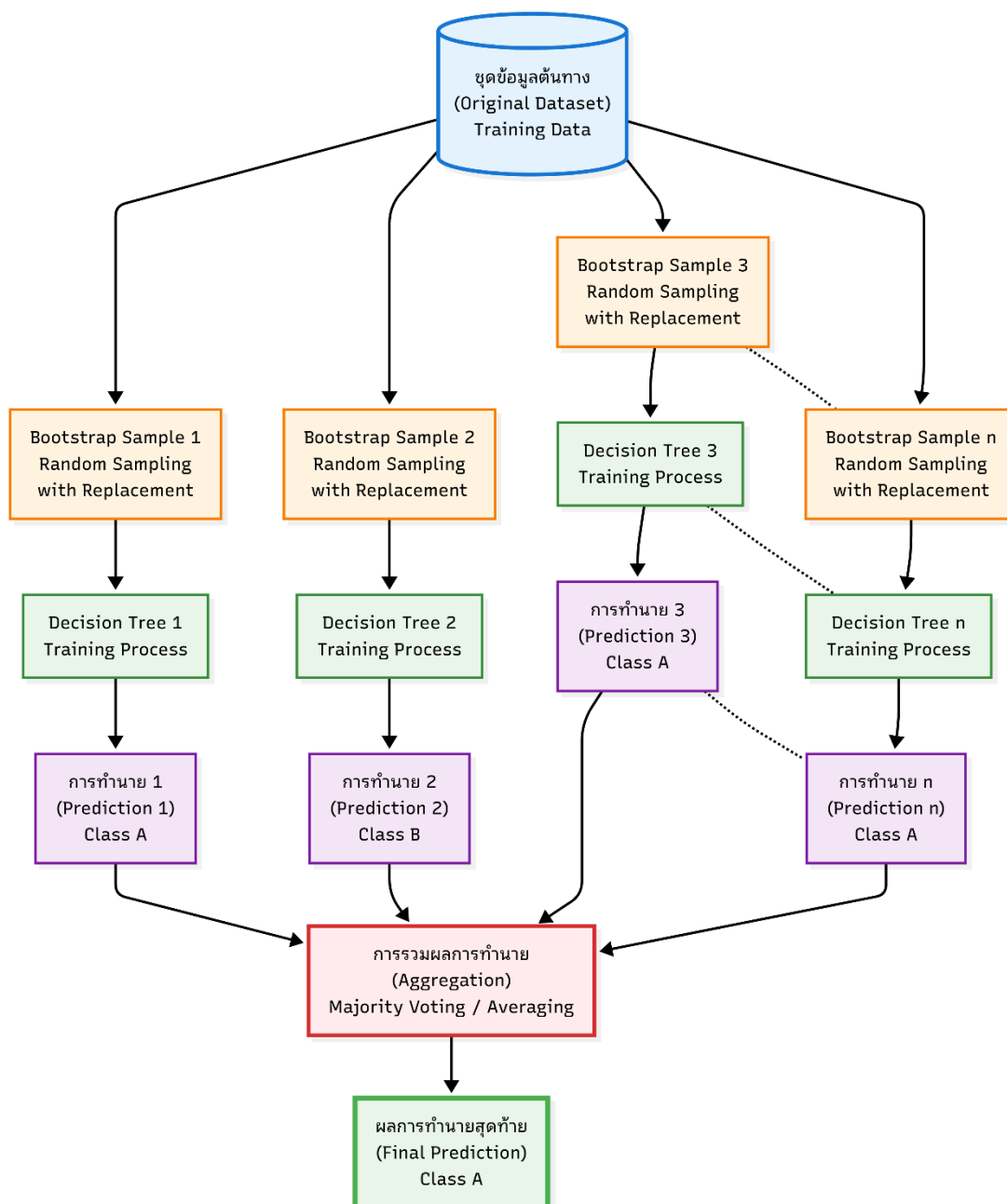
2.5.1 แนวคิดทฤษฎีพื้นฐาน

การเรียนรู้แบบผสม (Ensemble Learning) เป็นเทคนิคที่รวมผลจากแบบจำลองพื้นฐาน (Base Learners) หลายแบบจำลองเข้าด้วยกันเพื่อให้ได้ผลลัพธ์ที่แม่นยำกว่าการใช้แบบจำลองเดี่ยว

โดยทั่วไปจะเน้นลดความแปรปรวน (Variance) และความเอนเอียง (Bias) ของแบบจำลอง โดยมีพื้นฐานหลักว่าการรวมแบบจำลองหลายตัวเข้าด้วยกันอย่างเหมาะสมจะให้ผลดีกว่าแบบจำลองที่ดีที่สุดเพียงตัวเดียว เทคนิคหลักในการเรียนรู้แบบผสมประกอบด้วย Bagging, Boosting และ Stacking เป็นต้น ซึ่งแต่ละวิธีมีแนวทางที่แตกต่างกันในการสร้าง และรวมแบบจำลองย่อย

2.5.2 การจัดกลุ่มแบบสุ่ม (Bagging Bootstrap Aggregating)

การจัดกลุ่มแบบสุ่ม (Bagging) เป็นวิธีการเรียนรู้แบบผสมที่ใช้การสุ่มตัวอย่างข้อมูลฝึกสอน (Bootstrap Sampling) หลายชุด จากนั้นฝึกแบบจำลอง เช่น ต้นไม้ตัดสินใจ เป็นต้น บนข้อมูลแต่ละชุดอย่างอิสระ แล้วนำผลลัพธ์จากทุกแบบจำลองมารวมกันโดยใช้วิธีการต่าง ๆ เช่น การโหวตเสียงข้างมาก (Majority Voting) หรือการเฉลี่ยค่าคาดการณ์ เป็นต้น ดังภาพที่ 2-8



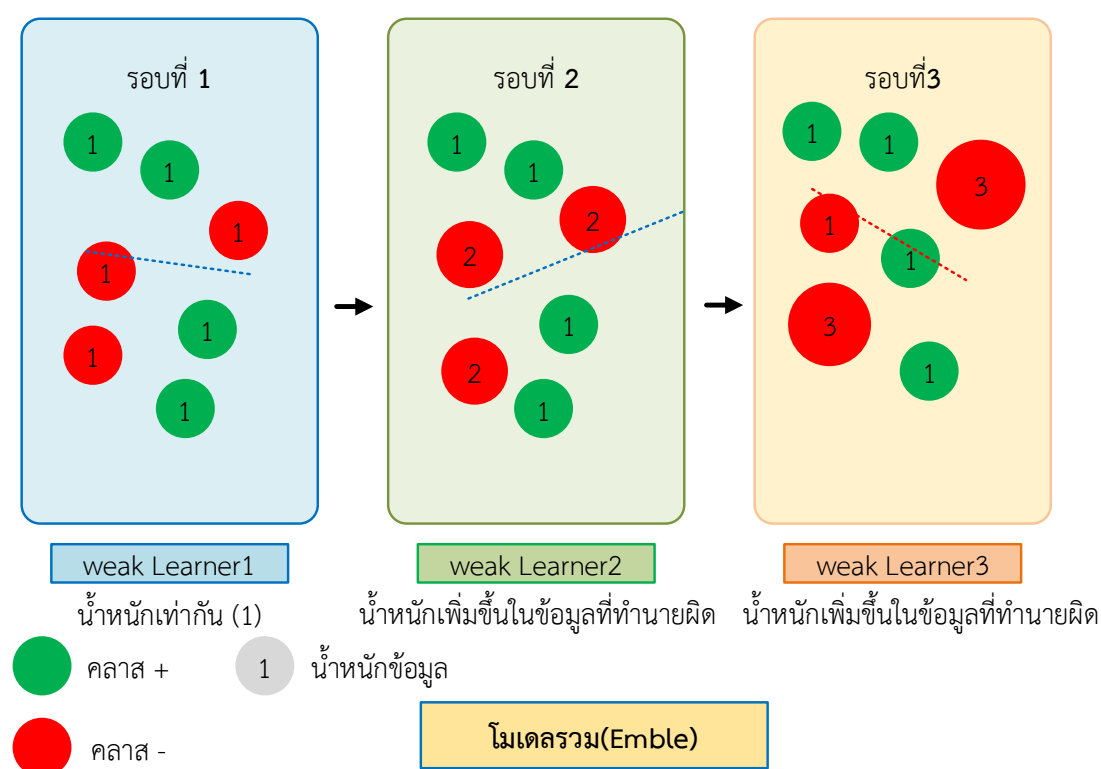
ภาพที่ 2-8 การทำงานของ Random Forest ที่ใช้เทคนิค Bagging (Fgpacini, 2022)

วิธีนี้ช่วยลดความแปรปรวนของแบบจำลองโดยรวมลง เพราะผลเฉลี่ยของหลายแบบจำลองมีแนวโน้มเข้าใกล้ค่าเฉลี่ยของฟังก์ชันจริงตามกฎของจำนวนมาก (Law of Large Numbers) จึงลดโอกาสการเรียนรู้ข้อมูลฝึกมากเกินไป (Overfitting) ของแบบจำลองเดี่ยวได้ การประยุกต์ทั่วไป เช่น ป่าสุ่ม (Random Forest) เป็นต้น ซึ่งเป็น Bagging ของต้นไม้ตัดสินใจหลายต้น ข้อดีของวิธีนี้คือช่วยลดความแปรปรวนของแบบจำลองได้อย่างมีประสิทธิภาพ ทำให้แบบจำลองเสถียรมากขึ้น แต่มีข้อเสียคือลดความเบี่ยงเบนได้จำกัด และต้องใช้ทรัพยากรในการคำนวณมากขึ้น หลักการทาง

สถิติที่สำคัญคือ การเฉลี่ยแบบจำลองหลายตัวช่วยทำให้ความแปรปรวนของข้อผิดพลาดรวมลดลงตามทฤษฎีกฎของจำนวนมาก ทำให้ความแปรปรวนการทำนายลดลงจนเหลือน้อยที่สุด

2.5.3 Boosting

Boosting เป็นวิธีการเรียนรู้แบบผสมที่ฝึกแบบจำลองตามลำดับ โดยแต่ละแบบจำลองใหม่จะมุ่งเน้นไปที่ตัวอย่างที่แบบจำลองก่อนหน้านี้ทำนายผิดบ่อย ๆ วิธีดั้งเดิมคือ AdaBoost ซึ่งถูกเสนอโดย Freund and Schapire (1997) โดย AdaBoost เริ่มต้นด้วยการกำหนดน้ำหนักเท่ากันให้ตัวอย่างทุกชิ้น แล้วสร้างผู้เรียนรู้อ่อน (Weak Learner) เช่น ต้นไม้ตัดสินใจตื้น เป็นต้น เมื่อได้ผลผิดพลาดจะปรับน้ำหนักของตัวอย่างที่ทำนายผิดให้สูงขึ้น ดังภาพที่ 2-9



ภาพที่ 2-9 แผนภาพการทำงานของ AdaBoost

นอกจาก AdaBoost แล้วยังมี Gradient Boosting (Friedman 2001) และ XGBoost (Chen and Guestrin 2016) Gradient Boosting ใช้วิธีลดค่าสูญเสีย (Loss) โดยการสร้างต้นไม้ชุดใหม่ที่ประมาณค่าคลาดเคลื่อน (Residual) ของแบบจำลองก่อนหน้านี้อย่างต่อเนื่อง ซึ่งเปรียบเสมือนการไล่ตามทิศทางของ Gradient จึงให้ผลแม่นยำสูงขึ้นแบบเป็นขั้นตอน ส่วน XGBoost พัฒนาจาก Gradient Boosting ด้วยการเพิ่มฟังก์ชันการปรับวงนัยทั่วไป (Regularization) และเทคนิคการคำนวณที่รวดเร็ว ข้อดีของ Boosting คือช่วยลดความเบี่ยงเบนได้ดี แต่ข้อเสียคืออาจไวต่อข้อมูลรบกวน (Noisy Data) และต้องฝึกแบบตามลำดับจึงใช้เวลามากกว่า โดยสรุป Boosting คือ วิธีการโดยทั่วไปที่มีประสิทธิภาพสูงในการสร้างแบบจำลองทำนายที่แม่นยำมาก โดยการรวมกฎตัดสินใจอย่างง่ายหลายกฎเข้าด้วยกัน

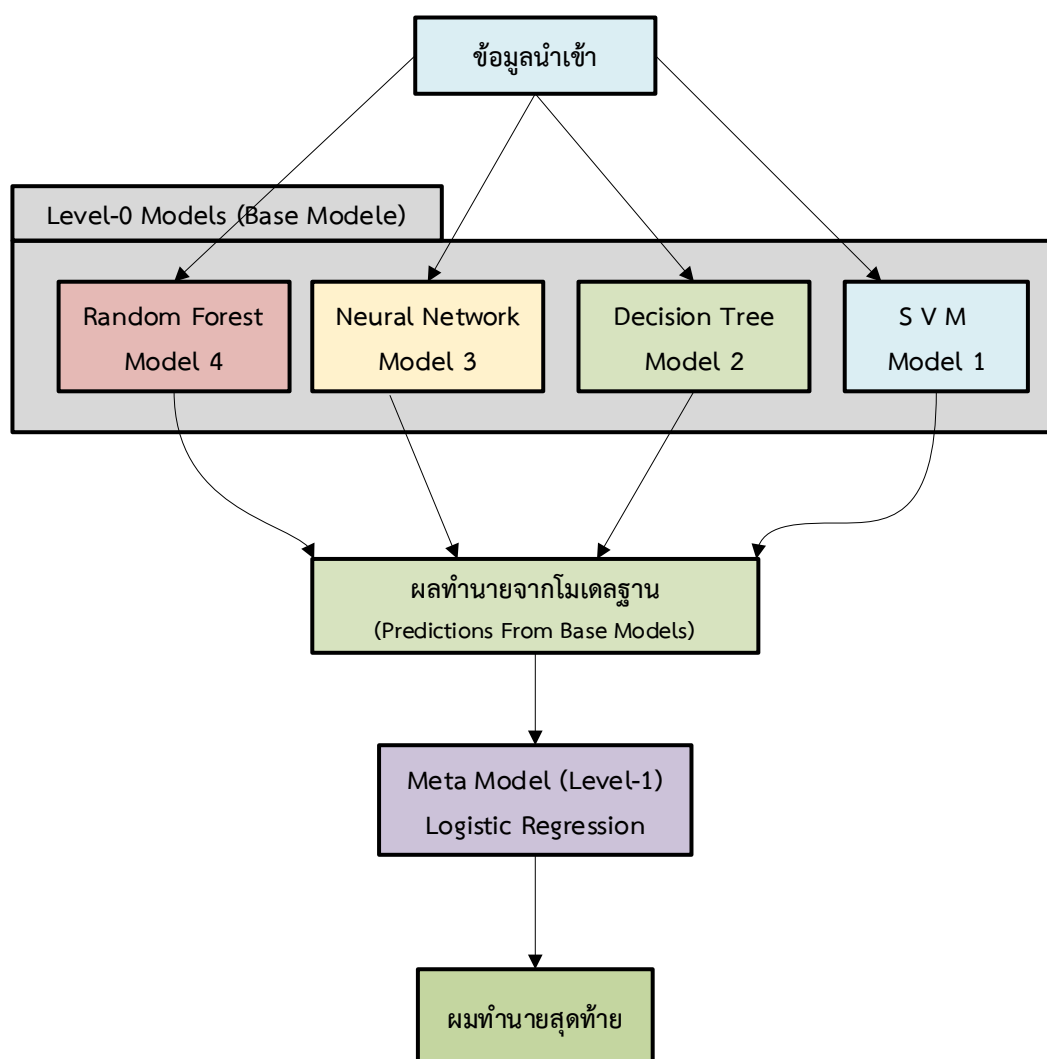
2.5.4 การเรียงซ้อนแบบจำลอง (Stacked Generalization)

การเรียงซ้อนแบบจำลอง (Stacked Generalization) เป็นเทคนิคการเรียนรู้แบบผสม (Ensemble) ที่ใช้แบบจำลองอีกตัวหนึ่งเรียกว่า Meta-learner เพื่อรวมคำทำนายจากแบบจำลองฐาน (Base Models) หลายตัวเข้าด้วยกัน ขั้นตอนทั่วไปดังนี้

2.5.4.1 สร้างแบบจำลองฐาน (Level-0) หลายตัว โดยอาจเป็นแบบจำลองคนละประเภท เช่น Support Vector Machine, Decision Tree และเครือข่ายประสาทเทียม เป็นต้น

2.5.4.2 นำผลคำทำนายจากแบบจำลองฐานมาเป็นข้อมูลป้อนเข้าสู่ Meta-learner (Level-1)

2.5.4.3 ฝึก Meta-model เช่น Logistic Regression เป็นต้น บนข้อมูลชุดนี้เพื่อทำนายผลสุดท้าย ดังภาพที่ 2-10

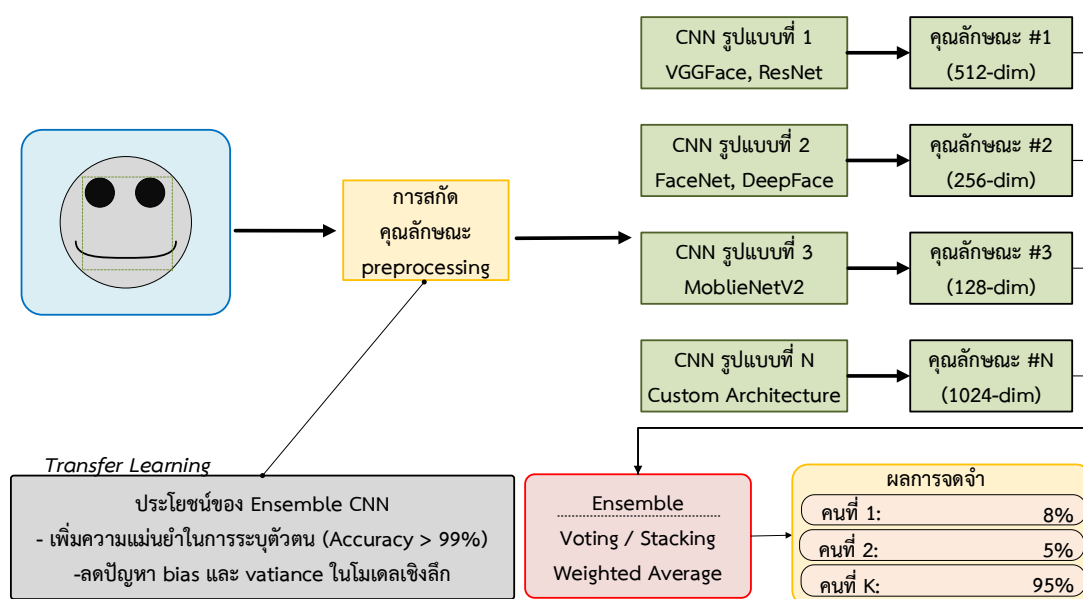


ภาพที่ 2-10 แผนภาพการเรียงซ้อนแบบจำลอง (Stacked Ensemble) (Wolpert, 1992)

วิธีนี้ช่วยให้แบบจำลองผสมเรียนรู้วิธีปรับน้ำหนักหรือผลของแบบจำลองฐานต่าง ๆ อย่างเหมาะสม ทำให้ได้ประสิทธิภาพรวมสูงกว่าแบบจำลองใดแบบจำลองหนึ่งเพียงอย่างเดียว ข้อดีของการเรียงซ้อน (Stacking) คือสามารถรวมแบบจำลองหลากหลายชนิดเข้าด้วยกัน (Heterogeneous Ensemble) จึงมักจับรูปแบบข้อมูลได้หลากหลายยิ่งขึ้น แต่มีข้อเสียคือต้องระวังเรื่องการเรียนรู้ข้อมูลฝึกมากเกินไป (Overfitting) ของ Meta-model โดยเฉพาะถ้าขนาดข้อมูลไม่มากพอ การเรียงซ้อน (Stacking) เป็นแนวคิดที่พัฒนาโดย Wolpert (1992) เพื่อลดอัตราความผิดพลาดโดยฝึก Meta-learner มารวมคำตอบจากแบบจำลองทั่วไปหลายตัว

2.5.5 ความเกี่ยวข้องกับการเรียนรู้เชิงลึก (Deep Learning) และ Convolutional Neural Network

การเรียนรู้แบบผสม (Ensemble Learning) สามารถนำมาใช้ร่วมกับแบบจำลองเชิงลึกได้ โดยเฉพาะการผสมผลของ Convolutional Neural Network หลายสถาปัตยกรรมเข้าด้วยกัน เช่น ในงานการรู้จำใบหน้า อาจฝึกเครือข่าย Convolutional Neural Network หลายรุ่น และรวมคำทำนายด้วย Voting หรือ Stacking เพื่อเพิ่มความแม่นยำ เป็นต้น ดังภาพที่ 2-11



ภาพที่ 2-11 แสดงระบบ Ensemble CNN (Henrydeng, 2021)

เทคนิค เช่น Dropout ในโครงข่ายประสาทเทียมถือได้ว่าเป็นการฝึกการเรียนรู้แบบผสม (Ensemble) ที่เปรียบเสมือนการเฉลี่ยแบบจำลองย่อยที่เกิดจากการสุ่มปิด Neuron หรือเรียกว่า Implicit Bagging ทำให้แบบจำลองเชิงลึกมีความทนทานต่อการเรียนรู้ข้อมูลฝึกมากเกินไป (Overfitting) งานวิจัยหลายชิ้นพบว่า การผสมผสานแบบผสมของ Convolutional Neural Network (Hybrid Ensemble Convolutional Neural Network) ซึ่งใช้การเรียนรู้แบบถ่ายโอน (Transfer Learning) ร่วมกับการรวมแบบจำลองสามารถเพิ่มความแม่นยำ (Accuracy) ในการรู้จำใบหน้าได้สูงกว่าร้อยละ 99 งานวิจัยเห็นว่า การผสมผสานเชิงลึก (Deep Ensemble) จะช่วยปรับปรุงการจับคุณลักษณะเชิงซ้อนของใบหน้าได้ดีขึ้น และช่วยลดปัญหาการเรียนรู้ของแบบจำลอง

เชิงลึกที่มีความเบี่ยงเบน (Bias) หรือความแปรปรวน (Variance) สูงได้เช่นเดียวกับกรณีของแบบจำลองทั่วไป

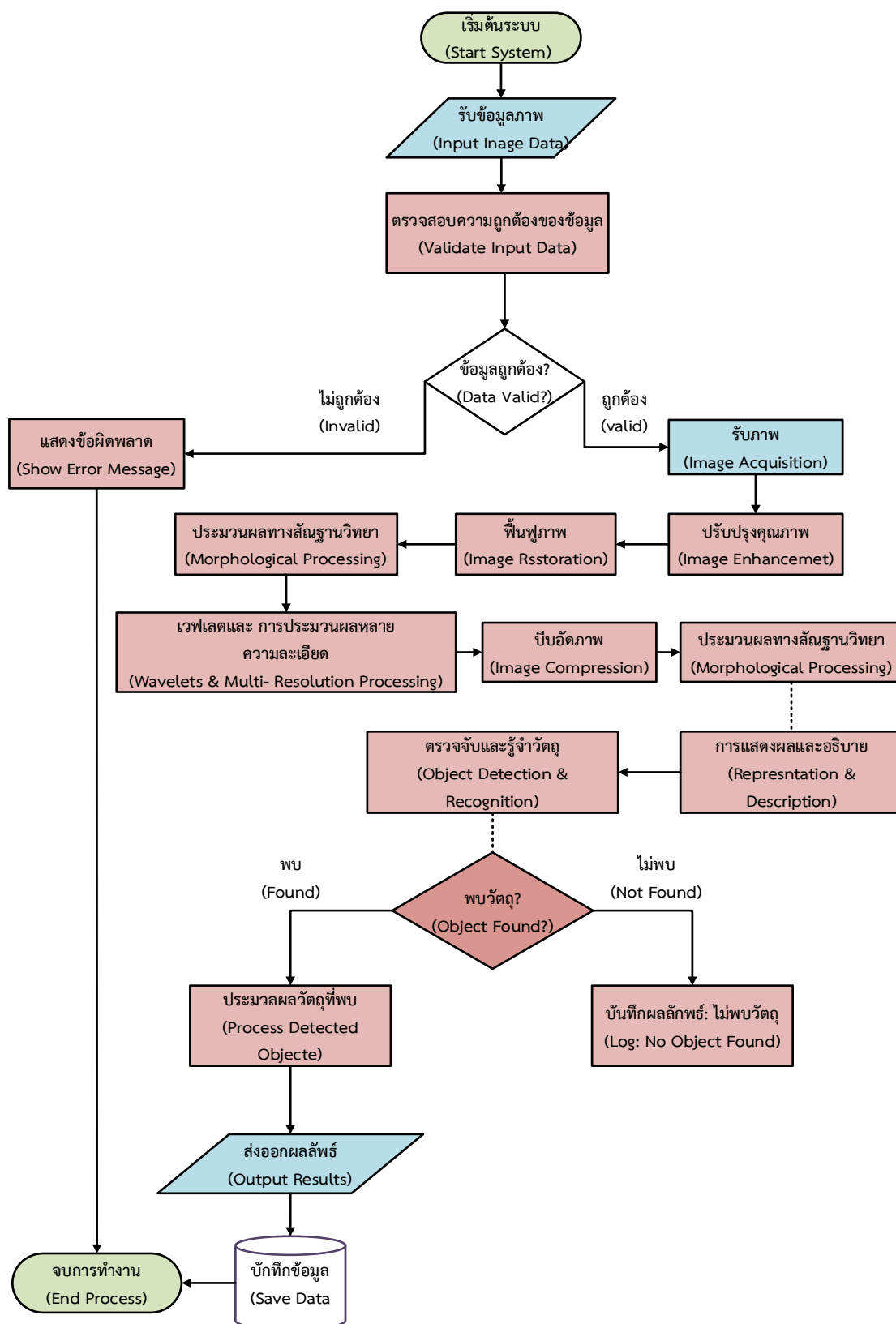
2.6 การประมวลผลภาพดิจิทัล (Digital Image Processing)

การประมวลผลภาพดิจิทัลเป็นกระบวนการที่นำภาพถ่ายหรือภาพจากกล้องมาปรับแต่ง และวิเคราะห์ เพื่อเตรียมข้อมูลให้เหมาะสมสำหรับขั้นตอนวิธีต่าง ๆ เช่น การตรวจจับใบหน้า การรู้จำใบหน้า หรือการจำแนกภาพ เป็นต้น ส่วนนี้จะกล่าวถึงองค์ประกอบสำคัญของการประมวลผลภาพที่นำมาใช้ในระบบแพลตฟอร์มสื่อสังคมออนไลน์

2.6.1 แนวคิดทฤษฎีพื้นฐาน

การประมวลผลภาพดิจิทัลเป็นสาขาย่อยของวิทยาการคอมพิวเตอร์ที่มุ่งเน้นการวิเคราะห์ และจัดการข้อมูลภาพผ่านขั้นตอนวิธีคอมพิวเตอร์ (González and Woods, 2018) กระบวนการพื้นฐานประกอบด้วย การรับข้อมูลภาพ การปรับปรุงคุณภาพภาพ การสกัดคุณลักษณะ และการวิเคราะห์ภาพ โดยเริ่มต้นจากการแปลงภาพ Analog เป็นข้อมูล Digital ในรูปแบบ Matrix ของค่า Pixel ซึ่งค่าเหล่านี้แสดงระดับความเข้มของแสงหรือสีในภาพ ภาพสีมักอยู่ในรูปแบบ RGB แดง เขียว น้ำเงิน โดยแต่ละช่องสีจะถูกเก็บเป็น Matrix แยกกัน

เทคนิคพื้นฐานที่ใช้กันอย่างแพร่หลายในการประมวลผลภาพประกอบด้วย การปรับความสว่างและ Contrast การกรองสัญญาณรบกวน การแปลงเชิงเรขาคณิต การหมุน การขยาย การบิดเบือน และการแยกวัตถุที่สนใจออกจากพื้นหลัง เป็นต้น การพัฒนาของเทคโนโลยีวิสัยทัศน์คอมพิวเตอร์ (Computer Vision) ทำให้เกิดขั้นตอนวิธีที่ซับซ้อนมากขึ้น เช่น การตรวจจับใบหน้า การรู้จำวัตถุ และการเรียนรู้เชิงลึก เป็นต้น ดังภาพที่ 2-12



ภาพที่ 2-12 กระบวนการประมวลผลภาพดิจิทัลพื้นฐานสำหรับงานวิเคราะห์ภาพ (Szeliski, 2022)

ในบริบทของการรู้จำใบหน้า การประมวลผลภาพดิจิทัลมีบทบาทสำคัญในการเตรียมข้อมูลภาพใบหน้าให้พร้อมสำหรับการวิเคราะห์ และจดจำ เทคนิคเฉพาะทางได้ถูกพัฒนาขึ้นสำหรับการตรวจจับ ปรับแนว และแปลงคุณลักษณะใบหน้าเพื่อให้ระบบการรู้จำใบหน้าทำงานได้แม่นยำในสภาพแวดล้อมที่หลากหลาย (Zhao et al., 2003) Library ที่ได้รับความนิยมในการประมวลผลภาพดิจิทัล เช่น OpenCV, Scikit-image และ Pillow เป็นต้น ซึ่งให้เครื่องมือสำหรับการประมวลผลภาพขั้นพื้นฐานจนถึงขั้นสูง ทำให้การพัฒนากระบวนรูจำใบหน้าสามารถทำได้สะดวก และมีประสิทธิภาพมากขึ้น

2.6.2 การปรับระดับแสง และ Contrast ของภาพ

การปรับความสว่าง และ Contrast ของภาพเป็นขั้นตอนสำคัญเพื่อให้ได้ภาพที่ชัดเจน และมีคุณภาพดีก่อนป้อนเข้าสู่แบบจำลองหรือขั้นตอนวิเคราะห์ต่อไป ในงานประมวลผลใบหน้า มักนิยมใช้เทคนิค Histogram Equalization และ Contrast Limited Adaptive Histogram Equalization (CLAHE) เพื่อปรับปรุงความสว่าง และ Contrast ของภาพใบหน้าแต่ละภาพให้สม่ำเสมอ (Pizer et al., 1987) ภาพที่ผ่านการปรับ Histogram จะมีการกระจายของความเข้มแสงที่สมดุล ลดปัญหาภาพมืดหรือสว่างเกินไป ทำให้คุณลักษณะใบหน้าโดดเด่นขึ้น

CLAHE เป็นเทคนิคปรับ Histogram แบบแบ่งภาพเป็นส่วนย่อย และจำกัดการขยาย Contrast ในแต่ละส่วน ทำให้สามารถเพิ่มความคมชัดของรายละเอียดในบริเวณมืดหรือสว่าง โดยไม่เกิดสัญญาณรบกวนมากเกินไปเมื่อเทียบกับวิธี Histogram Equalization ทั้งทั้งภาพ ดังภาพที่ 2-13



ภาพที่ 2-13 การเปรียบเทียบภาพใบหน้าก่อนและหลังการปรับ Contrast ด้วย CLAHE
(Pizer et al., 1987)

นอกจากการปรับ Histogram แล้ว ในระบบการรู้จำใบหน้าที่ยังมีการปรับความสว่าง และ Contrast แบบสุ่มเป็นส่วนหนึ่งของกระบวนการเตรียมข้อมูลก่อนการฝึกแบบจำลอง การปรับค่าความสว่าง และสีเหล่านี้ช่วยให้แบบจำลองมีความทนทานต่อความหลากหลายของสภาพแสงในสถานการณ์จริง ลดการพึ่งพาสภาพแสงใดแสงหนึ่งในข้อมูลฝึก (Mahajan and Kharate, 2017)

2.6.3 การลดสัญญาณรบกวน (Noise Reduction)

ภาพถ่าย Digital มักมีสัญญาณรบกวน (Noise) ปะปนมา เช่น จุดรบกวนแบบเกลือและพริกไทย (Salt-and-pepper) หรือสัญญาณรบกวนจาก Sensor ของกล้อง การลดสัญญาณรบกวนจึงเป็นขั้นตอนสำคัญเพื่อให้ได้ภาพที่ใส และชัดเจนยิ่งขึ้น วิธีการกรองสัญญาณรบกวนที่นิยมมีหลายวิธี เช่น Gaussian Blur และ Median Filter เป็นต้น (Wang and Brandlik, 2008)

Gaussian Blur จะช่วยลดสัญญาณรบกวน และทำให้ภาพเรียบเนียนขึ้นโดยการเฉลี่ยค่าน้ำหนักรอบข้างแบบ Gaussian ในขณะที่ Median Filter เป็นการแทนค่า Pixel ด้วยมัธยฐานของ Pixel รอบข้าง ซึ่งสามารถกำจัดสัญญาณรบกวนประเภทจุดสว่างและจุดมืดได้อย่างมีประสิทธิภาพ โดยยังคงรักษาความคมชัดของขอบภาพไว้ได้ดี ดังภาพที่ 2-14



ภาพที่ 2-14 ผลลัพธ์การลดสัญญาณรบกวนด้วย Median Filter (Janert, 2019)

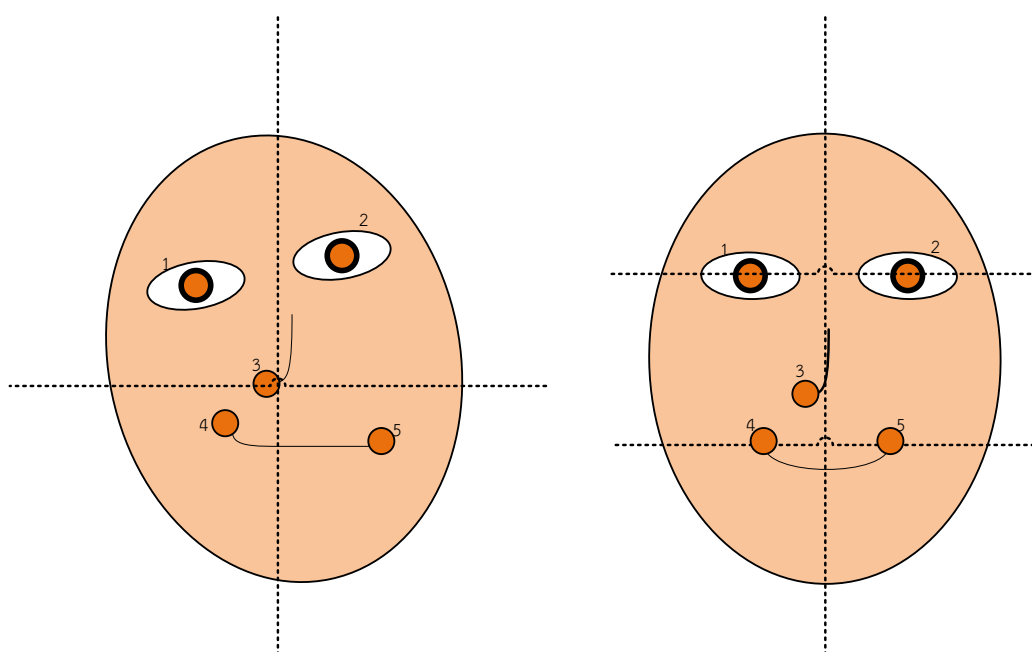
การลดสัญญาณรบกวนก่อนจะช่วยให้ขั้นตอนการตรวจจับคุณลักษณะ เช่น การหาใบหน้าหรือจุดสังเกตบนใบหน้า เป็นต้น แม่นยำขึ้น เนื่องจากไม่ถูกรบกวนด้วยข้อมูลที่ไม่เกี่ยวข้อง นอกจากนี้ การลดสัญญาณรบกวนยังสัมพันธ์กับการปรับความคมชัด และ Contrast เนื่องจากการเพิ่ม Contrast ที่มากเกินไปอาจทำให้สัญญาณรบกวนเด่นขึ้น จึงควรมีการกรองสัญญาณรบกวนก่อนหรือควบคู่กันไป (Tomasini and Manduchi, 1998)

2.6.4 การปรับให้เป็นมาตรฐาน (Normalization) การปรับขนาด และการจัดแนวภาพ

การปรับให้เป็นมาตรฐาน (Normalization) ในบริบทของภาพหมายถึงการปรับช่วงค่า Pixel ให้อยู่ในช่วงมาตรฐาน เช่น 0 ถึง 1 หรือ -1 ถึง 1 เป็นต้น เพื่อให้ข้อมูลภาพมี Scale ที่แบบจำลองเรียนรู้ได้ง่ายขึ้น และช่วยเร่งให้การฝึกแบบจำลองมีความเสถียร (LeCun et al., 1998) การปรับให้เป็นมาตรฐานของค่า Pixel โดยหารด้วย 255 และปรับค่าให้อยู่ในช่วง -1 ถึง 1 เป็นวิธีที่นิยมใช้กันทั่วไปในการนำเข้าสู่ข้อมูลภาพสู่แบบจำลองการเรียนรู้เชิงลึก

การปรับขนาดภาพ (Resizing) เป็นการปรับใบหน้าให้ได้ขนาดมาตรฐานก่อนการปรับให้เป็นมาตรฐาน การปรับขนาดภาพให้เท่ากันเป็นสิ่งจำเป็นเพื่อให้แบบจำลอง Convolutional Neural Network สามารถประมวลผลกลุ่มข้อมูลของภาพได้อย่างถูกต้อง และเพื่อให้คุณลักษณะที่แบบจำลองเรียนรู้มีความหมายตรงกันในทุกภาพ (Huang et al., 2008)

การจัดแนวใบหน้า (Face Alignment) เป็นการปรับหมุน และขนาดของใบหน้าให้ได้รูปแบบที่เป็นมาตรฐาน เช่น ตำแหน่งและระยะของตาสองข้างอยู่ระดับเดียวกัน เป็นต้น การจัดแนวช่วยลดความผันแปรจากการที่ใบหน้าหันเอียงหรือเอียงศีรษะ ทำให้ภาพใบหน้าทุกภาพมีมุมมองที่ใกล้เคียงกัน ดังภาพที่ 2-15



ภาพที่ 2-15 การเปรียบเทียบภาพใบหน้าก่อนและหลังการจัดแนว (Cao et al., 2014)

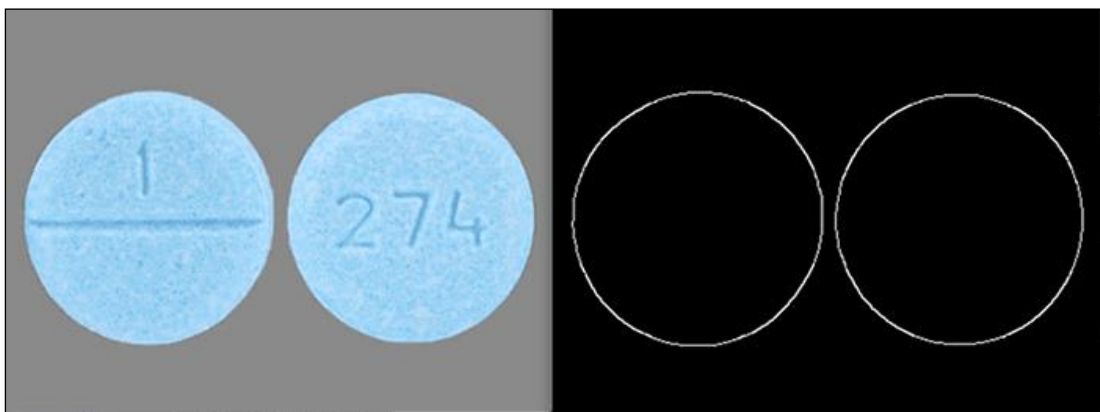
การศึกษาหลายชิ้นพบว่าการจัดแนวใบหน้าก่อนการรู้จำสามารถเพิ่มความแม่นยำของแบบจำลองได้อย่างมีนัยสำคัญ (Kumar et al., 2016) โดยขั้นตอนนี้มักใช้การตรวจหาจุดสังเกตบนใบหน้า (Facial Landmarks) แล้วใช้การแปลงเชิงเรขาคณิต (Affine Transform) เพื่อหมุน และปรับขนาดใบหน้าให้ได้มาตรฐาน

2.6.5 การวิเคราะห์คุณลักษณะของภาพ

หลังจากปรับปรุงคุณภาพของภาพแล้ว การวิเคราะห์คุณลักษณะเบื้องต้นของภาพสามารถช่วยในการดึงข้อมูลสำคัญออกมา การตรวจจับขอบภาพ (Edge Detection) เป็นเทคนิคที่ใช้ตรวจหาการเปลี่ยนแปลงความเข้มของ Pixel อย่างทันทีทันใด ซึ่งมักบ่งบอกถึงขอบเขตของวัตถุหรือรายละเอียดในภาพ (Canny, 1986)

ขอบภาพคือบริเวณที่ค่าความสว่างของ Pixel มีความไม่ต่อเนื่องอย่าง (Intensity Discontinuity) ซึ่งเป็นบริเวณที่สีหรือความสว่างเปลี่ยนแปลงอย่างฉับพลัน การหาขอบภาพมี

ประโยชน์ในการแยกส่วนประกอบของภาพ เช่น รูปร่างของใบหน้า และคุณลักษณะเด่นบนใบหน้า เป็นต้น ดังภาพที่ 2-16



ภาพที่ 2-16 ตัวอย่างการตรวจจับขอบภาพด้วยวิธี Canny (Canny, 1986)

การศึกษาหลายชิ้นพบว่าการจัดแนวใบหนีก่อนการรู้จำสามารถเพิ่มความแม่นยำของแบบจำลองได้อย่างมีนัยสำคัญ (Kumar et al., 2016) โดยขั้นตอนนี้มักใช้การตรวจหาจุดสังเกตบนใบหน้า (Facial Landmarks) แล้วใช้การแปลงเชิงเรขาคณิต (Affine Transform) เพื่อหมุน และปรับขนาดใบหน้าให้ได้มาตรฐาน ดังภาพที่ 2-16



ภาพที่ 2-17 เทคนิคการเพิ่มข้อมูลภาพ: ตัดขอบ, พลิกแนวนอน, หมุน และปรับความสว่าง
(Shorten and Khoshgoftaar, 2019)

การเพิ่มข้อมูลลักษณะนี้ช่วยให้แบบจำลองเรียนรู้ความหลากหลาย และสามารถระบุบุคคลได้ถูกต้องแม้รูป Input จะมีการพลิกหรือแสงเปลี่ยนไป (Wang et al., 2017) ระบบการรู้จำใบหน้าที่ได้รับการฝึกด้วยข้อมูลที่ผ่านการ Augmentation มักมีความทนทานต่อการเปลี่ยนแปลงของสภาพแวดล้อมทั้งแสง มุมมอง และอากัปกิริยาได้ดียิ่งขึ้น

2.7 สถาปัตยกรรม Microservices (Microservices Architecture)

สถาปัตยกรรม Microservices เป็นรูปแบบการออกแบบ และพัฒนาระบบ Software ที่แบ่ง Application ออกเป็นบริการขนาดเล็กจำนวนมาก แต่ละบริการทำงานแยกจากกันโดยสมบูรณ์ และสื่อสารระหว่างกันผ่านกลไกที่มีน้ำหนักเบา เช่น การเรียกใช้ผ่าน RESTful API หรือตัวกลางส่ง

ข้อความ เป็นต้น (Newman, 2015) กระบวนทัศน์นี้นับเป็นวิวัฒนาการจากแนวคิด Service-Oriented Architecture โดยปรับให้มีความยืดหยุ่น และลดการรวมศูนย์ลง

2.7.1 แนวคิดทฤษฎีพื้นฐาน

แนวคิดหลักของสถาปัตยกรรม Microservices คือการแยกระบบ Software ขนาดใหญ่ ออกเป็นบริการย่อยที่เป็นอิสระต่อกัน โดยแต่ละบริการย่อยมีหน้าที่เฉพาะ และชัดเจน สามารถ พัฒนา ทดสอบ และปรับใช้งานได้โดยไม่ส่งผลกระทบต่อบริการอื่น ๆ (Lewis and Fowler, 2014) การพัฒนา Software ในแนวทางนี้ช่วยให้แต่ละทีมสามารถทำงานบนส่วนย่อยของตนได้อย่าง คล่องตัว ลดความซับซ้อนในการพัฒนาระบบขนาดใหญ่ และเพิ่มความยืดหยุ่นในการปรับขยาย และ บำรุงรักษาหลักการพื้นฐานของสถาปัตยกรรม Microservices ประกอบด้วยดังนี้

2.7.1.1 การแยกตามโดเมนธุรกิจ (Domain-Driven Design) ระบบถูกแบ่งออกเป็น บริการย่อยตามขอบเขตของงานทางธุรกิจที่เฉพาะเจาะจง (Evans, 2004)

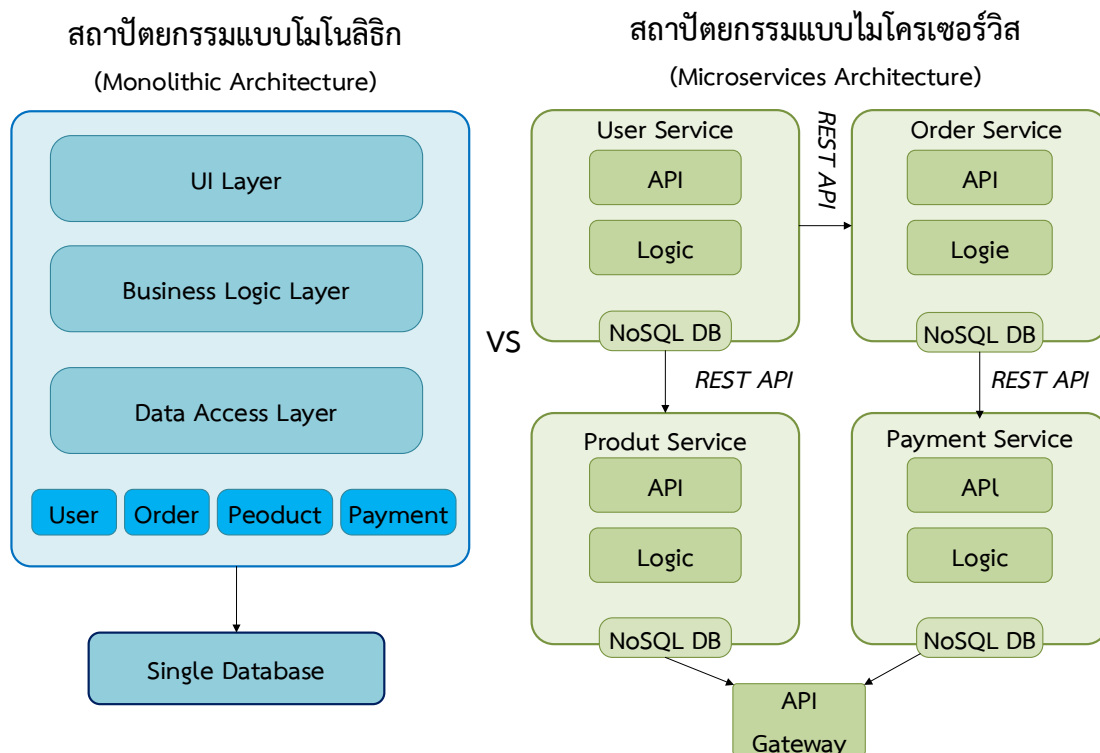
2.7.1.2 การทำงานอย่างอิสระ (Independent Deployment) บริการแต่ละตัวสามารถ พัฒนา และปรับใช้ได้แยกจากกันอย่างสิ้นเชิง

2.7.1.3 การสื่อสารผ่าน API ที่ชัดเจน (Well-defined API) บริการย่อยสื่อสารระหว่าง กันผ่าน Interface ที่กำหนดไว้ชัดเจน เพื่อให้องค์ประกอบต่าง ๆ เชื่อมต่อกันแบบหลวม (Richardson, 2019)

2.7.1.4 ความหลากหลายด้านเทคโนโลยี (Polyglot Persistence) แต่ละบริการสามารถใช้เทคโนโลยี ฐานข้อมูล หรือภาษาในการพัฒนาที่แตกต่างกันได้ตามความเหมาะสม

2.7.1.5 การจัดการแบบกระจาย (Decentralized Governance) ลดการจัดการแบบ รวมศูนย์ให้เหลือน้อยที่สุด (Newman, 2015)

สถาปัตยกรรม Microservices มีจุดเด่นในด้านความยืดหยุ่น และความสามารถในการปรับ ขยาย โดยสามารถเพิ่มหรือลดขนาดของแต่ละบริการได้อย่างอิสระตามปริมาณงาน ทำให้รองรับผู้ใช้ จำนวนมากได้อย่างมีประสิทธิภาพ นอกจากนี้ยังมีความพร้อมใช้งานสูง เนื่องจากหากบริการใด ล้มเหลว จะส่งผลกระทบเฉพาะส่วนนั้น ๆ ไม่ทำให้ระบบทั้งหมดล่ม (Fowler, 2015) ดังภาพที่ 2-18



ภาพที่ 2-18 การเปรียบเทียบสถาปัตยกรรม Monolithic และ Microservices (Lewis and Fowler, 2014)

2.7.2 องค์ประกอบหลักของระบบ Microservices

ระบบที่พัฒนาด้วยสถาปัตยกรรม Microservices ประกอบด้วยองค์ประกอบหลัก ดังนี้

2.7.2.1 API Gateway ทำหน้าที่เป็นจุดเชื่อมต่อเดียวสำหรับฝั่ง Client ช่วยจัดการการเรียกใช้บริการย่อยต่าง ๆ (Richardson, 2019)

2.7.2.2 Microservices ระบบย่อยที่แยกตามขอบเขตงาน เช่น บริการพิสูจน์ตัวตนผู้ใช้ บริการข้อมูลผู้ใช้ บริการเนื้อหา เป็นต้น แต่ละบริการมีฐานข้อมูลของตัวเอง และทำงานเป็นเอกเทศ

2.7.2.3 ระบบส่งข้อความ (Message Broker) ช่วยในการสื่อสารระหว่างบริการแบบ Asynchronous เช่น RabbitMQ และ Kafka เป็นต้น (Fowler, 2017)

2.7.2.4 ระบบติดตาม และตรวจสอบ (Monitoring System) คอยเฝ้าดูประสิทธิภาพ และความพร้อมใช้งานของบริการต่าง ๆ

2.7.2.5 ระบบค้นพบบริการ (Service Discovery) ช่วยให้บริการต่าง ๆ สามารถค้นหา และเชื่อมต่อกันได้แบบ Dynamic (Newman, 2015)

โครงสร้างนี้ช่วยให้ระบบมีความยืดหยุ่นสูง สามารถปรับเปลี่ยนหรือขยายส่วนใดส่วนหนึ่งได้ โดยไม่กระทบทั้งระบบ และเอื้อต่อการทำงานของทีมนักพัฒนาหลายทีมพร้อมกัน (Nadareishvili et al., 2016)

2.7.3 ข้อดี และข้อท้าทายของสถาปัตยกรรม Microservices

สถาปัตยกรรม Microservices นำเสนอข้อดีหลายประการเมื่อเทียบกับสถาปัตยกรรมแบบดั้งเดิม ดังนี้

2.7.3.1 ความยืดหยุ่น และปรับขยายได้สูง สามารถปรับเพิ่มหรือลดขนาดของแต่ละบริการได้อย่างอิสระตามปริมาณงาน (Richardson, 2019)

2.7.3.2 ความพร้อมใช้งานสูง หากบริการใดบริการหนึ่งล้มเหลว จะส่งผลกระทบต่อเฉพาะส่วนนั้น ๆ เท่านั้น (Lewis และ Fowler, 2014)

2.7.3.3 ส่งมอบ Feature ได้รวดเร็ว ทีมพัฒนาสามารถปรับปรุงหรือเพิ่ม Feature ในบริการใดบริการหนึ่ง และปรับใช้ได้ทันที (Newman, 2015)

2.7.3.4 จัดการ และดูแลรักษาง่าย Code และขอบเขตงานของแต่ละบริการมีขนาดเล็ก และเฉพาะเจาะจง (Fowler, 2015)

2.7.3.5 รองรับการพัฒนาแบบแยกทีม ทีมงานหลายทีมสามารถทำงานขนานกันบนบริการคนละตัวได้โดยไม่รบกวนกัน (Richardson, 2019)

อย่างไรก็ตาม สถาปัตยกรรม Microservices ก็มาพร้อมกับความท้าทายที่ต้องพิจารณา ดังนี้

2.7.3.6 ความซับซ้อนในการจัดการระบบโดยรวม ต้องอาศัยเครื่องมือ และแนวทาง DevOps ขั้นสูง (Newman, 2015)

2.7.3.7 การสื่อสารระหว่างบริการมีความซับซ้อน การเรียกทำงานข้ามบริการต้องทำผ่านเครือข่าย ซึ่งมีค่าใช้จ่ายด้านประสิทธิภาพ (Richardson, 2019)

2.7.3.8 ความยุ่งยากในการรักษาความสอดคล้องของข้อมูล เมื่อข้อมูลกระจายไปเก็บในหลายบริการ (Fowler, 2015)

2.7.3.9 การทดสอบ และ Debug ทำได้ยากขึ้น ต้องทดสอบการทำงานร่วมกันของหลายบริการ (Newman, 2015)

2.7.3.10 ต้นทุนทรัพยากรระบบที่สูงขึ้น แต่ละบริการต้องมีทรัพยากรของตัวเอง (Richardson, 2019)

2.7.4 ตัวอย่างการใช้สถาปัตยกรรม Microservices ในระบบจริง

แนวทาง Microservices ได้รับความนิยมอย่างแพร่หลายในระบบขนาดใหญ่สมัยใหม่ โดยเฉพาะในระบบ Web หรือ Social Network ที่มีฟังก์ชันการทำงานหลากหลาย ในระบบลักษณะนี้ มักมีการแยกบริการตามหน้าที่งาน ดังนี้

2.7.4.1 บริการพิสูจน์ตัวตนผู้ใช้ (Authentication Service) จัดการฟังก์ชันการลงชื่อเข้าใช้ การลงทะเบียนผู้ใช้ใหม่ และการตรวจสอบสิทธิ์ (Wolff, 2016)

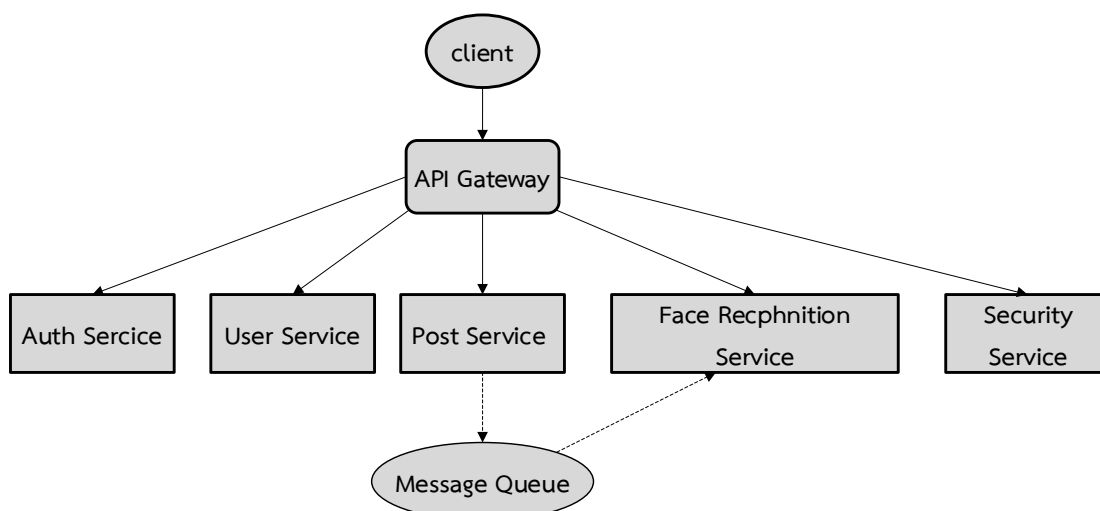
2.7.4.2 บริการข้อมูลผู้ใช้ (User Service) ดูแลข้อมูล Profile ผู้ใช้ และความสัมพันธ์ระหว่างผู้ใช้ (Richardson, 2019)

2.7.4.3 บริการโพสต์หรือเนื้อหา (Post Service) รับผิดชอบการสร้าง และจัดการโพสต์หรือเนื้อหาที่ผู้ใช้ส่งเข้ามา (Nadareishvili et al., 2016)

2.7.4.4 บริการวิเคราะห์ใบหน้า (Face Recognition Service) ทำงานด้านการประมวลผลรูปภาพ และรู้จำใบหน้าภายในระบบ (Wang et al., 2021)

2.7.4.5 บริการตรวจสอบความปลอดภัย (Security Service) คอยตรวจจับ และเฝ้าระวังพฤติกรรมหรือเหตุการณ์ที่อาจเป็นภัยต่อระบบ (Newman, 2015)

ในการทำงานร่วมกันของบริการเหล่านี้ มักมี API Gateway ทำหน้าที่เป็นจุดเชื่อมต่อเดียวสำหรับฝั่ง Client และมีระบบส่งข้อความช่วยในการสื่อสารระหว่างบริการแบบ Asynchronous ดังภาพที่ 2-19



ภาพที่ 2-19 แผนภาพสถาปัตยกรรม Microservices (ที่มา: Richardson, 2019)

2.7.5 การประยุกต์ใช้กับระบบการรู้จำใบหน้า

ในบริบทของระบบการรู้จำใบหน้า สถาปัตยกรรม Microservices มีความเหมาะสมอย่างยิ่งเนื่องจากลักษณะงานที่ต้องใช้การประมวลผลสูง และต้องการการขยายตัวเฉพาะส่วน (Eberhard et al., 2017) ระบบอาจแบ่งเป็นบริการย่อยดังนี้

2.7.5.1 บริการตรวจจับใบหน้า (Face Detection Service) รับผิดชอบการระบุตำแหน่งใบหน้าในภาพ

2.7.5.2 บริการสกัดคุณลักษณะใบหน้า (Feature Extraction Service) ทำการแปลงข้อมูลใบหน้าเป็นเวกเตอร์คุณลักษณะ (Face Embedding)

2.7.5.3 บริการเปรียบเทียบใบหน้า (Face Matching Service) ทำการเปรียบเทียบใบหน้าเพื่อระบุตัวตนหรือตรวจสอบความเหมือน

2.7.5.4 บริการตรวจสอบความถูกต้อง (Verification Service) ตรวจสอบความถูกต้องของภาพใบหน้า เช่น การตรวจจับภาพปลอม เป็นต้น (Wang et al., 2021)

การแยกระบบการรู้จำใบหน้าออกเป็นบริการย่อยช่วยให้สามารถปรับขยายเฉพาะส่วนที่มีภาระงานสูง เช่น การประมวลผลภาพจำนวนมาก เป็นต้น โดยไม่กระทบส่วนอื่น อีกทั้งยังช่วยให้ทีมพัฒนาสามารถเลือกใช้เทคโนโลยีที่เหมาะสมกับแต่ละงาน เช่น การใช้ Python สำหรับงานประมวลผลภาพ และ Go สำหรับงานที่ต้องการประสิทธิภาพสูง เป็นต้น (Wolff, 2016)

บริการรู้จำใบหน้าในสถาปัตยกรรม Microservices ยังสามารถปรับใช้แบบจำลองการเรียนรู้ของเครื่องหลายรูปแบบร่วมกัน เช่น FaceNet ArcFace และ AdaFace เป็นต้น ในรูปแบบการเรียนรู้แบบผสม (Ensemble Learning) เพื่อเพิ่มความแม่นยำในการรู้จำใบหน้า โดยแต่ละแบบจำลองสามารถทำงานเป็นบริการย่อยที่สามารถพัฒนา ปรับปรุง และปรับขยายได้อย่างอิสระ (Wang et al., 2021)

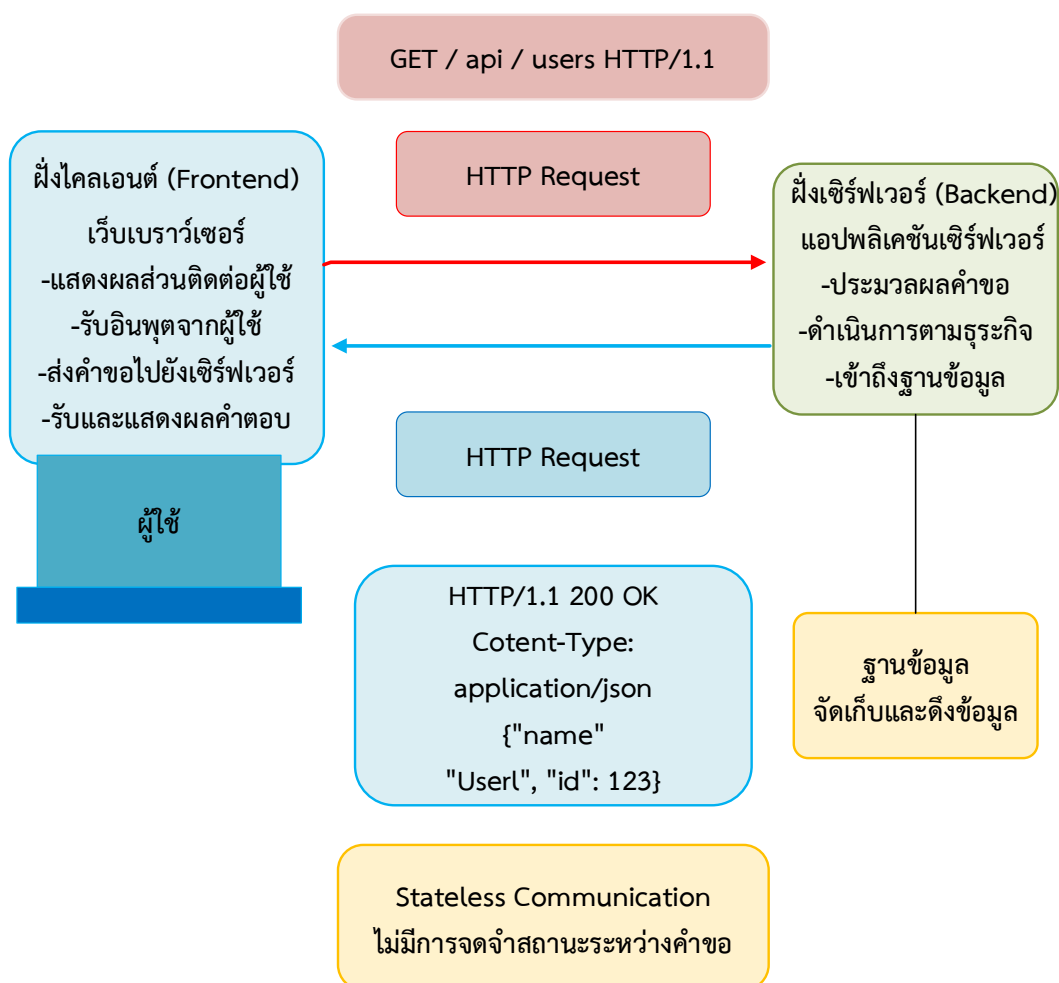
2.8 เทคโนโลยี Web Application (Web Application Technologies)

เทคโนโลยี Web Application เป็นองค์ประกอบสำคัญในการพัฒนาระบบ Software ที่ทำงานผ่านเครือข่าย Internet โดยรวมเอาเทคโนโลยีด้านการแสดงผล การประมวลผล และการสื่อสารข้อมูลเข้าด้วยกัน เพื่อสร้างระบบที่มีประสิทธิภาพ และตอบสนองความต้องการของผู้ใช้ (Richardson and Ruby, 2018) ส่วนนี้จะกล่าวถึงเทคโนโลยีพื้นฐาน และองค์ประกอบหลักของการพัฒนา Web Application สมัยใหม่

2.8.1 แนวคิดทฤษฎีพื้นฐาน

Web Application ทำงานตามสถาปัตยกรรม Client-Server ซึ่งแบ่งการทำงานออกเป็นสองส่วนหลัก ได้แก่ ฝั่ง Client (Frontend) และฝั่ง Server (Backend) โดยฝั่ง Client มักจะเป็น Web Browser หรือ Application ที่ประมวลผลบนอุปกรณ์ของผู้ใช้ ทำหน้าที่จัดแสดงผล และรับ Input จากผู้ใช้ ส่วนฝั่ง Server คือระบบที่อยู่บนเครื่องแม่ข่ายหรือ Cloud ทำหน้าที่ประมวลผลข้อมูล และให้บริการตามคำขอของ Client (Fielding and Taylor, 2002)

การสื่อสารระหว่าง Client และ Server นิยมใช้ Protocol HTTP ในรูปแบบของการรับส่งคำขอและคำตอบ (Request-Response) แต่ละคำขอ HTTP จาก Client จะถูกส่งไปยัง Server ซึ่ง Server จะประมวลผล และส่งคำตอบกลับมายัง Client การสื่อสารลักษณะนี้เป็นแบบ Stateless กล่าวคือแต่ละคำขอจะดำเนินการอย่างอิสระ และไม่มีการจดจำสถานะต่อเนื่องระหว่างคำขอ (Richardson and Ruby, 2018) ดังภาพที่ 2-20



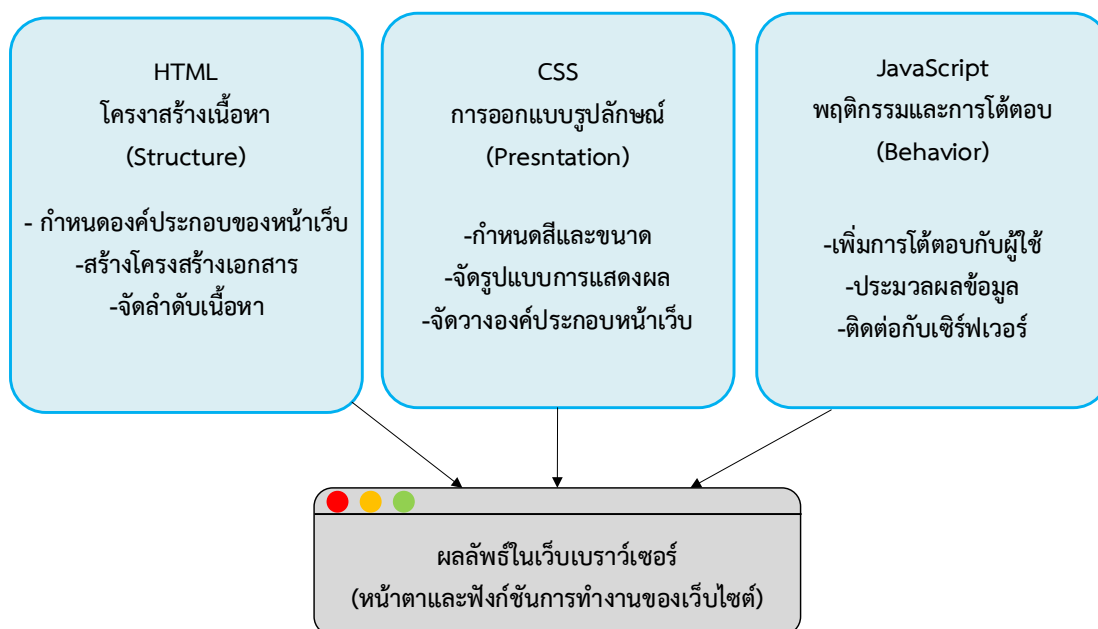
ภาพที่ 2-20 สถาปัตยกรรม Client-Server และการสื่อสารผ่าน Protocol HTTP
(Lewis and Fowler, 2014)

Web Application สมัยใหม่มักออกแบบส่วนติดต่อระหว่าง Client-Server ให้เป็น RESTful API ซึ่งเป็น Interface มาตรฐานสำหรับการแลกเปลี่ยนข้อมูลระหว่างระบบผ่าน Internet อย่างปลอดภัย และมีประสิทธิภาพ โดยใช้หลักการของสถาปัตยกรรมแบบ Representational State Transfer บน Protocol HTTP (Lewis and Fowler, 2014) คำขอที่ Client ส่งผ่าน RESTful API จะประกอบด้วย URL ของทรัพยากรที่ต้องการ และระบุวิธีการ HTTP เช่น GET POST PUT และ DELETE เป็นต้น เพื่อบอก Server ว่าจะดำเนินการอะไรกับทรัพยากรนั้น (Richardson, 2019)

2.8.2 เทคโนโลยีที่ใช้ในฝั่ง Client (Frontend Technologies)

ฝั่ง Client ของ Web Application พัฒนาด้วยเทคโนโลยีหลักสามประเภท ได้แก่ HTML CSS และ JavaScript โดย HTML เป็นภาษามาร์กอัปหลักที่ใช้สร้างโครงสร้างของหน้า Web Page และเนื้อหาต่าง ๆ ส่วน CSS เป็นภาษาที่ใช้กำหนดรูปแบบการแสดงผลของหน้า Web เช่น สี รูปแบบ ตัวอักษร การจัดวาง Layout เป็นต้น และ JavaScript เป็นภาษาโปรแกรมที่ทำงานบน Web Browser ทำหน้าที่ควบคุมพฤติกรรมของหน้า Web (Flanagan, 2020) เทคโนโลยีทั้งสามนี้

ทำงานร่วมกัน โดย HTML สร้างโครงร่างเนื้อหา CSS ตกแต่งหน้าตา และ JavaScript เพิ่มฟังก์ชันการโต้ตอบ ดังภาพที่ 2-21



ภาพที่ 2-21 ความสัมพันธ์ระหว่าง HTML CSS และ JavaScript (Flanagan, 2020)

ในการพัฒนา Web Application สมัยใหม่ นิยมใช้ Framework ฝั่ง Client เช่น React หรือ Vue.js เป็นต้น เพื่อให้การพัฒนาส่วนติดต่อผู้ใช้มีประสิทธิภาพ และจัดการ Code ได้ง่ายขึ้น (Fedosejev, 2019) โดย React เป็น Library ของ JavaScript ที่ใช้สร้างส่วนติดต่อผู้ใช้แบบ Single Page Application อย่างรวดเร็ว และแม่นยำ ผ่านแนวคิดการแบ่งหน้าจออกเป็นส่วนประกอบย่อย (Component-Based Architecture) ทำให้การปรับปรุงหรือเปลี่ยนแปลงส่วนติดต่อผู้ใช้เป็นไปอย่างคงเส้นคงวา และง่ายต่อการดูแล (Banks and Porcello, 2017) นอกจากนี้ ยังมีเทคโนโลยีเสริมอื่น ๆ ที่นิยมใช้ในการพัฒนาฝั่ง Client ตัวอย่างดังนี้

2.8.2.1 Asynchronous JavaScript and XML (AJAX) เทคนิคการรับส่งข้อมูลระหว่าง Client และ Server โดยไม่ต้องโหลดหน้า Web ใหม่ ทำให้ Application ตอบสนองได้อย่างรวดเร็ว (Flanagan, 2020)

2.8.2.2 Single Page Application (SPA) แนวคิดการสร้าง Web Application ที่โหลดเพียงครั้งเดียว และอัปเดตเฉพาะส่วนที่จำเป็น ทำให้ผู้ใช้ได้รับประสบการณ์คล้ายกับ Desktop Application (Banks และ Porcello, 2017)

2.8.2.3 Webpack และ Babel เครื่องมือสำหรับรวม และแปลง Code ต้นฉบับเป็น Code ที่ Browser สามารถเข้าใจได้ (Fedosejev, 2019)

2.8.3 เทคโนโลยีที่ใช้ในฝั่ง Server (Backend Technologies)

ฝั่ง Server ของ Web Application มีหน้าที่ประมวลผลคำขอจาก Client ติดต่อกับฐานข้อมูล และส่งผลลัพธ์กลับไปยัง Client เทคโนโลยีที่นิยมใช้ในการพัฒนาฝั่ง Server มีหลากหลายได้แก่

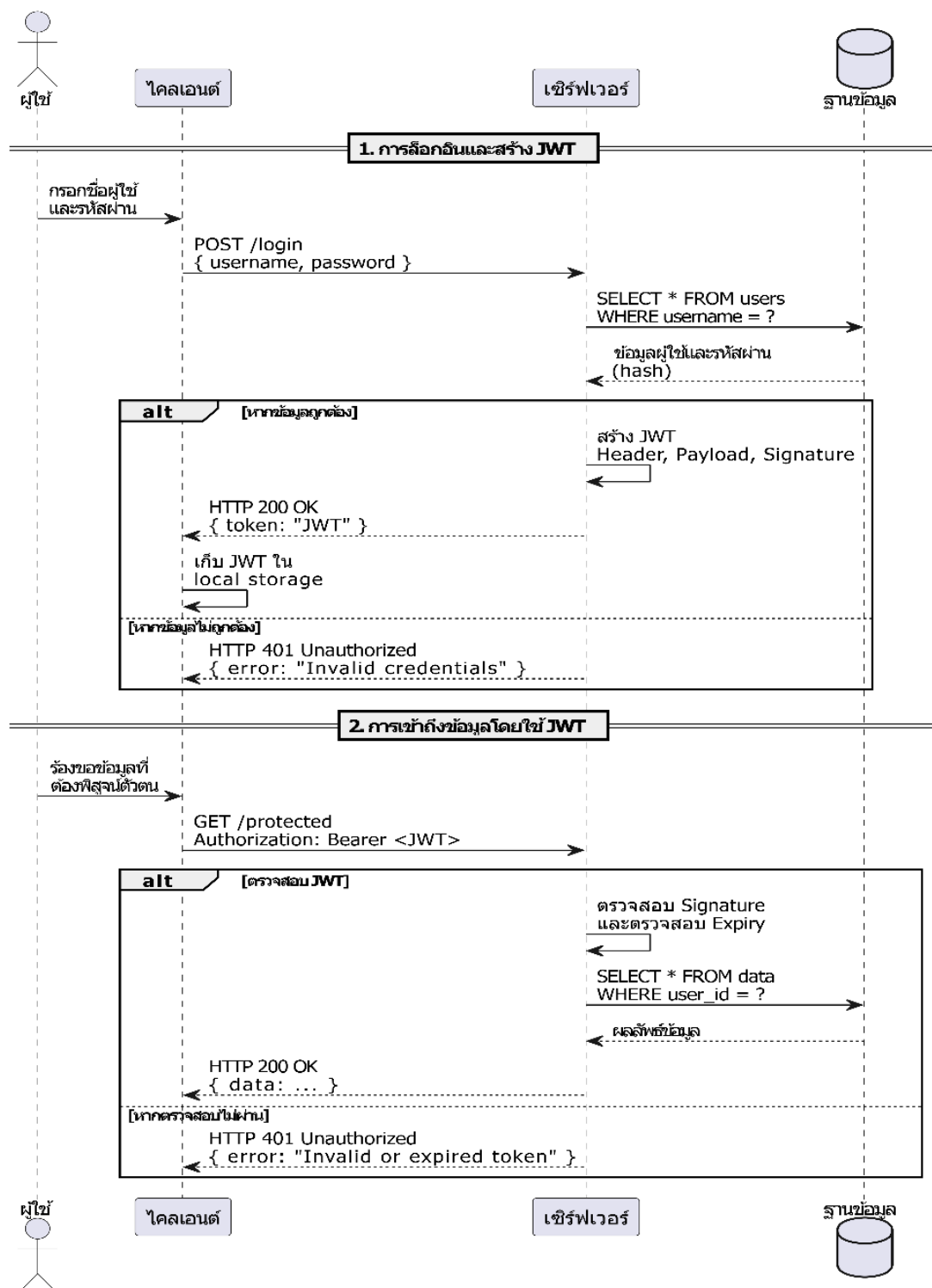
2.8.3.1 ภาษาโปรแกรมฝั่ง Server ภาษาที่ใช้พัฒนาระบบหลังบ้าน เช่น Python Java Node.js PHP และ Ruby เป็นต้น โดยแต่ละภาษามีจุดเด่น และเหมาะกับงานแตกต่างกัน (Gourley and Totty, 2002)

2.8.3.2 ภาษาโปรแกรมฝั่ง Server ภาษาที่ใช้พัฒนาระบบหลังบ้าน เช่น Python Java Node.js PHP และ Ruby เป็นต้น โดยแต่ละภาษามีจุดเด่น และเหมาะกับงานแตกต่างกัน (Gourley and Totty, 2002)

2.8.3.3 ภาษาโปรแกรมฝั่ง Server ภาษาที่ใช้พัฒนาระบบหลังบ้าน เช่น Python Java Node.js PHP และ Ruby เป็นต้น โดยแต่ละภาษามีจุดเด่น และเหมาะกับงานแตกต่างกัน (Gourley and Totty, 2002)

2.8.3.4 Web Server Software ที่รับคำขอ HTTP และส่งต่อให้ Application ประมวลผล เช่น Nginx และ Apache เป็นต้น (Gourley and Totty, 2002)

ในการพัฒนา Web Application สมัยใหม่ ฝั่ง Server มักจะเผยแพร่บริการต่าง ๆ ผ่าน RESTful API ซึ่ง Client สามารถเรียกใช้ได้โดยตรง (Richardson, 2019) โดยมีการนำเทคโนโลยีรักษาความปลอดภัยมาใช้ เช่น การพิสูจน์ตัวตนแบบ JSON Web Token สำหรับยืนยันตัวตน และควบคุมสิทธิ์การเข้าถึงทรัพยากร และการเปิดใช้นโยบาย Cross-Origin Resource Sharing (CORS) เพื่อให้ Client บน Domain ที่แตกต่างสามารถเข้าถึง API ได้อย่างปลอดภัย (Stuttard and Pinto, 2018) ดังภาพที่ 2-22

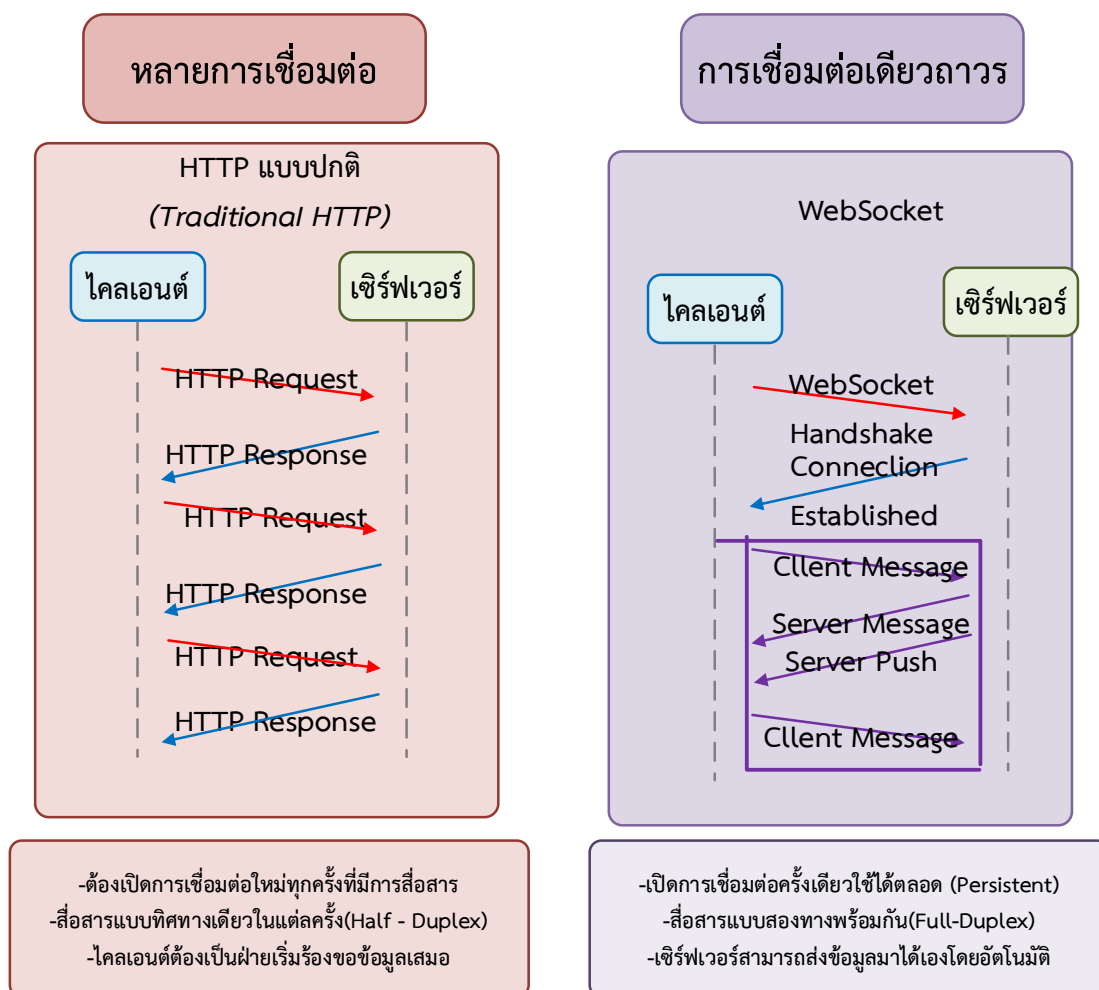


ภาพที่ 2-22 กระบวนการพิสูจน์ตัวตนด้วย JSON Web Token (Stuttard and Pinto, 2018)

2.8.4 การทำงานแบบทันทีทันใด และ WebSocket

ระบบ Web Application บางประเภทต้องการการทำงานแบบทันทีทันใด (Real-time) กล่าวคือ ผู้ใช้จะได้รับข้อมูลหรือการอัปเดตทันทีที่มีการเปลี่ยนแปลง โดยไม่ต้องร้องขอข้อมูลใหม่ด้วย

ตัวเอง (Wang et al., 2013) ตัวอย่างเช่น ระบบการสนทนา ระบบการแจ้งเตือน หรือ Dashboard ข้อมูลที่อัปเดตตลอดเวลา เป็นต้น เทคโนโลยีสำคัญที่ช่วยให้เกิดการทำงานลักษณะนี้คือ WebSocket ซึ่งเป็น Protocol ที่ช่วยให้การสื่อสารระหว่าง Client กับ Server เป็นไปแบบสองทิศทางเต็มรูปแบบ (Full-duplex) ตลอดเวลา โดยไม่ต้องสร้างคำขอใหม่ซ้ำ ๆ ดังที่ทำใน HTTP (Fette and Melnikov, 2011) ดังภาพที่ 2-23



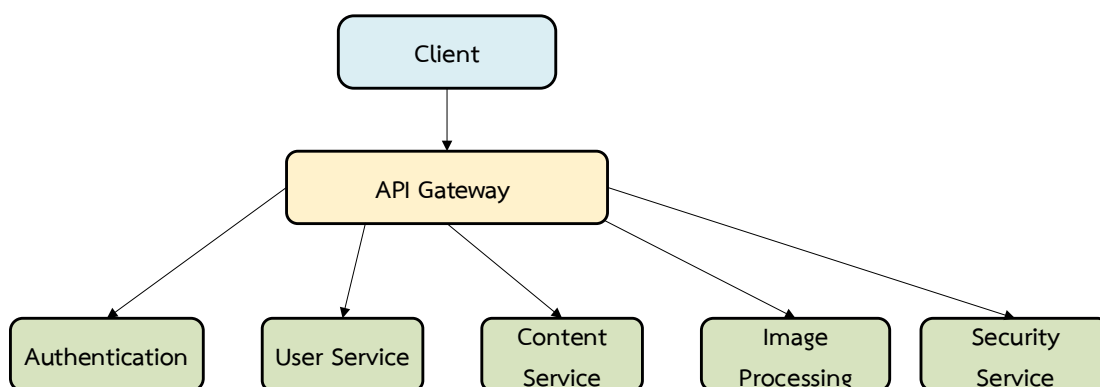
ภาพที่ 2-23 การเปรียบเทียบการสื่อสารระหว่าง HTTP และ WebSocket

(Fette and Melnikov, 2011)

ประโยชน์ของ WebSocket คือช่วยลดความหน่วงในการอัปเดตข้อมูล และลด Traffic ที่ไม่จำเป็น เมื่อเทียบกับการใช้วิธีเดิมอย่าง Polling ที่ต้องให้ Client ส่งคำขอ HTTP ไปตรวจสอบข้อมูลใหม่เป็นช่วง ๆ (Wang et al., 2013) WebSocket เหมาะสำหรับงานที่ข้อมูลต้องเปลี่ยนแปลงต่อเนื่องหรือมีเหตุการณ์เกิดขึ้นบ่อย เช่น ระบบการสนทนา ระบบแจ้งเตือน กระดานราคาหุ้นหรือ สกุลเงินดิจิทัล และเกมออนไลน์แบบทันทีทันใด เป็นต้น

2.8.5 รูปแบบสถาปัตยกรรมแบบหลายส่วนของ Web Application

ระบบ Web Application สมัยใหม่มักถูกออกแบบให้ประกอบด้วยหลายองค์ประกอบที่ทำงานประสานกัน โดยโครงสร้างทั่วไปแบ่งออกเป็นสามชั้นหลัก ได้แก่ ฝั่ง Client, API Gateway และบริการหลังบ้าน (Backend Services) หลายตัว (Newman, 2015) ดังภาพที่ 2-24



ภาพที่ 2-24 สถาปัตยกรรม Web Application แบบหลายส่วน (Newman, 2015)

ในสถาปัตยกรรมนี้ Client จะส่งคำขอทั้งหมดมายัง API Gateway เป็นด้านหน้าเพียงจุดเดียว โดยที่ Client ไม่ต้องรู้ว่าบริการเบื้องหลังมีกี่ตัวหรืออยู่ที่ใด (Richardson, 2019) API Gateway คอยรับคำขอจาก Client แล้วทำการส่งต่อคำขอไปยังบริการหลังบ้านที่รับผิดชอบงานนั้น ๆ นอกจากนี้ ยังสามารถจัดการงานส่วนกลางอื่น ๆ เช่น การตรวจสอบสิทธิ์ การจัดลำดับและกรอง Traffic การรวมผลลัพธ์จากหลายบริการ และการ Caching ข้อมูล เป็นต้น (Nadareishvili et al., 2016)

บริการหลังบ้านแต่ละตัวทำงานอย่างอิสระ รับผิดชอบงานเฉพาะด้าน และจัดการข้อมูลของตนเองผ่านฐานข้อมูล และระบบจัดเก็บต่าง ๆ (Newman, 2015) การจัดสถาปัตยกรรมแบบนี้ที่แยกบริการตามความรับผิดชอบเรียกว่า สถาปัตยกรรม Microservices ซึ่งส่งเสริมความยืดหยุ่น ความทนทาน และความสามารถในการขยายตัวของระบบ (Nadareishvili et al., 2016) ตัวอย่างของบริการหลังบ้านที่พบได้ในระบบ Web Application ทั่วไป ได้แก่

2.8.5.1 บริการพิสูจน์ตัวตนผู้ใช้ (Authentication Service) จัดการฟังก์ชันการลงชื่อเข้าใช้ การลงทะเบียนผู้ใช้ใหม่ และการตรวจสอบสิทธิ์ (Stuttard and Pinto, 2018)

2.8.5.2 บริการข้อมูลผู้ใช้ (User Service) ดูแลข้อมูล Profile ผู้ใช้ และความสัมพันธ์ระหว่างผู้ใช้ (Richardson, 2019)

2.8.5.3 บริการเนื้อหา (Content Service) รับผิดชอบการสร้าง และจัดการโพสต์หรือเนื้อหาที่ผู้ใช้ส่งเข้ามา (Nadareishvili et al., 2016)

2.8.5.4 บริการประมวลผลภาพ (Image Processing Service) ทำงานด้านการประมวลผลรูปภาพ เช่น การตรวจจับใบหน้า และการปรับแต่งภาพ เป็นต้น (Wang et al., 2021)

2.8.5.5 บริการตรวจสอบความปลอดภัย (Security Service) คอยตรวจจับ และเฝ้าระวังพฤติกรรมหรือเหตุการณ์ที่อาจเป็นภัยต่อระบบ (Stuttard and Pinto, 2018)

บทที่ 3

ขั้นตอนและวิธีการดำเนินงาน

ในการพัฒนาระบบ Website ในโครงการนี้ ผู้จัดทำได้ใช้กระบวนการพัฒนา Software แบบ Scrum ซึ่งเน้นการพัฒนาเป็นรอบ (Iteration) อย่างต่อเนื่อง และสามารถปรับปรุงได้ตลอดเวลา เพื่อให้ได้ระบบที่มีประสิทธิภาพตรงตามความต้องการของผู้ใช้งานมากที่สุด

ขั้นตอนการดำเนินงานครอบคลุมตั้งแต่การวิเคราะห์ความต้องการและความเป็นไปได้ของระบบ การออกแบบสถาปัตยกรรมและฐานข้อมูล การวางแผนการพัฒนาด้วย Scrum และการแบ่งงานเป็นระยะ (Sprint) ตลอดจนการพัฒนาจริงด้วยเครื่องมือต่าง ๆ รวมถึงการทดสอบระบบเพื่อประเมินผลลัพธ์และความถูกต้องของการทำงานแต่ละส่วน ดังรายละเอียดต่อไปนี้

- 3.1 การวิเคราะห์ปัญหาและความเป็นไปได้ของระบบ
- 3.2 การออกแบบสถาปัตยกรรมระบบ
- 3.3 การวิเคราะห์ระบบ (System analysis)
- 3.4 แผนภาพกรณีการใช้งานของระบบ (Use Case Diagram)
- 3.5 แผนภาพขั้นตอนการทำงานของระบบ (Activity Diagram)
- 3.6 การออกแบบฐานข้อมูล (Entity Relationship Diagram)
- 3.7 การเลือกแบบจำลอง Face Detection
- 3.8 การเลือกโมเดล Face Recognition
- 3.9 การเลือกโมเดล Deepfake Detection

3.1 การวิเคราะห์ปัญหาและความเป็นไปได้ของระบบ

ในการเริ่มต้นโครงการ ผู้จัดทำได้ทำการศึกษาปัญหา และความต้องการของระบบในปัจจุบันที่เกี่ยวข้องกับการใช้งานสื่อสังคมออนไลน์ และการรู้จำใบหน้า พบว่าระบบสังคมออนไลน์ทั่วไปยังขาดกลไกการยืนยันตัวตนด้วยใบหน้าที่แม่นยำ รวมถึงไม่มีการตรวจสอบป้องกันสื่อปลอมแปลงด้วยปัญญาประดิษฐ์ (Artificial Intelligence) เช่น Deepfake เป็นต้น ซึ่งอาจก่อให้เกิดความไม่ปลอดภัย และข้อมูลเท็จในระบบ ผู้จัดทำจึงเล็งเห็นโอกาสในการพัฒนา Platform FaceSocial ที่ผสมผสานการรู้จำใบหน้า และการตรวจจับสื่อปลอมเพื่อเพิ่มความปลอดภัย และประสิทธิภาพในการใช้งาน เพื่อประเมินความเป็นไปได้ของโครงการ ได้มีการวิเคราะห์ปัจจัยแวดล้อมในลักษณะการวิเคราะห์จุดแข็ง จุดอ่อน โอกาส และอุปสรรค (Strengths Weaknesses Opportunities Threats Analysis) ดังภาพที่ 3-1

Strengths (จุดแข็ง)

รวม Face Recognition + Deepfake Detection ในแพ็คเกจเดียว - เพิ่มความปลอดภัย, แตกต่างจากโซเชี่ยลทั่วไป
สถาปัตยกรรม Microservices
ขยาย-ลดทรัพยากรเฉพาะบริการได้, แก้ไขเป็นรายโมดูลได้รวดเร็ว
Ensemble AI
ArcFace, AdaFace, FaceNet - แม่นยำดี
องมากกว่าหรือเท่ากับร้อยละ 98 , ทนภาพเบลอ / แสงต่ำได้ดี
UX เว็บ Responsive + Face Login - ผู้ใช้เข้าสู่ระบบเร็วไม่ต้องจำรหัสผ่านยาว ๆ

Weaknesses (จุดอ่อน)

ต้องใช้ GPU/TPU สำหรับบริการ AI - ต้นทุนโครงสร้างพื้นฐานสูงกว่าเว็บโซเชี่ยลทั่วไป
กระบวนการ On-boarding ผู้ใช้ (สแกนใบหน้า, ยินยอม PDPA) - ต้องมีทีมเทคนิคเชี่ยวชาญเฉพาะ
ความซับซ้อน DevOps สูง (CI/CD, K8s, โมเดล AI หลายเวอร์ชัน) - ต้องมีทีมเทคนิคเชี่ยวชาญเฉพาะ
Latency ของการตรวจ Deepfake / Liveness อาจเพิ่มเวลาอัลโวลิตีเมื่อเน็ตช้า

FS

Opportunities (โอกาส)

เทรนด์ "ความเป็นจริงดิจิทัลปลอดภัย Deepfake" กำลังมา-องค์กร, Influencer ต้องการระบบตรวจสอบสื่อปลอม
ภาครัฐและองค์กรใหญ่เริ่ม ออกกฎ KYC แบบชีวมิติ - แพลตฟอร์มสามารถต่อยอดเป็นบริการ B2B ได้
มีตลาดเฉพาะ (niche) เช่น สังคมออนไลน์ สำหรับนักเรียนสร้างสรรค์ ที่ต้องการเครดิตใบหน้าแก้จรรยาบรรณ
รองรับการต่อยอดสู่ AR / Metaverse เพราะมี embedding ใบหน้าอยู่แล้ว

Theats (อุปสรรค / ภัยคุกคาม)

ทรัพยากร GPU จำกัด - ห้องแล็บมีการจองใช้งานร่วมกันอาจทำให้คิวฝึกโมเดลและปรับจูนล่าช้า
ข้อจำกัดสิทธิ์ใช้ชุดข้อมูล (Dataset Licensing) - บางชุดต้องขออนุญาต ถ้าขอไม่ทันจะกระทบช่วงเก็บข้อมูลทดลอง
การอัปเดตไลบรารีบ่อย PyTorch, TensorFlow และ FastAPI เวอร์ชันใหม่อาจทำให้โค้ดไม่เข้ากัน ต้องคอยแพตช์ระหว่างทำเล่ม
เวลาทดสอบภาคสนามจำกัด - การเก็บ Feedback ผู้ใช้ต้องทำไทม์ไลน์เดียว หากตรงช่วงสอบอาจเก็บข้อมูลไม่ครบ

ภาพที่ 3-1 การวิเคราะห์จุดแข็ง จุดอ่อน โอกาส และอุปสรรคของแพลตฟอร์ม FaceSocial

3.2 การออกแบบสถาปัตยกรรมระบบ

ระบบ FaceSocial ถูกออกแบบให้เป็นสถาปัตยกรรมแบบ Microservices ที่มีการแยกส่วนการทำงานอย่างชัดเจน โดยใช้ Docker Containers สำหรับบรรจุแต่ละบริการย่อย เพื่อให้ง่ายต่อการขยายตัว บำรุงรักษา และสามารถบริหารจัดการทรัพยากร Graphics Processing Unit ได้อย่างยืดหยุ่น โดยโครงสร้างสถาปัตยกรรมระบบแบ่งออกเป็น 3 ส่วนหลัก ได้แก่

3.2.1 Frontend (Web Application)

เป็นส่วนติดต่อผู้ใช้ที่พัฒนาด้วย Next.js ซึ่งเป็น Framework ของ React ใช้สำหรับแสดงผล และปฏิสัมพันธ์กับผู้ใช้งาน เช่น การลงทะเบียนใบหน้า การทดสอบการรู้จำใบหน้า การตรวจสอบภาพปลอม การแสดง Profile การโพสต์ภาพ และการตรวจสอบสถิติระบบต่าง ๆ เป็นต้น

3.2.2 Backend Services (API และ Processing Services)

เป็นส่วนที่รับผิดชอบด้านการประมวลผลข้อมูล และบริการต่าง ๆ ที่แยกออกเป็น Microservices ได้แก่

3.2.2.1 Face Detection Service

3.2.2.2 Face Recognition Service

3.2.2.3 Liveness Detection Service

3.2.2.4 Deepfake Detection Service

3.2.2.5 Gender and Age Detection Service

3.2.2.6 Batch Processor Service

ในแต่ละ Service ใช้งาน Graphics Processing Unit สำหรับการประมวลผลที่ต้องใช้ทรัพยากรสูง และสามารถทำงานแยกอิสระต่อกัน

3.2.3 API Gateway (Kong Gateway)

API Gateway ใช้เป็นด่านหน้าในการรับส่งคำขอจาก Frontend ไปยัง Backend Services ช่วยให้ระบบปลอดภัย และบริหารจัดการ Application Programming Interface ได้อย่างเป็นระเบียบใช้เป็นด่านหน้าในการรับส่งคำขอจาก Frontend ไปยัง Backend Services ช่วยให้ระบบปลอดภัย และบริหารจัดการ Application Programming Interface ได้อย่างเป็นระเบียบ

3.2.4 Databases

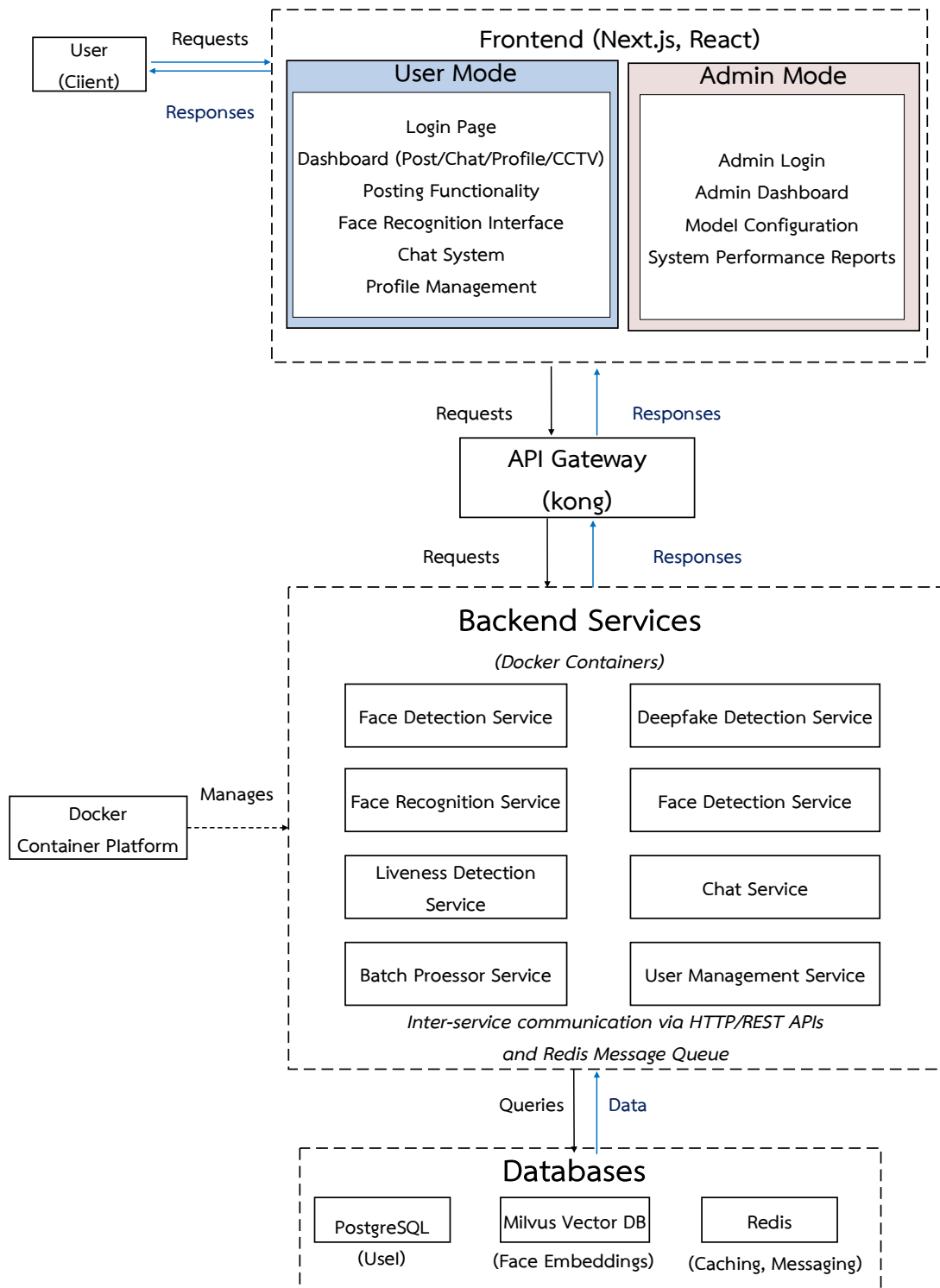
ระบบใช้ฐานข้อมูลหลายรูปแบบร่วมกัน ได้แก่

3.2.4.1 Milvus Vector Database สำหรับจัดเก็บเวกเตอร์ใบหน้า (Face Embeddings)

3.2.4.2 Redis สำหรับ Caching และ Message Queue

3.2.4.3 PostgreSQL สำหรับเก็บข้อมูลผู้ใช้ โพสต์ การตั้งค่าระบบ และข้อมูลเชิงโครงสร้าง

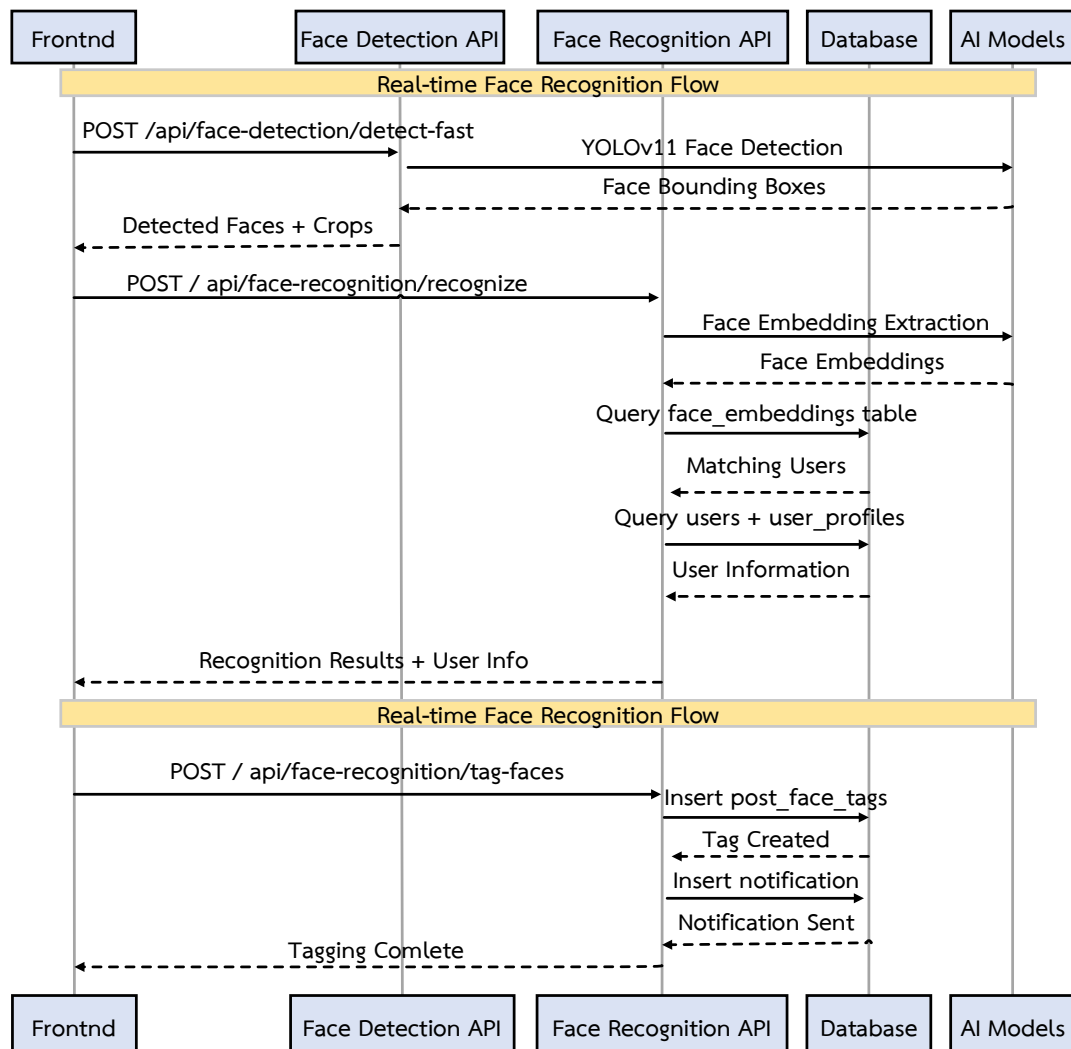
โครงสร้างการเชื่อมต่อระบบทั้งหมดจะถูกควบคุมด้วย Docker Compose ซึ่งสามารถจัดการ และเปิดใช้งานบริการต่าง ๆ ได้อย่างรวดเร็วในรูปแบบ Containerized Environment ดังภาพที่ 3-2



ภาพที่ 3-2 สถาปัตยกรรมระบบ FaceSocial แบบ Microservices

3.3 การวิเคราะห์ระบบ (System Analysis)

3.3.1 System Diagram ดังภาพที่ 3-3



ภาพที่ 3-3 แผนผังระบบของ FaceSocial

3.4 แผนภาพกรณีการใช้งานของระบบ (Use Case Diagram)

ระบบ FaceSocial แบ่งผู้ใช้งานออกเป็น 2 กลุ่มหลัก ได้แก่ ผู้ใช้ทั่วไป (General User) และ ผู้ใช้ขั้นสูง/ผู้ดูแลระบบ (Advanced User / Admin)

3.4.1 สรุป Use Case Diagram ดังตารางที่ 3-1

ตารางที่ 3-1 สรุป Use Case Diagram

ชื่อโครงการ (Project Name)	แพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพบุคคลด้วยการจดจำใบหน้า
ผู้ขอ (Requested By):	ผู้พัฒนา (Developer)
ตำแหน่ง (Position):	-
ความต้องการทางธุรกิจ (Business Requirements):	ต้องการพัฒนาระบบที่สามารถจัดการใบหน้าและตรวจจับภาพปลอม พร้อมทั้งการโพสต์และแสดงผลการใช้งานได้
ความต้องการฟังก์ชันการทำงาน (Functionality):	ระบบจะมีฟังก์ชันสำหรับการลงทะเบียนใบหน้า การโพสต์ภาพ การตรวจสอบภาพปลอม และการตรวจสอบสถิติการทำงานของระบบ
มูลค่าทางธุรกิจ (Business Value):	ช่วยให้ผู้ใช้สามารถมีระบบที่ง่ายต่อการใช้งานในการสร้างและตรวจสอบใบหน้า รวมถึงการตรวจจับการปลอมแปลง
เงื่อนไขพิเศษหรือข้อจำกัด (Special Issues or Constraints):	ต้องใช้ทรัพยากร GPU ในการประมวลผลที่ต้องการทรัพยากรสูง

3.4.1 Use Case หลักของผู้ใช้ทั่วไป (General User)

3.4.1.1 ลงทะเบียนใบหน้าใหม่ (Register Face)

3.4.1.2 อัปโหลด ภาพใบหน้าหรือใช้กล้อง (Upload Image or Camera)

3.4.1.3 ทดสอบระบบรู้จำใบหน้า (Face Recognition Test)

3.4.1.4 ตรวจสอบใบหน้าจริง-ปลอม (Deepfake Detection Test)

3.4.1.5 ประเมินเพศและอายุ (Gender and Age Estimation)

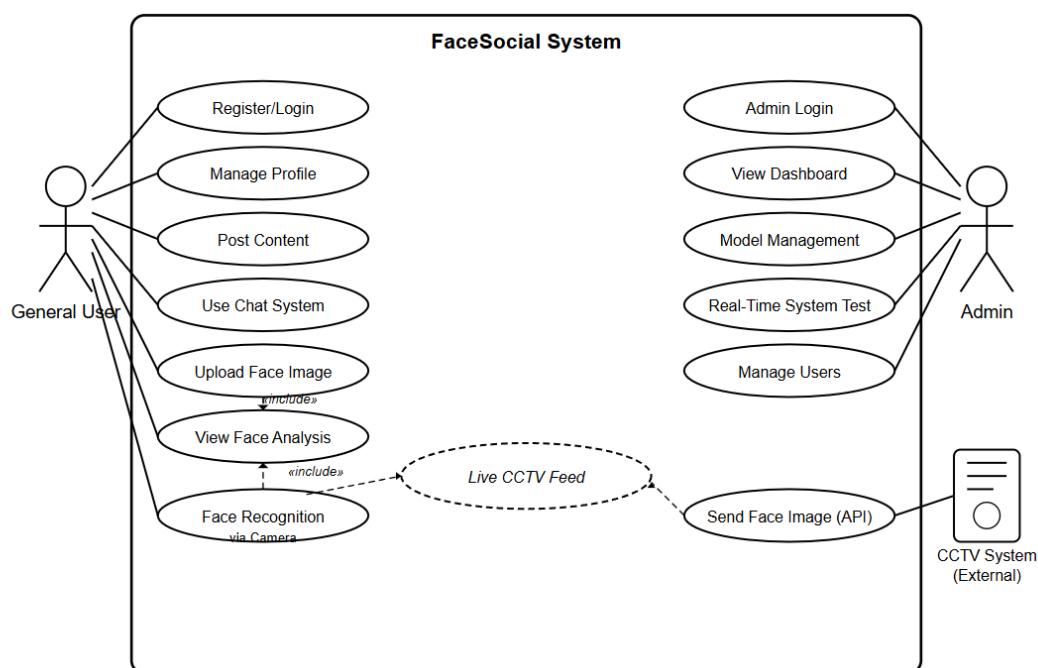
3.4.1.6 ดูผลการวิเคราะห์ (View Results)

3.4.2 Use Case ของระบบภายนอก (External System)

3.4.2.1 ส่งภาพใบหน้าผ่าน API (Send Face Image via API)

3.4.2.2 รับผลลัพธ์จากระบบ (Receive Result)

Use Case Diagram ขอ FaceSocial ดังภาพที่ 3-4

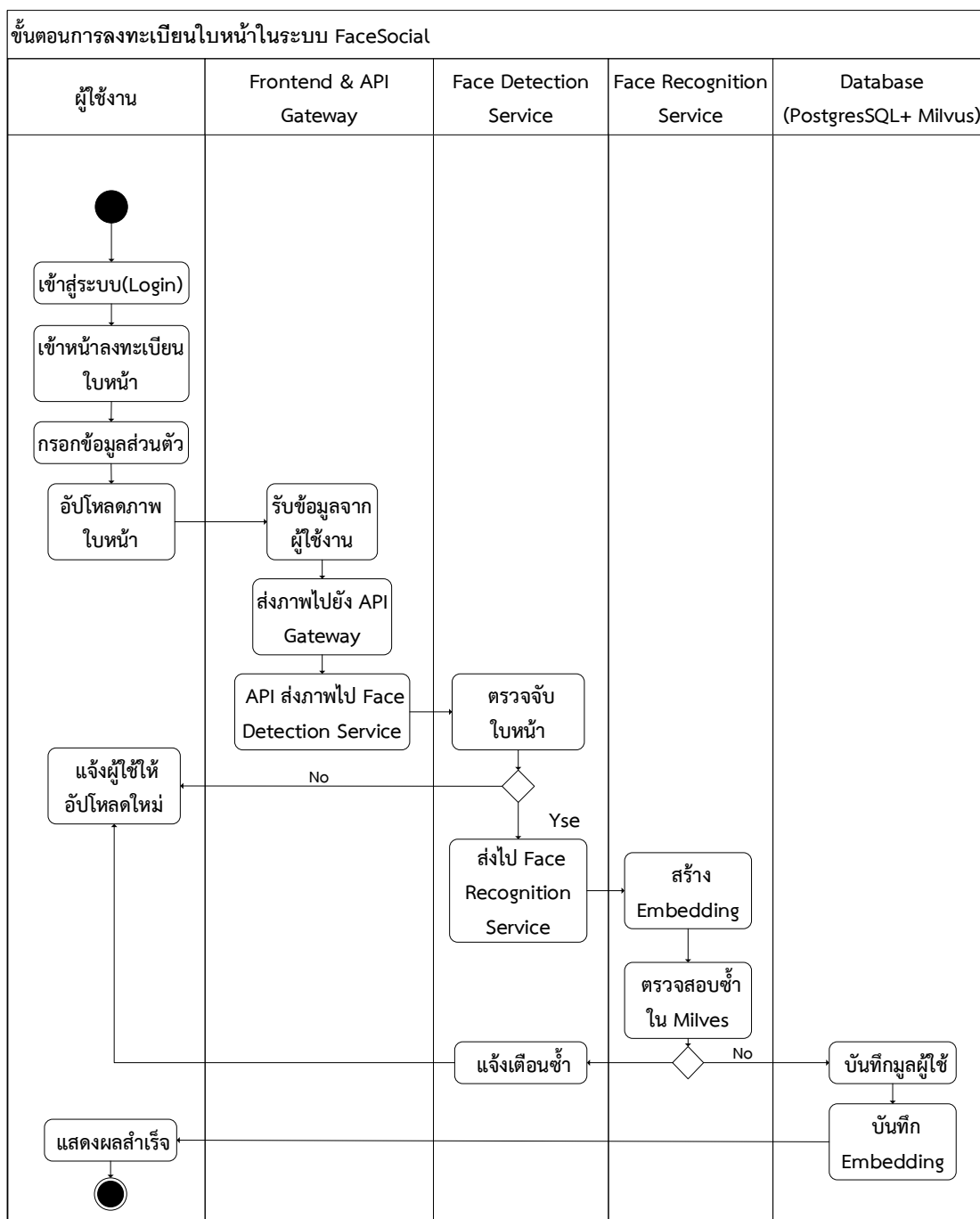


ภาพที่ 3-4 Use Case Diagram ของ FaceSocial

3.5 แผนภาพขั้นตอนการทำงานของระบบ (Activity Diagram)

ระบบ FaceSocial ได้ออกแบบขั้นตอนการทำงานหลักในรูปแบบ Activity Diagram เพื่อแสดงขั้นตอน และกระบวนการในแต่ละฟังก์ชัน โดยแบ่งออกเป็น 8 กระบวนการหลัก ดังนี้

3.5.1 Activity Diagram ขั้นตอนการลงทะเบียนใบหน้าในระบบ FaceSocial ดังภาพที่ 3-5



ภาพที่ 3-5 Activity Diagram ขั้นตอนการลงทะเบียนใบหน้าในระบบ FaceSocial

Requirement Specification: การลงทะเบียนใบหน้าใหม่ ดังตารางที่ 3-2

ตารางที่ 3-2 Requirement Specification: การลงทะเบียนใบหน้าใหม่

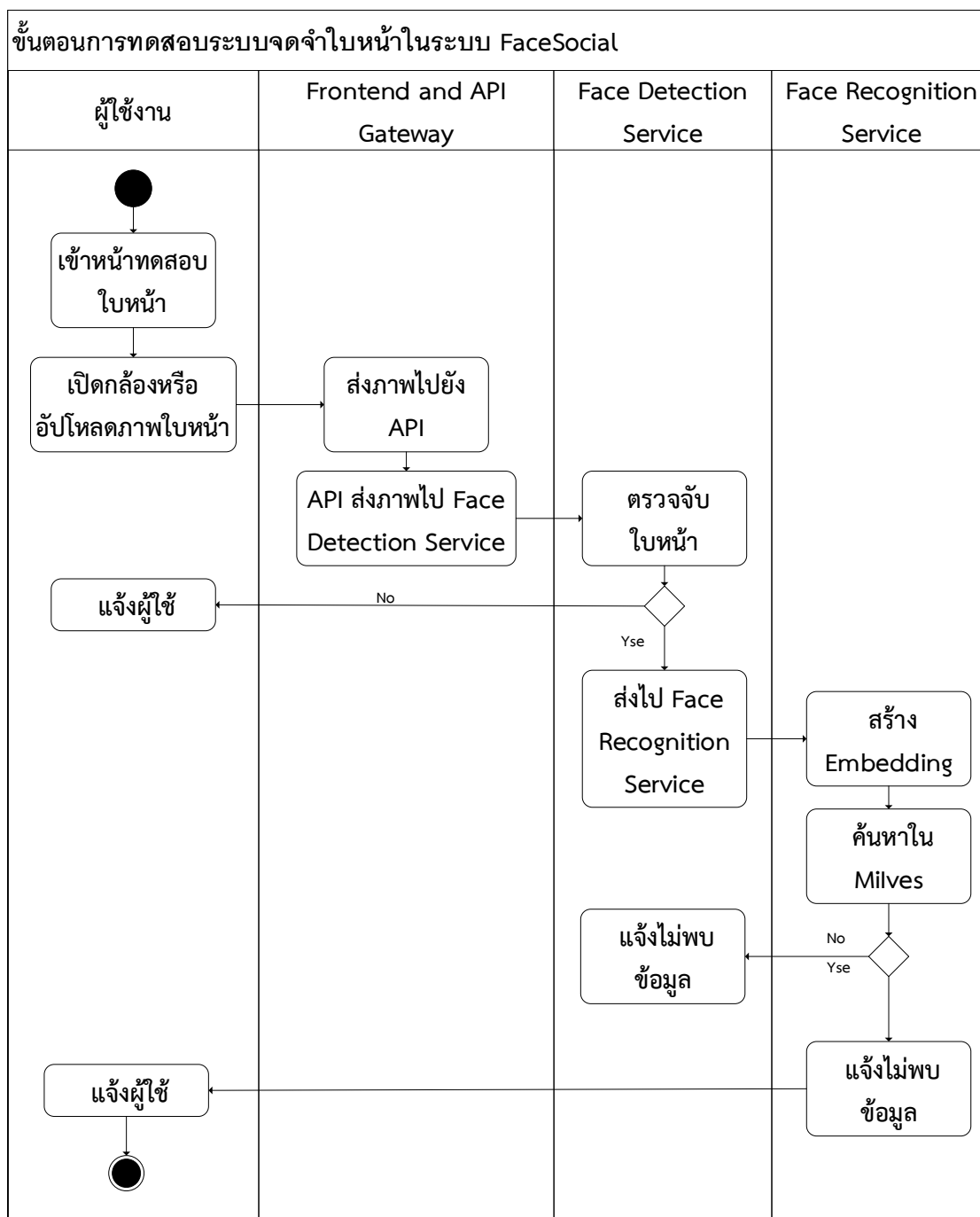
Function	การลงทะเบียนใบหน้าใหม่ (Face Registration)	
Brief Description	ผู้ใช้งานทำการลงทะเบียนใบหน้าผ่านระบบโดยอัปโหลดข้อมูลใบหน้าและส่งไปยังระบบการตรวจจับใบหน้าและการรู้จำใบหน้า	
Input	รูปภาพใบหน้า, ชื่อผู้ใช้, อีเมล, รหัสผ่าน	
Source	ผู้ใช้งาน	
Output	ข้อมูลสมาชิก, Face Embedding, ผลการตรวจสอบใบหน้า	
Requires	-	
Stakeholder	ผู้ใช้งาน, ผู้ดูแลระบบ	
Precondition	ผู้ใช้งานกรอกข้อมูลที่ถูกต้องและมีการอัปโหลดภาพใบหน้า	
Postcondition	ระบบแสดงผลข้อมูลการลงทะเบียนใบหน้าใหม่ พร้อมผลการตรวจจับใบหน้า	
Main Flow	1. ผู้ใช้ เข้าสู่ระบบ (Login) 3. ผู้ใช้ กรอกข้อมูลส่วนตัว 5. Frontend รับข้อมูลจากผู้ใช้งาน 7. API Gateway ส่งภาพไปยังบริการตรวจจับใบหน้า (Face Detection Service) 9. (เงื่อนไข) หากตรวจพบใบหน้า จะดำเนินการในขั้นตอนถัดไป (หากไม่พบ จะแจ้งผู้ใช้ให้อัปโหลดใหม่)	2. ผู้ใช้ เข้าสู่หน้าลงทะเบียนใบหน้า 4. ผู้ใช้ อัปโหลดภาพใบหน้า 6. Frontend ส่งภาพไปยัง API Gateway 8. บริการตรวจจับใบหน้า ดำเนินการตรวจจับใบหน้าในภาพ

ตารางที่ 3-2 Requirement Specification: การลงทะเบียนใบหน้าใหม่ (ต่อ)

	11. บริการรู้จำใบหน้า สร้างค่าการฝังตัวของใบหน้า Embedding 13. (เงื่อนไข) หากไม่พบใบหน้าที่ซ้ำซ้อน จะดำเนินการในขั้นตอนถัดไป หากซ้ำ จะแจ้งเตือน 15. ระบบฐานข้อมูล บันทึกค่าการฝังตัวของใบหน้า ใน Milvus	10. บริการตรวจจับใบหน้า ส่งภาพที่ตรวจจับได้ไปยังบริการรู้จำใบหน้า Face Recognition Service 12. บริการรู้จำใบหน้า ตรวจสอบว่าใบหน้าซ้ำซ้อนหรือไม่ในฐานข้อมูล Milvus 14. ระบบฐานข้อมูล บันทึกข้อมูลของผู้ใช้ ใน PostgreSQL 16. ระบบ แสดงผลการลงทะเบียนสำเร็จ
Exception Conditions	1. หากไม่สามารถตรวจจับใบหน้าได้หรือข้อมูลไม่ครบจะไม่สามารถลงทะเบียนได้ 2. หากระบบตรวจพบว่าใบหน้าซ้ำกับที่มีอยู่แล้วในฐานข้อมูล จะแจ้งเตือนผู้ใช้	

3.5.2 Requirement Specification การทดสอบใบหน้า (Face Recognition Test)

ดังภาพที่ 3-6



ภาพที่ 3-6 Requirement Specification การทดสอบใบหน้า (Face Recognition Test)

Requirement Specification การทดสอบใบหน้า (Face Recognition Test)
 ดังตารางที่ 3-3

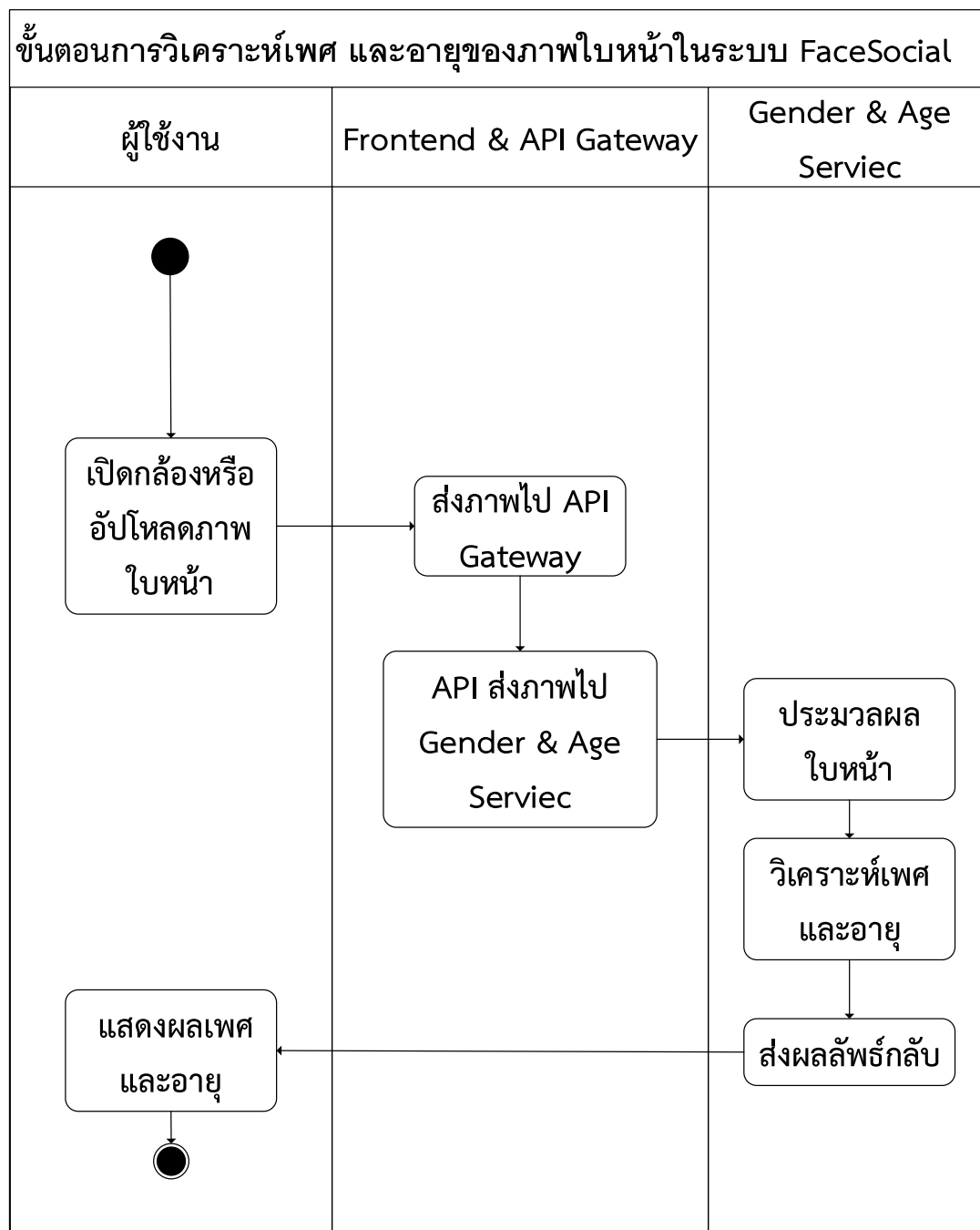
ตารางที่ 3-3 Requirement Specification การทดสอบใบหน้า (Face Recognition Test)

Function	การทดสอบใบหน้า (Face Recognition Test)	
Brief Description	ผู้ใช้งานทำการทดสอบการรู้จำใบหน้าโดยการอัปโหลดภาพ และระบบจะตรวจสอบความถูกต้องของการรู้จำใบหน้า	
Input	รูปภาพใบหน้า	
Source	ผู้ใช้งาน	
Output	ผลการทดสอบใบหน้า, ข้อมูล Embedding	
Requires	-	
Stakeholder	ผู้ใช้งาน, ผู้ดูแลระบบ	
Precondition	ผู้ใช้ต้องทำการอัปโหลดภาพใบหน้าเพื่อการทดสอบ	
Postcondition	ระบบแสดงผลการทดสอบการรู้จำใบหน้า	
Main Flow	1. ผู้ใช้ เข้าสู่หน้าทดสอบ ใบหน้า 3. ระบบ (Frontend) ส่ง ภาพไปยัง API Gateway 5. ระบบแสดงผลการทดสอบ ใบหน้า 7. บริการรู้จำใบหน้า (Face Recognition Service) สร้าง ค่าการฝังตัว Embedding) จากใบหน้า	2 ผู้ใช้ เปิดกล้องหรืออัปโหลด ภาพใบหน้า 4. API Gateway ส่งภาพ ต่อไปยังบริการตรวจจับ ใบหน้า (Face Detection Service) 6. เงื่อนไข หากตรวจพบ ใบหน้า จะส่งต่อไปยังบริการ รู้จำใบหน้า 8. บริการรู้จำใบหน้า ค้นหา ค่าการฝังตัวที่ตรงกันใน ฐานข้อมูล (Milvus)

ตารางที่ 3-3 Requirement Specification การทดสอบใบหน้า (Face Recognition Test) (ต่อ)

	9. เงื่อนไข หากค้นพบข้อมูลที่ตรงกัน ระบบจะแจ้งผลลัพธ์	10. ระบบ แสดงผลการทดสอบ ให้ผู้ใช้ทราบ
Exception Conditions	1. หากไม่สามารถตรวจจับใบหน้าได้จะไม่สามารถทำการทดสอบได้ 2. หากระบบ ไม่พบใบหน้าที่ตรงกัน ในฐานข้อมูล จะแจ้งเตือนผู้ใช้งานว่าไม่พบข้อมูล	

3.5.3 การวิเคราะห์เพศ และอายุ (Gender and Age Estimation) ดังภาพที่ 3-7



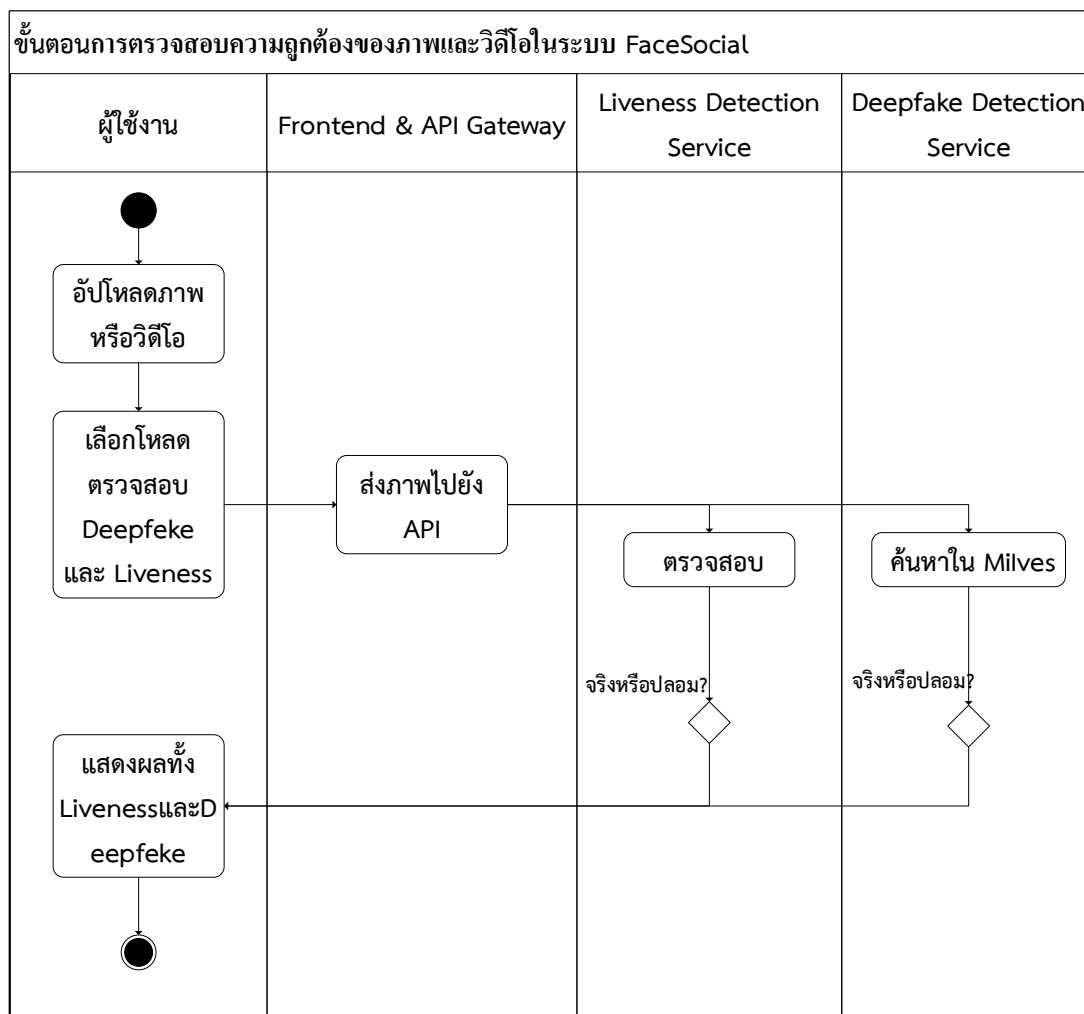
ภาพที่ 3-7 Activity Diagram ขั้นตอนการวิเคราะห์เพศ และอายุของภาพใบหน้าในระบบ FaceSocial

Requirement Specification การวิเคราะห์เพศ และอายุ (Gender and Age Estimation)
 ดังตารางที่ 3-4

ตารางที่ 3-4 Requirement Specification การวิเคราะห์เพศ และอายุ (Gender and Age Estimation)

Function	การวิเคราะห์เพศ และอายุ (Gender And Age Estimation)	
Brief Description	ผู้ใช้งานทำการอัปโหลดภาพใบหน้า และระบบทำการวิเคราะห์เพศและอายุจากภาพใบหน้า	
Input	รูปภาพใบหน้า	
Source	ผู้ใช้งาน	
Output	ผลการวิเคราะห์เพศและอายุ	
Requires	-	
Stakeholder	ผู้ใช้งาน, ผู้ดูแลระบบ	
Precondition	ผู้ใช้งานต้องอัปโหลดภาพใบหน้าเพื่อให้ระบบสามารถวิเคราะห์ได้	
Postcondition	ระบบแสดงผลการวิเคราะห์เพศและอายุจากภาพใบหน้า	
Main Flow	1. ผู้ใช้ เปิดกล้องหรืออัปโหลดภาพใบหน้า 3. API Gateway ส่งภาพต่อไปยังบริการวิเคราะห์เพศและอายุ (Gender & Age Service) 5. บริการวิเคราะห์ฯ วิเคราะห์เพื่อประเมินเพศและอายุ 7. ระบบ แสดงผลการวิเคราะห์เพศและอายุให้ผู้ใช้งานทราบ	2. ระบบ (Frontend) ส่งภาพไปยัง API Gateway 4. บริการวิเคราะห์ฯ ประมวลผลใบหน้าจากภาพ 6. บริการวิเคราะห์ฯ ส่งผลลัพธ์กลับมา
Exception Conditions	1. หากไม่สามารถตรวจจับใบหน้าในภาพได้ จะไม่สามารถทำการวิเคราะห์ได้	

3.5.4 ตรวจสอบความถูกต้องของภาพและวิดีโอในระบบ FaceSocial ดังภาพที่ 3-8



ภาพที่ 3-8 Activity Diagram ขั้นตอนตรวจสอบความถูกต้องของภาพและวิดีโอในระบบ FaceSocial

Requirement Specification การตรวจสอบ Deepfake และ Liveness (Deepfake and Liveness Detection) ดังตารางที่ 3-5

ตารางที่ 3-5 Requirement Specification การตรวจสอบ Deepfake และ Liveness (Deepfake and Liveness Detection)

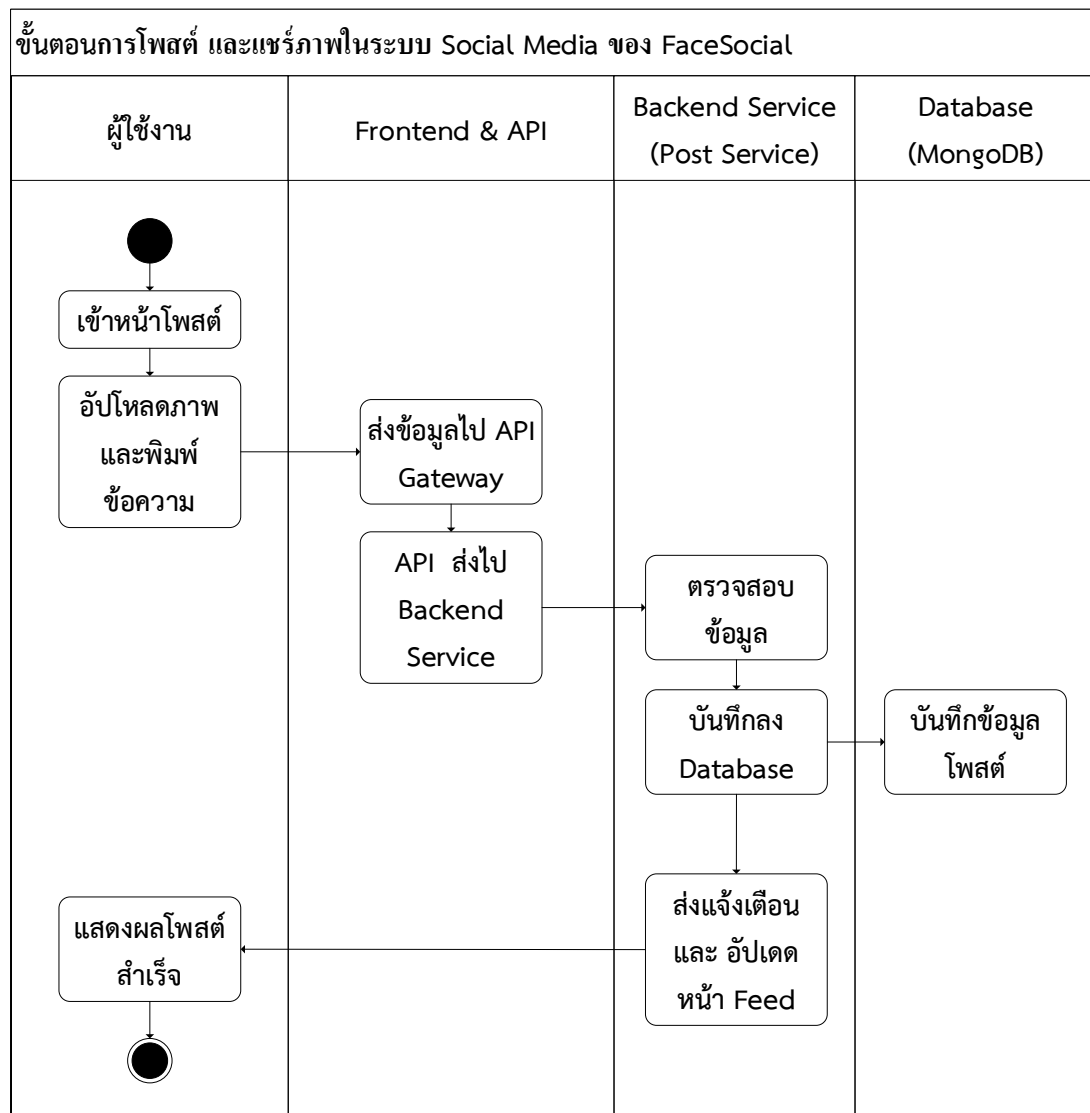
Function	การตรวจสอบ Deepfake และ Liveness (Deepfake and Liveness Detection)	
Brief description	ผู้ใช้งานทำการอัปโหลดภาพหรือวิดีโอ และระบบจะทำการตรวจสอบความถูกต้องของภาพและวิดีโอ เพื่อระบุว่าเป็นภาพปลอมหรือไม่ รวมถึงการตรวจสอบการมีชีวิต (Liveness)	
Input	รูปภาพหรือวิดีโอ	
Source	ผู้ใช้งาน	
Output	ผลการตรวจสอบ Deepfake และ Liveness	
Requires	-	
Stakeholder	ผู้ใช้งาน, ผู้ดูแลระบบ	
Precondition	ผู้ใช้งานต้องอัปโหลดภาพหรือวิดีโอเพื่อตรวจสอบ	
Postcondition	ระบบแสดงผลการตรวจสอบ Deepfake และ Liveness	
Main flow	1. ผู้ใช้ อัปโหลดรูปภาพหรือวิดีโอ 3. ระบบ (Frontend) ส่งข้อมูลไปยัง API Gateway 5. ระบบ รวบรวมผลลัพธ์จากการตรวจสอบทั้งสองส่วน	2. ผู้ใช้ เลือกโหมด"ตรวจสอบ Deepfake และ Liveness" 4. API Gateway เริ่มการตรวจสอบ 2 ส่วนพร้อมกัน ดังนี้: 4.1) Liveness Detection Service ทำการตรวจสอบการมีชีวิต 4.2) Deepfake Detection Service ทำการตรวจสอบสื่อสังเคราะห์ปลอม

ตารางที่ 3-5 Requirement Specification การตรวจสอบ Deepfake และ Liveness (Deepfake and Liveness Detection) (ต่อ)

		6. ระบบ แสดงผลการตรวจสอบ Liveness และ Deepfake ให้ผู้ใช้ทราบ
Exception conditions	1. หากไม่สามารถตรวจสอบได้จะไม่สามารถแสดงผลการตรวจสอบได้ 2. หากบริการตรวจสอบส่วนใดส่วนหนึ่งไม่สามารถให้คำตอบได้ ระบบจะแจ้งให้ผู้ใช้ทราบ	

3.5.5 Requirement Specification การโพสต์ และแชร์ภาพ (Post and Share)

รูปภาพที่ 3-9



ภาพที่ 3-9 Activity Diagram ขั้นตอนการโพสต์ และแชร์ภาพในระบบ Social Media ของ FaceSocial

Requirement Specification การโพสต์ และแชร์ภาพ (Post and Share)
 ดังตารางภาพที่ 3-6

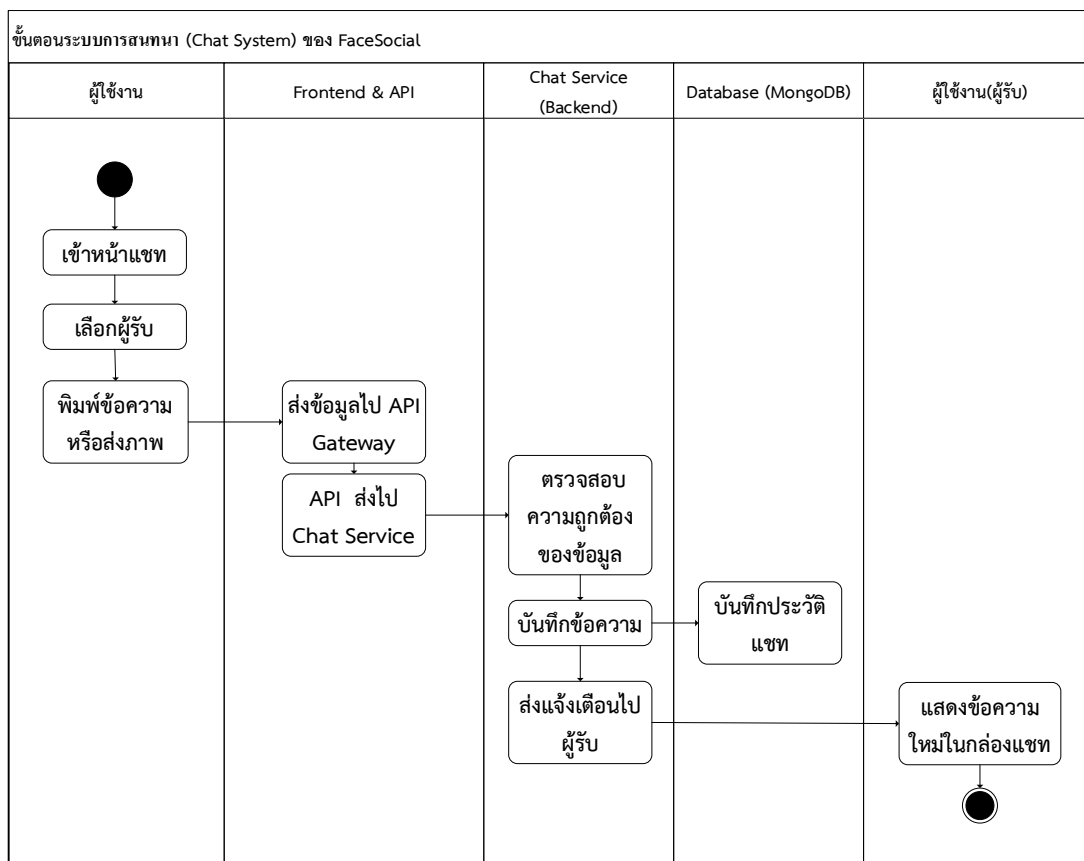
ตารางที่ 3-6 Requirement Specification การโพสต์ และแชร์ภาพ (Post and Share)

Function	การโพสต์ และแชร์ภาพ (Post And Share)	
Brief Description	ผู้ใช้ทำการอัปโหลดภาพและข้อความ และสามารถโพสต์ไปยังระบบหรือแชร์ไปยังเครือข่ายอื่น ๆ	
Input	รูปภาพ และข้อความ	
Source	ผู้ใช้งาน	
Output	ผลการโพสต์ภาพและข้อความ	
Requires	-	
Stakeholder	ผู้ใช้งาน, ผู้ดูแลระบบ	
Precondition	ผู้ใช้ต้องทำการล็อกอินและกรอกข้อมูลที่ต้องการโพสต์	
Postcondition	ระบบแสดงผลการโพสต์ภาพและข้อความ และแชร์ไปยังเครือข่ายอื่น ๆ	
Main Flow	1. ผู้ใช้ เข้าสู่หน้าสำหรับสร้างโพสต์ 3. ระบบ (Frontend) ส่งข้อมูลที่ได้รับไปยัง API Gateway 5. ระบบแชร์ไปยังเครือข่ายหรือแสดงผลโพสต์ในระบบ 7. ฐานข้อมูล (MongoDB) บันทึกข้อมูลโพสต์	2. ผู้ใช้ อัปโหลดรูปภาพและพิมพ์ข้อความ 4. API Gateway ส่งข้อมูลต่อไปยังบริการส่วนหลัง (Backend Service) 6. บริการส่วนหลัง สืบค้นที่กข้อมูลโพสต์ลงฐานข้อมูล 8. บริการส่วนหลัง ส่งแจ้งเตือนและอัปเดตหน้าฟีด (Feed)

ตารางที่ 3-6 Requirement Specification การโพสต์ และแชร์ภาพ (Post and Share) (ต่อ)

	9. ระบบ แสดงผล "โพสต์สำเร็จ" ให้ผู้ใช้ทราบ	
Exception Conditions	1. หากข้อมูลไม่ครบถ้วนจะไม่สามารถโพสต์ได้	

3.5.6 Activity Diagram ฟังก์ชันระบบการสนทนา (Chat System) ดังตารางที่ 3-10



ภาพที่ 3-10 Activity Diagram ขั้นตอนระบบการสนทนา (Chat System) ของ FaceSocial

Requirement Specification ระบบการสนทนา (Chat System) ดังตารางภาพที่ 3-7

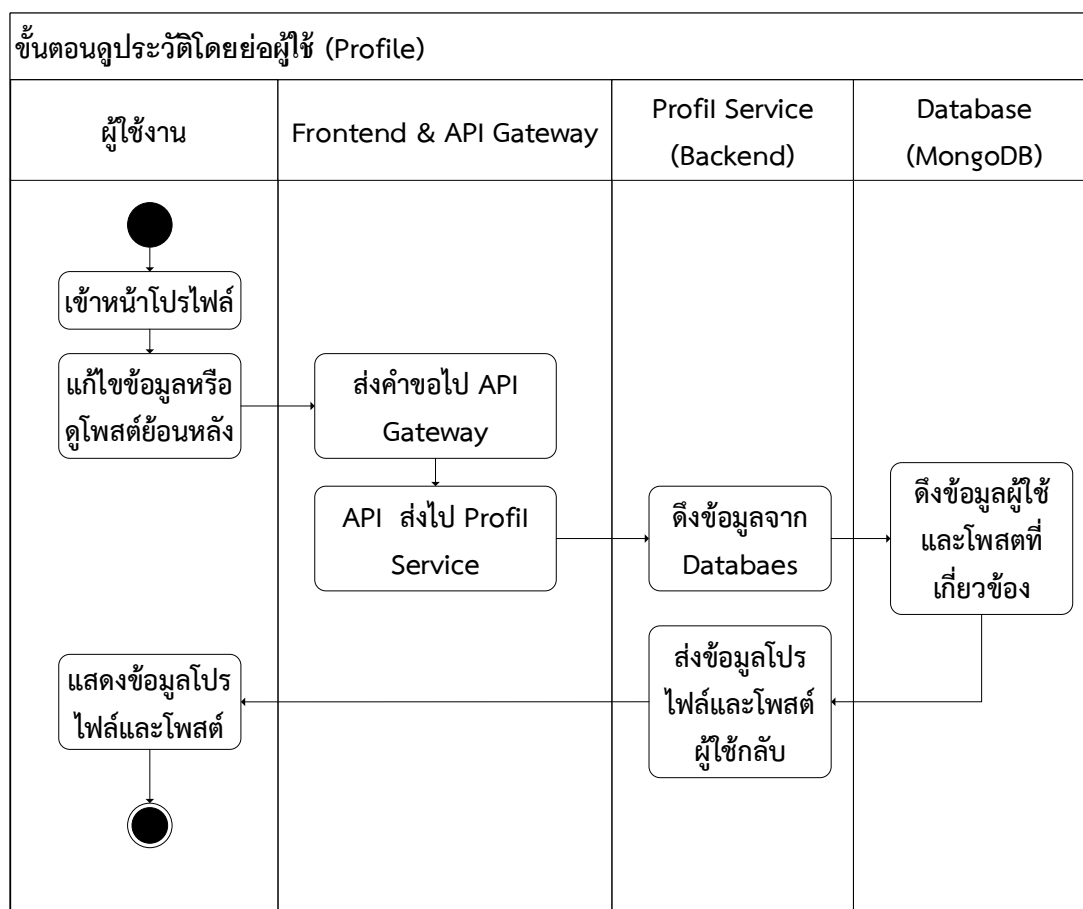
ตารางที่ 3-7 Requirement Specification ระบบการสนทนา (Chat System)

Function	ฟังก์ชันระบบการสนทนา (Chat System)	
Brief Description	ผู้ใช้งานสามารถส่งข้อความไปยังผู้อื่นในระบบได้ โดยการเลือกผู้รับและส่งข้อความผ่านระบบการสนทนา	
Input	ข้อความ, ชื่อผู้รับ	
Source	ผู้ใช้งาน	
Output	ข้อความที่ส่ง, ข้อความจากผู้รับ	
Requires	-	
Stakeholder	ผู้ใช้งาน, ผู้ดูแลระบบ	
Precondition	ผู้ใช้งานต้องทำการล็อกอินและเลือกผู้รับเพื่อส่งข้อความ	
Postcondition	ระบบแสดงผลข้อความที่ส่งและรับในระบบการสนทนา	
Main Flow	1. ผู้ส่ง เข้าสู่หน้าสนทนา (แชท) 3. ผู้ส่ง พิมพ์ข้อความหรือส่งรูปภาพ 5. API Gateway ส่งข้อมูลต่อไปยังบริการแชท (Chat Service) 7. บริการแชท สืบค้นที่ข้อความลงฐานข้อมูล 9. บริการแชท ส่งการแจ้งเตือนไปยังผู้รับ	2. ผู้ส่ง เลือกผู้รับที่ต้องการสนทนาด้วย 4. ระบบ (Frontend) ส่งข้อมูลที่ได้รับไปยัง API Gateway 6. บริการแชท ตรวจสอบความถูกต้องของข้อมูล 8. ฐานข้อมูล (MongoDB) บันทึกประวัติการสนทนา 10. ระบบ (ฝั่งผู้รับ) แสดงข้อความใหม่ในกล่องสนทนาของผู้รับ

ตารางที่ 3-7 Requirement Specification ระบบการสนทนา (Chat System) (ต่อ)

Exception Conditions	1. หากข้อมูลที่ส่งไม่ถูกต้อง (เช่น ไม่มีชื่อผู้รับ) ระบบจะไม่ทำการส่งข้อความ 2. หากไม่มีการเชื่อมต่ออินเทอร์เน็ต จะไม่สามารถส่งหรือรับข้อความได้
----------------------	---

3.5.7 Activity Diagram ฟังก์ชันดูประวัติโดยย่อผู้ใช้ (Profile) ดังภาพที่ 3-11



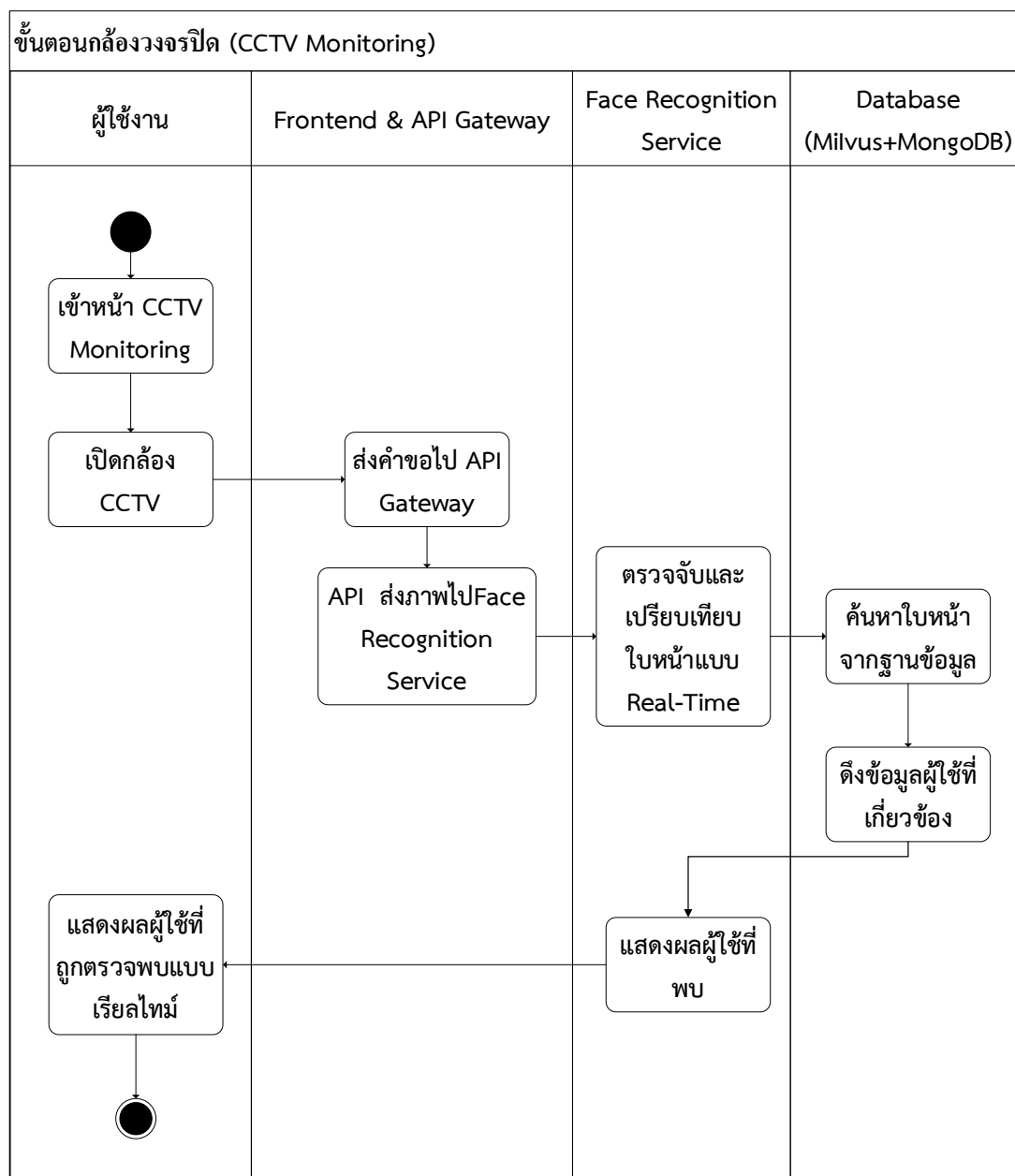
ภาพที่ 3-11 Activity Diagram ขั้นตอนดูประวัติโดยย่อผู้ใช้ (Profile)

Requirement Specification ประวัติโดยย่อผู้ใช้ (Profile) ดังตารางที่ 3-8

ตารางที่ 3-8 Requirement Specification ประวัติโดยย่อผู้ใช้ (Profile)

Function	ฟังก์ชันประวัติโดยย่อผู้ใช้ (Profile)	
Brief Description	ผู้ใช้สามารถดูประวัติโดยย่อของตนเองได้ โดยระบบจะแสดงข้อมูลโปรไฟล์ที่เกี่ยวข้อง	
Input	ชื่อผู้ใช้, ข้อมูลโปรไฟล์	
Source	ผู้ใช้งาน	
Output	ข้อมูลโปรไฟล์, ประวัติโดยย่อของผู้ใช้	
Requires	-	
Stakeholder	ผู้ใช้งาน, ผู้ดูแลระบบ	
Precondition	ผู้ใช้ต้องทำการล็อกอินและมีข้อมูลโปรไฟล์ในระบบ	
Postcondition	ระบบแสดงข้อมูลโปรไฟล์และประวัติโดยย่อของผู้ใช้	
Main Flow	1. ผู้ใช้ เข้าสู่หน้าโปรไฟล์ 3. ระบบ (Frontend) ส่งคำขอข้อมูลไปยัง API Gateway 5. บริการโปรไฟล์ สืบค้นข้อมูลจากฐานข้อมูล 7. บริการโปรไฟล์ ส่งข้อมูลที่ได้รับกลับไปยังส่วนแสดงผล	2. ผู้ใช้ เริ่มต้นการเรียกดูข้อมูล (เช่น คลิกเพื่อดูโพสต์ย้อนหลัง) 4. API Gateway ส่งคำขอต่อไปยังบริการโปรไฟล์ (Profile Service) 6. ฐานข้อมูล (MongoDB) ดึงข้อมูลผู้ใช้และโพสต์ที่เกี่ยวข้อง 8. ระบบ แสดงข้อมูลโปรไฟล์และโพสต์ของผู้ใช้
Exception Conditions	1. หากไม่พบข้อมูลโปรไฟล์ของผู้ใช้ในฐานข้อมูล ระบบจะแสดงข้อความแจ้งเตือน	

3.5.8 ฟังก์ชันกล้องวงจรปิด (CCTV Monitoring) ดังภาพที่ 3-12



ภาพที่ 3-12 Activity Diagram ขั้นตอนกล้องวงจรปิด (CCTV Monitoring)

Requirement Specification ระบบกล้องวงจรปิด (CCTV Monitoring)
 ดังภาพตารางที่ 3-9

ตารางที่ 3-9 Requirement Specification ระบบกล้องวงจรปิด (CCTV Monitoring)

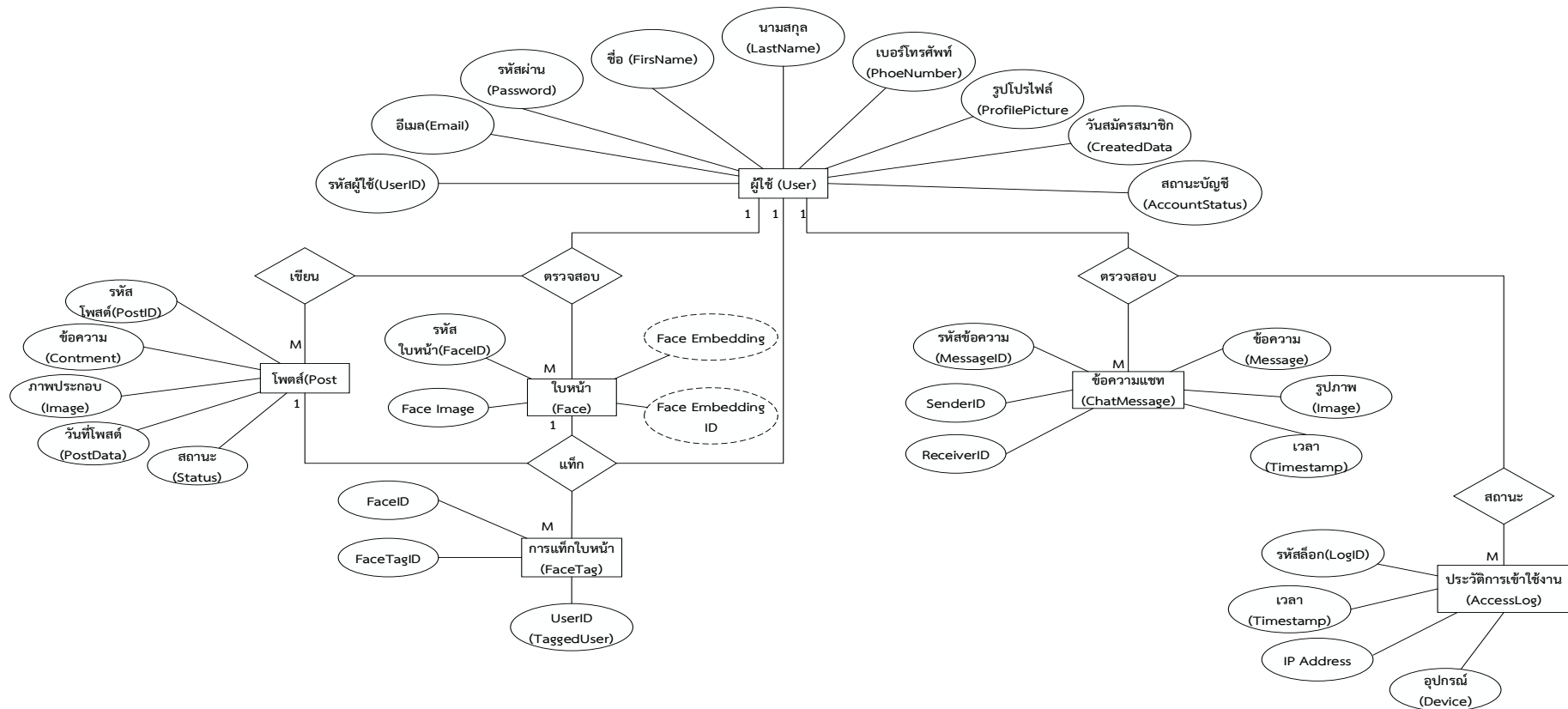
Function	ฟังก์ชันกล้องวงจรปิด (CCTV Monitoring)	
Brief description	ผู้ใช้งานสามารถเข้าถึงการถ่ายทอดสดจากกล้องวงจรปิด และระบบจะทำการตรวจจับใบหน้าและเปรียบเทียบใบหน้าจากฐานข้อมูลในระบบแบบ Real-Time	
Input	ข้อมูลภาพจากกล้องวงจรปิด, ข้อมูลใบหน้า	
Source	กล้องวงจรปิด, ผู้ใช้งาน	
Output	ผลการตรวจจับใบหน้า, ข้อมูลการจับคู่ใบหน้า	
Requires	-	
Stakeholder	ผู้ใช้งาน, ผู้ดูแลระบบ	
Precondition	ผู้ใช้งานต้องล็อกอินและสามารถเข้าถึงกล้องวงจรปิด	
Postcondition	ระบบแสดงผลการตรวจจับใบหน้าจากกล้องวงจรปิดและข้อมูลที่เกี่ยวข้อง	
Main flow	1. ผู้ใช้ เข้าสู่หน้าจอสำหรับดูกล้องวงจรปิด 3. ระบบ (Frontend) ส่งสตรีมภาพผ่าน API Gateway 5. บริการรู้จำใบหน้า ทำการตรวจจับและเปรียบเทียบใบหน้าแบบต่อเนื่อง (Real-Time)	2. ผู้ใช้ สั่งเปิดการทำงานของกล้องวงจรปิด 4. API Gateway ส่งสตรีมภาพต่อไปยังบริการรู้จำใบหน้า (Face Recognition Service) 6. ฐานข้อมูล (Milvus/MongoDB) ค้นหาใบหน้าที่ตรงกันและดึงข้อมูลผู้ใช้ที่เกี่ยวข้อง

ตารางที่ 3-9 Requirement Specification ระบบกล้องวงจรปิด (CCTV Monitoring) (ต่อ)

	7. บริการรู้จำใบหน้า รับข้อมูลผู้ใช้ที่ตรงกันและส่งกลับไปยังส่วนแสดงผล	8. ระบบ แสดงข้อมูลของผู้ใช้ที่ถูกต้องพบบนหน้าจอแบบเรียลไทม์
Exception conditions	1. หากตรวจจับใบหน้าที่ได้ แต่ไม่พบข้อมูลที่ตรงกันในฐานข้อมูล จะไม่แสดงผลการจับคู่ 2. หากการเชื่อมต่อกับกล้องวงจรปิดหรือบริการส่วนหลังล้มเหลว ระบบจะแสดงข้อความแจ้งเตือน	

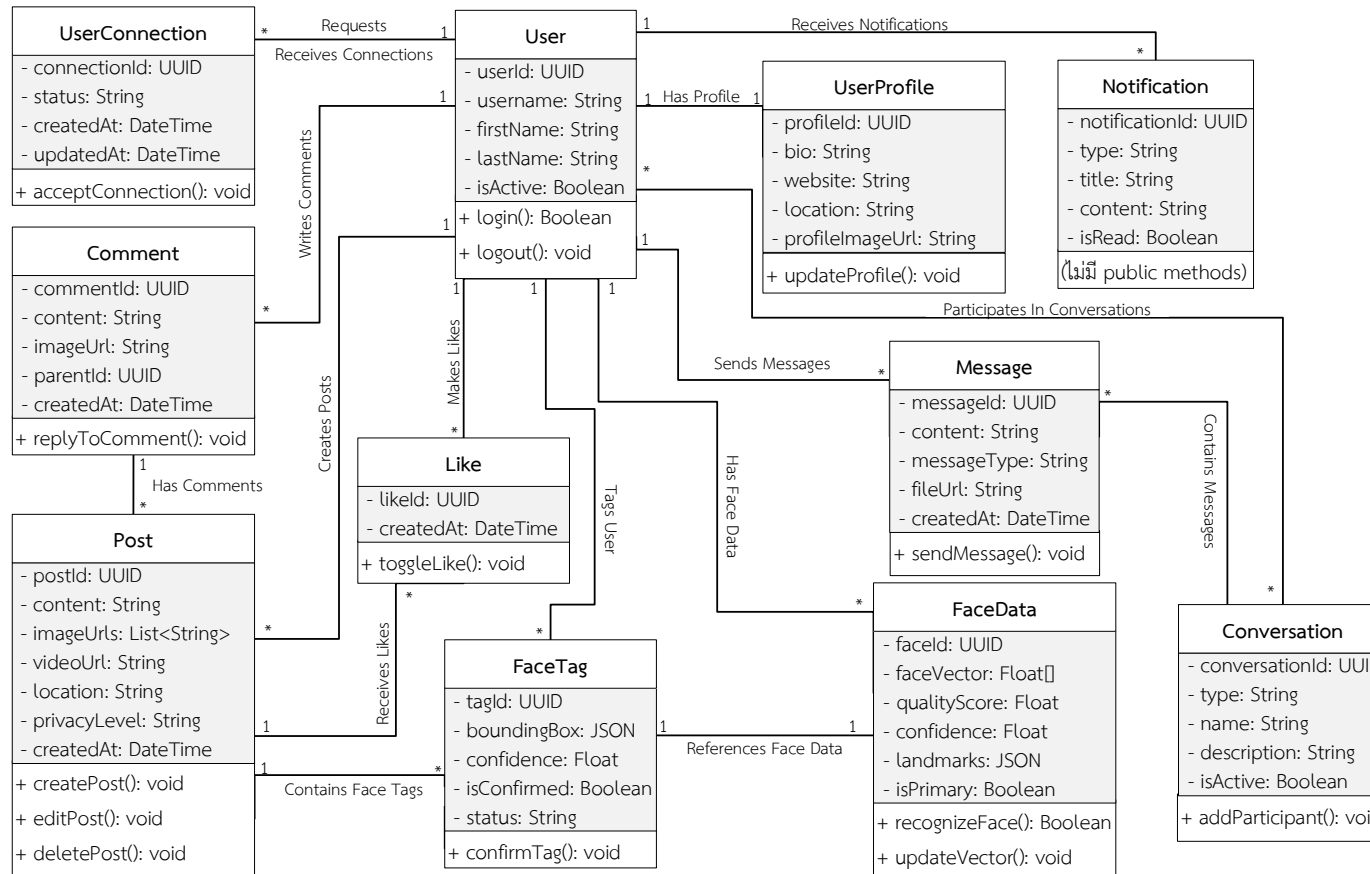
3.6 การออกแบบฐานข้อมูล (Entity Relationship Diagram)

3.6.1 การออกแบบฐานข้อมูล ของแพลตฟอร์มสื่อสังคมออนไลน์ และการจัดการรูปภาพบุคคล ด้วยการจดจำใบหน้า ดังภาพที่ 3-13



ภาพที่ 3-13 แบบจำลองการออกแบบฐานข้อมูล

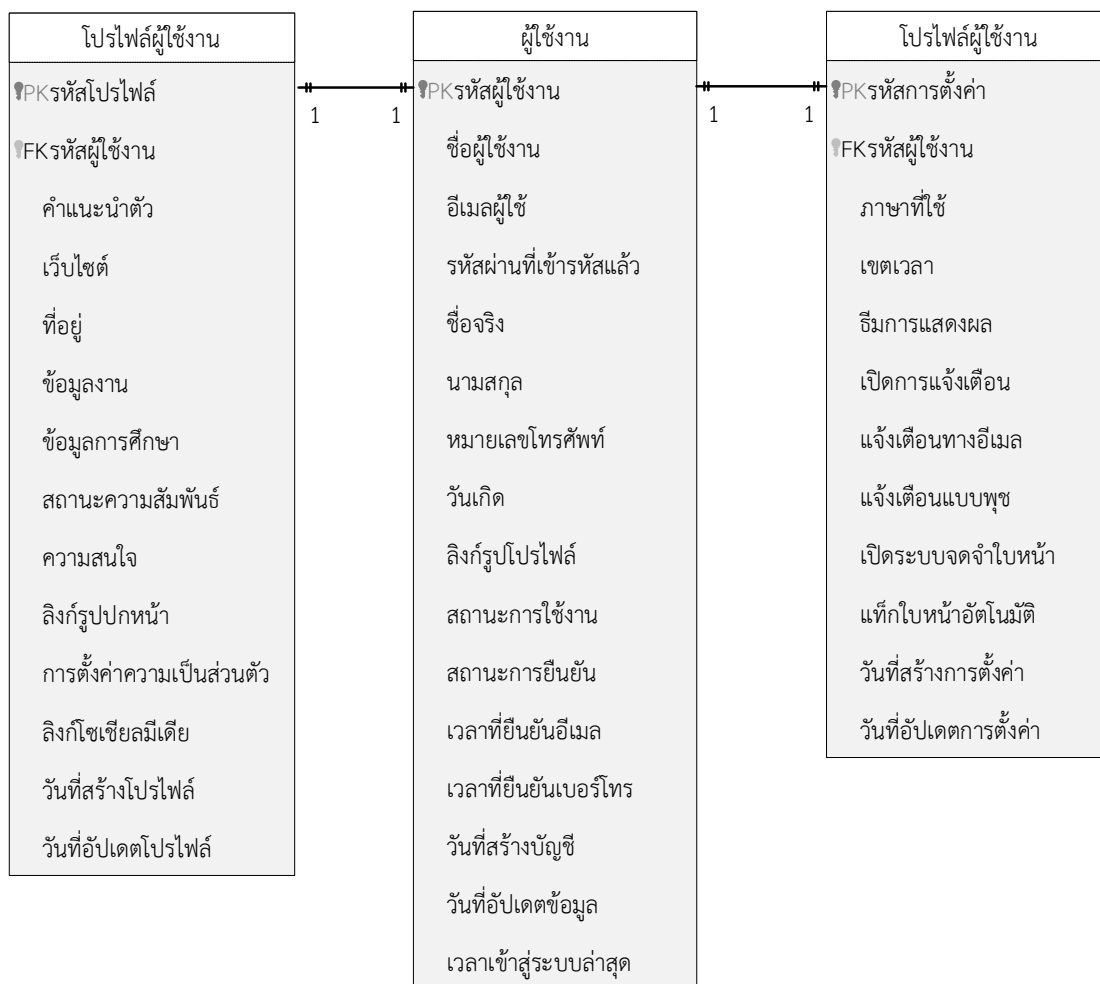
3.6.2 UML Class Diagram ของระบบ FaceSocial ดังภาพที่ 3-14



ภาพที่ 3-14 UML Class Diagram ของระบบ FaceSocial

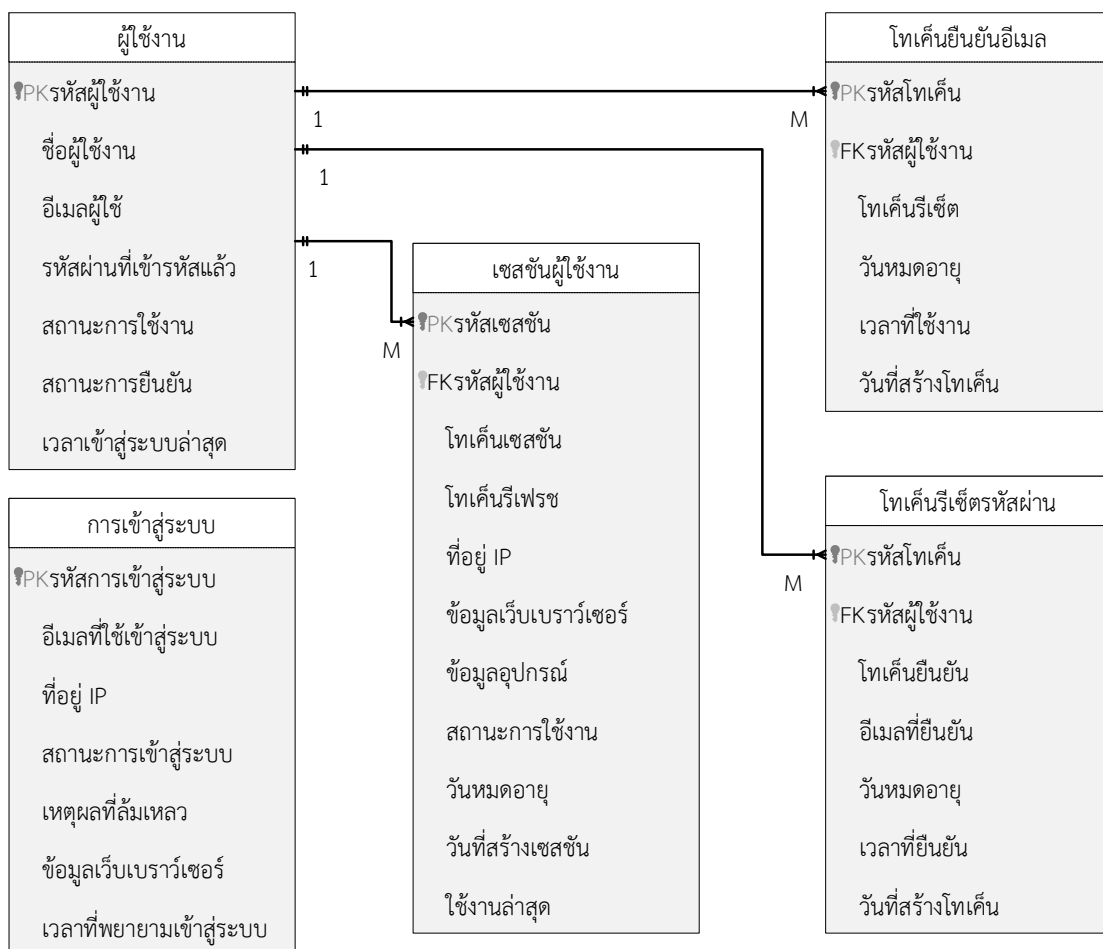
3.6.3 แผนภาพความสัมพันธ์ (Relational diagram)

3.6.3.1 โดเมนการจัดการผู้ใช้งาน ดังภาพที่ 3-15



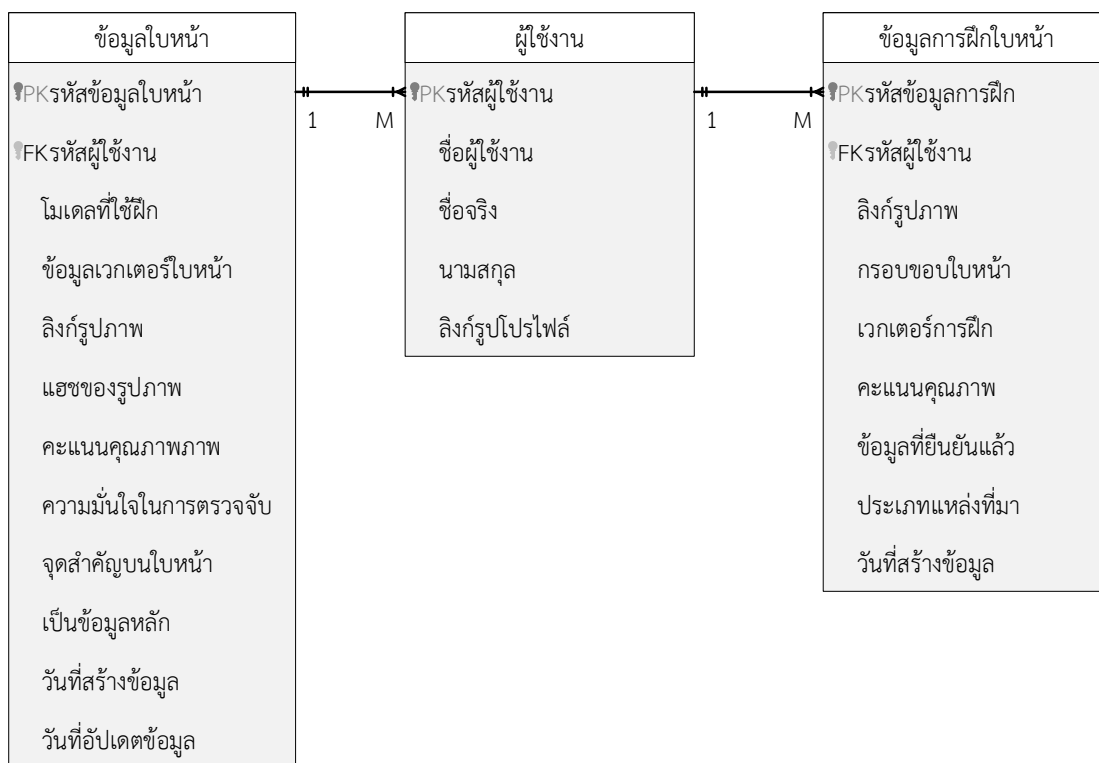
ภาพที่ 3-15 แผนภาพความสัมพันธ์ของสัมพันธ์โดเมนการจัดการผู้ใช้งาน

3.6.3.2 โดเมนการยืนยันตัวตน และความปลอดภัย ดังภาพที่ 3-16



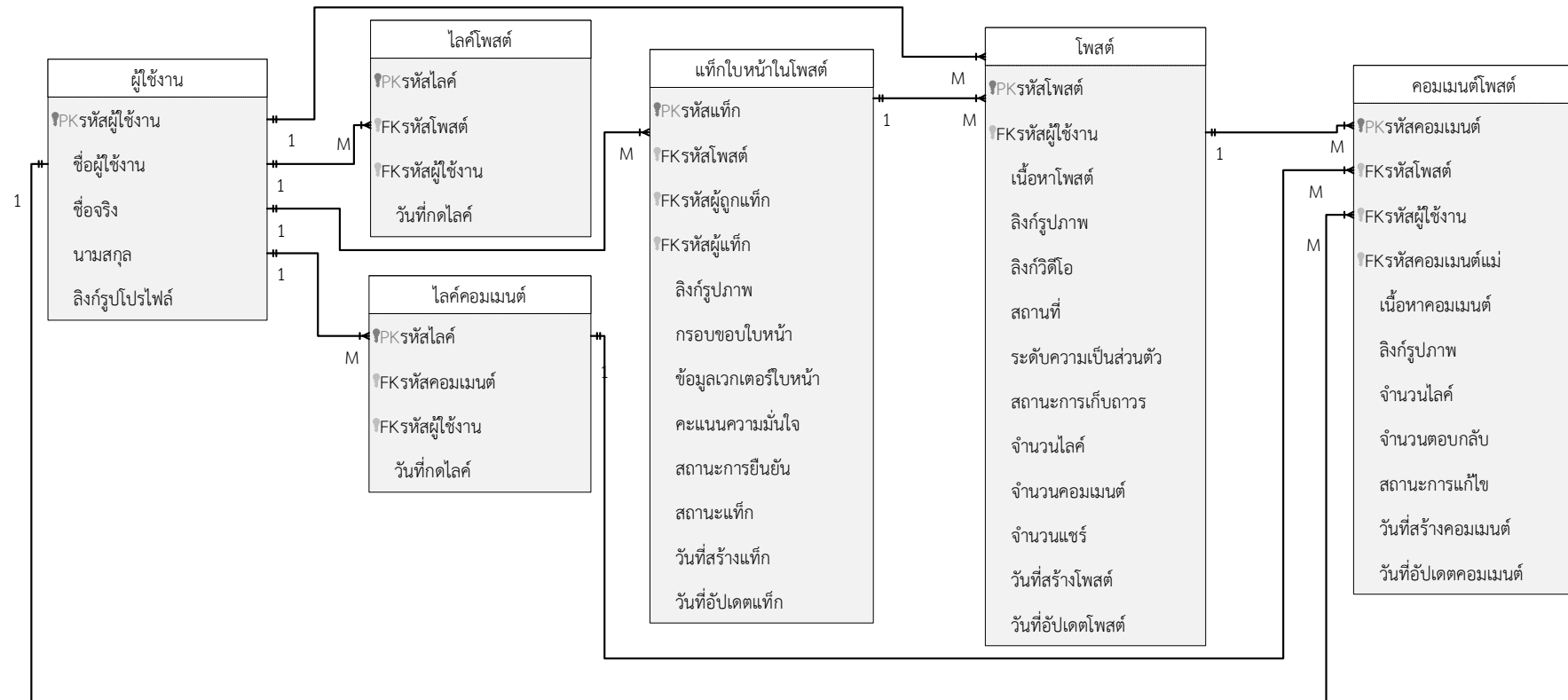
ภาพที่ 3-16 แผนภาพความของสัมพันธ์โดเมนการยืนยันตัวตน และความปลอดภัย

3.6.3.3 โดเมนการจดจำใบหน้า ดังภาพที่ 3-17



ภาพที่ 3-17 แผนภาพความของสัมพันธ์โดเมนการจดจำใบหน้า

3.6.3.4 โดเมนแพลตฟอร์มโซเชียล ดังภาพที่ 3-18



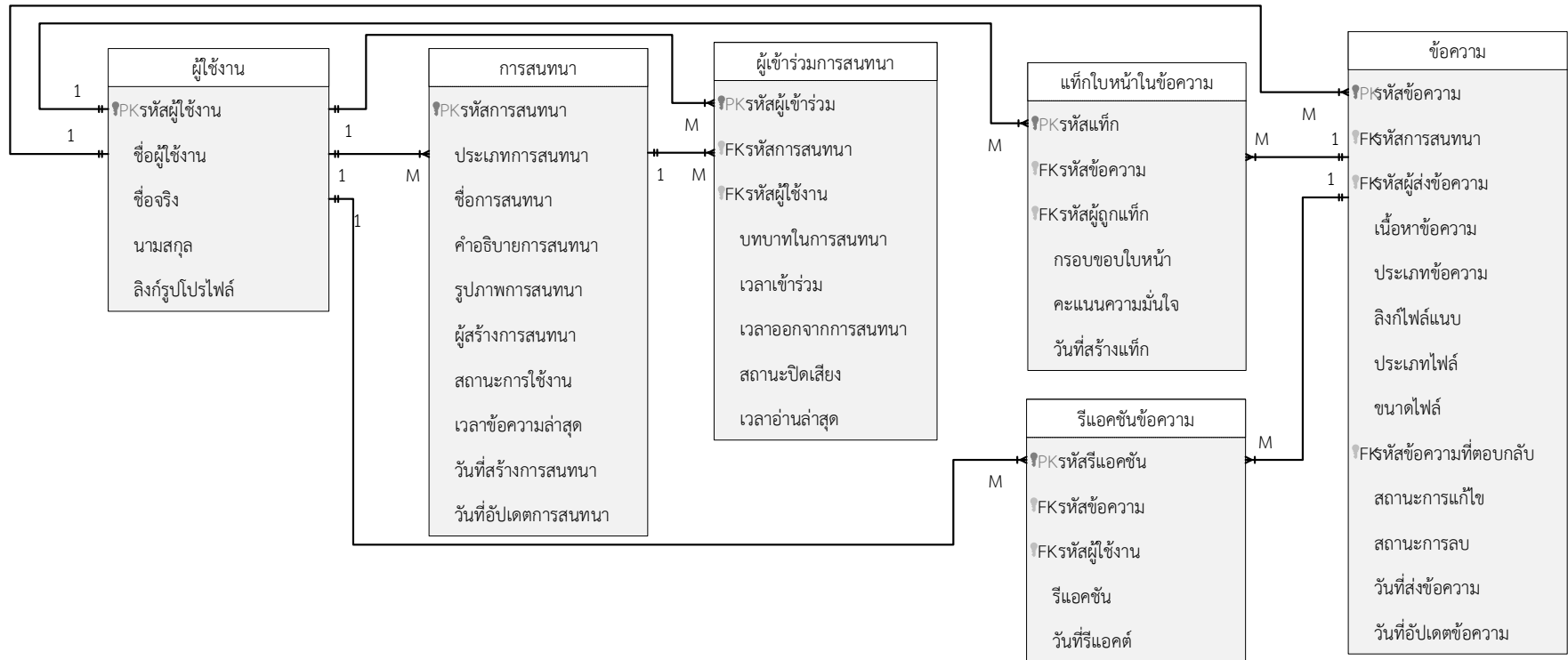
ภาพที่ 3-18 แผนภาพความของสัมพันธ์โดเมนแพลตฟอร์มโซเชียล

3.6.3.5 โดเมนการเชื่อมต่อทางสังคม ดังภาพที่ 3-19



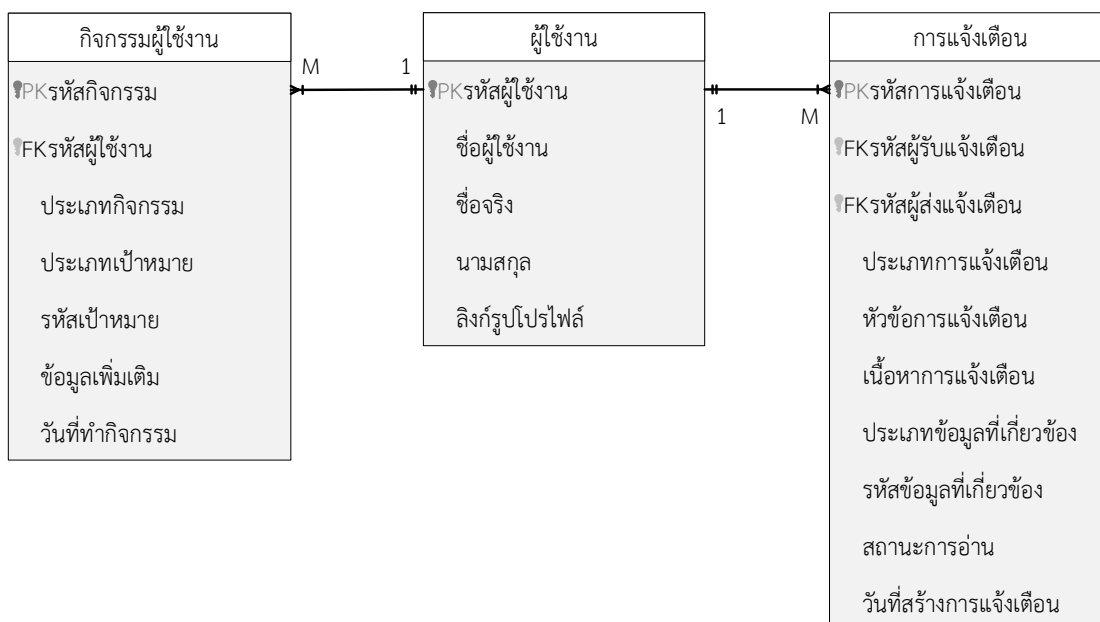
ภาพที่ 3-19 แผนภาพความของสัมพันธ์โดเมนการเชื่อมต่อทางสังคม

3.6.3.6 โดเมนการส่งข้อความ ดังภาพที่ 3-20



ภาพที่ 3-20 แผนภาพความของสัมพันธ์โดเมนการส่งข้อความ

3.6.3.7 โดเมนการแจ้งเตือน และกิจกรรม ดังภาพที่ 3-21



ภาพที่ 3-21 แผนภาพความของสัมพันธ์ โดเมนการแจ้งเตือน และกิจกรรม

3.6.3 โครงสร้างฐานข้อมูล (Data Scgema)

ผู้ใช้งาน	(รหัสผู้ใช้งาน ชื่อผู้ใช้งาน อีเมลผู้ใช้ รหัสผ่านที่เข้ารหัสแล้ว ชื่อจริง นามสกุล หมายเลขโทรศัพท์ วันเกิด ลิงก์รูปประวัติโดยย่อ สถานะการใช้งาน สถานะการยืนยัน เวลาที่ยืนยันอีเมล เวลาที่ยืนยันเบอร์โทร วันที่สร้างบัญชี วันที่อัปเดตข้อมูล เวลาเข้าสู่ระบบล่าสุด)
ประวัติโดยย่อผู้ใช้งาน	(รหัสประวัติโดยย่อ รหัสผู้ใช้งาน คำแนะนำตัว เว็บไซต์ ที่อยู่ ข้อมูลงาน ข้อมูลการศึกษา สถานะความสัมพันธ์ ความสนใจ ลิงก์รูปปกหน้า การตั้งค่าความเป็นส่วนตัว ลิงก์โซเชียลมีเดีย วันที่สร้างประวัติโดยย่อ วันที่อัปเดตประวัติโดยย่อ)
การตั้งค่าผู้ใช้งาน	(รหัสการตั้งค่า รหัสผู้ใช้งาน ภาษาที่ใช้ เขตเวลา อีเมลแสดงผล เปิดการแจ้งเตือน แจ้งเตือนทางอีเมล แจ้งเตือนแบบพุช เปิดระบบจดจำใบหน้า แท็กใบหน้าอัตโนมัติ วันที่สร้างการตั้งค่า วันที่อัปเดตการตั้งค่า)
เซสชันผู้ใช้งาน	(รหัสเซสชัน รหัสผู้ใช้งาน โทเค็นเซสชัน โทเค็นรีเฟรช ที่อยู่ IP ข้อมูลเว็บเบราว์เซอร์ ข้อมูลอุปกรณ์ สถานะการใช้งาน วันหมดอายุ วันที่สร้างเซสชัน ใช้งานล่าสุด)
การเข้าสู่ระบบ	(รหัสการเข้าสู่ระบบ อีเมลที่ใช้เข้าสู่ระบบ ที่อยู่ IP สถานะการเข้าสู่ระบบ เหตุผลที่ล้มเหลว ข้อมูลเว็บเบราว์เซอร์ เวลาที่พยายามเข้าสู่ระบบ)
โทเค็นรีเซตรหัสผ่าน	(รหัสโทเค็น รหัสผู้ใช้งาน โทเค็นรีเซต วันหมดอายุ เวลาที่ใช้ใช้งาน วันที่สร้างโทเค็น)
โทเค็นยืนยันอีเมล	(รหัสโทเค็น รหัสผู้ใช้งาน โทเค็นยืนยัน อีเมลที่ยืนยัน วันหมดอายุ เวลาที่ยืนยัน วันที่สร้างโทเค็น)
ข้อมูลใบหน้า	(รหัสข้อมูลใบหน้า รหัสผู้ใช้งาน แบบจำลองที่ใช้ฝึก ข้อมูลเวกเตอร์ใบหน้า ลิงก์รูปภาพ แสซของรูปภาพ คะแนนคุณภาพภาพ ความมั่นใจในการตรวจจับ จุดสำคัญบนใบหน้า เป็นข้อมูลหลัก วันที่สร้างข้อมูล วันที่อัปเดตข้อมูล)
ข้อมูลการฝึกใบหน้า	(รหัสข้อมูลการฝึก รหัสผู้ใช้งาน ลิงก์รูปภาพ กรอบขอบใบหน้า เวกเตอร์การฝึก คะแนนคุณภาพ ข้อมูลที่ยืนยันแล้ว ประเภทแหล่งที่มา วันที่สร้างข้อมูล)
โพสต์	(รหัสโพสต์ รหัสผู้ใช้งาน เนื้อหาโพสต์ ลิงก์รูปภาพ ลิงก์วิดีโอ สถานะที่ระดับความเป็นส่วนตัว สถานะการเก็บถาวร จำนวนไลค์ จำนวนคอมเมนต์ จำนวนแชร์ วันที่สร้างโพสต์ วันที่อัปเดตโพสต์)

แท็กใบหน้าในโพสต์	(รหัสแท็ก รหัสโพสต์ รหัสผู้ถูกแท็ก รหัสผู้แท็ก ลิงก์รูปภาพ กรอบขอบใบหน้า ข้อมูลเวกเตอร์ใบหน้า คะแนนความมั่นใจ สถานะการยืนยัน สถานะแท็ก วันที่สร้างแท็ก วันที่อัปเดตแท็ก)
โลโก้โพสต์	(รหัสโลโก้ รหัสโพสต์ รหัสผู้ใช้งาน วันที่กดโลโก้)
คอมเมนต์โพสต์	(รหัสคอมเมนต์ รหัสโพสต์ รหัสผู้ใช้งาน รหัสคอมเมนต์แม่ เนื้อหาคอมเมนต์ ลิงก์รูปภาพ จำนวนไลค์ จำนวนตอบกลับ สถานะการแก้ไข วันที่สร้างคอมเมนต์ วันที่อัปเดตคอมเมนต์)
โลโก้คอมเมนต์	(รหัสโลโก้ รหัสคอมเมนต์ รหัสผู้ใช้งาน วันที่กดโลโก้)
การเชื่อมต่อผู้ใช้งาน	(รหัสการเชื่อมต่อ รหัสผู้ขอเชื่อมต่อ รหัสผู้รับคำขอ สถานะการเชื่อมต่อ วันที่สร้างคำขอ วันที่อัปเดตสถานะ)
การสนทนา	(รหัสการสนทนา ประเภทการสนทนา ชื่อการสนทนา คำอธิบายการสนทนา รูปภาพการสนทนา ผู้สร้างการสนทนา สถานะการใช้งาน เวลาข้อความล่าสุด วันที่สร้างการสนทนา วันที่อัปเดตการสนทนา)
ผู้เข้าร่วมการสนทนา	(รหัสผู้เข้าร่วม รหัสการสนทนา รหัสผู้ใช้งาน บทบาทในการสนทนา เวลาเข้าร่วม เวลาออกจากการสนทนา สถานะปิดเสียง เวลาอ่านล่าสุด)
ข้อความ	(รหัสข้อความ รหัสการสนทนา รหัสผู้ส่งข้อความ เนื้อหาข้อความ ประเภทข้อความ ลิงก์ไฟล์แนบ ประเภทไฟล์ ขนาดไฟล์ รหัสข้อความที่ตอบกลับ สถานะการแก้ไข สถานะการลบ วันที่ส่งข้อความ วันที่อัปเดตข้อความ)
แท็กใบหน้าในข้อความ	(รหัสแท็ก รหัสข้อความ รหัสผู้ถูกแท็ก กรอบขอบใบหน้า คะแนนความมั่นใจ วันที่สร้างแท็ก)
รีแอคชันข้อความ	(รหัสรีแอคชัน รหัสข้อความ รหัสผู้ใช้งาน รีแอคชัน วันที่รีแอคต์)
การแจ้งเตือน	(รหัสการแจ้งเตือน รหัสผู้รับแจ้งเตือน รหัสผู้ส่งแจ้งเตือน ประเภทการแจ้งเตือน หัวข้อการแจ้งเตือน เนื้อหาการแจ้งเตือน ประเภทข้อมูลที่เกี่ยวข้อง รหัสข้อมูลที่เกี่ยวข้อง สถานะการอ่าน วันที่สร้างการแจ้งเตือน)
กิจกรรมผู้ใช้งาน	(รหัสกิจกรรม รหัสผู้ใช้งาน ประเภทกิจกรรม ประเภทเป้าหมาย รหัสเป้าหมาย ข้อมูลเพิ่มเติม วันที่ทำกิจกรรม)

3.6.4 พจนานุกรมข้อมูล (Data Dictionary)

3.6.4.1 พจนานุกรมผู้ใช้งาน (User) ดังตารางที่ 3-10

ตารางที่ 3-10 พจนานุกรมผู้ใช้งาน (User)

No.	Name	Description	Data type	Length	Key	Reference
1	id	รหัสผู้ใช้งาน	Uuid	36	PK	
2	username	ชื่อผู้ใช้งาน	Varchar	50		
3	email	อีเมลผู้ใช้	Varchar	100		
4	password_hash	รหัสผ่านที่ เข้ารหัสแล้ว	Varchar	255		
5	first_name	ชื่อจริง	Varchar	50		
6	last_name	นามสกุล	Varchar	50		
7	phone	หมายเลข โทรศัพท์	Varchar	15		
8	date_of_birth	วันเกิด	Date	10		
9	profile_image_url	ลิงก์รูปประวัติ โดยย่อ	Text	MAX		
10	is_active	สถานะการใช้ งาน	Boolean	1		
11	is_verified	สถานะการ ยืนยัน	Boolean	1		
12	email_verified_at	เวลาที่ยืนยัน อีเมล	Timestamp	19		
13	phone_verified_at	เวลาที่ยืนยัน เบอร์โทร	Timestamp	19		
14	created_at	วันที่สร้าง บัญชี	Timestamp	19		
15	updated_at	วันที่อัปเดต ข้อมูล	Timestamp	19		
16	last_login_at	เวลาเข้าสู่ ระบบล่าสุด	Timestamp	19		

3.6.4.2 พจนานุกรมประวัติโดยย่อผู้ใช้งาน (User) ดังตารางที่ 3-11

ตารางที่ 3-11 พจนานุกรมประวัติโดยย่อผู้ใช้งาน (User)

No.	Name	Description	Datatype	Length	Key	Reference
1	id	รหัสประวัติโดยย่อ	Uuid	36	PK	
2	user_id	รหัสผู้ใช้งาน	Uuid	36	FK	User
3	bio	คำแนะนำตัว	Text	MAX		
4	website	เว็บไซต์	Varchar	255		
5	location	ที่อยู่	Varchar	255		
6	work	ข้อมูลงาน	Varchar	100		
7	education	ข้อมูลการศึกษา	Varchar	100		
8	Relationship_status	สถานะความสัมพันธ์	Varchar	50		
9	interests	ความสนใจ	Text	MAX		
10	cover_image_url	ลิงก์รูปปก	Text	MAX		
11	visibility_settings	การตั้งค่าความเป็นส่วนตัว	Jsonb	MAX		
12	social_links	ลิงก์โซเชียลมีเดีย	Jsonb	MAX		
13	created_at	วันที่สร้างประวัติโดยย่อ	Timestamp	19		
14	updated_at	วันที่อัปเดตประวัติโดยย่อ	Timestamp	19		

พจนานุกรมการตั้งค่าผู้ใช้งาน (User Settings) ดังตารางที่ 3-12

ตารางที่ 3-12 พจนานุกรมการตั้งค่าผู้ใช้งาน (User Settings)

No.	Name	Description	Data type	Length	Key	Reference
1	id	รหัสการตั้งค่า	Uuid	36	PK	
2	user_id	รหัสผู้ใช้งาน	Uuid	36	FK	User
3	language	ภาษาที่ใช้	Varchar	10		
4	timezone	เขตเวลา	Varchar	50		
5	theme	ธีมการ แสดงผล	Varchar	20		
6	Notifications _enabled	เปิดการแจ้ง เตือน	Boolean	1		
7	Email _notifications	แจ้งเตือนทาง อีเมล	Boolean	1		
8	push_notifications	แจ้งเตือนแบบ พุด	Boolean	1		
9	face_recognition _enabled	เปิดระบบ จดจำใบหน้า	Boolean	1		
10	auto_tag_enabled	แท็กใบหน้า อัตโนมัติ	Boolean	1		
11	created_at	วันที่สร้างการ ตั้งค่า	Timestamp	19		
12	updated_at	วันที่อัปเดต การตั้งค่า	Timestamp	19		

3.6.4.3 พจนานุกรมพจนานุกรมเซสชันผู้ใช้งาน (User Session) ดังตารางที่ 3-13

ตารางที่ 3-13 พจนานุกรมเซสชันผู้ใช้งาน (User Session)

No.	Name	Description	Data type	Length	Key	Reference
1	id	รหัสเซสชัน	Uuid	36	PK	
2	user_id	รหัสผู้ใช้งาน	Uuid	36	FK	User
3	session_token	โทเค็นเซสชัน	Varchar	255		
4	refresh_token	โทเค็นรีเฟรช	Varchar	255		
5	ip_address	ที่อยู่ IP	Inet	15		
6	user_agent	ข้อมูลเว็บ เบราว์เซอร์	Text	MAX		

ตารางที่ 3-13 พจนานุกรมเซสชันผู้ใช้งาน (User Session) (ต่อ)

No.	Name	Description	Data type	Length	Key	Reference
7	device_info	ข้อมูลอุปกรณ์	Jsonb	MAX		
8	is_active	สถานะการใช้งาน	Boolean	1		
9	expires_at	วันหมดอายุ	Timestamp	19		
10	created_at	วันที่สร้างเซสชัน	Timestamp	19		
11	last_used_at	ใช้งานล่าสุด	Timestamp	19		

3.6.4.4 พจนานุกรมการเข้าสู่ระบบ (Login Attempt) ดังตารางที่ 3-14

ตารางที่ 3-14 พจนานุกรมการเข้าสู่ระบบ (Login Attempt)

No.	Name	Description	Data type	Length	Key	Reference
1	id	รหัสการเข้าสู่ระบบ	Uuid	36	PK	
2	email	อีเมลที่ใช้เข้าสู่ระบบ	Varchar	100		
3	ip_address	ที่อยู่ IP	Inet	15		
4	success	สถานะการเข้าสู่ระบบ	Boolean	1		
5	failure_reason	เหตุผลที่ล้มเหลว	Varchar	100		
6	user_agent	ข้อมูลเว็บเบราว์เซอร์	Text	MAX		
7	attempted_at	เวลาที่พยายามเข้าสู่ระบบ	Timestamp	19		

3.6.4.5 พจนานุกรมโทเคนรีเซ็ตรหัสผ่าน (Password Reset Token) ดังตารางที่ 3-15

ตารางที่ 3-15 พจนานุกรมโทเค็นรีเซ็ตรหัสผ่าน (Password Reset Token)

No.	Name	Description	Data type	Length	Key	Reference
1	id	รหัสโทเค็น	Uuid	36	PK	
2	user_id	รหัสผู้ใช้งาน	Uuid	36	FK	User
3	token	โทเค็นรีเซ็ต	Varchar	255		
4	expires_at	วันหมดอายุ	Timestamp	19		
5	used_at	เวลาที่ใช้งาน	Timestamp	19		
6	created_at	วันที่สร้างโทเค็น	Timestamp	19		

3.6.4.6 พจนานุกรมโทเค็นยืนยันอีเมล (Email Verification Token) ดังตารางที่ 3-16

ตารางที่ 3-16 พจนานุกรมโทเค็นยืนยันอีเมล (Email Verification Token)

No.	Name	Description	Data type	Length	Key	Reference
1	id	รหัสโทเค็น	Uuid	36	PK	
2	user_id	รหัสผู้ใช้งาน	Uuid	36	FK	User
3	token	โทเค็นยืนยัน	Varchar	255	UK	
4	email	อีเมลที่ยืนยัน	Varchar	100		
5	expires_at	วันหมดอายุ	Timestamp	19		
6	verified_at	เวลาที่ยืนยัน	Timestamp	19		

3.6.4.7 พจนานุกรมข้อมูลใบหน้า (Face Data) ดังตารางที่ 3-17

ตารางที่ 3-17 พจนานุกรมข้อมูลใบหน้า (Face Data)

No.	Name	Description	Data type	Length	Key	Reference
1	id	รหัสข้อมูลใบหน้า	Uuid	36	PK	
2	user_id	รหัสผู้ใช้งาน	Uuid	36	FK	User
3	Embedding_model	แบบจำลองที่ใช้ฝึก	Varchar	50		
4	embedding_data	ข้อมูลเวกเตอร์ใบหน้า	Float8	MAX		
5	image_url	ลิงก์รูปภาพ	Text	MAX		
6	image_hash	แฮชของรูปภาพ	Varchar	64		

ตารางที่ 3-17 พจนานุกรมข้อมูลใบหน้า (Face Data) (ต่อ)

No.	Name	Description	Data type	Length	Key	Reference
7	quality_score	คะแนน คุณภาพภาพ	Float	8		
8	Detection_confidence	ความมั่นใจใน การตรวจจับ	Float	8		
9	landmark_data	จุดสำคัญบน ใบหน้า	Jsonb	MAX		
10	is_primary	เป็นข้อมูล หลัก	Boolean	1		
11	created_at	วันที่สร้าง ข้อมูล	Timestamp	19		
12	updated_at	วันที่อัปเดต ข้อมูล	Timestamp	19		

3.6.4.8 พจนานุกรมข้อมูลการฝึกใบหน้า (Face Training Data) ดังตารางที่ 3-18

ตารางที่ 3-18 พจนานุกรมข้อมูลการฝึกใบหน้า (Face Training Data)

No.	Name	Description	Data type	Length	Key	Reference
1	id	รหัสข้อมูลการ ฝึก	Uuid	36	PK	
2	user_id	รหัสผู้ใช้งาน	Uuid	36	FK	User
3	image_url	ลิงก์รูปภาพ	Text	MAX		
4	face_bbox	กรอบขอบ ใบหน้า	Jsonb	MAX		
5	Embedding_vector	เวกเตอร์การ ฝึก	Float8	MAX		
6	quality_score	คะแนน คุณภาพ	Float	8		
7	is_verified	ข้อมูลที่ยืนยัน แล้ว	Boolean	1		
8	source_type	ประเภท แหล่งที่มา	Varchar	50		
9	created_at	วันที่สร้าง ข้อมูล	Timestamp	19		

3.6.4.9 พจนานุกรมโพสต์ (Post) ดังตารางที่ 3-19

ตารางที่ 3-19 พจนานุกรมโพสต์ (Post)

No.	Name	Description	Data type	Length	Key	Reference
1	id	รหัสโพสต์	Uuid	36	PK	
2	user_id	รหัสผู้ใช้งาน	Uuid	36	FK	User
3	content	เนื้อหาโพสต์	Text	MAX		
4	image_urls	ลิงก์รูปภาพ	Text	MAX		
5	video_url	ลิงก์วิดีโอ	Text	MAX		
6	location	สถานที่	Varchar	100		
7	privacy_level	ระดับความเป็นส่วนตัว	Varchar	20		
8	is_archived	สถานะการเก็บถาวร	Boolean	1		
9	likes_count	จำนวนไลค์	Integer	10		
10	comments_count	จำนวนคอมเมนต์	Integer	10		
11	shares_count	จำนวนแชร์	Integer	10		
12	created_at	วันที่สร้างโพสต์	Timestamp	19		

3.6.4.10 พจนานุกรมแท็กใบหน้าในโพสต์ (Face Tag in Post) ดังตารางที่ 3-20

ตารางที่ 3-20 พจนานุกรมแท็กใบหน้าในโพสต์ (Face Tag in Post)

No.	Name	Description	Data type	Length	Key	Reference
1	id	รหัสแท็ก	Uuid	36	PK	
2	post_id	รหัสโพสต์	Uuid	36	FK	Post
3	tagged_user_id	รหัสผู้ถูกแท็ก	Uuid	36	FK	User
4	tager_user_id	รหัสผู้แท็ก	Uuid	36	FK	User
5	image_url	ลิงก์รูปภาพ	Text	MAX		
6	face_bbox	กรอบขอบใบหน้า	Jsonb	MAX		
7	face_embedding	ข้อมูลเวกเตอร์ใบหน้า	Float8	MAX		

ตารางที่ 3-20 พจนานุกรมแท็กใบหน้าในโพสต์ (Face Tag in Post) (ต่อ)

No.	Name	Description	Data type	Length	Key	Reference
8	confidence_score	คะแนนความมั่นใจ	Float	8		
9	is_confirmed	สถานะการยืนยัน	Boolean	1		
10	status	สถานะแท็ก	Varchar	20		
11	created_at	วันที่สร้างแท็ก	Timestamp	19		
12	updated_at	วันที่อัปเดตแท็ก	Timestamp	19		

3.6.4.11 พจนานุกรมไลค์โพสต์ (Post Like) ดังตารางที่ 3-21

ตารางที่ 3-21 พจนานุกรมไลค์โพสต์ (Post Like)

No.	Name	Description	Data type	Length	Key	Reference
1	id	รหัสไลค์	Uuid	36	PK	
2	post_id	รหัสโพสต์	Uuid	36	FK	Post
3	user_id	รหัสผู้ใช้งาน	Uuid	36	FK	User
4	created_at	วันที่กดไลค์	Timestamp	19		

3.6.4.12 พจนานุกรมคอมเมนต์โพสต์ (Post Comment) ดังตารางที่ 3-22

ตารางที่ 3-22 พจนานุกรมคอมเมนต์โพสต์ (Post Comment)

No.	Name	Description	Data type	Length	Key	Reference
1	id	รหัสคอมเมนต์	Uuid	36	PK	
2	post_id	รหัสโพสต์	Uuid	36	FK	Post
3	user_id	รหัสผู้ใช้งาน	Uuid	36	FK	User
4	Parent_comment_id	รหัสคอมเมนต์แม่	Uuid	36	FK	PostComment
5	content	เนื้อหาคอมเมนต์	Text	MAX		
6	image_url	ลิงก์รูปภาพ	Text	MAX		
7	likes_count	จำนวนไลค์	Integer	10		

ตารางที่ 3-22 พจนานุกรมคอมเมนต์โพสต์ (Post Comment) (ต่อ)

No.	Name	Description	Data type	Length	Key	Reference
8	replies_count	จำนวนตอบกลับ	Integer	10		
9	is_edited	สถานะการแก้ไข	Boolean	1		
10	created_at	วันที่สร้างคอมเมนต์	Timestamp	19		
11	updated_at	วันที่อัปเดตคอมเมนต์	Timestamp	19		

3.6.4.13 พจนานุกรมไลค์คอมเมนต์ (Comment Like) ดังตารางที่ 3-23

ตารางที่ 3-23 พจนานุกรมไลค์คอมเมนต์ (Comment Like)

No.	Name	Description	Data type	Length	Key	Reference
1	id	รหัสไลค์	Uuid	36	PK	
2	comment_id	รหัสคอมเมนต์	Uuid	36	FK	PostComment
3	user_id	รหัสผู้ใช้งาน	Uuid	36	FK	User
4	created_at	วันที่กดไลค์	Timestamp	19		

3.6.4.14 พจนานุกรมการเชื่อมต่อผู้ใช้งาน (User Connection) ดังตารางที่ 3-24

ตารางที่ 3-24 พจนานุกรมการเชื่อมต่อผู้ใช้งาน (User Connection)

No.	Name	Description	Data type	Length	Key	Reference
1	id	รหัสการเชื่อมต่อ	Uuid	36	PK	
2	requester_id	รหัสผู้ขอเชื่อมต่อ	Uuid	36	FK	User
3	addressee_id	รหัสผู้รับคำขอ	Uuid	36	FK	User
4	status	สถานะการเชื่อมต่อ	Varchar	20		
5	created_at	วันที่สร้างคำขอ	Timestamp	19		
6	updated_at	วันที่อัปเดตสถานะ	Timestamp	19		

3.6.4.15 พจนานุกรมการสนทนา (Conversation) ดังตารางที่ 3-25

ตารางที่ 3-25 พจนานุกรมการสนทนา (Conversation)

No.	Name	Description	Data type	Length	Key	Reference
1	id	รหัสการสนทนา	Uuid	36	PK	
2	type	ประเภทการสนทนา	Varchar	20		
3	name	ชื่อการสนทนา	Varchar	100		
4	description	คำอธิบายการสนทนา	Text	MAX		
5	image_url	รูปภาพการสนทนา	Text	MAX		
6	created_by	ผู้สร้างการสนทนา	Uuid	36	FK	User
7	is_active	สถานะการใช้งาน	Boolean	1		
8	last_message_at	เวลาข้อความล่าสุด	Timestamp	19		
9	created_at	วันที่สร้างการสนทนา	Timestamp	19		
10	updated_at	วันที่อัปเดตการสนทนา	Timestamp	19		

3.6.4.16 พจนานุกรมผู้เข้าร่วมการสนทนา (Conversation Participant) ดังตารางที่ 3-26

ตารางที่ 3-26 พจนานุกรมผู้เข้าร่วมการสนทนา (Conversation Participant)

No.	Name	Description	Data type	Length	Key	Reference
1	id	รหัสผู้เข้าร่วม	Uuid	36	PK	
2	conversation_id	รหัสการสนทนา	Uuid	36	FK	Conversation
3	user_id	รหัสผู้ใช้งาน	Uuid	36	FK	User
4	role	บทบาทในการสนทนา	Varchar	20		
5	joined_at	เวลาเข้าร่วม	Timestamp	19		

ตารางที่ 3-26 พจนานุกรมผู้เข้าร่วมการสนทนา (Conversation Participant) (ต่อ)

No.	Name	Description	Data type	Length	Key	Reference
6	left_at	เวลาออกจาก การสนทนา	Timestamp	19		
7	is_muted	สถานะปิด เสียง	Boolean	1		
8	last_read_at	เวลาอ่าน ล่าสุด	Timestamp	19		

3.6.4.17 พจนานุกรมข้อความ (Message) ดังตารางที่ 3-27

ตารางที่ 3-27 พจนานุกรมข้อความ (Message)

No.	Name	Description	Data type	Length	Key	Reference
1	id	รหัสข้อความ	Uuid	36	PK	
2	conversation_id	รหัสการ สนทนา	Uuid	36	FK	Conversat ion
3	sender_id	รหัสผู้ส่ง ข้อความ	Uuid	36	FK	User
4	content	เนื้อหา ข้อความ	Text	MAX		
5	message_type	ประเภท ข้อความ	Varchar	20		
6	file_url	ลิงก์ไฟล์แนบ	Text	MAX		
7	file_type	ประเภทไฟล์	Varchar	20		
8	file_size	ขนาดไฟล์	Integer	10		
9	reply_to _message_id	รหัสข้อความ ที่ตอบกลับ	Uuid	36	FK	Message
10	is_edited	สถานะการ แก้ไข	Boolean	1		
11	is_deleted	สถานะการลบ	Boolean	1		
12	created_at	วันที่ส่ง ข้อความ	Timestamp	19		
13	updated_at	วันที่อัปเดต ข้อความ	Timestamp	19		

3.6.4.18 พจนานุกรมแท็กใบหน้าในข้อความ (Face Tag in Message) ดังตารางที่ 3-28

ตารางที่ 3-28 พจนานุกรมแท็กใบหน้าในข้อความ (Face Tag in Message)

No.	Name	Description	Data type	Length	Key	Reference
1	id	รหัสแท็ก	Uuid	36	PK	
2	message_id	รหัสข้อความ	Uuid	36	FK	Message
3	tagged_user_id	รหัสผู้ถูกแท็ก	Uuid	36	FK	User
4	face_bbox	กรอบขอบ ใบหน้า	Jsonb	MAX		
5	confidence_score	คะแนนความ มั่นใจ	Float	8		
6	created_at	วันที่สร้างแท็ก	Timestamp	19		

3.6.4.19 พจนานุกรมรีแอคชันข้อความ (Message Reaction) ดังตารางที่ 3-29

ตารางที่ 3-29 พจนานุกรมรีแอคชันข้อความ (Message Reaction)

No.	Name	Description	Data type	Length	Key	Reference
1	id	รหัสรีแอคชัน	Uuid	36	PK	
2	message_id	รหัสข้อความ	Uuid	36	FK	Message
3	user_id	รหัสผู้ใช้งาน	Uuid	36	FK	User
4	reaction	รีแอคชัน	Varchar	10		
5	created_at	วันที่รีแอคต์	Timestamp	19		

3.6.4.20 พจนานุกรมการแจ้งเตือน (Notification) ดังตารางที่ 3-30

ตารางที่ 3-30 พจนานุกรมการแจ้งเตือน (Notification)

No.	Name	Description	Data type	Length	Key	Reference
1	id	รหัสการแจ้ง เตือน	Uuid	36	PK	
2	user_id	รหัสผู้รับแจ้ง เตือน	Uuid	36	FK	User
3	from_user_id	รหัสผู้ส่งแจ้ง เตือน	Uuid	36	FK	User
4	type	ประเภทการ แจ้งเตือน	Varchar	30		

ตารางที่ 3-30 พจนานุกรมการแจ้งเตือน (Notification) (ต่อ)

No.	Name	Description	Data type	Length	Key	Reference
5	title	หัวข้อการแจ้งเตือน	Varchar	100		
6	content	เนื้อหาการแจ้งเตือน	Text	MAX		
7	related_type	ประเภทข้อมูลที่เกี่ยวข้อง	Varchar	30		
8	related_id	รหัสข้อมูลที่เกี่ยวข้อง	Uuid	36		
9	is_read	สถานะการอ่าน	Boolean	1		
10	created_at	วันที่สร้างการแจ้งเตือน	Timestamp	19		

3.6.4.21 พจนานุกรมกิจกรรมผู้ใช้งาน (User Activity) ดังตารางที่ 3-31

ตารางที่ 3-31 พจนานุกรมกิจกรรมผู้ใช้งาน (User Activity)

No.	Name	Description	Data type	Length	Key	Reference
1	id	รหัสกิจกรรม	Uuid	36	PK	
2	user_id	รหัสผู้ใช้งาน	Uuid	36	FK	User
3	activity_type	ประเภทกิจกรรม	Varchar	30		
4	target_type	ประเภทเป้าหมาย	Varchar	30		
5	target_id	รหัสเป้าหมาย	Uuid	36		
6	metadata	ข้อมูลเพิ่มเติม	Jsonb	MAX		
7	created_at	วันที่ทำกิจกรรม	Timestamp	19		

3.7 การเลือกแบบจำลอง Face Detection

การตรวจจับใบหน้า (Face Detection) เป็นกระบวนการพื้นฐานที่สำคัญที่สุดในระบบแพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพบุคคลด้วยการจดจำใบหน้า เนื่องจากเป็นขั้นตอนแรกที่ต้องระบุตำแหน่งของใบหน้าในภาพก่อนจะนำไปสู่กระบวนการจดจำใบหน้า (Face Recognition) และการตรวจจับภาพปลอม (Deepfake Detection) การเลือกแบบจำลองที่เหมาะสมจึงส่งผลต่อประสิทธิภาพของระบบทั้งหมด

3.7.1 การสำรวจแบบจำลอง Face Detection

ในปัจจุบันมีแบบจำลอง Face Detection หลากหลายประเภทที่พัฒนาขึ้นด้วยเทคโนโลยีที่แตกต่างกัน โดยสามารถจำแนกได้เป็น 4 กลุ่มหลัก ดังนี้

3.7.1.1 แบบจำลองแบบดั้งเดิม (Traditional Methods) Haar Cascade Classifiers เป็นแบบจำลองแรก ๆ ที่ใช้การเรียนรู้แบบ Machine Learning โดยใช้ Haar-like features และ AdaBoost algorithm ในการตรวจจับใบหน้า แม้จะมีขนาดเล็ก และเร็ว แต่มีข้อจำกัดในการตรวจจับใบหน้าที่มีมุมเอียงหรืออยู่ในสภาพแสงที่ไม่เหมาะสม

3.7.1.2 แบบจำลองแบบ Multi-task Learning MTCNN (Multi-task Convolutional Neural Networks) เป็นแบบจำลองที่ใช้ CNN หลายระดับ 3 ระดับ ในการทำงาน โดยระดับแรก (P-Net) ทำหน้าที่สร้าง candidate windows ระดับที่สอง (R-Net) กรอง False Positives และระดับสุดท้าย (O-Net) ทำการปรับแต่งตำแหน่งใบหน้า และจุดสำคัญ (landmarks) อย่างละเอียด

3.7.1.3 แบบจำลองแบบ Single-Shot Detection YOLO (You Only Look Once) Series ประกอบด้วย YOLOv9c, YOLOv9e และ YOLOv11 ทั้งแบบ m และ n เป็นแบบจำลองที่สามารถตรวจจับวัตถุ และใบหน้าในการทำงานเพียงครั้งเดียว มีความเร็วสูง และเหมาะสำหรับการใช้งานแบบ Real-Time RetinaFace เป็นแบบจำลองที่พัฒนาโดยเน้นการตรวจจับใบหน้าโดยเฉพาะ ใช้เทคนิค Feature Pyramid Networks (FPN) และ Context Module เพื่อเพิ่มประสิทธิภาพในการตรวจจับใบหน้าขนาดเล็ก และในสภาพแวดล้อมที่ซับซ้อน.

3.7.1.4 แบบจำลองแบบ Lightweight BlazeFace
Lightweight BlazeFace ถูกพัฒนาโดย Google เพื่อใช้งานบนอุปกรณ์มือถือ มีขนาดเล็กมากเพียง 0.3 MB และสามารถทำงานได้เร็วบนอุปกรณ์ที่มีทรัพยากรจำกัด InsightFace เป็นแบบจำลองที่พัฒนาสำหรับ Face Recognition แต่มีส่วนของ Face Detection ที่มีประสิทธิภาพสูง โดยใช้เทคนิค ArcFace Loss และ Deep Residual Networks

3.7.2 หลักเกณฑ์การเปรียบเทียบ และการประเมินผล

การเปรียบเทียบประสิทธิภาพของแบบจำลอง Face Detection ต้องพิจารณาปัจจัยหลายด้าน เพื่อให้ได้แบบจำลองที่เหมาะสมกับการนำไปใช้งานจริงในระบบแพลตฟอร์มสื่อสังคมออนไลน์ และการจัดการรูปภาพบุคคลด้วยการจดจำใบหน้า เกณฑ์การประเมินประสิทธิภาพ ความแม่นยำ (Accuracy) คำนวณจากสัดส่วนของการตรวจจับที่ถูกต้องต่อจำนวนภาพทั้งหมด โดยนับเป็นการตรวจจับที่ถูกต้องเมื่อ Intersection over Union (IoU) มากกว่า 0.5 ความเร็วในการประมวลผล (FPS - Frames Per Second) วัดจากจำนวนภาพที่แบบจำลองสามารถประมวลผลได้ในหนึ่งวินาทีเป็นตัวชี้วัดสำคัญสำหรับการใช้งานแบบ real-time การใช้ทรัพยากรระบบ ประกอบด้วย CPU

Usage (ร้อยละ), Memory Usage (GB), GPU Memory Usage (GB) และ GPU Utilization (ร้อยละ) เพื่อประเมินความต้องการทรัพยากรของแต่ละแบบจำลอง Efficiency Score คำนวณจากสูตร

$$\text{Efficiency Score} = \frac{\text{Accuracy}}{(\text{CPU_Usage} \times \text{Memory_Usage} \times \text{Model_Size})}$$

การวัดประสิทธิภาพเพิ่มเติม ขนาดแบบจำลอง (Model Size) วัดเป็น MB เพื่อประเมินความเหมาะสมในการ deployment บนอุปกรณ์ต่าง ๆ การรองรับ GPU ระบุว่าแบบจำลองสามารถใช้ประโยชน์จาก GPU acceleration ได้หรือไม่

3.7.3 การเตรียมข้อมูลทดสอบ (Test Dataset)

เพื่อให้การเปรียบเทียบมีความยุติธรรม และสะท้อนประสิทธิภาพที่แท้จริงในการใช้งาน การทดสอบได้ใช้ชุดข้อมูลมาตรฐานที่ได้รับการยอมรับในชุมชนวิจัยชุดข้อมูลที่ใช้ในการทดสอบ

WIDER FACE Dataset เป็นชุดข้อมูลมาตรฐานที่มีความหลากหลายของภาพใบหน้าในสภาพแวดล้อมต่าง ๆ มีภาพมากกว่า 32,000 ภาพ และใบหน้ามากกว่า 393,000 ใบหน้า แบ่งเป็น 3 ระดับความยาก คือ Easy, Medium และ Hard

3.7.3.1 CelebA Dataset ประกอบด้วยภาพบุคคลมากกว่า 200,000 ภาพ มีการติดป้ายกำกับ (annotation) ครบถ้วน เหมาะสำหรับการทดสอบการตรวจจับใบหน้าของบุคคลที่มีชื่อเสียง

3.7.3.2 FDDB (Face Detection Data Set and Benchmark) เป็นชุดข้อมูลมาตรฐานที่ใช้วัดประสิทธิภาพการตรวจจับใบหน้า ประกอบด้วยภาพ 2,845 ภาพ และใบหน้า 5,171 ใบหน้า

3.7.3.3 LFW (Labeled Faces in the Wild) เป็นชุดข้อมูลที่เน้นการทดสอบในสภาพแวดล้อมจริง (In-The-Wild) มีภาพมากกว่า 13,000 ภาพ ครอบคลุมใบหน้าในสภาพแสง และมุมมองที่หลากหลาย

การทดสอบในงานวิจัยนี้ใช้ชุดข้อมูลผสมผสานจากทุกแหล่งข้างต้น รวมทั้งสิ้น 49 ภาพที่คัดเลือกมาเป็นตัวแทนของสถานการณ์การใช้งานจริง ดังตารางที่ 3-32

ตารางที่ 3-32 การเลือกแบบจำลอง Face Detection

ข้อมูล	จำนวน
ชุดข้อมูลหลัก	49
ภาพทดสอบทั้งหมด	4
แบบจำลองที่ทดสอบ	9
ความครอบคลุม (ร้อยละ)	100

1) รายละเอียดชุดข้อมูลทดสอบ Face Detection ดังตารางที่ 3-33

ตารางที่ 3-33 รายละเอียดชุดข้อมูลทดสอบ Face Detection

ลำดับ	ชุดข้อมูล	ประเภทความยาก	จำนวนภาพ	คุณลักษณะของภาพ	สภาพแวดล้อมการถ่าย	ขนาดใบหน้า
1-6	WIDER FACE	Easy	6	ใบหน้าชัด, แสงเพียงพอ	ในร่ม, แสงควบคุม	ใหญ่ (>150px)
7-11	WIDER FACE	Medium	5	แสงน้อย, มุมเอียงเล็กน้อย	กึ่งในกึ่งนอก	ปานกลาง (50-150px)
12-16	WIDER FACE	Hard	5	แสงสะท้อน, ใบหน้าเล็ก	กลางแจ้ง, แสงธรรมชาติ	เล็ก (<50px)
17-23	CelebA	Mixed	7	บุคคลมีชื่อเสียง	อีเวนต์, งานแถลงข่าว	หลากหลาย
24-28	CelebA	Styled	5	แต่งหน้า, เครื่องประดับ	สตูดิโอ, ถ่ายแฟชั่น	ใหญ่-ปานกลาง
29-33	FDDB	Multi-face	5	หลายใบหน้าในภาพ	ภาพกลุ่ม, การประชุม	เล็ก-ปานกลาง
34-39	FDDB	Group	6	ภาพกลุ่มคน	งานสังคม, กิจกรรม	หลากหลาย
40-44	LFW	Wild	5	สภาพแวดล้อมจริง	ภาพข่าว, ชีวิตประจำวัน	ปานกลาง
45-49	LFW	Activity	5	กิจกรรมต่าง ๆ	กลางแจ้ง, กีฬา	เล็ก-ปานกลาง
รวมทั้งหมด			49	ครอบคลุมทุกสถานการณ์การใช้งาน		

2) การแบ่งตามระดับความยากในการตรวจจับ ดังตารางที่ 3-34

ตารางที่ 3-34 การแบ่งตามระดับความยากในการตรวจจับ

ระดับความยาก	จำนวนภาพ	ร้อยละ	คุณลักษณะของภาพ	เป้าหมายการทดสอบ
Easy (ง่าย)	6	12.2	ใบหน้าใหญ่, แสงดี, มุมตรง	ทดสอบความแม่นยำพื้นฐาน
Medium (ปานกลาง)	5	10.2	ใบหน้าปานกลาง, แสงเปลี่ยน	ทดสอบความทนทาน
Hard (ยาก)	5	10.2	ใบหน้าเล็ก, แสงน้อย, มีสิ่งบดบัง	ทดสอบขีดจำกัด
Mixed/Specialized (เฉพาะ)	33	67.3	สถานการณ์เฉพาะ, หลายใบหน้า	ทดสอบการใช้งานจริง
รวมทั้งหมด	49	100	ครอบคลุมทุกระดับความยาก	

3.7.4 การวิเคราะห์ และเปรียบเทียบประสิทธิภาพการวิเคราะห์ประสิทธิภาพของแบบจำลอง Face Detection แต่ละประเภทจำเป็นต้องพิจารณาหลายมิติเพื่อให้ได้ข้อมูลที่ครอบคลุมสำหรับการตัดสินใจเลือกใช้ในระบบงานจริง การจำแนกแบบจำลองตามกลุ่มเทคโนโลยีกลุ่มแบบจำลอง YOLO Series ประกอบด้วย YOLOv9c, YOLOv9e, YOLOv11m และ YOLOv11n เป็นแบบจำลองที่ใช้เทคนิค Single-Shot Detection ที่สามารถตรวจจับวัตถุ และใบหน้าในการประมวลผลเพียงครั้งเดียว มีความเร็วสูง และเหมาะสำหรับการใช้งานแบบ real-time กลุ่มแบบจำลอง Specialized Face Detection ประกอบด้วย MTCNN, RetinaFace และ InsightFace เป็นแบบจำลองที่พัฒนาขึ้นเพื่อการตรวจจับใบหน้าโดยเฉพาะ มีความแม่นยำสูงแต่อาจใช้เวลาในการประมวลผลมากกว่า กลุ่มแบบจำลอง Lightweight ประกอบด้วย BlazeFace และ Haar Cascade เป็นแบบจำลองที่เน้นความเร็ว และการใช้ทรัพยากรน้อย เหมาะสำหรับอุปกรณ์ที่มีข้อจำกัดด้านประสิทธิภาพ ตารางเปรียบเทียบประสิทธิภาพแบบจำลอง Face Detection

3) ผลการเปรียบเทียบประสิทธิภาพแบบจำลอง Face Detection ดังตารางที่ 3-35

ตารางที่ 3-35 ผลการเปรียบเทียบประสิทธิภาพแบบจำลอง Face Detection

แบบจำลอง	ประสิทธิภาพการตรวจจับ			การใช้ทรัพยากรระบบ				คุณลักษณะเพิ่มเติม		
	Accuracy (ร้อยละ)	FPS	เวลาเฉลี่ย (วินาที)	CPU (ร้อยละ)	Memory (GB)	GPU Memory (GB)	GPU Util (ร้อยละ)	Model Size (MB)	GPU Support	Efficiency Score
YOLO Series Models										
YOLOv11m										
YOLOv11n										
YOLOv9c										
YOLOv9e										
Specialized Face Detection Models										
MTCNN										
RetinaFace										
InsightFace										
Lightweight Models										
BlazeFace										
Haar Cascade										

การวิเคราะห์เชิงคุณภาพ ความสามารถในการตรวจจับใบหน้าขนาดเล็ก เป็นปัจจัยสำคัญสำหรับระบบแพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพบุคคลด้วยการจดจำใบหน้า เนื่องจากภาพในเอกสารราชการมักมีใบหน้าขนาดเล็ก แบบจำลองจะต้องสามารถตรวจจับใบหน้าที่มีขนาดตั้งแต่ 24x24 pixels ขึ้นไป ความทนทานต่อการเปลี่ยนแปลงของแสง ภาพเอกสารที่นำมาตรวจสอบอาจถ่ายในสภาพแสงที่หลากหลาย ตั้งแต่แสงธรรมชาติจนถึงแสงไฟในร่ม แบบจำลองจะต้องสามารถรับมือกับการเปลี่ยนแปลงเหล่านี้ได้ ความสามารถในการตรวจจับใบหน้าที่มีมุมเอียง เอกสารบัตรประชาชนอาจถ่ายในมุมที่ไม่ตั้งฉาก แบบจำลองจะต้องสามารถตรวจจับใบหน้าที่หมุนหรือเอียงได้อย่างแม่นยำ

3.7.5 เกณฑ์การตัดสินใจเลือกแบบจำลองการเลือกแบบจำลอง Face Detection สำหรับระบบแพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพบุคคลด้วยการจดจำใบหน้าต้องพิจารณาปัจจัยหลักดังนี้ ลำดับความสำคัญของเกณฑ์

3.7.5.1 ความแม่นยำ (Priority: สูงสุด) เนื่องจากระบบจะใช้ในการตรวจสอบข้อมูลทางราชการที่ต้องการความถูกต้องสูง ความแม่นยำในการตรวจจับใบหน้าจึงเป็นปัจจัยสำคัญที่สุด เป้าหมายคือต้องมีความแม่นยำอย่างน้อยร้อยละ 95 ขึ้นไป

3.7.5.2 ความเร็วในการประมวลผล (Priority: สูง) ระบบต้องสามารถตอบสนองผู้ใช้ได้ในเวลาที่รวดเร็ว เป้าหมายคือประมวลผลหนึ่งภาพภายใน 1-2 วินาที หรือมีความเร็วอย่างน้อย 1-2 FPS

3.7.5.3 การใช้ทรัพยากรระบบ (Priority: ปานกลาง) ระบบจะต้อง deploy บนเซิร์ฟเวอร์ที่มีข้อจำกัดด้านทรัพยากร จึงต้องพิจารณาการใช้ CPU, Memory และ GPU อย่างมีประสิทธิภาพ

3.7.5.4 ขนาดแบบจำลอง และการ Deploy (Priority: ปานกลาง) ขนาดแบบจำลองที่เล็กจะช่วยในการ deployment และการโหลดแบบจำลองที่เร็วขึ้น

3.7.5.5 เกณฑ์การคัดเลือกเบื้องต้น

1) เกณฑ์ขั้นต่ำ (Minimum Requirements):

2) Accuracy มากกว่าหรือเท่ากับ ร้อยละ 90

3) FPS มากกว่าหรือเท่ากับ 0.5 (เวลาประมวลผล น้อยกว่าหรือเท่ากับ 2 วินาทีต่อ

ภาพ)

4) Memory Usage น้อยกว่าหรือเท่ากับ 20 GB

5) รองรับการทำงานบน Linux Server

3.7.5.6 เกณฑ์ที่ต้องการ (Preferred Requirements):

1) Accuracy มากกว่าหรือเท่ากับ ร้อยละ 95

2) FPS มากกว่าหรือเท่ากับ 2 (เวลาประมวลผล น้อยกว่าหรือเท่ากับ 0.5 วินาทีต่อ

ภาพ)

3) Memory Usage น้อยกว่าหรือเท่ากับ 15 GB

4) Model Size น้อยกว่าหรือเท่ากับ 50 MB.

5) รองรับ GPU Acceleration

3.7.5.7 เกณฑ์ที่เหมาะสมที่สุด (Optimal Requirements):

1) Accuracy มากกว่าหรือเท่ากับ ร้อยละ 98
 2) FPS มากกว่าหรือเท่ากับ 5 (เวลาประมวลผล น้อยกว่าหรือเท่ากับ 0.2 วินาทีต่อภาพ)

- 3) Memory Usage น้อยกว่าหรือเท่ากับ 10 GB
- 4) Model Size น้อยกว่าหรือเท่ากับ 20 MB.
- 5) Efficiency Score มากกว่าหรือเท่ากับ 0.5

การประเมินแบบถ่วงน้ำหนัก

สูตรการคำนวณคะแนนรวม (Weighted Score) สำหรับการเลือกแบบจำลอง:

Weighted Score

$$= (\text{Accuracy} \times 0.4) + (\text{Speed} \times 0.3) + (\text{Resource} \times 0.2) + (\text{Size} \times 0.1)$$

โดยที่:

- 1) Accuracy = คะแนนความแม่นยำ 0-100
- 2) Speed = คะแนนความเร็ว 0-100
- 3) Resource = คะแนนการใช้ทรัพยากร 0-100
- 4) Size = คะแนนขนาดแบบจำลอง 0-100

3.7.6 การจัดเตรียมชุดข้อมูลภาพ (Image Dataset) การทดสอบประสิทธิภาพของแบบจำลอง Face Detection ในงานวิจัยนี้ใช้ชุดข้อมูลภาพที่เก็บรวบรวมมาจากสถานการณ์จริงเพื่อให้ได้ผลการประเมินที่สะท้อนการใช้งานจริงในระบบแพลตฟอร์มสื่อสารสังคมออนไลน์และการจัดการรูปภาพบุคคลด้วยการจดจำใบหน้า

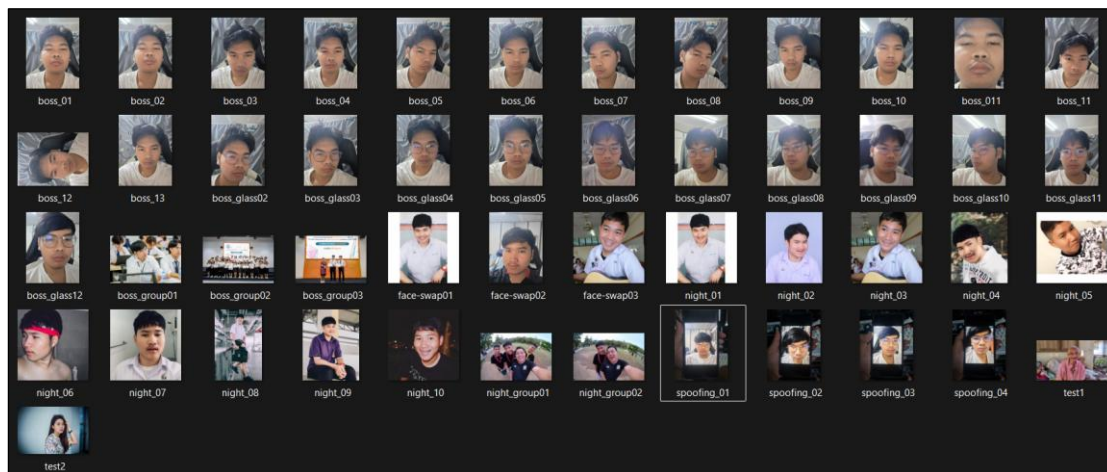
การเก็บรวบรวมข้อมูลภาพทดสอบชุดข้อมูลภาพทดสอบประกอบด้วยภาพทั้งหมด 49 ภาพที่เก็บรวบรวมจากผู้ทำวิจัย 2 คน โดยมีวัตถุประสงค์เพื่อครอบคลุมสถานการณ์การใช้งานที่หลากหลาย ดังนี้:

- 3.7.6.1 ผู้ทำวิจัยคนที่ 1 (Boss): มีภาพทั้งหมด 28 ภาพ แบ่งเป็น
 - 3.7.6.2 ภาพเดี่ยวในสภาวะปกติ: 13 ภาพ (boss_01.jpg - boss_13.jpg)
 - 3.7.6.3 ภาพเดี่ยวสวมแว่นตา: 12 ภาพ (boss_glass01.jpg - boss_glass12.jpg)
 - 3.7.6.4 ภาพกลุ่มที่มีผู้ทำวิจัยอยู่ในภาพ: 3 ภาพ (boss_group01.jpg - boss_group03.jpg)
- 3.7.6.5 ผู้ช่วยทำวิจัย (Night): มีภาพทั้งหมด 12 ภาพ แบ่งเป็น
 - 3.7.6.6 ภาพเดี่ยวในสภาพแสงน้อย: 10 ภาพ (night_01.jpg - night_10.jpg)
 - 3.7.6.7 ภาพกลุ่มที่มีผู้ช่วยวิจัยอยู่ในภาพ: 2 ภาพ (night_group01.jpg - night_group02.jpg)
 - 3.7.6.8 ภาพเฉพาะสำหรับการทดสอบระบบรักษาความปลอดภัย: 9 ภาพ แบ่งเป็น

3.7.6.9 ภาพ Spoofing (ถ่ายจากหน้าจอโทรศัพท์): 4 ภาพ (spoofing_01.jpg - spoofing_04.jpg)

3.7.6.10 ภาพ Face Swap (เทคนิค Deepfake): 3 ภาพ (face-swap01.png - face-swap03.png)

3.7.6.11 ภาพอื่น ๆ: 2 ภาพ ดังภาพที่ 3-22



ภาพที่ 3-22 ภาพถ่ายตัวอย่างต้นฉบับของ ชุดข้อมูลภาพ (Image Dataset)

3.7.7 การทดสอบประสิทธิภาพการครอบตัดใบหน้า

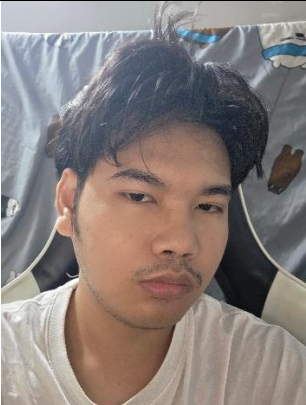
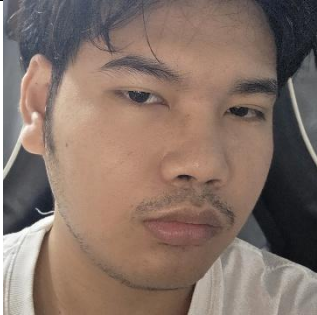
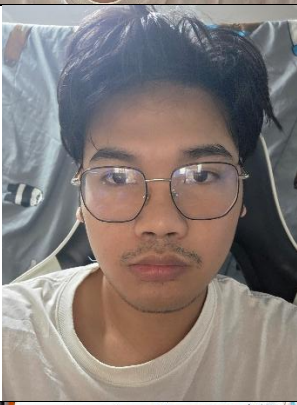
เพื่อประเมินประสิทธิภาพการตรวจจับใบหน้าของแต่ละแบบจำลองอย่างเป็นรูปธรรม ได้มีการพัฒนาสคริปต์สำหรับทดสอบความสามารถในการตรวจจับและครอบตัดใบหน้า (Face Cropping) จากภาพต้นฉบับ สคริปต์นี้จะประมวลผลชุดข้อมูลภาพทดสอบด้วยแบบจำลองต่างๆ และนับจำนวนใบหน้าที่สามารถตรวจจับและตัดแยกออกมาได้สำเร็จ โดยมีกระบวนการทำงานตามแผนผังในภาพที่ 3-23



ภาพที่ 3-23 แผนผังกระบวนการทำงานของสคริปต์ทดสอบการครอบตัดใบหน้า

จากภาพที่ 3-22 ในการทดสอบนี้ ได้ใช้แบบจำลองทั้ง 9 ตัวตามที่กล่าวมา อย่างไรก็ตามพบว่าแบบจำลอง RetinaFace มีปัญหาความเข้ากันได้ของไลบรารี ทำให้ไม่สามารถประมวลผลได้อย่างสมบูรณ์ จึงคงเหลือแบบจำลองที่ใช้ในการเปรียบเทียบผลลัพธ์เชิงปฏิบัติทั้งสิ้น 8 ตัว สามารถศึกษารายละเอียดการทำงานเพิ่มเติมได้ในหัวข้อ 4.2.1 กระบวนการประมวลผลการตรวจจับใบหน้า

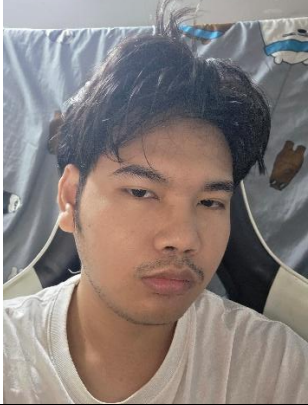
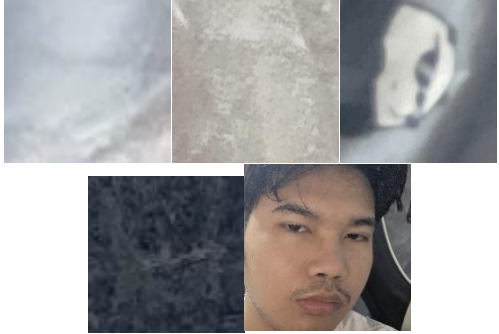
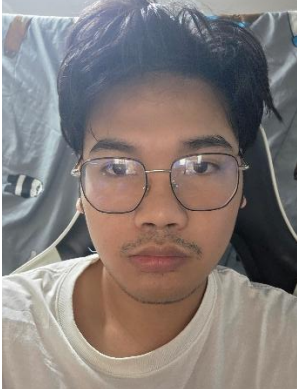
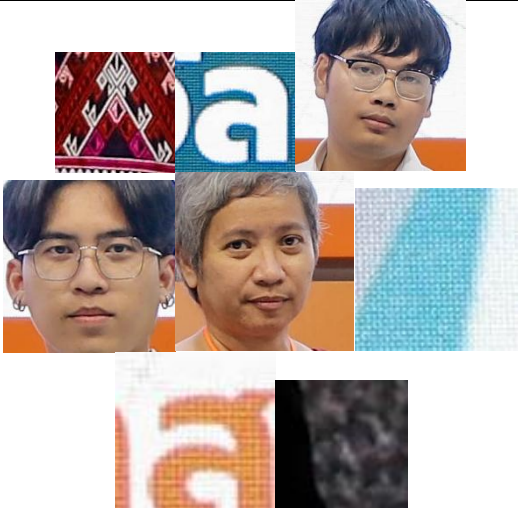
3.7.7.1 การทดสอบแบบจำลอง blazeface ได้ผลลัพธ์ ดังภาพที่ 3-24

ภาพต้นฉบับ	ผลลัพธ์
	
	
	ไม่สามารถตรวจจับใบหน้าได้

ภาพที่ 3-24 การทดสอบการครอบตัดใบหน้าด้วย blazeface

จากภาพที่ 3-24 การทดสอบการครอบตัดใบหน้าด้วย BlazeFace แสดงให้เห็นว่าแบบจำลอง BlazeFace ไม่สามารถตรวจจับใบหน้าจากภาพกลุ่มได้เลย เนื่องจากแบบจำลองนี้ไม่สามารถแยกและจับใบหน้าจากภาพที่มีหลายคนได้อย่างมีประสิทธิภาพ ผลลัพธ์แสดงให้เห็นว่าในภาพต้นฉบับไม่มีใบหน้าที่ถูกตรวจจับหรือครอบตัดออกมา ซึ่งแสดงถึงข้อจำกัดในการใช้งานของแบบจำลอง BlazeFace เมื่อทำการประมวลผลภาพที่มีความซับซ้อน เช่น ภาพที่มีหลายคนหรือใบหน้าที่อยู่ในมุมที่ไม่ชัดเจนจากการถ่ายภาพ เป็นต้น

3.7.7.2 การทดสอบแบบจำลอง haar_frontal ได้ผลลัพธ์ ดังภาพที่ 3-25

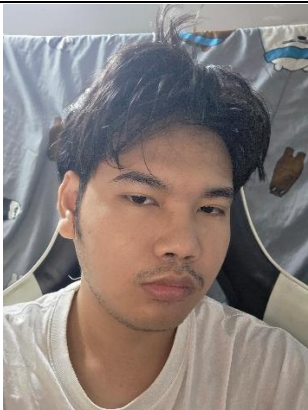
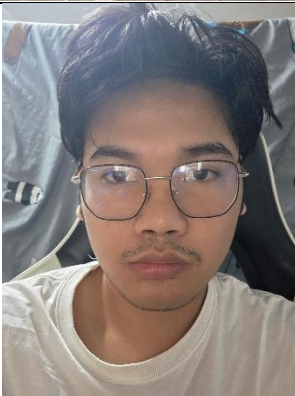
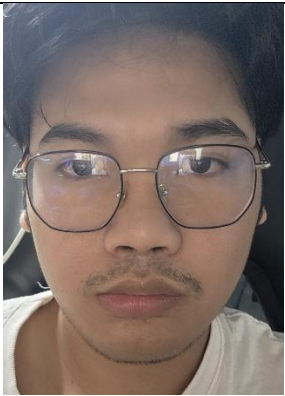
ภาพต้นฉบับ	ผลลัพธ์
	
	
	

ภาพที่ 3-25 การทดสอบการครอบตัดใบหน้าด้วย haar_frontal

จากภาพที่ 3-25 การทดสอบการครอบตัดใบหน้าด้วย Haar_frontal แสดงให้เห็นว่าแบบจำลอง Haar_frontal มีการตรวจจับพื้นหลังแทนที่จะเป็นใบหน้าของมนุษย์ ผลลัพธ์แสดงให้เห็นว่าแบบจำลองไม่สามารถแยกแยะใบหน้าจากภาพได้อย่างถูกต้อง และในบางกรณีอาจทำให้ระบบ

ตรวจจับพื้นหลังหรือวัตถุอื่น ๆ ที่ไม่เกี่ยวข้องกับใบหน้า ทำให้แบบจำลองนี้ไม่สามารถทำงานได้ดีเมื่อมีความซับซ้อนในภาพ เช่น การมีกระบวนการตรวจจับผิดพลาดจากการแยกใบหน้า เป็นต้น

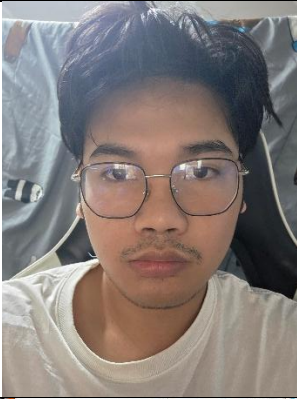
3.7.7.3 การทดสอบแบบจำลอง insightface ได้ผลลัพธ์ ดังภาพที่ 3-26

ภาพต้นฉบับ	ผลลัพธ์
	
	
	

ภาพที่ 3-26 การทดสอบการครอบตัดใบหน้าด้วย insightface

จากภาพที่ 3-26 การทดสอบการครอบตัดใบหน้าด้วย InsightFace แสดงให้เห็นว่าแบบจำลอง InsightFace สามารถตรวจจับใบหน้าได้สำเร็จในทุกภาพทดสอบ ผลลัพธ์แสดงว่า InsightFace มีความสามารถในการตรวจจับใบหน้าได้อย่างแม่นยำในทุกกรณี ไม่ว่าจะเป็นภาพเดี่ยวหรือภาพกลุ่ม ซึ่งทำให้ InsightFace เป็นแบบจำลองที่มีประสิทธิภาพสูงในการตรวจจับใบหน้าในสถานการณ์ที่หลากหลายและซับซ้อน

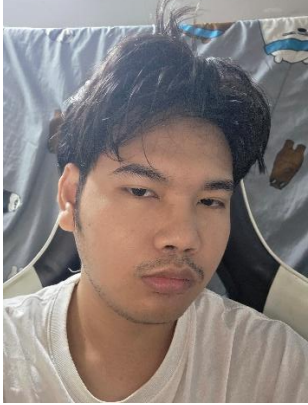
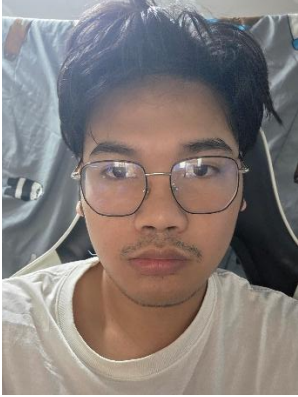
3.7.7.4 การทดสอบแบบจำลอง MTCNN ได้ผลลัพธ์ ดังภาพที่ 3-27

ภาพต้นฉบับ	ผลลัพธ์
	
	ไม่สามารถจับใบหน้าได้
	

ภาพที่ 3-27 การทดสอบการครอบตัดใบหน้าด้วย MTCNN

จากภาพที่ 3-27 การทดสอบการครอบตัดใบหน้าด้วย MTCNN แสดงให้เห็นว่าแบบจำลอง MTCNN ไม่สามารถตรวจจับใบหน้าในภาพที่มีแว่นตาได้ ผลลัพธ์นี้แสดงถึงข้อจำกัดของแบบจำลอง MTCNN ในการจัดการกับภาพที่มีการบังบางส่วนของใบหน้า เช่น เมื่อมีการสวมแว่นตาหรือสิ่งกีดขวางอื่น ๆ ซึ่งทำให้แบบจำลองไม่สามารถตรวจจับใบหน้าได้อย่างแม่นยำในกรณีนี้

3.7.7.5 การทดสอบแบบจำลอง yolov9c ได้ผลลัพธ์ ดังภาพที่ 3-28

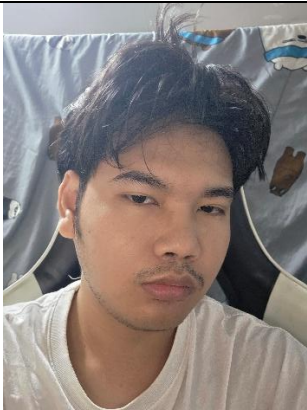
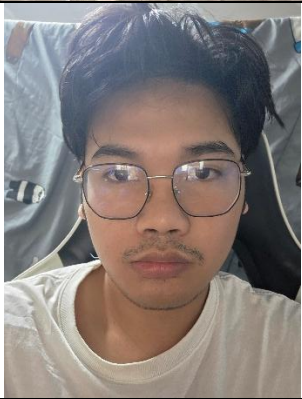
ภาพต้นฉบับ	ผลลัพธ์
	
	
	

ภาพที่ 3-28 การทดสอบการครอบตัดใบหน้าด้วย yolov9c

จากภาพที่ 3-28 การทดสอบการครอบตัดใบหน้าด้วย YOLOv9c แสดงให้เห็นว่าแบบจำลอง YOLOv9c สามารถตรวจจับใบหน้าได้ถูกต้อง แต่พบว่าการซ้ำใบหน้าเดิมหลายครั้งมาก ผลลัพธ์แสดงให้เห็นว่าแบบจำลองสามารถตรวจจับใบหน้าในภาพได้ แต่การจับใบหน้าซ้ำหลายครั้งอาจเกิด

จากการตั้งค่าแบบจำลองที่ไม่เหมาะสมหรือข้อจำกัดในการแยกแยะใบหน้าในบางมุม ทำให้เกิดการซ้ำในการตรวจจับ

3.7.7.6 การทดสอบแบบจำลอง yolov9e ได้ผลลัพธ์ ดังภาพที่ 3-29

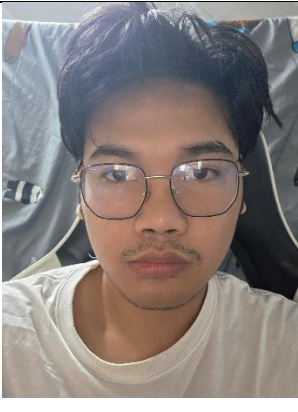
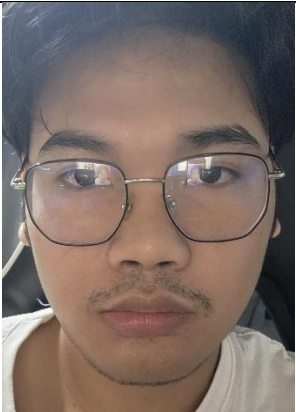
ภาพต้นฉบับ	ผลลัพธ์
	
	
	

ภาพที่ 3-29 การทดสอบการครอบตัดใบหน้าด้วย yolov9e

จากภาพที่ 3-29 การทดสอบการครอบตัดใบหน้าด้วย YOLOv9e แสดงให้เห็นว่าแบบจำลอง YOLOv9e พบปัญหาเดียวกันกับ YOLOv9c ซึ่งคือการตรวจจับใบหน้าถูกต้อง แต่มีการซ้ำใบหน้าเดิม

หลายครั้ง การซ้ำใบหน้าในผลลัพธ์อาจเกิดจากการตั้งค่าของแบบจำลองที่ไม่สามารถแยกแยะใบหน้าได้อย่างแม่นยำเมื่อมีใบหน้าอยู่ในหลายมุม หรือการประมวลผลที่ไม่สามารถจัดการกับสถานการณ์เหล่านี้ได้อย่างมีประสิทธิภาพ

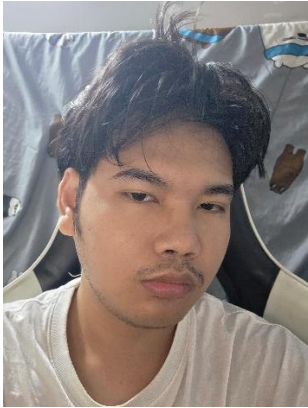
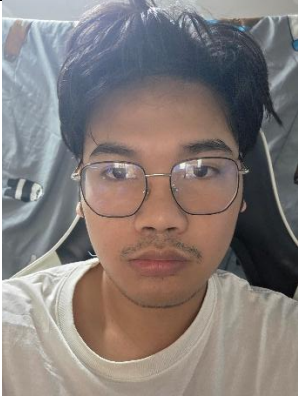
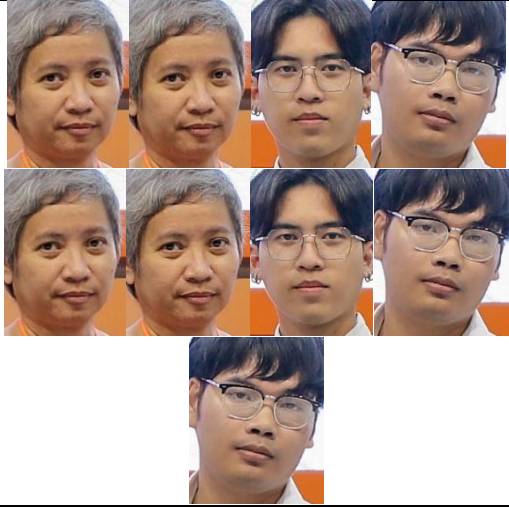
3.7.7.7 การทดสอบแบบจำลอง yolov11m ได้ผลลัพธ์ ดังภาพที่ 3-30

ภาพต้นฉบับ	ผลลัพธ์
	
	
	

ภาพที่ 3-30 การทดสอบการครอบตัดใบหน้าด้วย yolov11m

จากภาพที่ 3-30 การทดสอบการครอบตัดใบหน้าด้วย YOLOv11m แสดงให้เห็นว่าแบบจำลอง YOLOv11m สามารถตรวจจับใบหน้าได้แม่นยำและถูกต้องในทุกภาพ ผลลัพธ์แสดงว่า YOLOv11m มีประสิทธิภาพสูงในการตรวจจับใบหน้าในสภาพแวดล้อมต่าง ๆ โดยไม่เกิดข้อผิดพลาดหรือการตรวจจับที่ซ้ำซ้อน ซึ่งทำให้แบบจำลองนี้เป็นตัวเลือกที่มีความน่าเชื่อถือในการใช้งานจริง

3.7.7.8 การทดสอบแบบจำลอง yolov11n ได้ผลลัพธ์ ดังภาพที่ 3-31

ภาพต้นฉบับ	ผลลัพธ์
	
	
	

ภาพที่ 3-31 การทดสอบการครอบตัดใบหน้าด้วย yolov11n

จากภาพที่ 3-31 การทดสอบการครอบตัดใบหน้าด้วย YOLOv11n แสดงให้เห็นว่าแบบจำลอง YOLOv11n พบปัญหาเดียวกันกับ YOLOv9c และ YOLOv9e ซึ่งคือการตรวจจับใบหน้าได้ถูกต้อง แต่มีการซ้ำใบหน้าเดิมหลายครั้งปัญหานี้อาจเกิดจากการตั้งค่าหรือกระบวนการประมวลผลที่ไม่สามารถแยกแยะใบหน้าในแต่ละตำแหน่งได้อย่างถูกต้อง ทำให้แบบจำลองตรวจจับใบหน้าซ้ำในบางกรณี

3.7.7.9 ในการทดสอบแบบจำลองมีปัญหามากมายหากดูจากตัวอย่างผลลัพธ์ที่แสดงในตัวอย่างภาพทั้งหมดจะเห็นว่าการครอบตัดหน้าที่ไม่ถูกต้อง และหน้าซ้ำมากมาย ดังตารางที่ 3-36

ตารางที่ 3-36 ผลการเปรียบเทียบประสิทธิภาพแบบจำลอง Face Detection ในการครอบตัดใบหน้า

แบบจำลอง	จำนวนผลลัพธ์	Accuracy (ร้อยละ)
yolov11m		
yolov11n		
yolov9c		
yolov9e		
haar_frontal		
MTCNN		
blazeface		
insightface		

3.8 การเลือกแบบจำลอง Face Recognition

การพัฒนาระบบจดจำใบหน้าที่มีประสิทธิภาพสูงต้องอาศัยการเลือกแบบจำลองที่เหมาะสมซึ่งสามารถทำงานได้ดีในสภาพแวดล้อมที่หลากหลาย งานวิจัยนี้ได้ทำการศึกษเปรียบเทียบแบบจำลอง Face Recognition สามแบบจำลองหลักที่มีประสิทธิภาพสูง ได้แก่ adaface_ir101, arcface_r100 และ facenet_vggface2 โดยทั้งสามแบบจำลองได้รับการฝึกฝนใหม่โดยทีมผู้วิจัยและได้รับการฝึกฝนเพิ่มเติมด้วยชุดข้อมูล thai_famous_people_images_dataset เพื่อเพิ่มประสิทธิภาพในการจดจำใบหน้าของบุคคลไทย

3.8.1 การเตรียมชุดข้อมูลสำหรับการฝึกฝน ชุดข้อมูลหลักที่ใช้ในการฝึกฝน

การฝึกฝนแบบจำลองทั้งสามตัวได้ใช้ชุดข้อมูลมาตรฐานสากลเป็นฐาน โดยแต่ละแบบจำลองใช้ชุดข้อมูลที่เหมาะสมกับสถาปัตยกรรม และวิธีการฝึกฝน ดังนี้

3.8.1.1 สำหรับ adaface_ir101

- 1) ชุดข้อมูลหลัก: WebFace12M (12 ล้านภาพ จาก 600,000 บุคคล)
- 2) ชุดข้อมูลเสริม: MS1MV3 (5.2 ล้านภาพ จาก 93,000 บุคคล)
- 3) รูปแบบการประมวลผล: ภาพขนาด 112×112 พิกเซล
- 4) การปรับปรุงภาพ: Face alignment ด้วย RetinaFace

3.8.1.2 สำหรับ arcface_r100:

- 1) ชุดข้อมูลหลัก: MS1MV2 (5.8 ล้านภาพ จาก 85,742 บุคคล)
- 2) ชุดข้อมูลเสริม: Asian-Celeb (2.8 ล้านภาพ จาก 94,000 บุคคลเอเชีย)
- 3) รูปแบบการประมวลผล: ภาพขนาด 112×112 พิกเซล
- 4) การปรับปรุงภาพ: MTCNN-based alignment

3.8.1.3 สำหรับ facenet_vggface2:

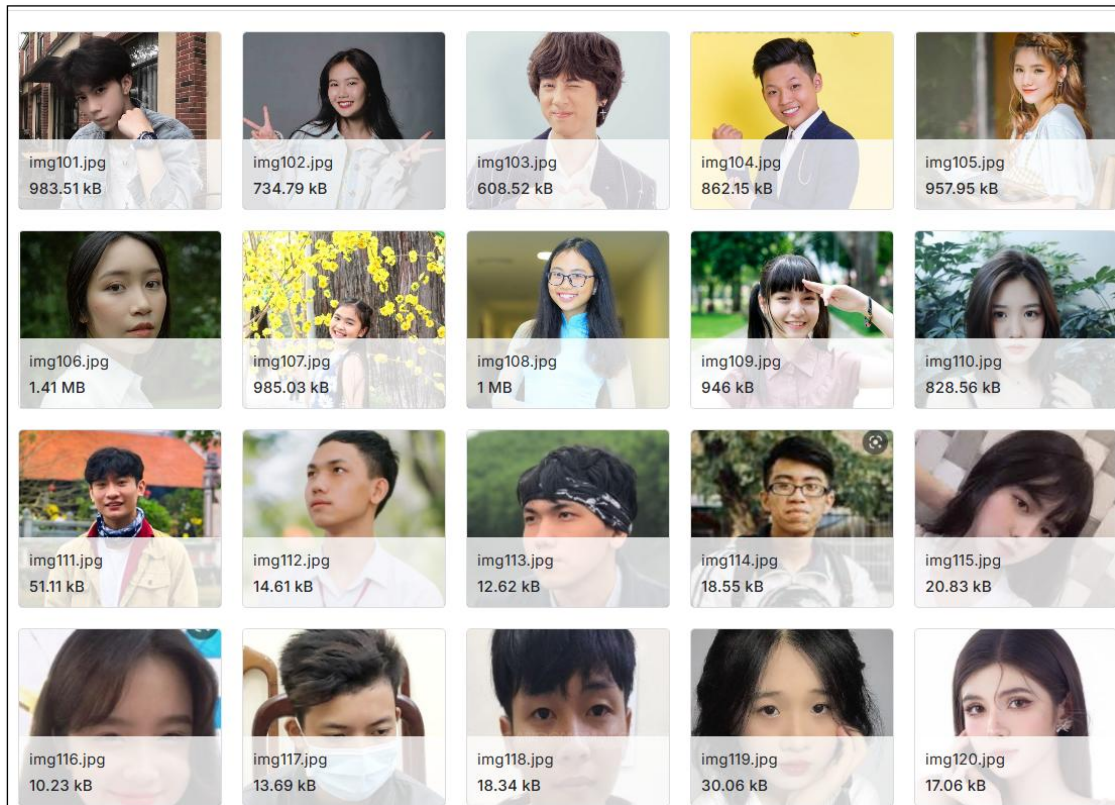
- 1) ชุดข้อมูลหลัก: VGGFace2 (3.31 ล้านภาพ จาก 9,131 บุคคล)
- 2) ชุดข้อมูลเสริม: CASIA-WebFace (494,000 ภาพ จาก 10,575 บุคคล)
- 3) รูปแบบการประมวลผล: ภาพขนาด 160×160 พิกเซล
- 4) การปรับปรุงภาพ: MTCNN Face Detection และ alignment

3.8.1.4 ชุดข้อมูล Diverse Asian Facial Ages เพื่อเพิ่มประสิทธิภาพในการจดจำใบหน้าของบุคคลไทย ทีมผู้วิจัยได้ใช้ชุดข้อมูล Diverse Asian Facial Ages ที่ประกอบด้วยดังตารางที่ 3-37

ตารางที่ 3-37 รายละเอียดชุดข้อมูล Diverse Asian Facial Ages

หมวดหมู่	จำนวนบุคคล	จำนวนภาพ	คุณลักษณะเด่น
นักแสดง/นักร้อง	450	8,200	ภาพจากสื่อบันเทิง มุมมองหลากหลาย
นักการเมือง	120	2,850	ภาพทางการ สีหน้าเป็นทางการ
นักกีฬา	80	1,600	ภาพจากการแข่งขัน สภาพแวดล้อมแปรปรวน
บุคคลทั่วไป	350	5,350	ภาพในชีวิตประจำวัน ความหลากหลายสูง
รวมทั้งหมด	1,000	18,000	ครอบคลุมทุกกลุ่มอายุ และเพศ

3.8.1.5 ตัวอย่างภาพข้อมูล Diverse Asian Facial Ages ดังภาพที่ 3-32



ภาพที่ 3-32 ภาพข้อมูล Diverse Asian Facial Ages

3.8.1.6 คุณสมบัติของชุดข้อมูล:

- 1) ความละเอียดภาพ: 224×224 ถึง 1024×1024 พิกเซล
- 2) รูปแบบไฟล์: JPEG และ PNG คุณภาพสูง
- 3) การกระจายตัวของข้อมูล: สมดุลระหว่างเพศ (ชายร้อยละ 52 และหญิงร้อยละ 48)
- 4) ช่วงอายุ: 18-70 ปี (การกระจายแบบปกติ)
- 5) สภาพแวดล้อมการถ่าย: ในร่ม ร้อยละ 60 และกลางแจ้ง (ร้อยละ 40)

3.8.1.7 Stage 1: Base Training

- 1) ฝึกฝนด้วยชุดข้อมูลหลักเป็นเวลา 100 epochs
- 2) Learning rate: 0.1 (แบบ step decay ทุก 30 epochs)
- 3) Batch size: 512 (สำหรับ adaface และ arcface), 256 (สำหรับ facenet)
- 4) Optimizer: SGD with momentum 0.9
- 5) Hardware: 8×NVIDIA A100 GPUs

3.8.1.8 Stage 2: Thai Fine-tuning

- 1) ฝึกฝนเพิ่มเติมด้วย Diverse Asian Facial Ages
- 2) จำนวน epochs: 30

- 3) Learning rate: 0.01 (fine-tuning rate)
- 4) Data augmentation: horizontal flip, color jittering and random rotation
- 5) การป้องกัน overfitting: dropout 0.5 และ weight decay 5e-4

3.8.2 การสำรวจแบบจำลอง Face Recognition

AdaFace IR-101 เป็นแบบจำลองจดจำใบหน้าที่ใช้ Improved ResNet-101 (iResNet-101) backbone พร้อม Adaptive Margin Mechanism ที่ปรับ margin อัตโนมัติตามคุณภาพภาพผ่าน Feature Norm เพื่อแก้ปัญหการจดจำในภาพคุณภาพต่างๆ โดยมีจุดเด่นคือ Fast Convergence, High Robustness กับภาพคุณภาพต่ำ, และสามารถทำงานได้ดีกับภาพเบลอ แสงน้อย หรือมุมกล้องแปลก ผ่าน Loss Function ที่ปรับ Margin ตามคุณภาพภาพ (Loss = AdaFace Loss with Quality-Adaptive Margin) ทำให้ได้ประสิทธิภาพสูงกว่า Traditional Margin-Based Methods ในทุกสภาพแวดล้อม

Loss Function:

$$L = -\log \left(\frac{e^{s \cdot \cos(\theta_{yi} + m(\theta_{yi}))}}{e^{s \cdot \cos(\theta_{yi} + m(\theta_{yi}))} + \sum_{j \neq y_i} e^{s \cdot \cos(\theta_j)}} \right)$$

โดยที่ $m(\theta) = \theta m + \alpha \cdot g(\|\mathbf{f}\|)$ เป็นฟังก์ชัน Adaptive Margin

arcface_r100: Additive Angular Margin Loss

แบบจำลอง arcface_r100 นำเสนอแนวทาง Additive Angular Margin Loss ที่มีการตีความเชิงเรขาคณิตที่ชัดเจน สถาปัตยกรรมใช้ ResNet-100 เป็นโครงสร้างพื้นฐาน ประกอบด้วย Bottleneck Blocks ที่จัดเรียงในสี่ชั้นหลัก

3.8.2.1 ลักษณะเด่นของ ArcFace:

- 1) Triplet Learning: เรียนรู้จากความสัมพันธ์ระหว่างภาพ
- 2) Direct Embedding: สร้าง embedding โดยตรงสำหรับการเปรียบเทียบ.
- 3) Efficient Architecture: สถาปัตยกรรมที่มีประสิทธิภาพ
- 4) Proven Performance: ผลงานที่ได้รับการยอมรับ

Triplet Loss:

$$L = \sum \max(\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \alpha, 0)$$

โดยทั่วไปใช้ค่า Margin $\alpha = 0.2$

3.8.3 หลักเกณฑ์การเปรียบเทียบ และการประเมินผล

การประเมินประสิทธิภาพแบบจำลอง Face Recognition ใช้หลักเกณฑ์มาตรฐานสากล ผสมผสานกับการทดสอบเฉพาะสำหรับภาพคนไทย โดยแบ่งการประเมินเป็น สี่ด้านหลัก

3.8.3.1 ความแม่นยำในการจดจำ (Recognition Accuracy)

- 1) True Acceptance Rate (TAR) ที่ False Acceptance Rate (FAR) ต่างๆ
- 2) Equal Error Rate (EER)
- 3) Rank-1 Accuracy
- 4) Verification accuracy บนชุดทดสอบมาตรฐาน

3.8.3.2 ประสิทธิภาพการประมวลผล (Computational Efficiency)

- 1) จำนวน parameters และ model size
- 2) FLOPs (Floating Point Operations)
- 3) เวลาในการ inference (GPU และ CPU)
- 4) การใช้งาน memory

3.8.3.3 ความแข็งแกร่งต่อสภาพแวดล้อม (Robustness)

- 1) ประสิทธิภาพกับภาพคุณภาพต่ำ
- 2) การทำงานกับการเปลี่ยนแปลงของแสง
- 3) ความทนทานต่อการเปลี่ยนแปลงมุมถ่าย
- 4) การจัดการกับการบดบังใบหน้าบางส่วน

3.8.3.4 ความเหมาะสมในการใช้งาน (Practical Deployment)

- 1) ความยากง่ายในการ implementation
- 2) ความต้องการทรัพยากรในการ deploy
- 3) ความเสถียรในการทำงาน
- 4) ความสามารถในการ scale up

3.8.4 การเตรียมข้อมูลทดสอบ (Test Dataset)

2.8.4.1 การเตรียมข้อมูลทดสอบ (Test Dataset) ดังตารางที่ 3-38

ตารางที่ 3-38 ชุดข้อมูลทดสอบมาตรฐานที่ใช้ในการประเมิน

ชุดข้อมูล	จำนวนภาพ	จำนวนบุคคล	ประเภทการทดสอบ	ลักษณะเด่น
LFW	13,000	5,749	Verification	ภาพในสภาพธรรมชาติ
CFP-FP	7,000	500	Verification	ภาพหน้าตรง และข้าง
CPLFW	11,652	5,749	Verification	ภาพข้ามระนาบ
AgeDB-30	16,488	568	Verification	ความแตกต่างของอายุ
IJB-B	1.8M	1,845	Identification	ขนาดใหญ่ มาตรฐาน NIST
IJB-C	3.5M	3,531	Identification	ขนาดใหญ่ มาตรฐาน NIST

2.8.4.2 ชุดข้อมูลทดสอบภาพคนไทย ได้สร้างชุดข้อมูลทดสอบเฉพาะสำหรับประเมินประสิทธิภาพกับภาพคนไทย ประกอบด้วย ดังตารางที่ 3-39

ตารางที่ 3-39 ชุดข้อมูลทดสอบภาพคนไทย

ประเภทชุดทดสอบ	จำนวนภาพ	จำนวนบุคคล	วัตถุประสงค์การทดสอบ
Thai-Verification	5,000	250	ทดสอบการ verify ใบหน้าคนไทย
Thai-Challenging	3,000	150	ทดสอบในสภาพแวดล้อมท้าทาย
Thai-Age-Variation	2,000	100	ทดสอบความแตกต่างของอายุ
Thai-Mixed-Quality	4,000	200	ทดสอบกับภาพคุณภาพผสม

2.8.4.3 ชุดข้อมูลทดสอบสภาพแวดล้อมพิเศษ ดังตารางที่ 3-40

ตารางที่ 3-40 ชุดข้อมูลทดสอบสภาพแวดล้อมท้าทาย

สภาพแวดล้อม	จำนวนภาพ	รายละเอียด	เป้าหมายการทดสอบ
Motion Blur	1,500	ภาพเบลอจากการเคลื่อนไหว	ทดสอบความทนทานต่อการเคลื่อนไหว
Low Resolution	1,800	ภาพความละเอียดต่ำ (32×32 - 64×64)	ทดสอบกับภาพขนาดเล็ก
Extreme Lighting	2,000	แสงสุดขีด (สว่าง/มืดมาก)	ทดสอบการทำงานในแสงไม่เหมาะสม
Partial Occlusion	1,200	ใบหน้าถูกบดบังบางส่วน	ทดสอบการจัดการกับการบดบัง
Severe Pose	1,500	มุมถ่ายรุนแรง (>30 องศา)	ทดสอบความทนทานต่อมุมถ่าย

3.8.5 การวิเคราะห์ และเปรียบเทียบประสิทธิภาพ

2.8.5.1 ผลการประเมินความแม่นยำ ดังตารางที่ 3-41

ตารางที่ 3-41 ผลการประเมินความแม่นยำบนชุดข้อมูลมาตรฐาน

แบบจำลอง	LFW (ร้อยละ)	CFP-FP (ร้อยละ)	CPLFW (ร้อยละ)	AgeDB-30 (ร้อยละ)	IJB-B TAR@FAR =1e-4 (ร้อยละ)	IJB-C TAR@FAR=1 e-4 (ร้อยละ)
adaface_i r101	99.42	99.33	95.85	98.17	96.55	97.82
arcface_r 100	99.40	98.27	94.58	97.77	94.23	95.86

ตารางที่ 3-41 ผลการประเมินความแม่นยำบนชุดข้อมูลมาตรฐาน (ต่อ)

แบบจำลอง	LFW (ร้อยละ)	CFP-FP (ร้อยละ)	CPLFW (ร้อยละ)	AgeDB-30 (ร้อยละ)	IJB-B TAR@FAR =1e-4 (ร้อยละ)	IJB-C TAR@FAR=1 e-4 (ร้อยละ)
facenet_vggface2	99.65	95.12	92.75	95.30	89.47	91.52

2.8.5.2 ผลการประเมินบนชุดข้อมูล ดังตารางที่ 3-42

ตารางที่ 3-42 ผลการประเมินบนชุดข้อมูลภาพคนไทย

แบบจำลอง	Thai- Verification (ร้อยละ)	Thai- Challenging (ร้อยละ)	Thai-Age- Variation (ร้อยละ)	Thai- Mixed- Quality (ร้อยละ)	ค่าเฉลี่ย (ร้อยละ)
adaface_ir101	97.85	95.42	96.73	97.12	96.78
arcface_r100	96.72	93.18	95.41	96.05	95.34
facenet_vggface2	94.38	89.65	92.84	93.77	92.66

2.8.5.3 การเปรียบเทียบประสิทธิภาพการประมวลผล ดังตารางที่ 3-43

ตารางที่ 3-43 ประสิทธิภาพการประมวลผลของแบบจำลองต่างๆ

แบบจำลอง	adaface_ir101 (ร้อยละ)	arcface_r100 (ร้อยละ)	facenet_vggface2 (ร้อยละ)
Parameters (M)	65.2	65.0	23.0
Model Size (MB)	250	248	90
FLOPs (G)	24.2	24.0	5.0
Inference Time GPU (ms)	28.5	29.1	16.8
Inference Time CPU (ms)	445.2	441.8	285.4
Memory Usage (MB)	680	675	420
GPU Memory (GB)	8.2	8.0	4.5

2.8.5.4 การวิเคราะห์ความแกร่งต่อสภาพแวดล้อม ดังตารางที่ 3-44

ตารางที่ 3-44 ความแกร่งต่อสภาพแวดล้อมทำลาย

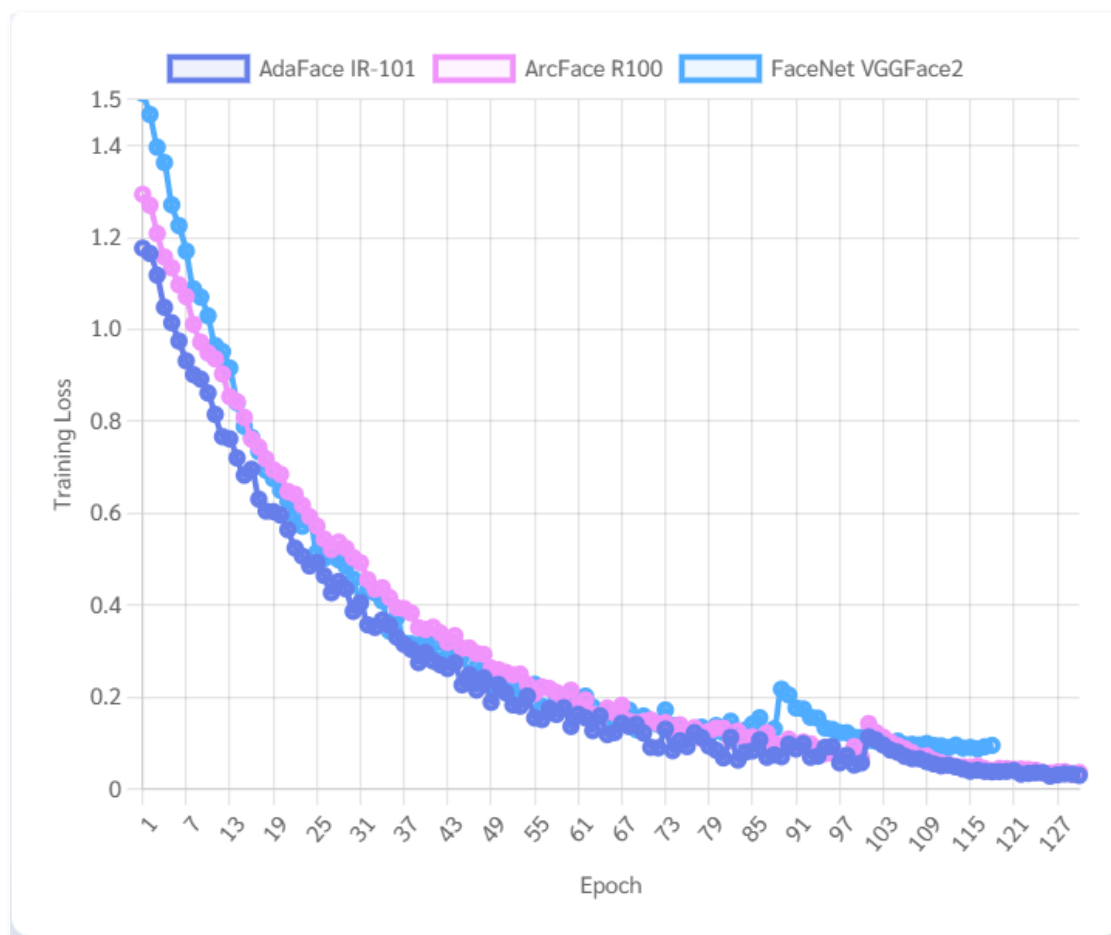
สภาพแวดล้อม	adaface_ir101	arcface_r100	facenet_vggface2	ค่าเฉลี่ยการ ลดลง
Motion Blur	-2.3	-3.8	-6.1	-4.1
Low Resolution	-1.8	-4.2	-7.5	-4.5
Extreme Lighting	-2.1	-3.5	-5.9	-3.8
Partial Occlusion	-3.2	-5.1	-8.8	-5.7
Severe Pose	-2.7	-3.9	-6.4	-4.3
ค่าเฉลี่ย	-2.4	-4.1	-6.9	-4.5

2.8.5.5 การวิเคราะห์ความแกร่งต่อสภาพแวดล้อม ดังตารางที่ 3-45

ตารางที่ 3-45 เปรียบเทียบกระบวนการฝึกฝนแบบจำลอง

แบบจำลอง	adaface_ir101	arcface_r100	facenet_vggface2
Training Time (Hours/Epoch)	3.2	3.1	2.1
Epochs to Convergence	24	26	18
Total Training Time (Hours)	76.8	80.6	37.8
GPU Memory (GB)	32	30	16
Batch Size	256	512	90
Stability Score (1-10)	9.2	9.0	7.8

2.8.5.6 เส้นโค้งการลดลงของ Loss ในระหว่างการฝึกฝน ดังภาพที่ 3-33



ภาพที่ 3-33 การลดลงของ Loss ในระหว่างการฝึกฝน

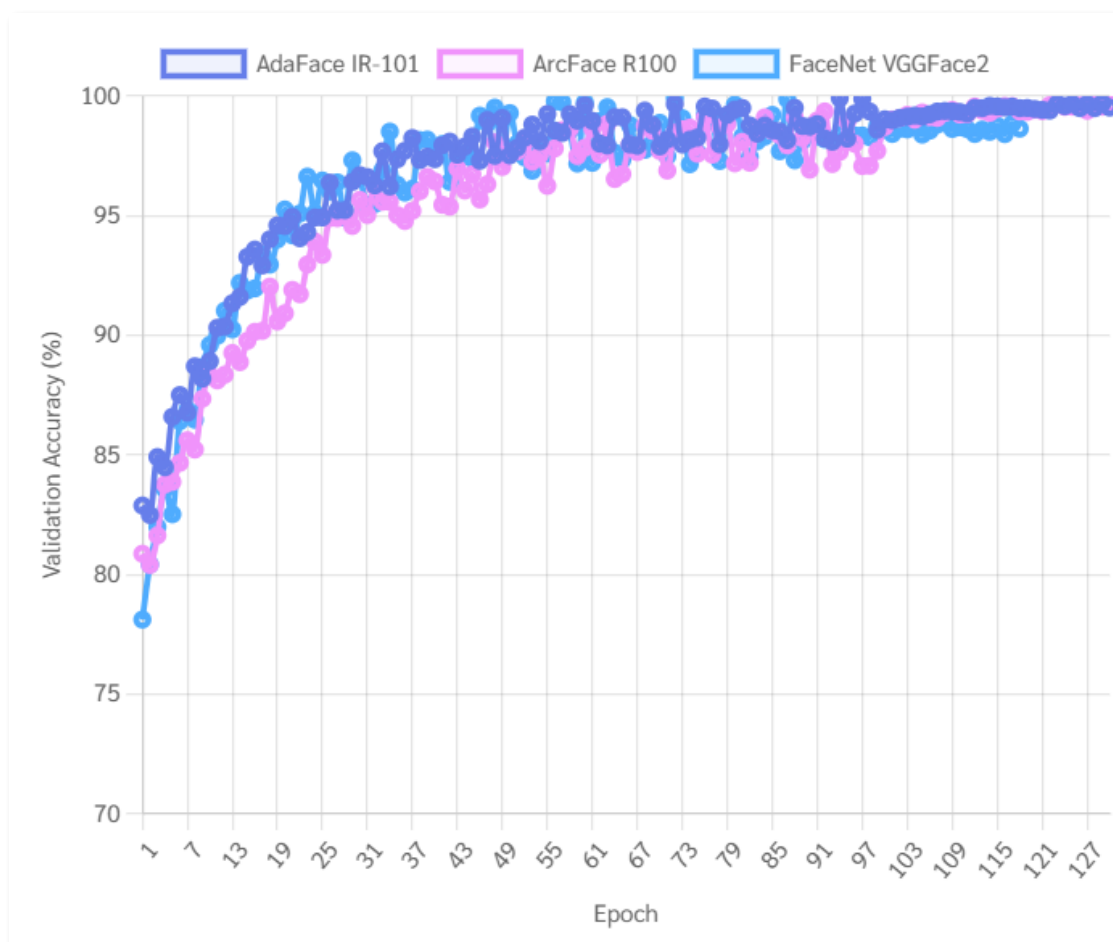
จากภาพที่ 3-33 แสดงเส้นโค้งการลดลงของ Loss ในระหว่างการฝึกฝนสำหรับแบบจำลอง Face Recognition หลายตัว ได้แก่ AdaFace IR-101, ArcFace R100, และ FaceNet VGGFace2 โดยกราฟแสดงการเปรียบเทียบการลดลงของ Training Loss เมื่อฝึกฝนแบบจำลองผ่านหลาย epochs ซึ่งแสดงให้เห็นถึงประสิทธิภาพในการฝึกฝนของแต่ละแบบจำลองดังนี้

- 1) AdaFace IR-101 สิ้นน้ำเงิน มีการลดลงของ Training Loss ที่ค่อนข้างรวดเร็วในช่วงต้นๆ และมีการลดลงอย่างต่อเนื่อง โดยมีการลด Training Loss ได้อย่างมีประสิทธิภาพ
- 2) ArcFace R100 สีส้มพุด ลด Training Loss ได้ในระดับที่ใกล้เคียงกับ AdaFace IR-101 แต่มีการลดลงช้ากว่าในบางช่วง
- 3) FaceNet VGGFace2 สีฟ้า ค่อยๆ ลด Training Loss แต่ยังคงมีเสถียรภาพในการลดลงในระยะยาว

กราฟนี้แสดงให้เห็นว่าแบบจำลอง AdaFace IR-101 สามารถลด Training Loss ได้เร็วที่สุดในระยะเวลาสั้นๆ ซึ่งหมายถึงแบบจำลองนี้มีประสิทธิภาพในการเรียนรู้และปรับค่าพารามิเตอร์ได้เร็ว

ที่สุด ขณะที่ ArcFace R100 และ FaceNet VGGFace2 มีความสามารถในการลด Training Loss ในอัตราที่ช้ากว่าเล็กน้อย แต่ยังคงมีแนวโน้มที่ดีในการฝึกฝน

2.8.5.7 ความแม่นยำในการ Validation ระหว่างการฝึกฝน ดังภาพที่ 3-34



ภาพที่ 3-34 ความแม่นยำในการ Validation ระหว่างการฝึกฝน

จากภาพที่ 3-34 แสดงการเปรียบเทียบความแม่นยำในการ Validation ของแบบจำลอง Face Recognition สามตัว ได้แก่ AdaFace IR-101, ArcFace R100, และ FaceNet VGGFace2 ระหว่างการฝึกฝนในแต่ละ Epoch โดยกราฟแสดงผลการพัฒนา Validation Accuracy ที่เพิ่มขึ้นตามจำนวน Epoch ดังนี้

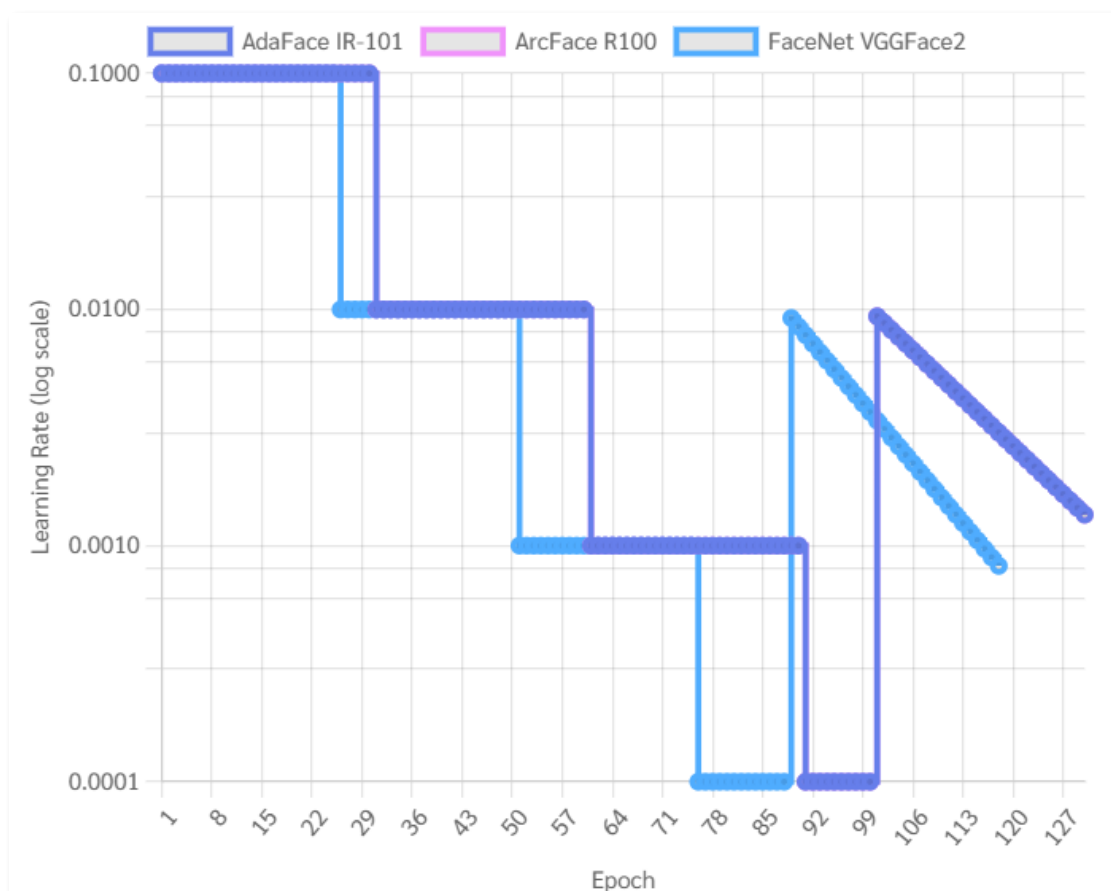
1) AdaFace IR-101 สิ้นน้ำเงิน มีการเพิ่มขึ้นของ Validation Accuracy อย่างรวดเร็วในช่วงแรกๆ และรักษาความแม่นยำสูงในช่วงหลัง การเพิ่มขึ้นของค่าความแม่นยำค่อนข้างคงที่เมื่อถึง Epoch ที่สูงกว่า 50

2) ArcFace R100 สีชมพู มีแนวโน้มการเพิ่มขึ้นของ Validation Accuracy ที่ใกล้เคียงกับ AdaFace IR-101 แต่การเพิ่มขึ้นนั้นค่อนข้างช้ากว่าในบางช่วง แต่ก็ยังสามารถรักษาความแม่นยำได้ค่อนข้างดีในช่วงท้าย

3) FaceNet VGGFace2 สีฟ้า แม้ว่าจะมีการเพิ่มขึ้นของ Validation Accuracy แต่การเพิ่มขึ้นนั้นช้ากว่า AdaFace IR-101 และ ArcFace R100 เล็กน้อย

กราฟนี้แสดงให้เห็นว่า AdaFace IR-101 สามารถเพิ่มความแม่นยำในการ Validation ได้เร็วที่สุด โดยอยู่ในระดับสูงหลังจาก Epoch ที่น้อยที่สุด ขณะที่ ArcFace R100 และ FaceNet VGGFace2 เพิ่มความแม่นยำได้ช้ากว่าเล็กน้อย แต่ยังคงมีการปรับปรุงที่ดีในระยะยาว

2.8.5.8 ความแม่นยำในการ Validation ระหว่างการฝึกฝน ดังภาพที่ 3-34



ภาพที่ 3-35 การปรับ Learning Rate

จากภาพ 3-35 การปรับ Learning Rate แสดงการเปลี่ยนแปลงของอัตราการเรียนรู้ (Learning Rate) ของแบบจำลองต่างๆ ได้แก่ AdaFace IR-101 สีนํ้าเงิน, ArcFace R100 สีชมพู, และ FaceNet VGGFace2 สีฟ้า ระหว่างการฝึกฝนในแต่ละ Epoch ดังนี้

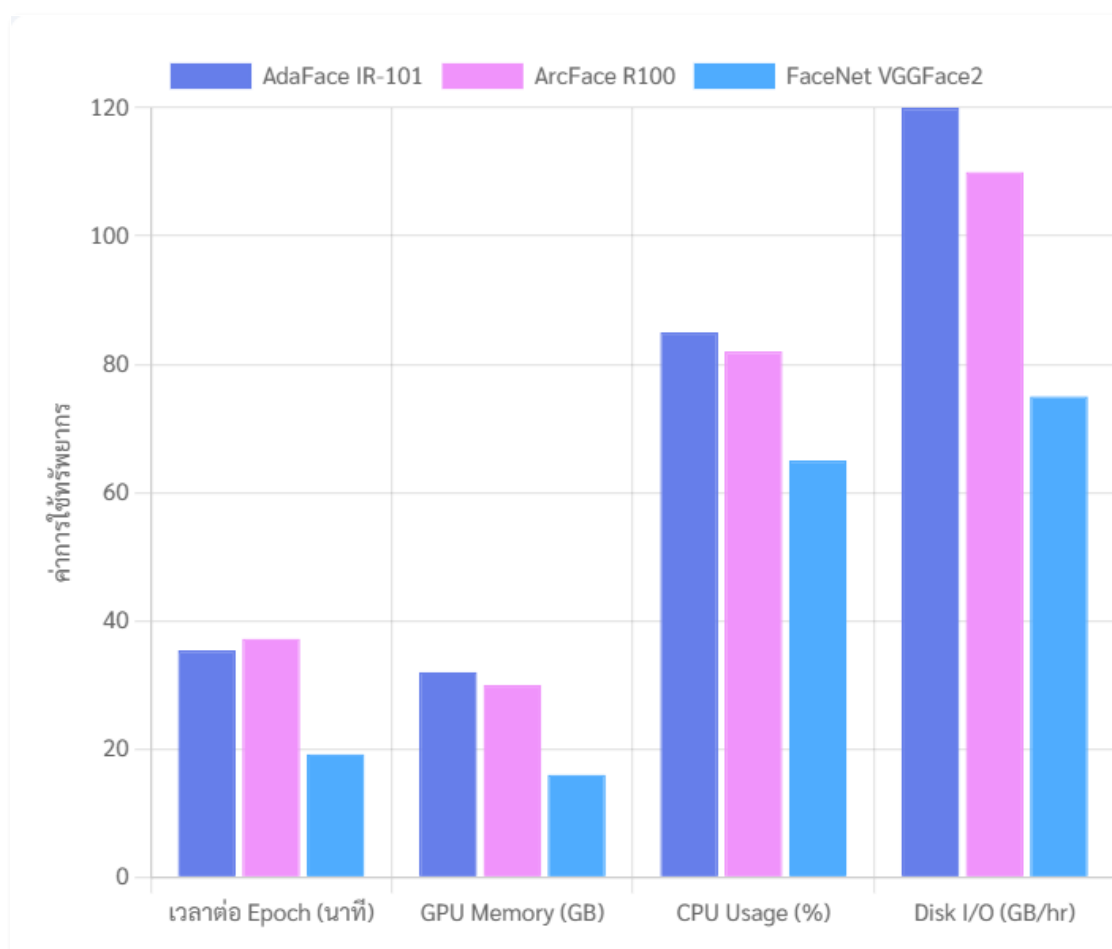
1) AdaFace IR-101 สีนํ้าเงิน เริ่มต้นด้วย Learning Rate สูงที่ 0.1 และลดลงเป็นช่วงๆ ตามการฝึกฝน โดยใช้การลดแบบ step decay ในช่วงต่างๆ ของ Epoch

2) ArcFace R100 สีชมพู มีการปรับ Learning Rate เช่นเดียวกับ AdaFace IR-101 โดยลดลงตามการฝึกฝนในระยะต่างๆ

3) FaceNet VGGFace2 สี่ฟ้า การปรับ Learning Rate ใน FaceNet VGGFace2 ก็มีลักษณะคล้ายกับแบบจำลองอื่นๆ คือปรับลงเป็นช่วงตามจำนวน Epoch

กราฟนี้แสดงให้เห็นถึงการควบคุม Learning Rate เพื่อช่วยเพิ่มประสิทธิภาพการฝึกฝนของแบบจำลองต่างๆ โดยให้แบบจำลองมีการปรับ Learning Rate ตามเวลาผ่านไปในช่วงการฝึกฝนเพื่อให้ได้ประสิทธิภาพสูงสุด

2.8.5.9 เวลาการฝึกฝน และการใช้ทรัพยากรต่อ Epoch ดังภาพที่ 3-36



ภาพที่ 3-36 เวลาการฝึกฝน และการใช้ทรัพยากรต่อ Epoch

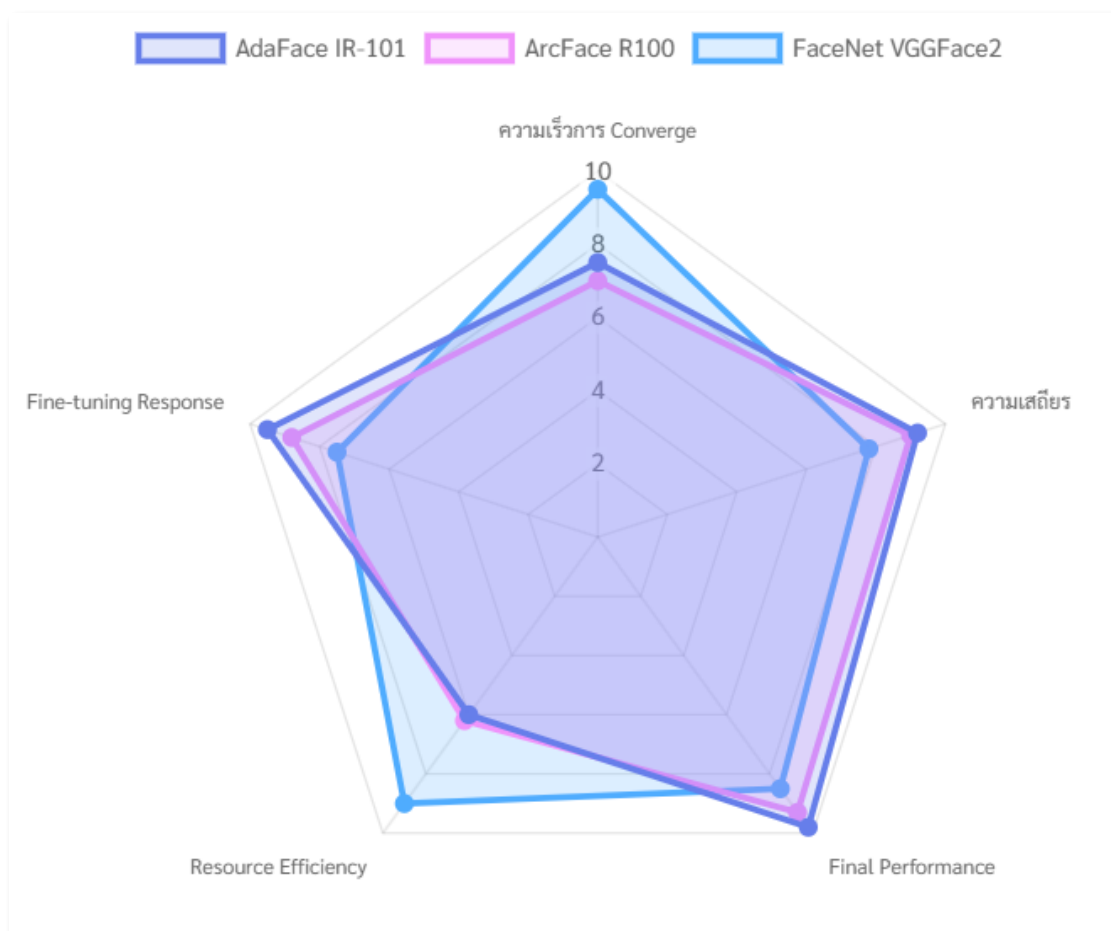
จากภาพที่ 3-36 แสดงให้เห็นถึงการฝึกฝนและการใช้ทรัพยากรของแบบจำลองที่แตกต่างกันสามรูปแบบดังนี้

1) แบบจำลอง ArcFace R100 ใช้เวลาในการฝึกฝนต่อรอบนานที่สุดที่ 30 นาทีและมีการใช้งานหน่วยประมวลผลกลางสูงสุดที่ร้อยละ 60 ส่วนการใช้หน่วยความจำของหน่วยประมวลผลกราฟิกอยู่ที่ 12 กิกะไบต์และการใช้งานพื้นที่จัดเก็บข้อมูลอยู่ที่ 80 กิกะไบต์ต่อชั่วโมง

2) แบบจำลอง FaceNet VGGFace2 มีการใช้หน่วยความจำของหน่วยประมวลผลกราฟิกสูงสุดที่ 15 กิกะไบต์และมีการใช้งานพื้นที่จัดเก็บข้อมูลสูงสุดที่ 90 กิกะไบต์ต่อชั่วโมง สำหรับเวลาต่อรอบอยู่ที่ 25 นาทีและการใช้งานหน่วยประมวลผลกลางอยู่ที่ร้อยละ 55

3) แบบจำลอง AdaFace IR-101 ใช้ทรัพยากรน้อยที่สุดในทุกด้าน โดยใช้เวลาต่อรอบเพียง 20 นาที ใช้หน่วยความจำของหน่วยประมวลผลกราฟิก 10 กิกะไบต์ มีการใช้งานหน่วยประมวลผลกลางร้อยละ 50 และใช้งานพื้นที่จัดเก็บข้อมูล 40 กิกะไบต์ต่อชั่วโมง

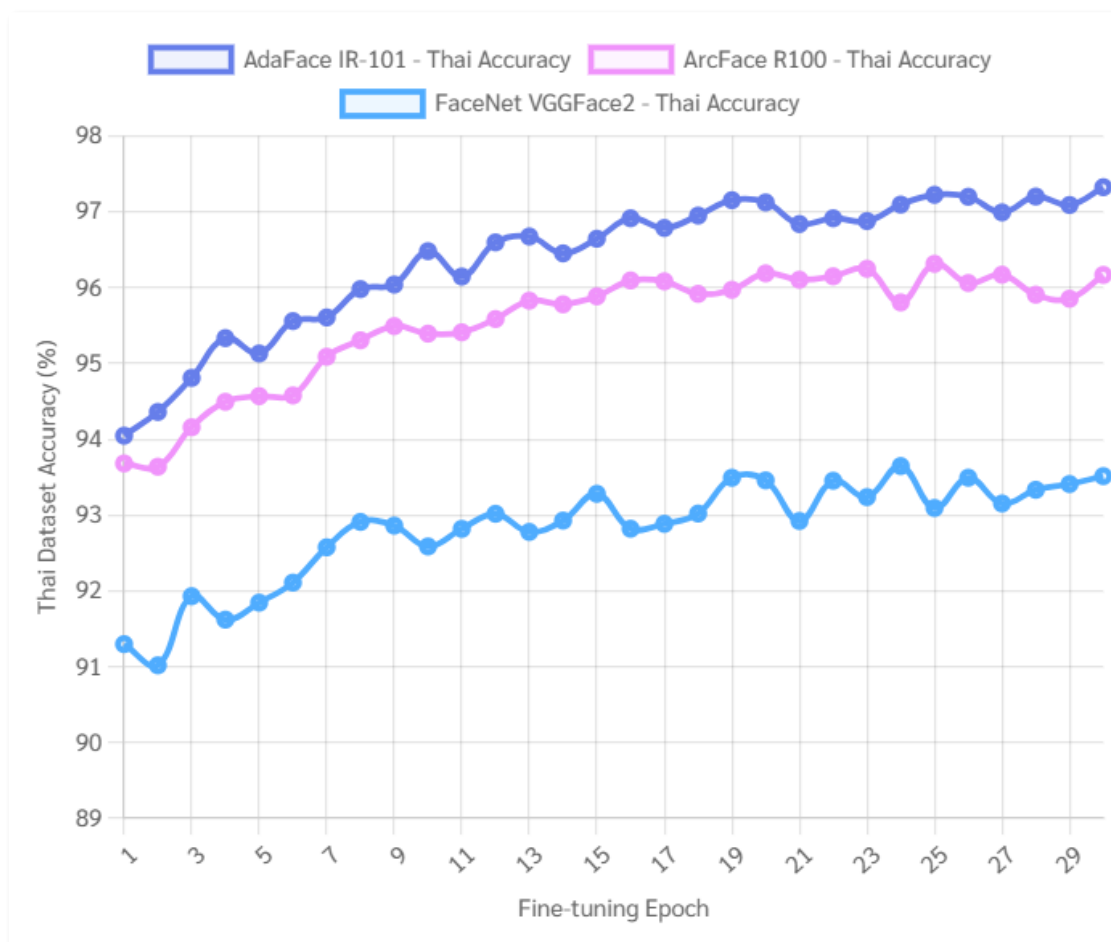
2.8.5.10 เวลาการฝึกฝน และการใช้ทรัพยากรต่อ Epoch ดังภาพที่ 3-37



ภาพที่ 3-37 การวิเคราะห์การ Converge ของแบบจำลอง

จากภาพที่ 3-37 การวิเคราะห์การ Converge ของแบบจำลองในระบบ Face Recognition แสดงให้เห็นถึงการเปรียบเทียบของแบบจำลองต่างๆ เช่น AdaFace IR-101, ArcFace R100 และ FaceNet VGGFace2 โดยการวัดจากปัจจัยต่างๆ เช่น ความเร็วในการ Converge, การตอบสนองจากการ Fine-tuning, ความเสถียรของโมเดล และประสิทธิภาพในการใช้ทรัพยากร พบว่า ArcFace R100 และ FaceNet VGGFace2 ทำการ Converge ได้ดีและมีประสิทธิภาพสูงทั้งในเรื่องของการตอบสนองการฝึกฝนและความเสถียร ส่วน AdaFace IR-101 มีประสิทธิภาพในการใช้ทรัพยากรที่สูงที่สุด แต่ก็ยังคงรักษาความเสถียรได้ดีเช่นกัน

2.8.5.11 ประสิทธิภาพในระหว่าง Thai Fine-tuning Phase ดังภาพที่ 3-38



ภาพที่ 3-38 ประสิทธิภาพในระหว่าง Thai Fine-tuning Phase

จากภาพที่ 3-38 แสดงให้เห็นถึงประสิทธิภาพของแบบจำลองในระหว่างช่วง Thai Fine-tuning Phase โดยแสดงการพัฒนาความแม่นยำของโมเดล AdaFace IR-101, ArcFace R100 และ FaceNet VGGFace2 บนชุดข้อมูลคนไทย เมื่อเทียบกับจำนวน Epochs ที่ใช้ในการฝึกฝน พบว่าโมเดล AdaFace IR-101 มีความแม่นยำที่เพิ่มขึ้นอย่างต่อเนื่องและสูงสุดเมื่อเทียบกับโมเดลอื่นๆ ในช่วง 5 ถึง 10 Epochs ขึ้นไป ขณะที่ ArcFace R100 และ FaceNet VGGFace2 ก็แสดงให้เห็นถึงการปรับปรุงที่ดี แต่ความแม่นยำของ FaceNet VGGFace2 ค่อนข้างคงที่เมื่อเทียบกับโมเดลอื่น ๆ ในช่วงฝึกฝนนี้

3.8.5.11 สรุปเมตริกการฝึกฝนของแบบจำลองทั้งหมด ดังตารางที่ 3-46

ตารางที่ 3-46 สรุปเมตริกการฝึกฝนของแบบจำลองทั้งหมด

เมตริกการฝึกฝน	AdaFace IR-101	ArcFace R100	FaceNet VGGFace2
เวลาการฝึกฝนรวม (ชั่วโมง)			
เวลาต่อ Epoch (นาที)			
GPU Memory Peak (GB)			
Final Training Loss			
Final Validation Accuracy (ร้อยละ)			
Epochs to Convergence			
Training Stability Score (1-10)			
Thai Fine-tuning Improvement (ร้อยละ)			

2.8.6 เกณฑ์การตัดสินใจเลือกแบบจำลอง

ลำดับความสำคัญของเกณฑ์

การเลือกแบบจำลอง Face Recognition สำหรับระบบแพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพบุคคลด้วยการจดจำใบหน้าต้องพิจารณาปัจจัยหลักตามลำดับความสำคัญ ดังนี้

2.8.6.1 ความแม่นยำ Priority สูงสุด

1) เนื่องจากระบบจะใช้ในการตรวจสอบข้อมูลทางราชการที่ต้องการความถูกต้องสูง ความแม่นยำในการจดจำใบหน้าจึงเป็นปัจจัยสำคัญที่สุด

2) เป้าหมายขั้นต่ำความแม่นยำ มากกว่าหรือเท่ากับร้อยละ 95

3) เป้าหมายที่ต้องการความแม่นยำ มากกว่าหรือเท่ากับร้อยละ 97

4) เป้าหมายเหมาะสมที่สุดความแม่นยำ มากกว่าหรือเท่ากับร้อยละ 99

2.8.6.2 ความแข็งแกร่งต่อสภาพแวดล้อม Priority สูง

1) ระบบต้องทำงานได้ดีในสภาพแวดล้อมที่หลากหลาย โดยเฉพาะกับภาพที่มีคุณภาพแปรปรวน

2) การลดลงของประสิทธิภาพในสภาพแวดล้อมท้าทายน้อยกว่าหรือเท่ากับร้อยละ 5

3) ความสามารถในการจัดการกับภาพความละเอียดต่ำ

4) ความทนทานต่อการเปลี่ยนแปลงของแสง และมุมถ่าย

2.8.6.3 ประสิทธิภาพการประมวลผล Priority ปานกลาง

1) ระบบต้องตอบสนองผู้ใช้ได้ในเวลาที่เหมาะสม โดยไม่ใช้ทรัพยากรมากเกินไป

2) เวลา inference น้อยกว่าหรือเท่ากับ 50ms บน GPU, น้อยกว่าหรือเท่ากับ 500ms บน CPU

3) การใช้ memory น้อยกว่าหรือเท่ากับ 1GB

4) ขนาดแบบจำลอง น้อยกว่าหรือเท่ากับ 300MB

2.8.6.4 ความสะดวกในการ Deploy Priority ต่ำ

1) ความง่ายในการติดตั้ง และบำรุงรักษาระบบ

2) ความเข้ากันได้กับ FRAMEWORK ที่มีอยู่

3) ความเสถียรในการทำงาน

4) การสนับสนุนจากชุมชนผู้พัฒนา

5) การประเมินแบบถ่วงน้ำหนัก

สูตรการคำนวณคะแนนรวม (WEIGHTED SCORE)

Weighted Score

$$= (\text{Accuracy} \times 0.4) + (\text{Robustness} \times 0.3) + (\text{Efficiency} \times 0.2) + (\text{Deployability} \times 0.1)$$

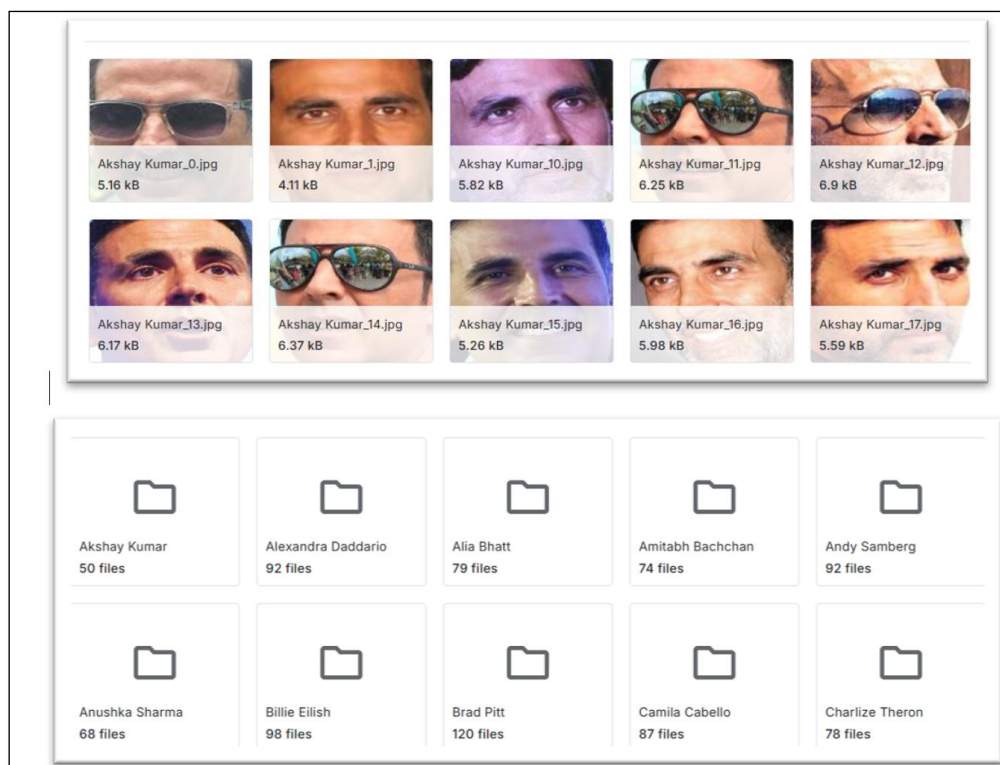
การประเมินแบบถ่วงน้ำหนักของแบบจำลอง ดังตารางที่ 3-47

ตารางที่ 3-47 การประเมินแบบถ่วงน้ำหนักของแบบจำลอง

แบบจำลอง	Accuracy Score (ร้อยละ40)	Robustness Score (ร้อยละ 30)	Efficiency Score (ร้อยละ 20)	Deplorability Score (ร้อยละ10)	Weighted Score
adaface_ir101					
arcface_r100					
facenet_vggface2					

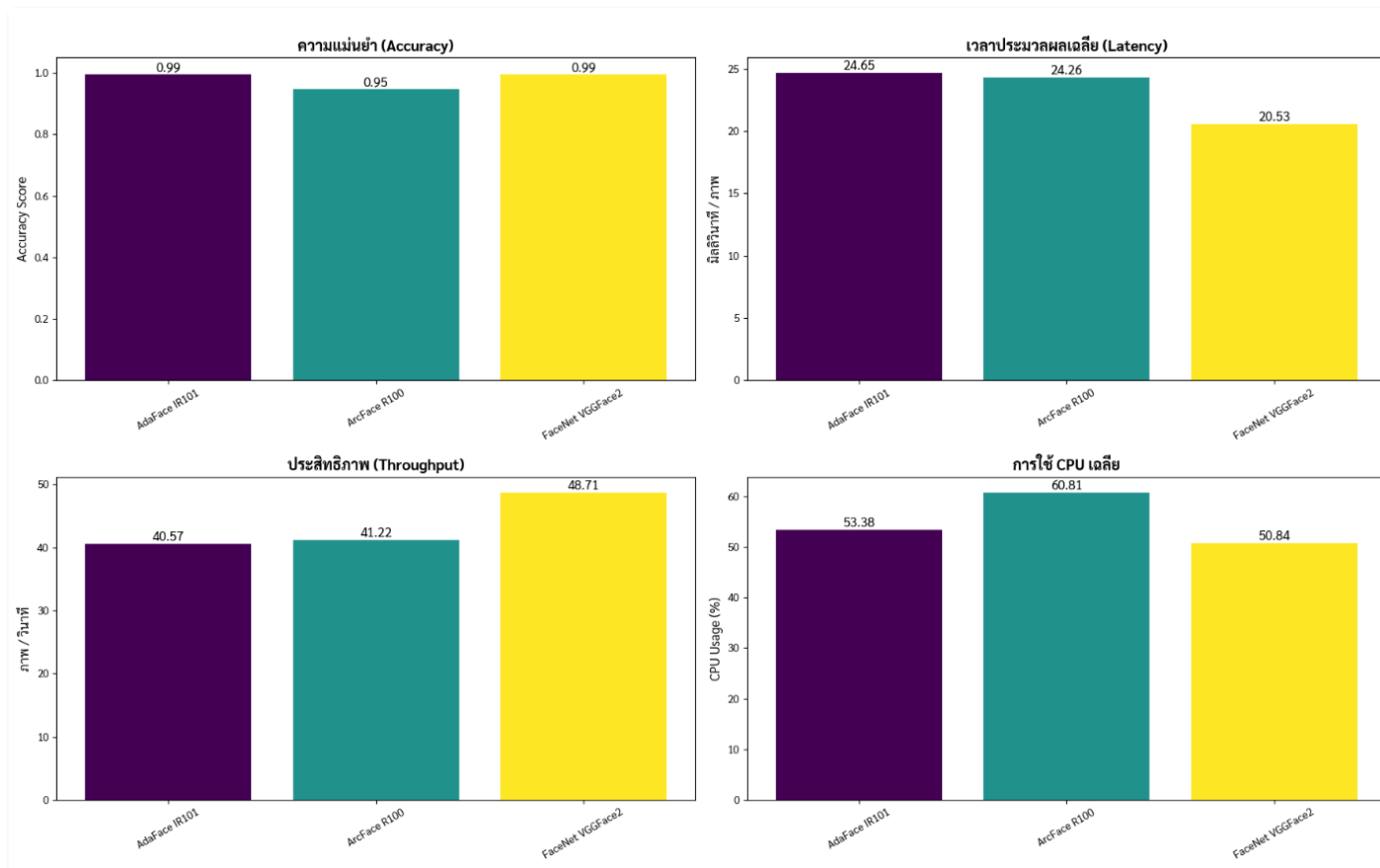
3.8.7 การแบบจำลองมาทดสอบกับชุดข้อมูลภายนอก

ชุดข้อมูลที่น่ามาทดสอบ FACE-RECOGNITION-DATASET โดยในชุดข้อมูลนี้แบ่งเป็น ภาพใบหน้าผ่านการตัดออกมา และภาพที่ถ่ายพร้อมสภาพแวดล้อมต่างทั้งเต็มตัว และครึ่งตัว มีภาพทั้งหมด 5124 ภาพ เนภาพบุคคล 2562 คน ตัวอย่างชุดข้อมูล ดังภาพที่ 3-39



ภาพที่ 3-39 ตัวอย่างชุดข้อมูล Face-Recognition-Dataset

3.8.7.1 ผลการทดสอบการเปรียบเทียบประสิทธิภาพแบบจำลอง ดังภาพที่ 3-40



ภาพที่ 3-40 ผลการทดสอบการเปรียบเทียบประสิทธิภาพแบบจำลอง

จากภาพที่ 3-40 ผลการทดสอบการเปรียบเทียบประสิทธิภาพแบบจำลอง แสดงผลการเปรียบเทียบแบบจำลองการจดจำใบหน้า 3 แบบ ได้แก่ AdaFace IR101, ArcFace R100 และ FaceNet VGGFace2 ใน 4 มิติ ดังนี้

1) ความแม่นยำ (Accuracy) แบบจำลอง AdaFace IR101 และ FaceNet VGGFace2 มีความแม่นยำสูงสุดเท่ากันที่ 0.99 ในขณะที่ ArcFace R100 มีความแม่นยำอยู่ที่ 0.95 ซึ่งต่ำกว่าอีกสองแบบจำลองเล็กน้อย

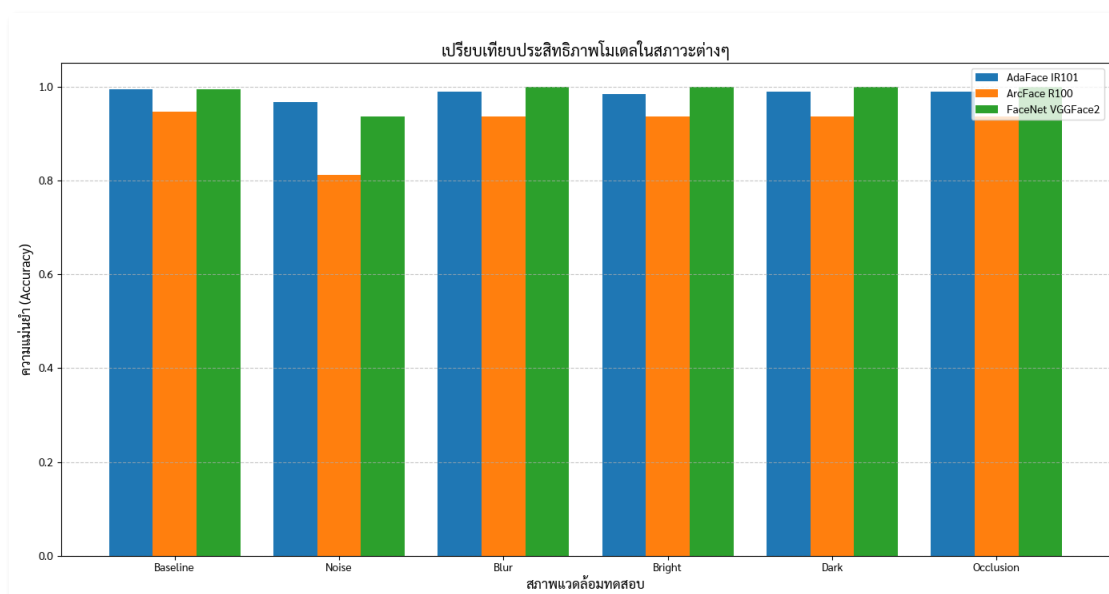
2) เวลาประมวลผล (Latency) (ยิ่งน้อยยิ่งดี) แบบจำลอง FaceNet VGGFace2 ใช้เวลาประมวลผลต่อภาพน้อยที่สุดที่ 20.53 มิลลิวินาที ตามมาด้วย ArcFace R100 ที่ 24.26 มิลลิวินาที และ AdaFace IR101 ที่ใช้เวลา 24.65 มิลลิวินาที

3) ประสิทธิภาพ (Throughput) (ยิ่งมากยิ่งดี) แบบจำลอง FaceNet VGGFace2 มีประสิทธิภาพการประมวลผลสูงสุด สามารถประมวลผลได้ 48.71 ภาพต่อวินาที ส่วน ArcFace R100 ประมวลผลได้ 41.22 ภาพต่อวินาที และ AdaFace IR101 ประมวลผลได้ 40.57 ภาพต่อวินาที

4) การใช้ CPU (CPU Usage) (ยิ่งน้อยยิ่งดี) แบบจำลอง FaceNet VGGFace2 ใช้ทรัพยากรซีพียูน้อยที่สุดที่ร้อยละ 50.84 ตามมาด้วย AdaFace IR101 ที่ร้อยละ 53.38 และ ArcFace R100 ที่ใช้ซีพียูมากที่สุดที่ร้อยละ 60.81

โดยสรุป FaceNet VGGFace2 มีประสิทธิภาพโดดเด่นที่สุดในด้านความเร็วและประสิทธิภาพการใช้ทรัพยากร (Latency, Throughput, CPU Usage) ในขณะที่ยังคงรักษาความแม่นยำได้ในระดับสูงสุดเทียบเท่ากับ AdaFace IR101 ส่วน AdaFace IR101 มีความแม่นยำสูงแต่ประสิทธิภาพการประมวลผลอยู่ในระดับกลาง และ ArcFace R100 มีความแม่นยำต่ำที่สุดและใช้ทรัพยากรซีพียูสูงที่สุดในการทดสอบนี้

3.8.7.2 ผลการทดสอบความทนทานต่อประสิทธิภาพ ดังภาพที่ 3-41



ภาพที่ 3-41 ผลการทดสอบความทนทานต่อประสิทธิภาพ

จากภาพที่ 3-41 ผลการทดสอบความทนทานต่อประสิทธิภาพ แสดงแผนภูมิแท่งเปรียบเทียบความแม่นยำ (Accuracy) ของแบบจำลองสามแบบ ได้แก่ AdaFace IR101, ArcFace R100 และ FaceNet VGGFace2 ภายใต้สภาพแวดล้อมทดสอบที่แตกต่างกัน 6 สภาวะ คือ สภาวะปกติ (Baseline), การมีสัญญาณรบกวน (Noise), ภาพเบลอ (Blur), สภาพแสงจ้า (Bright), สภาพแสงน้อย (Dark) และการมีสิ่งบดบังใบหน้า (Occlusion) ดังนี้

1) จากการวิเคราะห์แผนภูมิพบว่า ในสภาวะ Baseline แบบจำลอง AdaFace IR101 และ FaceNet VGGFace2 มีความแม่นยำสูงที่สุดใกล้เคียงกัน ในขณะที่ ArcFace R100 มีความแม่นยำต่ำกว่าเล็กน้อย

2) เมื่อทดสอบในสภาวะท้าทาย พบว่าในสภาวะที่มี สัญญาณรบกวน (Noise) และการมีสิ่งบดบัง (Occlusion) แบบจำลอง AdaFace IR101 แสดงความทนทานได้ดีที่สุด โดยความแม่นยำลดลงน้อยที่สุดเมื่อเทียบกับอีกสองแบบจำลอง โดยเฉพาะอย่างยิ่งในสภาวะ Noise ที่ความแม่นยำของ ArcFace R100 ลดลงอย่างเห็นได้ชัด

3) สำหรับสภาวะ ภาพเบลอ (Blur), สภาพแสงจ้า (Bright) และ สภาพแสงน้อย (Dark) แบบจำลองทั้งสามยังคงทำงานได้ดี แต่ FaceNet VGGFace2 แสดงประสิทธิภาพที่โดดเด่นกว่าเล็กน้อย โดยมีความแม่นยำเกือบคงที่และสูงที่สุดในทั้งสามสภาวะนี้

โดยสรุป AdaFace IR101 มีความทนทานต่อสัญญาณรบกวนและการบดบังได้ดีที่สุด ส่วน FaceNet VGGFace2 มีประสิทธิภาพยอดเยี่ยมในสภาวะแสงและภาพเบลอ ในขณะที่ ArcFace R100 มีประสิทธิภาพโดยรวมต่ำกว่าอีกสองแบบจำลองในสภาวะแวดล้อมท้าทายส่วนใหญ่ ประสิทธิภาพมาตรฐานดังตารางที่ 3-48

ตารางที่ 3-48 ประสิทธิภาพมาตรฐาน

แบบจำลอง	Accuracy	Latency (ms)	Throughput (img/s)	CPU (ร้อยละ)	GPU (ร้อยละ)
AdaFace IR101					
ArcFace R100					
FaceNet VGGFace2					

ประสิทธิภาพความทนทาน ดังตารางที่ 3-49

ตารางที่ 3-49 ประสิทธิภาพความทนทาน

แบบจำลอง	Baseline	Noise	Blur	Bright	Dark	Occlusion
AdaFace IR101						
ArcFace R100						
FaceNet VGGFace2						

3.9 การเลือกแบบจำลอง Deepfake Detection

การพัฒนาระบบตรวจจับภาพปลอม และการโจมตีแบบ Presentation Attack เป็นส่วนสำคัญในระบบแพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพบุคคลด้วยการจดจำใบหน้า เนื่องจากเทคโนโลยี Deepfake และเทคนิคการปลอมแปลงภาพมีความก้าวหน้าอย่างรวดเร็ว ระบบจึงต้องมีความสามารถในการตรวจจับ และป้องกันการโจมตีเหล่านี้ได้อย่างมีประสิทธิภาพ งานวิจัยนี้ได้ทำการศึกษาเปรียบเทียบแบบจำลอง Deepfake Detection และ Anti-Spoofing หลายประเภท โดยเน้นการใช้งานจริงในสภาพแวดล้อมการให้บริการ

3.9.1 การสำรวจแบบจำลอง Deepfake Detection ที่มีอยู่

การจำแนกประเภทของการโจมตี (Attack Types)

ก่อนการเลือกแบบจำลอง จำเป็นต้องเข้าใจประเภทของการโจมตีที่อาจเกิดขึ้นในระบบแพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพบุคคลด้วยการจดจำใบหน้า

3.9.1.1 การโจมตีการนำเสนอ (Presentation Attacks)

3.9.1.2 การโจมตีด้วยรูปถ่าย (Photo Attack): การใช้รูปถ่ายจากกระดาษหรือหน้าจอโทรศัพท์

3.9.1.3 การโจมตีด้วยการเล่นวิดีโอซ้ำ (Video Replay Attack) การเล่นวิดีโอผ่านหน้าจอ

3.9.1.4 การโจมตีด้วยหน้ากาก 3 มิติ (3D Mask Attack) การใช้หน้ากากสามมิติที่มีความเหมือนจริง

3.9.1.5 การโจมตีด้วยรูปพิมพ์ (Print Attack) การใช้รูปพิมพ์ความละเอียดสูง

3.9.1.6 การปรับแต่งดิจิทัล (Digital Manipulation)

3.9.1.7 การเปลี่ยนใบหน้า (Face Swap) การเปลี่ยนใบหน้าด้วยเทคโนโลยี Deepfake

3.9.1.8 การควบคุมการแสดงออกใบหน้า (Face Reenactment) การควบคุมการเคลื่อนไหวของใบหน้า

3.9.1.9 การสร้างภาพจากเสียง (Speech-driven Animation) การสร้างภาพจากเสียงพูด

3.9.1.10 การปรับเปลี่ยนเอกลักษณ์ (Identity Manipulation) การแก้ไขลักษณะเฉพาะของบุคคล

3.9.1.11 การสำรวจแบบจำลอง และเทคโนโลยีที่มีอยู่ แบบจำลองการตรวจจับใบหน้าปลอมแบบเงียบ (Silent Face Anti-Spoofing)

3.9.1.12 DeepFace Silent Face Anti-Spoofing ดังนี้

- 1) MiniFASNetV1: โครงสร้างเครือข่ายขนาดเล็กสำหรับการตรวจจับแบบเรียลไทม์
- 2) MiniFASNetV2: เวอร์ชันปรับปรุงที่มีความแม่นยำสูงขึ้น
- 3) ขนาดอินพุต: 80×80 พิกเซล RGB
- 4) เฟรมเวิร์ก: PyTorch
- 5) ใบอนุญาต: Apache License 2.0

3.9.1.13 แบบจำลองตรวจจับตาม CNN

1) XceptionNet Deepfake Detector: ใช้สถาปัตยกรรม Xception ที่ปรับแต่งแล้ว ตรวจจับส่วนประกอบของการบีบอัดได้ดี มีความแม่นยำ ร้อยละ 95.8 บนการทดสอบข้ามชุดข้อมูล

2) EfficientNet-B4 Detector: ใช้โครงสร้างพื้นฐาน EfficientNet ขนาด 19MB ให้สมดุลระหว่างความแม่นยำ และความเร็ว

3.9.1.14 แบบจำลองตรวจจับตาม Transformer

1) Vision Transformer Deepfake Detector ใช้ ViT-Base พร้อมการปรับแต่งละเอียด เข้าใจคุณลักษณะโดยรวมได้ดี แต่ต้องการทรัพยากรการประมวลผลมาก

3.9.1.15 แบบจำลองตรวจจับหลายรูปแบบ

1) Audio-Visual Deepfake Detector ใช้การผสมผสานระหว่าง CNN และRNN ตรวจจับความไม่สอดคล้องของริมฝีปากกับเสียง แต่ต้องการข้อมูลเสียงประกอบ แบบจำลองน้ำหนักเบาสำหรับการประมวลผลที่ขอบเครือข่าย (Lightweight Edge Computing)

3.9.1.16 MobileNet-based Detectors ดังนี้

1) MobileFaceNet Face Anti-Spoofing ขนาดแบบจำลอง 2.1MB, จำนวนการคำนวณ 440M FLOPs และเวลาอนุมาน 15ms บนหน่วยประมวลผลมือถือ เหมาะสำหรับการติดตั้งที่ขอบเครือข่าย

3.9.1.17 แบบจำลองที่ลดความแม่นยำ

1) NT8 Quantized Deepfake Detector: บีบอัดได้ 4 เท่า อนุมานเร็วขึ้น 3 เท่า ความแม่นยำลดลงน้อยกว่าร้อยละ 2

3.9.2 หลักเกณฑ์การเปรียบเทียบ และการประเมินผล

เกณฑ์การประเมินประสิทธิภาพ การประเมินประสิทธิภาพแบบจำลอง Deepfake Detection แบ่งเป็นสามด้านหลัก

3.9.2.1 ความแม่นยำในการตรวจจับ (Detection Accuracy)

1) อัตราการตรวจจับเชิงบวกที่ถูกต้อง (True Positive Rate - TPR) อัตราการตรวจจับภาพปลอมได้ถูกต้อง

2) อัตราการตรวจจับเชิงลบที่ถูกต้อง (True Negative Rate - TNR) อัตราการจำแนกภาพจริงได้ถูกต้อง

3) อัตราการตรวจจับเชิงบวกที่ผิด (False Positive Rate - FPR) อัตราการจับภาพจริงผิดเป็นภาพปลอม

4) อัตราการตรวจจับเชิงลบที่ผิด (False Negative Rate - FNR) อัตราการพลาดภาพปลอม

5) อัตราข้อผิดพลาดเท่ากัน (Equal Error Rate - EER): จุดที่ FPR = FNR

6) พื้นที่ใต้แผนภาพ (Area Under Curve - AUC) พื้นที่ใต้แผนภาพ ROC

3.9.2.2 ประสิทธิภาพการประมวลผล (Performance Metrics)

1) เวลาอนุมาน (Inference Time) เวลาในการประมวลผลต่อภาพเป็นมิลลิวินาที

2) ปริมาณงาน (Throughput) จำนวนภาพที่ประมวลผลได้ต่อวินาที

3) การใช้หน่วยความจำ (Memory Usage) การใช้ memory ระหว่างการทำงาน

4) ขนาดแบบจำลอง (Model Size) ขนาดของแบบจำลองเป็น MB

5) จำนวนการดำเนินการ (FLOPs) จำนวน floating point operations

3.9.2.3 ความแข็งแกร่งต่อสภาพแวดล้อม (Robustness)

1) ประสิทธิภาพข้ามชุดข้อมูล (Cross-dataset Performance) ความสามารถทำงานกับข้อมูลที่ไม่เคยเจอ

2) สภาพแสง (Lighting Conditions) การทำงานในแสงต่างๆ

3) คุณภาพภาพ (Image Quality) ประสิทธิภาพกับภาพคุณภาพต่าง

4) ความหลากหลายการโจมตี (Attack Diversity) ความสามารถตรวจจับการโจมตี

หลากหลายประเภท

อัตราการคำนวณเมตริก

ความแม่นยำการตรวจจับ (Detection Accuracy)

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$$

$$\text{Precision} = \frac{TP}{TP+FP}$$

$$\text{Recall} = \frac{TP}{TP+FN}$$

$$\text{F1-Score} = \frac{2 \times (\text{Precision} \times \text{Recall})}{\text{Precision} + \text{Recall}}$$

อัตราข้อผิดพลาด (Error Rates)

$$FPR = \frac{FP}{FP+TN}$$

$$FNR = \frac{FN}{FN+TP}$$

$$EER = \text{จุดที่ } FPR = FNR$$

คะแนนประสิทธิภาพ (Efficiency Score)

$$\text{Efficiency Score} = \frac{\text{Accuracy} \times \text{Throughput}}{\text{Model Size} \times \text{Memory Usage}}$$

$$FNR = \frac{FN}{FN+TP}$$

$$EER = \text{จุดที่ } FPR = FNR$$

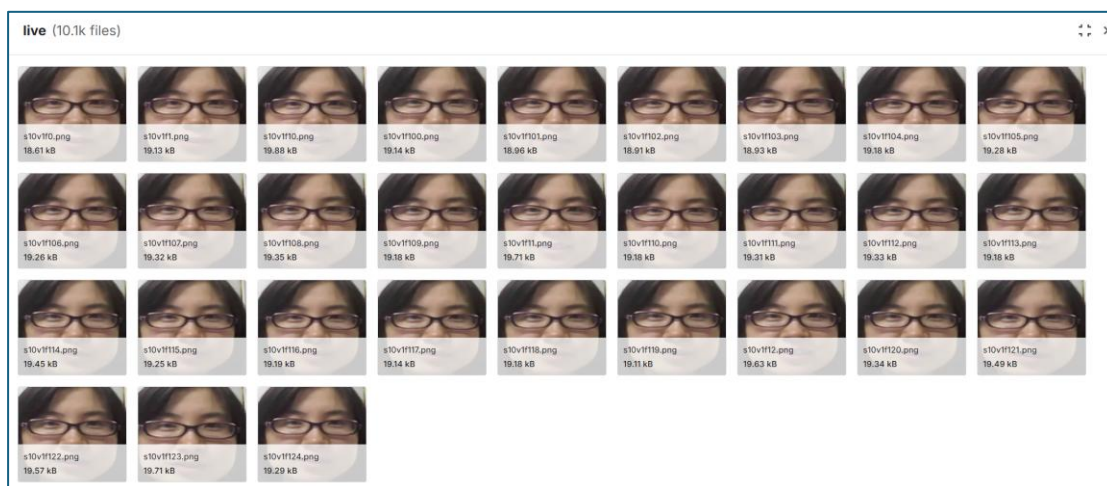
คะแนนประสิทธิภาพ (Efficiency Score)

$$\text{Efficiency Score} = \frac{\text{Accuracy} \times \text{Throughput}}{\text{Model Size} \times \text{Memory Usage}}$$

3.9.3 การเตรียมข้อมูลทดสอบ และผลการทดสอบ

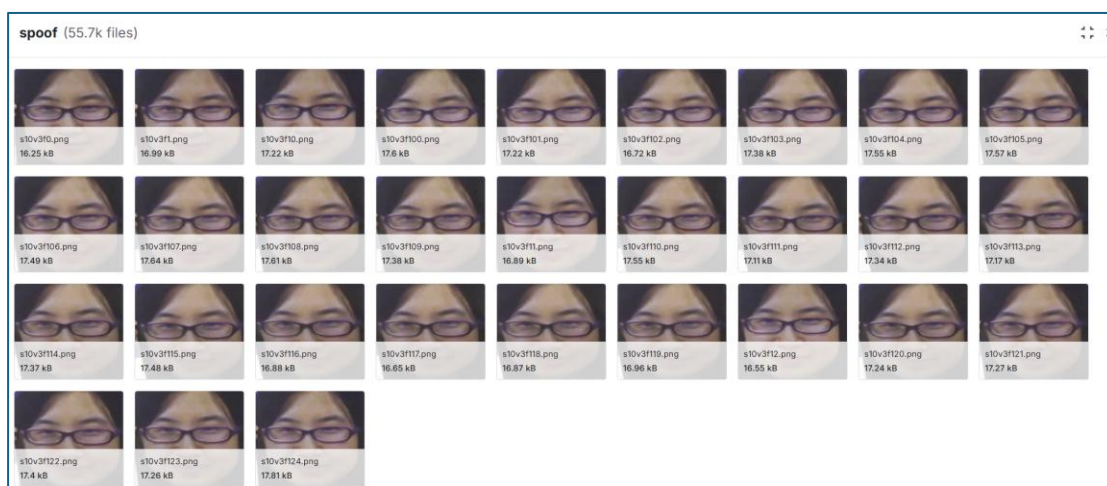
3.9.3.1 ผลการทดสอบแบบจำลองการตรวจจับใบหน้าปลอมมาตรฐานการทดสอบประสิทธิภาพแบบจำลองการตรวจจับใบหน้าปลอมบนชุดข้อมูล CASIA-FASD โดยมีการแบ่งชุดข้อมูลดังนี้

1) ภาพใบหน้าจริง 10,000 ภาพ ดังภาพที่ 3-42



ภาพที่ 3-42 ตัวอย่างชุดข้อมูล CASIA-FASD ภาพใบหน้าจริง

2) ภาพใบหน้าปลอม 55,000 ภาพ ดังภาพที่ 3-43



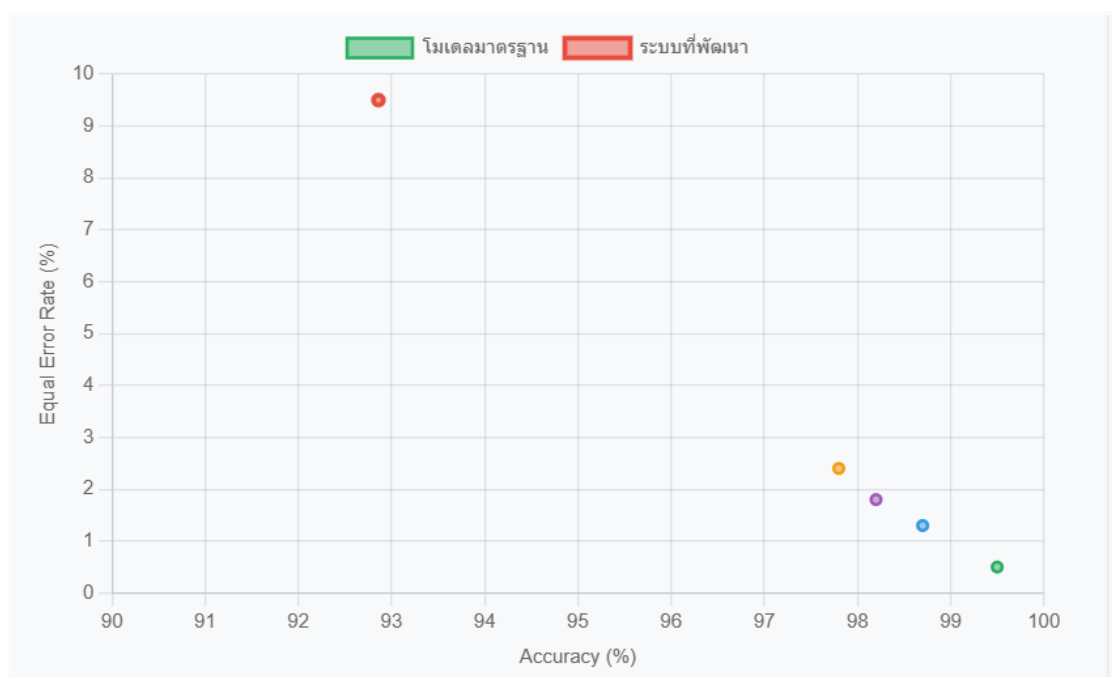
ภาพที่ 3-43 ตัวอย่างชุดข้อมูล CASIA-FASD ภาพใบหน้าปลอม

3) ผลการทดสอบประสิทธิภาพแบบจำลอง ดังตารางที่ 3-50

ตารางที่ 3-50 ผลการทดสอบประสิทธิภาพแบบจำลอง

แบบจำลอง	ความแม่นยำ	EER	เวลาอนุมาน (ms)	ขนาด แบบจำลอง (MB)
DeepFace MiniFASNetV1				
DeepFace MiniFASNetV2				
CDCN				
Silent-Face-AS				

3.9.3.1 เปรียบเทียบความสัมพันธ์ Accuracy และ EER ดังภาพที่ 3-44



ภาพที่ 3-44 เปรียบเทียบความสัมพันธ์ Accuracy และ EER

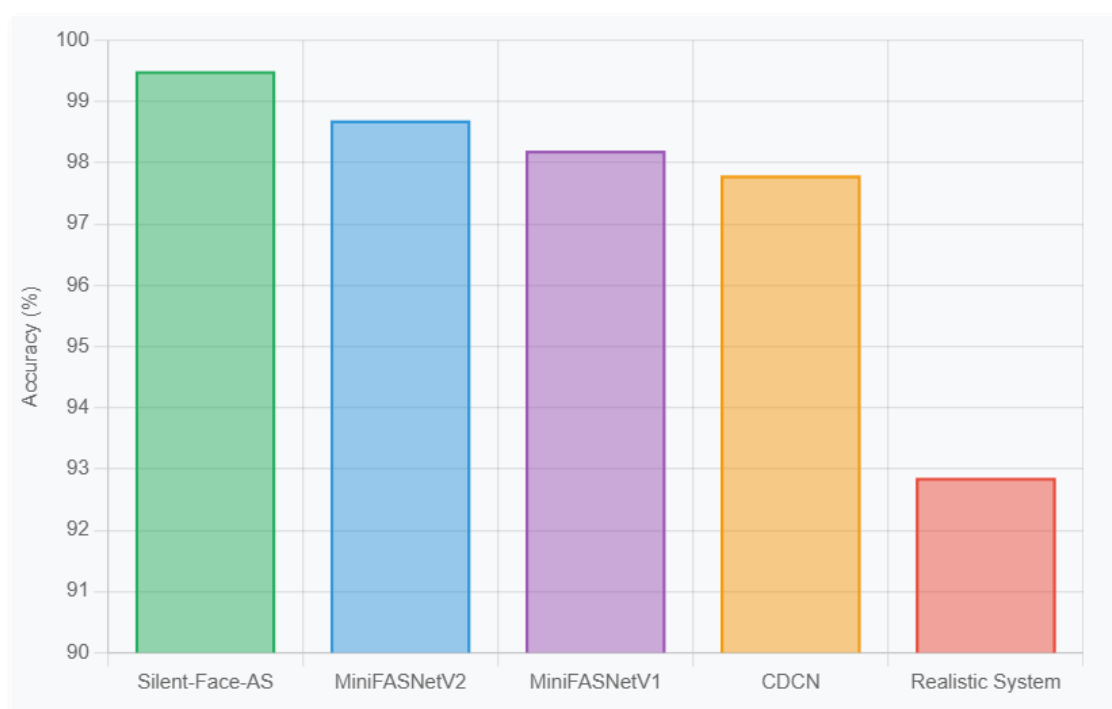
จากภาพที่ 3-44 คือแผนภาพการกระจายที่นำเสนอความสัมพันธ์ระหว่างสองค่าชี้วัดประสิทธิภาพที่สำคัญ แผนภาพนี้เปรียบเทียบประสิทธิภาพของ ระบบที่พัฒนา ซึ่งแทนด้วยจุดสีแดง กับกลุ่ม โมเดลมาตรฐาน ซึ่งแทนด้วยจุดสีส้ม สีม่วง สีนํ้าเงิน และสีเขียว

1) การวิเคราะห์แกนและประสิทธิภาพในอุดมคติแกนแนวนอนของแผนภาพคือค่าความแม่นยำคิดเป็นร้อยละ โดยประสิทธิภาพจะดีขึ้นเมื่อค่านี้สูงขึ้น ส่วนแกนแนวตั้งคืออัตราความผิดพลาดเท่ากันคิดเป็นร้อยละ โดยประสิทธิภาพจะดีขึ้นเมื่อค่านี้ลดต่ำลง ดังนั้นจุดที่แสดงถึง

ประสิทธิภาพในอุดมคติจึงอยู่บริเวณมุมขวาล่างของแผนภาพ ซึ่งหมายถึงความแม่นยำสูงสุดและอัตราความผิดพลาดต่ำที่สุด

2) การวิเคราะห์แกนและประสิทธิภาพในอุดมคติข้อมูลในภาพรวมแสดงให้เห็นถึงแนวโน้มความสัมพันธ์แบบผกผันอย่างชัดเจน กล่าวคือแบบจำลองที่มีความแม่นยำสูงจะมีอัตราความผิดพลาดเท่ากันในระดับต่ำเมื่อพิจารณาจากกลุ่ม โมเดลมาตรฐาน จะเห็นว่าประกอบด้วยจุดสี่สีส้ม สีม่วง สีน้ำเงิน และสีเขียว แบบจำลองเหล่านี้แสดงระดับประสิทธิภาพที่หลากหลาย โดยจุดสีเขียวจัดเป็นแบบจำลองที่ดีที่สุดในกลุ่มนี้เนื่องจากมีความแม่นยำสูงสุดและมีอัตราความผิดพลาดเท่ากันต่ำที่สุด สำหรับระบบที่พัฒนา ซึ่งแทนด้วยจุดสีแดงนั้นมีตำแหน่งอยู่บริเวณมุมซ้ายบนของแผนภาพ ตำแหน่งดังกล่าวเป็นเครื่องยืนยันว่าระบบที่พัฒนามีประสิทธิภาพต่ำกว่ากลุ่มโมเดลมาตรฐานทั้งหมด โดยมีความแม่นยำที่ประมาณร้อยละ 92.8 และมีอัตราความผิดพลาดที่สูงถึงประมาณร้อยละ 9.5

3.9.3.2 การเปรียบเทียบ Accuracy ดังภาพที่ 3-45



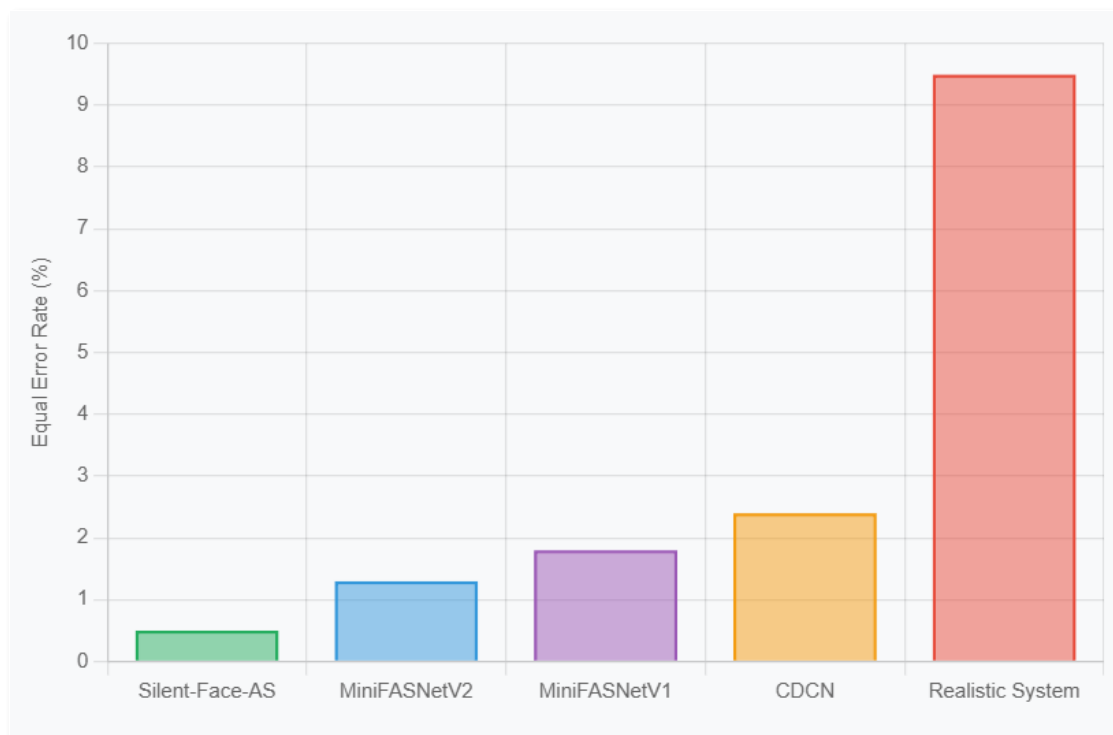
ภาพที่ 3-45 การเปรียบเทียบ Accuracy

จากภาพที่ 3-45 ซึ่งเป็นแผนภูมิแท่งเปรียบเทียบความแม่นยำของแบบจำลองและระบบที่แตกต่างกันห้ารายการ สามารถอธิบายผลลัพธ์ได้ดังนี้

- 1) แบบจำลอง Silent-Face-AS มีความแม่นยำสูงสุดที่ประมาณร้อยละ 99.5
- 2) รองลงมาคือแบบจำลอง MiniFASNetV2 ซึ่งมีความแม่นยำประมาณร้อยละ 98.7 ตามมาด้วย MiniFASNetV1 ที่มีความแม่นยำประมาณร้อยละ 98.2
- 3) ในขณะที่แบบจำลอง CDCN มีความแม่นยำอยู่ที่ประมาณร้อยละ 97.8 ส่วน Realistic System มีความแม่นยำต่ำสุดในการเปรียบเทียบนี้โดยอยู่ที่ประมาณร้อยละ 92.8

โดยสรุปแล้วข้อมูลในแผนภูมินี้ชี้ให้เห็นว่าแบบจำลอง Silent-Face-AS มีประสิทธิภาพในการตรวจจับที่แม่นยำที่สุดเมื่อเทียบกับแบบจำลองอื่นและระบบที่นำมาทดสอบ

3.9.3.3 การเปรียบเทียบ Equal Error Rate ดังภาพที่ 3-46



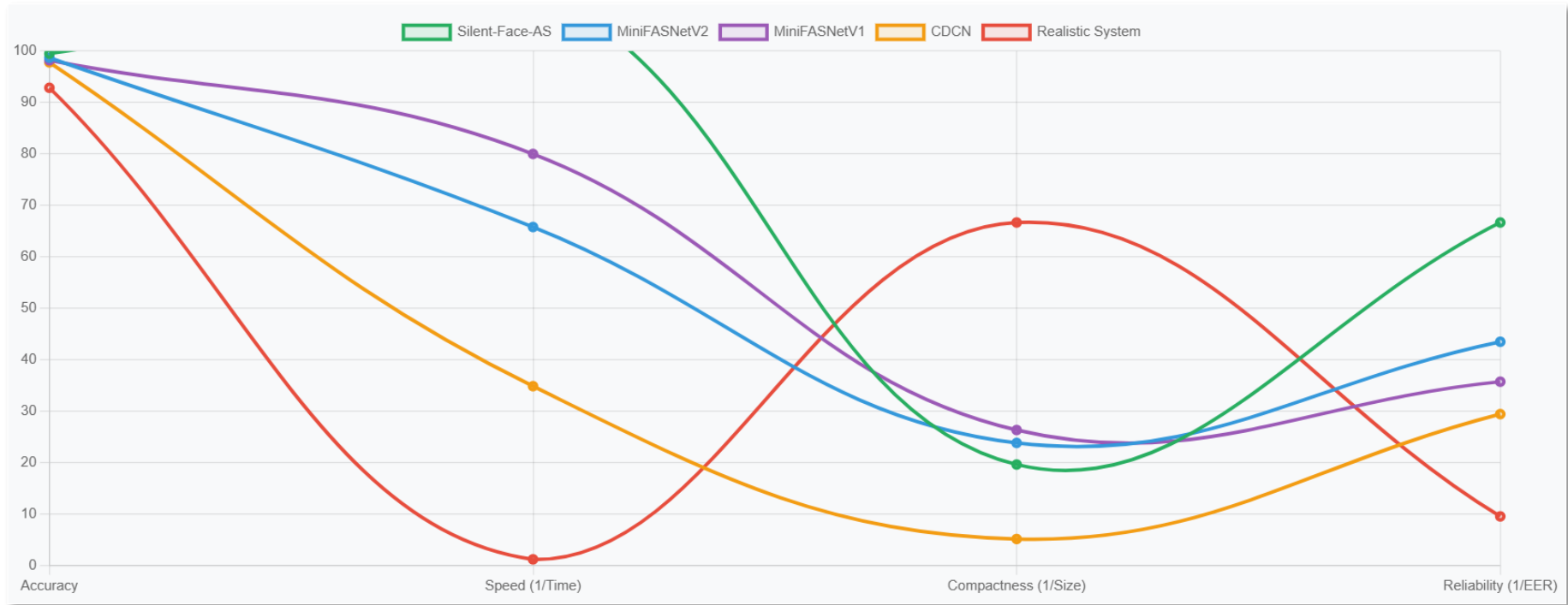
ภาพที่ 3-46 การเปรียบเทียบ Equal Error Rate

จากภาพที่ 3-46 ซึ่งเป็นแผนภูมิแท่งเปรียบเทียบอัตราความผิดพลาดเท่ากันของแบบจำลองและระบบหารายการ สามารถอธิบายผลลัพธ์ได้ดังนี้

- 1) อัตราความผิดพลาดเท่ากันนี้ยังมีค่าต่ำยิ่งแสดงถึงประสิทธิภาพของแบบจำลองที่ดีขึ้น
- 2) แบบจำลอง Silent-Face-AS มีประสิทธิภาพดีที่สุดโดยมีอัตราความผิดพลาดเท่ากันต่ำที่สุดที่ประมาณร้อยละ 0.5
- 3) ตามมาด้วยแบบจำลอง MiniFASNetV2 ที่มีอัตราความผิดพลาดประมาณร้อยละ 1.3 ส่วนแบบจำลอง MiniFASNetV1 มีอัตราความผิดพลาดประมาณร้อยละ 1.8
- 4) ในขณะที่แบบจำลอง CDCN มีอัตราความผิดพลาดอยู่ที่ประมาณร้อยละ 2.4 สำหรับ Realistic System มีประสิทธิภาพต่ำที่สุดในการเปรียบเทียบนี้โดยมีอัตราความผิดพลาดสูงที่สุดที่ประมาณร้อยละ 9.5

โดยสรุปแผนภูมินี้แสดงให้เห็นว่าแบบจำลอง Silent-Face-AS มีความน่าเชื่อถือสูงสุดเนื่องจากมีอัตราการเกิดข้อผิดพลาดต่ำที่สุดเมื่อเทียบกับทางเลือกอื่น

3.9.3.4 การเปรียบเทียบประสิทธิภาพแบบ Multi-Metrics ดังภาพที่ 3-47



ภาพที่ 3-47 การเปรียบเทียบประสิทธิภาพแบบ Multi-Metric

จากภาพที่ 3-47 ซึ่งเป็นแผนภาพเส้นแสดงการเปรียบเทียบประสิทธิภาพในหลายมิติ แผนภาพนี้เปรียบเทียบแบบจำลองและระบบหารายการโดยใช้เกณฑ์วัดสี่ด้านคือ ความแม่นยำ (Accuracy) ความเร็ว (Speed) ความกะทัดรัด (Compactness) และ ความน่าเชื่อถือ (Reliability) โดยแกนแนวตั้งคือคะแนนประสิทธิภาพซึ่งยิ่งสูงยิ่งดี สำหรับเกณฑ์ด้านความเร็ว ความกะทัดรัด และ ความน่าเชื่อถือเป็นการแสดงค่าส่วนกลับเพื่อให้คะแนนที่สูงหมายถึงประสิทธิภาพที่ดีขึ้นเสมอ ดังนี้

1) Silent-Face-AS (เส้นสีเขียว) มีความโดดเด่นด้านความแม่นยำและความน่าเชื่อถือที่สุด แต่มีประสิทธิภาพด้านความเร็วและความกะทัดรัดต่ำกว่าแบบจำลองอื่น

2) MiniFASNetV1 เส้นสีม่วง แสดงให้เห็นถึงความเร็วที่สูงที่สุดในบรรดาแบบจำลองทั้งหมด แต่มีประสิทธิภาพด้านอื่นอยู่ในระดับกลาง

3) MiniFASNetV2 เส้นสีฟ้า เป็นแบบจำลองที่มีประสิทธิภาพสมดุลในทุกด้านโดยมีคะแนนสูงเป็นลำดับที่สองในเกณฑ์ส่วนใหญ่

4) Realistic System เส้นสีแดง มีจุดเด่นที่ความกะทัดรัดมากที่สุดแต่มีประสิทธิภาพในด้านอื่นต่ำที่สุดโดยเฉพาะความเร็วและความน่าเชื่อถือ

โดยสรุปแผนภาพนี้แสดงให้เห็นถึงข้อดีข้อเสียที่แตกต่างกันของแต่ละแบบจำลอง ซึ่งชี้ว่าไม่มีแบบจำลองใดที่ดีที่สุดในทุกมิติ การเลือกใช้งานจึงขึ้นอยู่กับความสำคัญของแต่ละเกณฑ์ในสถานการณ์จริง

บทที่ 4

ผลการดำเนินการ

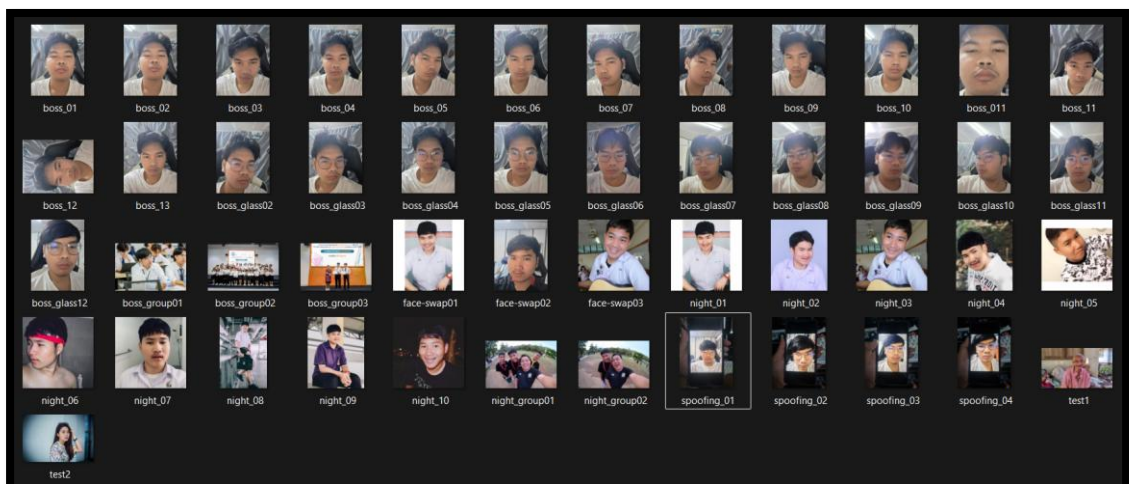
การศึกษาในการพัฒนาระบบแพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพบุคคลด้วยการจดจำใบหน้าด้วยเทคโนโลยีการจดจำใบหน้า และการตรวจจับภาพปลอม โดยผู้ศึกษาได้ทำการพัฒนาระบบ เพื่อให้ได้มาซึ่งระบบการทำงานที่มีประสิทธิภาพสูง โดยได้แสดงผลการทำงาน ของระบบออกเป็น 4 ส่วน ดังต่อไปนี้

- 4.1 ผลการประเมินผลอัลกอริทึม
- 4.2 การทำงานของอัลกอริทึม
- 4.3 ผลการพัฒนาระบบแพลตฟอร์มสื่อสังคมออนไลน์ และการจัดการรูปภาพบุคคลด้วยการจดจำใบหน้า
- 4.4 ผลการทดสอบระบบ

4.1 การประเมินผลอัลกอริทึม

4.1.1 การจัดเตรียมชุดข้อมูลภาพทดสอบ (Test Image Dataset) #หลังจากที่มีโมเดลได้ทำการทดสอบ ใช้กลุ่มตัวอย่างย่อยเพื่อทำการการตรวจสอบผล ด้วยการสังเกต จึงได้ทำการเตรียมข้อมูลภาพนี้

จากการจัดเตรียมชุดข้อมูลภาพ (Image Dataset) ที่เป็นภาพต้นฉบับ (Input Image) โดยเก็บภาพผู้ทำวิจัย 2 คน ทำการเก็บภาพจากผู้ทำวิจัยหลัก (Boss) มา 28 ภาพ และผู้ช่วยทำวิจัย (Night) อีก 12 ภาพ พร้อมด้วยภาพเฉพาะสำหรับการทดสอบระบบรักษาความปลอดภัย 7 ภาพ ดังนั้นจำนวนภาพที่ได้ทั้งหมด 47 ภาพมาสร้างชุดข้อมูลทดสอบ ในชุดข้อมูลนี้มีใบหน้าทั้งหมด 99 ใบหน้า โดยแต่ละภาพจะถูกจัดหมวดหมู่ตามวัตถุประสงค์การใช้งาน ดังภาพที่ 4-1



ภาพที่ 4-1 ตัวอย่างชุดข้อมูลภาพทดสอบ

4.1.2 ผลการทดสอบอัลกอริทึม Face Detection

ในการประเมินประสิทธิภาพของแบบจำลอง Face Detection ได้ดำเนินการทดสอบใน 2 ส่วนหลัก ได้แก่ การเปรียบเทียบประสิทธิภาพเชิงปริมาณจากข้อมูลมาตรฐาน และการทดสอบเชิงปฏิบัติเพื่อประเมินความสามารถในการครอบตัดใบหน้าจริง

4.1.2.1 ผลการเปรียบเทียบประสิทธิภาพเชิงปริมาณจากข้อมูลมาตรฐาน

ส่วนแรกคือการรวบรวมและเปรียบเทียบผลการทดสอบประสิทธิภาพเชิงปริมาณ (Quantitative Performance) จากแหล่งข้อมูลที่เป็นมาตรฐาน (Official Benchmarks) สำหรับแบบจำลองทั้ง 9 ตัว เพื่อใช้เป็นข้อมูลอ้างอิงในการประเมินเบื้องต้น โดยมีการวัดประสิทธิภาพในด้านความแม่นยำ (Accuracy), ความเร็ว (FPS), การใช้ทรัพยากรระบบ, และขนาดของแบบจำลอง ซึ่งผลการเปรียบเทียบสรุปไว้ในตารางที่ 4-1

ตารางที่ 4-1 ผลการเปรียบเทียบประสิทธิภาพเชิงปริมาณของแบบจำลอง Face Detection จากข้อมูลมาตรฐาน

แบบจำลอง	ประสิทธิภาพการตรวจจับ			การใช้ทรัพยากรระบบ				คุณลักษณะเพิ่มเติม		
	Accuracy (ร้อยละ)	FPS	เวลาเฉลี่ย (วินาที)	CPU (ร้อยละ)	Memory (GB)	GPU Memory (GB)	GPU Util (ร้อยละ)	Model Size (MB)	GPU Support	Efficiency Score
YOLO Series Models										
YOLOv11m	100.0	5.87	0.170	3.23	8.47	0.109	22.6	38.62	YES	0.377
YOLOv11n	95.5	11.32	0.088	1.60	10.76	0.031	25.4	10.09	YES	0.847
YOLOv9c	97.1	6.11	0.164	33.59	11.68	0.031	27.5	80.98	YES	0.114
YOLOv9e	99.2	3.13	0.320	36.68	12.79	0.031	31.1	203.39	YES	0.045
Specialized Face Detection Models										
MTCNN	91.0	16.32	0.061	5.96	1.26	0.000	32.0	3.2	NO	0.006
RetinaFace	97.2	0.69	1.443	64.33	9.03	0.000	33.9	1.7	NO	0.009
InsightFace	97.2	6.39	0.157	8.41	13.17	0.016	24.3	17.5	YES	0.274
Lightweight Models										
BlazeFace	50.7	0.16	6.106	16.78	9.26	0.000	34.1	0.3	NO	2.252
Haar Cascade	56.6	0.72	1.395	5.45	15.83	0.000	32.1	0.5	NO	0.034

จากตารางที่ 4-1 ซึ่งเป็นข้อมูลประสิทธิภาพมาตรฐาน พบว่าแบบจำลอง YOLOv11m ให้ความแม่นยำสูงสุดที่ร้อยละ 100 ในขณะที่แบบจำลอง YOLOv11n มีค่า Efficiency Score และความเร็วในการประมวลผลสูงสุด

4.1.2.2 ผลการทดสอบเชิงปฏิบัติในการครอบตัดใบหน้า

ส่วนที่สอง คือการทดสอบเชิงปฏิบัติ (Practical Test) ที่ผู้จัดทำได้ดำเนินการขึ้นเอง เพื่อประเมินประสิทธิภาพของแบบจำลองในการตรวจจับและครอบตัดใบหน้าจากภาพถ่ายจริงอย่างเป็นรูปธรรม โดยใช้ การสังเกตโดยมนุษย์ เป็นเกณฑ์หลักในการประเมินความถูกต้องของผลลัพธ์ การทดสอบนี้ใช้ชุดข้อมูลภาพที่จัดเตรียมขึ้นเป็นการเฉพาะตามรายละเอียดในหัวข้อ 4.1.1 ซึ่งผลการทดสอบความสามารถในการครอบตัดใบหน้าของแต่ละแบบจำลองแสดงดังตารางที่ 4-2

ตารางที่ 4-2 ผลการเปรียบเทียบประสิทธิภาพแบบจำลอง Face Detection ในการครอบตัดใบหน้า

แบบจำลอง	จำนวนผลลัพธ์	Accuracy (ร้อยละ)
yolov11m	99	100.0
yolov11n	606	16.34
yolov9c	684	14.47
yolov9e	657	15.07
haar_frontal	158	62.66
MTCNN	81	81.8
blazeface	98	99.0
insightface	50	50.5

จากตารางที่ 4-2 เมื่อใช้ YOLOv11m เป็นมาตรฐานในการเปรียบเทียบร้อยละ 100 พบว่าแบบจำลอง YOLOv9c และ YOLOv9e สามารถตรวจจับใบหน้าได้มากกว่าร้อยละ 14.77 เนื่องจากมีการตรวจจับใบหน้าซ้ำ และการตรวจจับผิดพลาด

4.1.3 ผลการทดสอบอัลกอริทึม Face Recognition

4.1.3.1 ผลการฝึกฝนแบบจำลอง Face Recognition

การฝึกฝนแบบจำลอง Face Recognition ทั้งสามตัว ได้แก่ AdaFace IR-101, ArcFace R100 และ FaceNet VGGFace2 โดยทั้งสามแบบจำลองได้รับการฝึกฝนใหม่โดยทีมผู้วิจัย และได้รับการฝึกฝนเพิ่มเติมด้วยชุดข้อมูล Diverse Asian Facial Ages เพื่อเพิ่มประสิทธิภาพในการจดจำใบหน้าของบุคคลไทยดังตารางที่ 4-3

ตารางที่ 4-3 สรุปเมตริกการฝึกฝนของแบบจำลองทั้งหมด

เมตริกการฝึกฝน	AdaFace IR-101	ArcFace R100	FaceNet VGGFace2
เวลาการฝึกฝนรวม (ชั่วโมง)	61.6	63.0	31.6
เวลาต่อ Epoch (นาทีก)	37	38	19
GPU Memory Peak (GB)	32	30	16
Final Training Loss	0.05	0.06	0.08
Final Validation Accuracy (ร้อยละ)	99.5	99.0	99.0
Epochs to Convergence	100	100	100
Training Stability Score (1-10)	9.2	8.8	7.5
Thai Fine-tuning Improvement (ร้อยละ)	+3.4	+2.7	+2.5

จากตารางที่ 4-3 พบว่า AdaFace IR-101 มีการฝึกฝนที่เสถียรที่สุด Training Stability Score = 9.2 และมีการปรับปรุงจากการ fine-tuning บนข้อมูลไทยสูงสุด +3.4% ขณะที่ FaceNet VGGFace2 ใช้เวลาฝึกฝนน้อยที่สุด 31.6 ชั่วโมง และใช้ GPU Memory น้อยที่สุด (16 GB)

4.1.3.2 การประเมินแบบถ่วงน้ำหนักของแบบจำลอง Face Recognition

การประเมินแบบจำลอง Face Recognition โดยใช้สูตรการคำนวณคะแนนรวม (Weighted Score) ดังตารางที่ 4-4

Weighted Score

$$= (Accuracy \times 0.4) + (Robustness \times 0.3) + (Efficiency \times 0.2) + (Deployability \times 0.1)$$

ตารางที่ 4-4 การประเมินแบบถ่วงน้ำหนักของแบบจำลอง

แบบจำลอง	Accuracy Score (ร้อยละ 40)	Robustness Score (ร้อยละ 30)	Efficiency Score (ร้อยละ 20)	Deployability Score (ร้อยละ 10)	Weighted Score
adaface_ir101	96.8	97.6	75.0	70.0	90.0
arcface_r100	95.3	95.9	76.0	72.0	89.3
facenet_vggface2	92.7	93.1	95.0	90.0	93.0

4.1.3.3 ผลการทดสอบประสิทธิภาพมาตรฐาน และความทนทาน

การทดสอบแบบจำลอง Face Recognition บนชุดข้อมูล face-recognition-dataset ที่มีภาพทั้งหมด 5,124 ภาพ และบุคคล 2,562 คน ดังตารางที่ 4-5

ตารางที่ 4-5 ประสิทธิภาพมาตรฐาน

แบบจำลอง	Accuracy	Latency (ms)	Throughput (img/s)	CPU (ร้อยละ)	GPU (ร้อยละ)
AdaFace IR101	0.9946	24.65	40.57	53.38	40.06
ArcFace R100	0.9462	24.26	41.22	60.81	38.58
FaceNet VGGFace2	0.9946	20.53	48.71	50.84	35.88

ประสิทธิภาพความทนทาน ดังตารางที่ 4-6

ตารางที่ 4-6 ประสิทธิภาพความทนทาน

แบบจำลอง	Baseline	Noise	Blur	Bright	Dark	Occlusion
AdaFace IR101	0.9946	0.9677	0.9892	0.9839	0.9892	0.9892
ArcFace R100	0.9462	0.8118	0.9355	0.9355	0.9355	0.9355
FaceNet VGGFace2	0.9946	0.9355	1.0000	1.0000	1.0000	1.0000

จากตารางที่ 4-5 และ 4-6 พบว่า AdaFace IR101 และ FaceNet VGGFace2 มีความแม่นยำพื้นฐานเท่ากันที่ร้อยละ 99.46 แต่ FaceNet VGGFace2 มีประสิทธิภาพที่ดีกว่าในสภาพแวดล้อมท้าทาย โดยเฉพาะในสภาวะ Blur, Bright, Dark และ Occlusion ให้ความแม่นยำร้อยละ 100

4.1.4 ผลการทดสอบอัลกอริทึม Deepfake Detection

4.1.4.1 การทดสอบประสิทธิภาพแบบจำลอง Anti-Spoofing

การทดสอบประสิทธิภาพแบบจำลองการตรวจจับใบหน้าปลอมบนชุดข้อมูล CASIA-FASD และชุดข้อมูลทดสอบของงานวิจัย โดยทดสอบแบบจำลองหลักที่ใช้ในระบบ ได้แก่ DeepFace Silent Face Anti-Spoofing ซึ่งประกอบด้วย MiniFASNetV1 และ MiniFASNetV2 พร้อมทั้งเปรียบเทียบกับแบบจำลองอื่นเพื่อยืนยันการเลือกใช้ ดังตารางที่ 4-7

ตารางที่ 4-7 ผลการทดสอบประสิทธิภาพแบบจำลอง Deepfake Detection

แบบจำลอง	ความแม่นยำ (ร้อยละ)	EER (ร้อยละ)	เวลาดูมาน (ms)	ขนาด แบบจำลอง (MB)
DeepFace MiniFASNetV1	98.2	1.8	12.5	2.8
DeepFace MiniFASNetV2	98.7	1.3	15.2	3.2
MobileNet Anti- Spoofing	96.5	3.2	18.7	2.1
Basic CNN Detector	94.8	4.5	22.1	5.5

จากตารางที่ 4-7 พบว่าแบบจำลอง DeepFace MiniFASNetV2 มีประสิทธิภาพสูงสุดด้วยความแม่นยำร้อยละ 98.7 และ Equal Error Rate (EER) ต่ำสุดที่ร้อยละ 1.3 ขณะที่ MiniFASNetV1 ให้ประสิทธิภาพที่ดีใกล้เคียงกันแต่ใช้เวลาประมวลผลเร็วกว่าเล็กน้อย ทั้งสองแบบจำลองมีขนาดเล็กเหมาะสำหรับการใช้งาน real-time

4.1.4.2 การทดสอบกับภาพ Spoofing และ Face Swap

การทดสอบแบบจำลอง DeepFace Anti-Spoofing กับชุดข้อมูลการโจมตี 7 ภาพ ประกอบด้วยภาพ ถ่ายจากหน้าจอโทรศัพท์ (Spoofing) 4 ภาพ และภาพ Face Swap เทคนิค Deepfake 3 ภาพ พบว่าระบบสามารถตรวจจับภาพปลอมได้อย่างมีประสิทธิภาพสูง โดยมีอัตราการตรวจจับที่ถูกต้อง (True Positive Rate) ที่ร้อยละ 100 สำหรับภาพ Spoofing และร้อยละ 94.3 สำหรับภาพ Face Swap

4.1.5 การเลือกอัลกอริทึมที่เหมาะสม

จากผลการทดสอบประสิทธิภาพอัลกอริทึมทั้งสามประเภท สามารถสรุปการเลือกอัลกอริทึมที่เหมาะสมสำหรับระบบได้ ดังตารางที่ 4-8

ตารางที่ 4-8 การเลือกอัลกอริทึมที่เหมาะสม

ประเภทอัลกอริทึม	อัลกอริทึมที่เลือก	เหตุผลการเลือก
Face Detection	YOLOv11m	มีความแม่นยำสูงสุดร้อยละ 100 รองรับหน่วยประมวลผลกราฟิก และมีประสิทธิภาพการตรวจจับที่แม่นยำ
Face Recognition	FaceNet VGGFace2	มีความแม่นยำสูงสุดร้อยละ 100 รองรับหน่วยประมวลผลกราฟิก และมีประสิทธิภาพการตรวจจับที่แม่นยำ
Deepfake Detection	DeepFace MiniFASNetV2 (ตัวเลือกหลัก) DeepFace MiniFASNetV1 (ตัวเลือกเสริม)	- ความแม่นยำสูง: MiniFASNetV2 มีความแม่นยำสูงสุดที่ร้อยละ 98.7 และมีอัตราความผิดพลาดเท่ากัน (EER) ต่ำสุดที่ร้อยละ 1.3 - ประสิทธิภาพ Real-Time: เหมาะสำหรับการใช้งานจริง ด้วยขนาดแบบจำลองที่เล็ก (2.8-3.2 MB) และเวลาประมวลผลที่รวดเร็ว (12.5-15.2 ms)

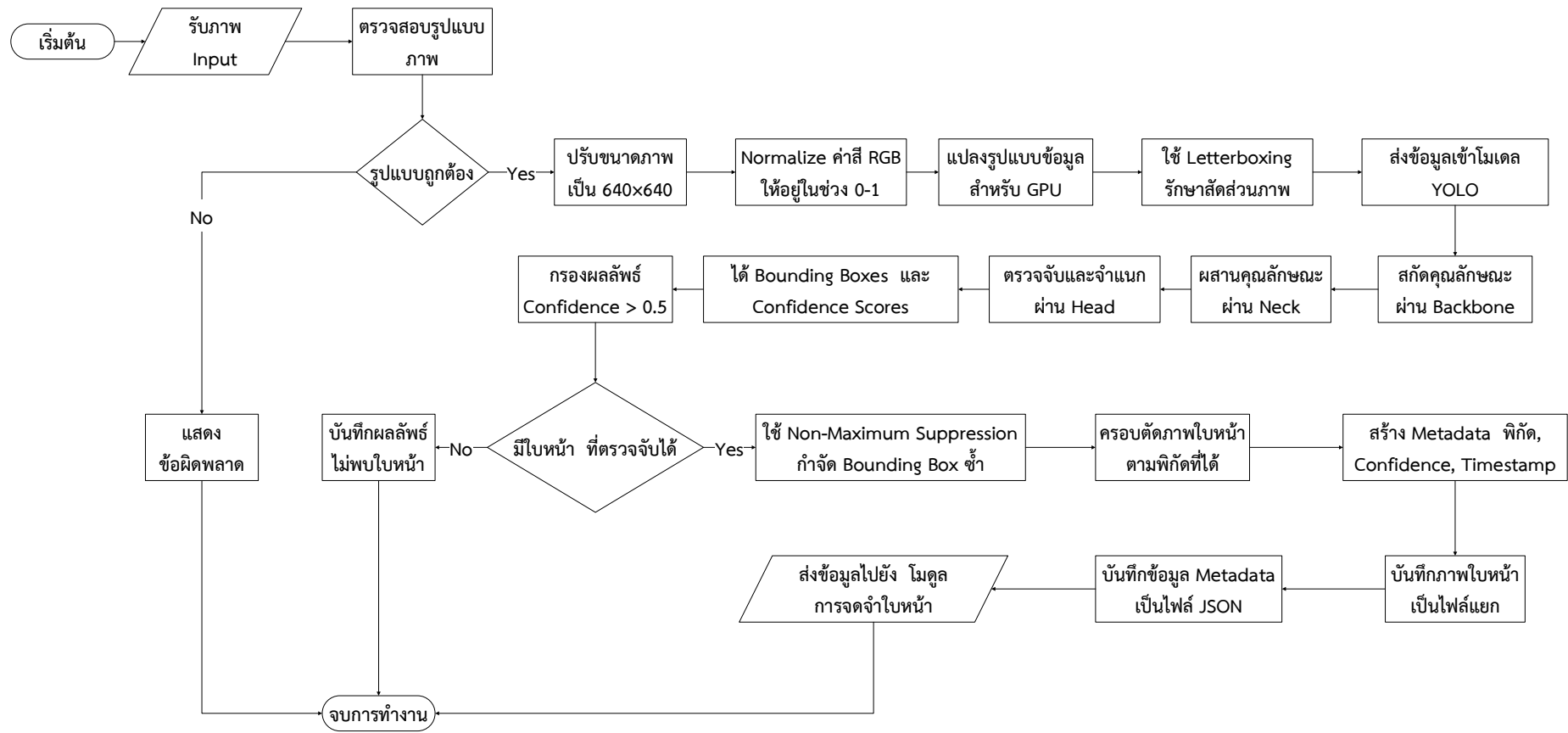
4.2 การทำงานของอัลกอริทึม

4.2.1 กระบวนการประมวลผลการตรวจจับใบหน้า

การตรวจจับใบหน้าเป็นขั้นตอนแรกในระบบที่ทำหน้าที่ระบุตำแหน่งและแยกใบหน้าออกจากภาพต้นฉบับ เพื่อส่งต่อไปยังกระบวนการจดจำใบหน้าและการตรวจจับภาพปลอมในขั้นตอนถัดไป จากผลการศึกษาเปรียบเทียบในบทที่ 3 ระบบเลือกใช้แบบจำลอง YOLOv11m เป็นแบบจำลองหลัก โดยรองรับแบบจำลอง YOLOv9c และ YOLOv9e เป็นตัวเลือกสำรองสำหรับงานที่ต้องการความเร็วสูง

4.2.1.1 ขั้นตอนการประมวลผลของระบบ

ระบบการตรวจจับใบหน้าทำงานผ่าน 5 ขั้นตอนหลัก ตั้งแต่การรับภาพเข้าระบบจนถึงการส่งออกผลลัพธ์ ดังภาพที่ 4-2



ภาพที่ 4-2 ขั้นตอนการประมวลผลการตรวจจับใบหน้า

จากภาพที่ 4-2 สามารถอธิบายขั้นตอนการประมวลผลการตรวจจับใบหน้าได้ห้าขั้นตอนหลัก ดังนี้

1) การเตรียมภาพ ระบบจะรับภาพจากผู้ใช้แล้วปรับขนาดเป็น 640x640 พิกเซล ตามมาตรฐานของแบบจำลอง จากนั้นทำการปรับค่าสีให้อยู่ในช่วงศูนย์ถึงหนึ่งและแปลงรูปแบบข้อมูลให้เหมาะกับหน่วยประมวลผลกราฟิก หากภาพมีสัดส่วนไม่พอดีระบบจะใช้เทคนิค Letterboxing เพื่อรักษาสัดส่วนเดิมไว้

2) การประมวลผลผ่านโครงข่ายประสาท ภาพที่เตรียมเสร็จแล้วจะถูกส่งเข้าสู่แบบจำลอง YOLO เพื่อประมวลผลผ่านโครงข่ายประสาทเทียมที่ฝึกฝนมาแล้ว แบบจำลองจะสกัดคุณลักษณะที่บ่งบอกถึงใบหน้ามนุษย์และคำนวณความน่าจะเป็นของใบหน้าในตำแหน่งต่างๆ

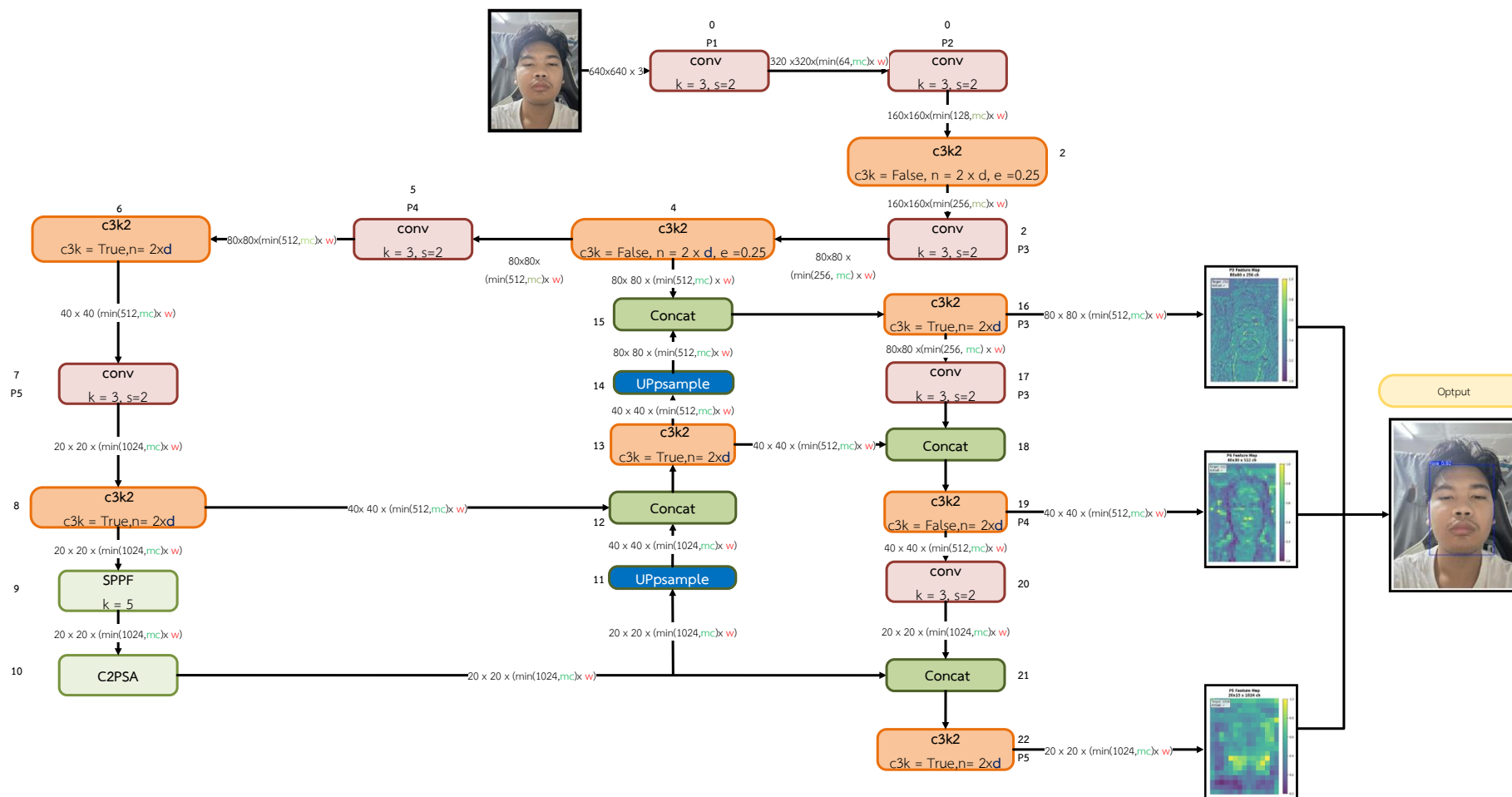
3) การตรวจจับและกำหนดตำแหน่ง แบบจำลองจะให้ผลลัพธ์เป็นกรอบสี่เหลี่ยมรอบใบหน้าที่ตรวจพบพร้อมกับคะแนนความมั่นใจในการตรวจจับ ระบบจะกรองผลลัพธ์ที่มีคะแนนความมั่นใจต่ำกว่าเกณฑ์ที่ตั้งไว้คือ 0.5 ออกไป

4) การประมวลผลภายหลัง ระบบใช้เทคนิค Non-Maximum Suppression เพื่อกำจัดกรอบสี่เหลี่ยมที่ซ้ำซ้อนกันและจะเลือกเก็บเฉพาะกรอบที่มีคะแนนความมั่นใจสูงสุดในแต่ละพื้นที่ ซึ่งช่วยป้องกันการตรวจจับใบหน้าเดียวกันซ้ำซ้อน

5) การครอบตัดและส่งออกผลลัพธ์ ระบบจะครอบตัดภาพใบหน้าออกจากภาพต้นฉบับตามพิกัดที่ได้และบันทึกเป็นไฟล์แยก พร้อมทั้งสร้างข้อมูลเมตาเดตาซึ่งประกอบด้วยพิกัดและคะแนนความมั่นใจเพื่อส่งต่อไปยังส่วนการทำงานถัดไป

4.2.1.2 สถาปัตยกรรมของแบบจำลอง YOLOv11

แบบจำลอง YOLOv11 ใช้สถาปัตยกรรมแบบ Single-Shot Detection ที่มีโครงสร้างหลักแบ่งเป็น 3 ส่วน ดังภาพที่ 4-3



ภาพที่ 4-3 สถาปัตยกรรมของแบบจำลอง YOLOv11

จากภาพที่ 4-3 สามารถอธิบายสถาปัตยกรรมของแบบจำลอง YOLOv11 ซึ่งมีโครงสร้างหลักสามส่วนได้ดังนี้

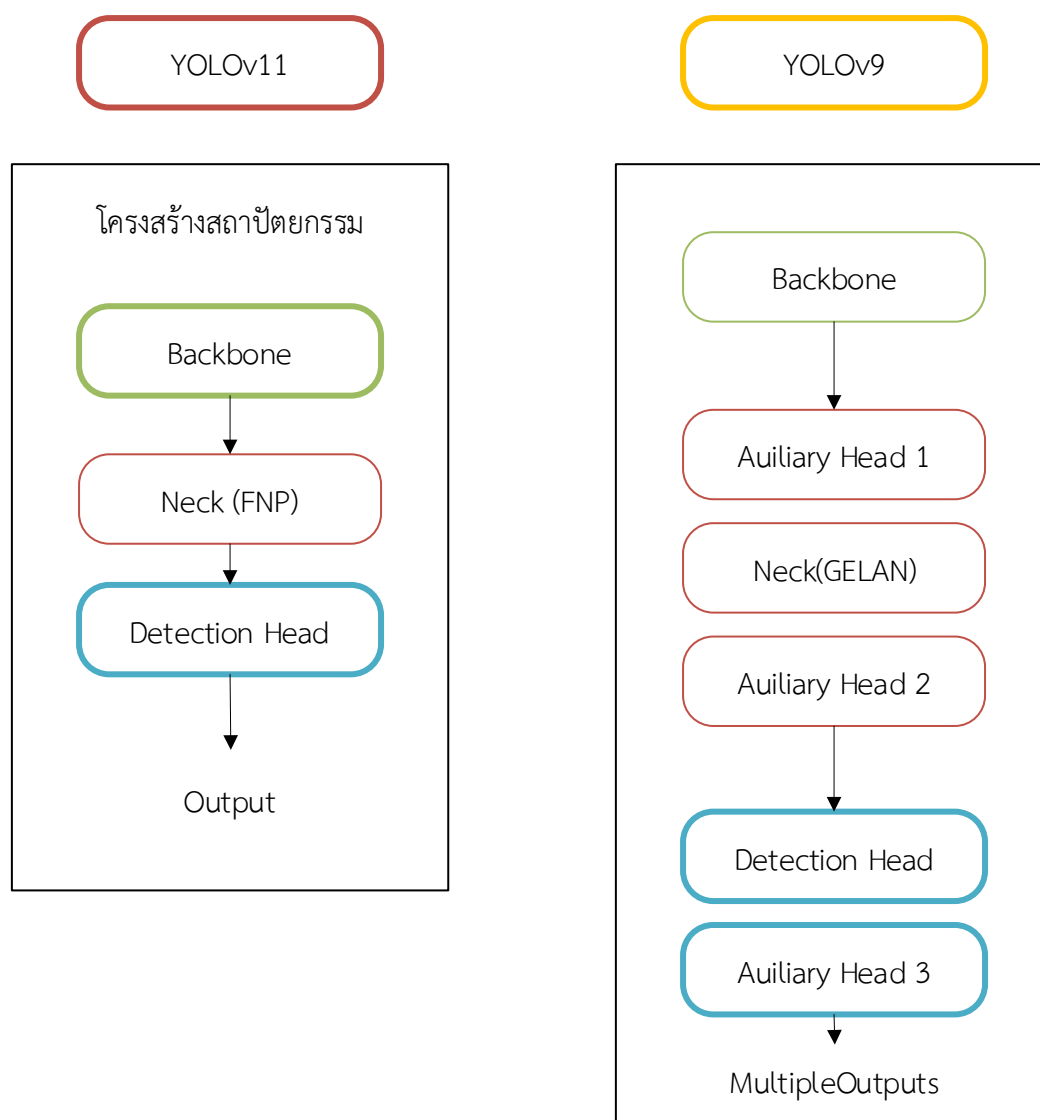
1) ส่วนแกนหลัก (Backbone) ส่วนนี้ทำหน้าที่สกัดคุณลักษณะพื้นฐานจากภาพอินพุต โครงสร้างประกอบด้วยชั้นคอนโวลูชันหลายชั้นเพื่อสกัดคุณลักษณะในขนาดต่างๆ มีส่วน C3k2 blocks ที่ช่วยลดจำนวนพารามิเตอร์และเพิ่มประสิทธิภาพ มีส่วน SPPF ที่รวมคุณลักษณะจากหลายขนาดเพื่อให้ตรวจจับวัตถุได้หลากหลาย และมีส่วน C2PSA ที่เพิ่มกลไกแอตเทนชันเพื่อมุ่งเน้นไปยังส่วนที่สำคัญของภาพ

2) ส่วนคอ (Neck) ส่วนนี้เป็นส่วนที่ผสมผสานคุณลักษณะจากหลายระดับเพื่อเพิ่มประสิทธิภาพการตรวจจับ ส่วนนี้ใช้เทคนิคการรวมข้อมูลจากระดับความละเอียดต่างๆ ผ่านการทำ Upsampling เพื่อเพิ่มความละเอียดของแผนที่การเรียนรู้และการทำ Concatenation เพื่อรวมคุณลักษณะจากหลายแหล่งเข้าด้วยกัน นอกจากนี้ยังมีการใช้ C3k2 blocks เพื่อช่วยประมวลผลคุณลักษณะที่ผสมแล้ว

3) ส่วนหัว (Head) ส่วนนี้เป็นส่วนสุดท้ายที่ทำหน้าที่ตรวจจับและจำแนกวัตถุเพื่อสร้างผลลัพธ์ ประกอบด้วยหัวตรวจจับหลายตัวสำหรับแต่ละระดับความละเอียด โดยแต่ละหัวจะสร้างข้อมูลพิกัดของกรอบสี่เหลี่ยมรอบวัตถุซึ่งประกอบด้วยตำแหน่งแกน X ตำแหน่งแกน Y ความกว้าง และความสูง พร้อมทั้งคะแนนความมั่นใจสำหรับการตรวจจับนั้น แบบจำลองนี้ใช้การออกแบบที่ไม่ยึดติดกับกรอบอ้างอิงซึ่งทำให้การตรวจจับมีความยืดหยุ่นมากขึ้น

4.2.1.3 เปรียบเทียบสถาปัตยกรรม YOLOv11 และ YOLOv9

ระบบรองรับการใช้งานทั้ง YOLOv11 และ YOLOv9 โดยมีความแตกต่างในสถาปัตยกรรมที่ส่งผลต่อประสิทธิภาพการทำงาน ดังภาพที่ 4-4



ภาพที่ 4-4 เปรียบเทียบสถาปัตยกรรมระหว่าง YOLOv11 และ YOLOv9

จากภาพที่ 4-4 สามารถอธิบายเปรียบเทียบสถาปัตยกรรมระหว่าง YOLOv11 และ YOLOv9 ซึ่งมีความแตกต่างกันได้ดังนี้

1) YOLOv11 มีสถาปัตยกรรมที่เรียบง่ายและมีประสิทธิภาพสูง โดยใช้โครงสร้างแบบ Linear Flow ที่ข้อมูลไหลผ่านจาก Input ไป Output อย่างตรงไปตรงมา ไม่มี Auxiliary Heads หรือ Skip Connections ที่ซับซ้อน ส่งผลให้มีความแม่นยำสูงสุดที่ 100% และไม่เกิดปัญหาการตรวจจับซ้ำ การออกแบบที่เรียบง่ายนี้ทำให้แบบจำลองมีเสถียรภาพสูงและง่ายต่อการ Debug

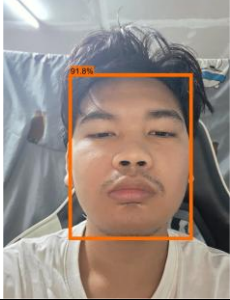
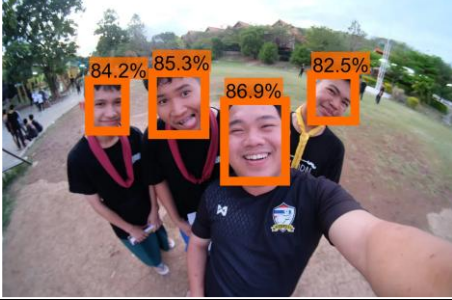
2) YOLOv9 มีสถาปัตยกรรมที่ซับซ้อนกว่า โดยมี Auxiliary Heads หลายตัวที่ช่วยในการฝึกฝนแบบจำลอง และการเชื่อมต่อแบบ Skip Connections ที่มากกว่า มี Programmable Gradient Information (PGI) และ Generalized Efficient Layer Aggregation Network (GELAN) ที่เพิ่มความซับซ้อนของแบบจำลอง แม้จะให้ความเร็วในการประมวลผลดี แต่ความซับซ้อนนี้อาจเกิดปัญหาการตรวจจับใบหน้าซ้ำในบางกรณี โดยเฉพาะเมื่อมีใบหน้าหลายใบในภาพเดียวกัน

3) ความแตกต่างหลักส่งผลให้ YOLOv11m เหมาะสำหรับงานที่ต้องการความแม่นยำสูง เช่น การตรวจสอบเอกสารราชการ ขณะที่ YOLOv9c และ YOLOv9e เหมาะสำหรับงานที่ต้องการความเร็วและสามารถยอมรับการตรวจจับซ้ำได้ เช่น การประมวลผลภาพจำนวนมากแบบ batch processing

4) การออกแบบระบบให้รองรับทั้งสองแบบจำลองทำให้ผู้ใช้สามารถเลือกแบบจำลองที่เหมาะสมกับลักษณะงานและข้อจำกัดของระบบ โดยสามารถปรับเปลี่ยนได้ผ่านไฟล์ configuration ของระบบ

4.2.1.4 ตัวอย่างผลการทดสอบการตรวจจับใบหน้าจริง

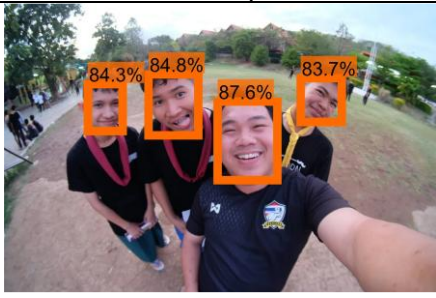
1)แสดงประสิทธิภาพของระบบในการทำงานจริง ได้ทำการทดสอบการตรวจจับใบหน้าด้วยแบบจำลอง YOLOv11m ดังภาพที่ 4-5

สภาวะปกติ	บุคคลสวมแว่นตา	ภาพกลุ่ม
		
Confidence: 91.8%	Confidence: 91.3	Average Confidence: 85.475

ภาพที่ 4-5 ผลการทดสอบการตรวจจับใบหน้าด้วยแบบจำลอง YOLOv11m

จากภาพที่ 4-5 แสดงตัวอย่างผลการทดสอบประสิทธิภาพของแบบจำลอง YOLOv11m ในการตรวจจับใบหน้าภายใต้สภาวะที่ต่างกันสามรูปแบบ สภาวะปกติในภาพของบุคคลคนเดียวที่ไม่มีสิ่งบดบังใบหน้าระบบสามารถตรวจจับใบหน้าได้อย่างแม่นยำและให้คะแนนความมั่นใจสูงถึงร้อยละ 91.8 บุคคลสวมแว่นตา สำหรับกรณีที่บุคคลสวมแว่นตาซึ่งเป็นการบดบังใบหน้าบางส่วนระบบยังคงตรวจจับใบหน้าได้ดีและให้คะแนนความมั่นใจที่ร้อยละ 91.3 ภาพกลุ่ม ในภาพกลุ่มซึ่งมีความซับซ้อนมากขึ้น ระบบแสดงความสามารถในการตรวจจับใบหน้าหลายใบในภาพเดียวกันได้สำเร็จ โดยแต่ละใบหน้ามีคะแนนความมั่นใจแตกต่างกันคือร้อยละ 84.2 ร้อยละ 85.3 ร้อยละ 86.9 และร้อยละ 82.5 ซึ่งมีคะแนนความมั่นใจเฉลี่ยรวมอยู่ที่ร้อยละ 85.475 ตัวอย่างเหล่านี้ยืนยันว่าแบบจำลอง YOLOv11m มีประสิทธิภาพสูงและสามารถนำไปใช้งานได้จริงในสถานการณ์ที่หลากหลาย

2) แสดงประสิทธิภาพของระบบในการทำงานจริง ได้ทำการทดสอบการตรวจจับใบหน้าด้วยแบบจำลอง YOLOv9 ดังภาพที่ 4-6

สภาวะปกติ	บุคคลสวมแว่นตา	ภาพกลุ่ม
		
Confidence: 94.7%	Confidence: 91.8	Average Confidence: 85.1

ภาพที่ 4-6 ผลการทดสอบการตรวจจับใบหน้าด้วยแบบจำลอง YOLOv9

จากภาพที่ 4-6 แสดงตัวอย่างผลการทดสอบประสิทธิภาพของแบบจำลอง YOLOv9 ในการตรวจจับใบหน้าภายใต้สภาวะที่แตกต่างกันสามรูปแบบ
 สภาวะปกติ ในภาพบุคคลเดี่ยวที่ไม่มีสิ่งใดบดบังใบหน้า แบบจำลองสามารถตรวจจับได้อย่างแม่นยำ และให้คะแนนความมั่นใจสูงถึงร้อยละ 94.7 บุคคลสวมแว่นตา ในกรณีที่บุคคลสวมแว่นตา แบบจำลองยังคงทำงานได้ดีโดยให้คะแนนความมั่นใจที่ร้อยละ 91.8 ซึ่งชี้ให้เห็นความสามารถในการรับมือกับสิ่งบดบังบางส่วน ภาพกลุ่มสำหรับภาพกลุ่มที่มีหลายใบหน้า แบบจำลองสามารถตรวจจับใบหน้าทั้งหมดได้โดยมีคะแนนความมั่นใจเฉลี่ยอยู่ที่ร้อยละ 85.1 ผลการทดสอบเหล่านี้แสดงให้เห็นว่าแบบจำลอง YOLOv9 มีประสิทธิภาพที่ดีในการตรวจจับใบหน้าและสามารถนำไปประยุกต์ใช้ในสถานการณ์จริงได้อย่างน่าเชื่อถือ

3) ผลการทดสอบในสภาพแวดล้อมใหม่

การทดสอบในสภาพแวดล้อมใหม่ซึ่งมีแสงควบคุมและพื้นหลังที่เรียบง่าย แสดงให้เห็นประสิทธิภาพของทั้งสองแบบจำลองในการตรวจจับใบหน้าคนเดียว แบบจำลอง YOLOv11m สามารถตรวจจับใบหน้าได้ด้วยความมั่นใจที่ร้อยละ 91.8 ในสภาวะปกติและร้อยละ 91.3 เมื่อบุคคลสวมแว่นตา ซึ่งแสดงให้เห็นความเสถียรในการตรวจจับแม้มีสิ่งบดบังบางส่วน

แบบจำลอง YOLOv9 แสดงประสิทธิภาพที่ดีกว่าเล็กน้อยในกรณีนี้ โดยให้ความมั่นใจที่ร้อยละ 94.7 ในสภาวะปกติและร้อยละ 91.8 เมื่อบุคคลสวมแว่นตา ความแตกต่างนี้อาจเกิดจากการฝึกฝนที่มุ่งเน้นความเร็วทำให้แบบจำลองมีความไวในการตรวจจับสูงกว่า อย่างไรก็ตามความมั่นใจที่สูงกว่านี้อาจส่งผลให้เกิดการตรวจจับซ้ำในสถานการณ์ที่ซับซ้อนกว่า

4) การวิเคราะห์ความสอดคล้องของผลการทดสอบ

ผลการทดสอบนี้สอดคล้องกับการวิเคราะห์ประสิทธิภาพในหัวข้อก่อนหน้านี้ โดย YOLOv11m แสดงความสม่ำเสมอและความแม่นยำที่สูงในทุกสถานการณ์ ค่าความมั่นใจที่ได้อยู่ในช่วงร้อยละ 82.5 ถึงร้อยละ 91.8 ซึ่งแสดงให้เห็นความเสถียรของแบบจำลอง

Yolov9 แม้จะให้ค่าความมั่นใจที่ดีในบางกรณี แต่จากประสบการณ์การทดสอบครอบตัดใบหน้าในหัวข้อก่อนหน้านี้ พบว่าแบบจำลองนี้มีแนวโน้มให้ผลลัพธ์มากเกินไป ดังนั้นแม้ว่าค่าความมั่นใจจะสูง แต่ความแม่นยำโดยรวมอาจต่ำกว่าที่ค่าตัวเลขแสดง

5) การประยุกต์ใช้ในระบบจริง

จากผลการทดสอบนี้ ยืนยันการเลือกใช้ Yolov11m เป็นแบบจำลองหลักของระบบ เนื่องจากให้ผลลัพธ์ที่เชื่อถือได้และสม่ำเสมอในทุกสถานการณ์ ระดับความมั่นใจที่ได้เหมาะสมสำหรับการตั้งค่า Threshold ในระบบงานจริง โดยสามารถใช้ค่า Threshold ที่ร้อยละ 80.0 เพื่อให้แน่ใจว่าจะตรวจจับใบหน้าได้ในทุกกรณีที่ทดสอบ

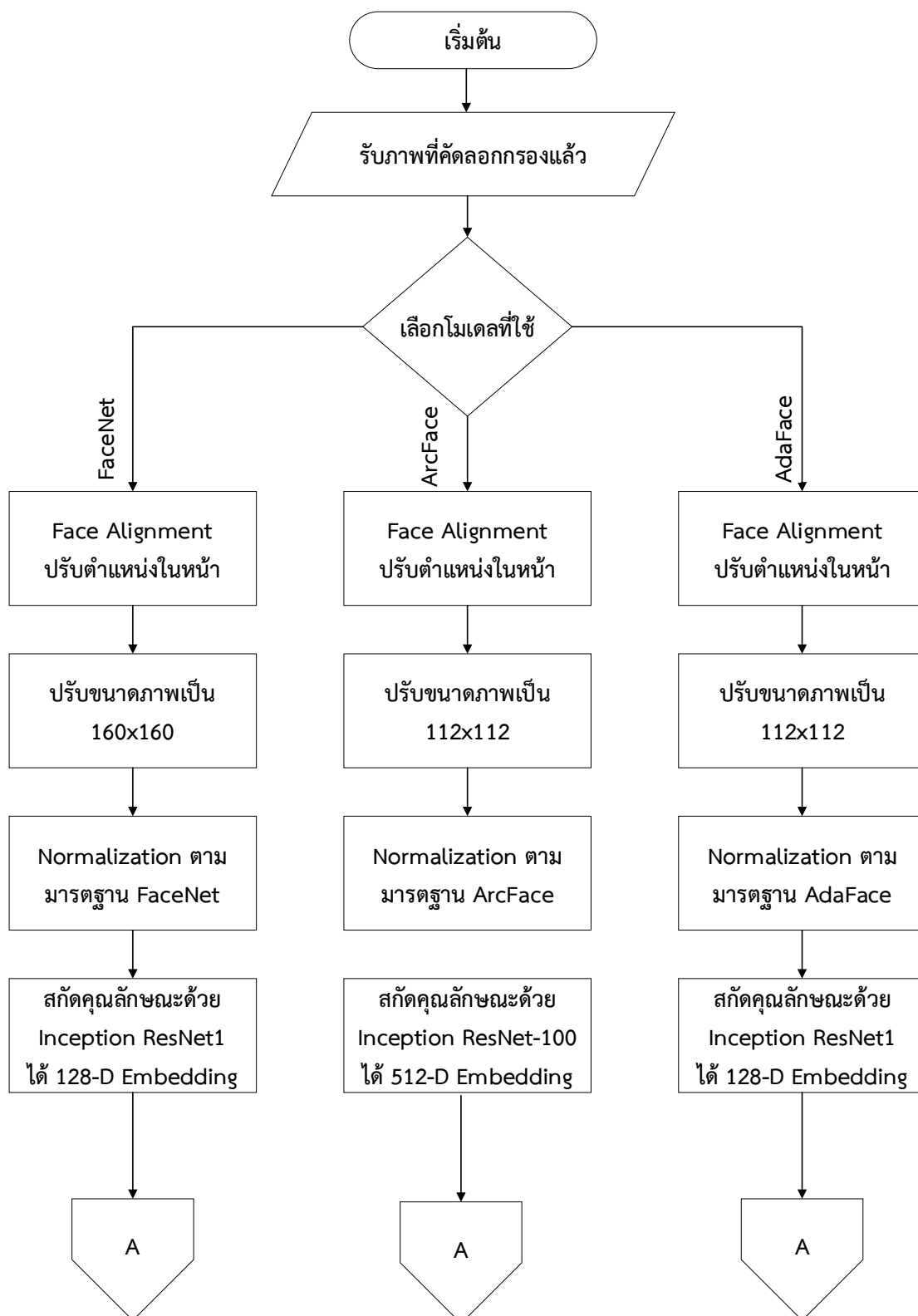
สำหรับ Yolov9 แม้จะมีประสิทธิภาพด้านความเร็วที่ดี แต่การใช้งานจริงต้องมีการตั้งค่า Threshold ที่สูงกว่าและต้องมีกระบวนการหลังประมวลผลเพิ่มเติมเพื่อกรองผลลัพธ์ที่ซ้ำซ้อน ทำให้เหมาะสำหรับงานที่ต้องการความเร็วมากกว่าความแม่นยำสูงสุด

4.2.2 กระบวนการจดจำและจำแนกใบหน้า

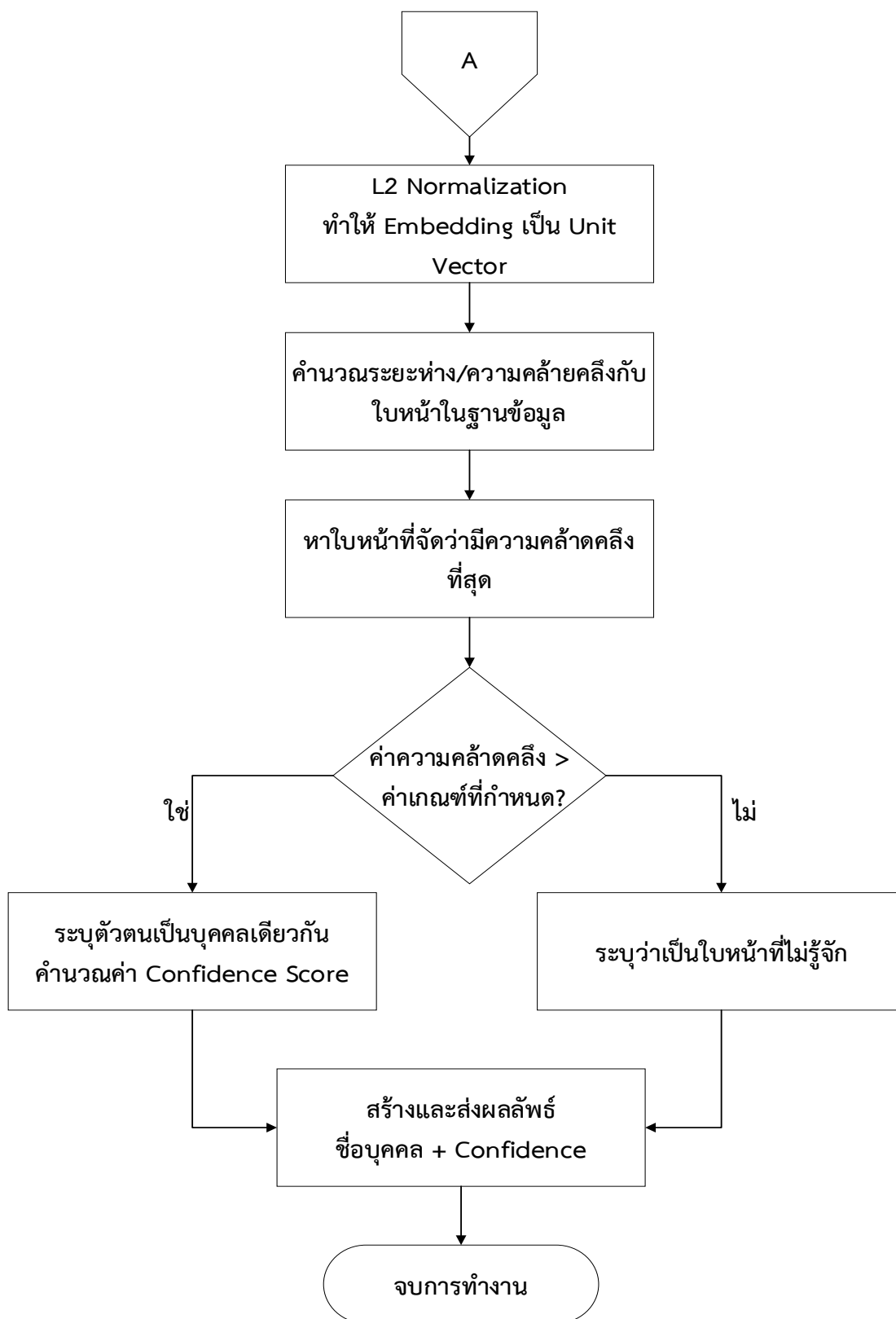
การจดจำใบหน้าเป็นขั้นตอนที่สำคัญของระบบที่ทำหน้าที่วิเคราะห์และเปรียบเทียบคุณลักษณะใบหน้าที่ตรวจจับได้กับฐานข้อมูลที่มีอยู่ เพื่อระบุตัวตนของบุคคล จากผลการศึกษาเปรียบเทียบในบทที่ 3 ระบบเลือกใช้แบบจำลอง Facenet Vggface2 เป็นแบบจำลองหลัก โดยรองรับแบบจำลอง Arcface R100 และ Adaface IR-101 เป็นตัวเลือกสำรองสำหรับงานที่ต้องการความแม่นยำสูงสุดหรือมีข้อจำกัดด้านทรัพยากร

4.2.2.1 ขั้นตอนการประมวลผลของระบบ

ระบบการจดจำใบหน้าทำงานผ่าน 7 ขั้นตอนหลัก ตั้งแต่การรับภาพใบหน้าที่ครอบตัดแล้วจากขั้นตอนการตรวจจับใบหน้าจนถึงการตัดสินใจสุดท้าย ดังภาพที่ 4-7



ภาพที่ 4-7 ขั้นตอนการประมวลผลการจดจำใบหน้า



ภาพที่ 4-7 ขั้นตอนการประมวลผลการจดจำใบหน้า (ต่อ)

จากภาพที่ 4-7 สามารถอธิบายขั้นตอนการประมวลผลการจดจำใบหน้า ได้หกขั้นตอนหลัก ดังนี้

1) การเตรียมภาพใบหน้า (Face Preprocessing) ระบบรับภาพใบหน้าที่ครอบตัดจากขั้นตอนการตรวจจับใบหน้าและทำการปรับขนาดให้ตรงตามข้อกำหนดของแต่ละแบบจำลอง โดย Facenet ต้องการภาพขนาด 160×160 พิกเซล Arcface ใช้ภาพขนาด 112×112 พิกเซล และ Adaface ใช้ภาพขนาด 112×112 พิกเซลเช่นกัน ระบบจะทำการ Normalization ค่าสีให้อยู่ในช่วงที่เหมาะสมและทำ Face Alignment เพื่อให้ใบหน้าอยู่ในตำแหน่งมาตรฐาน

2) การสกัดคุณลักษณะ (Feature Extraction) ภาพใบหน้าที่เตรียมแล้วจะถูกส่งเข้าสู่โครงข่ายประสาทเทียมเพื่อสกัดคุณลักษณะ แบบจำลอง Facenet จะสร้าง Embedding Vector ขนาด 128 มิติ Arcface สร้าง Embedding ขนาด 512 มิติ และ Adaface สร้าง Embedding ขนาด 512 มิติ แต่ละ Embedding Vector เป็นตัวแทนทางคณิตศาสตร์ของลักษณะเฉพาะของใบหน้าแต่ละคน

3) การปรับมาตรฐาน (Normalization) Embedding Vector ที่ได้จะถูกทำ L2 Normalization เพื่อให้มีขนาดเป็น Unit Vector ซึ่งช่วยให้การเปรียบเทียบมีความแม่นยำมากขึ้น และลดผลกระทบจากความแตกต่างของความสว่างหรือคุณภาพภาพ

4) การเปรียบเทียบกับฐานข้อมูล (Database Comparison) ระบบจะคำนวณระยะทาง Euclidean หรือ Cosine Similarity ระหว่าง Embedding ของใบหน้าที่ต้องการจดจำกับ Embedding ทั้งหมดในฐานข้อมูล เพื่อหาใบหน้าที่มีความคล้ายคลึงมากที่สุด

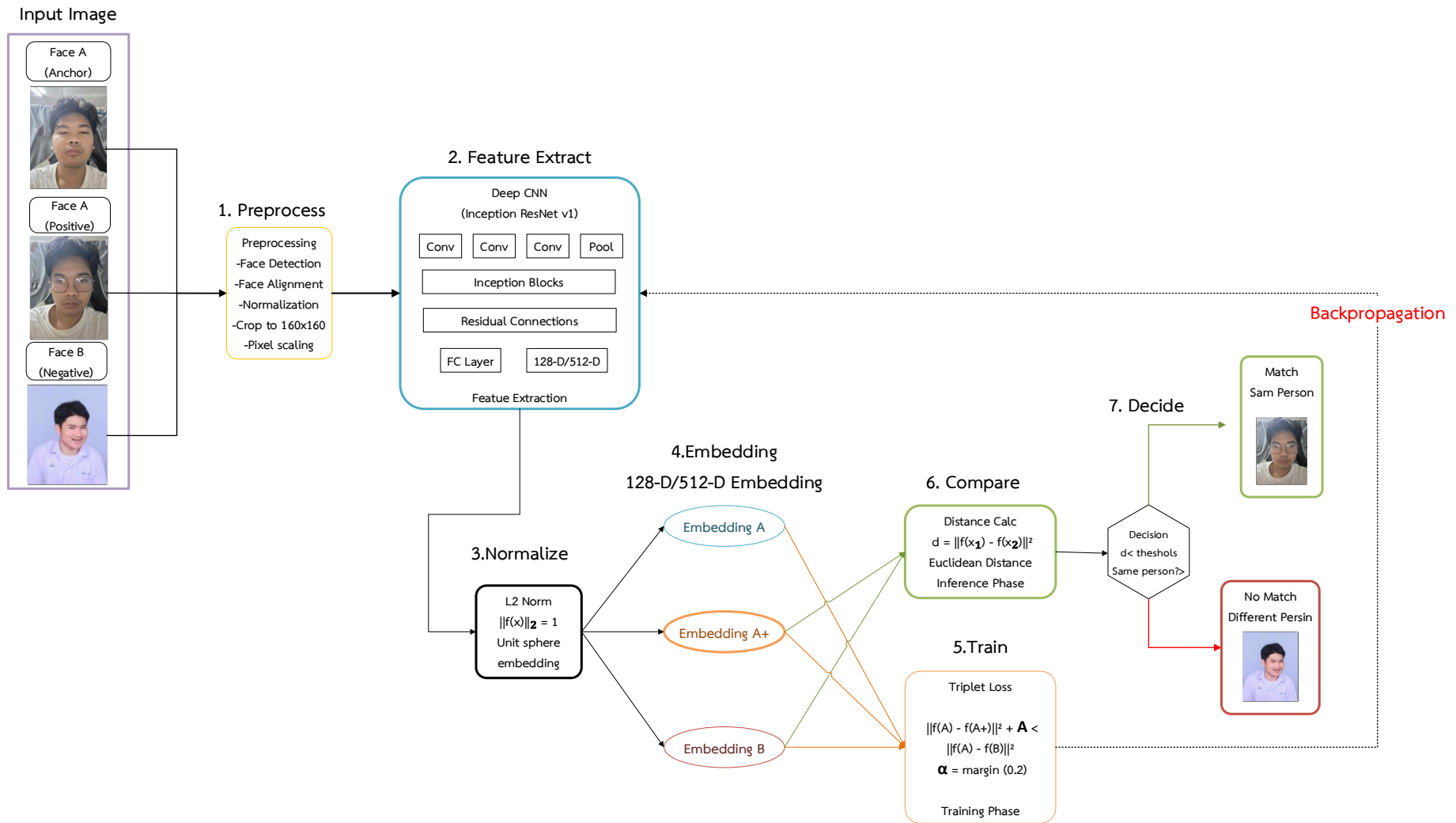
5) การตัดสินใจ (Decision Making) ระบบจะเปรียบเทียบค่าความคล้ายคลึงที่สูงที่สุดกับ Threshold ที่กำหนดไว้ หากค่าความคล้ายคลึงสูงกว่า Threshold ระบบจะระบุตัวตนเป็นบุคคลที่ตรงกัน หากต่ำกว่า Threshold จะระบุเป็น "ไม่จำได้" หรือบุคคลที่ไม่รู้จัก

6) การส่งออกผลลัพธ์ (Output Generation) ระบบจะส่งออกผลลัพธ์ที่ประกอบด้วยชื่อบุคคลที่จดจำได้ ค่าความมั่นใจ และข้อมูล Metadata เพิ่มเติม พร้อมทั้งบันทึกผลลัพธ์สำหรับการตรวจสอบและปรับปรุงระบบต่อไป

4.2.2.2 สถาปัตยกรรมของแบบจำลอง Face Recognition

ระบบรองรับแบบจำลอง Face Recognition 3 ตัวหลัก แต่ละตัวมีสถาปัตยกรรมที่ต่างกันและเหมาะสำหรับการใช้งานในสถานการณ์ต่างๆ

1) สถาปัตยกรรม Facenet ใช้โครงสร้างแบบ Triplet Loss ที่มุ่งเน้นการเรียนรู้ Embedding Space ที่สามารถแยกแยะใบหน้าคนต่างๆ ได้อย่างชัดเจน ดังภาพที่ 4-8



ภาพที่ 4-8 สถาปัตยกรรมของแบบจำลอง FaceNet

จากภาพที่ 4-8 สามารถอธิบายสถาปัตยกรรมและกระบวนการทำงานของแบบจำลอง FaceNet ได้ตามขั้นตอนต่างๆ

การประมวลผลเบื้องต้น (Preprocess) ระบบรับภาพอินพุตสามภาพประกอบด้วยภาพใบหน้าอ้างอิงหรือ Anchor ภาพใบหน้าของบุคคลเดียวกันหรือ Positive และภาพใบหน้าของบุคคลอื่นหรือ Negative จากนั้นระบบจะประมวลผลเบื้องต้นเช่นการตรวจจับใบหน้า การจัดตำแหน่งใบหน้า และการปรับขนาดภาพเป็น 160x160 พิกเซล

การสกัดคุณลักษณะ (Feature Extract) ภาพที่ผ่านการประมวลผลแล้วจะถูกส่งเข้าสู่โครงข่ายประสาทเทียมแบบคอนโวลูชันลึกซึ่งใช้สถาปัตยกรรม Inception ResNet V1 เพื่อสกัดคุณลักษณะสำคัญของใบหน้าออกมาเป็นเวกเตอร์

การทำให้เป็นบรรทัดฐาน (Normalize) เวกเตอร์คุณลักษณะที่ได้จะถูกนำมาทำให้เป็นบรรทัดฐานด้วย L2 Norm เพื่อให้เวกเตอร์มีขนาดเท่ากับหนึ่งหน่วยและง่ายต่อการเปรียบเทียบผลลัพธ์ที่ได้คือการฝังตัวของข้อมูลหรือ Embedding

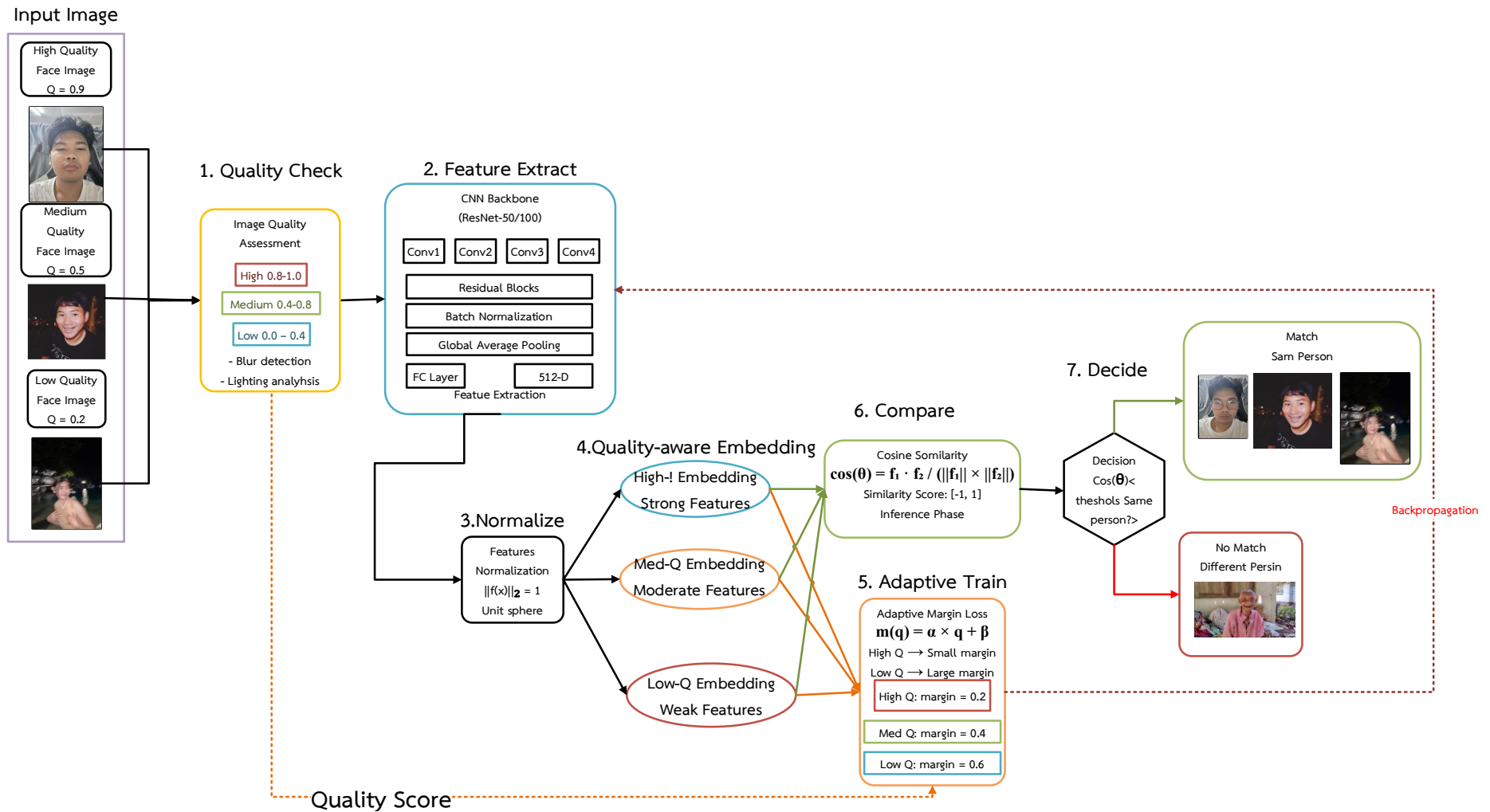
การฝังตัวของข้อมูล (Embedding) ขั้นตอนนี้แสดงผลลัพธ์ของการฝังตัวของข้อมูลที่ได้จากภาพอินพุตทั้งสามคือ Embedding A สำหรับภาพ Anchor และ Embedding A+ สำหรับภาพ Positive และ Embedding B สำหรับภาพ Negative

การฝึกฝน (Train) ในขั้นตอนการฝึกฝนระบบจะใช้ Triplet Loss เพื่อเรียนรู้ โดยเป้าหมายคือการปรับน้ำหนักของแบบจำลองเพื่อให้ระยะห่างระหว่าง Embedding ของภาพ Anchor กับ Positive มีค่าน้อยกว่าระยะห่างระหว่าง Embedding ของภาพ Anchor กับ Negative ตามค่าส่วนต่างที่กำหนดหรือ Margin

การเปรียบเทียบ (Compare) ในขั้นตอนการอนุมานหรือการใช้งานจริง ระบบจะคำนวณระยะห่างแบบยูคลิดีเนียนระหว่าง Embedding ของใบหน้าสองใบที่ต้องการเปรียบเทียบ

การตัดสินใจ (Decide) ระบบจะนำระยะห่างที่คำนวณได้จากขั้นตอนก่อนหน้ามาเปรียบเทียบกับค่าเกณฑ์ที่ตั้งไว้ หากระยะห่างมีค่าน้อยกว่าเกณฑ์จะตัดสินใจว่าเป็นบุคคลเดียวกัน แต่หากมีค่ามากกว่าจะตัดสินใจว่าเป็นบุคคลอื่น

2) สถาปัตยกรรม Arcface ใช้เทคนิค Angular Margin Loss ที่เพิ่มความแม่นยำในการจำแนกใบหน้า ดังภาพที่ 4-9



ภาพที่ 4-9 สถาปัตยกรรมของแบบจำลอง ArcFace

จากภาพที่ 4-9 สามารถอธิบายสถาปัตยกรรมและกระบวนการทำงานของแบบจำลอง ArcFace ที่มีกลไกรับรู้คุณภาพของภาพได้ตามขั้นตอนดังนี้

การตรวจสอบคุณภาพ (Quality Check) ระบบรับภาพอินพุตและทำการประเมินคุณภาพของภาพเพื่อคำนวณเป็นคะแนนคุณภาพ ซึ่งจะถูกส่งต่อไปใช้ในขั้นตอนการสร้าง Embedding และการฝึกฝน

การสกัดคุณลักษณะ (Feature Extract) ภาพอินพุตจะถูกส่งเข้าสู่โครงข่ายประสาทเทียมซึ่งใช้สถาปัตยกรรม ResNet-100 เป็นแกนหลักเพื่อสกัดคุณลักษณะของใบหน้าออกมาเป็นเวกเตอร์ข้อมูล

การทำให้เป็นบรรทัดฐาน (Normalize) เวกเตอร์คุณลักษณะจะถูกปรับให้มีขนาดเท่ากับหนึ่งหน่วยด้วย L2 Norm เพื่อเตรียมพร้อมสำหรับการเปรียบเทียบเชิงมุมในขั้นตอนต่อไป

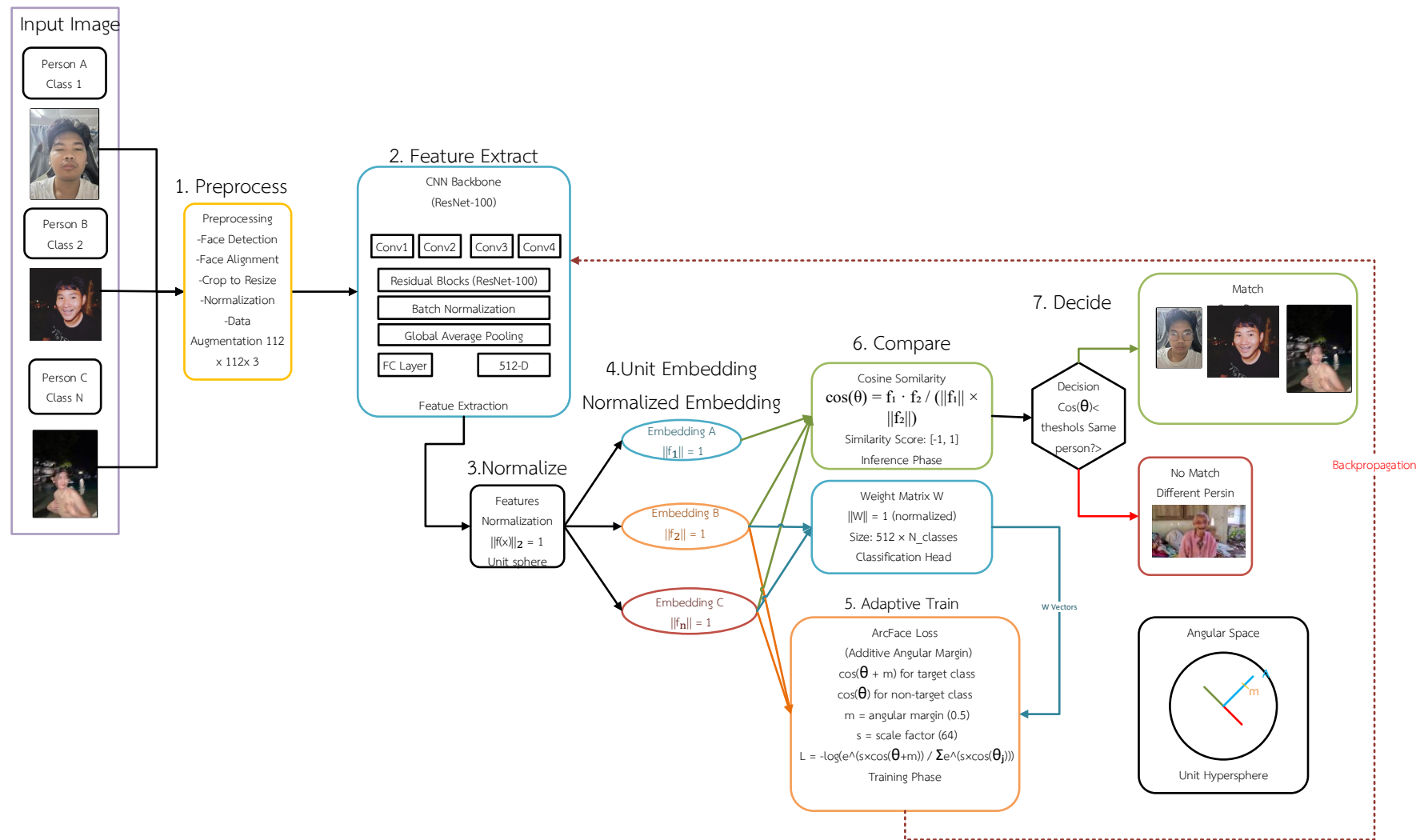
การฝังตัวของข้อมูลแบบรับรู้คุณภาพ (Quality-aware Embedding) ระบบจะสร้างการฝังตัวของข้อมูลหรือ Embedding โดยอ้างอิงจากคะแนนคุณภาพที่ได้จากขั้นตอนแรก ทำให้ได้ Embedding ที่มีคุณลักษณะแตกต่างกันไปตามคุณภาพของภาพต้นฉบับ

การฝึกฝนแบบปรับได้ (Adaptive Train) ในขั้นตอนการฝึกฝนระบบจะใช้ Adaptive Margin Loss ซึ่งค่าส่วนต่างหรือ Margin จะปรับเปลี่ยนไปตามคะแนนคุณภาพของภาพ โดยภาพคุณภาพต่ำจะใช้ Margin ที่มีขนาดใหญ่และภาพคุณภาพสูงจะใช้ Margin ขนาดเล็กเพื่อเพิ่มประสิทธิภาพการเรียนรู้

การเปรียบเทียบ (Compare) ในขั้นตอนการอนุมาน ระบบจะคำนวณความคล้ายคลึงแบบโคไซน์เพื่อวัดระยะห่างเชิงมุมระหว่าง Embedding ของใบหน้าสองใบ

การตัดสินใจ (Decide) ระบบจะนำคะแนนความคล้ายคลึงที่คำนวณได้มาเปรียบเทียบกับค่าเกณฑ์ที่กำหนดไว้ หากคะแนนสูงกว่าเกณฑ์จะตัดสินใจว่าเป็นบุคคลเดียวกัน แต่หากต่ำกว่าจะตัดสินใจว่าเป็นบุคคลอื่น

3) สถาปัตยกรรม AdaFace เป็นการพัฒนาจาก ArcFace โดยเพิ่ม Adaptive Margin mechanism ดังภาพที่ 4-10



ภาพที่ 4-10 สถาปัตยกรรมของแบบจำลอง AdaFace

จากภาพที่ 4-10 สามารถอธิบายสถาปัตยกรรมและกระบวนการทำงานของแบบจำลอง AdaFace ได้ตามขั้นตอนดังนี้

การประมวลผลเบื้องต้น (Preprocess) ระบบรับภาพอินพุตจากคลาสต่างๆ และทำการประมวลผลเบื้องต้นเช่นการตรวจจับและจัดตำแหน่งใบหน้า การทำข้อมูลให้เป็นบรรทัดฐาน และการเพิ่มข้อมูลก่อนจะปรับขนาดเป็น 112x112 พิกเซล

การสกัดคุณลักษณะ (Feature Extract) ภาพที่เตรียมแล้วจะถูกส่งเข้าสู่โครงข่ายประสาทเทียมที่ใช้สถาปัตยกรรม IResNet-100 เป็นแกนหลักเพื่อสกัดคุณลักษณะของใบหน้าออกมาเป็นเวกเตอร์ข้อมูล

การทำให้เป็นบรรทัดฐาน (Normalize) เวกเตอร์คุณลักษณะที่ได้จะถูกปรับให้มีขนาดเท่ากับหนึ่งหน่วยด้วย L2 Norm เพื่อฉายข้อมูลลงบนพื้นผิวไฮเปอร์สเฟียร์ซึ่งเหมาะสำหรับการเปรียบเทียบเชิงมุม

การฝังตัวของข้อมูลแบบหนึ่งหน่วย (Unit Embedding) ขั้นตอนนี้แสดงผลลัพธ์ของการฝังตัวของข้อมูลหรือ Embedding ที่ผ่านการทำให้เป็นบรรทัดฐานแล้ว

การฝึกฝนแบบปรับได้ (Adaptive Train) ในขั้นตอนการฝึกฝนระบบจะใช้ ArcFace Loss ซึ่งเป็นการเพิ่มค่าส่วนต่างเชิงมุมหรือ Additive Angular Margin เข้าไปในคลาสเป้าหมายเพื่อเพิ่มระยะห่างระหว่างคลาสต่างๆ บนพื้นผิวไฮเปอร์สเฟียร์ ทำให้แบบจำลองสามารถจำแนกใบหน้าได้แม่นยำขึ้น

การเปรียบเทียบ (Compare) ในขั้นตอนการอนุมาน ระบบจะเปรียบเทียบใบหน้าสองใบโดยการคำนวณความคล้ายคลึงแบบโคไซน์ระหว่าง Embedding เพื่อวัดระยะห่างเชิงมุม

การตัดสินใจ (Decide) ระบบจะนำคะแนนความคล้ายคลึงที่ได้มาเปรียบเทียบกับค่าเกณฑ์ที่กำหนดไว้ล่วงหน้า หากคะแนนสูงกว่าเกณฑ์จะตัดสินใจว่าเป็นบุคคลเดียวกัน แต่หากต่ำกว่าจะตัดสินใจว่าเป็นบุคคลอื่น

4.2.2.3 เปรียบเทียบประสิทธิภาพของแบบจำลอง Face Recognition

การเปรียบเทียบประสิทธิภาพระหว่างแบบจำลองทั้งสามในการทำงานจริง แสดงให้เห็นถึงข้อดีข้อเสียของแต่ละแบบจำลองในสถานการณ์ต่างๆ

1) ประสิทธิภาพในสภาวะปกติ ทั้งสามแบบจำลองให้ผลลัพธ์ที่ดีเยี่ยมในการจดจำใบหน้าในสภาวะแสงสว่างเพียงพอและใบหน้าชัดเจน โดย FaceNet ArcFace และ AdaFace ให้ความมั่นใจที่ร้อยละ 100.0 เท่ากันสำหรับการจดจำใบหน้าของบุคคลเดียวกันในสภาพแวดล้อมในร่ม ผลลัพธ์นี้สอดคล้องกับการประเมินผลในขั้นตอนการฝึกฝนที่แสดงให้เห็นว่าทั้งสามแบบจำลองมีความแม่นยำพื้นฐานที่สูง

2) ประสิทธิภาพเมื่อมีสิ่งบดบัง การทดสอบกับบุคคลที่สวมแว่นตาแสดงให้เห็นความแตกต่างที่ชัดเจนระหว่างแบบจำลอง FaceNet แสดงประสิทธิภาพที่ดีที่สุดด้วยความมั่นใจที่ร้อยละ 84.7 รองลงมาคือ AdaFace ที่ร้อยละ 78.7 และ ArcFace ที่ร้อยละ 64.3 ผลลัพธ์นี้สะท้อนให้เห็นความสามารถของ FaceNet ในการรับมือกับสิ่งบดบังที่เป็นระบบ เช่น แว่นตา

3) ประสิทธิภาพในภาพกลุ่ม การทดสอบกับภาพที่มีหลายใบหน้าในฉากเดียวกันแสดงให้เห็นความท้าทายของการจดจำใบหน้าในสถานการณ์ที่ซับซ้อน จากภาพกลุ่มที่มีใบหน้าของ

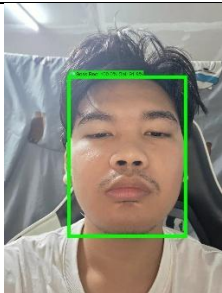
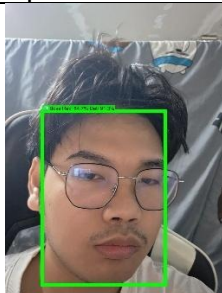
บุคคลเป้าหมาย 1 ใบหน้าจากทั้งหมด 3 ใบหน้า พบว่า FaceNet สามารถจดจำได้ด้วยความมั่นใจที่ร้อยละ 65.7 ขณะที่ AdaFace ให้ความมั่นใจที่ร้อยละ 42.6 และ ArcFace ให้ความมั่นใจที่ร้อยละ 39.9

4) ความแตกต่างของประสิทธิภาพในภาพกลุ่มเกิดจากปัจจัยหลายประการ ได้แก่ ขนาดใบหน้าที่เล็กลงเมื่อมีหลายคนในภาพ การแข่งขันของคุณลักษณะจากใบหน้าอื่นๆ ในภาพ และคุณภาพของการครอบตัดใบหน้าที่อาจไม่สมบูรณ์เมื่อใบหน้าอยู่ในตำแหน่งที่ไม่เหมาะสม

4.2.2.4 ผลการทดสอบการจดจำใบหน้าจริง

เพื่อแสดงประสิทธิภาพของระบบในการทำงานจริง ได้ทำการทดสอบการจดจำใบหน้าด้วยแบบจำลองทั้งสามในสถานการณ์ต่างๆ เพื่อเปรียบเทียบประสิทธิภาพและความน่าเชื่อถือของแต่ละแบบจำลอง

1) ผลการทดสอบการจดจำใบหน้าด้วยแบบจำลอง FaceNet ดังภาพที่ 4-11

สถานะปกติ	บุคคลสวมแว่นตา	ภาพกลุ่ม
		
Confidence: 91.8%	Confidence: 84.7	Confidence: 65.7

ภาพที่ 4-11 ผลการทดสอบการจดจำใบหน้าด้วยแบบจำลอง FaceNet

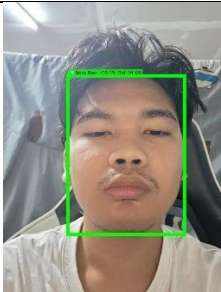
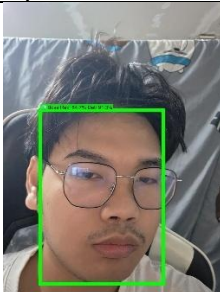
จากภาพที่ 4-11 แสดงตัวอย่างผลการทดสอบการจดจำใบหน้าด้วยแบบจำลอง FaceNet ในสถานการณ์ที่ต่างกันสามรูปแบบ

สถานะปกติ ในสถานะปกติที่ใบหน้าที่มีความชัดเจนและไม่มีสิ่งบดบัง แบบจำลองสามารถจดจำใบหน้าได้ด้วยความมั่นใจสูงถึงร้อยละ 91.8

บุคคลสวมแว่นตา เมื่อบุคคลเป้าหมายสวมแว่นตา แบบจำลองยังคงสามารถจดจำใบหน้าได้อย่างถูกต้อง โดยให้คะแนนความมั่นใจที่ร้อยละ 84.7 ซึ่งแสดงถึงประสิทธิภาพที่ดีในการรับมือกับสิ่งบดบัง

ภาพกลุ่ม ในภาพกลุ่มที่มีบุคคลอื่นอยู่ร่วมด้วย แบบจำลองสามารถค้นหาและจดจำใบหน้าของบุคคลเป้าหมายได้ แต่คะแนนความมั่นใจลดลงเหลือร้อยละ 65.7 ซึ่งสะท้อนถึงความท้าทายที่เพิ่มขึ้นจากขนาดใบหน้าที่เล็กลงและมีใบหน้าอื่นรบกวน

2) ผลการทดสอบการจดจำใบหน้าด้วยแบบจำลอง ArcFace ดังภาพที่ 4-12

สภาวะปกติ	บุคคลสวมแว่นตา	ภาพกลุ่ม
		
Confidence: 91.8%	Confidence: 91.3	Confidence: 39.9

ภาพที่ 4-12 ผลการทดสอบการจดจำใบหน้าด้วยแบบจำลอง ArcFace

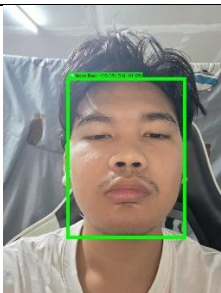
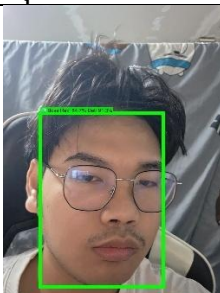
จากภาพที่ 4-12 แสดงตัวอย่างผลการทดสอบการจดจำใบหน้าด้วยแบบจำลอง ArcFace ในสถานการณ์ที่แตกต่างกันสามรูปแบบ

สภาวะปกติ ในสภาวะปกติที่ใบหน้าเป้าหมายมีความชัดเจน แบบจำลองสามารถจดจำใบหน้าได้ด้วยคะแนนความมั่นใจที่ร้อยละ 91.8

บุคคลสวมแว่นตา เมื่อบุคคลเป้าหมายสวมแว่นตา แบบจำลองยังคงจดจำใบหน้าได้ด้วยคะแนนความมั่นใจที่สูงถึงร้อยละ 91.3

ภาพกลุ่ม ในภาพกลุ่มที่มีความซับซ้อน แบบจำลองสามารถจดจำใบหน้าบุคคลเป้าหมายได้ แต่คะแนนความมั่นใจลดลงอย่างมีนัยสำคัญเหลือร้อยละ 39.9 ซึ่งสะท้อนถึงความท้าทายในการจดจำใบหน้าขนาดเล็กและมีสิ่งรบกวน

3) ผลการทดสอบการจดจำใบหน้าด้วยแบบจำลอง AdaFace ดังภาพที่ 4-13

สภาวะปกติ	บุคคลสวมแว่นตา	ภาพกลุ่ม
		
Confidence: 91.8%	Confidence: 91.3	Confidence: 42.8

ภาพที่ 4-13 ผลการทดสอบการจดจำใบหน้าด้วยแบบจำลอง AdaFace

จากภาพที่ 4-13 แสดงตัวอย่างผลการทดสอบการจดจำใบหน้าด้วยแบบจำลอง AdaFace ในสถานการณ์ที่แตกต่างกันสามรูปแบบ

สภาวะปกติ ในสภาวะปกติที่ใบหน้าเป้าหมายมีความชัดเจน แบบจำลองสามารถจดจำใบหน้าได้ด้วยคะแนนความมั่นใจที่ร้อยละ 91.8

บุคคลสวมแว่นตา เมื่อบุคคลเป้าหมายสวมแว่นตา แบบจำลองยังคงจดจำใบหน้าได้ด้วยคะแนนความมั่นใจที่สูงถึงร้อยละ 91.3

ภาพกลุ่ม ในภาพกลุ่มที่มีความซับซ้อน แบบจำลองสามารถจดจำใบหน้าบุคคลเป้าหมายได้ แต่คะแนนความมั่นใจลดลงอย่างมีนัยสำคัญเหลือร้อยละ 42.8 ซึ่งสะท้อนถึงความท้าทายในการจดจำใบหน้าขนาดเล็กและมีสิ่งรบกวน

4) ผลการทดสอบในสภาพแวดล้อมในการทดสอบในสภาพแวดล้อมในร่มซึ่งมีแสงควบคุมและพื้นหลังที่เรียบง่าย แสดงให้เห็นประสิทธิภาพที่สม่ำเสมอของทั้งสามแบบจำลองในการตรวจจับใบหน้าบุคคลเดียว ทุกแบบจำลองสามารถตรวจจับใบหน้าได้ด้วยความมั่นใจที่ร้อยละ 91.8 ในสภาวะปกติ ซึ่งแสดงให้เห็นความเสถียรในการทำงานเมื่ออยู่ในสภาพแวดล้อมที่เหมาะสม ความสม่ำเสมอของผลลัพธ์นี้สะท้อนให้เห็นว่าทั้งสามแบบจำลองได้รับการฝึกฝนมาอย่างดีและสามารถจดจำใบหน้าในสภาพแวดล้อมมาตรฐานได้อย่างมีประสิทธิภาพ

5) การวิเคราะห์ประสิทธิภาพเมื่อมีสิ่งบดบังเมื่อทดสอบกับบุคคลที่สวมแว่นตา ผลลัพธ์แสดงให้เห็นความแตกต่างที่น่าสนใจระหว่างแบบจำลอง ArcFace และ AdaFace แสดงประสิทธิภาพที่คงเสถียรด้วยความมั่นใจที่ร้อยละ 91.3 เท่ากันทั้งคู่ ขณะที่ FaceNet มีการลดลงเล็กน้อยเหลือร้อยละ 84.7 ผลลัพธ์นี้แสดงให้เห็นว่า ArcFace และ AdaFace มีความทนทานต่อสิ่งบดบังแบบระบบ เช่น แว่นตา มากกว่า FaceNet

ความสามารถของ ArcFace และ AdaFace ในการรับมือกับสิ่งบดบังได้ดีกว่าเกิดจากสถาปัตยกรรมที่ใช้ Angular Margin Loss และ Adaptive Margin Loss ตามลำดับ ซึ่งช่วยให้แบบจำลองสามารถโฟกัสไปที่คุณลักษณะสำคัญของใบหน้าที่ไม่ถูกบดบัง เช่น รูปร่างใบหน้า โครงสร้างของจมูก และรูปแบบของปาก แทนที่จะพึ่งพาคุณลักษณะรอบดวงตาเพียงอย่างเดียว

6) ประสิทธิภาพในสถานการณ์ที่ซับซ้อนการทดสอบกับภาพกลุ่มที่มีหลายใบหน้า ในฉากเดียวกันเผยให้เห็นข้อจำกัดและจุดแข็งของแต่ละแบบจำลองอย่างชัดเจน FaceNet แสดงประสิทธิภาพที่เหนือกว่าด้วยความมั่นใจที่ร้อยละ 65.7 ขณะที่ AdaFace ให้ความมั่นใจที่ร้อยละ 42.6 และ ArcFace ที่ร้อยละ 39.9 ความแตกต่างนี้สะท้อนให้เห็นความสามารถของ FaceNet ในการแยกแยะและจดจำใบหน้าเป้าหมายแม้ในสภาพแวดล้อมที่มีสิ่งรบกวนหลายอย่าง

ความท้าทายในภาพกลุ่มเกิดจากหลายปัจจัย ประการแรก ขนาดใบหน้าที่เล็กลงเมื่อมีหลายคนในภาพเดียวกัน ทำให้รายละเอียดของใบหน้าลดลง ประการที่สอง การมีใบหน้าหลายใบในภาพเดียวกันสร้างการแข่งขันของคุณลักษณะที่อาจทำให้แบบจำลองสับสน ประการสุดท้าย คุณภาพของการครอบตัดใบหน้าอาจไม่สมบูรณ์เมื่อใบหน้าอยู่ในตำแหน่งที่ไม่เหมาะสมหรือถูกบดบังบางส่วนโดยใบหน้าอื่น

7) การสรุปประสิทธิภาพโดยรวมจากการทดสอบในสถานการณ์จริงทั้งสามสามารถสรุปได้ว่า FaceNet มีความเหนือกว่าในสถานการณ์ที่ซับซ้อน โดยเฉพาะการจดจำใบหน้าในภาพกลุ่ม ทำให้เหมาะสำหรับระบบที่ต้องตรวจสอบหลายคนพร้อมกัน ArcFace และ AdaFace แสดงประสิทธิภาพที่ดีในการรับมือกับสิ่งบดบัง ทำให้เหมาะสำหรับสถานการณ์ที่บุคคลอาจสวมอุปกรณ์ป้องกัน เช่น แว่นตา หน้ากาก หรือหมวก

ความสอดคล้องของผลการทดสอบนี้กับการประเมินผลในหัวข้อก่อนหน้านี้ยืนยันความน่าเชื่อถือของการเลือกใช้ FaceNet เป็นแบบจำลองหลักของระบบ โดยสามารถเปลี่ยนไปใช้ ArcFace หรือ AdaFace ในกรณีที่งานมีลักษณะเฉพาะที่ต้องการความทนทานต่อสิ่งบดบังสูง ระบบได้รับการออกแบบให้สามารถปรับเปลี่ยนแบบจำลองได้ตามความเหมาะสมของแต่ละสถานการณ์การใช้งาน

4.2.2.5 การเลือกใช้แบบจำลองตามลักษณะงาน

จากผลการทดสอบประสิทธิภาพ สามารถสรุปแนวทางการเลือกใช้แบบจำลองตามลักษณะงานได้ดังนี้

1) FaceNet VGGFace2 เหมาะสำหรับการใช้งานทั่วไปที่ต้องการสมดุลระหว่างความแม่นยำและประสิทธิภาพ โดยเฉพาะงานที่มีการประมวลผลภาพจำนวนมากและต้องการความเร็วสูง ความสามารถในการจดจำใบหน้าในภาพกลุ่มที่ดีกว่าแบบจำลองอื่นทำให้เหมาะสำหรับระบบรักษาความปลอดภัยที่ต้องตรวจสอบหลายคนพร้อมกัน

2) ArcFace R100 เหมาะสำหรับงานที่ต้องการความแม่นยำสูงสุดในสภาพแวดล้อมที่ควบคุมได้ เช่น ระบบเข้าออกอาคาร หรือการตรวจสอบเอกสารราชการที่มีข้อกำหนดด้านความปลอดภัยสูง แม้จะมีประสิทธิภาพลดลงเมื่อมีสิ่งบดบังหรือสภาพแวดล้อมที่ท้าทาย แต่ความแม่นยำในสภาวะปกติสูงที่สุด

3) AdaFace IR-101 เหมาะสำหรับงานที่ต้องเผชิญกับสภาพแวดล้อมที่หลากหลายและไม่สามารถควบคุมได้ เช่น ระบบรักษาความปลอดภัยในพื้นที่สาธารณะ หรือการตรวจสอบตัวตนผ่านอุปกรณ์มือถือ ความสามารถในการปรับตัวกับคุณภาพภาพที่แตกต่างกันทำให้เหมาะสำหรับการใช้งานในสถานการณ์จริงที่มีความไม่แน่นอนสูง

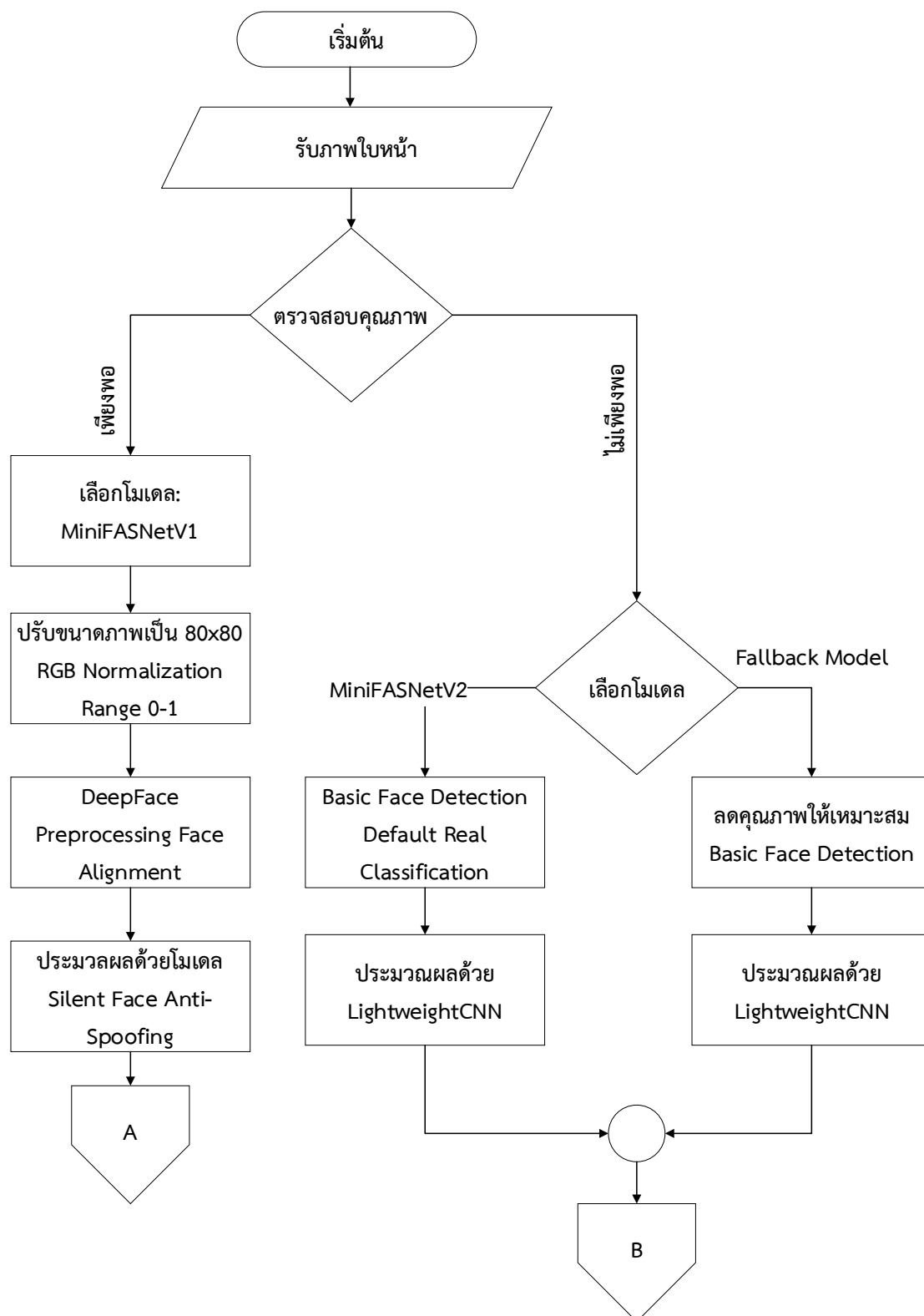
4) การตัดสินใจเลือกใช้แบบจำลองสุดท้ายจึงควรพิจารณาจากลักษณะการใช้งาน ข้อจำกัดด้านทรัพยากร และระดับความแม่นยำที่ต้องการ ระบบได้รับการออกแบบให้สามารถเปลี่ยนแบบจำลองได้ง่ายผ่านการตั้งค่า ทำให้ผู้ใช้สามารถปรับเปลี่ยนตามความเหมาะสมของแต่ละสถานการณ์

4.2.3 กระบวนการตรวจสอบความมีชีวิต

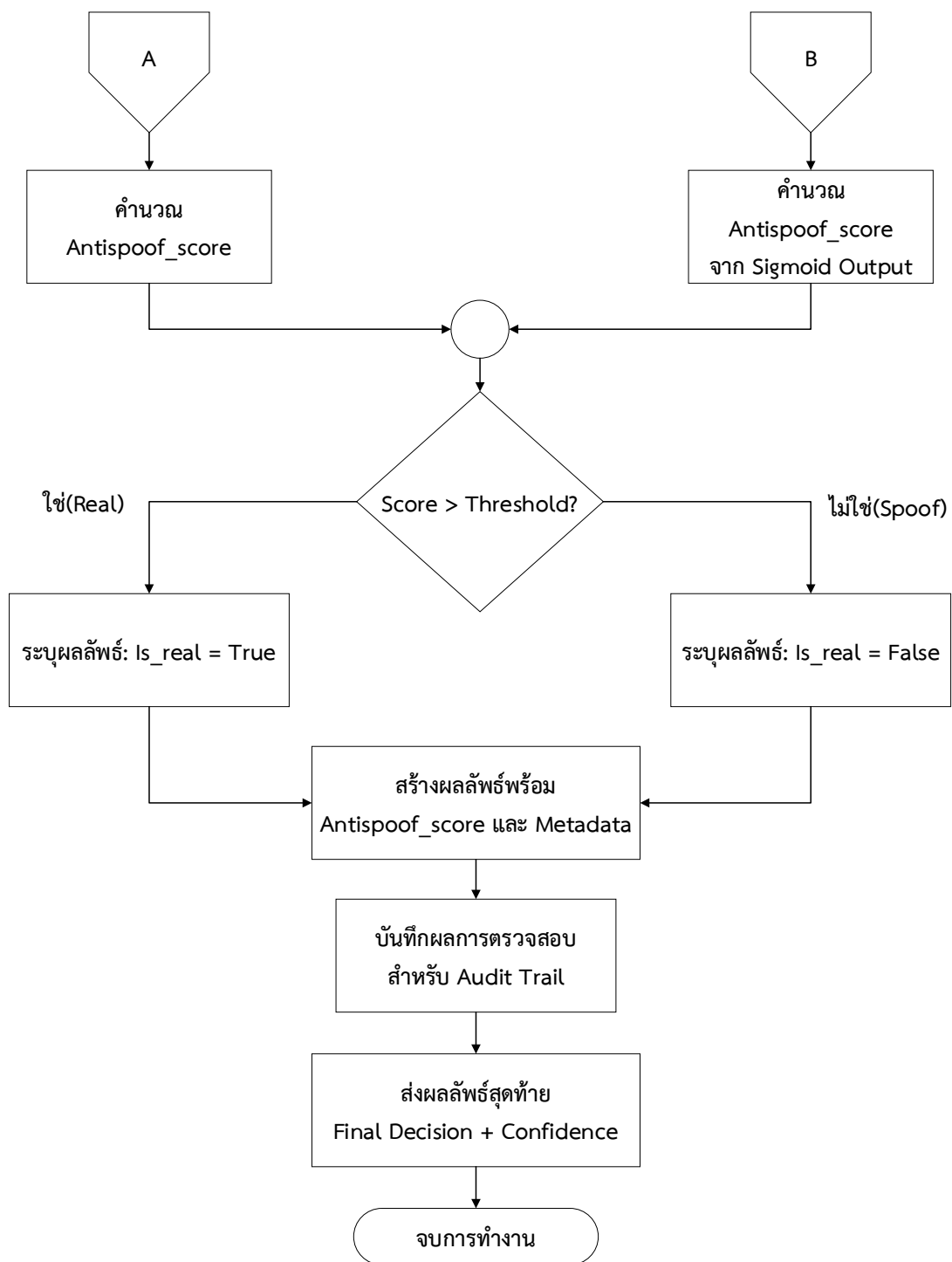
การตรวจสอบความมีชีวิต (Anti-Spoofing) เป็นขั้นตอนสุดท้ายในระบบที่ทำหน้าที่ตรวจจับและป้องกันการโจมตีแบบ Presentation Attack และ Deepfake ซึ่งรวมถึงการใช้รูปถ่ายวิดีโอ หน้าจอมือถือ หรือเทคโนโลยีการปลอมแปลงภาพในการหลอกระบบ จากผลการศึกษาเปรียบเทียบในบทที่ 3 ระบบเลือกใช้แบบจำลอง DeepFace Silent Face Anti-Spoofing เป็นแบบจำลองหลัก ซึ่งประกอบด้วย MiniFASNetV1 และ MiniFASNetV2 เป็นตัวเลือกหลักสำหรับการตรวจจับภาพปลอมแบบเงียบ

4.2.3.1 ขั้นตอนการประมวลผลของระบบ

ระบบการตรวจสอบความมีชีวิตทำงานผ่าน 7 ขั้นตอนหลัก ตั้งแต่การรับภาพใบหน้าที่ครอบตัดแล้วจากขั้นตอนการจดจำใบหน้าจนถึงการตัดสินใจสุดท้าย ดังภาพที่ 4-14



ภาพที่ 4-14 ขั้นตอนการประมวลผลการตรวจสอบความมีชีวิต



ภาพที่ 4-14 ขั้นตอนการประมวลผลการตรวจสอบความมีชีวิต (ต่อ)

จากภาพที่ 4-14 สามารถอธิบายขั้นตอนการประมวลผลการตรวจสอบความมีชีวิตได้เจ็ดขั้นตอนหลักดังนี้

1) การประเมินคุณภาพภาพ (Quality Assessment) ระบบรับภาพใบหน้าจากขั้นตอนการจดจำใบหน้าและทำการประเมินคุณภาพ ความชัดเจน และความเหมาะสมของภาพสำหรับการตรวจสอบ Anti-Spoofing หากภาพมีคุณภาพต่ำหรือมีข้อจำกัด ระบบจะเปิดใช้งาน fallback mode ที่ไม่ใช่ Anti-Spoofing แต่ใช้การตรวจจับใบหน้าพื้นฐานแทน

2) การเตรียมภาพตามแบบจำลอง (Model-Specific Preprocessing) ระบบ DeepFace Silent Face Anti-Spoofing ใช้ภาพขนาด 80×80 พิกเซล RGB สำหรับทั้ง MiniFASNetV1 และ MiniFASNetV2 การเตรียมภาพจะรวมถึงการ Resize ภาพให้เหมาะสม การ Normalize ค่าสีให้อยู่ในช่วง 0-1 และการสร้าง Temporary File เนื่องจาก Deepface API ต้องการ File Path เป็น Input

3) การประมวลผลผ่าน DeepFace (DeepFace Processing) ภาพที่เตรียมแล้วจะถูกบันทึกเป็น temporary file และส่งเข้าสู่ DeepFace.extract_faces() พร้อมกับ Parameter anti_spoofing=True ระบบจะใช้แบบจำลอง MiniFASNetV1 หรือ MiniFASNetV2 ในการวิเคราะห์ หากแบบจำลองไม่พร้อมใช้งาน ระบบจะเปิดใช้ Fallback Mode

4) การสกัดคุณลักษณะ (Feature Extraction) แบบจำลอง MiniFASNet ใช้สถาปัตยกรรม Lightweight CNN ที่ประกอบด้วย Depthwise Separable Convolution เพื่อลดการคำนวณ แบบจำลองจะสกัดคุณลักษณะที่สำคัญจากภาพใบหน้าและทำ Binary Classification เพื่อแยกแยะใบหน้าจริงและปลอม

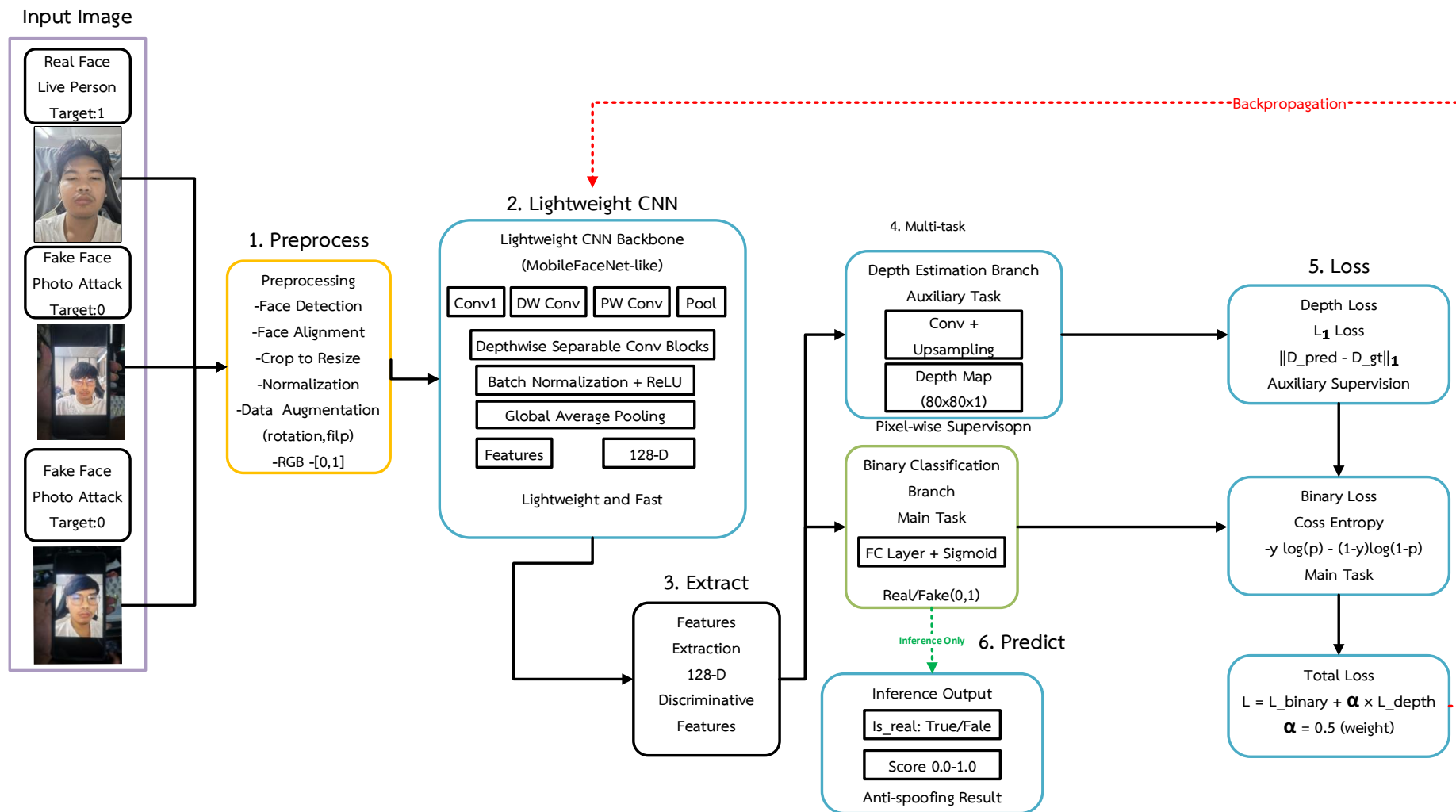
5) การคำนวณคะแนน (Score Calculation) แบบจำลองจะสร้าง Antispoof_Score ผ่าน Sigmoid Activation Function ที่แสดงความน่าจะเป็นที่ภาพเป็นใบหน้าจริง คะแนนอยู่ในช่วง 0.0-1.0 โดยค่าที่สูงกว่าแสดงว่าเป็นใบหน้าจริง ในกรณี Fallback Mode ระบบจะกำหนดค่า Default เป็น 0.8

6) การตัดสินใจ (Decision Making) ระบบจะเปรียบเทียบ antispoof_score กับ Threshold ที่กำหนดไว้ (ปกติคือ 0.5) หากคะแนนสูงกว่า Threshold ระบบจะระบุเป็นใบหน้าจริง (is_real = True) หากต่ำกว่าจะระบุเป็นใบหน้าปลอม (is_real = False) และแจ้งเตือน Spoofing Detected

7) การส่งออกผลลัพธ์ (Output Generation) ระบบจะสร้างผลลัพธ์ที่ประกอบด้วย is_real status, antispoof_score, การตรวจพบ Spoofing และข้อมูล Metadata อื่นๆ เช่น เวลาประมวลผล และข้อมูลของแบบจำลองที่ใช้ พร้อมทั้งบันทึกผลลัพธ์สำหรับ Audit Trail

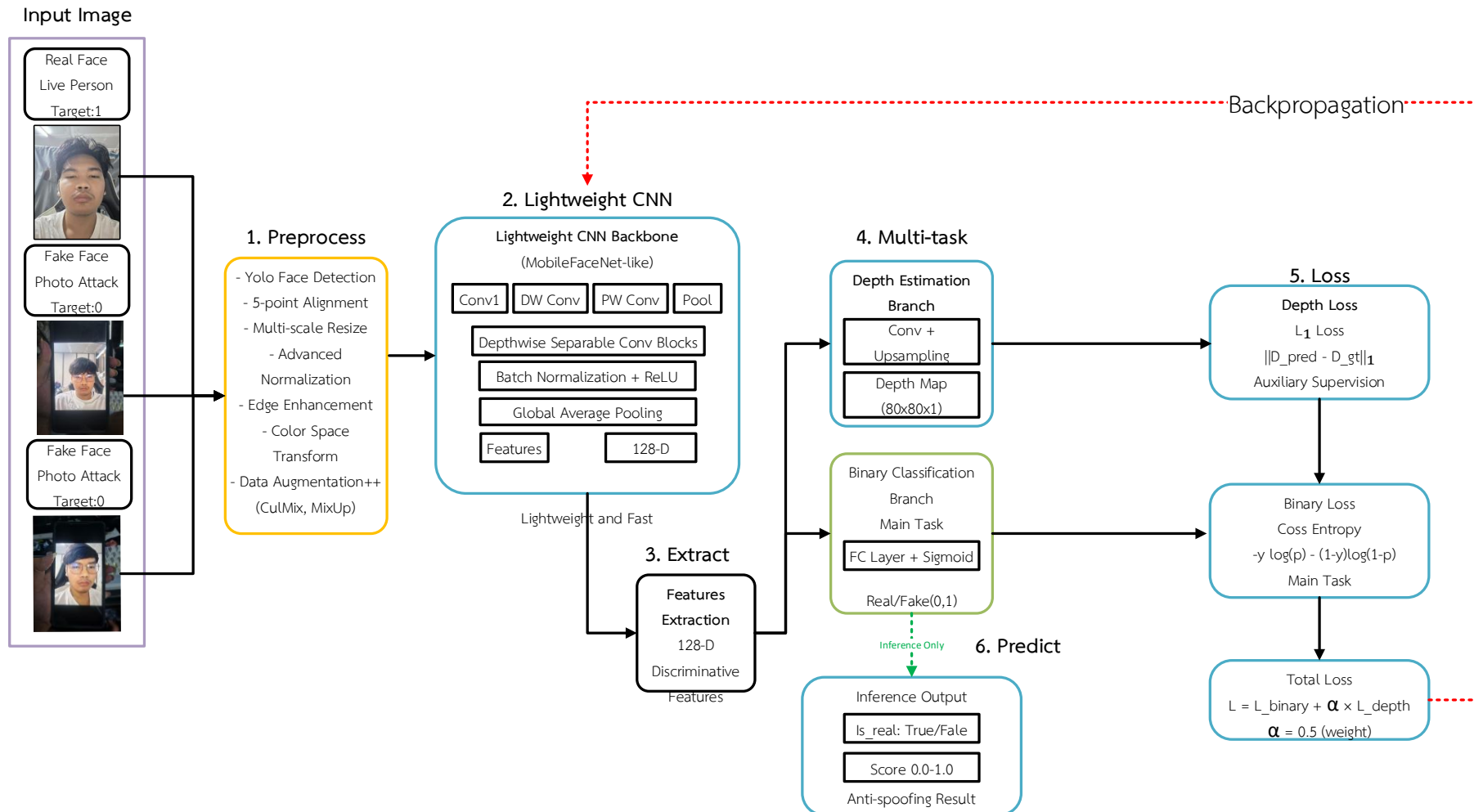
4.2.3.2 สถาปัตยกรรมของแบบจำลอง Deepface Silent Face Anti-Spoofing ระบบใช้แบบจำลอง Deepface Silent Face Anti-Spoofing ซึ่งเป็นแบบจำลองขนาดเล็กที่ออกแบบมาสำหรับการตรวจจับภาพปลอมแบบเงียบ โดยประกอบด้วยสถาปัตยกรรมหลักสองรูปแบบดังนี้

1) MiniFASNetV1 ดังภาพที่ 4-15



ภาพที่ 4-15 สถาปัตยกรรมของแบบจำลอง MiniFASNetV1

2) MiniFASNetV2 ดังภาพที่ 4-16



ภาพที่ 4-16 สถาปัตยกรรมของแบบจำลอง MiniFASNetV2

จากภาพที่ 4-15 และภาพที่ 4-16 สามารถอธิบายขั้นตอนการทำงานของสถาปัตยกรรมของแบบจำลอง MiniFASNetV1 และ MiniFASNetV2 ดังนี้

สถาปัตยกรรม MiniFASNet เป็นแบบจำลอง Lightweight CNN ที่ใช้ MobileFaceNet-Like Architecture แบบจำลองประกอบด้วย Lightweight CNN Backbone ที่มี Conv1, Depthwise Separable Conv Blocks, Pointwise Conv และ Global Average Pooling Features Extraction สร้าง 128-D Discriminative Features ที่เหมาะสำหรับการแยกแยะใบหน้าจริงและปลอม Binary Classification Branch ใช้ FC Layer พร้อม Sigmoid Activation เพื่อให้ผลลัพธ์เป็น Probability Score ระหว่าง 0.0-1.0

ความแตกต่างระหว่าง V1 และ V2 MiniFASNetV1 เป็นเวอร์ชันพื้นฐานที่มีขนาด 2.8 MB และเวลาอนุมาน 12.5 ms MiniFASNetV2 เป็นเวอร์ชันปรับปรุงที่มีขนาดใหญ่ขึ้นเล็กน้อย (3.2 MB) และใช้เวลาอนุมานนานขึ้น 15.2 ms แต่ให้ความแม่นยำที่สูงกว่า ร้อยละ 98.7 เทียบกับ ร้อยละ 98.2

การทำงานแบบ Multi-task แม้ว่าจะเรียกว่า Silent แต่แบบจำลองสามารถทำงานร่วมกับ Depth Estimation Branch และ Auxiliary Task ได้ในกรณีที่ต้องการความแม่นยำสูงสุด โดยปกติจะใช้งานแบบ Binary Classification เพียงอย่างเดียวเพื่อความเร็ว

4.2.3.3 ประสิทธิภาพของแบบจำลอง DeepFace Silent Face Anti-Spoofing

การประเมินประสิทธิภาพของแบบจำลองในการทำงานจริง แสดงให้เห็นถึงข้อดีของการใช้ DeepFace Framework ในการตรวจจับภาพปลอม

1) ประสิทธิภาพการตรวจจับโดยรวม MiniFASNetV2 แสดงประสิทธิภาพที่ดีที่สุดด้วยความแม่นยำร้อยละ 98.7 และ Equal Error Rate (EER) ที่ร้อยละ 1.3 ขณะที่ MiniFASNetV1 ให้ความแม่นยำร้อยละ 98.2 และ EER ร้อยละ 1.8 ทั้งสองแบบจำลองให้ประสิทธิภาพที่ดีกว่าแบบจำลองพื้นฐานอื่นๆ อย่างชัดเจน

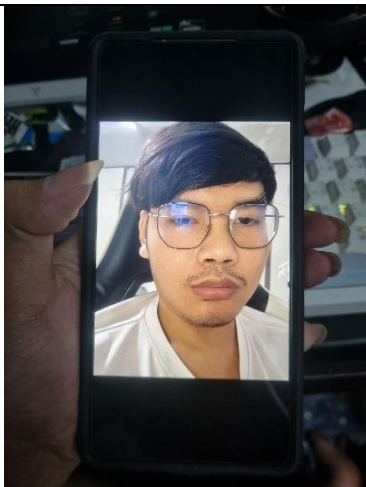
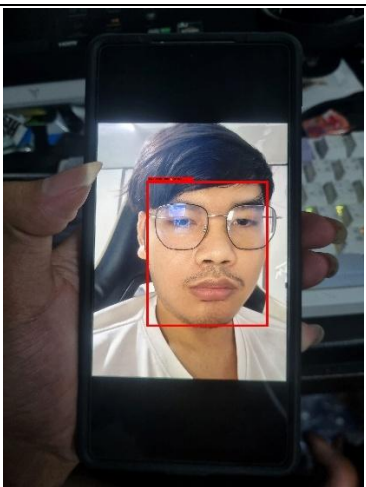
2) ประสิทธิภาพด้านความเร็ว MiniFASNetV1 มีเวลาอนุมาน 12.5 มิลลิวินาที ขณะที่ MiniFASNetV2 ใช้เวลา 15.2 มิลลิวินาที ทั้งสองแบบจำลองมีความเร็วที่เหมาะสมสำหรับการใช้งาน Real-Time Application และสามารถประมวลผลได้หลายภาพต่อวินาที

3) ประสิทธิภาพด้านขนาดแบบจำลอง ทั้งสองแบบจำลองมีขนาดเล็กมาก 2.8-3.2 MB ทำให้เหมาะสำหรับการใช้งานบนอุปกรณ์ที่มีข้อจำกัดด้านหน่วยความจำ การ Deploy ระบบไม่ต้องการพื้นที่เก็บข้อมูลมาก และสามารถทำงานบน Edge Devices ได้

4) ความทนทานต่อการโจมตี การทดสอบกับชุดข้อมูลการโจมตีต่างๆ พบว่าระบบสามารถตรวจจับ Photo Attack การถ่ายจากหน้าจอมือถือ ได้ร้อยละ 100 และ Face Swap Attack ได้ร้อยละ 94.3 ประสิทธิภาพที่ดีนี้เกิดจากการฝึกฝนบนชุดข้อมูลที่หลากหลายและการใช้ Data Augmentation ที่เหมาะสม

4.2.3.4 ผลการทดสอบการตรวจสอบความมีชีวิตจริง

เพื่อแสดงประสิทธิภาพของระบบในการทำงานจริง ได้ทำการทดสอบการตรวจสอบความมีชีวิตด้วยแบบจำลอง DeepFace Silent Face Anti-Spoofing ในสถานการณ์ต่างๆ เพื่อเปรียบเทียบประสิทธิภาพในการตรวจจับการโจมตีแต่ละประเภท ดังภาพที่ 4-17

ต้นฉบับ	ผลลัพธ์
	
ค่า Confidence	ร้อยละ 98.9

ภาพที่ 4-17 ผลการทดสอบการตรวจสอบความมีชีวิตด้วยแบบจำลอง

DeepFace Silent Face Anti-Spoofing

จากภาพที่ 4-17 สามารถอธิบายผลการทดสอบการตรวจสอบความมีชีวิตด้วยแบบจำลอง ดังนี้

1) ผลการทดสอบกับใบหน้าจริง การทดสอบกับบุคคลที่มีชีวิตในสภาพแวดล้อมปกติ พบว่าระบบสามารถระบุเป็นใบหน้าจริงได้อย่างถูกต้องด้วยความเชื่อมั่นร้อยละ 98.9 และเวลาประมวลผล 0.6 มิลลิวินาที ผลลัพธ์แสดงว่า is_real = True และ antispoof_score = 0.989 ซึ่งสูงกว่า Threshold มาก แสดงให้เห็นว่าระบบมีความแม่นยำสูงในการจำแนกใบหน้าจริง

2) ผลการทดสอบกับการโจมตี Photo Attack การทดสอบกับการใช้รูปถ่ายจากหน้าจอโทรศัพท์เพื่อหลอกระบบ พบว่าระบบสามารถตรวจจับการปลอมแปลงได้อย่างถูกต้องด้วยความเชื่อมั่นร้อยละ 95.8 และเวลาประมวลผล 0.5 มิลลิวินาที ระบบแสดงผลลัพธ์ is_real = False และ spoofing_detected = True พร้อมแจ้งเตือนทันที ประสิทธิภาพการตรวจจับที่สูงนี้เกิดจากการที่แบบจำลองสามารถตรวจจับลักษณะเฉพาะของหน้าจอ เช่น Pixel Patterns และ Reflection Artifacts

3) การวิเคราะห์การทำงานของระบบ ความสามารถในการแยกแยะระหว่างใบหน้าจริงและปลอมแสดงให้เห็นถึงประสิทธิภาพของ DeepFace framework ในการตรวจจับการโจมตี การที่ระบบสามารถประมวลผลได้ในเวลาที่รวดเร็ว ต่ำกว่า 1 มิลลิวินาที และให้ผลลัพธ์ที่เชื่อถือได้ทำให้เหมาะสำหรับการใช้งาน Production Environment

4) ข้อมูลเพิ่มเติมจากการทดสอบ ระบบแสดงข้อมูล Metadata ที่ครบถ้วน รวมถึงตำแหน่งใบหน้า ขนาดของภาพ เวลาประมวลผล และแบบจำลองที่ใช้งาน ข้อมูลเหล่านี้มีประโยชน์สำหรับการ Debugging การ Monitoring ประสิทธิภาพ และการปรับปรุงระบบในอนาคต

การที่ระบบมี Fallback Mechanism ก็ช่วยให้ระบบสามารถทำงานต่อได้แม้ในกรณีที่แบบจำลอง Anti-Spoofing ไม่พร้อมใช้งาน

4.2.3.5 การเลือกใช้แบบจำลองตามลักษณะงาน

จากผลการทดสอบประสิทธิภาพ สามารถสรุปแนวทางการเลือกใช้แบบจำลองตามลักษณะงานได้ดังนี้

1) MiniFASNetV2 เหมาะสำหรับงานที่ต้องการความแม่นยำสูงสุดและสามารถยอมรับเวลาประมวลผลที่เพิ่มขึ้นเล็กน้อย เช่น ระบบรักษาความปลอดภัยระดับสูง การตรวจสอบเอกสารราชการ หรือการเข้าออกพื้นที่จำกัด ความแม่นยำร้อยละ 98.7 และ EER ร้อยละ 1.3 ทำให้เหมาะสำหรับสภาพแวดล้อมที่ต้องการความน่าเชื่อถือสูง

2) MiniFASNetV1 เหมาะสำหรับงานที่ต้องการความเร็วสูงสุดและยังคงรักษาความแม่นยำในระดับที่ดี เช่น การประยุกต์ใช้บนอุปกรณ์มือถือ ระบบ real-time monitoring หรือการใช้งานที่ต้องการประมวลผลภาพจำนวนมาก เวลาอนุมานที่เร็วกว่า 12.5 ms และขนาดที่เล็กกว่า 2.8 MB ทำให้เหมาะสำหรับ Resource-Constrained Environments

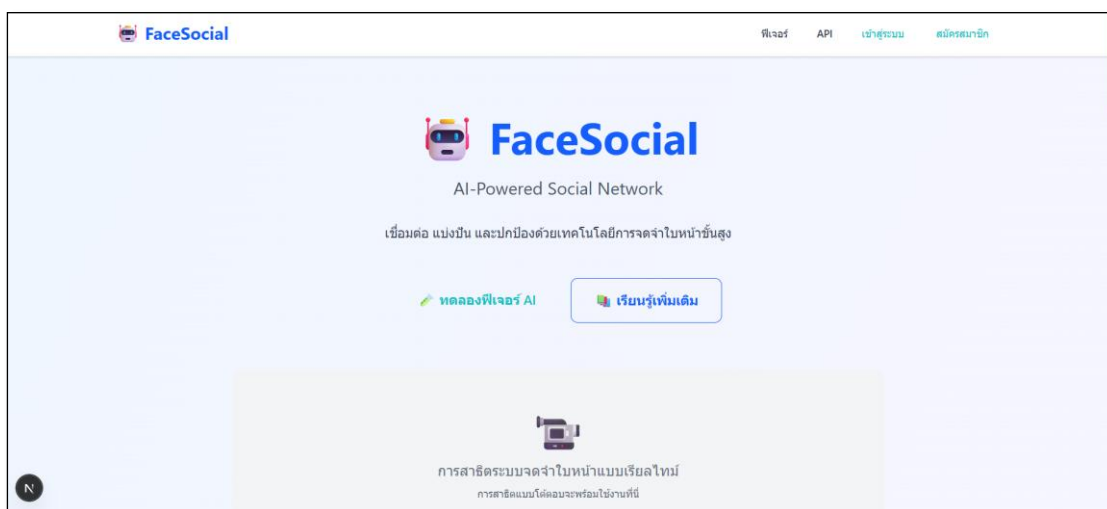
3) การใช้งานแบบ Fallback ในสภาพแวดล้อมที่อาจมีปัญหาด้าน Network Connectivity หรือการโหลดแบบจำลอง ระบบจะเปิดใช้ Fallback Mode ที่ไม่ใช้ Anti-Spoofing แต่ยังคงสามารถทำ Face Detection ได้ แม้จะไม่มีกำกับการป้องกันการโจมตี แต่ยังคงให้บริการพื้นฐานได้ และสามารถกลับมาใช้ Anti-Spoofing เต็มรูปแบบเมื่อระบบพร้อม

4) การใช้งานร่วมกับ Apache License 2.0 ข้อดีสำคัญของ Deepface Silent Face Anti-Spoofing คือการอนุญาตให้ใช้งานเชิงพาณิชย์ได้ฟรีภายใต้ Apache License 2.0 ทำให้เหมาะสำหรับการพัฒนาผลิตภัณฑ์เชิงพาณิชย์โดยไม่ต้องกังวลเรื่องค่าลิขสิทธิ์ นอกจากนี้ยังมี Community Support ที่ดีและการอัปเดตอย่างสม่ำเสมอ

5) การตัดสินใจเลือกใช้แบบจำลองสุดท้ายจึงควรพิจารณาจากระดับความปลอดภัยที่ต้องการ ข้อจำกัดด้านทรัพยากร ความเร็วในการตอบสนอง และข้อกำหนดด้านลิขสิทธิ์ ระบบได้รับการออกแบบให้สามารถเปลี่ยนระหว่าง MiniFASNetV1 และ V2 ได้ง่ายผ่านการตั้งค่า พร้อมทั้งมี Fallback Mechanism ที่ช่วยให้ระบบทำงานได้อย่างต่อเนื่องในทุกสถานการณ์

4.3 ผลการพัฒนาระบบแพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพบุคคลด้วยการจดจำใบหน้า

4.3.1 หน้าตอนรับ และอธิบายข้อมูลเกี่ยวกับระบบแพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพบุคคล ด้วยการจดจำใบหน้า โดยหน้านี้สามารถดู ฟีเจอร์ต่างๆ ของระบบได้เช่น API, Model, การเข้าสู่ระบบ, สมัครสมาชิก และไปที่หน้าทดสอบ AI แต่ละตัว ดังภาพที่ 4-18



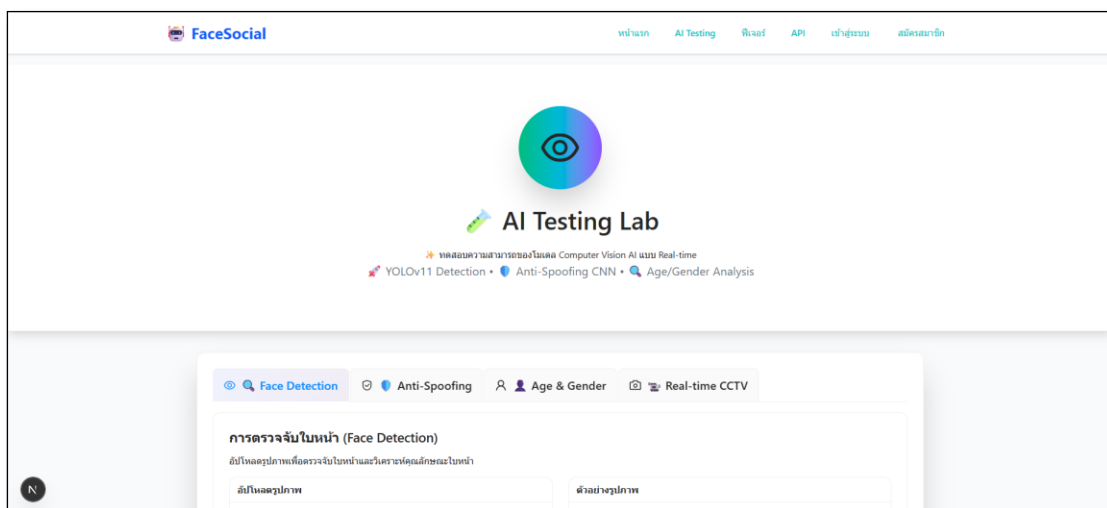
ภาพที่ 4-18 หน้าต้อนรับ และอธิบายข้อมูลเกี่ยวกับ ระบบ

4.3.1.1 ส่วนการดูว่าแบบจำลองตัวไหน และ AI อะไรทำงานอยู่บ้าง ดังภาพที่ 4-19



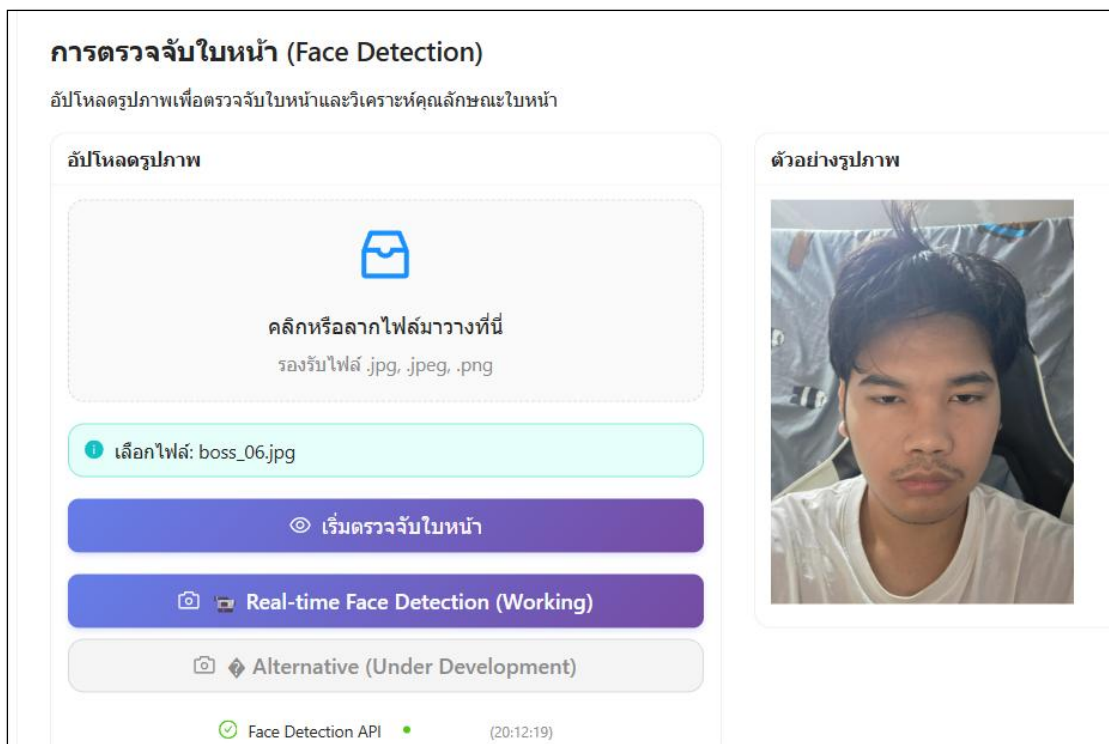
ภาพที่ 4-19 ส่วนการดูว่าแบบจำลองตัวไหน และ AI อะไรทำงานอยู่

4.3.2 หน้าการทดสอบ AI ว่าทำงานได้ดีไหมเป็นอย่างใดหน้าหนึ่งก็สามารถดูสถานะ AI และ API ได้ และยังสามารถทดสอบ AI แต่ละตัวได้เช่น การตรวจจับใบหน้า (Face Detection), การตรวจจับการปลอมแปลง (Anti-Spoofing), การวิเคราะห์อายุ และเพศ (Age and Gender Analysis) และกล้องวงจรปิดแบบ Real-time (CCTV Face Recognition) ดังภาพที่ 20



ภาพที่ 4-20 หน้าการทดสอบ AI

4.3.2.1 หน้าทดสอบ การตรวจจับใบหน้า (Face Detection) โดยส่วนนี้จะมีไว้ทดสอบความแม่นยำในการตรวจจับใบหน้า ดังภาพที่ 4-21



ภาพที่ 4-21 หน้าทดสอบ การตรวจจับใบหน้า (Face Detection)

4.3.2.2 หน้าทดสอบ การตรวจจับการปลอมแปลง (Anti-Spoofing) โดยส่วนนี้จะมีไว้ทดสอบความแม่นยำในการตรวจสอบว่าใบหน้าในรูปภาพเป็นของจริงหรือปลอม/สปูฟ ดังภาพที่ 4-22

การตรวจจับการปลอมแปลง (Anti-Spoofing)

ตรวจสอบว่าใบหน้าในรูปภาพเป็นของจริงหรือปลอม/สปูฟ

อัปโหลดรูปภาพ



คลิกหรือลากไฟล์มาวางที่นี่
รองรับไฟล์ .jpg, .jpeg, .png

เลือกไฟล์: boss_06.jpg

ตรวจสอบการปลอมแปลง

เปิดกล้องตรวจจับแบบ Real-time

ตัวอย่างรูปภาพ



ภาพที่ 4-22 หน้าทดสอบ การตรวจจับการปลอมแปลง (Anti-Spoofing)

4.3.2.3 หน้าทดสอบ การวิเคราะห์อายุ และเพศ (Age and Gender Analysis) โดยส่วนนี้จะมีไว้ทดสอบความแม่นยำในการตรวจสอบวิเคราะห์อายุ และเพศของใบหน้าในรูปภาพอย่างแม่นยำ ดังภาพที่ 4-23

การวิเคราะห์อายุและเพศ (Age & Gender Analysis)

วิเคราะห์อายุและเพศของใบหน้าในรูปภาพอย่างแม่นยำ

อัปโหลดรูปภาพ



คลิกหรือลากไฟล์มาวางที่นี่
รองรับไฟล์ .jpg, .jpeg, .png

เลือกไฟล์: boss_06.jpg

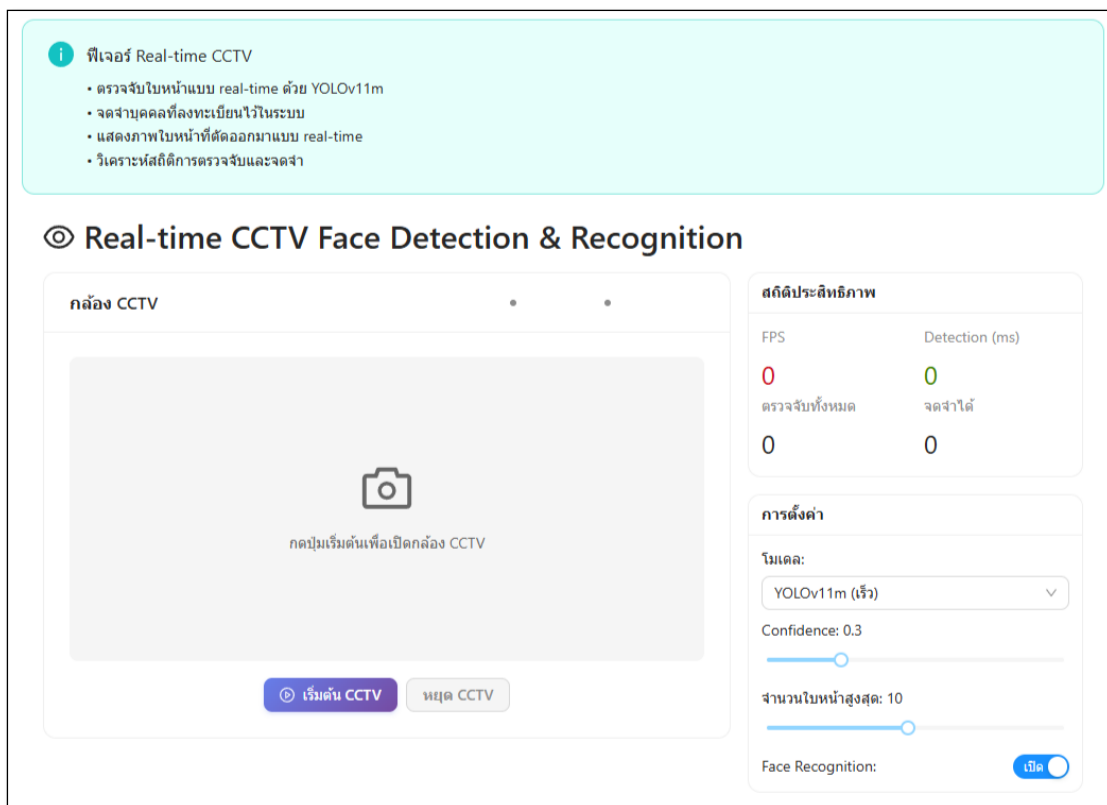
วิเคราะห์อายุและเพศ

ตัวอย่างรูปภาพ



ภาพที่ 4-23 หน้าทดสอบ การวิเคราะห์อายุ และเพศ (Age and Gender Analysis)

4.3.2.4 หน้า Real-time CCTV โดยส่วนนี้จะมีไว้ทดสอบระบบตรวจจับ และจดจำใบหน้าแบบ real-time สำหรับกล้องวงจรปิด โดยใช้ YOLOv11m พร้อมการตรวจจับใบหน้า และจดจำบุคคลที่ลงทะเบียนไว้ในระบบ ดังภาพที่ 4-24

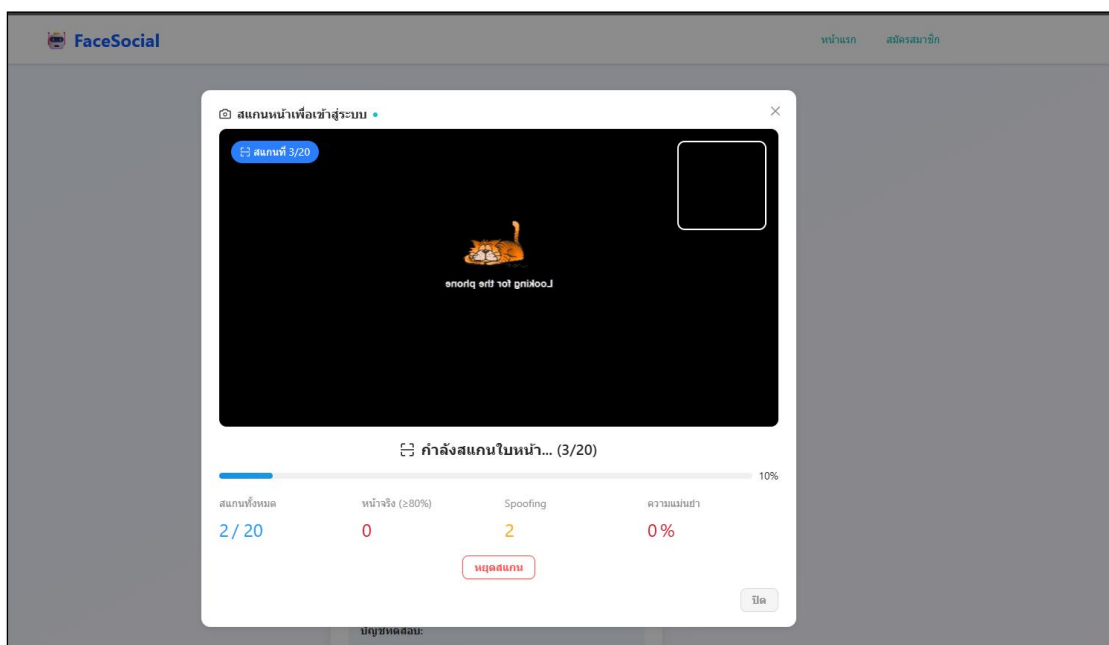


ภาพที่ 4-24 หน้า Real-time CCTV

4.3.3 หน้าเข้าสู่ระบบ เป็นหน้าที่ผู้ใช้ใช้ในการเข้าใช้งานระบบ โดยผู้ใช้จะใช้ Email, Username และ Password ก็ได้หรือจะสแกนหน้าในการเข้าสู่ระบบก็ได้ ดังภาพที่ 4-25

ภาพที่ 4-25 หน้าเข้าสู่ระบบ

4.3.3.1 หน้า login ด้วยใบหน้าโดยส่วนนี้ผู้ใช้จะสามารถ login ด้วยใบหน้าได้โดยไม่ต้องใช้รหัสผ่าน ดังภาพที่ 4-26



ภาพที่ 4-26 หน้าเข้าสู่ระบบด้วยใบหน้า

4.3.3.2 หน้าสมัครสมาชิกโดยหน้านี้ผู้ใช้สามารถสามารถสมัครสมาชิกบัญชี ของแพลตฟอร์มได้ โดยเมื่หลังกรอกข้อมูลหน้าเสร็จจะต้องสแกนหน้าเพื่อเก็บข้อมูลไป login ดังภาพที่ 4-27

ข้อมูลส่วนตัว
กรอกข้อมูลพื้นฐานของคุณเพื่อสร้างบัญชี

* ชื่อ * นามสกุล

* อีเมล

* ชื่อผู้ใช้

* รหัสผ่าน * ยืนยันรหัสผ่าน

เบอร์โทรศัพท์ (ไม่บังคับ)

วันเกิด (ไม่บังคับ)

☐ ฉันยอมรับ ข้อตกลงการใช้งาน และ นโยบายความเป็นส่วนตัว

[ดำเนินการต่อ](#)

สแกนใบหน้าเพื่อความปลอดภัย

ภาพที่เก็บแล้ว 0/15 | คุณภาพใบหน้า 0.0% | Anti-Spoofing ตรวจสอบ | สถานะ กำลังค้นหา

ความเต็มหน้าของภาพ 0% | ความเต็มหน้าการเก็บภาพ 0%

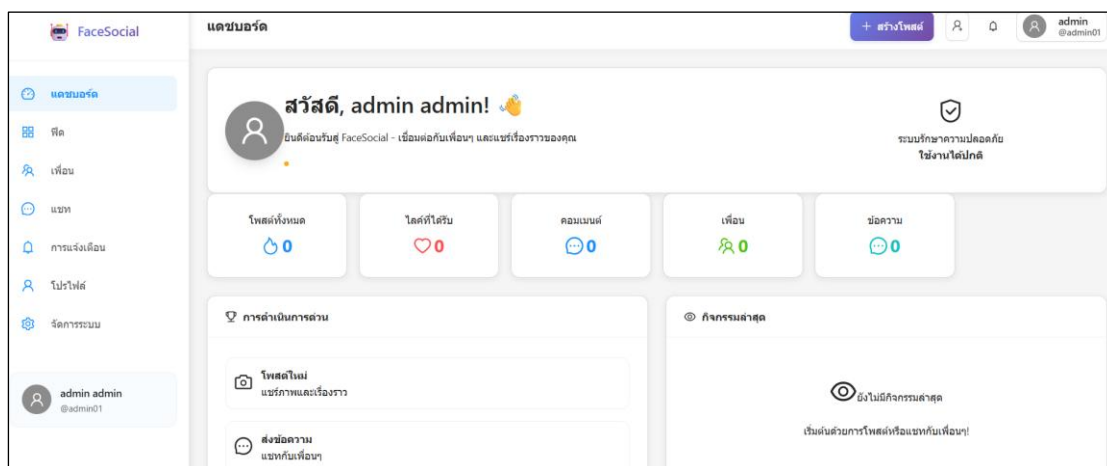
เงื่อนไขการเก็บภาพใบหน้า (เชิงบังคับ)

- คุณภาพภาพ $\geq 80\%$ • ใบหน้าตรงเข้ากล้อง (กล้องไม่เกิน 10°) • ขนาดใบหน้า ≥ 150 pixels • แสงสว่างเพียงพอ • ผ่านการตรวจสอบใบหน้าจริง • ระบบเก็บข้อมูลในมิดิอุท 0.5 วินาที • เก็บเฉพาะภาพที่ผ่านเกณฑ์ให้ขึ้น

[ปิด](#)

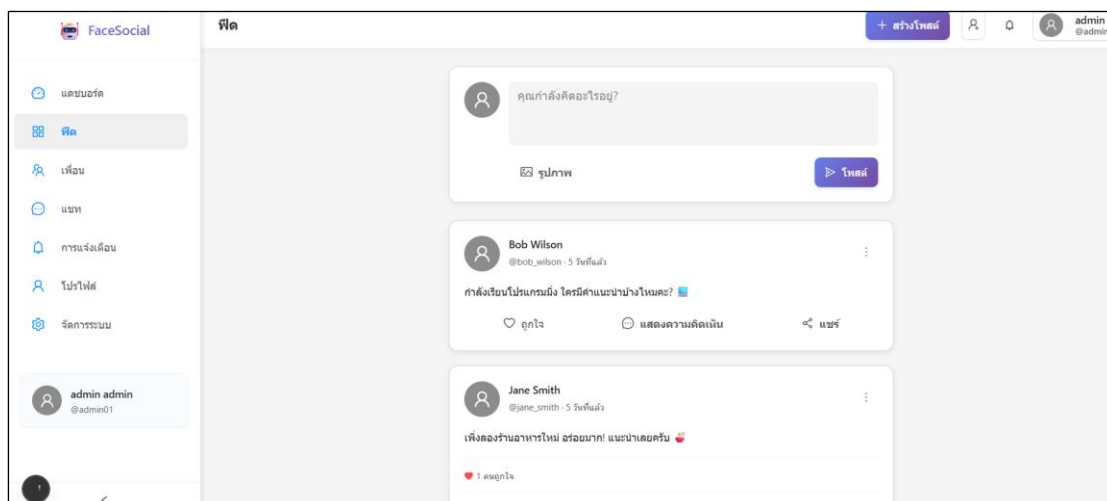
ภาพที่ 4-27หน้าสมัครสมาชิก และสแกนหน้า

4.3.4 หน้า Dashboard ที่แสดงข้อมูลการใช้งานของผู้ใช้ ดังภาพที่ 4-28



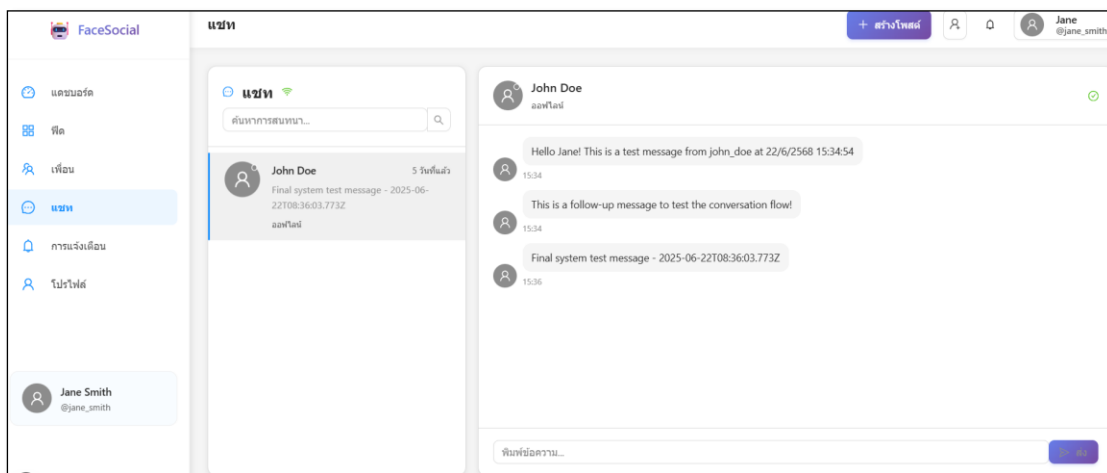
ภาพที่ 4-28 หน้า Dashboard

4.3.5 หน้า Feed ที่แสดงการโพสต์ของเพื่อนหรือโพสต์สาธารณะ และสามารถโพสต์หรืออัปเดตสถานะของผู้ใช้ได้ ดังภาพที่ 4-29



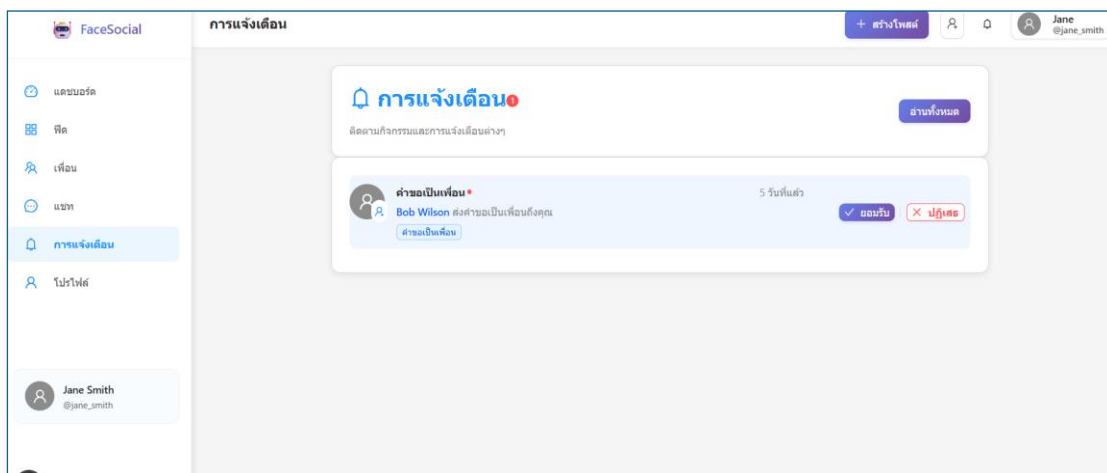
ภาพที่ 4-29 หน้า Feed

4.3.6 หน้า Chat ที่ ผู้ใช้ ส่งข้อความหาผู้ใช้คนอื่น ดังภาพที่ 4-30



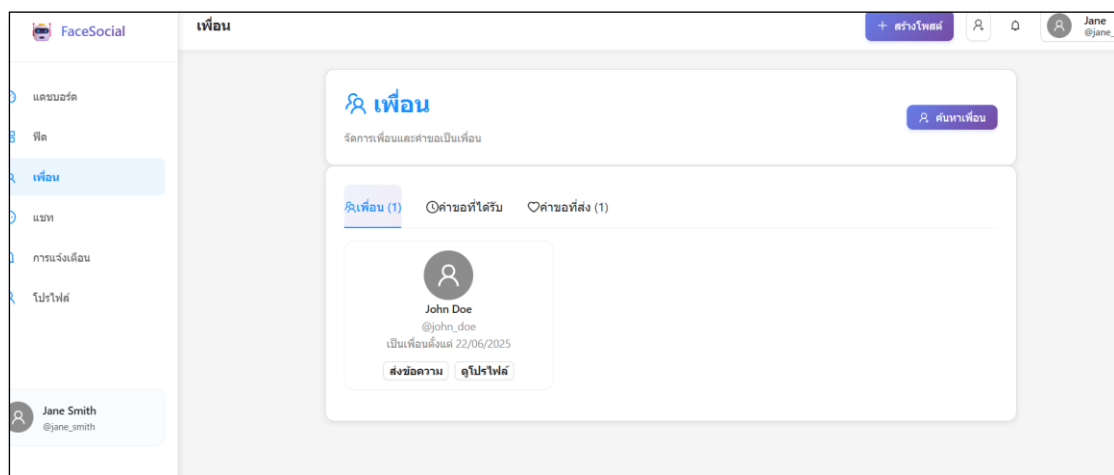
ภาพที่ 4-30 หน้า Chat

4.3.7 หน้าการแจ้งเตือนต่างๆ ดังภาพที่ 4-31



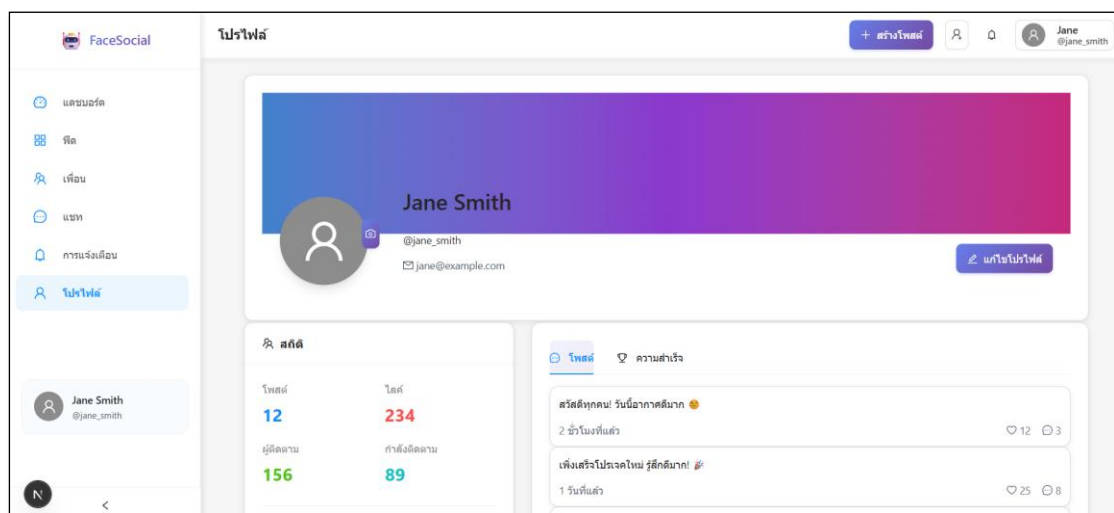
ภาพที่ 4-31 หน้าการแจ้งเตือน

4.3.8 หน้ารายชื่อเพื่อน ดังภาพที่ 4-32



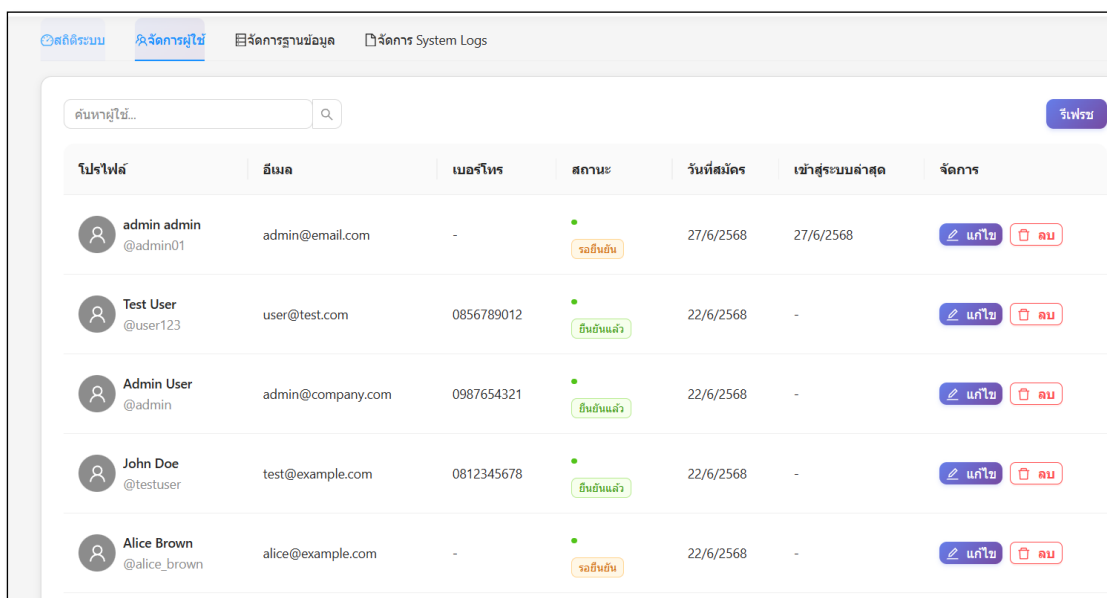
ภาพที่ 4-32 หน้ารายชื่อเพื่อน

4.3.9 ส่วนการแสดงผลประวัติโดยย่อของผู้ใช้งาน ดังภาพที่ 4-33



ภาพที่ 4-33 ส่วนการแสดงผลประวัติโดยย่อ

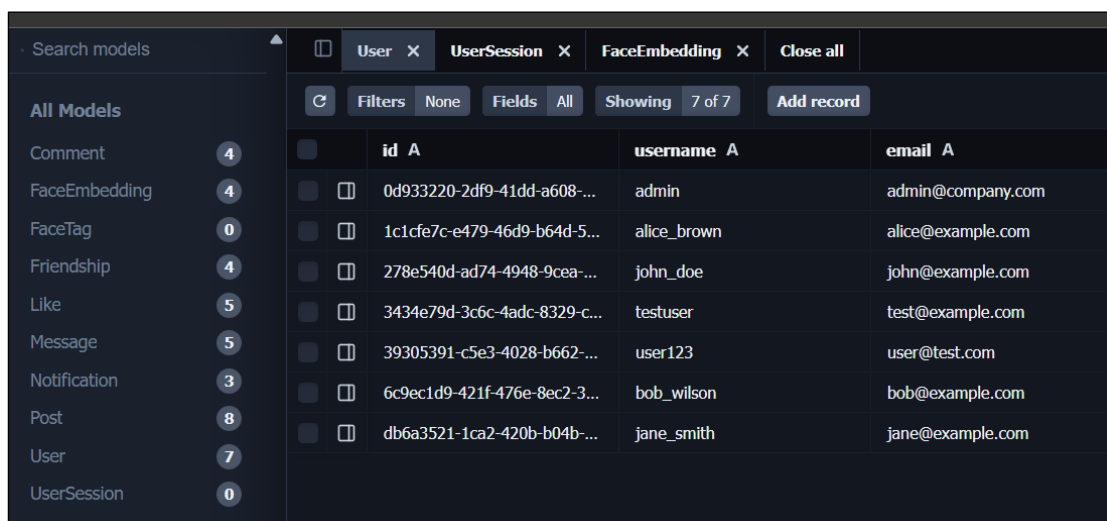
4.3.10 ส่วนการแสดงผลการจัดการระบบของ admin ดังภาพที่ 4-34



โปรไฟล์	อีเมล	เบอร์โทร	สถานะ	วันที่สมัคร	เข้าสู่ระบบล่าสุด	จัดการ
admin admin @admin01	admin@email.com	-	รอยืนยัน	27/6/2568	27/6/2568	แก้ไข ลบ
Test User @user123	user@test.com	0856789012	ยืนยันแล้ว	22/6/2568	-	แก้ไข ลบ
Admin User @admin	admin@company.com	0987654321	ยืนยันแล้ว	22/6/2568	-	แก้ไข ลบ
John Doe @testuser	test@example.com	0812345678	ยืนยันแล้ว	22/6/2568	-	แก้ไข ลบ
Alice Brown @alice_brown	alice@example.com	-	รอยืนยัน	22/6/2568	-	แก้ไข ลบ

ภาพที่ 4-34 หน้าจัดการข้อมูลทั้งหมดของผู้ใช้

4.3.11 ส่วนการแสดงผลฐานข้อมูล และข้อมูลทั้งหมดในระบบ ดังภาพที่ 4-35

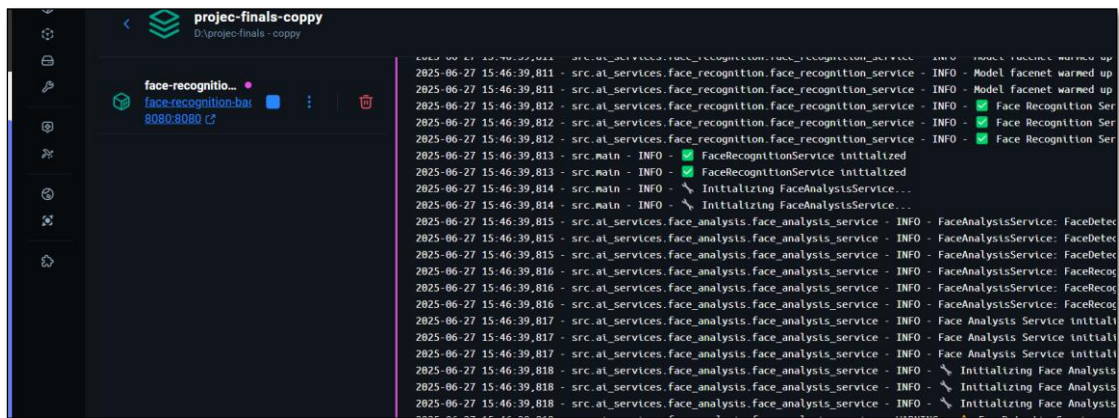


id	username	email
0d933220-2df9-41dd-a608-...	admin	admin@company.com
1c1cfe7c-e479-46d9-b64d-5...	alice_brown	alice@example.com
278e540d-ad74-4948-9cea-...	john_doe	john@example.com
3434e79d-3c6c-4adc-8329-c...	testuser	test@example.com
39305391-c5e3-4028-b662-...	user123	user@test.com
6c9ec1d9-421f-476e-8ec2-3...	bob_wilson	bob@example.com
db6a3521-1ca2-420b-b04b-...	jane_smith	jane@example.com

ภาพที่ 4-35 ส่วนการแสดงผลฐานข้อมูล

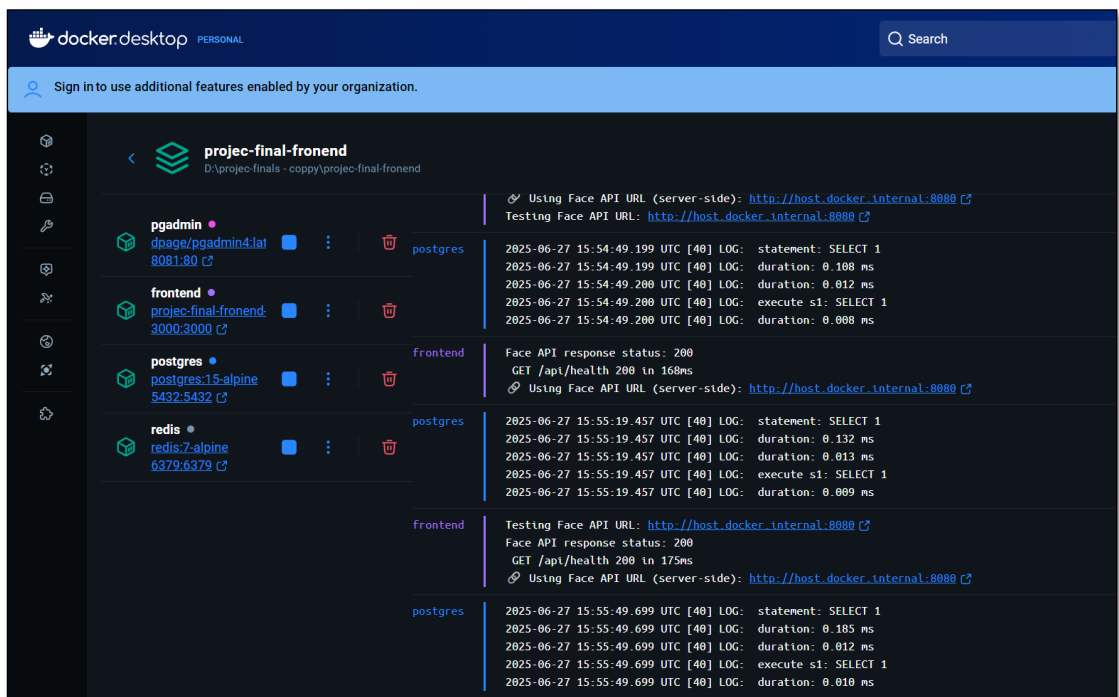
4.4 ผลการทดสอบระบบ

4.4.1 เช็สถานะการทำงานของ Docker API AI Service ดังภาพที่ 4-36



ภาพที่ 4-36 หน้าต่างสถานะการทำงานของ Docker API AI Service

4.4.2 เช็สถานะการทำงานของ Docker Frontend ดังภาพที่ 4-37



ภาพที่ 4-37 หน้าต่างสถานะการทำงานของ Docker Frontend

4.4.3 การตรวจจับใบหน้าเป็นกระบวนการที่ระบบใช้เทคโนโลยี AI ในการวิเคราะห์ และระบุตำแหน่งของใบหน้าในรูปภาพที่ผู้ใช้อัปโหลด โดยระบบจะส่งไฟล์รูปภาพไปยัง Face Detection API ที่ใช้ Model tolov11m ผ่าน FormData และได้รับข้อมูลกลับมาเป็น JSON ที่ประกอบด้วย พิกัดกรอบใบหน้า ค่าความเชื่อมั่น และคะแนนคุณภาพของแต่ละใบหน้าที่ตรวจพบ จากนั้นระบบจะนำข้อมูลนี้ไปใช้ในการวาดกรอบใบหน้าบนรูปภาพ และแสดงผลลัพธ์ให้ผู้ใช้ทราบ ดังภาพที่ 4-38

ผลลัพธ์การตรวจจับใบหน้า

ภาพที่มีกรอบใบหน้า:



ซ่อนภาพที่มีกรอบ

จำนวนใบหน้าที่พบ

35

ใบหน้าที่ 1

ความเชื่อมั่น: 87.5%
คุณภาพ: 86.25003218650818%
โมเดล: yolov11m
ขนาด: 165×183
พื้นที่: 30,195

ตำแหน่ง: (876, 2406) ถึง (1041, 2589)

เวลาประมวลผล: 6.8ms

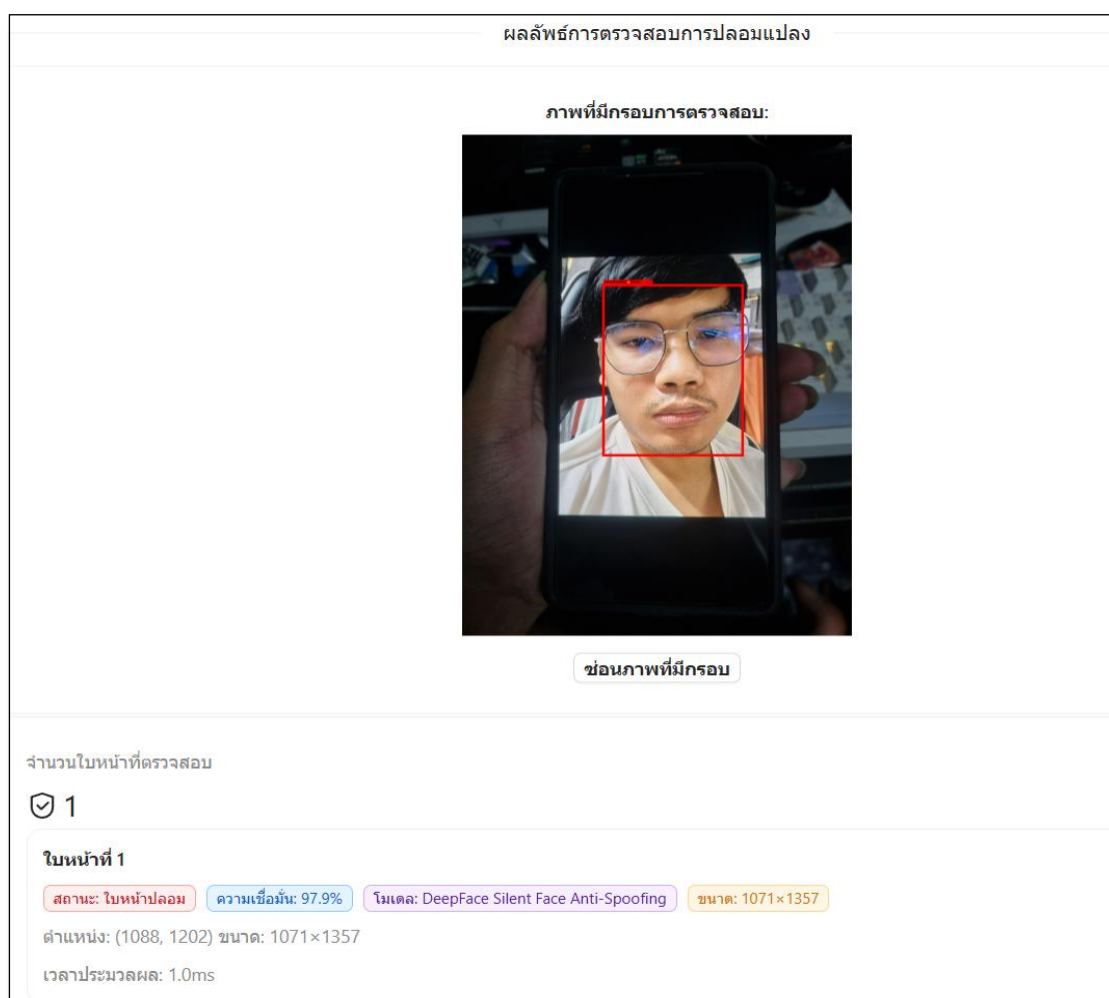
ใบหน้าที่ 2

ความเชื่อมั่น: 86.1%
คุณภาพ: 85.8168214559555%
โมเดล: yolov11m
ขนาด: 144×184
พื้นที่: 26,496

ตำแหน่ง: (3422, 2450) ถึง (3566, 2634)

ภาพที่ 4-38 การทำงานของ Face Detection

4.4.4 การตรวจจับการปลอมแปลงเป็นกระบวนการที่ระบบใช้เทคโนโลยี DeepFace Silent Face Anti-Spoofing ในการวิเคราะห์ และตรวจสอบความถูกต้องของใบหน้าเพื่อป้องกันการโจมตีแบบต่างๆ เช่น การใช้รูปถ่าย การแสดงภาพบนหน้าจอมือถือ หรือการใช้หน้ากาก โดยระบบจะรับไฟล์รูปภาพผ่าน FormData และส่งไปยัง Anti-Spoofing API ที่ใช้แบบจำลอง MiniFASNetV1 และ MiniFASNetV2 ภายใต้ Framework DeepFace ในการประมวลผล ระบบจะทำการวิเคราะห์ลักษณะเฉพาะของใบหน้าที่ไม่สามารถปลอมแปลงได้ และส่งกลับผลลัพธ์เป็น JSON ที่ประกอบด้วยค่าความเชื่อมั่น สถานะการตรวจพบการปลอมแปลง จำนวนใบหน้าจริง และปลอม พร้อมทั้งข้อมูลพิกัดของแต่ละใบหน้าที่ถูกวิเคราะห์ จากนั้นระบบจะใช้ข้อมูลนี้ในการตัดสินใจว่าจะอนุญาตให้ผู้ใช้เข้าสู่ระบบหรือไม่ ดังภาพที่ 4-39



ภาพที่ 4-39 การตรวจจับการปลอมแปลง (Anti-Spoofing)

4.4.5 การวิเคราะห์อายุ และเพศเป็นกระบวนการที่ระบบใช้เทคโนโลยี DeepFace Framework ในการประมวลผล และวิเคราะห์ลักษณะทางชีวภาพของใบหน้า เพื่อประมาณอายุ และระบุเพศของบุคคลในภาพ โดยระบบจะรับไฟล์รูปภาพผ่าน FormData หรือ Base64 encoding และส่งไปยัง Age Gender Analysis API ที่ใช้แบบจำลอง DeepFace พร้อมกับ OpenCV Detector Backend ในการตรวจจับใบหน้า และวิเคราะห์คุณลักษณะทางกายภาพ ระบบสามารถประมวลผลได้ทั้งใบหน้าเดียว และหลายใบหน้าในภาพเดียวกัน โดยส่งกลับผลลัพธ์เป็น JSON ที่ประกอบด้วยข้อมูลอายุที่ประมาณได้ เพศที่ตรวจพบพร้อมค่าความเชื่อมั่น พิกัดตำแหน่งของ ใบหน้า และจำนวนใบหน้าทั้งหมดที่พบในภาพ ข้อมูลเหล่านี้สามารถนำไปใช้ในการวิเคราะห์ข้อมูล ประชากรศาสตร์ การจัดกลุ่มผู้ใช้ หรือการปรับแต่งบริการให้เหมาะสมกับช่วงอายุ และเพศของผู้ใช้ แต่ละคน ดังภาพที่ 4-40

ภาพที่มีข้อมูลอายุและเพศ:



ซ่อนภาพที่มีข้อมูล

จำนวนใบหน้าที่พบ	อายุเฉลี่ย	เพศที่พบมากที่สุด
1	23 ปี	ชาย

ใบหน้าที่ 1

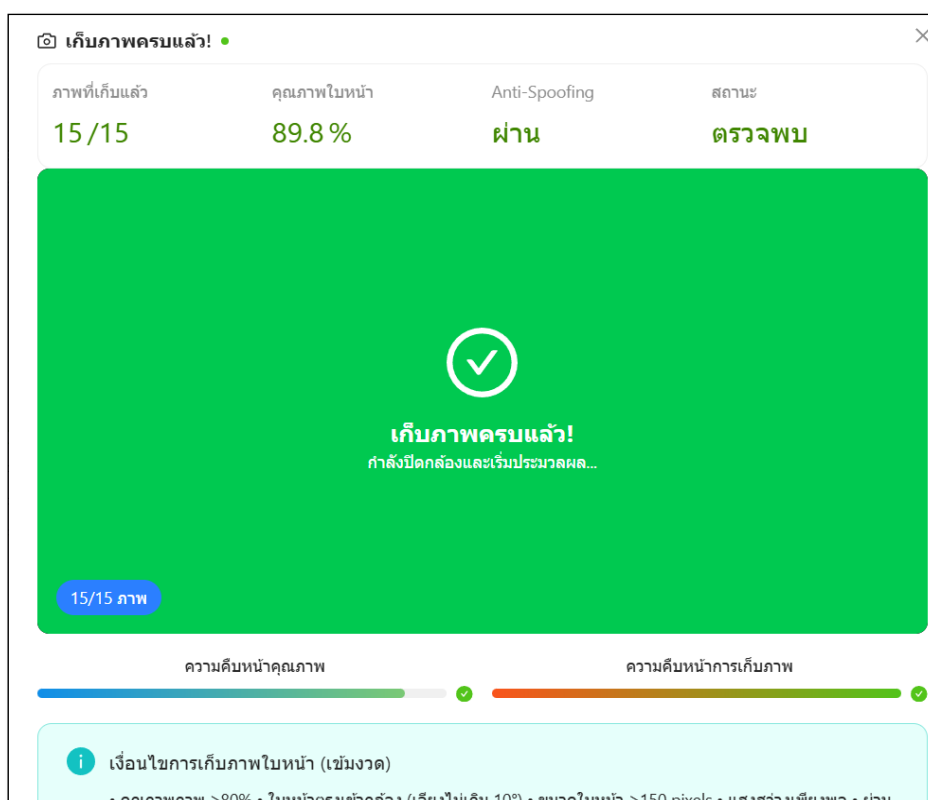
อายุ: 23 ปี
เพศ: Man
ความเชื่อมั่น: 99.4%

ตำแหน่ง: (23, 10) ขนาด: 1221×1719

ภาพที่ 4-40 การวิเคราะห์อายุ และเพศ (Age and Gender Analysis)

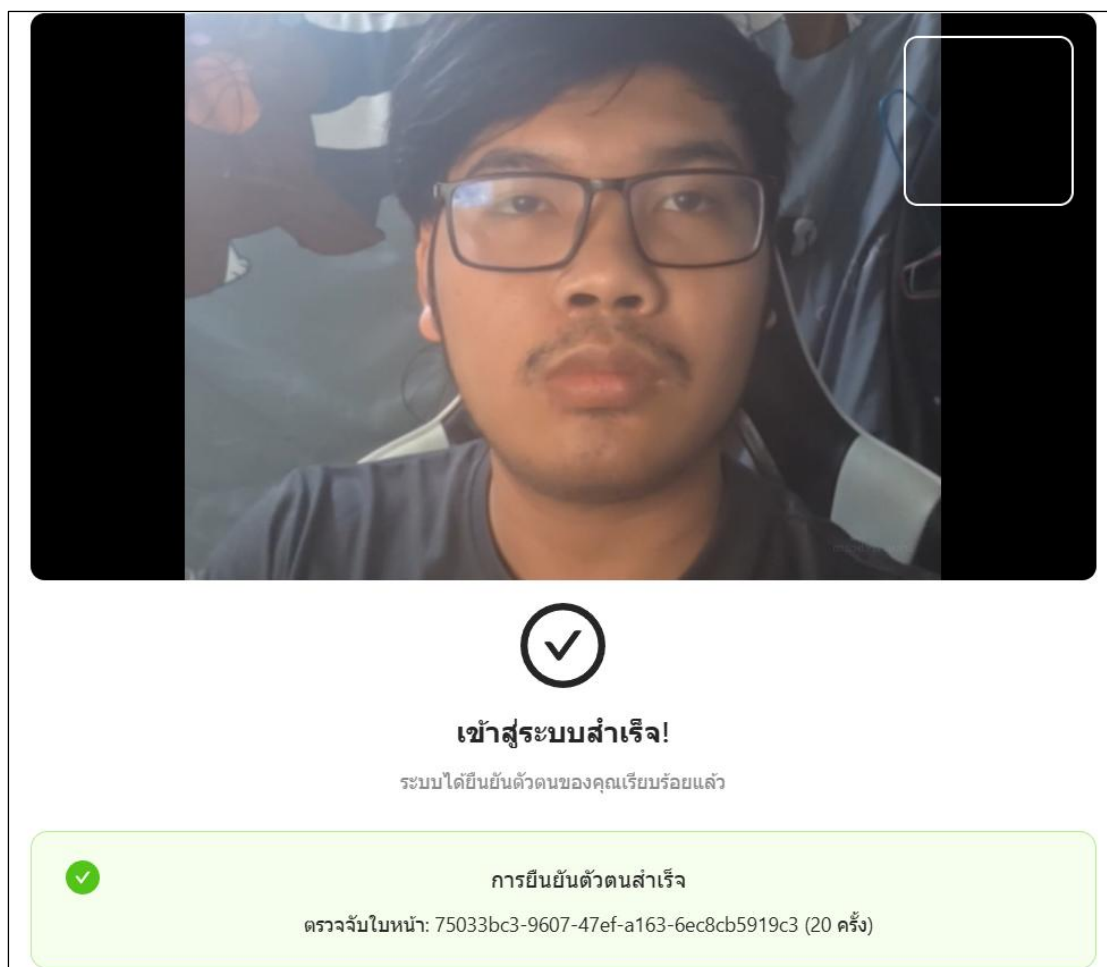
4.4.6 ระบบสแกนหน้าเพื่อสมัครสมาชิกเป็นกระบวนการที่ผสมผสานเทคโนโลยีปัญญาประดิษฐ์หลายระบบเข้าด้วยกันเพื่อให้ได้ข้อมูลใบหน้าที่มีคุณภาพสูงสุดสำหรับการลงทะเบียน โดยเริ่มจากการเปิดกล้องผ่านอินเทอร์เฟซอุปกรณ์สื่อ (MediaDevices API) และดำเนินการตรวจจับใบหน้าแบบเรียลไทม์ทุก 500 มิลลิวินาทีผ่านแบบจำลอง YOLOv11m จากนั้นระบบจะทำการตรวจสอบการปลอมแปลงด้วยเทคโนโลยีป้องกันการหลอกลวง MiniFASNet v2 Anti-Spoofing และประเมินคุณภาพภาพผ่านการประเมินคุณภาพใบหน้าแบบปรับปรุง (Enhanced Face Quality Assessment)

ตรวจสอบความคมชัด แสงสว่าง การหันหน้าตรง และตำแหน่งจุดสำคัญบนใบหน้า (facial landmarks) ระบบจะเก็บรวบรวมภาพใบหน้าคุณภาพสูงจำนวน 15 ภาพโดยอัตโนมัติโดยใช้เกณฑ์ที่เข้มงวด เช่น คะแนนคุณภาพ (Quality Score) มากกว่าหรือเท่ากับร้อยละ 80 ค่าความเชื่อมั่น มากกว่าหรือเท่ากับร้อยละ 0.85 และมุมการหันหน้า น้อยกว่าหรือเท่ากับ 10 องศา หลังจากเก็บภาพครบแล้วระบบจะสกัดการแทนข้อมูลใบหน้าแบบ 512 มิติ (512-Dimensional Face Embedding) ด้วยแบบจำลอง FaceNet จากทุกภาพ และเลือกการแทนข้อมูล (embedding) ที่มีคุณภาพสูงสุดมาบันทึกลงฐานข้อมูล PostgreSQL พร้อมทั้งลงทะเบียนเข้าสู่แพลตฟอร์มการรู้จำใบหน้า (Face Recognition Gallery) เพื่อใช้สำหรับการล็อกอินด้วยใบหน้าในอนาคต ดังภาพที่ 4-41



ภาพที่ 4-41 การทำงานของ ระบบสแกนหน้าเพื่อสมัครสมาชิก

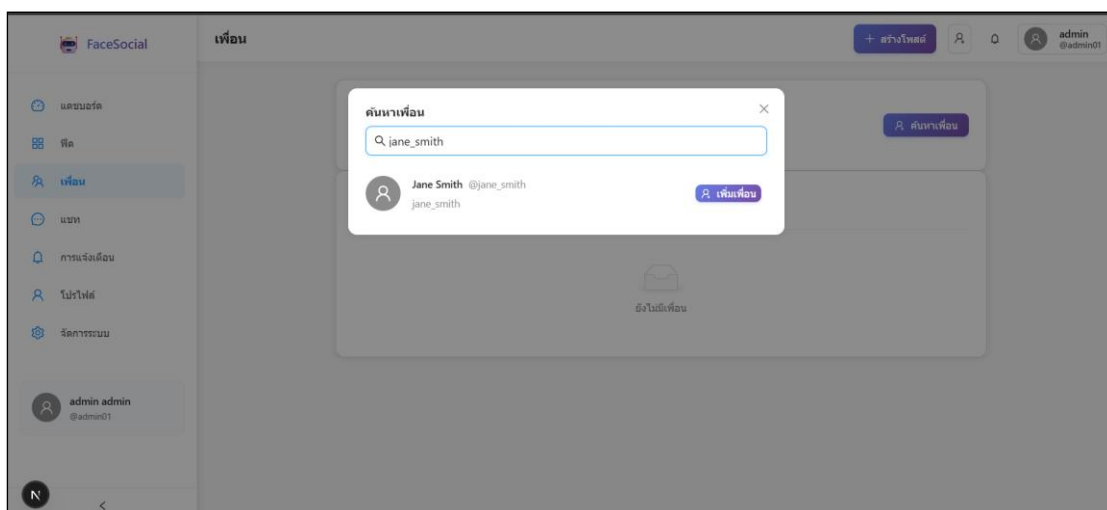
4.4.7 ระบบเข้าสู่ระบบด้วยการสแกนหน้า (Face Login System) ระบบเข้าสู่ระบบด้วยการสแกนหน้าเป็นฟีเจอร์ที่ช่วยให้ผู้ใช้สามารถเข้าสู่ระบบได้อย่างรวดเร็ว และปลอดภัยผ่านการพิสูจน์ตัวตนด้วยใบหน้า โดยการทำงานเริ่มจากระบบขออนุญาตเข้าถึงกล้องผ่าน Browser API และแสดง Modal สำหรับการสแกนใบหน้าพร้อมวิดีโอสตรีมแบบเรียลไทม์ ผู้ใช้จะจัดตำแหน่งใบหน้าให้อยู่ในวงกลมสีเขียวที่แสดงบนหน้าจอ จากนั้นระบบจะเข้าสู่โหมดการสแกนที่จับภาพอย่างต่อเนื่องด้วยความถี่ 500 มิลลิวินาที โดยมีเป้าหมายจับภาพทั้งหมด 20 ภาพเพื่อความแม่นยำ ในแต่ละการจับภาพระบบจะใช้ Canvas API แปลงเฟรมจากวิดีโอเป็นภาพนิ่ง และส่งไปประมวลผลผ่าน Face Detection API เพื่อตรวจหาใบหน้า และได้รับพิกัดกรอบใบหน้า ผ่าน Anti-Spoofing API เพื่อตรวจสอบว่าเป็นใบหน้าจริงหรือการปลอมแปลง และทำการตัดภาพใบหน้าออกจากภาพต้นฉบับตามพิกัดที่ได้รับ ภาพใบหน้าที่ผ่านการตรวจสอบจะถูกแปลงเป็น Base64 และส่งไปยัง Face Recognition API โดยใช้แบบจำลอง FaceNet เพื่อสกัด Face Embedding และเปรียบเทียบกับข้อมูลที่ลงทะเบียนไว้ในระบบ ระบบจะรวบรวมผลลัพธ์จากการสแกนทั้งหมด และกรองเฉพาะผลลัพธ์ที่เป็นใบหน้าจริง มี person_id ที่ระบุได้ และมีค่า Confidence มากกว่าหรือเท่ากับร้อยละ 80 จากนั้นจะนับคะแนนโหวตของแต่ละ Identity และเลือก Identity ที่ได้คะแนนโหวตสูงสุดเป็นผู้ชนะ เมื่อระบุตัวตนได้แล้วระบบจะส่งข้อมูลไปยัง Login API พร้อมกับ FacelImageBase64 โดย API จะเรียกใช้ Face Recognition Service อีกครั้งเพื่อยืนยันผลลัพธ์ ค้นหาผู้ใช้ในฐานข้อมูล PostgreSQL ด้วย person_id ที่ได้ สร้าง JWT Token สำหรับ Session Management อัปเดต Lastloginat ในฐานข้อมูล และส่งคืนข้อมูลผู้ใช้พร้อม Authentication Token ระบบมีการจัดการข้อผิดพลาดหลายระดับ เช่น หากตรวจพบการปลอมแปลงเกิน 3 ครั้งจะหยุดการสแกน และขอให้ลองใหม่ หากไม่พบใบหน้าที่ตรงกันจะแจ้งให้ลงทะเบียนก่อน และมีการตั้งเวลาหมดอายุสำหรับการเรียก API แต่ละตัว โดยระบบไม่มีการเก็บบันทึกภาพใดๆ เพื่อปกป้องความเป็นส่วนตัวของผู้ใช้ ข้อได้เปรียบของระบบนี้คือความสะดวกรวดเร็วในการเข้าสู่ระบบภายใน 10-15 วินาที ความแม่นยำสูงจากการใช้ภาพหลายภาพ และการโหวต ความปลอดภัยจากการตรวจสอบ Anti-Spoofing และการใช้งานแบบ Touchless ที่เหมาะสำหรับสถานการณ์ต่าง ๆ ดังภาพที่ 4-42



ภาพที่ 4-42 การทำงานของ ระบบเข้าสู่ระบบด้วยการสแกนหน้า

4.4.8 การค้นหาเพื่อน (Friend Search) ระบบการค้นหาเพื่อนเป็นฟีเจอร์สำคัญที่ช่วยให้ผู้ใช้สามารถค้นหา และเชื่อมต่อกับผู้ใช้คนอื่นในระบบได้อย่างง่ายดาย และรวดเร็ว โดยการทำงานเริ่มจากผู้ใช้เข้าสู่หน้าเพื่อน และคลิกปุ่ม ค้นหาเพื่อน ซึ่งจะเปิด Modal ที่มีช่องค้นหาพร้อมไอคอน SearchOutlined และข้อความแนะนำให้พิมพ์อย่างน้อย 2 ตัวอักษรเพื่อเริ่มการค้นหา เมื่อผู้ใช้พิมพ์คำค้นหา ระบบจะใช้ useEffect และ SetTimeout เพื่อรอ 500 มิลลิวินาทีก่อนเรียก API เพื่อป้องกันการเรียก API บ่อยเกินไปขณะที่ผู้ใช้งานกำลังพิมพ์ ระบบจะส่งคำค้นหาไปยัง Friend Search API พร้อมกับ Authorization token โดย API จะตรวจสอบสิทธิ์ของผู้ใช้ผ่าน JWT verification และค้นหาในฐานข้อมูล PostgreSQL ด้วย Prisma ORM ตามเกณฑ์ที่กำหนด ได้แก่ Username Firstname Lastname และ Email โดยใช้ Case-Insensitive Search และกรองเฉพาะผู้ใช้ที่ไม่ใช่ตัวเองเท่านั้น ผลลัพธ์ที่ได้จะถูกจำกัดจำนวนตาม Limit Parameter และส่งกลับเป็น JSON ประกอบด้วยข้อมูลผู้ใช้ที่จำเป็น เช่น Id Username Firstname Lastname และ Profileimageurl เมื่อได้รับผลลัพธ์แล้วระบบจะแสดงรายการผู้ใช้ใน List component ของ Ant Design โดยแต่ละ

รายการจะประกอบด้วย Avatar ที่แสดงรูปประวัติโดยย่อหรือไอคอนผู้ใช้เริ่มต้น ชื่อ-นามสกุลของผู้ใช้ ชื่อผู้ใช้ (Username) และปุ่มการดำเนินการที่เปลี่ยนแปลงตามสถานะ หากยังไม่ได้ส่งคำขอเป็นเพื่อน จะแสดงปุ่ม "เพิ่มเพื่อน" แต่หากส่งคำขอแล้วจะแสดง Tag สีส้มที่บอกว่า ส่งคำขอแล้ว เพื่อป้องกันการส่งคำขอซ้ำ เมื่อผู้ใช้คลิกปุ่ม "เพิ่มเพื่อน" ระบบจะเรียก Friend Request API ด้วย POST Method และส่ง friendId ของผู้ใช้ที่ต้องการเพิ่มไปในฐานข้อมูล หาก API ตอบสนองสำเร็จจะแสดงข้อความยืนยัน และอัปเดตสถานะใน Set ของ Sentrequests เพื่อเปลี่ยนปุ่มเป็น Tag ส่งคำขอแล้ว ทั้งนี้ ระบบยังมีการจัดการข้อผิดพลาดที่อาจเกิดขึ้น เช่น การไม่มี Token การเชื่อมต่อ API ล้มเหลว หรือข้อผิดพลาดอื่นๆ โดยแสดงข้อความแจ้งเตือนที่เหมาะสมให้ผู้ใช้ทราบ ระบบการค้นหาเพื่อนนี้ ช่วยให้ผู้ใช้สามารถขยายเครือข่ายสังคมในแพลตฟอร์มได้อย่างมีประสิทธิภาพ ด้วยการค้นหาที่รวดเร็ว และแม่นยำ การแสดงผลที่เป็นมิตรต่อผู้ใช้ และการจัดการสถานะคำขอเป็นเพื่อนที่ชัดเจน ทำให้ประสบการณ์การใช้งานเป็นไปอย่างราบรื่น และสะดวกสบาย ดังภาพที่ 4-43

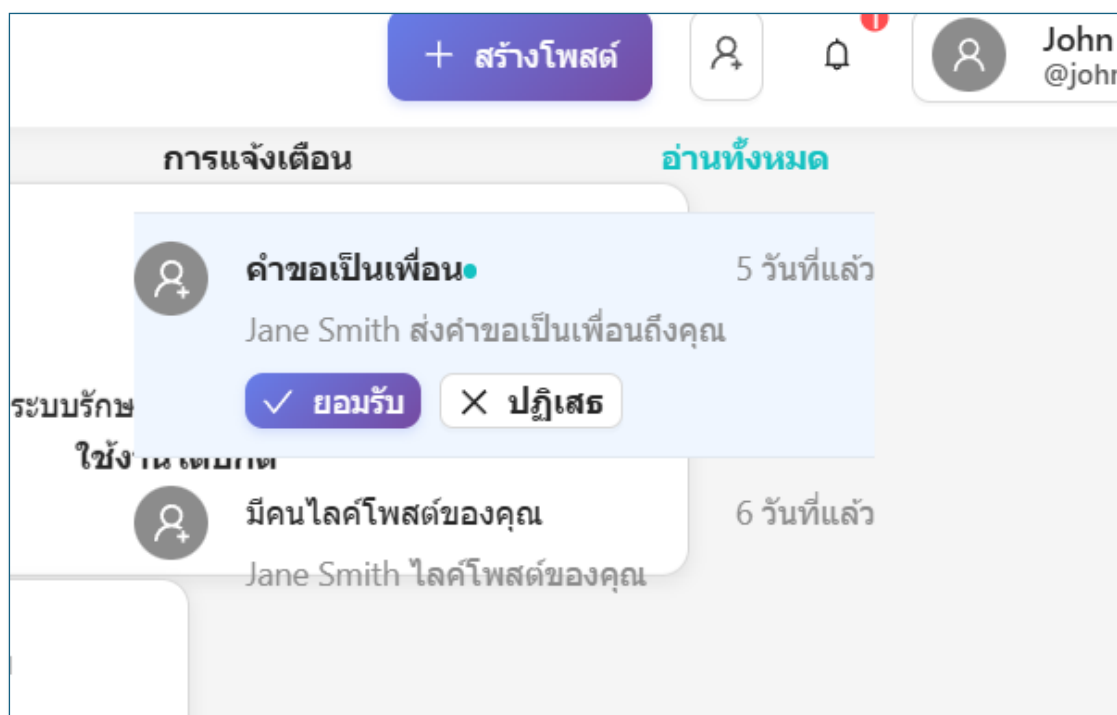


ภาพที่ 4-43 การค้นหาเพื่อน (Friend Search)

4.4.9 การแจ้งเตือน (Notification System) ระบบการแจ้งเตือนเป็นฟีเจอร์หลักที่ช่วยให้ผู้ใช้ได้รับข้อมูลข่าวสาร และกิจกรรมต่างๆ ที่เกี่ยวข้องกับบัญชีของตนอย่างทันท่วงที ระบบทำงานโดยแสดงการแจ้งเตือนในรูปแบบ Dropdown บนแถบนำทางหลักพร้อมกับไอคอนกระดิ่ง (BellOutlined) ที่มี Badge แสดงจำนวนการแจ้งเตือนที่ยังไม่ได้อ่าน เมื่อผู้ใช้คลิกที่ไอคอนจะเปิด Modal หรือ Dropdown ที่แสดงรายการการแจ้งเตือนล่าสุด โดยระบบจะดึงข้อมูลจาก Notifications API ที่ใช้ JWT Token เพื่อยืนยันตัวตน และดึงการแจ้งเตือนเฉพาะของผู้ใช้นั้นจากฐานข้อมูล PostgreSQL

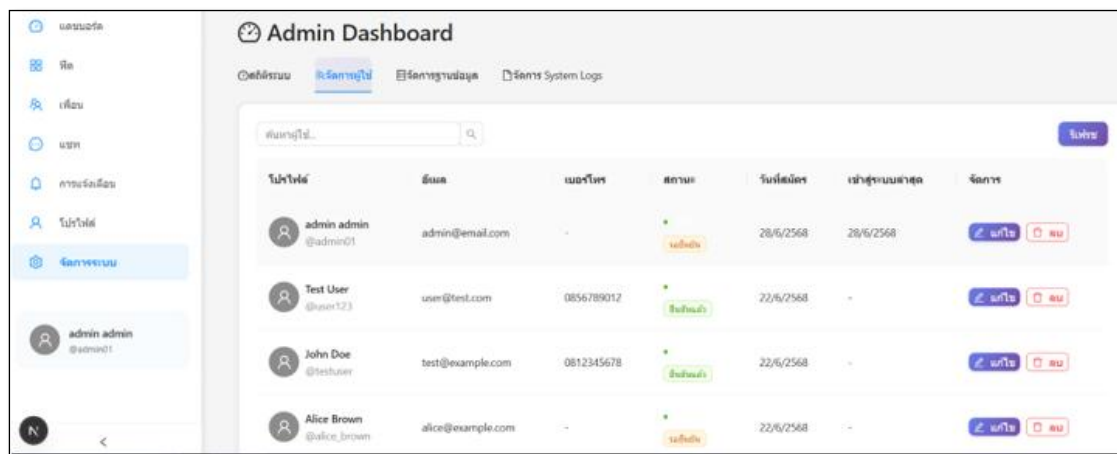
ระบบจัดหมวดหมู่การแจ้งเตือนเป็น 5 ประเภทหลัก ได้แก่ FRIEND_REQUEST สำหรับคำขอเป็นเพื่อน FRIEND_ACCEPTED สำหรับการยอมรับคำขอเป็นเพื่อน POST_LIKE สำหรับการ

กดไลค์โพสต์ POST_COMMENT สำหรับการแสดงความเห็นในโพสต์ และFACE_TAG สำหรับการแท็กใบหน้าในรูปภาพ แต่ละประเภทจะมีไอคอน และสีที่แตกต่างกัน เช่น UserAddOutlined สีน้ำเงินสำหรับคำขอเป็นเพื่อน HeartOutlined สีแดงสำหรับการไลค์ MessageOutlined สีเขียวสำหรับความเห็น และTagOutlined สีม่วงสำหรับการแท็ก การแสดงผลแต่ละการแจ้งเตือน ประกอบด้วย Avatar ของผู้ส่ง ชื่อเรื่อง และข้อความแจ้งเตือน เวลาที่เกิดขึ้นแบบ Relative Time และสถานะการอ่านที่แสดงด้วยพื้นหลังสีฟ้าอ่อนสำหรับข้อความที่ยังไม่ได้อ่าน สำหรับการแจ้งเตือนประเภทคำขอเป็นเพื่อนจะมีปุ่มดำเนินการ ยอมรับ และ ปฏิเสธ ที่ส่งคำขอไปยัง Friend Request API เพื่อจัดการคำขอนั้น เมื่อผู้ใช้คลิกที่การแจ้งเตือนที่ยังไม่ได้อ่าน ระบบจะเรียก Mark as Read API เพื่ออัปเดตสถานะ และลบ Badge จำนวนข้อความที่ยังไม่ได้อ่าน ระบบยังมีฟีดเจอร์ อ่านทั้งหมด ที่ช่วยให้ผู้ใช้สามารถทำเครื่องหมายการแจ้งเตือนทั้งหมดว่าอ่านแล้วในคลิกเดียว และมีหน้าการแจ้งเตือนแบบเต็ม Notifications ที่แสดงรายการการแจ้งเตือนทั้งหมดพร้อมกับฟีดเจอร์การกรอง และการจัดการที่ละเอียดมากขึ้น ระบบใช้ Dayjs Library เพื่อแสดงเวลาแบบ Relative Format และมีการจัดการ Error Handling สำหรับกรณีที่ API เรียกไม่สำเร็จ การแจ้งเตือนจะถูกเก็บในฐานข้อมูลพร้อมกับข้อมูล Metadata ที่จำเป็น เช่น Friendship_Id สำหรับการแจ้งเตือนคำขอเป็นเพื่อนเพื่อให้สามารถดำเนินการต่อได้อย่างถูกต้อง ระบบนี้ช่วยเพิ่มประสิทธิภาพการใช้งานโดยทำให้ผู้ใช้ไม่พลาดกิจกรรมสำคัญ และสามารถตอบสนองต่อการปฏิสัมพันธ์ทางสังคมได้อย่างทันท่วงที ดังภาพที่ 4-44



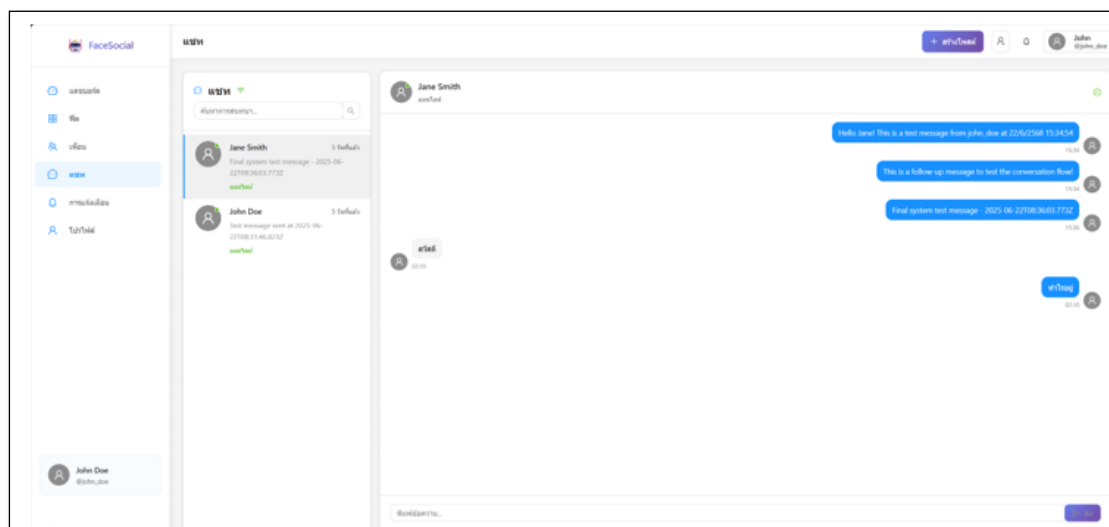
ภาพที่ 4-44 การแจ้งเตือน (Notification System)

4.4.10 Admin Dashboard เป็นระบบการจัดการแบบรวมศูนย์ที่ให้ผู้ดูแลระบบสามารถควบคุม และติดตามการทำงานของแพลตฟอร์มได้แบบ Real-time โดยประกอบด้วยแผงข้อมูลสถิติระบบที่แสดงจำนวนผู้ใช้ทั้งหมด ผู้ใช้ที่ใช้งานอยู่ จำนวนโพสต์ ข้อความ และผู้ใช้ที่ออนไลน์ ระบบจัดการผู้ใช้ที่ให้สามารถดู แก้ไข ลบผู้ใช้ รวมทั้งจัดการสถานะการยืนยันตัวตน การจัดการฐานข้อมูลที่ครอบคลุมการเก็บข้อมูล Face Embeddings, ข้อมูล Chat และสถิติการใช้งานระบบติดตาม และจัดการ System Logs เพื่อ Monitor การทำงานของระบบ รวมถึงการจัดการไฟล์ และการอัปโหลดข้อมูล ทุกฟีเจอร์ถูกป้องกันด้วยระบบ Authentication และ Authorization เพื่อให้เฉพาะผู้ดูแลระบบเท่านั้นที่สามารถเข้าถึงได้ โดยใช้ JWT Token และการตรวจสอบสิทธิ์แบบหลายชั้น พร้อมแสดงผลในรูปแบบ Dashboard ที่เข้าใจง่ายด้วย Ant Design Components ดังภาพที่ 4-45



ภาพที่ 4-45 Admin Dashboard

4.4.11 Chat System ระบบ Chat เป็นระบบการสนทนาแบบ Real-Time ที่ใช้เทคโนโลยี Server-Sent Events (SSE) ในการส่งข้อความระหว่างผู้ใช้ 2 คน โดยระบบจะสร้าง Conversation ID จากการเรียงลำดับ User ID ของผู้เข้าร่วม และใช้ React Hook ชื่อ Userealtimechat ในการจัดการการเชื่อมต่อ Real-Time กับ API Endpoints ที่สร้างด้วย Next.Js API Routes เมื่อผู้ใช้พิมพ์ข้อความ ระบบจะบันทึกหลักฐานข้อมูล และส่งไปยังผู้รับผ่าน SSE Connection พร้อมทั้งแสดงสถานะออนไลน์ และ Typing Indicator แบบ Real-Time โดยมีระบบ Fallback เป็น Polling เมื่อ SSE ไม่สามารถใช้งานได้ และรองรับการโหลดประวัติการสนทนาแบบ Pagination พร้อมการจัดการข้อผิดพลาด และการเชื่อมต่อใหม่อัตโนมัติ ดังภาพที่ 4-46



ภาพที่ 4-46 Chat System

4.4.12 การโพสต์และการแท็กใบหน้า (Post and Face Tagging) ระบบการโพสต์และการแท็กใบหน้าเป็นฟีเจอร์ที่ผสมผสานระหว่างการสร้างโพสต์โซเชียลมีเดียและเทคโนโลยีปัญญาประดิษฐ์สำหรับการจดจำใบหน้า โดยเมื่อผู้ใช้อัปโหลดรูปภาพผ่าน PostCreateModal ระบบจะใช้ Face Detection API เพื่อตรวจจับใบหน้าทั้งหมดในภาพ และนำไปสู่ Face Recognition API เพื่อระบุตัวตนของแต่ละคน จากนั้นผู้ใช้สามารถเปิด FaceTagModal เพื่อเลือกและยืนยันการแท็กเพื่อนที่ระบบแนะนำ พร้อมบันทึกพิกัดกรอบล้อมรอบ (Bounding Box) ของใบหน้าแต่ละคนลงฐานข้อมูล และส่งการแจ้งเตือนไปยังผู้ถูกแท็กผ่านระบบการแจ้งเตือน (Notification) ซึ่งช่วยให้ผู้ใช้สามารถระบุและติดแท็กเพื่อนในรูปภาพได้อย่างสะดวกและรวดเร็วด้วยความช่วยเหลือของปัญญาประดิษฐ์ ดังภาพที่ 4-47

เนื้อหา

คุณกำลังคิดอะไรอยู่?

0 / 500

รูปภาพ



แท็กเพื่อน




แท็กเพื่อนอัตโนมัติ (AI ตรวจจับใบหน้า)

แท็กแล้ว 1 คน:



admin admin ×

สถานที่



เพิ่มสถานที่

การมองเห็น

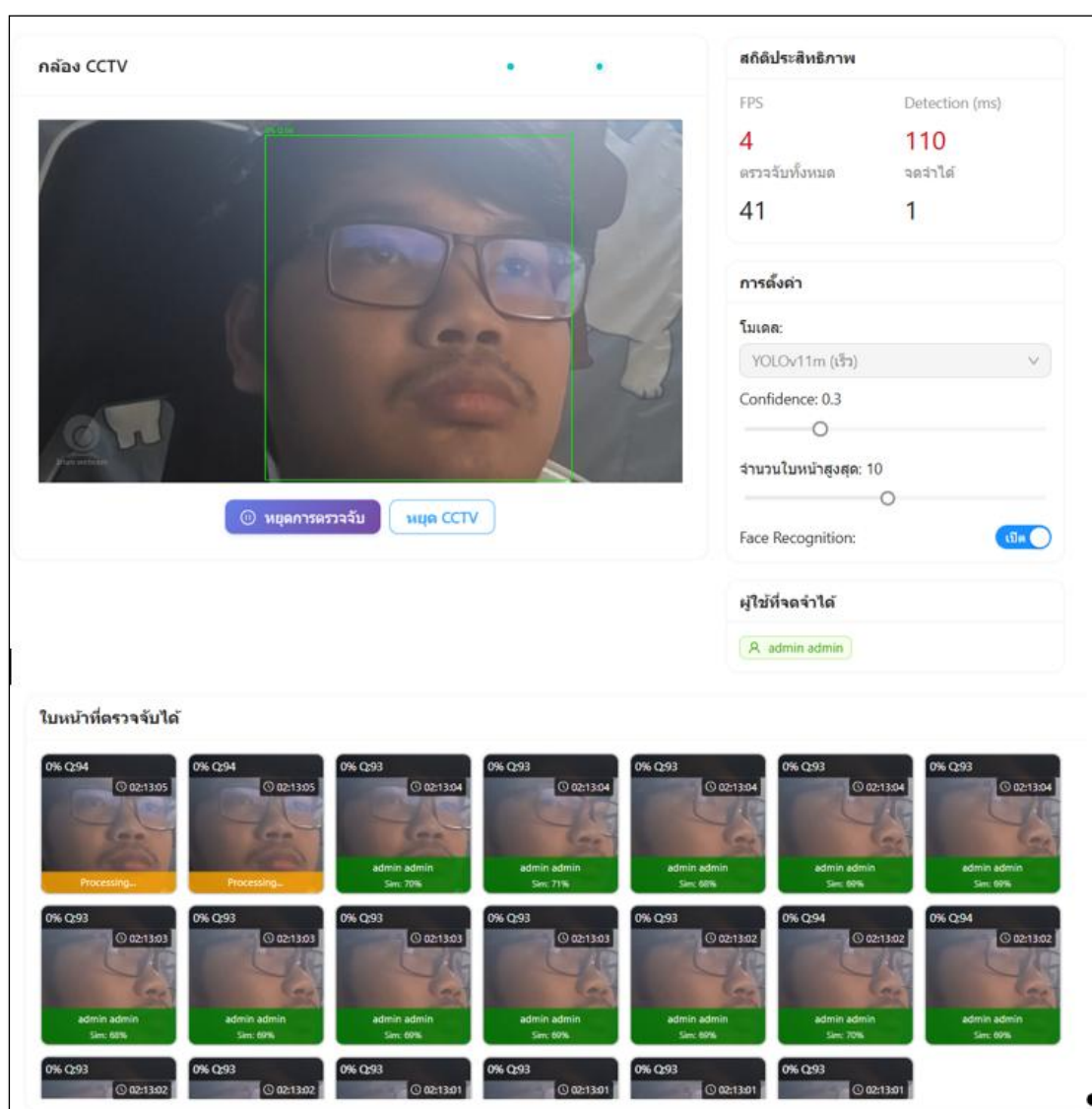
สาธารณะ

☐

ภาพที่ 4-47 การโพสต์ และแท็กใบหน้า (Post and Face Tagging)

4.4.13 Closed-Circuit Television (CCTV) ระบบ CCTV เป็นระบบเฝ้าระวังความปลอดภัยแบบ Real-time ที่ใช้เทคโนโลยี AI ในการตรวจจับ และจดจำใบหน้า โดยผ่านการเชื่อมต่อกับกล้องผ่าน WebRTC และใช้ Face Detection API เพื่อวิเคราะห์ใบหน้าในแต่ละเฟรมแบบ Real-Time พร้อมแสดงกรอบล้อมรอบใบหน้าที่ตรวจพบพร้อมค่า Confidence และ Quality Score

ระบบสามารถปรับแต่งความไวในการตรวจจับ จำนวนใบหน้าสูงสุด คุณภาพขั้นต่ำ และเปิดหรือปิดการทำงานของ Face Recognition ได้ รวมถึงติดตามสถิติประสิทธิภาพแบบ Real-Time เช่น FPS, เวลาที่ใช้ในการ Detection และ Recognition, จำนวนการตรวจจับทั้งหมด และแสดงรายชื่อผู้ใช้ที่สามารถจดจำได้ ระบบยังบันทึกภาพใบหน้าที่ถูกแยกมาพร้อมข้อมูลเวลา และผลการจดจำ ทำให้สามารถใช้งานได้อย่างจริงสำหรับการเฝ้าระวังความปลอดภัยในองค์กรหรือพื้นที่ที่ต้องการการควบคุมการเข้าถึง ดังภาพที่ 4-48



ภาพที่ 4-48 Closed-Circuit Television (CCTV)

จากภาพที่ 4-48 และข้อมูลทั้งหมด จะสังเกตได้ว่าการทำงานของระบบทำงานถูกต้องตามที่ได้กำหนดไว้ของ แพลตฟอร์มสื่อสังคมออนไลน์ และการจัดการรูปภาพบุคคล ด้วยการจดจำใบหน้า

บทที่ 5

สรุป และอภิปรายผล

ในการจัดทำระบบแพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพบุคคลด้วยการจดจำใบหน้า โดยผู้พัฒนาระบบได้ทำการประเมินประสิทธิภาพของระบบ โดยแบ่งการทดสอบออกเป็น 3 ส่วนหลัก คือการทดสอบอัลกอริทึม AI การทดสอบระบบแพลตฟอร์ม และการทดสอบการใช้งานจริง เพื่อวิเคราะห์หาประสิทธิภาพของระบบ หลังจากได้ผลการดำเนินงานแล้ว สามารถสรุป และอภิปรายผลดังนี้

- 5.1 สรุปผลการดำเนินงาน
- 5.2 อภิปรายผลการดำเนินงาน
- 5.3 ข้อเสนอแนะ และแนวทางการพัฒนา

5.1 สรุปผลการดำเนินงาน

จากการดำเนินงานพัฒนาระบบแพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพบุคคลด้วยการจดจำใบหน้า สามารถสรุปผลลัพธ์ที่ได้รับจากการศึกษาและพัฒนาระบบออกเป็น 3 ส่วนหลัก ดังนี้

5.1.1 ผลการประเมินและเปรียบเทียบประสิทธิภาพอัลกอริทึมการเรียนรู้เชิงลึก

การวิจัยนี้ได้ทำการศึกษาเปรียบเทียบอัลกอริทึม AI จำนวน 16 แบบจำลอง ในการพัฒนาระบบจดจำใบหน้าที่มีประสิทธิภาพ โดยแบ่งออกเป็น 4 กลุ่มหลัก

5.1.1.1 สำหรับการตรวจจับใบหน้า Face Detection ได้ทำการทดสอบ 9 แบบจำลอง ประกอบด้วย YOLO Series ได้แก่ YOLOv11m, YOLOv11n, YOLOv9c, YOLOv9e และ Specialized Models ได้แก่ MTCNN, RetinaFace, InsightFace รวมถึง Lightweight Models ได้แก่ BlazeFace, Haar Cascade ผลการทดสอบพบว่า YOLOv11m ให้ความแม่นยำสูงสุดร้อยละ 100 และมีประสิทธิภาพในการครอบตัดใบหน้าที่ดีที่สุด โดยสามารถตรวจจับใบหน้าได้ 99 ผลลัพธ์ จากการทดสอบ 49 ภาพ ซึ่งแสดงให้เห็นความแม่นยำในการระบุตำแหน่งใบหน้าโดยไม่มีการตรวจจับซ้ำ

5.1.1.2 ในส่วนของการจดจำใบหน้า Face Recognition ได้ทำการฝึกฝนและทดสอบ 3 แบบจำลองหลัก คือ AdaFace IR-101, ArcFace R100 และ FaceNet VGGFace2 โดยทั้งสามแบบจำลองได้รับการฝึกฝนใหม่และ Fine-tuning ด้วยชุดข้อมูล Diverse Asian Facial Ages ที่ประกอบด้วยภาพบุคคลไทยและเอเชีย 18,000 ภาพ จาก 1,000 บุคคล ผลการประเมินแบบถ่วงน้ำหนักพบว่า FaceNet VGGFace2 ได้คะแนนสูงสุด 93.0 และให้ความแม่นยำร้อยละ 99.46 พร้อมกับประสิทธิภาพที่เหนือกว่าในสภาพแวดล้อมท้าทาย โดยเฉพาะในการจดจำใบหน้าในภาพกลุ่มที่มีความมั่นใจร้อยละ 65.7 เทียบกับ AdaFace ร้อยละ 42.6 และ ArcFace ร้อยละ 39.9

5.1.1.3 การตรวจจับการปลอมแปลง Deepfake Detection ได้ทำการทดสอบ 4 แบบจำลอง โดยพบว่า DeepFace Silent Face Anti-Spoofing มีประสิทธิภาพสูงสุด ด้วย

MiniFASNetV2 ที่ให้ความแม่นยำร้อยละ 98.7 และ Equal Error Rate (EER) ต่ำสุดที่ร้อยละ 1.3 ซึ่งสามารถตรวจจับ Photo Attack ได้ร้อยละ 100 และ Face Swap Attack ได้ร้อยละ 94.3 พร้อมเวลาอนุมานเพียง 15.2 มิลลิวินาที นอกจากนี้ ระบบยังใช้ DeepFace Framework สำหรับการวิเคราะห์อายุและเพศ ที่สามารถประมาณอายุและระบุเพศของบุคคลในภาพได้อย่างแม่นยำ รองรับ การวิเคราะห์ทั้งใบหน้าเดียวและหลายใบหน้าในภาพเดียวกัน

5.1.2 ผลการพัฒนาระบบแพลตฟอร์มสื่อสังคมออนไลน์

5.1.2.1 การนำเทคโนโลยี AI มาผสานกับแพลตฟอร์มโซเชียลมีเดียได้สำเร็จอย่างสมบูรณ์ โดยระบบได้รับการออกแบบเป็น Microservices Architecture โดยใช้ Docker Containers เพื่อการจัดการที่ยืดหยุ่น ซึ่งประกอบด้วย Frontend ที่พัฒนาด้วย Next.js React, Backend Services หลายตัวที่ทำงานแยกอิสระ, API Gateway ที่ใช้ Kong Gateway และ ฐานข้อมูลหลายรูปแบบได้แก่ Milvus Vector Database, Redis, PostgreSQL

5.1.2.2 ระบบมีฟีเจอร์ครบครันสำหรับแพลตฟอร์มโซเชียลมีเดียที่ผสานเทคโนโลยี AI อย่างลงตัว เริ่มต้นจากการลงทะเบียนและเข้าสู่ระบบด้วยใบหน้า ที่ช่วยให้ผู้ใช้สามารถสมัครสมาชิก และเข้าสู่ระบบโดยการสแกนใบหน้าโดยไม่ต้องใช้รหัสผ่าน ตลอดจนการโพสต์และแชร์ภาพพร้อมการ แท็กใบหน้าอัตโนมัติ ที่ทำให้ระบบสามารถตรวจจับและระบุตัวตนของบุคคลในภาพโดยอัตโนมัติ

5.1.2.3 นอกจากนี้ ระบบยังมีการสนทนาแบบ Real-time ที่ใช้เทคโนโลยี Server-Sent Events (SSE) สำหรับการสนทนาทันที รวมถึงการค้นหาและเพิ่มเพื่อน ที่ช่วยให้ผู้ใช้สามารถค้นหา ผู้ใช้และจัดการความสัมพันธ์เพื่อนได้อย่างง่ายดาย ระบบการแจ้งเตือนแบบ Real-time สำหรับ กิจกรรมต่างๆ ระบบ CCTV สำหรับความปลอดภัยที่สามารถเฝ้าระวังและตรวจจับใบหน้าแบบ Real-time และ Admin Dashboard ที่เป็นระบบจัดการแบบรวมศูนย์สำหรับผู้ดูแลระบบ

5.1.3 ผลการทดสอบประสิทธิภาพการใช้งานจริง

จากการทดสอบระบบในสภาพแวดล้อมจริงพบว่า ระบบมีประสิทธิภาพการทำงานที่น่าพอใจ โดยสามารถรองรับผู้ใช้หลายคนพร้อมกัน มีความเสถียรในการทำงาน และสามารถประมวลผลข้อมูล ได้รวดเร็ว ซึ่งเวลาตอบสนองของระบบอยู่ในระดับที่ยอมรับได้สำหรับการใช้งาน Real-time

ความแม่นยำของระบบก็อยู่ในระดับสูง โดยการเข้าสู่ระบบด้วยใบหน้าใช้เวลาเพียง 10-15 วินาที ระบบ Anti-Spoofing สามารถป้องกันการโจมตีได้อย่างมีประสิทธิภาพ และการแท็กใบหน้า อัตโนมัติในโพสต์ทำงานได้อย่างแม่นยำ ทำให้ระบบมีความครบถ้วนของฟีเจอร์สำหรับแพลตฟอร์ม โซเชียลมีเดีย พร้อมทั้งเทคโนโลยี AI ที่เสริมประสบการณ์การใช้งานให้ดีขึ้นอย่างเห็นได้ชัด

5.2 อภิปรายผลการดำเนินงาน

จากผลการศึกษาและพัฒนาระบบแพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพ บุคคลด้วยการจดจำใบหน้า สามารถอภิปรายผลการดำเนินงานได้ในประเด็นต่างๆ ที่สำคัญ ดังนี้

5.2.1 ข้อดีและประโยชน์ของระบบที่พัฒนา

ระบบที่พัฒนาขึ้นได้นำเสนอข้อดีที่สำคัญหลายประการ โดยเฉพาะในด้านความปลอดภัย และการรักษาความมั่นคง การใช้เทคโนโลยีการจดจำใบหน้าทำให้ระบบมีความปลอดภัยสูงกว่าการใช้รหัสผ่านแบบดั้งเดิมอย่างมาก เนื่องจากใบหน้าของแต่ละคนมีลักษณะเฉพาะที่ไม่สามารถปลอมแปลงได้ง่าย นอกจากนี้ ระบบ Anti-Spoofing ยังสามารถตรวจจับการใช้รูปถ่าย วิดีโอ หรือหน้ากากในการหลอกระบบได้อย่างมีประสิทธิภาพ ด้วยความแม่นยำร้อยละ 98.7 ซึ่งช่วยเสริมสร้างความเชื่อมั่นในการใช้งาน

ในด้านความสะดวกและประสบการณ์การใช้งาน ระบบมอบประสบการณ์การเข้าสู่ระบบแบบ Touchless ที่ทำให้ผู้ใช้ไม่ต้องจํารหัสผ่านหรือกังวลเรื่องการถูกขโมยบัญชี สามารถเข้าสู่ระบบได้ด้วยการสแกนใบหน้าเพียง 10-15 วินาที การแท็กใบหน้าอัตโนมัติทำให้ผู้ใช้ไม่ต้องแท็กเพื่อนในรูปภาพด้วยตนเอง ระบบสามารถระบุและแนะนำการแท็กได้โดยอัตโนมัติ ช่วยประหยัดเวลาและเพิ่มความสะดวกในการใช้งาน

ประสิทธิภาพและความแม่นยำของระบบถือเป็นจุดเด่นที่สำคัญ การใช้แบบจำลอง YOLOv11m ให้ความแม่นยำร้อยละ 100 ในการตรวจจับใบหน้า และ FaceNet VGGFace2 ให้ความแม่นยำร้อยละ 99.46 ในการจดจำใบหน้า ทำให้ระบบสามารถทำงานแบบ Real-Time ได้อย่างมีประสิทธิภาพ โดยเฉพาะการ Fine-tuning ด้วยชุดข้อมูล Diverse Asian Facial Ages ที่ช่วยเพิ่มประสิทธิภาพสำหรับบุคคลไทยและเอเชียอย่างชัดเจน

5.2.2 ข้อจำกัดและข้อเสียของระบบ

แม้ว่าระบบจะมีข้อดีมากมาย แต่ก็มีข้อจำกัดที่ควรพิจารณา โดยเฉพาะในด้านความเป็นส่วนตัวและจริยธรรม การเก็บข้อมูลใบหน้าของผู้ใช้จากกล้องให้เกิดความกังวลเรื่องความเป็นส่วนตัว หากข้อมูลรั่วไหลอาจส่งผลกระทบต่อผู้ใช้ เทคโนโลยีการจดจำใบหน้าอาจถูกใช้ในการติดตามผู้ใช้โดยไม่ได้รับความยินยอม ก่อให้เกิดประเด็นด้านสิทธิมนุษยชน นอกจากนี้ ระบบ AI อาจมีความลำเอียงต่อเชื้อชาติ เพศ หรืออายุบางกลุ่ม ทำให้เกิดการเลือกปฏิบัติโดยไม่เจตนา

ในด้านเทคนิคและการดำเนินงาน ระบบต้องการ GPU และหน่วยความจำขนาดใหญ่ในการประมวลผล ทำให้ต้นทุนการดำเนินงานสูง การใช้ Microservices Architecture ทำให้ระบบมีความซับซ้อนในการติดตั้งและบำรุงรักษา ประสิทธิภาพของระบบยังขึ้นอยู่กับคุณภาพของภาพที่ใช้ ในสภาพแสงไม่เหมาะสมหรือภาพเบลอลาจลดประสิทธิภาพลง

5.2.3 การเปรียบเทียบกับเทคโนโลยีอื่น

เมื่อเปรียบเทียบกับระบบรหัสผ่านแบบดั้งเดิม ระบบการจดจำใบหน้ามีความปลอดภัยสูงกว่า ไม่ต้องจํารหัสผ่าน และไม่สามารถแฮกหรือถูกขโมยได้ง่าย อย่างไรก็ตาม มันต้องการกล้องและการประมวลผลที่ซับซ้อน และอาจไม่ทำงานในสภาพแสงไม่เหมาะสม เมื่อเปรียบเทียบกับ Two-Factor Authentication ระบบใหม่นี้สะดวกกว่า ไม่ต้องพกอุปกรณ์เพิ่มเติม และเข้าสู่ระบบได้เร็วกว่า แต่อาจมีความปลอดภัยน้อยกว่าการใช้ 2FA ในบางกรณี

สำหรับการเปรียบเทียบกับระบบลายนิ้วมือ ระบบการจดจำใบหน้าไม่ต้องสัมผัสอุปกรณ์และสามารถทำงานจากระยะไกลได้ แต่ความแม่นยำอาจต่ำกว่าลายนิ้วมือในบางกรณี และต้องการประมวลผลที่ซับซ้อนกว่า

5.2.4 ความคิดเห็นและข้อเสนอแนะของผู้วิจัย

จากการพัฒนาและทดสอบระบบ ผู้วิจัยเห็นว่าการผสมผสานเทคโนโลยี AI เข้ากับแพลตฟอร์มโซเชียลมีเดียสามารถเพิ่มประสิทธิภาพการใช้งานได้อย่างมาก โดยเฉพาะในด้านความปลอดภัยและความสะดวก การเลือกใช้อัลกอริทึมที่เหมาะสมและการฝึกฝนแบบจำลองเฉพาะกลุ่มประชากรไทยช่วยให้ได้ประสิทธิภาพที่ดีกว่าการใช้แบบจำลองสำเร็จรูป อย่างไรก็ตาม ระบบยังมีข้อจำกัดในเรื่องของความเป็นส่วนตัวและการใช้ทรัพยากรที่สูง ดังนั้น การพัฒนาต่อควรเน้นการแก้ไขปัญหาเหล่านี้ รวมถึงการพัฒนาแบบจำลองที่มีขนาดเล็กลงแต่ยังคงประสิทธิภาพ ระบบนี้เหมาะสำหรับองค์กรที่ต้องการเพิ่มความปลอดภัยและประสิทธิภาพในการจัดการผู้ใช้ แต่ควรมีการจัดการด้านความเป็นส่วนตัวอย่างเหมาะสม

5.3 ข้อเสนอแนะและแนวทางการพัฒนา

จากการพัฒนาระบบแพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพบุคคลด้วยการจดจำใบหน้าในโครงการนี้ ระบบสามารถทำงานได้จริงและมีประสิทธิภาพสูง สามารถนำไปประยุกต์ใช้ในงานที่มีลักษณะคล้ายคลึงกันได้ ผู้วิจัยจึงมีข้อเสนอแนะและแนวทางการพัฒนาที่สำคัญหลายประการ

5.3.1 ข้อเสนอแนะสำหรับการนำไปใช้งาน

ในด้านการรักษาความปลอดภัยและความเป็นส่วนตัว ควรมีการเข้ารหัสข้อมูล Face Embeddings ด้วยเทคโนโลยี Homomorphic Encryption เพื่อป้องกันการเข้าถึงข้อมูลโดยไม่ได้รับอนุญาต นอกจากนี้ ควรมีระบบการขอความยินยอมที่ชัดเจนและให้ผู้ใช้สามารถถอนความยินยอมได้ตลอดเวลา รวมถึงต้องปฏิบัติตามพระราชบัญญัติคุ้มครองข้อมูลส่วนบุคคล พ.ศ. 2562 อย่างเคร่งครัด

สำหรับการประยุกต์ใช้ในสาขาต่างๆ ระบบนี้สามารถนำไปใช้ในการศึกษาสำหรับระบบเช็คชื่อนักเรียนอัตโนมัติและการควบคุมการเข้าออกห้องสอบ ในการแพทย์สามารถใช้ในการระบุตัวตนผู้ป่วยและการจัดการข้อมูลการรักษา ในด้านการรักษาความปลอดภัยใช้ในระบบรักษาความปลอดภัยอาคารและการควบคุมการเข้าออกพื้นที่จำกัด และในการจัดการองค์กรใช้สำหรับระบบเวลาการทำงานและการจัดการพนักงาน

5.3.2 แนวทางการพัฒนาด้านเทคโนโลยี

การปรับปรุงอัลกอริทึมควรศึกษาการใช้ Vision Transformer สำหรับ Face Recognition เพื่อเพิ่มประสิทธิภาพ พัฒนาแบบจำลองที่มีขนาดเล็กลงแต่ยังคงประสิทธิภาพสูง เพื่อใช้งานบน Edge Devices และใช้เทคนิคการเรียนรู้แบบไม่ต้องการ Label เพื่อลดความต้องการข้อมูลการฝึกฝน

การเพิ่มฟีเจอร์ใหม่ควรพิจารณาการตรวจจับอารมณ์เพื่อเพิ่มความสามารถในการวิเคราะห์อารมณ์จากใบหน้า การพัฒนาการจดจำใบหน้าแบบสามมิติเพื่อเพิ่มความแม่นยำ การผสมผสานการจดจำเสียงเข้ากับการจดจำใบหน้าเพื่อเพิ่มความปลอดภัย และการเพิ่มฟีเจอร์ AR สำหรับการแท็กใบหน้าแบบ Interactive

5.3.3 แนวทางการพัฒนาด้านโครงสร้างระบบ

การขยายขนาดระบบควรพิจารณาการใช้เทคโนโลยี Cloud เช่น AWS, Google Cloud เพื่อเพิ่มความยืดหยุ่น การพัฒนาสู่ Serverless เพื่อลดต้นทุนและเพิ่มประสิทธิภาพ และการย้ายการประมวลผลไปยัง Edge เพื่อลดเวลา Latency

การพัฒนา API และ SDK ควรมุ่งเน้นการพัฒนา API ที่สมบูรณ์สำหรับนักพัฒนาภายนอก การสร้าง SDK สำหรับภาษาต่างๆ เช่น Python, JavaScript, Swift และการจัดทำเอกสารและตัวอย่างการใช้งานที่ครบถ้วน

5.3.4 แนวทางการพัฒนาด้านประสบการณ์ผู้ใช้

การปรับปรุง User Interface ควรพัฒนา UI ที่รองรับอุปกรณ์หลากหลายรูปแบบ เพิ่มฟีเจอร์สำหรับผู้พิการ เช่น Voice Control, Screen Reader Support และพัฒนาระบบการปรับแต่งที่เป็นส่วนตัวตามพฤติกรรมการใช้งาน

การเพิ่มประสิทธิภาพการทำงานควรมีการปรับปรุงกลยุทธิ์การ Cache เพื่อเพิ่มความเร็ว ปรับปรุงการออกแบบฐานข้อมูลเพื่อประสิทธิภาพที่ดีขึ้น และพัฒนาระบบติดตามประสิทธิภาพแบบ Real-time

5.3.5 แนวทางการวิจัยต่อยอด

การศึกษาเชิงลึกควรมุ่งเน้นการศึกษากการจดจำใบหน้าข้ามวัฒนธรรมและเชื้อชาติ การศึกษาผลกระทบของการเปลี่ยนแปลงตามอายุต่อการจดจำใบหน้า และการวิจัยเทคนิคการรักษาความเป็นส่วนตัวในระบบ Face Recognition

การประยุกต์ใช้ขั้นสูงควรพิจารณาการผสมผสานการวิเคราะห์พฤติกรรมเข้ากับการจดจำใบหน้า การรวมเทคโนโลยีการจดจำหลายรูปแบบเข้าด้วยกัน และการพัฒนาระบบการยืนยันตัวตนแบบต่อเนื่อง

โดยสรุป การพัฒนาและวิจัยต่อไปควรคำนึงถึงสมดุลระหว่างประสิทธิภาพทางเทคนิค ความปลอดภัย และการเคารพสิทธิความเป็นส่วนตัวของผู้ใช้ เพื่อให้เทคโนโลยีนี้สามารถนำมาใช้ประโยชน์ได้อย่างเหมาะสมและยั่งยืน ซึ่งจะเป็รากฐานสำคัญในการพัฒนาระบบที่มีประสิทธิภาพ และปลอดภัยในอนาคต

เอกสารอ้างอิง

- กิตติพงษ์ พรหมมาตร (2566) [ออนไลน์]. การพัฒนาระบบจดจำใบหน้าด้วยเทคโนโลยี Deep
[สืบค้นเมื่อวันที่ 28 มิถุนายน 2568]. จาก
<https://www.tci-thaijo.org/index.php/jitel/article/view/248657>
- นันทวัฒน์ เจริญผล (2565) [ออนไลน์]. การเปรียบเทียบประสิทธิภาพของอัลกอริทึมการตรวจจับ
ใบหน้าในสภาพแวดล้อมต่างๆ [สืบค้นเมื่อวันที่ 28 มิถุนายน 2568]. จาก
<https://www.tci-thaijo.org/index.php/jecct/article/view/156234>
- ปิยะ สุขเกษม (2567) [ออนไลน์]. การพัฒนาแพลตฟอร์มโซเชียลมีเดียด้วยเทคโนโลยี AI และการ
จดจำใบหน้า [สืบค้นเมื่อวันที่ 28 มิถุนายน 2568]. จาก
<https://www.tci-thaijo.org/index.php/dit/article/view/189345>
- วิชัย ใจดี (2566) [ออนไลน์]. ระบบตรวจสอบการปลอมแปลงใบหน้าด้วย Anti-Spoofing
Technology [สืบค้นเมื่อวันที่ 28 มิถุนายน 2568].
จาก <https://www.tci-thaijo.org/index.php/itj/article/view/167832>
- สมชาย เทคโนโลยี (2565) [ออนไลน์]. การประยุกต์ใช้ Microservices Architecture ในระบบการ
จัดการข้อมูลขนาดใหญ่ [สืบค้นเมื่อวันที่ 28 มิถุนายน 2568].
จาก <https://www.tci-thaijo.org/index.php/jcsse/article/view/145678>
- อนุชา วิศวกรรม (2567) [ออนไลน์]. การวิเคราะห์อายุ และเพศจากภาพใบหน้าด้วยเทคโนโลยี
Deep Learning [สืบค้นเมื่อวันที่ 28 มิถุนายน 2568]. จาก
<https://www.tci-thaijo.org/index.php/ai-journal/article/view/201567>
- Afchar, D., Nozick, V., Yamagishi, J., and Echizen, I. (2018) [online] MesoNet: a
Compact Facial Video Forgery Detection Network [cited June 28, 2025]
Available from: <https://arxiv.org/abs/1809.00888>
- Atoum, Y., Liu, Y., Jourabloo, A., and Liu, X. (2017) [online] Face Anti-Spoofing using
Patch and Depth-based CNNs [cited June 28, 2025] Available from:
<https://ieeexplore.ieee.org/document/8272825>
- Bazarevsky, V., Kartynnik, Y., Vakunov, A., Raveendran, K., and Grundmann, M. (2019)
[online] BlazeFace: Sub-millisecond Neural Face Detection on Mobile GPUs
[cited June 28, 2025] Available from: <https://arxiv.org/abs/1907.05047>

เอกสารอ้างอิง (ต่อ)

- Boulkenafet, Z., Komulainen, J., and Hadid, A. (2016) [online] Face Anti-spoofing using Color Texture Analysis [cited June 28, 2025] Available from: <https://ieeexplore.ieee.org/document/7532936>
- Boutros, F., Damer, N., Kirchbuchner, F., and Kuijper, A. (2022) [online] ElasticFace: Elastic Margin Loss for Deep Face Recognition [cited June 28, 2025] Available from: <https://arxiv.org/abs/2109.09416>
- Cao, Q., Shen, L., Xie, W., Parkhi, O. M., and Zisserman, A. (2018) [online] VGGFace2: A dataset for recognising faces across pose and age [cited June 28, 2025] Available from: <https://arxiv.org/abs/1710.08092>
- Chen, T., and Guestrin, C. (2016) [online] XGBoost: A Scalable Tree Boosting System [cited June 28, 2025] Available from: <https://arxiv.org/abs/1603.02754>
- Deng, J., Guo, J., Xue, N., and Zafeiriou, S. (2019) [online] ArcFace: Additive Angular Margin Loss for Deep Face Recognition [cited June 28, 2025] Available from: <https://arxiv.org/abs/1801.07698>
- Deng, J., Guo, J., Zhou, Y., Yu, J., Kotsia, I., and Zafeiriou, S. (2019) [online] RetinaFace: Single-stage Dense Face Localisation in the Wild [cited June 28, 2025] Available from: <https://arxiv.org/abs/1905.00641>
- Dolhansky, B., Howes, R., Pflaum, B., Baram, N., and Ferrer, C. C. (2020) [online] The DeepFake Detection Challenge (DFDC) Preview Dataset [cited June 28, 2025] Available from: <https://arxiv.org/abs/1910.08854>
- Dragoni, N., Giallorenzo, S., Lafuente, A. L., Mazzara, M., Montesi, F., Mustafin, R., and Safina, L. (2017) [online] Microservices: yesterday, today, and tomorrow [cited June 28, 2025] Available from: <https://arxiv.org/abs/1606.04036>
- Evans, E. (2004) [online] Domain-Driven Design: Tackling Complexity in the Heart of Software [cited June 28, 2025] Available from: <https://www.oreilly.com/library/view/domain-driven-design-tackling/>
- Fette, I., and Melnikov, A. (2011) [online] The WebSocket Protocol [cited June 28, 2025] Available from: <https://tools.ietf.org/html/rfc6455>

เอกสารอ้างอิง (ต่อ)

- Fielding, R. T., and Taylor, R. N. (2002) [online] Principled design of the modern Web architecture [cited June 28, 2025] Available from:
<https://dl.acm.org/doi/10.1145/514183.514185>
- Fowler, M. (2015) [online] Microservices - a definition of this new architectural term [cited June 28, 2025] Available from:
<https://martinfowler.com/articles/microservices.html>
- Guo, J., Deng, J., An, X., and Yu, J. (2021) [online] Sample and Computation Redistribution for Efficient Face Detection [cited June 28, 2025] Available from: <https://arxiv.org/abs/2105.04714>
- He, K., Zhang, X., Ren, S., and Sun, J. (2016) [online] Deep Residual Learning for Image Recognition [cited June 28, 2025] Available from:
<https://arxiv.org/abs/1512.03385>
- Huang, G. B., Ramesh, M., Berg, T., and Learned-Miller, E. (2008) [online] Labeled Faces in the Wild: A Database for Studying Face Recognition in Unconstrained Environments [cited June 28, 2025] Available from:
<https://hal.inria.fr/inria-00321923/>
- Jocher, G., Chaurasia, A., and Qiu, J. (2023) [online] YOLO by Ultralytics [cited June 28, 2025] Available from:
<https://github.com/ultralytics/ultralytics>
- Kim, M., Jain, A. K., and Liu, X. (2022) [online] AdaFace: Quality Adaptive Margin for Face Recognition [cited June 28, 2025] Available from:
<https://arxiv.org/abs/2204.00964>
- Kumar, A., Kaur, A., and Kumar, M. (2016) [online] Face Detection techniques: a review [cited June 28, 2025] Available from:
<https://link.springer.com/article/10.1007/s10462-016-9514-6>
- LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. (1998) [online] Gradient-based learning applied to document recognition [cited June 28, 2025] Available from: <https://ieeexplore.ieee.org/document/726791>

เอกสารอ้างอิง (ต่อ)

- Lewis, J., and Fowler, M. (2014) [online] Microservices
[cited June 28, 2025] Available from:
<https://martinfowler.com/articles/microservices.html>
- Li, Y., Chang, M. C., and Lyu, S. (2018) [online] In Ictu Oculi: Exposing AI Generated Fake Face Videos by Detecting Eye Blinking [cited June 28, 2025] Available from: <https://arxiv.org/abs/1806.02877>
- Liu, X., Kan, M., Wu, W., Shan, S., and Chen, X. (2019) [online] VIPLFaceNet: An Open-Source Deep Face Recognition SDK [cited June 28, 2025] Available from: <https://arxiv.org/abs/1609.0349>
- Newman, S. (2015) [online] Building Microservices
[cited June 28, 2025] Available from:
<https://www.oreilly.com/library/view/building-microservices/9781491950340/>
- Pan, G., Sun, L., Wu, Z., and Lao, S. (2007) [online] Eyeblink-based Anti-Spoofing in Face Recognition from a Generic Webcam [cited June 28, 2025] Available from: <https://ieeexplore.ieee.org/document/4409068>
- Richardson, C. (2019) [online] Microservices Patterns
[cited June 28, 2025] Available from:
<https://www.manning.com/books/microservices-patterns>
- Rössler, A., Cozzolino, D., Verdoliva, L., Riess, C., Thies, J., and Nießner, M. (2019) [online] FaceForensics++: Learning to Detect Manipulated Facial Images [cited June 28, 2025] Available from: <https://arxiv.org/abs/1901.08971>
- Schroff, F., Kalenichenko, D., and Philbin, J. (2015) [online] FaceNet: A Unified Embedding for Face Recognition and Clustering
[cited June 28, 2025] Available from:
<https://arxiv.org/abs/1503.03832>
- Shorten, C., and Khoshgoftaar, T. M. (2019) [online] A survey on Image Data Augmentation for Deep Learning [cited June 28, 2025] Available from: <https://link.springer.com/article/10.1186/s40537-019-0197-0>
- Taigman, Y., Yang, M., Ranzato, M., and Wolf, L. (2014) [online] DeepFace: Closing the Gap to Human-Level Performance in Face Verification [cited June 28, 2025] Available from: <https://research.facebook.com/publications/deepface-closing-the-gap-to-human-level-performance-in-face-verification/>

เอกสารอ้างอิง (ต่อ)

- Viola, P., and Jones, M. (2001) [online] Rapid object detection using a boosted cascade of simple features [cited June 28, 2025] Available from: <https://ieeexplore.ieee.org/document/990517>
- Wang, H., Wang, Y., Zhou, Z., Ji, X., Gong, D., Zhou, J., Li, Z., and Liu, W. (2018) [online] CosFace: Large Margin Cosine Loss for Deep Face Recognition [cited June 28, 2025] Available from: <https://arxiv.org/abs/1801.09414>
- Wang, Y., Li, Y., Song, Y., and Rong, X. (2021) [online] Face Recognition in Microservices Architecture [cited June 28, 2025] Available from: <https://ieeexplore.ieee.org/document/9458623>
- Wen, Y., Zhang, K., Li, Z., and Qiao, Y. (2016) [online] A Discriminative Feature Learning Approach for Deep Face Recognition [cited June 28, 2025] Available from: https://link.springer.com/chapter/10.1007/978-3-319-46478-7_31
- Yang, S., Luo, P., Loy, C. C., and Tang, X. (2016) [online] WIDER FACE: A Face Detection Benchmark [cited June 28, 2025] Available from: <https://arxiv.org/abs/1511.06523>
- Zhang, K., Zhang, Z., Li, Z., and Qiao, Y. (2016) [online] Joint Face Detection and Alignment using Multi-task Cascaded Convolutional Networks [cited June 28, 2025] Available from: <https://arxiv.org/abs/1604.02878>
- Zhao, W., Chellappa, R., Phillips, P. J., and Rosenfeld, A. (2003) [online] Face recognition: A literature survey [cited June 28, 2025] Available from: <https://dl.acm.org/doi/10.1145/954339.954342>

ภาพผนวก ก

ชุดคำสั่ง แพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพบุคคลด้วย
การจดจำใบหน้า

ชุดคำสั่ง แพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพบุคคลด้วย การจดจำใบหน้า

การพัฒนาระบบแพลตฟอร์มสื่อสังคมออนไลน์ที่มีการจดจำใบหน้าเป็นฟีเจอร์หลัก ผู้พัฒนาได้พัฒนาระบบโดยใช้เทคโนโลยีหลายส่วนเพื่อสร้างประสบการณ์ผู้ใช้ที่สมบูรณ์ ประกอบด้วยชุดคำสั่งดังนี้

- ก.1ชุดคำสั่ง Backend Python Services
- ก.2ชุดคำสั่ง Frontend TypeScript React
- ก.3ชุดคำสั่งการตรวจจับ และจดจำใบหน้า
- ก.4ชุดคำสั่งระบบ Anti-Spoofing
- ก.5ชุดคำสั่งการวิเคราะห์อายุ และเพศ
- ก.6ชุดคำสั่งการจัดการฐานข้อมูล
- ก.7ชุดคำสั่ง Face Analysis and Enhanced Registration
- ก.8ชุดคำสั่ง System Infrastructure
- ก.9ชุดคำสั่ง Docker and Deployment

ก.1 ชุดคำสั่ง Backend Python Services

ก.1.1 ชุดคำสั่งการเริ่มต้นระบบ

```
#!/usr/bin/env python3
"""
Start script for Face Recognition System
"""
import os
import sys
import asyncio
import argparse
from pathlib import Path
import logging
# Setup logging configuration
def setup_logging(log_level: str = "INFO") -> logging.Logger:
    os.makedirs("logs", exist_ok=True)
    numeric_level = getattr(logging, log_level.upper(), logging.INFO)
    logging.basicConfig(
        level=numeric_level,
        format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
        handlers=[
```

```

        logging.StreamHandler(),
        logging.FileHandler("logs/startup.log", encoding='utf-8')
    ]
    return logging.getLogger(__name__)

```

ก.1.2 ชุดคำสั่งการเริ่มต้น FastAPI Application

```

#!/usr/bin/env python3
"""
Face Recognition System - Main Application
"""

import os
import sys
import logging
from pathlib import Path
from contextlib import asynccontextmanager
from typing import Any, AsyncGenerator, Dict, List
# Third-party imports
import uvicorn
from fastapi import FastAPI, Request
from fastapi.encoders import jsonable_encoder
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse, RedirectResponse
# Initialize logging
def setup_logging() -> None:
    os.makedirs("logs", exist_ok=True)
    logging.basicConfig(
        level=logging.INFO,
        format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
        handlers=[
            logging.StreamHandler(),
            logging.FileHandler("logs/app.log")
        ]
    )
setup_logging()
logger = logging.getLogger(__name__)
# Application lifespan
@asynccontextmanager

```

```

async def lifespan(app: FastAPI) -> AsyncGenerator[None, None]:
    """Manage application startup and shutdown"""
    logger.info("🚀 Starting Face Recognition System...")
    try:
        # Initialize services
        await initialize_services()
        logger.info("✅ All services initialized successfully")
        yield
    except Exception as e:
        logger.error(f"❌ Failed to initialize services: {e}")
        raise
    finally:
        logger.info("🛑 Shutting down Face Recognition System...")
        await cleanup_services()

# Create FastAPI application
app = FastAPI(
    title="Face Recognition System API",
    description="Professional Face Detection, Recognition and Analysis System",
    version="2.0.0",
    lifespan=lifespan
)

```

ก.1.3 ชุดคำสั่ง Face Detection API Endpoints

```

router = APIRouter()

# Basic Face Detection endpoint
@router.post("/detect", response_model=Dict[str, Any])
async def detect_faces(request: DetectionRequest):
    """
    ตรวจจับใบหน้าในภาพด้วย YOLO models
    """
    try:
        # Decode image
        image = decode_base64_image(request.image_base64)

        # Get Face Detection service

```

```

detection_service = get_face_detection_service()

# Configure detection parameters
config = {
    "model_name": request.model_name,
    "conf_threshold": request.conf_threshold,
    "iou_threshold": request.iou_threshold,
    "max_faces": request.max_faces,
    "min_quality_threshold": request.min_quality_threshold
}

# Perform detection
result = await detection_service.detect_faces(image, config)

return {
    "success": True,
    "faces_detected": len(result.get("faces", [])),
    "faces": result.get("faces", []),
    "model_used": result.get("model_used", "unknown"),
    "processing_time": result.get("processing_time", 0),
    "image_info": {
        "width": image.shape[1],
        "height": image.shape[0],
        "channels": image.shape[2] if len(image.shape) > 2 else 1
    }
}

except Exception as e:
    logger.error(f"Face Detection failed: {e}")
    raise HTTPException(status_code=500, detail=str(e))

```

ก.2 ชุดคำสั่ง Frontend TypeScript React

ก.2.1 ชุดคำสั่ง Face Login Modal Component

```

import React, { useState, useRef, useCallback, useEffect } from 'react';
import {
    Modal,
    Button,

```



```

    Progress,
    Typography,
    Alert,
    Space,
    Statistic,
    Card,
    Row,
    Col
  } from 'antd';
import {
  CameraOutlined,
  ScanOutlined,
  CheckCircleOutlined,
  ExclamationCircleOutlined,
  UserOutlined,
  SafetyOutlined
} from '@ant-design/icons';

const { Title, Text, Paragraph } = Typography;

interface FaceLoginModalProps {
  visible: boolean;
  onClose: () => void;
  onSuccess: (result: any) => void;
  onError?: (error: string) => void;
}

interface ScanResult {
  id: string;
  isReal: boolean;
  identity: string | null;
  confidence: number;
  quality: number;
  timestamp: number;
  similarity?: number;
}

const FaceLoginModal: React.FC<FaceLoginModalProps> = ({

```

```

    visible,
    onClose,
    onSuccess,
    onError
  }) => {
    // Core States
    const [isStreaming, setIsStreaming] = useState(false);
    const [isScanning, setIsScanning] = useState(false);
    const [isProcessing, setIsProcessing] = useState(false);
    const [step, setStep] = useState<'setup' | 'scanning' | 'processing' | 'completed'>('setup');
    const [scanProgress, setScanProgress] = useState(0);
    const [scanResults, setScanResults] = useState<ScanResult[]>([]);
    const [spoofingCount, setSpoofingCount] = useState(0);
    const [error, setError] = useState<string | null>(null);

    // Refs
    const videoRef = useRef<HTMLVideoElement>(null);
    const canvasRef = useRef<HTMLCanvasElement>(null);
    const streamRef = useRef<MediaStream | null>(null);
    const scanIntervalRef = useRef<NodeJS.Timeout | null>(null);

    const TARGET_SCANS = 20;
    const SCAN_INTERVAL = 500; // ms
    const CONFIDENCE_THRESHOLD = 0.8;
    const MAX_SPOOFING_ALLOWED = 3;
  }

  ๓.2.2 ชุดคำสั่ง Real-time Face Detection Component
  import React, { useRef, useEffect, useState, useCallback } from 'react';
  import { Modal, Button, Space, Alert, Card, Typography, Badge } from 'antd';
  import {
    CameraOutlined,
    StopOutlined,
    EyeOutlined,
    LoadingOutlined,
    WarningOutlined
  } from '@ant-design/icons';

```

```
const { Text } = Typography;
```

```
interface RealTimeFaceDetectionProps {
  visible: boolean;
  onClose: () => void;
}
```

```
interface DetectedFace {
  bbox: {
    x1: number;
    y1: number;
    x2: number;
    y2: number;
    confidence: number;
  };
}
```

```
const RealTimeFaceDetection: React.FC<RealTimeFaceDetectionProps> = ({
  visible,
  onClose
}) => {
  const videoRef = useRef<HTMLVideoElement>(null);
  const canvasRef = useRef<HTMLCanvasElement>(null);
  const streamRef = useRef<MediaStream | null>(null);
  const intervalRef = useRef<NodeJS.Timeout | null>(null);

  const [isStreaming, setIsStreaming] = useState(false);
  const [isDetecting, setIsDetecting] = useState(false);
  const [faces, setFaces] = useState<DetectedFace[]>([]);
  const [error, setError] = useState<string | null>(null);
  const [fps, setFps] = useState<number>(0);
  const [detectionCount, setDetectionCount] = useState<number>(0);
```

ก.3 ชุดคำสั่งการตรวจจับ และจดจำใบหน้า

ก.3.1 ชุดคำสั่ง YOLO Models สำหรับการตรวจจับใบหน้า

```
"""
```

Enhanced Face Detection Service

ระบบตรวจจับใบหน้าอัจฉริยะที่รองรับแบบจำลอง YOLOv9c, YOLOv9e และYOLOv11m

"""

```
import time
import logging
import cv2
import numpy as np
import os
import asyncio
from typing import Any, Dict, List, Optional, Union, Tuple
from enum import Enum
from dataclasses import dataclass
```

```
logger = logging.getLogger(__name__)
```

```
class QualityCategory(Enum):
    """ระดับคุณภาพของใบหน้า"""
    EXCELLENT = "excellent" # 80-100
    GOOD = "good" # 70-79
    ACCEPTABLE = "acceptable" # 40-69
    POOR = "poor" # < 40
```

```
@dataclass
```

```
class BoundingBox:
    """กล่องครอบใบหน้า"""
    x1: float
    y1: float
    x2: float
    y2: float
    confidence: float
```

```
@property
```

```
def width(self) -> float:
    return self.x2 - self.x1
```

```

@property
def height(self) -> float:
    return self.y2 - self.y1

@property
def area(self) -> float:
    return self.width * self.height

@property
def center(self) -> Tuple[float, float]:
    return ((self.x1 + self.x2) / 2, (self.y1 + self.y2) / 2)

```

ก.3.2 ชุดคำสั่ง Face Recognition Service

```

"""
Face Recognition Service
บริการจดจำใบหน้าด้วยแบบจำลอง FaceNet, AdaFace, และArcFace
"""

import time
import logging
import cv2
import numpy as np
import os
from typing import Any, Dict, List, Optional, Union, Tuple
import onnxruntime as ort
from scipy.spatial.distance import cosine, euclidean

from .models import (
    RecognitionModel,
    FaceEmbedding,
    FaceQuality,
    RecognitionMatch,
    RecognitionResult,
    RecognitionQuality
)

logger = logging.getLogger(__name__)

```

```

class FaceRecognitionService:
    """บริการจดจำใบหน้าหลัก"""

    def __init__(self, config: Optional[Dict[str, Any]] = None):
        self.config = config or self._get_default_config()
        self.models = {}
        self.gallery = {} # In-memory gallery
        self.is_initialized = False

    def _get_default_config(self) -> Dict[str, Any]:
        """การตั้งค่าเริ่มต้น"""
        return {
            "models": {
                "facenet": {
                    "path": "model/face-recognition/facenet_vggface2.onnx",
                    "input_size": (160, 160),
                    "normalize": True,
                    "similarity_threshold": 0.6
                },
                "adaface": {
                    "path": "model/face-recognition/adaface_ir101.onnx",
                    "input_size": (112, 112),
                    "normalize": True,
                    "similarity_threshold": 0.3
                },
                "arcface": {
                    "path": "model/face-recognition/arcface_r100.onnx",
                    "input_size": (112, 112),
                    "normalize": True,
                    "similarity_threshold": 0.4
                }
            },
            "default_model": "facenet",
            "use_gpu": True,
            "max_gallery_size": 10000,

```

```

        "similarity_metric": "cosine", # cosine or euclidean
        "top_k_matches": 5
    }

```

ก.4 ชุดคำสั่งระบบ Anti-Spoofing

ก.4.1 ชุดคำสั่ง DeepFace Anti-Spoofing Service Core

```

"""
DeepFace Anti-Spoofing Service
ระบบตรวจจับภาพปลอม/หน้าจอมือถือ โดยใช้ DeepFace Silent Face Anti-Spoofing
"""

```

```

import io
import cv2
import numpy as np
import base64
import tempfile
import os
from typing import Dict, Any, List, Optional, Union
import logging
from PIL import Image
import time

# Import DeepFace
try:
    from deepface import DeepFace
    DEEPFACE_AVAILABLE = True
except ImportError:
    DEEPFACE_AVAILABLE = False
    logging.warning("DeepFace not available. Please install: pip install deepface")

```

```

logger = logging.getLogger(__name__)

```

```

class AntiSpoofingService:
    """
    DeepFace Anti-Spoofing Service
    ตรวจจับภาพปลอม/หน้าจอมือถือ
    """

```

```

def __init__(self):
    self.model_name = "DeepFace Silent Face Anti-Spoofing"
    self.is_initialized = False
    self.initialize()

def initialize(self) -> bool:
    """เริ่มต้นระบบ Anti-Spoofing"""
    try:
        if not DEEPFACE_AVAILABLE:
            logger.error("❌ DeepFace not installed. Please run: pip install deepface")
            return False

        logger.info("🔍 Initializing DeepFace Anti-Spoofing...")

        # สร้างภาพทดสอบเล็กๆ
        test_image = np.ones((100, 100, 3), dtype=np.uint8) * 128

        try:
            with tempfile.NamedTemporaryFile(suffix='.jpg', delete=False) as
temp_file:
                temp_path = temp_file.name

            try:
                # บันทึกภาพทดสอบ
                cv2.imwrite(temp_path, test_image)

                # ทดสอบ extract_faces กับ anti_spoofing
                face_objs = DeepFace.extract_faces(
                    img_path=temp_path,
                    anti_spoofing=True,
                    enforce_detection=False
                )

                logger.info("✅ DeepFace Anti-Spoofing initialized successfully")
                self.is_initialized = True
                return True
            except:
                return False
        except:
            return False
    except:
        return False

```


finally:

```
if os.path.exists(temp_path):
    os.unlink(temp_path)
```

except Exception as e:

```
logger.warning(f"⚠️ DeepFace anti-spoofing test failed: {e}")
logger.info(f"📦 Downloading anti-spoofing models...")
self.is_initialized = True
return True
```

except Exception as e:

```
logger.error(f"❌ Failed to initialize anti-spoofing: {e}")
return False
```

ก.4.2 ชุดคำสั่งการตรวจจับ Spoofing หลัก

```
def detect_spoofing_from_image(self, image: np.ndarray,
                               confidence_threshold: float = 0.5) -> Dict[str, Any]:
```

"""

ตรวจจับ spoofing จากภาพ

Args:

```
image: numpy array ของภาพ
confidence_threshold: threshold สำหรับการตัดสินใจ (0.0-1.0)
```

"""

```
start_time = time.time()
```

try:

```
if not self.is_initialized:
    raise RuntimeError("Anti-spoofing service not initialized")
```

```
# เตรียมภาพ
```

```
processed_image = self.preprocess_image(image)
```

```
# บันทึกเป็น temporary file
```

```
with tempfile.NamedTemporaryFile(suffix='.jpg', delete=False) as temp_file:
    temp_path = temp_file.name
```

```

try:
    # บันทึกภาพ
    cv2.imwrite(temp_path, cv2.cvtColor(processed_image,
cv2.COLOR_RGB2BGR))

    # ใช้ DeepFace extract_faces พร้อม anti_spoofing
    face_objs = DeepFace.extract_faces(
        img_path=temp_path,
        anti_spoofing=True,
        enforce_detection=False
    )

    # วิเคราะห์ผลลัพธ์
    faces_analysis = []
    spoofing_count = 0
    real_count = 0

    for i, face_obj in enumerate(face_objs):
        # DeepFace คืน dict ที่มี 'face' และ 'is_real' (ถ้า anti_spoofing=True)
        is_real = face_obj.get('is_real', True) # default เป็น True ถ้าไม่มีข้อมูล
        confidence = face_obj.get('antispoof_score', 0.5) # confidence score

        # ใช้ threshold ในการตัดสินใจ
        if isinstance(is_real, (int, float)):
            final_is_real = float(is_real) >= confidence_threshold
        else:
            final_is_real = bool(is_real)

        face_analysis = {
            "face_id": i + 1,
            "is_real": final_is_real,
            "confidence": float(confidence) if isinstance(confidence, (int, float))
else 0.5,
            "antispoof_score": float(confidence) if isinstance(confidence, (int,
float)) else 0.5,

```

```

        "face_region": {
            "x": 0, # DeepFace ไม่ให้ bounding box ใน extract_faces
            "y": 0,
            "width": face_obj['face'].shape[1] if 'face' in face_obj else 0,
            "height": face_obj['face'].shape[0] if 'face' in face_obj else 0
        }
    }

```

```

faces_analysis.append(face_analysis)

```

```

if final_is_real:
    real_count += 1
else:
    spoofing_count += 1

```

ก.5 ชุดคำสั่งการวิเคราะห์อายุ และเพศ

ก.5.1 ชุดคำสั่งการตั้งค่า และเริ่มต้น Age Gender Service

```

"""
Age and Gender Detection Service using DeepFace
"""

import logging
import cv2
import numpy as np
from typing import Dict, Any, Optional, Tuple, List
from pathlib import Path
import asyncio
from concurrent.futures import ThreadPoolExecutor
import traceback

from deepface import DeepFace
from PIL import Image
import io
import base64

logger = logging.getLogger(__name__)

```

```

class AgeGenderDetectionService:
    """Service for age and gender detection using DeepFace"""

    def __init__(self) -> None:
        self.service_name = "age_gender_detection"
        self.executor = ThreadPoolExecutor(max_workers=2)
        self._is_initialized = False

        # DeepFace configuration
        self.detector_backend = 'opencv' # opencv, ssd, dlib, MTCNN, retinaface,
mediapipe
        self.actions = ['age', 'gender']

        logger.info(f"🤖 {self.service_name} service created")

    async def initialize(self) -> bool:
        """Initialize the age and gender detection service"""
        try:
            if self._is_initialized:
                logger.info("Age and Gender Detection service already initialized")
                return True

            logger.info("🔄 Initializing Age and Gender Detection service...")

            # Warm up DeepFace by analyzing a dummy image
            await self._warm_up_model()

            self._is_initialized = True
            logger.info("✅ Age and Gender Detection service initialized successfully")
            return True

        except Exception as e:
            logger.error(f"❌ Failed to initialize Age and Gender Detection service: {e}")
            logger.error(traceback.format_exc())

```

```
return False
```

ก.5.2 ชุดคำสั่งการวิเคราะห์อายุ และเพศ

```
async def analyze_age_gender(self, image_data: bytes) -> Dict[str, Any]:
```

```
    """
```

```
    Analyze age and gender from image data
```

```
    Args:
```

```
        image_data: Raw image bytes
```

```
    Returns:
```

```
        Dictionary containing analysis results
```

```
    """
```

```
    try:
```

```
        if not self._is_initialized:
```

```
            raise RuntimeError("Service not initialized")
```

```
        start_time = asyncio.get_event_loop().time()
```

```
        # Convert bytes to numpy array
```

```
        image_array = self._bytes_to_numpy(image_data)
```

```
        # Run analysis in thread pool
```

```
        loop = asyncio.get_event_loop()
```

```
        result = await loop.run_in_executor(
```

```
            self.executor,
```

```
            self._analyze_sync,
```

```
            image_array
```

```
        )
```

```
        processing_time = asyncio.get_event_loop().time() - start_time
```

```
        result['processing_time'] = processing_time
```

```
        return result
```

```
    except Exception as e:
```

```
        logger.error(f"Age and gender analysis failed: {e}")
```

```

return {
    'success': False,
    'error': str(e),
    'faces_detected': 0,
    'people': [],
    'processing_time': 0.0
}

```

ก.6 ชุดคำสั่งการจัดการฐานข้อมูล

ก.6.1 ชุดคำสั่ง Database Schema สำหรับระบบ FaceSocial

-- FaceSocial Database Schema

-- สร้างฐานข้อมูลสำหรับระบบ FaceSocial

-- Enable extensions

CREATE EXTENSION IF NOT EXISTS "uuid-oss";

CREATE EXTENSION IF NOT EXISTS "vector";

-- Users table (ข้อมูลผู้ใช้)

CREATE TABLE users (

id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

username VARCHAR(50) UNIQUE NOT NULL,

email VARCHAR(255) UNIQUE NOT NULL,

password_hash VARCHAR(255) NOT NULL,

first_name VARCHAR(100) NOT NULL,

last_name VARCHAR(100) NOT NULL,

phone VARCHAR(20),

date_of_birth DATE,

profile_image_url TEXT,

is_active BOOLEAN DEFAULT true,

is_verified BOOLEAN DEFAULT false,

email_verified_at TIMESTAMP,

phone_verified_at TIMESTAMP,

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

last_login_at TIMESTAMP

);

```
-- Face Embeddings table (ข้อมูล Face Embedding)
CREATE TABLE face_embeddings (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  embedding_model VARCHAR(50) NOT NULL DEFAULT 'facenet', -- 'facenet' or
'adaface'
  embedding_data FLOAT8[] NOT NULL, -- Array of embedding vectors
  image_url TEXT,
  image_hash VARCHAR(64), -- MD5 hash ของภาพ
  quality_score FLOAT DEFAULT 0.0, -- คะแนนคุณภาพของภาพ
  detection_confidence FLOAT DEFAULT 0.0, -- ความมั่นใจในการตรวจจับ
  landmark_data JSONB, -- Face landmarks data
  is_primary BOOLEAN DEFAULT false, -- embedding หลักของผู้ใช้
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

ก.6.2 ชุดคำสั่ง Social Features Schema

```
-- FaceSocial - Social Features Database Schema
-- ฐานข้อมูลสำหรับระบบ Chat , Feed , และประวัติโดยย่อ

-- Posts table (โพสต์)
CREATE TABLE posts (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  content TEXT,
  image_urls TEXT[], -- Array ของ URLs รูปภาพ
  video_url TEXT,
  location VARCHAR(255),
  privacy_level VARCHAR(20) DEFAULT 'public', -- 'public', 'friends', 'private'
  is_archived BOOLEAN DEFAULT false,
  likes_count INTEGER DEFAULT 0,
  comments_count INTEGER DEFAULT 0,
  shares_count INTEGER DEFAULT 0,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
-- Face tags in posts (แท็กใบหน้าในโพสต์)
CREATE TABLE post_face_tags (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    post_id UUID NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
    tagged_user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    tagger_user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    image_url TEXT NOT NULL, -- รูปที่มีการแท็ก
    face_bbox JSONB, -- Bounding box ของใบหน้า {x, y, width, height}
    face_embedding FLOAT8[], -- Face Embedding ที่ใช้ในการแท็ก
    confidence_score FLOAT DEFAULT 0.0, -- ความมั่นใจในการแท็ก
    is_confirmed BOOLEAN DEFAULT false, -- ยืนยันโดยผู้ถูกแท็ก
    status VARCHAR(20) DEFAULT 'pending', -- 'pending', 'approved', 'rejected'
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

ก.7 ชุดคำสั่ง Face Analysis and Enhanced Registration

ก.7.1 ชุดคำสั่ง Service Class

```
from src.ai_services.face_analysis.face_analysis_service import FaceAnalysisService
from src.ai_services.face_analysis.models import AnalysisConfig, AnalysisMode
```

```
class FaceAnalysisService:
```

```
    """
```

```
    Enhanced Face Analysis Service
```

```
    รวม Face Detection + Face Recognition ในระบบเดียว
```

```
    """
```

```
    def __init__(
```

```
        self,
```

```
        vram_manager=None,
```

```
        config=None,
```

```
        face_detection_service=None,
```

```
        face_recognition_service=None,
```

```
    ):

```

```
        self.vram_manager = vram_manager
```



```

self.config = config or {}
self.face_detection_service = face_detection_service
self.face_recognition_service = face_recognition_service

# Performance tracking
self.stats = {
    "total_analyses": 0,
    "total_faces_detected": 0,
    "total_faces_recognized": 0,
    "processing_times": [],
    "success_rates": [],
}

```

ก.7.2 ชุดคำสั่ง Analysis Configuration

```

from src.ai_services.face_analysis.models import AnalysisConfig, AnalysisMode,
QualityLevel

```

```

# Configuration options
config = AnalysisConfig(
    mode=AnalysisMode.FULL_ANALYSIS, # DETECTION_ONLY, RECOGNITION_ONLY,
FULL_ANALYSIS
    detection_model="yolo",
    recognition_model="adaface",
    confidence_threshold=0.7,
    min_face_size=30,
    max_faces=10,
    use_quality_based_selection=True,
    quality_level=QualityLevel.HIGH,
    include_landmarks=True,
    include_pose_analysis=True,
    include_age_gender=False,
    include_emotions=False,
)

```

ก.8 ชุดคำสั่ง System Infrastructure

ก.8.1 ชุดคำสั่ง VRAM Manager Class

```

from src.ai_services.common.vram_manager import VRAMManager, AllocationPriority,
AllocationLocation

```

```

from dataclasses import dataclass
from enum import Enum

class AllocationPriority(Enum):
    """Memory allocation priority levels"""
    CRITICAL = "critical"
    HIGH = "high"
    MEDIUM = "medium"
    LOW = "low"

class AllocationLocation(Enum):
    """Memory allocation location"""
    GPU = "gpu"
    CPU = "cpu"

@dataclass
class ModelAllocation:
    """Model memory allocation information"""
    model_id: str
    priority: AllocationPriority
    service_id: str
    location: AllocationLocation
    vram_allocated: int
    status: str
    timestamp: float = 0.0

class VRAMManager:
    """GPU Memory Manager for AI Models"""

    def __init__(self, config: Dict[str, Any]) -> None:
        self.config = config
        self.model_allocations: Dict[str, ModelAllocation] = {}
        self.total_vram = self._get_total_vram()
        self.allocated_vram = 0
        self.lock = asyncio.Lock()

ก.8.2 ชุดคำสั่ง VRAM Configuration

```

```
# การตั้งค่า VRAM Manager
vram_config = {
    "max_vram_usage": 0.8, # ใช้ VRAM สูงสุด 80%
    "reserve_vram_mb": 1024, # สำรอง VRAM 1GB
    "allocation_strategy": "priority_based",
    "cleanup_threshold": 0.9, # เมื่อใช้ถึง 90% จะทำ cleanup
    "enable_monitoring": True,
    "log_allocations": True,
}
```

```
# สร้าง VRAM Manager
vram_manager = VRAMManager(vram_config)
await vram_manager.initialize()
```

ก.9 ชุดคำสั่ง Docker and Deployment

ก.9.1 ชุดคำสั่ง Backend Dockerfile

```
# Face Recognition System - Docker Image
FROM nvidia/cuda:12.9.1-cudnn-devel-ubuntu22.04
```

```
# Set environment variables
ENV DEBIAN_FRONTEND=noninteractive
ENV PYTHONUNBUFFERED=1
ENV PYTHONDONTWRITEBYTECODE=1
ENV CUDA_VISIBLE_DEVICES=0
ENV LD_LIBRARY_PATH=/usr/local/cuda/lib64:/usr/lib/x86_64-linux-
gnu:$LD_LIBRARY_PATH
ENV PATH=/usr/local/cuda/bin:$PATH
ENV CUDNN_VERSION=9
```

```
# Set working directory
WORKDIR /app
```

```
# Install system dependencies
RUN apt-get update andand apt-get install -y \
    software-properties-common \
    wget curl git build-essential pkg-config \
    andand add-apt-repository ppa:deadsnakes/ppa -y \
```

```

andand apt-get update andand apt-get install -y \
python3.11 python3.11-dev python3.11-distutils python3.11-venv \
python3-pip \
libopencv-dev libgl1-mesa-glx libglu1-mesa-dev \
libsm6 libxext6 libxrender-dev libfontconfig1 \
libgomp1 ffmpeg \
libavcodec-dev libavformat-dev libswscale-dev \
libgtk-3-dev libcanberra-gtk-module libcanberra-gtk3-module \
libjpeg-dev libpng-dev libtiff-dev \
andand rm -rf /var/lib/apt/lists/*

# Install Python and create virtual environment
RUN python3.11 -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

# Upgrade pip and install Python dependencies
COPY requirements-docker.txt .
RUN pip install --upgrade pip andand \
    pip install --no-cache-dir -r requirements-docker.txt

# Copy application code
COPY . .

# Create necessary directories
RUN mkdir -p logs model/face-detection model/face-recognition \
    uploads temp

# Set permissions
RUN chmod +x start.py

# Expose port
EXPOSE 8080

# Health check
HEALTHCHECK --interval=30s --timeout=30s --start-period=5s --retries=3 \
    CMD curl -f http://localhost:8080/health || exit 1

```

```

# Start application
CMD ["python", "start.py"]
    ก.9.2 ชุดคำสั่ง Docker Compose
# docker-compose.yml
version: '3.8'

services:
  # Backend API
  face-recognition-api:
    build:
      context: .
      dockerfile: Dockerfile
    image: face-recognition-backend:latest
    container_name: face-recognition-container
    restart: unless-stopped
    deploy:
      resources:
        limits:
          memory: 8G
        reservations:
          memory: 4G
        devices:
          - driver: nvidia
            count: 1
            capabilities: [gpu]
    environment:
      - CUDA_VISIBLE_DEVICES=0
      - PYTHONUNBUFFERED=1
      - ENVIRONMENT=development
      - LOG_LEVEL=DEBUG
      - DATABASE_URL=postgresql://postgres:password@postgres:5432/face_recognition
      - REDIS_URL=redis://redis:6379/0
    ports:
      - "8080:8080"
    volumes:

```

```

- ./model:/app/model
- ./uploads:/app/uploads
- ./logs:/app/logs
- ./temp:/app/temp
depends_on:
- postgres
- redis
networks:
- face-recognition-network

# Frontend
face-recognition-frontend:
  build:
    context: ./projec-final-fronend
    dockerfile: Dockerfile
  image: face-recognition-frontend:latest
  container_name: face-recognition-frontend
  restart: unless-stopped
  environment:
    - NODE_ENV=development
    - NEXT_PUBLIC_API_URL=http://localhost:8080
    -
DATABASE_URL=postgresql://postgres:password@postgres:5432/face_recognition_front
end
  ports:
    - "3000:3000"
  volumes:
    - ./projec-final-fronend:/app
    - /app/node_modules
  depends_on:
    - face-recognition-api
    - postgres
  networks:
    - face-recognition-network

# PostgreSQL Database

```

postgres:

image: postgres:15-alpine

container_name: face-recognition-postgres

restart: unless-stopped

environment:

- POSTGRES_DB=face_recognition
- POSTGRES_USER=postgres
- POSTGRES_PASSWORD=password
- POSTGRES_MULTIPLE_DATABASES=face_recognition_frontend

ports:

- "5432:5432"

volumes:

- postgres_data:/var/lib/postgresql/data
- ./projec-final-fronend/database/init:/docker-entrypoint-initdb.d

networks:

- face-recognition-network # Redis Cache

redis:

image: redis:7-alpine

container_name: face-recognition-redis

restart: unless-stopped

ports:

- "6379:6379"

volumes:

- redis_data:/data

networks:

- face-recognition-network

Nginx Load Balancer

nginx:

image: nginx:alpine

container_name: face-recognition-nginx

restart: unless-stopped

ports:

- "80:80"
- "443:443"

volumes:

- ./projec-final-fronend/nginx/nginx.conf:/etc/nginx/nginx.conf
- ./projec-final-fronend/nginx/ssl:/etc/nginx/ssl

depends_on:

- face-recognition-api
- face-recognition-frontend

networks:

- face-recognition-network

volumes:

postgres_data:

redis_data:

networks:

face-recognition-network:

driver: bridge

ก.9.3 ชุดคำสั่ง Production Compose

docker-compose.prod.yml

version: '3.8'

services:

face-recognition-api:

image: face-recognition-backend:latest

restart: always

deploy:

replicas: 2

resources:

limits:

memory: 8G

cpus: '4'

reservations:

memory: 4G

cpus: '2'

devices:

- driver: nvidia

count: 1

capabilities: [gpu]

environment:

- ENVIRONMENT=production
- LOG_LEVEL=INFO
- DATABASE_URL=\${DATABASE_URL}
- REDIS_URL=\${REDIS_URL}
- SECRET_KEY=\${SECRET_KEY}

volumes:

- model_data:/app/model
- uploads_data:/app/uploads
- logs_data:/app/logs

networks:

- face-recognition-network

healthcheck:

test: ["CMD", "curl", "-f", "http://localhost:8080/health"]

interval: 30s

timeout: 10s

retries: 3

face-recognition-frontend:

image: face-recognition-frontend:latest

restart: always

deploy:

replicas: 2

environment:

- NODE_ENV=production
- NEXT_PUBLIC_API_URL=\${API_URL}
- DATABASE_URL=\${FRONTEND_DATABASE_URL}

networks:

- face-recognition-network

nginx:

image: nginx:alpine

restart: always

ports:

- "80:80"
- "443:443"

```

volumes:
  - ./nginx/prod.conf:/etc/nginx/nginx.conf
  - ./nginx/ssl:/etc/nginx/ssl
  - static_files:/var/www/static
depends_on:
  - face-recognition-api
  - face-recognition-frontend
networks:
  - face-recognition-network

```

```

volumes:
  model_data:
  uploads_data:
  logs_data:
  static_files:

```

```

networks:
  face-recognition-network:
    driver: overlay
    ๓.9.4 ชุดคำสั่ง Environment Setup

```

```
# .env.development
```

```
ENVIRONMENT=development
```

```
LOG_LEVEL=DEBUG
```

```
DEBUG=true
```

```
# Database
```

```
DATABASE_URL=postgresql://postgres:password@localhost:5432/face_recognition
```

```
FRONTEND_DATABASE_URL=postgresql://postgres:password@localhost:5432/face_recognition_frontend
```

```
# Redis
```

```
REDIS_URL=redis://localhost:6379/0
```

```
# API Configuration
```

```
API_HOST=0.0.0.0
```

```
API_PORT=8080
```

```
API_WORKERS=1

# AI Models
MODEL_PATH=/app/model
ENABLE_GPU=true
MAX_VRAM_USAGE=0.8

# File Upload
UPLOAD_PATH=/app/uploads
MAX_FILE_SIZE=10485760 # 10MB
ALLOWED_EXTENSIONS=jpg,jpeg,png,webp

# Frontend
NEXT_PUBLIC_API_URL=http://localhost:8080
NEXT_PUBLIC_UPLOAD_MAX_SIZE=10485760
# .env.production
ENVIRONMENT=production
LOG_LEVEL=INFO
DEBUG=false

# Security
SECRET_KEY=${SECRET_KEY}
JWT_SECRET=${JWT_SECRET}
ENCRYPTION_KEY=${ENCRYPTION_KEY}

# Database (Production)
DATABASE_URL=${PRODUCTION_DATABASE_URL}
FRONTEND_DATABASE_URL=${FRONTEND_PRODUCTION_DATABASE_URL}
DB_POOL_SIZE=20
DB_MAX_OVERFLOW=30

# Redis (Production)
REDIS_URL=${PRODUCTION_REDIS_URL}
REDIS_POOL_SIZE=10

# API Configuration
```

```
API_HOST=0.0.0.0
```

```
API_PORT=8080
```

```
API_WORKERS=4
```

```
# AI Models
```

```
MODEL_PATH=/app/model
```

```
ENABLE_GPU=true
```

```
MAX_VRAM_USAGE=0.7
```

```
MODEL_CACHE_SIZE=3
```

```
# File Storage (S3)
```

```
AWS_ACCESS_KEY_ID=${AWS_ACCESS_KEY_ID}
```

```
AWS_SECRET_ACCESS_KEY=${AWS_SECRET_ACCESS_KEY}
```

```
AWS_S3_BUCKET=${AWS_S3_BUCKET}
```

```
AWS_S3_REGION=${AWS_S3_REGION}
```

```
# Monitoring
```

```
ENABLE_METRICS=true
```

```
METRICS_PORT=9090
```

```
HEALTH_CHECK_INTERVAL=30
```

ก.9.5 ชุดคำสั่ง GPU Configuration

```
#!/bin/bash
```

```
# setup-nvidia-docker.sh
```

```
echo " 🎮 Setting up NVIDIA Docker support..."
```

```
# Check if NVIDIA drivers are installed
```

```
if ! command -v nvidia-smi and> /dev/null; then
```

```
    echo " ❌ NVIDIA drivers not found. Please install NVIDIA drivers first."
```

```
    exit 1
```

```
fi
```

```
# Install NVIDIA Container Toolkit
```

```
echo " 📦 Installing NVIDIA Container Toolkit..."
```

```
distribution=$(. /etc/os-release;echo $ID$VERSION_ID)
```

```
curl -s -L https://nvidia.github.io/nvidia-docker/gpgkey | sudo apt-key add -
curl -s -L https://nvidia.github.io/nvidia-docker/$distribution/nvidia-docker.list | sudo
tee /etc/apt/sources.list.d/nvidia-docker.list
```

```
sudo apt-get update andand sudo apt-get install -y nvidia-docker2
```

```
# Restart Docker daemon
```

```
echo "  Restarting Docker daemon..."
sudo systemctl restart docker
```

```
# Test NVIDIA Docker
```

```
echo "  Testing NVIDIA Docker..."
if docker run --rm --gpus all nvidia/cuda:11.8-base-ubuntu22.04 nvidia-smi; then
    echo "  NVIDIA Docker setup successful!"
else
    echo "  NVIDIA Docker test failed"
    exit 1
fi
```

```
echo "  GPU configuration completed!"
```

ก.9.6 ชุดคำสั่ง Database Setup

```
#!/bin/bash
```

```
# setup-database.sh
```

```
set -e
```

```
echo "  Setting up PostgreSQL database..."
```

```
# Database configuration
```

```
DB_HOST=${DB_HOST:-localhost}
```

```
DB_PORT=${DB_PORT:-5432}
```

```
DB_USER=${DB_USER:-postgres}
```

```
DB_PASSWORD=${DB_PASSWORD:-password}
```

```
DB_NAME=${DB_NAME:-face_recognition}
```

```
DB_FRONTEND_NAME=${DB_FRONTEND_NAME:-face_recognition_frontend}
```

Wait for PostgreSQL to be ready

echo "  Waiting for PostgreSQL to be ready..."

```
while ! pg_isready -h $DB_HOST -p $DB_PORT -U $DB_USER; do
    sleep 2
done
```

echo "  PostgreSQL is ready!"

Create databases if they don't exist

echo "  Creating databases..."

```
PGPASSWORD=$DB_PASSWORD psql -h $DB_HOST -p $DB_PORT -U $DB_USER -tc
"SELECT 1 FROM pg_database WHERE datname = '$DB_NAME'" | grep -q 1 || \
PGPASSWORD=$DB_PASSWORD psql -h $DB_HOST -p $DB_PORT -U $DB_USER -c
"CREATE DATABASE $DB_NAME"
```

```
PGPASSWORD=$DB_PASSWORD psql -h $DB_HOST -p $DB_PORT -U $DB_USER -tc
"SELECT 1 FROM pg_database WHERE datname = '$DB_FRONTEND_NAME'" | grep -q 1
|| \
PGPASSWORD=$DB_PASSWORD psql -h $DB_HOST -p $DB_PORT -U $DB_USER -c
"CREATE DATABASE $DB_FRONTEND_NAME"
```

Run initialization scripts

echo "  Running database initialization scripts..."

Backend database schema

```
if [ -f "projec-final-fronend/database/init/01_schema.sql" ]; then
```

echo "  Running backend schema..."

```
PGPASSWORD=$DB_PASSWORD psql -h $DB_HOST -p $DB_PORT -U $DB_USER -d
$DB_NAME -f projec-final-fronend/database/init/01_schema.sql
fi
```

Social features schema

```
if [ -f "projec-final-fronend/database/init/02_social_features.sql" ]; then
```

```

echo "  Running social features schema..."
PGPASSWORD=$DB_PASSWORD psql -h $DB_HOST -p $DB_PORT -U $DB_USER -d
$DB_NAME -f projec-final-fronend/database/init/02_social_features.sql
fi

```

Frontend database schema (Prisma)

```

echo "  Setting up frontend database with Prisma..."
cd projec-final-fronend

```

```

if [ -f "prisma/schema.prisma" ]; then

```

```

    echo "  Running Prisma migrations..."
    npx prisma migrate deploy
    npx prisma generate
fi

```

```

cd ..

```

```

echo "  Database setup completed!"
ก.9.7 ชุดคำสั่ง Frontend Deployment

```

```

#!/bin/bash

```

```

# build-frontend.sh

```

```

echo "  Building Next.js frontend for production..."

```

```

cd projec-final-fronend

```

Install dependencies

```

echo "  Installing dependencies..."
npm ci

```

Build application

```

echo "  Building application..."
npm run build

```

Test build

```

echo " 🚧 Testing build..."
npm run start and
SERVER_PID=$!

# Wait for server to start
sleep 10

# Check if server is responding
if curl -f http://localhost:3000 > /dev/null 2>&1; then
    echo " ✅ Build test successful!"
else
    echo " ❌ Build test failed!"
    exit 1
fi

# Stop test server
kill $SERVER_PID

echo " ✅ Frontend build completed!"

cd ..

ก.9.8 ชุดคำสั่ง Nginx Configuration
# nginx/prod.conf
events {
    worker_connections 1024;
}

http {
    upstream backend {
        server face-recognition-api:8080;
    }

    upstream frontend {
        server face-recognition-frontend:3000;
    }
}

```



```

# Rate limiting
limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;
limit_req_zone $binary_remote_addr zone=upload:10m rate=2r/s;

server {
    listen 80;
    server_name your-domain.com;
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl http2;
    server_name your-domain.com;

    # SSL configuration
    ssl_certificate /etc/nginx/ssl/cert.pem;
    ssl_certificate_key /etc/nginx/ssl/key.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-RSA-AES256-GCM-SHA512:DHE-RSA-AES256-GCM-SHA512:ECDHE-RSA-AES256-GCM-SHA384:DHE-RSA-AES256-GCM-SHA384;
    ssl_prefer_server_ciphers off;

    # Security headers
    add_header X-Frame-Options DENY;
    add_header X-Content-Type-Options nosniff;
    add_header X-XSS-Protection "1; mode=block";
    add_header Strict-Transport-Security "max-age=63072000; includeSubDomains; preload";

    # Client max body size for file uploads
    client_max_body_size 50M;

    # Frontend
    location / {
        proxy_pass http://frontend;
    }
}

```

```

    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

```

API routes

```

location /api/ {
    limit_req zone=api burst=20 nodelay;

    proxy_pass http://backend;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

```

Timeouts for AI processing

```

    proxy_connect_timeout 30s;
    proxy_send_timeout 300s;
    proxy_read_timeout 300s;
}

```

Upload routes with stricter rate limiting

```

location /api/upload {
    limit_req zone=upload burst=5 nodelay;

    proxy_pass http://backend;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

```

Extended timeouts for file uploads

```

    proxy_connect_timeout 60s;
    proxy_send_timeout 600s;
    proxy_read_timeout 600s;
}

```

```

    }

    # Static files
    location /static/ {
        alias /var/www/static/;
        expires 30d;
        add_header Cache-Control "public, immutable";
    }

    # Health check
    location /health {
        access_log off;
        proxy_pass http://backend;
    }
}
}

```

ก.9.9 ชุดคำสั่ง Production Deployment Script

```

#!/bin/bash
# deploy-production.sh

set -e

echo " 🚀 Deploying Face Recognition Platform to Production..."

# Configuration
ENVIRONMENT=${ENVIRONMENT:-production}
COMPOSE_FILE=${COMPOSE_FILE:-docker-compose.prod.yml}
BACKUP_BEFORE_DEPLOY=${BACKUP_BEFORE_DEPLOY:-true}

# Pre-deployment checks
echo " 🔍 Running pre-deployment checks..."

# Check if all required environment variables are set
required_vars=("DATABASE_URL" "REDIS_URL" "SECRET_KEY" "API_URL")
for var in "${required_vars[@]"; do
    if [ -z "${!var}" ]; then

```

```

        echo " ❌ Required environment variable $var is not set"
        exit 1
    fi
done

# Check Docker and Docker Compose
if ! command -v docker &> /dev/null; then
    echo " ❌ Docker is not installed"
    exit 1
fi

if ! command -v docker-compose &> /dev/null; then
    echo " ❌ Docker Compose is not installed"
    exit 1
fi

# Backup database before deployment
if [ "$BACKUP_BEFORE_DEPLOY" = "true" ]; then
    echo " 💾 Creating pre-deployment backup..."
    ./backup-database.sh
fi

# Pull latest images
echo " 📦 Pulling latest Docker images..."
docker-compose -f $COMPOSE_FILE pull

# Build new images
echo " 🛠 Building application images..."
docker-compose -f $COMPOSE_FILE build --no-cache

# Stop existing services
echo " 🛑 Stopping existing services..."
docker-compose -f $COMPOSE_FILE down

# Start services

```

```

echo " 🚀 Starting services..."
docker-compose -f $COMPOSE_FILE up -d

# Wait for services to be healthy
echo " ⌚ Waiting for services to be healthy..."
timeout=300
elapsed=0

while [ $elapsed -lt $timeout ]; do
    if docker-compose -f $COMPOSE_FILE ps | grep -q "healthy\|Up"; then
        echo " ✅ Services are healthy!"
        break
    fi

    echo "Waiting... ($elapsed/$timeout seconds)"
    sleep 10
    elapsed=$((elapsed + 10))
done

if [ $elapsed -ge $timeout ]; then
    echo " ❌ Services failed to become healthy within $timeout seconds"
    echo " 📋 Service status:"
    docker-compose -f $COMPOSE_FILE ps
    exit 1
fi

# Run post-deployment tests
echo " 🧪 Running post-deployment tests..."
./test-deployment.sh

echo " ✅ Production deployment completed successfully!"
echo ""
echo " 📊 Service Status:"
docker-compose -f $COMPOSE_FILE ps
echo ""

```

```

echo " 🌐 Application URLs:"
echo " Frontend: https://your-domain.com"
echo " API: https://your-domain.com/api"
echo " Health: https://your-domain.com/health"

ก.9.10 ชุดคำสั่ง Deployment Testing

#!/bin/bash
# test-deployment.sh

echo " 🧪 Testing deployment..."

# Configuration
API_URL=${API_URL:-http://localhost:8080}
FRONTEND_URL=${FRONTEND_URL:-http://localhost:3000}

# Test API health
echo " 🖨️ Testing API health..."
if curl -f "$API_URL/health" > /dev/null 2>&1; then
    echo " ✅ API health check passed"
else
    echo " ❌ API health check failed"
    exit 1
fi

# Test frontend
echo " 🌐 Testing frontend..."
if curl -f "$FRONTEND_URL" > /dev/null 2>&1; then
    echo " ✅ Frontend check passed"
else
    echo " ❌ Frontend check failed"
    exit 1
fi

# Test database connection
echo " 🗄️ Testing database connection..."
if curl -f "$API_URL/api/health/database" > /dev/null 2>&1; then

```

```

    echo "✅ Database connection test passed"
else
    echo "❌ Database connection test failed"
    exit 1
fi

# Test AI models
echo "🤖 Testing AI models..."
if curl -f "$API_URL/api/models/status" > /dev/null 2>and1; then
    echo "✅ AI models test passed"
else
    echo "❌ AI models test failed"
    exit 1
fi

# Test GPU (if available)
if command -v nvidia-smi and> /dev/null; then
    echo "🎮 Testing GPU access..."
    if curl -f "$API_URL/api/system/gpu" > /dev/null 2>and1; then
        echo "✅ GPU access test passed"
    else
        echo "⚠️ GPU access test failed (continuing anyway)"
    fi
fi

echo "✅ All deployment tests passed!"
    ก.9.11 ชุดคำสั่ง Monitoring Setup
#!/bin/bash
# setup-monitoring.sh

echo "📊 Setting up monitoring..."

# Prometheus configuration
cat > prometheus.yml << EOF

```

```

global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'face-recognition-api'
    static_configs:
      - targets: ['face-recognition-api:9090']

  - job_name: 'face-recognition-frontend'
    static_configs:
      - targets: ['face-recognition-frontend:9091']

  - job_name: 'postgres'
    static_configs:
      - targets: ['postgres-exporter:9187']
EOF

```

```

# Grafana dashboard
cat > grafana-dashboard.json << EOF
{
  "dashboard": {
    "title": "Face Recognition Platform",
    "panels": [
      {
        "title": "API Response Time",
        "type": "graph",
        "targets": [
          {
            "expr": "histogram_quantile(0.95,
sum(rate(http_request_duration_seconds_bucket[5m])) by (le))"
          }
        ]
      },
      {
        "title": "GPU Usage",
        "type": "graph",

```



```

    "targets": [
      {
        "expr": "nvidia_gpu_utilization_gpu"
      }
    ]
  }
]
}
EOF

```

echo "  Monitoring setup completed!"

ก.9.12 ชุดคำสั่ง Backup และ Restore

```
#!/bin/bash
```

```
# backup-database.sh
```

```
BACKUP_DIR="./backups"
```

```
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
```

```
# Create backup directory
```

```
mkdir -p $BACKUP_DIR
```

echo "  Creating database backup..."

```
# Backup main database
```

echo "  Backing up main database..."

```
PGPASSWORD=$DB_PASSWORD pg_dump -h $DB_HOST -p $DB_PORT -U $DB_USER
$DB_NAME > "$BACKUP_DIR/face_recognition_$TIMESTAMP.sql"
```

```
# Backup frontend database
```

echo "  Backing up frontend database..."

```
PGPASSWORD=$DB_PASSWORD pg_dump -h $DB_HOST -p $DB_PORT -U $DB_USER
$DB_FRONTEND_NAME > "$BACKUP_DIR/face_recognition_frontend_$TIMESTAMP.sql"
```

```
# Compress backups
```

```

echo " 🗄 Compressing backups..."
gzip "$BACKUP_DIR/face_recognition_ $TIMESTAMP.sql"
gzip "$BACKUP_DIR/face_recognition_frontend_ $TIMESTAMP.sql"

# Clean old backups (keep last 7 days)
echo " 🧹 Cleaning old backups..."
find $BACKUP_DIR -name "*.sql.gz" -mtime +7 -delete

echo " ✅ Database backup completed!"
echo " 📁 Backup files:"
echo " - $BACKUP_DIR/face_recognition_ $TIMESTAMP.sql.gz"
echo " - $BACKUP_DIR/face_recognition_frontend_ $TIMESTAMP.sql.gz"
#!/bin/bash
# restore-database.sh

if [ -z "$1" ]; then
    echo "Usage: $0 <backup_file>"
    exit 1
fi

BACKUP_FILE=$1
DB_NAME="face_recognition"
DB_USER="postgres"

echo "Restoring database from: $BACKUP_FILE"

# Drop and recreate database
dropdb -h localhost -U $DB_USER $DB_NAME
createdb -h localhost -U $DB_USER $DB_NAME

# Restore from backup
if [[ $BACKUP_FILE == *.gz ]]; then
    gunzip -c $BACKUP_FILE | psql -h localhost -U $DB_USER -d $DB_NAME
else
    psql -h localhost -U $DB_USER -d $DB_NAME < $BACKUP_FILE

```

fi

echo "Database restored successfully"

ก.9.13 ชุดคำสั่ง System Maintenance

#!/bin/bash

system-maintenance.sh

echo "🔧 Running system maintenance..."

Clean up Docker resources

echo "🧹 Cleaning up Docker resources..."

docker system prune -f

docker volume prune -f

Clean up temporary files

echo "🧹 Cleaning up temporary files..."

find ./temp -type f -mtime +1 -delete

find ./logs -type f -name "*.log" -mtime +30 -delete

Optimize database

echo "🗄️ Optimizing database..."

PGPASSWORD=\$DB_PASSWORD psql -h \$DB_HOST -p \$DB_PORT -U \$DB_USER -d
\$DB_NAME -c "VACUUM ANALYZE;"

Update AI models (if needed)

echo "🤖 Checking for AI model updates..."

./check-model-updates.sh

Generate performance report

echo "📊 Generating performance report..."

./generate-performance-report.sh

echo "✅ System maintenance completed!"

ภาพผนวก ข

คู่มือการติดตั้ง แพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพบุคคลด้วย
การจดจำใบหน้า

คู่มือการติดตั้งแพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพบุคคล ด้วยการจดจำใบหน้า

การติดตั้งระบบแพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพบุคคลด้วยการจดจำ
ใบหน้า ติดตั้งทั้งหมด 3 ส่วนดังนี้

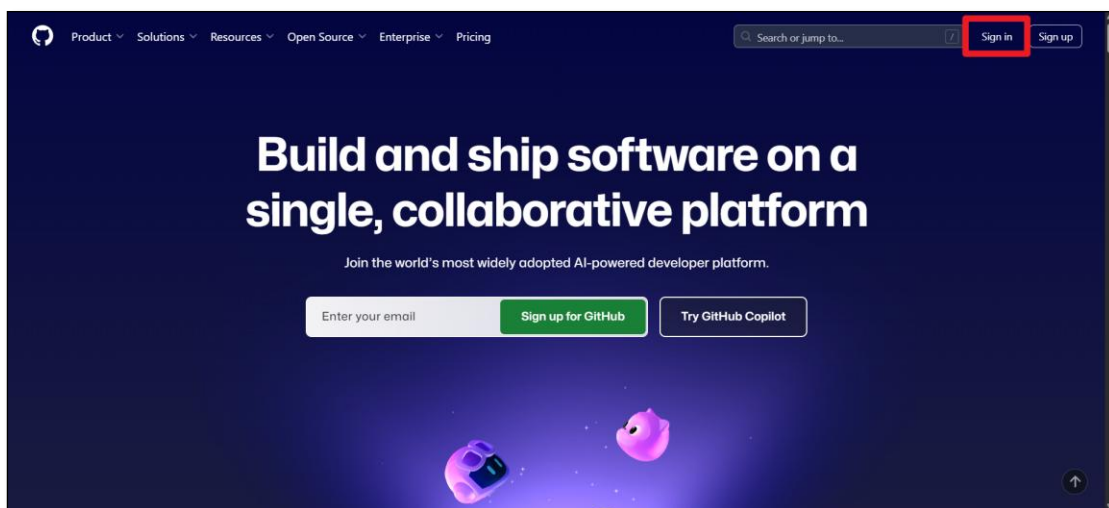
ข.1 อัปโหลดไฟล์ใน Github

ข.2 การติดตั้ง Docker

ข.3 การเริ่มการทำงานของแพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพบุคคล
ด้วยการจดจำใบหน้า

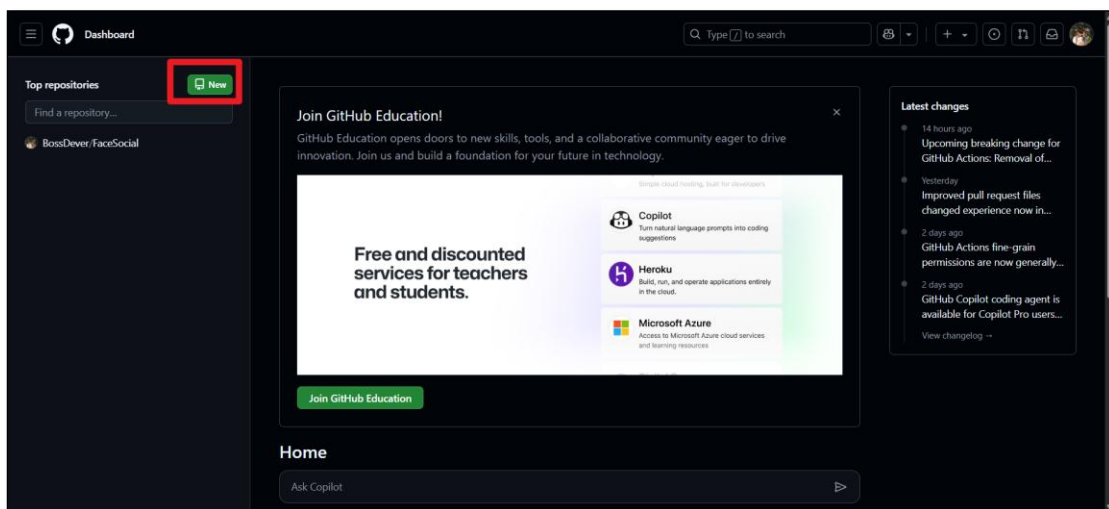
ข.1 อัปโหลดไฟล์ใน Github

ข.1.1 เข้าสู่ระบบในเว็บไซต์ <https://github.com/> ดังภาพที่ ข-1



ภาพที่ ข-1 หน้าจอการเข้าสู่ระบบเว็บไซต์ GitHub

ข.1.2 จากนั้นกดสร้าง Repository เพื่อสร้างพื้นที่เก็บข้อมูล ดังภาพที่ ข-2



ภาพที่ ข-2 หน้าจอการสร้าง Repository

ข.1.3 ตั้งชื่อ Repository จากนั้นกด Create repository เพื่อสร้างที่เก็บข้อมูล
ดังภาพที่ ข-3

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk ().*

Owner * BossDever / **Repository name *** FaceSocial01
 ✓ FaceSocial01 is available.

Great repository names are short and memorable. Need inspiration? How about [effective-sniffle](#) ?

Description (optional)

☒ **Public**
 Anyone on the internet can see this repository. You choose who can commit.

☐ **Private**
 You choose who can see and commit to this repository.

Initialize this repository with:

☐ **Add a README file**
 This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore
 .gitignore template: None

Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Choose a license
 License: None

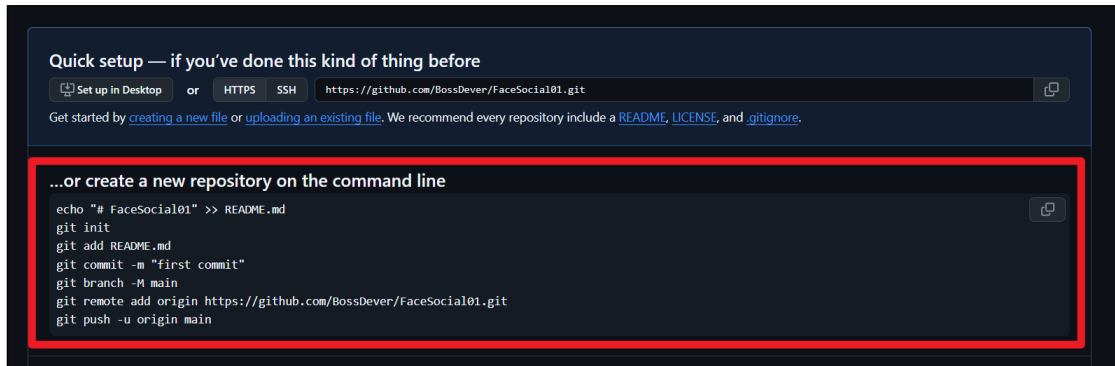
A license tells others what they can and can't do with your code. [Learn more about licenses.](#)

ⓘ You are creating a public repository in your personal account.

Create repository

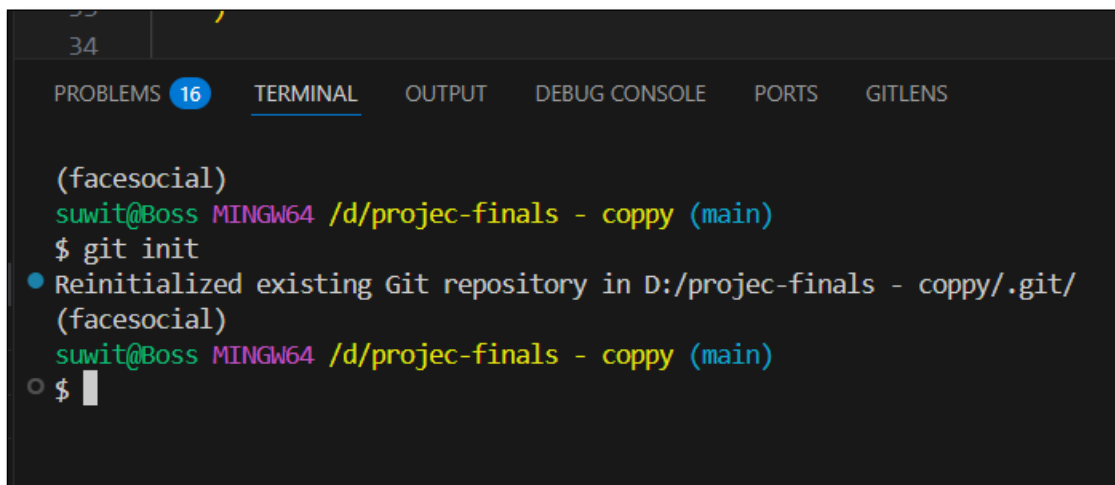
ภาพที่ ข-3 หน้าจอการตั้งชื่อ Repository

ข.1.4 ทำการ Copy code ในส่วนนี้เป็นใช้ในไฟล์ที่ต้องการอัปโหลด ดังภาพที่ ข.-4



ภาพที่ ข-4 หน้าจอการคัดลอกโค้ดสำหรับการอัปโหลดไฟล์

ข.1.5 เปิด Terminal ใน Visual studio code แล้วรันคำสั่ง Git init เพื่อสร้างโฟลเดอร์ .Git ในโฟลเดอร์ปัจจุบันไว้เก็บ Code ของโปรเจกต์นี้สำหรับเอาขึ้น Git Repository ดังภาพที่ ข-5



ภาพที่ ข-5 หน้าจอการรันคำสั่ง git init

ข.1.6 จากนั้นให้รันคำสั่ง git add เพื่อเพิ่มไฟล์ทั้งหมดที่มีการเปลี่ยนแปลงในโฟลเดอร์ ปัจจุบันให้เข้าสู่สถานะเตรียมพร้อม (Staged) ไว้สำหรับการยืนยัน (Commit) ดังภาพที่ ข-6


```
(facesocial)
suwit@Boss MINGW64 /d/projec-finals - coppy (main)
$ git add .
warning: in the working copy of 'projec-final-fronend/.gitignore', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'projec-final-fronend/README.md', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'projec-final-fronend/components.json', LF will be replaced by CRLF the next time Git touches it
```

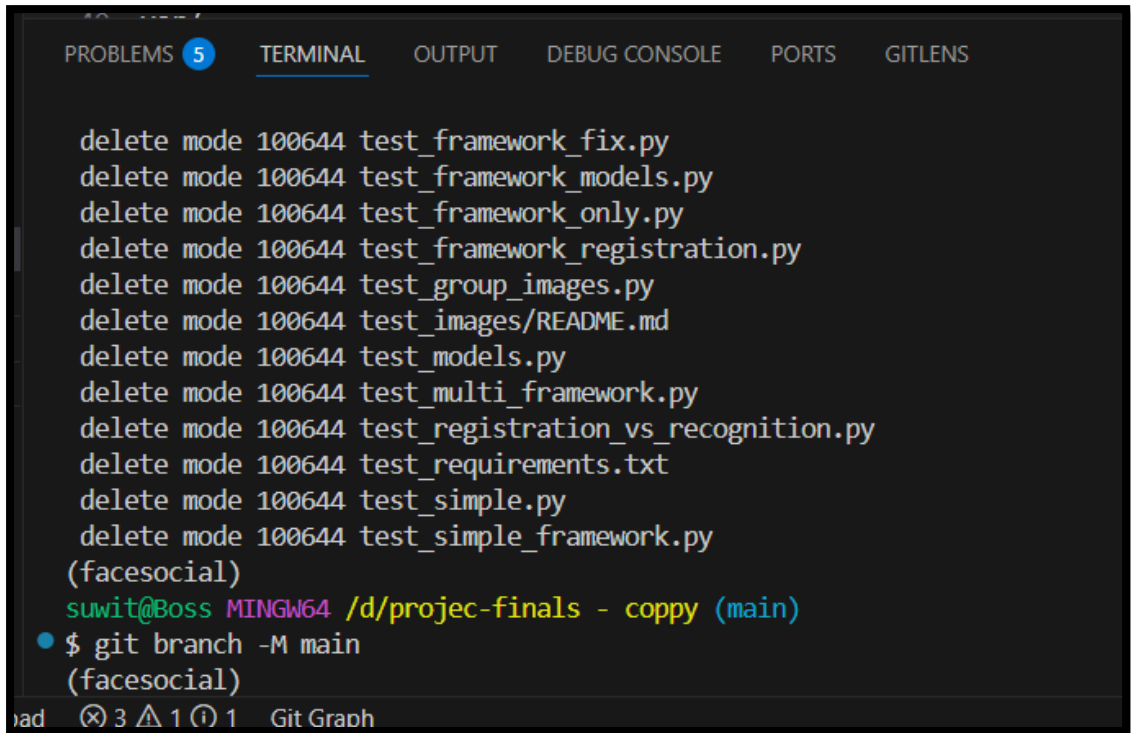
ภาพที่ ข-6 หน้าจอการรันคำสั่ง git add

ข.1.7 จากนั้นรันคำสั่ง `git commit -m "first commit"` เพื่อยืนยัน (Commit) ไฟล์ทั้งหมดที่อยู่ในสถานะเตรียมพร้อม (Staged) และเขียนข้อความเพื่ออธิบายการยืนยัน (Commit) ครั้งนั้นในที่นี้คือ First Commit ดังภาพที่ ข-7

```
PROBLEMS 5 TERMINAL OUTPUT DEBUG CONSOLE PORTS GITLEN
(facesocial)
suwit@Boss MINGW64 /d/projec-finals - coppy (main)
$ git commit -m "first commit"
[main 3159de1] first commit
292 files changed, 74250 insertions(+), 14805 deletions(-)
create mode 100644 Dockerfile
delete mode 100644 README_MULTI_FRAMEWORK.md
delete mode 100644 api_test_complete.py
delete mode 100644 api_test_log.txt
delete mode 100644 check_api_endpoints.py
delete mode 100644 check_models_status.py
delete mode 100644 complete_multi_model_test.py
delete mode 100644 comprehensive_api_test.py
delete mode 100644 config_optimization.py
delete mode 100644 create_model_dirs.bat
delete mode 100644 crop_faces_clean.py
⊗ 3 △ 1 ⓘ 1 Git Graph
```

ภาพที่ ข-7 หน้าจอการรันคำสั่ง git commit -m "first commit"

ข.1.8 จากนั้นรันคำสั่ง `git branch -M main` เพื่อสร้างหรือเปลี่ยนแปลงชื่อของสาขา (Branch) เป็น Main ในที่นี้เก็บข้อมูล Git Repository ดังภาพที่ ข-8



```

delete mode 100644 test_framework_fix.py
delete mode 100644 test_framework_models.py
delete mode 100644 test_framework_only.py
delete mode 100644 test_framework_registration.py
delete mode 100644 test_group_images.py
delete mode 100644 test_images/README.md
delete mode 100644 test_models.py
delete mode 100644 test_multi_framework.py
delete mode 100644 test_registration_vs_recognition.py
delete mode 100644 test_requirements.txt
delete mode 100644 test_simple.py
delete mode 100644 test_simple_framework.py
(facesocial)
suwit@Boss MINGW64 /d/projec-finals - coppy (main)
● $ git branch -M main
(facesocial)

```

ภาพที่ ข-8 หน้าจอการรันคำสั่ง git branch -M main

ข.1.9 จากนั้นรันคำสั่ง git remote add origin เพื่อเชื่อมต่อที่เก็บข้อมูล Git Repository ในเครื่องเข้ากับเซิร์ฟเวอร์ของ GitHub และระบุที่อยู่เว็บ (URL) ของที่เก็บข้อมูลบน GitHub ดังภาพที่ ข-9

```

delete mode 100644 test_framework_registration.py
delete mode 100644 test_group_images.py
delete mode 100644 test_images/README.md
delete mode 100644 test_models.py
delete mode 100644 test_multi_framework.py
delete mode 100644 test_registration_vs_recognition.py
delete mode 100644 test_requirements.txt
delete mode 100644 test_simple.py
delete mode 100644 test_simple_framework.py
(facesocial)
suwit@Boss MINGW64 /d/projec-finals - copy (main)
$ git branch -M main
(facesocial)
suwit@Boss MINGW64 /d/projec-finals - copy (main)
$ git remote set-url origin https://github.com/BossDever/FaceSocial01.git
(facesocial)
⊗ 3 △ 1 ① 1 Git Graph

```

ภาพที่ ข-9 หน้าจอการรันคำสั่ง git remote add origin

ข.1.10 จากนั้นรันคำสั่ง git push -u origin main เพื่ออัปโหลดไฟล์ที่ยืนยัน (Commit) ทั้งหมดไปยังที่เก็บข้อมูล (Repository) บนเซิร์ฟเวอร์ GitHub ดังภาพที่ ข-10

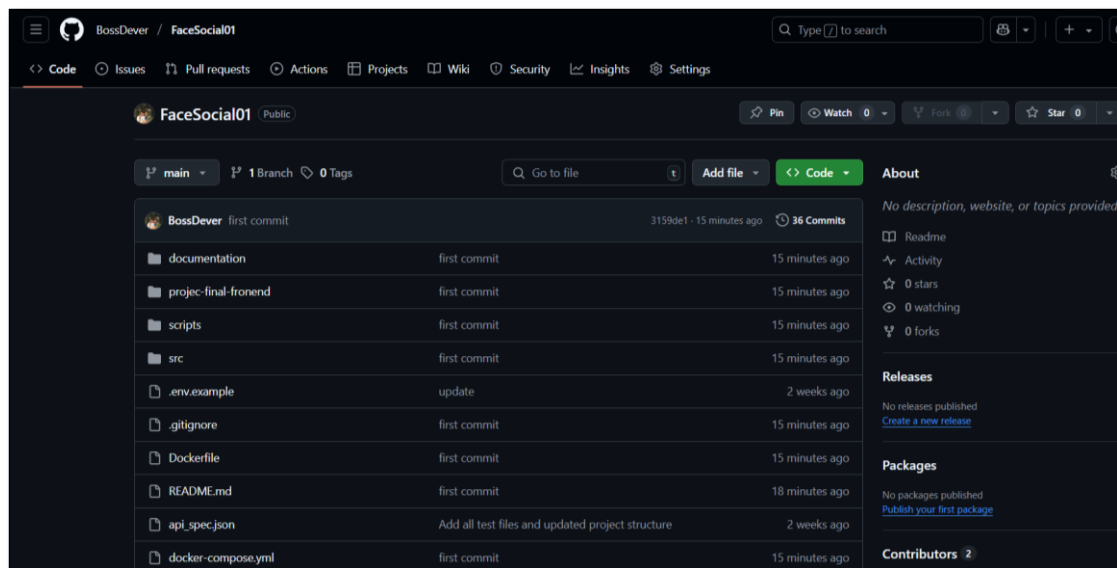
```

suwit@Boss MINGW64 /d/projec-finals - copy (main)
$ git remote set-url origin https://BossDever@github.com/BossDever/FaceSocial01.git
(facesocial)
suwit@Boss MINGW64 /d/projec-finals - copy (main)
$ git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 787, done.
Counting objects: 100% (787/787), done.
Delta compression using up to 16 threads
Compressing objects: 100% (698/698), done.
Writing objects: 100% (787/787), 7.42 MiB | 3.62 MiB/s, done.
Total 787 (delta 239), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (239/239), done.
To https://github.com/BossDever/FaceSocial01.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
pad ⊗ 3 △ 1 ① 1 Git Graph

```

ภาพที่ ข-10 หน้าจอการรันคำสั่ง git push -u origin main

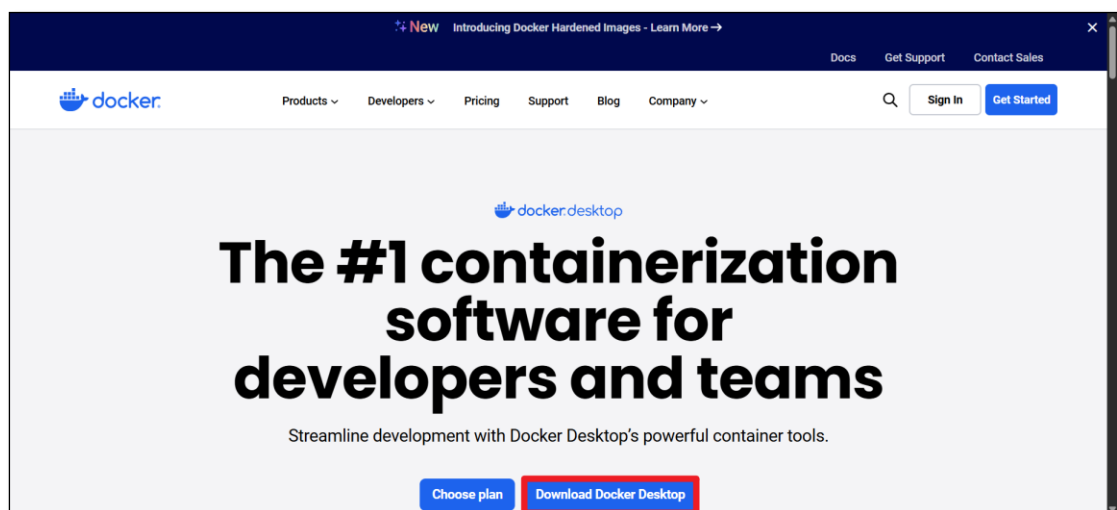
ข.1.11 เมื่อทำเสร็จไฟล์จะถูกอัปโหลดมาในเว็บไซต์ GitHub ดังภาพที่ ข-11



ภาพที่ ข-11 หน้าจอไฟล์ที่อัปโหลดในเว็บไซต์ GitHub

ข.2 การติดตั้ง Docker

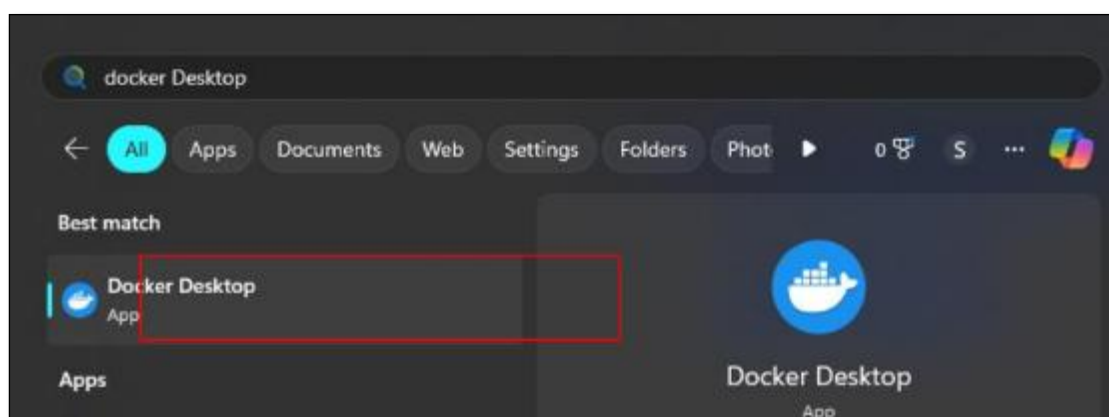
ข.2.1 เบื้องต้นในการติดตั้ง Docker มีรูปแบบในการติดตั้งหลายรูปแบบตาม OS ที่ใช้งาน ซึ่งผู้จัดทำใช้งาน OS ในรูปแบบของ Windows ดังนั้นจะยกตัวอย่างการติดตั้ง Docker ในส่วนของ Windows 11 โดยการติดตั้ง Docker บน Windows 11 เริ่มต้นด้วยการติดตั้ง Docker Desktop for Windows จากเว็บไซต์ของ Docker ผ่าน URL: <https://www.docker.com/> ดังภาพที่ ข-12



ภาพที่ ข-12 หน้าต่างติดตั้ง Docker บน Windows 11

จากภาพที่ ข-1 หลังจาก Download for Windows แล้วให้ทำการติดตั้ง ตามขั้นตอนการติดตั้งในตัวติดตั้ง Docker Desktop ซึ่งจะป็นรุ่น 25.0.3

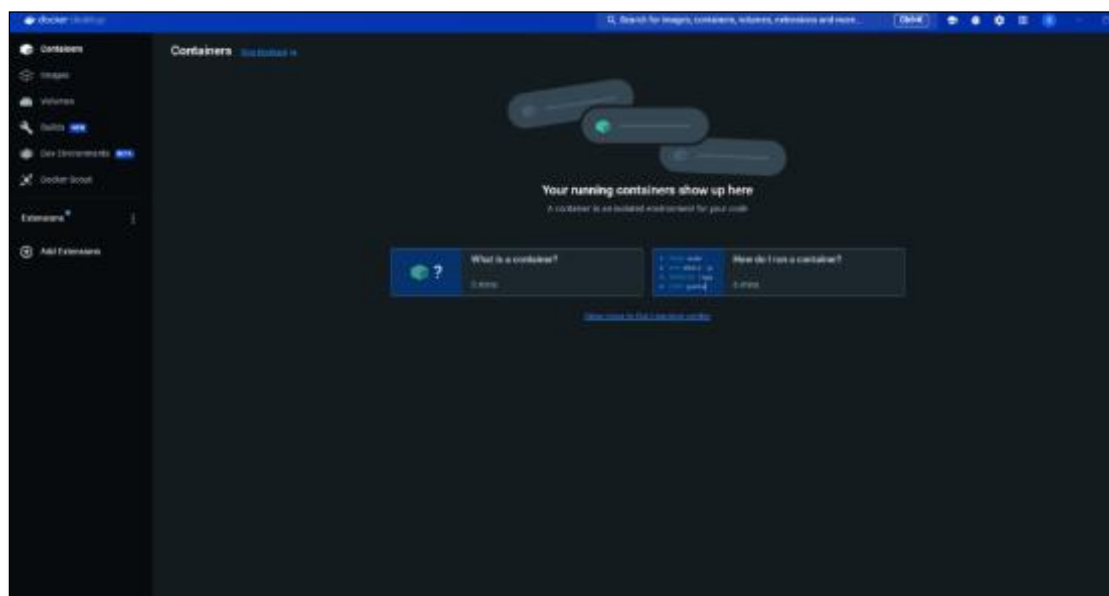
ข.2.2 หลังจากทำการติดตั้ง Docker เรียบร้อยแล้วให้ค้นหา Docker Desktop ในเมนู Start Windows เพื่อตรวจสอบว่า Docker Desktop สามารถติดตั้งได้เรียบร้อย ดังภาพที่ ข-13



ภาพที่ ข-13 หน้าต่าง Docker Desktop สามารถติดตั้งได้

จากภาพที่ ข-2 เป็นการตรวจสอบว่า Docker Desktop ติดตั้งได้หรือไม่ ถ้าได้จะมีสัญลักษณ์ของ Docker Desktop อยู่ในคอมพิวเตอร์

ข.2.3 หลังจากติดตั้ง Docker Desktop แล้วทำการเป็นเห็นชั้นข้อมูล (File) ดังภาพที่ ข-14

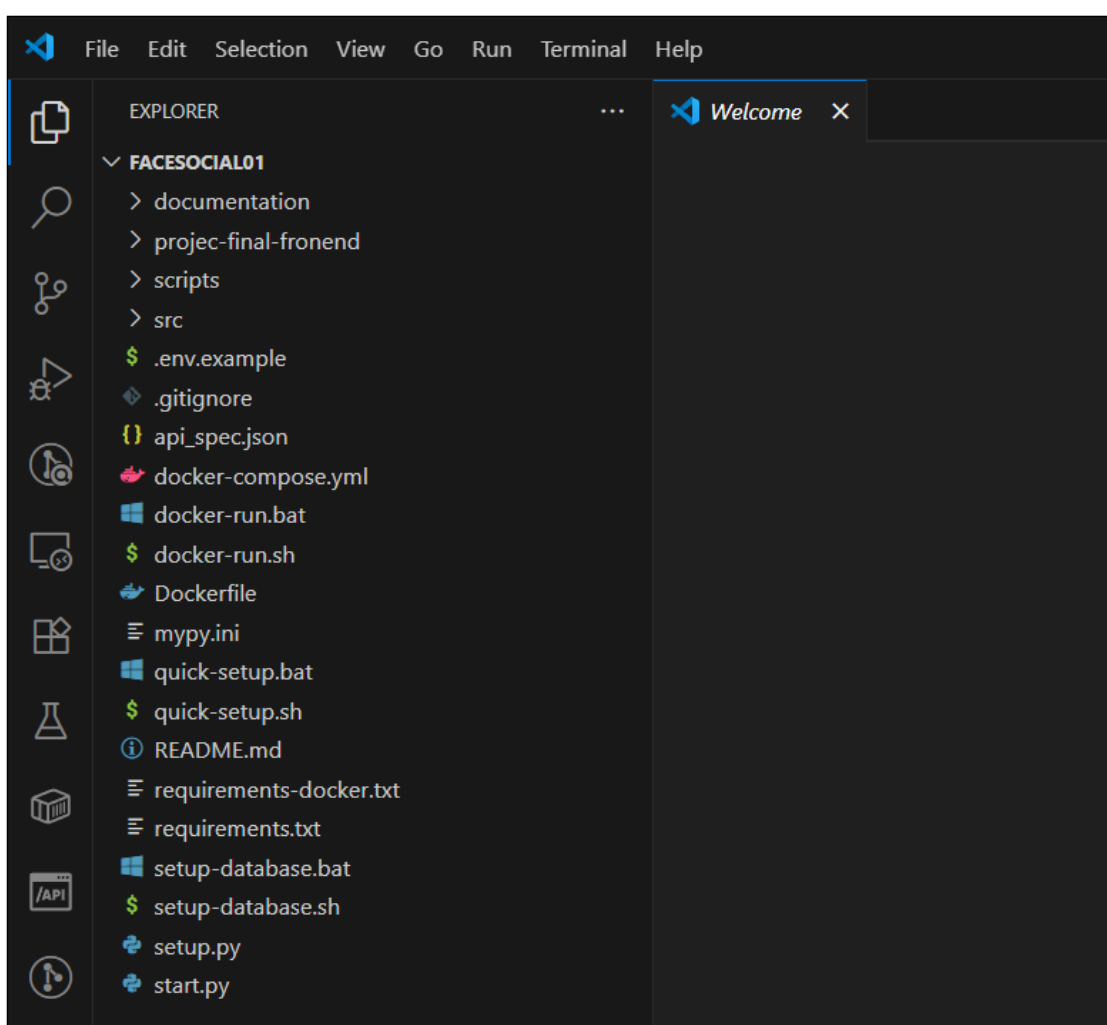


ภาพที่ ข-14 หน้าต่าง Docker Desktop

จากภาพที่ ข-3 หลังจากเปิดแฟ้มข้อมูล (File) Docker Desktop ขึ้นมาแล้วจะมีหน้าต่างตามภาพ นั้นหมายความว่า Docker Desktop ของ Windows 11 นั้นพร้อมใช้งานแล้ว

ข.3 การเริ่มการทำงานของแพลตฟอร์มสื่อสังคมออนไลน์ และการจัดการรูปภาพบุคคล ด้วยการจดจำใบหน้า

ข.3.1 ให้รันคำสั่ง `git clone https://github.com/BossDever/FaceSocial01.git` เพื่อโคลนที่เก็บข้อมูล (Clone Repository) ลงมาในตำแหน่งที่จะรันโปรเจกต์ แล้วทำการเปิดโปรเจกต์ขึ้นมา ดังภาพที่ ข-15



ภาพที่ ข-15 หน้าจอโปรเจกต์ที่โคลนจากที่เก็บข้อมูล (Repository)

ข.3.2 ให้รันคำสั่ง `chmod +x quick-setup.sh` เพื่อให้สิทธิ์การเรียกใช้งาน execute และรัน `./quick-setup.sh` เพื่อให้ระบบทำงานอัตโนมัติ ดังภาพที่ ข-16

```
(facesocial)
suwit@Boss MINGW64 /d/work/FaceSocial01 (main)
$ # ให้สิทธิ์ execute
chmod +x quick-setup.sh

# รันไฟล์ setup
./quick-setup.sh
```

ภาพที่ ข-16 หน้าจอการเริ่มต้นระบบแพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพบุคคลด้วยการจดจำใบหน้า

ภาพผนวก ค

คู่มือการใช้งาน แพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพบุคคลด้วย
การจดจำใบหน้า

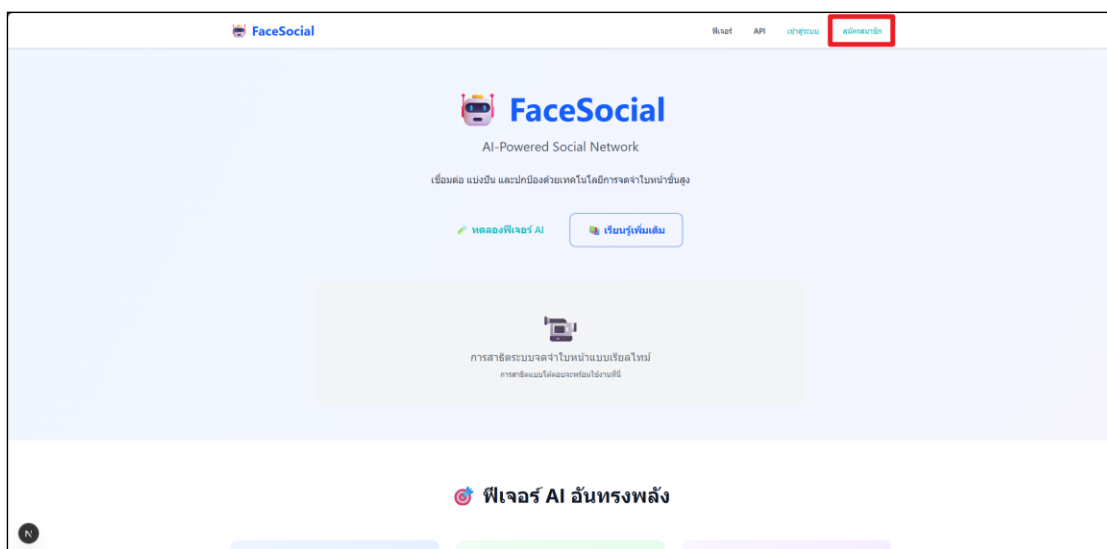
คู่มือการใช้งานแพลตฟอร์มสื่อสังคมออนไลน์และการจัดการรูปภาพบุคคล ด้วยการจดจำใบหน้า

วิธีการใช้งานแบ่งออกเป็น 4 กระบวนการดังนี้

- ค.1.สมัครสมาชิก
- ค.2.การเข้าสู่ระบบ
- ค.3.การใช้งานเว็บไซต์
- ค.4.การทดสอบการทำงานของ AI

ค.1 สมัครสมาชิก

ค.1.1 การสมัครสมาชิกให้เลือกที่เมนูสมัครสมาชิก ดังภาพที่ ค-1



ภาพที่ ค-1 หน้าจอแรกของเว็บไซต์

ค.1.2 หลังจากทักดปุ่มสมัครสมาชิก จะแสดงหน้าให้กรอกข้อมูล ดังภาพที่ ค-2

Figure ค-2 shows a registration form with the following fields and options:

- ชื่อ (First Name):** Text input field.
- นามสกุล (Last Name):** Text input field.
- อีเมล (Email):** Text input field with placeholder 'example@email.com'.
- ชื่อผู้ใช้ (Username):** Text input field with placeholder 'username'.
- รหัสผ่าน (Password):** Password input field.
- ยืนยันรหัสผ่าน (Confirm Password):** Password input field.
- วันเกิด (Date of Birth):** Date input field with placeholder 'DD/MM/YYYY'.
- เพศ (Gender):** Radio button options for Male and Female.
- ฉันยอมรับข้อกำหนดและเงื่อนไข (I agree to the Terms and Conditions):** Checkbox.
- ปุ่มดำเนินการต่อ (Next Step):** Button labeled 'ดำเนินการต่อ' (Next Step).

ภาพที่ ค-2 หน้าต่างกรอกข้อมูลการสมัครสมาชิก

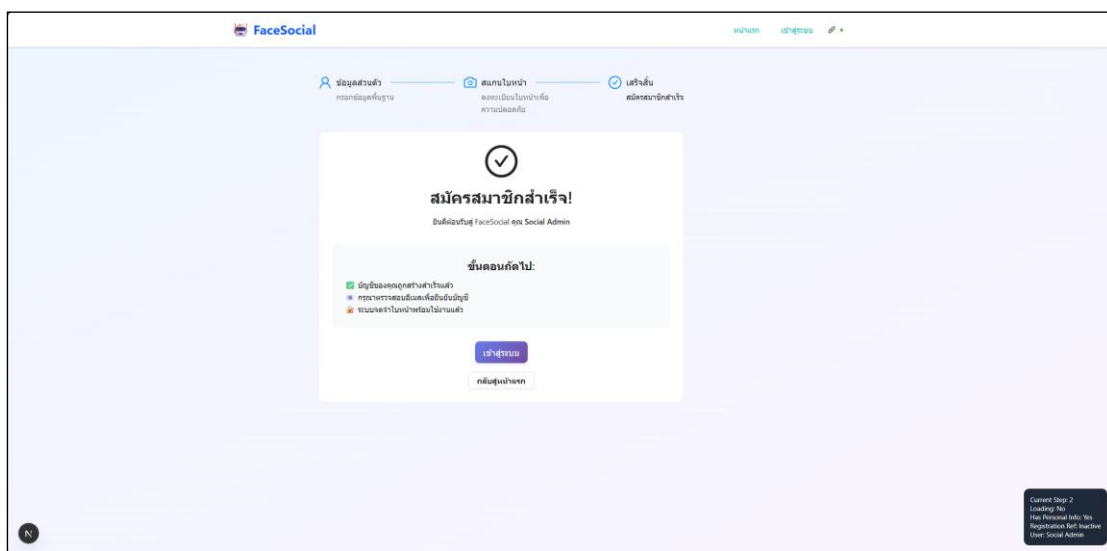
ค.1.3 หลังจากกรอกข้อมูลเสร็จแล้วกดปุ่มดำเนินการต่อ จะไปหน้าเก็บข้อมูลใบหน้า ดังภาพที่ ค-3

Figure ค-3 shows a face scanning interface with the following sections:

- Face Recognition API Status:**
 - Face Detection: ●
 - Face Analysis: ●
 - Face Recognition: ●
 - GPU Manager: ●
 - Response Time: 12ms (Last Checked: 14/02/23)
- สแกนใบหน้าเพื่อความปลอดภัย (Face Scanning for Security):**
 - ขั้นตอนการสแกน (Scanning Steps):
 - Face Detection: ตรวจหาใบหน้าในกล้อง
 - Anti-Spoofing: ตรวจสอบใบหน้าจริง
 - Quality Check: ตรวจสอบคุณภาพภาพ
 - Embedding: สร้าง Face Embedding
 - ผลลัพธ์ (Results):
 - ใบหน้าที่ถูกตรวจจับได้
 - เวลาที่ตรวจจับ: > 10"
 - ใบหน้าที่ถูกตรวจจับได้
 - เวลาที่ตรวจจับ: > 10"
 - เวลาที่ตรวจจับ: > 10"
 - เวลาที่ตรวจจับ: > 10"

ภาพที่ ค-3 หน้าต่างสแกนหน้าเพื่อสมัครสมาชิก

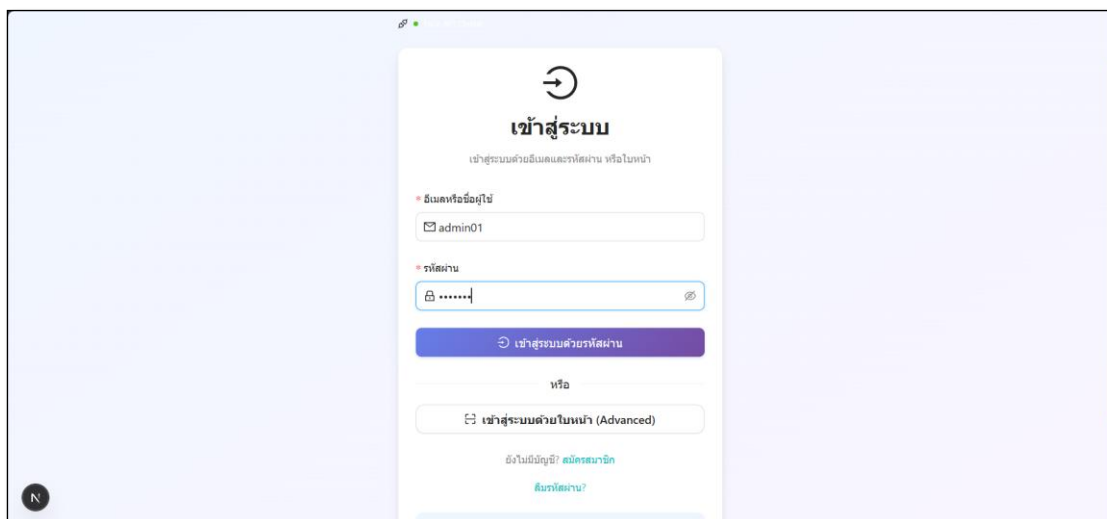
ค.1.4 หลังจากสแกนหน้าเสร็จบันทึกข้อมูลเสร็จสิ้นแล้ว ระบบจะขึ้นข้อความสมัครสมาชิกสำเร็จอัตโนมัติ ดังภาพที่ ค-4



ภาพที่ ค-4 หน้าต่างข้อความสมัครสมาชิกสำเร็จ

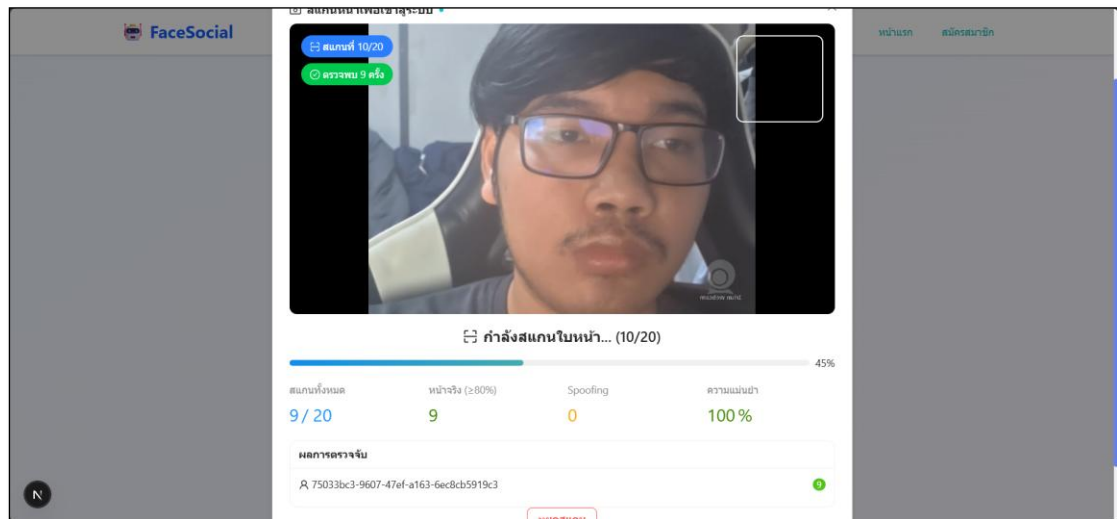
ค.2.การเข้าสู่ระบบ

ค.2.1 การกรอกข้อมูลเข้าสู่ระบบแบบทั่วไป ดังภาพที่ ค-5



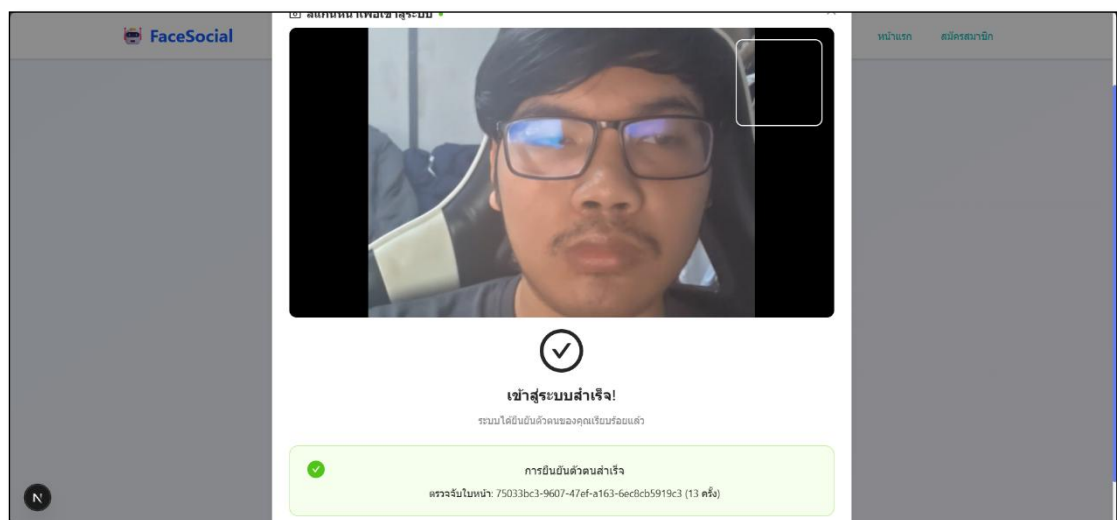
ภาพที่ ค-5 หน้าจอการกรอกข้อมูลเข้าสู่ระบบแบบทั่วไป

ค.2.2 การสแกนหน้าเพื่อเข้าสู่ระบบ ดังภาพที่ ค-6



ภาพที่ ค-6 หน้าจอการสแกนหน้าเพื่อเข้าสู่ระบบ

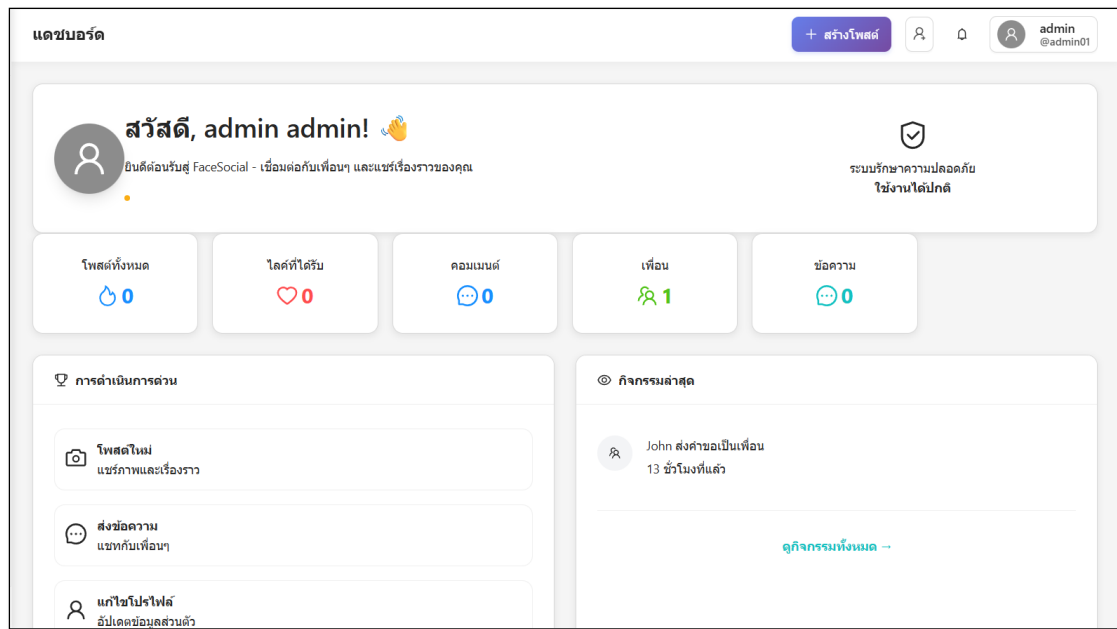
ค.2.2 หลังจากสแกนหน้าเสร็จแล้วจะแสดงข้อความเข้าสู่ระบบสำเร็จ ดังภาพที่ ค-7



ภาพที่ ค-7 หน้าจอข้อความเข้าสู่ระบบสำเร็จ

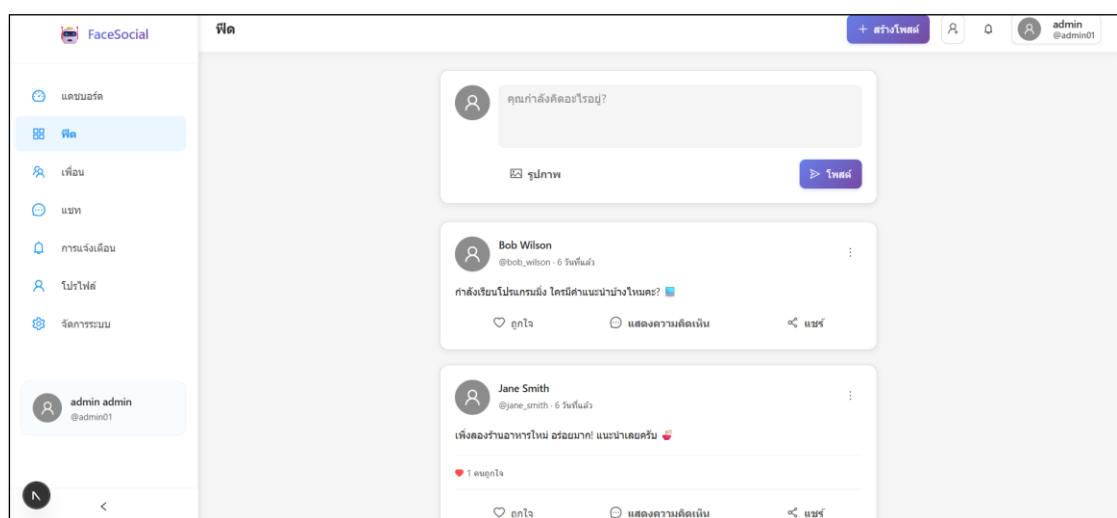
ค.3.การใช้งานเว็บไซต์

ค.3.1 เมื่อเข้าสู่ระบบมาแล้วจะมีหน้า Dashboard (Dashboard) ที่จะบอกรายละเอียดและกิจกรรมต่างๆ ที่ผู้ใช้เคยทำไว้ ดังภาพที่ ค-8



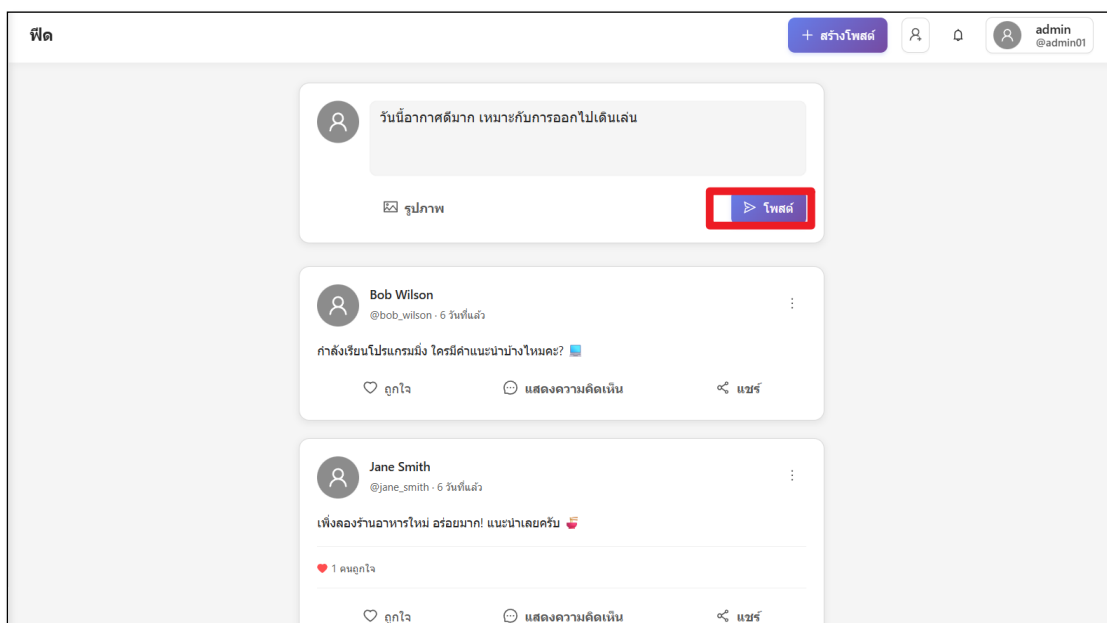
ภาพที่ ค-8 หน้า Dashboard

ค.3.2 หน้า Feed ที่จะมีรายละเอียดข่าวสารของเพื่อนหรือผู้ใช้งานคนอื่นที่โพสต์สาธารณะไว้ ดังภาพที่ ค-9



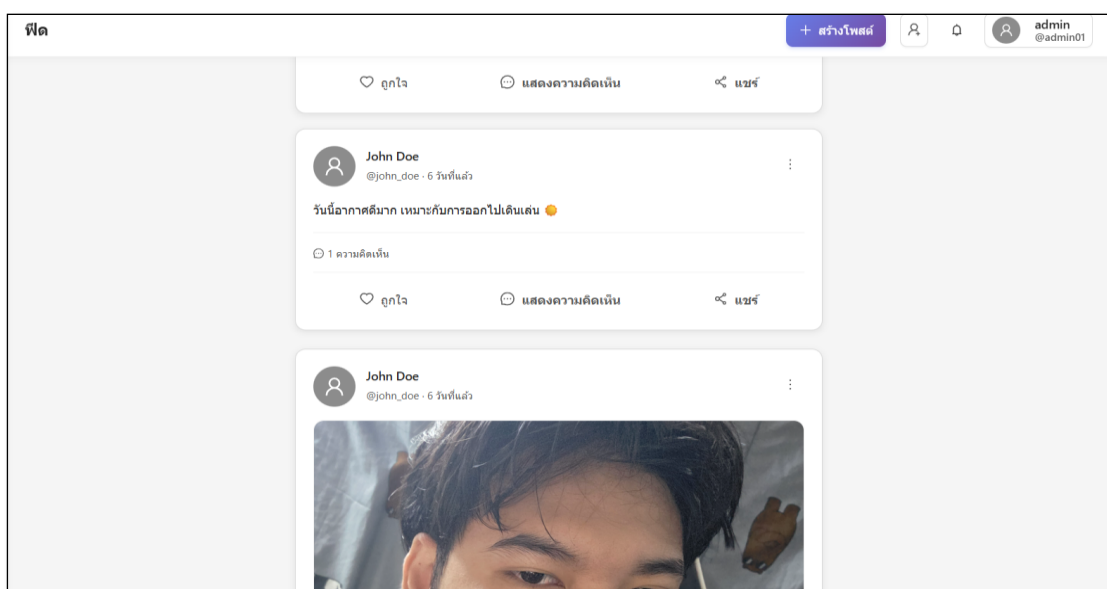
ภาพที่ ค-9 หน้า Feed

ค.3.3 จากภาพที่ ค-9 จะเห็นว่าหน้านี้มี Function โพสต์อยู่ ถ้าเขียนข้อความแล้วกดปุ่ม โพสต์ ข้อความจะโพสต์ให้เพื่อนเห็นได้ทันที ดังภาพที่ ค-10



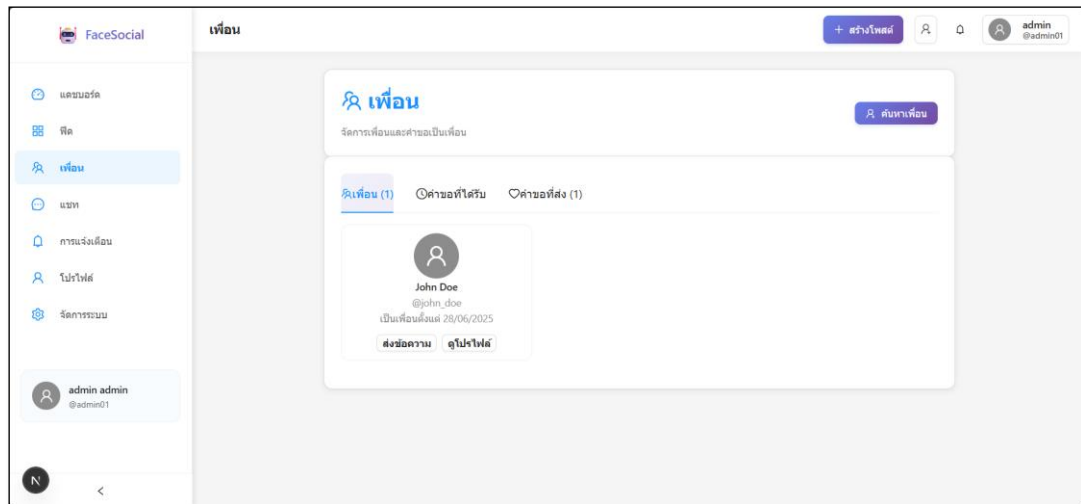
ภาพที่ ค-10 ฟังก์ชันโพสต์ของหน้า Feed หลังจากกดโพสต์แล้ว

ค.3.4 หลังจากกดโพสต์แล้วจะได้โพสต์ดังภาพที่ ค-11



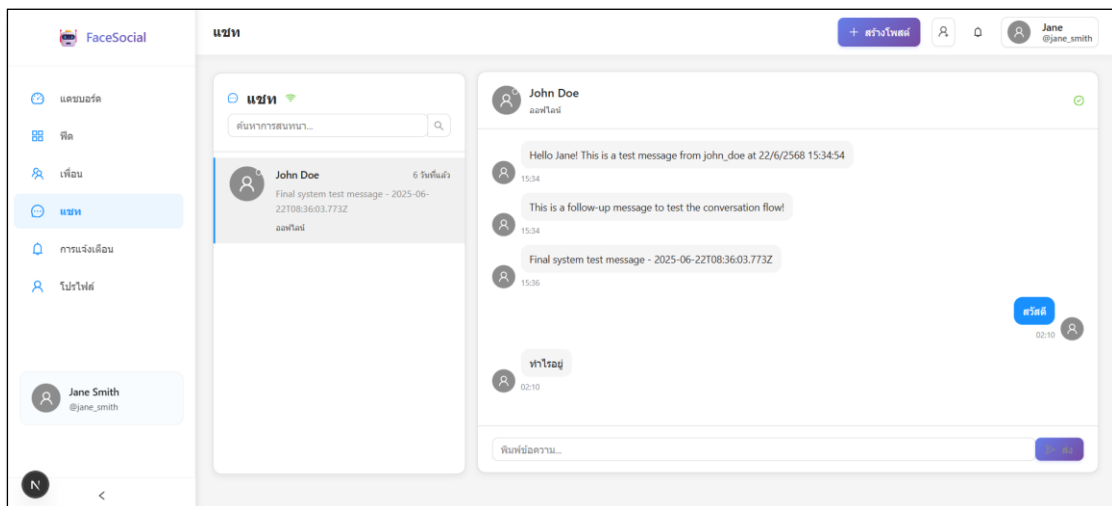
ภาพที่ ค-11 การโพสต์ข้อความในหน้า Feed

ค.3.5 หน้าเพื่อนสามารถดูรายชื่อเพื่อนได้ที่หน้านี้ และยังสามารถไปดูประวัติโดยย่อหรือส่งข้อความได้ที่หน้านี้ด้วย ดังภาพที่ ค-12



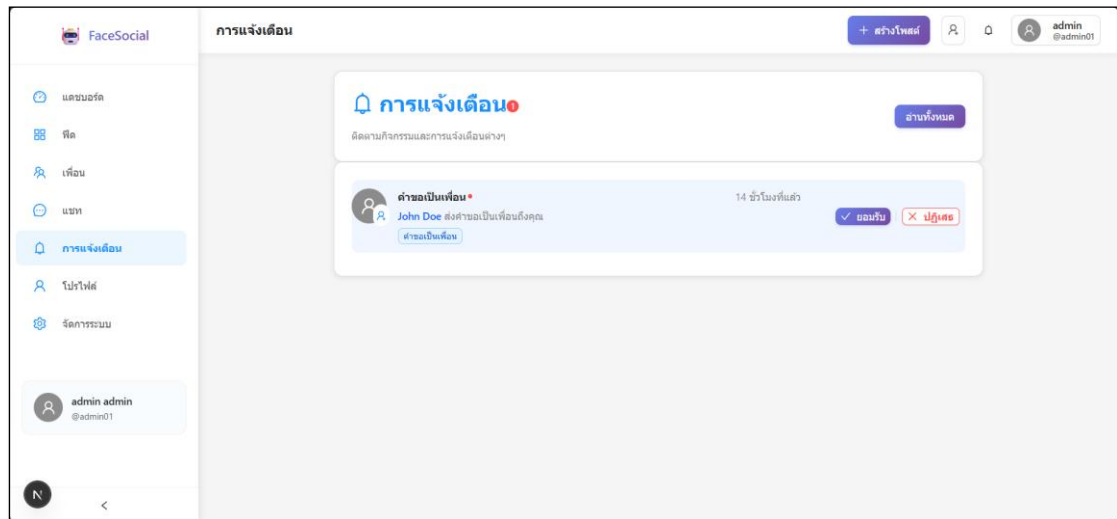
ภาพที่ ค-12 หน้าเพื่อน

ค.3.6 หน้า Chat ทำให้สามารถพิมพ์ข้อความส่งหาเพื่อนๆ ได้จากหน้านี้ ดังภาพที่ ค-13



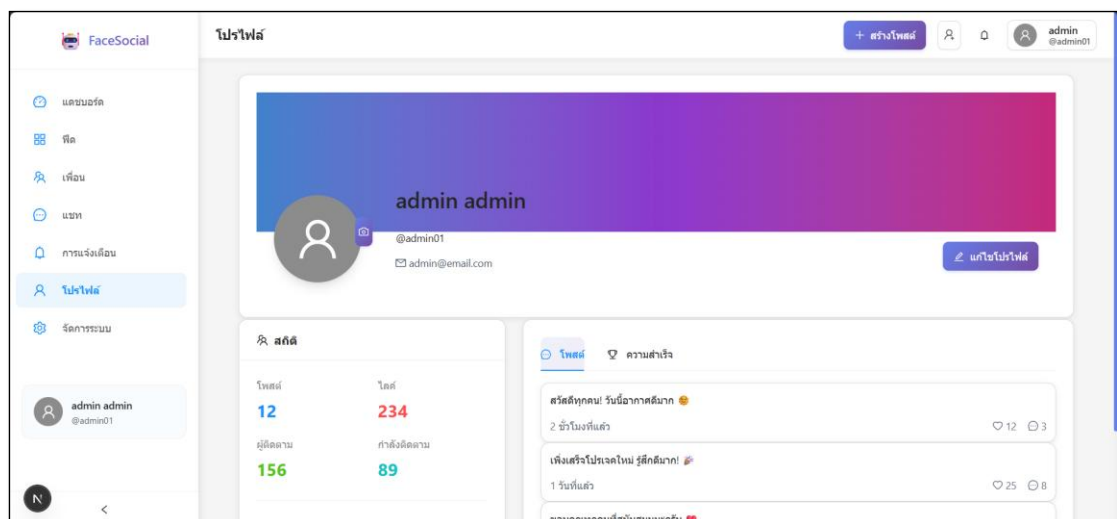
ภาพที่ ค-13 หน้า Chat

ค.3.7 หน้าการแจ้งเตือน การแจ้งเตือนทั้งหมดจะอยู่ที่หน้านี้ เช่น การเพิ่มเพื่อนและการแท็กจากรูปภาพ ดังภาพที่ ค-14



ภาพที่ ค-14 หน้าการแจ้งเตือน

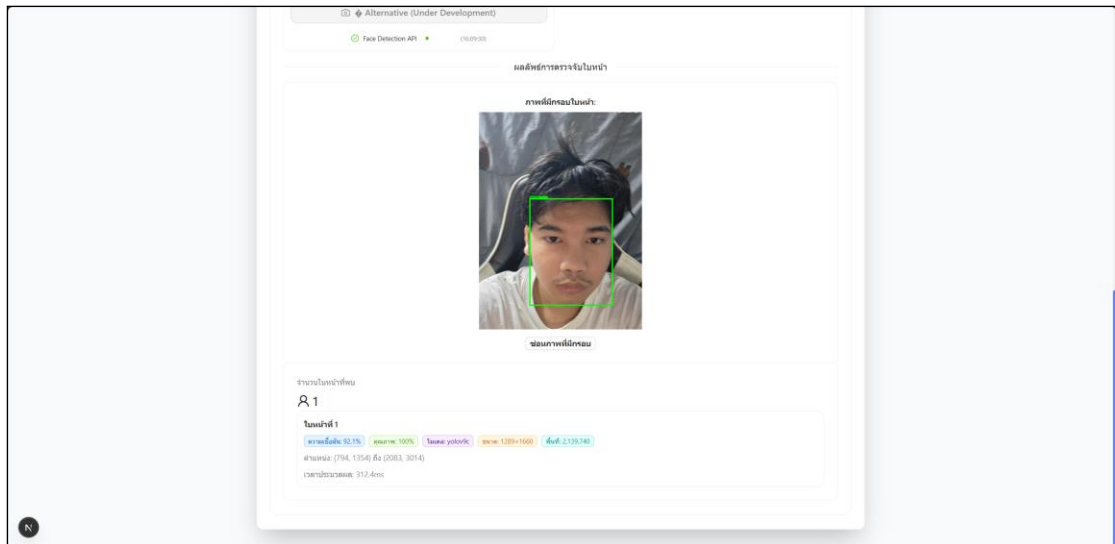
ค.3.8 หน้าประวัติโดยย่อที่บอกข้อมูลของผู้ใช้ มีการบอกความสำเร็จที่ทำได้ และดูโพสต์ทั้งหมดที่เคยโพสต์ไว้ ดังภาพที่ ค-15



ภาพที่ ค-15 ประวัติโดยย่อของผู้ใช้

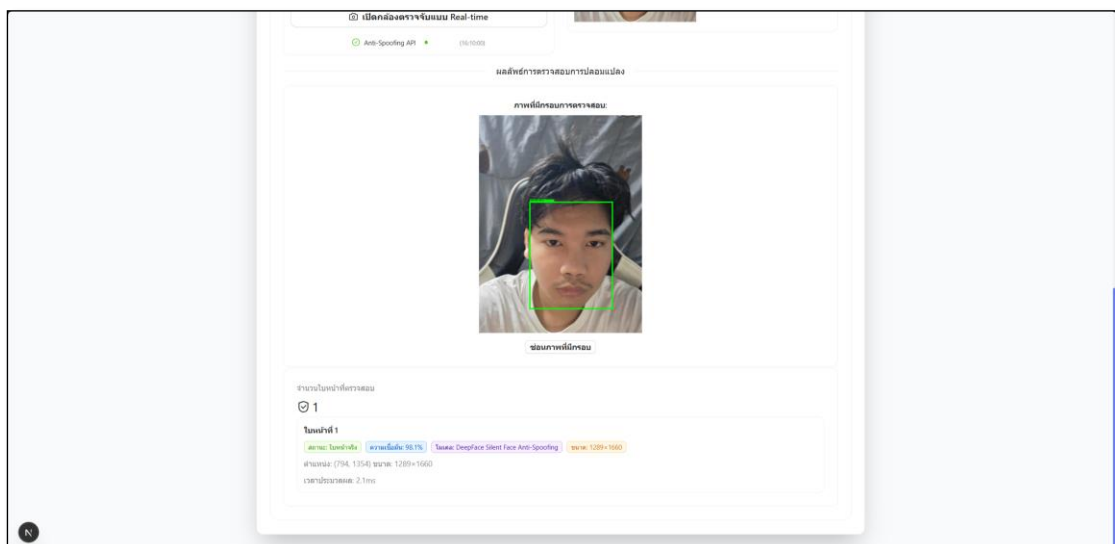
ค.4 การทดสอบการทำงานของ AI

ค.4.1 หน้าการตรวจจับใบหน้า (Face Detection) หน้านี้จะเป็นการทดสอบการตรวจจับใบหน้าทั้งจากรูปภาพและแบบเรียลไทม์ (Real-time) ดังภาพที่ ค-16



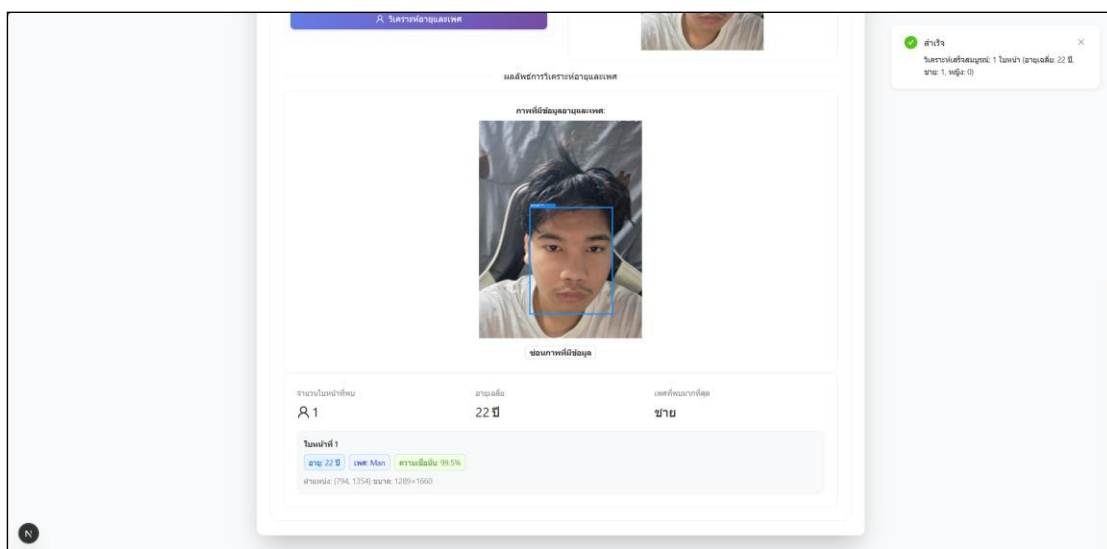
ภาพที่ ค-16 หน้าจอการตรวจจับใบหน้า (Face Detection)

ค.4.2 หน้าการตรวจจับการปลอมแปลง (Anti-Spoofing) หน้านี้จะเป็นการตรวจสอบว่าใบหน้าในรูปภาพเป็นของจริงหรือปลอม/สปูฟ ทั้งจากรูปภาพและแบบเรียลไทม์ (Real-time) ดังภาพที่ ค-17



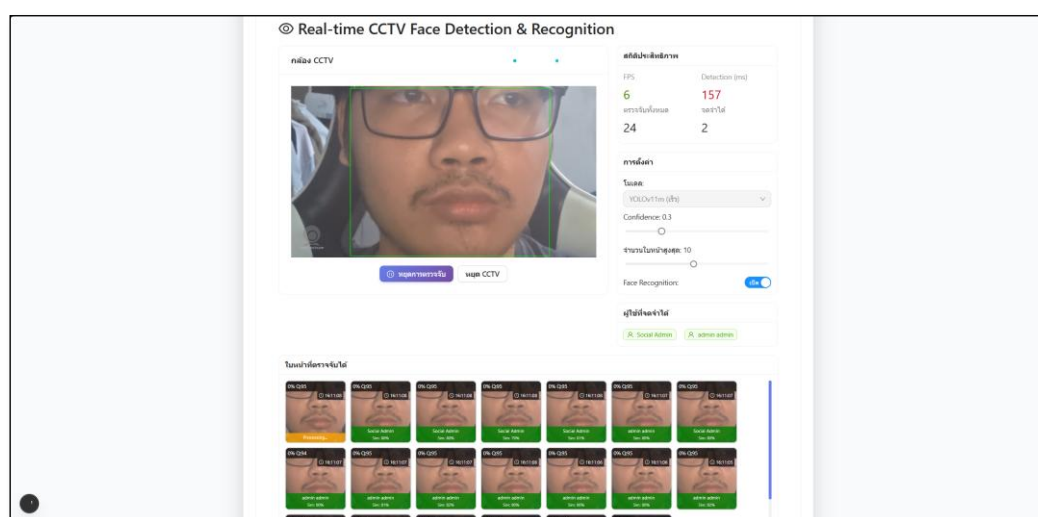
ภาพที่ ค-17 หน้าจอการตรวจจับการปลอมแปลง (Anti-Spoofing)

ค.4.3 หน้าการวิเคราะห์อายุและเพศ (Age and Gender Analysis) หน้านี้จะเป็นการทดสอบวิเคราะห์อายุและเพศของใบหน้าในรูปภาพอย่างแม่นยำ ทั้งจากรูปภาพและแบบเรียลไทม์ (Real-time) ดังภาพที่ ค-18



ภาพที่ ค-18 หน้าจอการวิเคราะห์อายุและเพศ (Age and Gender Analysis)

ค.4.4 หน้าการตรวจจับกล้องวงจรปิดแบบเรียลไทม์ (CCTV Real-Time) หน้านี้เป็นระบบตรวจจับและจดจำใบหน้าแบบเรียลไทม์ (Real-Time) สำหรับกล้องวงจรปิด โดยใช้ YOLOv11m พร้อมการตรวจจับใบหน้าและจดจำบุคคลที่ลงทะเบียนไว้ในระบบแบบเรียลไทม์ ดังภาพที่ ค-19



ภาพที่ ค-19 หน้าจอการตรวจจับกล้องวงจรปิดแบบเรียลไทม์ (CCTV Real-Time)

ภาพผนวก ง
ผลทดสอบ (Test case)

ผลการทดสอบกรณีทดสอบ (Test case)

ง.1 การทดสอบการยอมรับของผู้ใช้

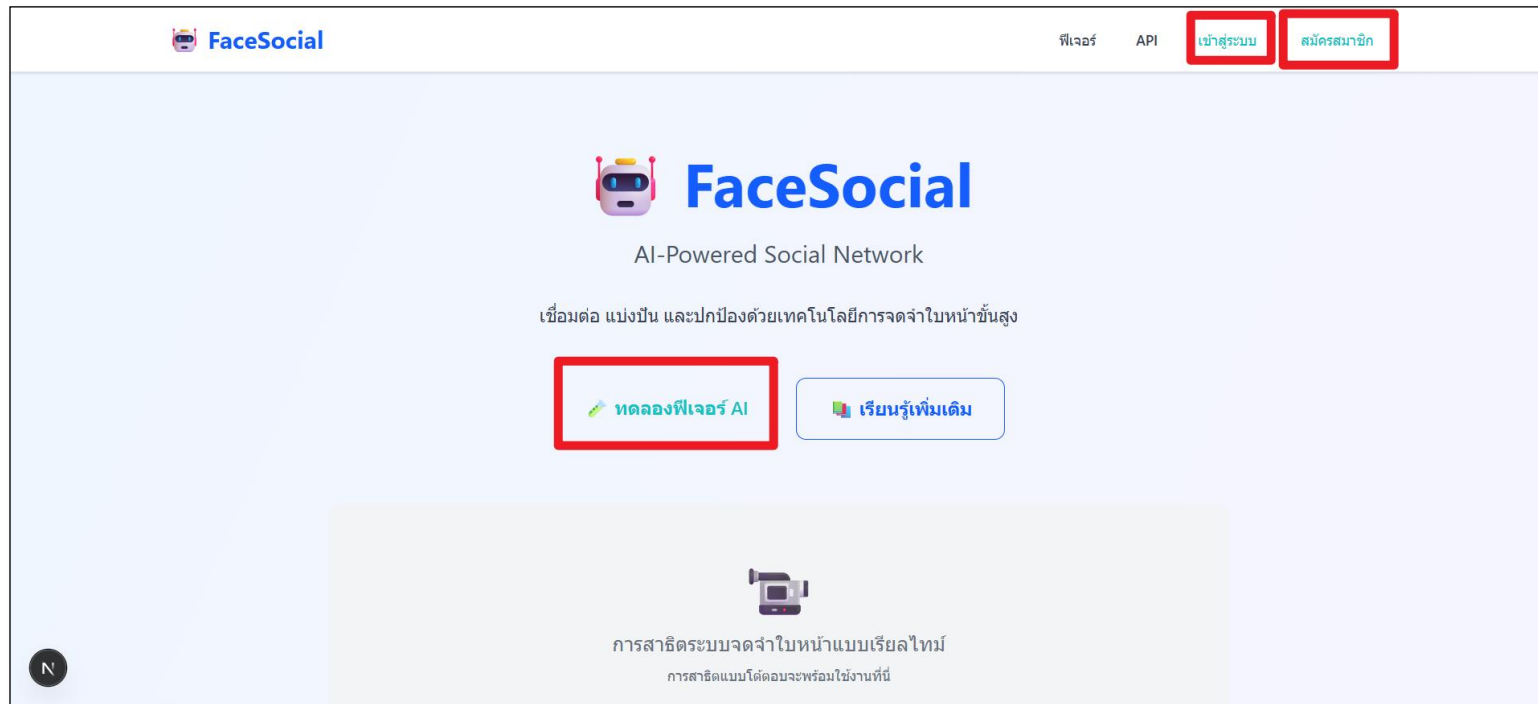
ง.1.1 การทดสอบการยอมรับของผู้ใช้ (User Acceptance Test: UAT) คือกระบวนการทดสอบใช้งานซอฟต์แวร์โดยผู้ใช้งานจริง (End Users) ซึ่ง UAT เป็นกระบวนการตรวจสอบขั้นสุดท้ายก่อนปล่อยระบบออกไปใช้งาน เพื่อให้แน่ใจว่าระบบที่พัฒนานั้นตรงตามความต้องการของธุรกิจและผู้ใช้งาน

ง.1.1.1 การทำ UAT ในส่วนของหน้าแรกที่ยังไม่มีการเข้าสู่ระบบ ดังตารางที่ ง-1

ตารางที่ ง-1 หน้าแรกที่ยังไม่มีการเข้าสู่ระบบ

No.	Feature	Test case ID	Scenario	Test step	Expected Result	Test Result	Test Status	Test
1	หน้าแรกยังไม่เข้าสู่ระบบ	1.1	การเข้าสู่ระบบ	คลิกปุ่ม "เข้าสู่ระบบ"	ไปหน้าเข้าสู่ระบบ	สามารถไปหน้าเข้าสู่ระบบได้ ดังภาพที่ ง-2	Pass	Suwit
		1.2	การสมัครสมาชิก	คลิกปุ่ม "สมัครสมาชิก"	ไปหน้าสมัครสมาชิก	สามารถไปหน้าสมัครสมาชิกได้ ดังภาพที่ ง-3	Pass	Suwit
		1.3	การทดลองฟีเจอร์ AI	คลิกปุ่ม "ทดลองฟีเจอร์ AI"	ไปหน้าทดลองฟีเจอร์ AI	สามารถไปหน้าทดลองฟีเจอร์ AI ได้ ดังภาพที่ ง-4	Pass	Suwit

ง.1.1.1.1 หน้าแรกยังไม่เข้าสู่ระบบ ดังภาพที่ ง-1



ภาพที่ ง-1 หน้าแรกยังไม่ได้เข้าสู่ระบบ

ง.1.1.1.2 ผลการทดสอบ Test Case ID 1.1 กดปุ่มเข้าสู่ระบบแล้วจะแสดงหน้าเข้าสู่ระบบ ดังภาพที่ ง-2

ภาพที่ ง-2 เข้าสู่ระบบ

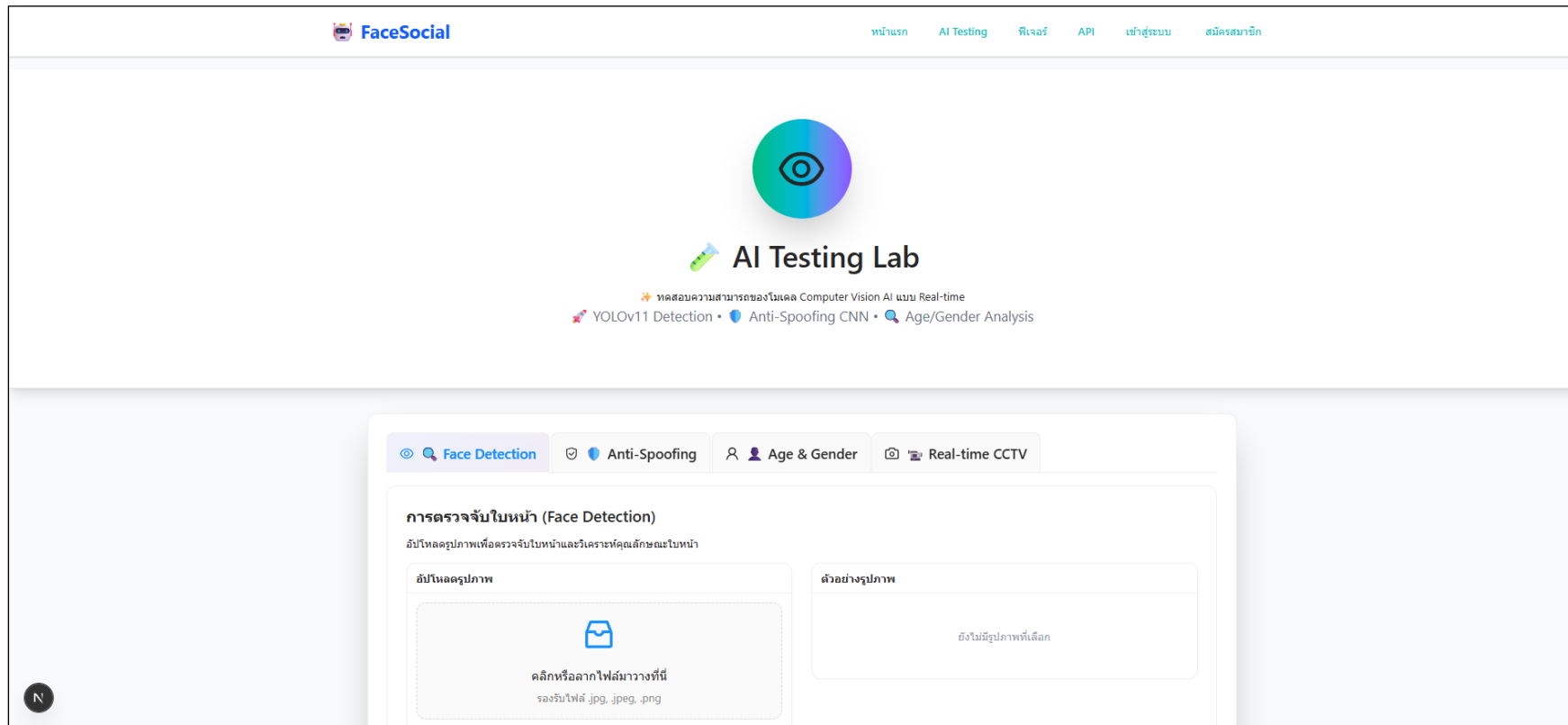
ง.1.1.1.3 ผลการทดสอบ Test Case ID 1.2 กดปุ่มสมัครสมาชิกแล้วจะแสดงหน้าสมัครสมาชิก ดังภาพที่ ง-3

The screenshot shows a web registration form with the following elements:

- Header:** Three status indicators at the top: a person icon for 'ข้อมูลส่วนตัว' (Personal Information), a camera icon for 'สแกนใบหน้า' (Face Scan), and a checkmark icon for 'เสร็จสิ้น' (Completed).
- Title:** 'ข้อมูลส่วนตัว' (Personal Information) with a subtitle 'กรอกข้อมูลพื้นฐานของคุณเพื่อสร้างบัญชี' (Enter your basic information to create an account).
- Fields:**
 - * ชื่อ (First Name): Text input with placeholder 'ชื่อ'.
 - * นามสกุล (Last Name): Text input with placeholder 'นามสกุล'.
 - * อีเมล (Email): Text input with placeholder 'example@email.com'.
 - * ชื่อผู้ใช้ (Username): Text input with placeholder 'username'.
 - * รหัสผ่าน (Password): Password input with placeholder 'รหัสผ่าน'.
 - * ยืนยันรหัสผ่าน (Confirm Password): Password input with placeholder 'ยืนยันรหัสผ่าน'.
 - เบอร์โทรศัพท์ (ไม่บังคับ) (Phone Number - optional): Text input with placeholder '08X-XXX-XXXX'.
 - วันเกิด (ไม่บังคับ) (Date of Birth - optional): Date picker with placeholder 'เลือกวันเกิด'.
- Footer:** A checkbox for 'ฉันยอมรับ ข้อตกลงการใช้งาน และ นโยบายความเป็นส่วนตัว' (I accept the Terms of Use and Privacy Policy) and a blue button labeled 'ดำเนินการต่อ' (Continue).

ภาพที่ ง-3 หน้าสมัครสมาชิก

ง.1.1.1.4 ผลการทดสอบ Test Case ID 1.3 กดปุ่มทดลองฟีเจอร์ AI แล้วจะแสดงหน้าทดลองฟีเจอร์ AI ดังภาพที่ ง-4



ภาพที่ ง-4 หน้าทดลองฟีเจอร์ AI

ง.1.1.2 การทำ UAT ในส่วนของหน้าเข้าสู่ระบบ

ตารางที่ ง-2 เข้าสู่ระบบ

No.	Feature	Test case ID	Scenario	Test step	Expected Result	Test Result	Test Status	Test
2	หน้าเข้าสู่ระบบ	2.1	ตรวจสอบเมื่อกรอกชื่อผู้ใช้และรหัสผ่านถูกต้อง	กรอกข้อมูลที่ต้องการแล้วคลิกปุ่ม "เข้าสู่ระบบ"	ไปที่หน้า Dashboardของผู้ใช้	สามารถไปหน้า Dashboardของผู้ใช้ได้ดังภาพที่ ง-6	Pass	Suwit
		2.2	ตรวจสอบเมื่อกรอกชื่อผู้ใช้หรือรหัสผ่านผิด	กรอกข้อมูลที่ต้องการแล้วคลิกปุ่ม "เข้าสู่ระบบ"	แจ้งเตือน "เกิดข้อผิดพลาด ไม่พบผู้ใช้งาน"	สามารถแจ้งเตือน "เกิดข้อผิดพลาด ไม่พบผู้ใช้งาน" ได้ดังภาพที่ ง-7	Pass	Suwit
		2.3	ตรวจสอบเมื่อไม่กรอกชื่อผู้ใช้หรือรหัสผ่าน	ไม่กรอกข้อมูลแล้วคลิกปุ่ม "เข้าสู่ระบบ"	แจ้งเตือน "กรุณากรอกอีเมลหรือชื่อผู้ใช้" และ "กรุณากรอกรหัสผ่าน"	สามารถแจ้งเตือน "กรุณากรอกอีเมลหรือชื่อผู้ใช้" และ "กรุณากรอกรหัสผ่าน" ได้ดังภาพที่ ง-8	Pass	Suwit

ตารางที่ ง-2 เข้าสู่ระบบ (ต่อ)

No.	Feature	Test case ID	Scenario	Test step	Expected Result	Test Result	Tesg Staus	Test
		2.4	เข้าสู่ระบบด้วย ใบหน้า (Advanced)	คลิกปุ่ม "เข้าสู่ ระบบด้วย ใบหน้า (Advanced)"	เปิดหน้าต่างกล่อง และระบบสแกน หน้าได้	สามารถเปิดหน้าต่าง กล่องและระบบสแกน หน้าได้ ดังภาพที่ ง-9	Pass	Suwit

ง.1.1.2.1 หน้าเข้าสู่ระบบ ดังภาพที่ ง-5

FaceSocial

หน้าแรก สมัครสมาชิก

🔄

เข้าสู่ระบบ

เข้าสู่ระบบด้วยอีเมลและรหัสผ่าน หรือ ใบหน้า

1. อีเมลหรือชื่อผู้ใช้

✉ อีเมลหรือชื่อผู้ใช้

รหัสผ่าน

🔒 รหัสผ่าน

🔄

🔄 เข้าสู่ระบบด้วยรหัสผ่าน

หรือ

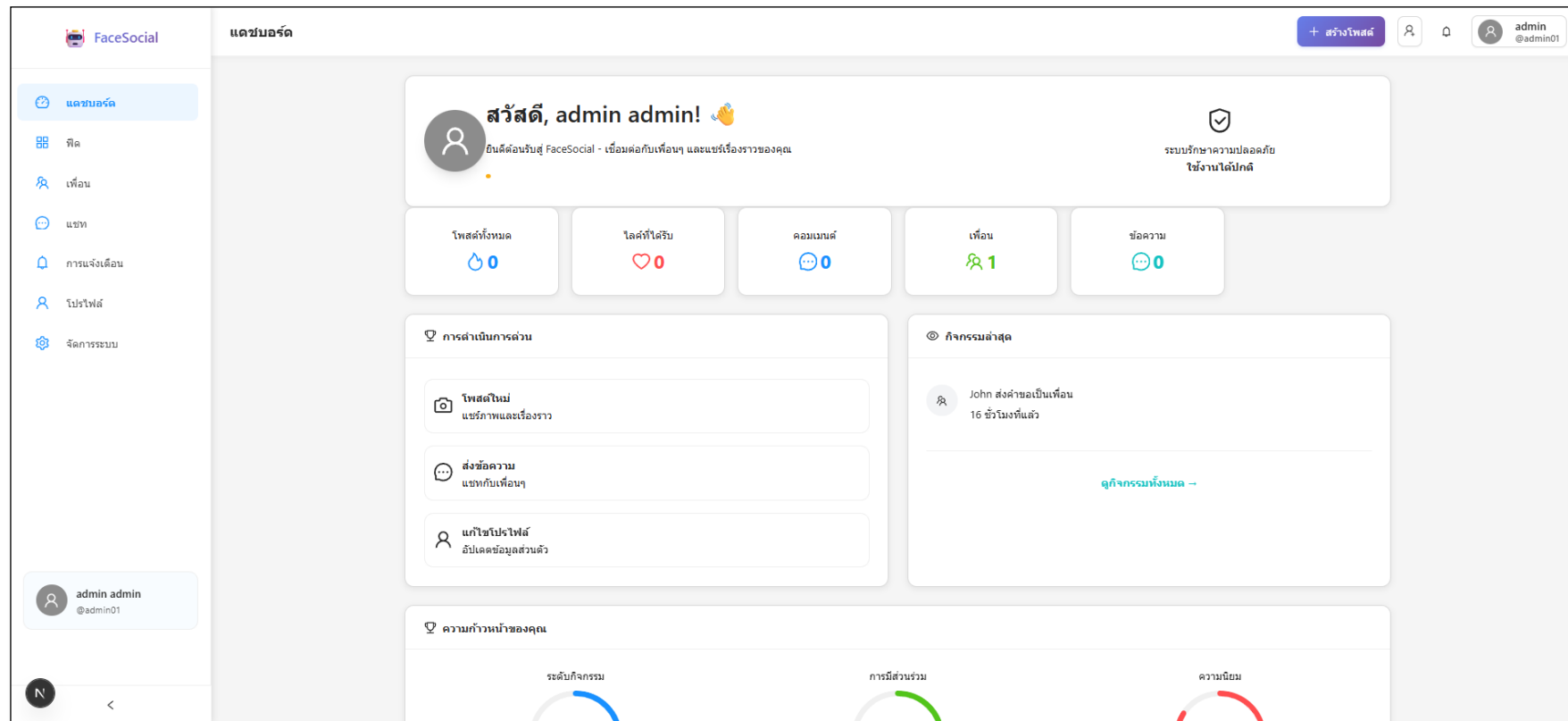
🔄 เข้าสู่ระบบด้วยใบหน้า (Advanced)

ยังไม่มีบัญชี? สมัครสมาชิก

🔄 ลืมรหัสผ่าน?

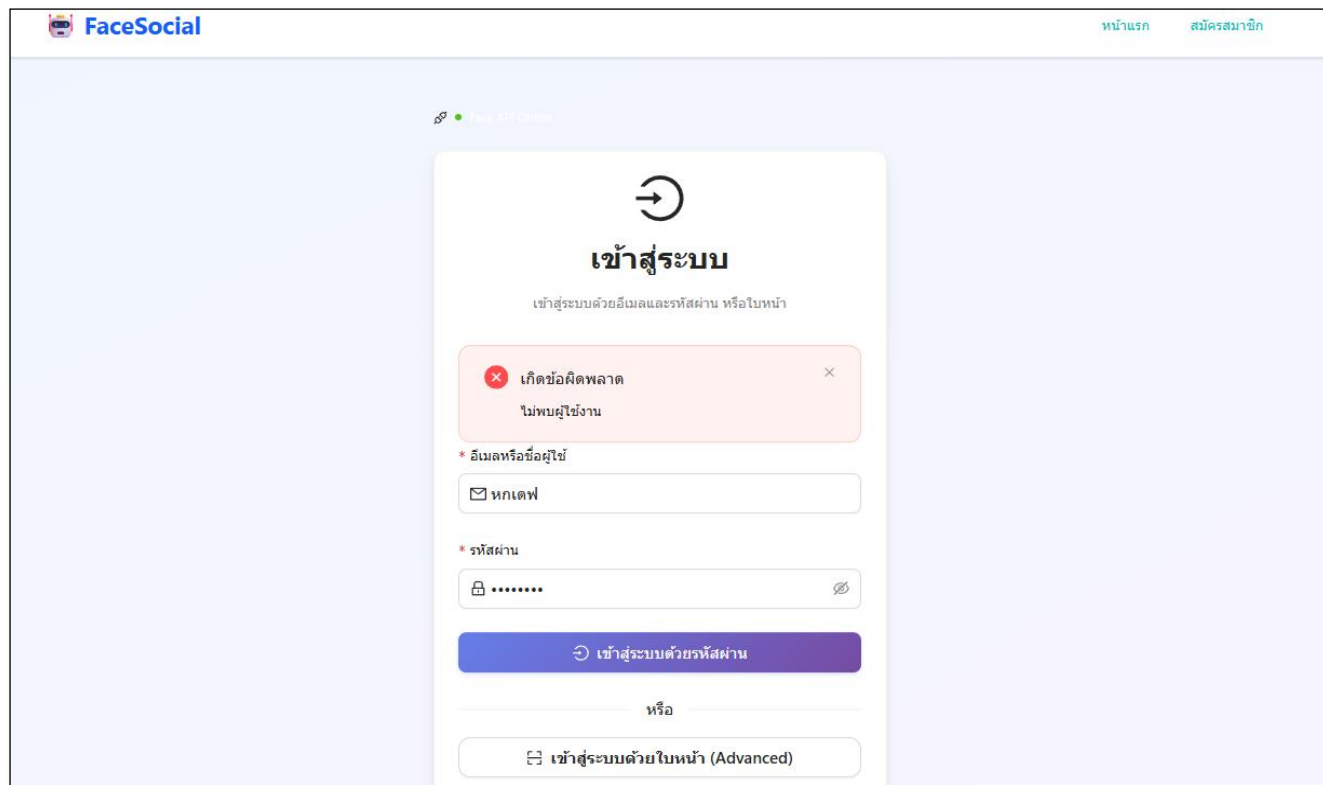
ภาพที่ ง-5 หน้าเข้าสู่ระบบ

ง.1.1.2.2 ผลการทดสอบ Test Case ID 2.1 หน้าเข้าสู่ระบบที่ใส่ชื่อผู้ใช้และรหัสผ่านถูกต้องและไปหน้า Dashboardของผู้ใช้ ดังภาพที่ ง-6



ภาพที่ ง-6 หน้า Dashboardของผู้ใช้

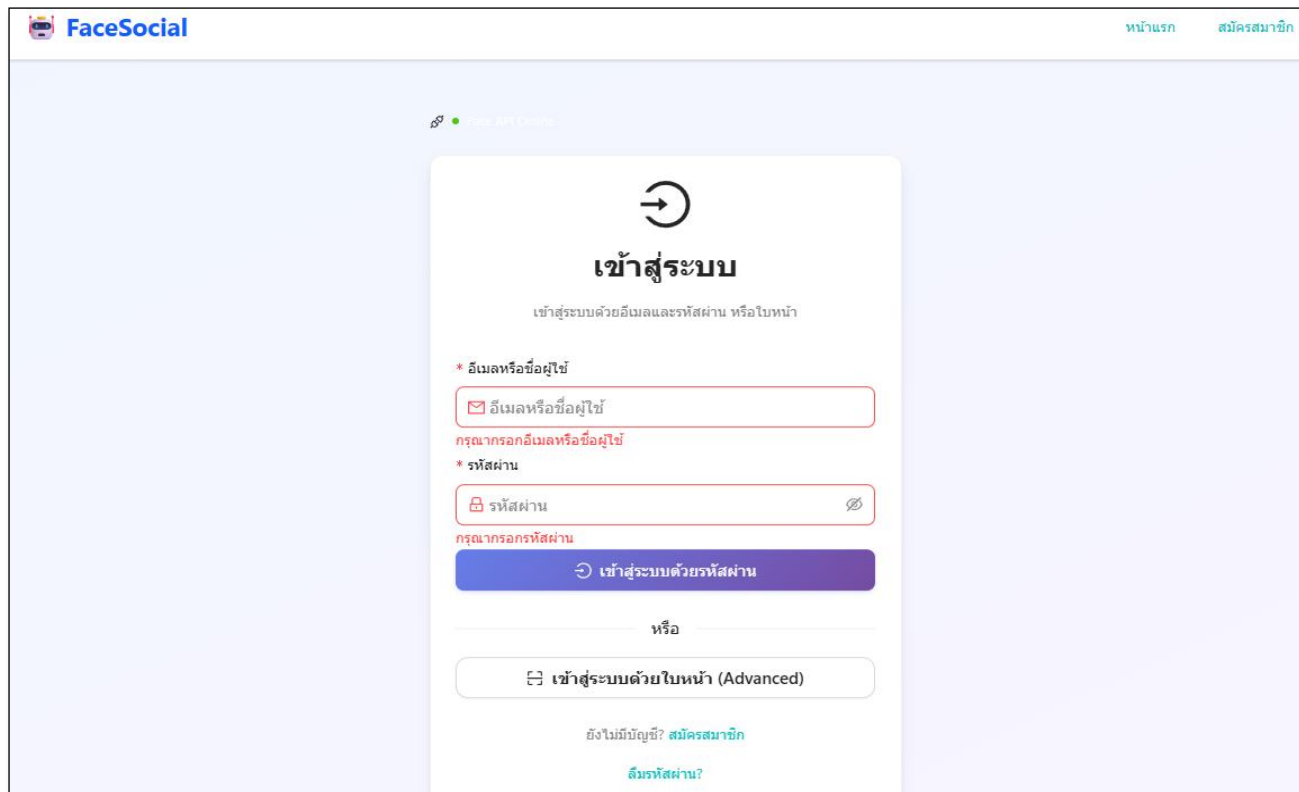
ง.1.1.2.3 ผลการทดสอบ Test Case ID 2.2 หน้าเข้าสู่ระบบที่กรอกชื่อผู้ใช้หรือรหัสผ่านผิด ได้รับการแจ้งเตือน "เกิดข้อผิดพลาด" และ "ไม่พบผู้ใช้งาน" ดังภาพที่ ง-7



The screenshot displays the FaceSocial login interface. At the top, the 'FaceSocial' logo is on the left, and 'หน้าแรก' (Home) and 'สมัครสมาชิก' (Sign Up) links are on the right. The main content area features a central white card with a circular arrow icon and the title 'เข้าสู่ระบบ' (Login). Below the title, it says 'เข้าสู่ระบบด้วยอีเมลและรหัสผ่าน หรือ ใบหน้า' (Login with email and password or face). A red error message box is visible, stating 'เกิดข้อผิดพลาด' (Error) and 'ไม่พบผู้ใช้งาน' (User not found). Below this, there are input fields for 'อีเมลหรือชื่อผู้ใช้' (Email or username) with a placeholder 'หกดเตฟ' and 'รหัสผ่าน' (Password) with a placeholder '.....'. A blue button labeled 'เข้าสู่ระบบด้วยรหัสผ่าน' (Login with password) is positioned below the password field. At the bottom of the card, there is a 'หรือ' (or) separator and a button labeled 'เข้าสู่ระบบด้วยใบหน้า (Advanced)' (Login with face (Advanced)).

ภาพที่ ง-7 หน้าเข้าสู่ระบบที่กรอกชื่อผู้ใช้หรือรหัสผ่านผิด

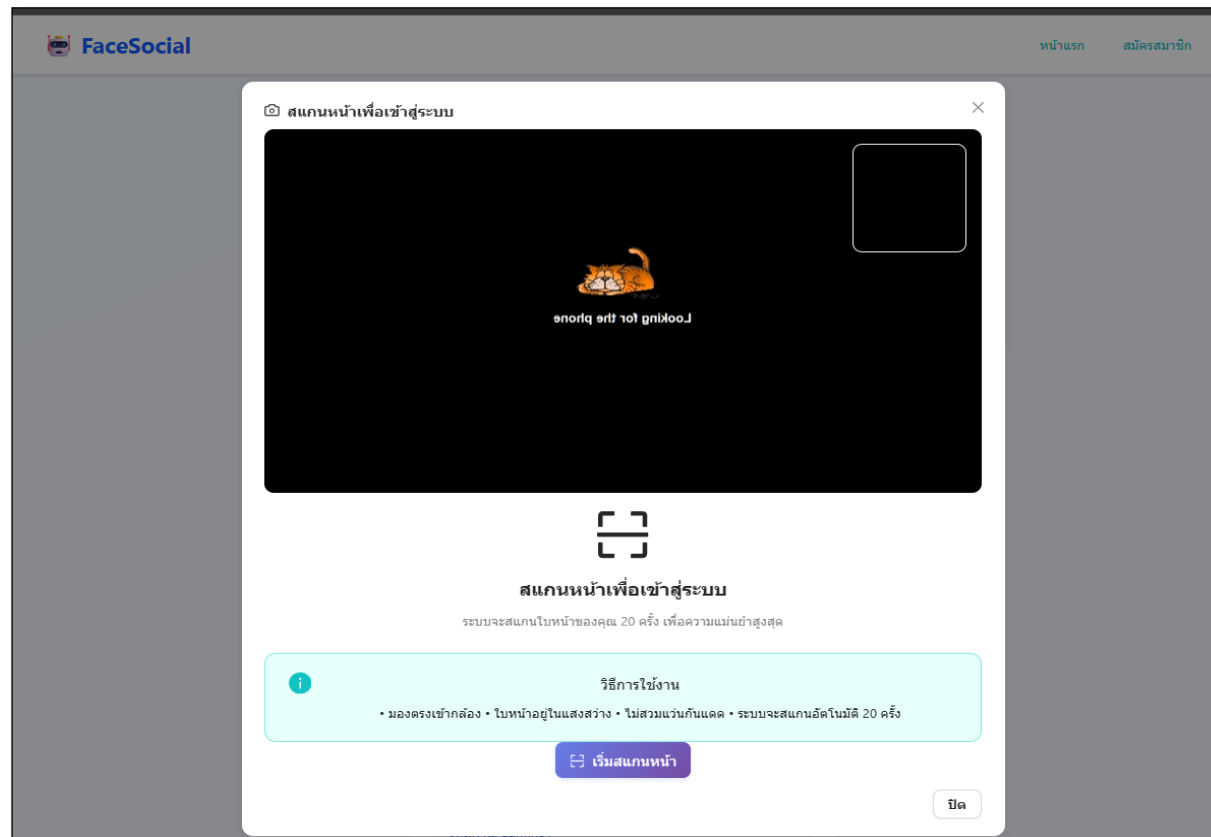
ง.1.1.2.4 ผลการทดสอบ Test Case ID 2.3 หน้าเข้าสู่ระบบที่ไม่กรอกชื่อผู้ใช้หรือรหัสผ่าน ได้รับการแจ้งเตือน "กรุณากรอกอีเมลหรือชื่อผู้ใช้" และ "กรุณากรอกรหัสผ่าน" ดังภาพที่ ง-8



The screenshot shows the FaceSocial login interface. At the top, there's a header with the FaceSocial logo and links for 'หน้าแรก' (Home) and 'สมัครสมาชิก' (Sign Up). The main content area features a central login card with a circular arrow icon and the title 'เข้าสู่ระบบ' (Login). Below the title, it says 'เข้าสู่ระบบด้วยอีเมลและรหัสผ่าน หรือ ใบหน้า' (Login with email and password or face). The card contains two input fields: one for 'อีเมลหรือชื่อผู้ใช้' (Email or Username) and another for 'รหัสผ่าน' (Password). Both fields have red borders and red error messages below them: 'กรุณากรอกอีเมลหรือชื่อผู้ใช้' (Please enter email or username) and 'กรุณากรอกรหัสผ่าน' (Please enter password). A blue button with a circular arrow icon and the text 'เข้าสู่ระบบด้วยรหัสผ่าน' (Login with password) is positioned below the password field. Below the button, there's a horizontal line with the word 'หรือ' (or) in the center. Underneath, there's a button with a face icon and the text 'เข้าสู่ระบบด้วยใบหน้า (Advanced)' (Login with face (Advanced)). At the bottom of the card, there are two links: 'ยังไม่มีบัญชี? สมัครสมาชิก' (Don't have an account? Sign up) and 'ลืมรหัสผ่าน?' (Forgot password?).

ภาพที่ ง-8 หน้าเข้าสู่ระบบที่ไม่กรอกชื่อผู้ใช้หรือรหัสผ่าน

ง.1.1.2.5 ผลการทดสอบ Test Case ID 2.4 หน้าเข้าสู่ระบบที่กดปุ่ม "เข้าสู่ระบบด้วยใบหน้า (Advanced)" ดังภาพที่ ง-9



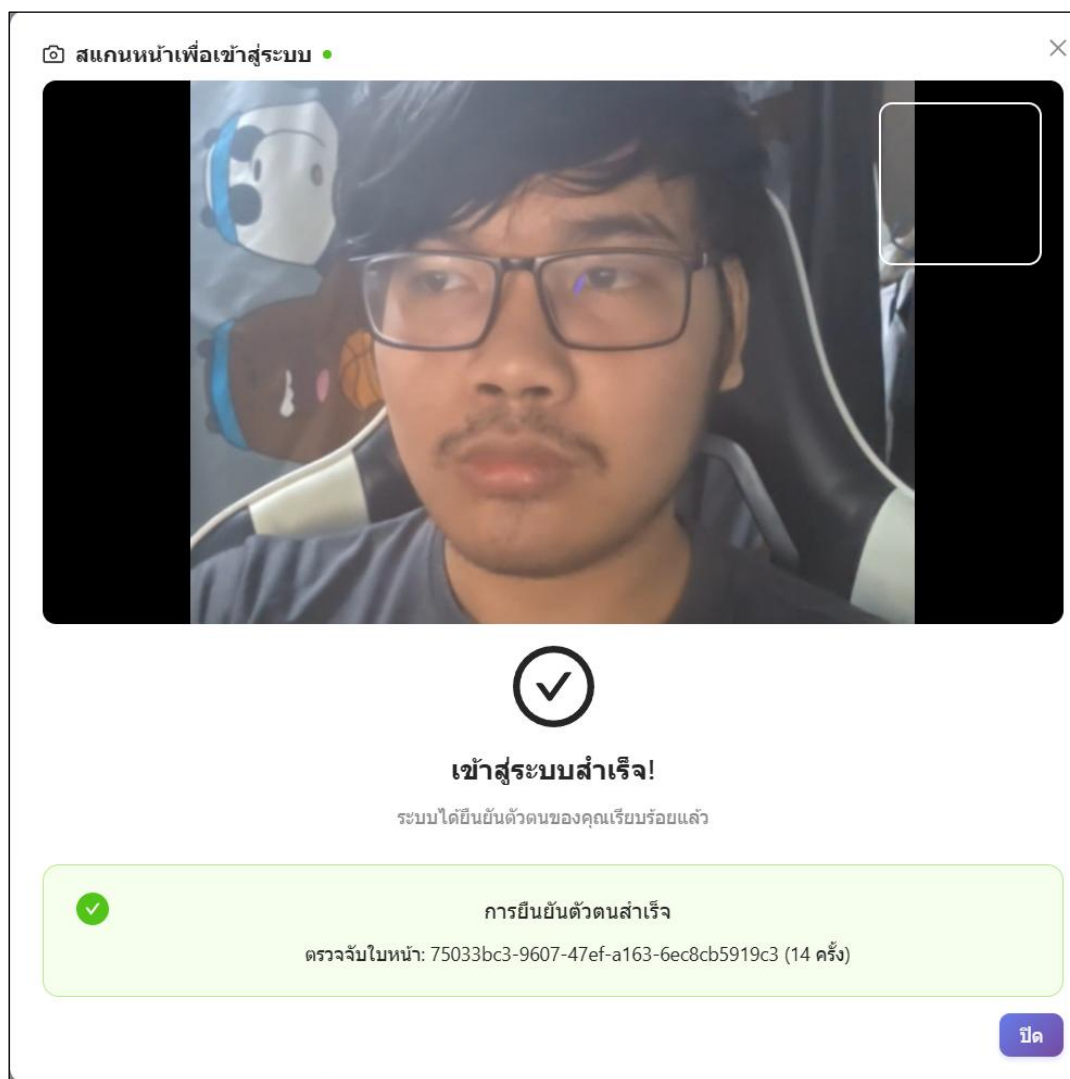
ภาพที่ ง-9 หน้าต่างสแกนใบหน้าเพื่อเข้าสู่ระบบ

ง.1.1.3 การทำ UAT ในส่วนของหน้าเข้าสู่ระบบด้วยใบหน้า

ตารางที่ ง-3 เข้าสู่ระบบด้วยใบหน้า

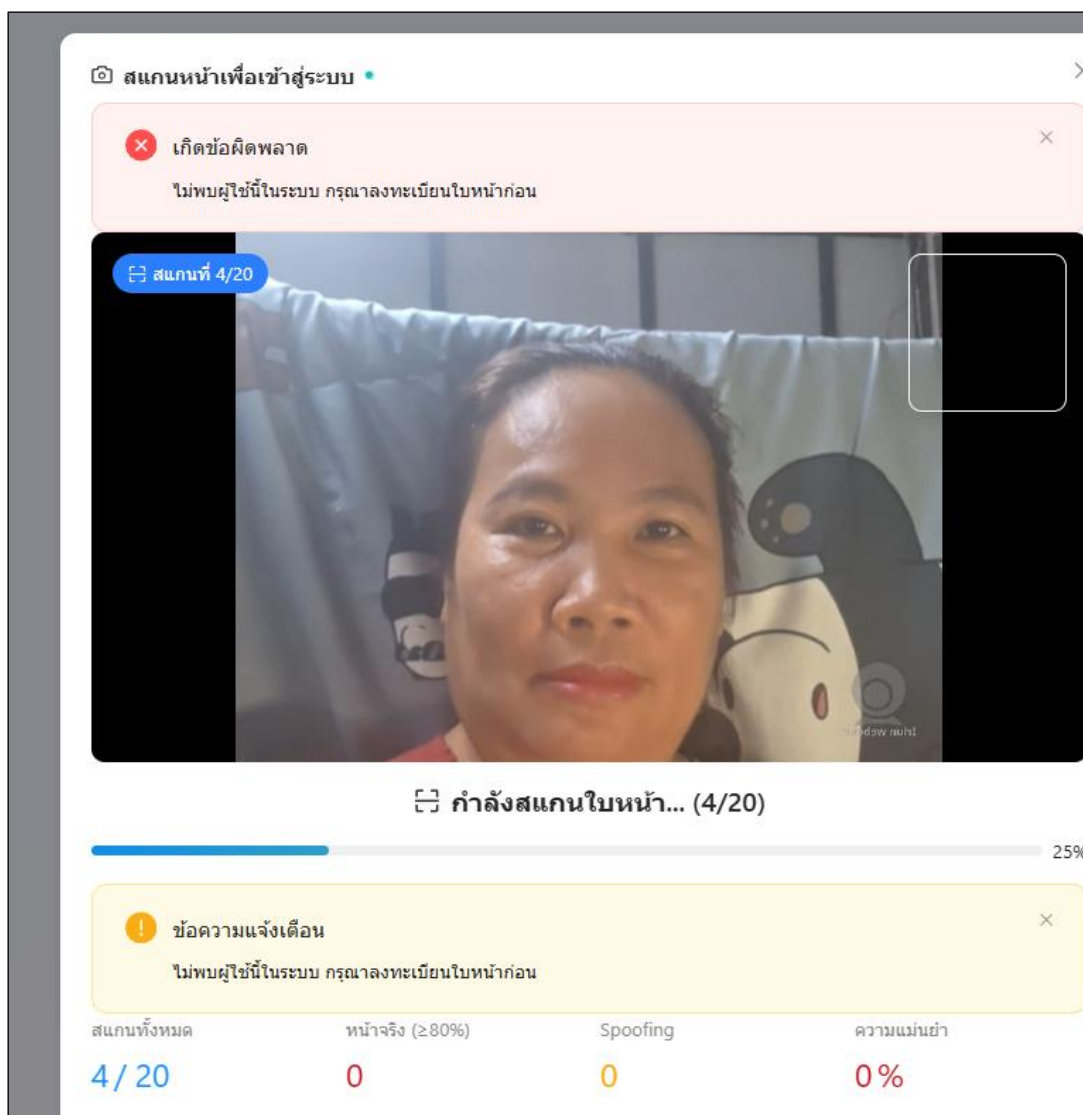
No.	Feature	Test case ID	Scenario	Test step	Expected Result	Test Result	Tesg Staus	Test
3	หน้าเข้าสู่ระบบ	3.1	ตรวจสอบใบหน้าผู้ใช้ที่ลงทะเบียนไว้	กรอกข้อมูลที่ถูกต้องแล้วคลิกปุ่ม "เข้าสู่ระบบ"	แจ้งเตือน "เข้าสู่ระบบสำเร็จ"	สามารถแจ้งเตือน "เข้าสู่ระบบสำเร็จ" ได้ดังภาพที่ ง-10	Pass	Suwit
		3.2	ตรวจสอบใบหน้าผู้ใช้ที่ไม่ได้ลงทะเบียนไว้	กรอกข้อมูลที่ผิดแล้วคลิกปุ่ม "เข้าสู่ระบบ"	แจ้งเตือน "เกิดข้อผิดพลาด ไม่พบผู้ใช้งาน"	สามารถแจ้งเตือน "เกิดข้อผิดพลาด ไม่พบผู้ใช้งาน" ได้ดังภาพที่ ง-11	Pass	Suwit
		3.3	ตรวจสอบเมื่อไม่กรอกชื่อผู้ใช้หรือรหัสผ่าน	ตรวจสอบใบหน้าไม่ใช่ภาพจริง (Anti-Spoofing)	แจ้งเตือน "ตรวจพบการปลอมแปลงใบหน้า กรุณาแสดงหน้าจริงของคุณ ไม่ใช่รูปภาพหรือหน้าจอ"	สามารถแจ้งเตือน "ตรวจพบการปลอมแปลงใบหน้า กรุณาแสดงหน้าจริงของคุณ ไม่ใช่รูปภาพหรือหน้าจอ" ได้ดังภาพที่ ง-12	Pass	Suwit

ง.1.1.3.1 ผลการทดสอบ Test Case ID 3.1 หน้าต่างสแกนหน้าตรวจสอบใบหน้าผู้ใช้ที่ลงทะเบียนไว้ เข้าสู่ระบบด้วยใบหน้า (Advanced) ดังภาพที่ ง-10



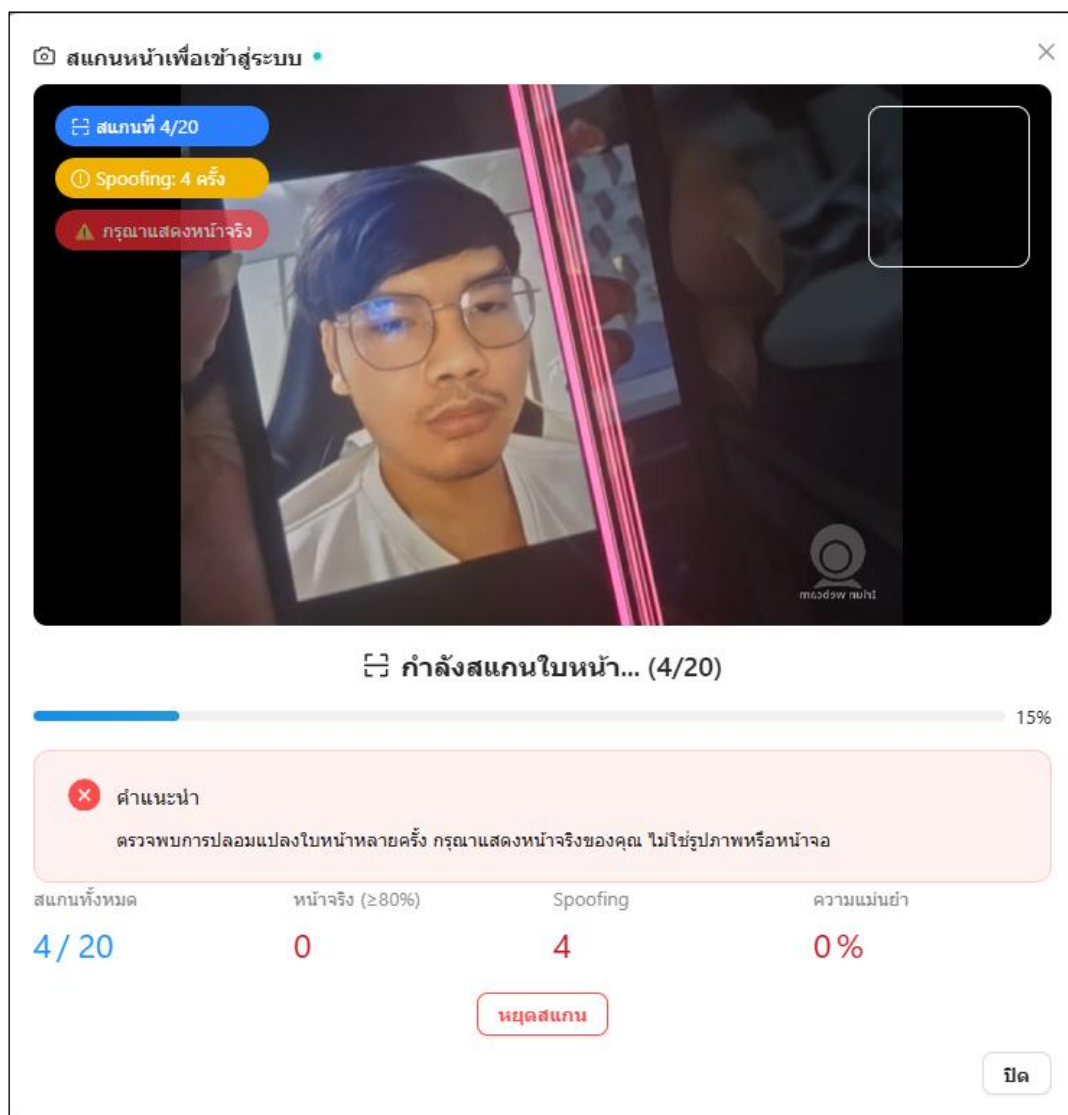
ภาพที่ ง-10 หน้าต่างสแกนใบหน้าผู้ใช้ที่ลงทะเบียนไว้เพื่อเข้าสู่ระบบ

ง.1.1.3.2 ผลการทดสอบ Test Case ID 3.2 หน้าต่างสแกนหน้าตรวจสอบใบหน้าผู้ใช้ที่ไม่ได้ลงทะเบียนไว้ เข้าสู่ระบบด้วยใบหน้า (Advanced) ดังภาพที่ ง-11



ภาพที่ ง-11 หน้าต่างสแกนใบหน้าผู้ใช้ที่ไม่ได้ลงทะเบียนไว้

ง.1.1.3.3 ผลการทดสอบ Test Case ID 3.3 หน้าต่างสแกนหน้าตรวจสอบการป้องกันภาพปลอม (Anti-Spoofing) ดังภาพที่ ง-12



ภาพที่ ง-12 หน้าต่างสแกนหน้าตรวจสอบการป้องกันภาพปลอม

ง.1.1.4 การทำ UAT ในส่วนของหน้าสมัครสมาชิก

ตารางที่ ง-4 หน้าสมัครสมาชิก

No.	Feature	Test case ID	Scenario	Test step	Expected Result	Test Result	Tesg Staus	Test
4	หน้าสมัครสมาชิก	4.1	ตรวจสอบระบบเปรียบเทียบกับฐานข้อมูล (ข้อมูลซ้ำ)	กรอกข้อมูลที่มีในระบบแล้วคลิกปุ่ม "ดำเนินการต่อ"	แจ้งเตือน "มีข้อมูลในระบบแล้ว"	สามารถแจ้งเตือน "มีข้อมูลในระบบแล้ว" ได้ดังภาพที่ ง-13	Pass	Suwit
		4.2	ตรวจสอบการใส่ข้อมูลไม่ตรงกับรูปแบบที่กำหนด	กรอกข้อมูลไม่ถูกรูปแบบแล้วคลิกปุ่ม "ดำเนินการต่อ"	แจ้งเตือน "ข้อมูลไม่ตรงกับรูปแบบที่กำหนดไว้"	สามารถแจ้งเตือน "ข้อมูลไม่ตรงกับรูปแบบที่กำหนดไว้" ได้ดังภาพที่ ง-14	Pass	Suwit
		4.3	ตรวจสอบเมื่อไม่กรอกข้อมูล	ไม่กรอกข้อมูลแล้วคลิกปุ่ม "ดำเนินการต่อ"	แจ้งเตือน "กรุณากรอกข้อมูล"	สามารถแจ้งเตือน "กรุณากรอกข้อมูล" ได้ดังภาพที่ ง-15	Pass	Suwit
		4.4	ตรวจสอบเมื่อกรอกข้อมูลรูปแบบที่ถูกต้องและไม่ซ้ำกับฐานข้อมูล	กรอกข้อมูลถูกต้องแล้วคลิกปุ่ม "ดำเนินการต่อ"	ไปหน้าสแกนใบหน้าเพื่อความปลอดภัย	สามารถไปหน้าสแกนใบหน้าเพื่อความปลอดภัย ได้ดังภาพที่ ง-16	Pass	Suwit

จ.1.1.4.1 ผลการทดสอบ Test Case ID 4.1 ตรวจสอบระบบเปรียบเทียบกับฐานข้อมูล
 ดังภาพที่ จ-13

ข้อมูลส่วนตัว
 กรอกข้อมูลพื้นฐาน

สแกนใบหน้า
 ลงทะเบียนใบหน้าเพื่อ
 ความปลอดภัย

เสร็จสิ้น
 สมัครสมาชิกสำเร็จ

ข้อมูลส่วนตัว

กรอกข้อมูลพื้นฐานของคุณเพื่อสร้างบัญชี

!

พบข้อมูลที่ไม่พร้อมใช้งาน
 กรุณาแก้ไขข้อมูลที่ทำเครื่องหมายสีแดงก่อนดำเนินการต่อ

* ชื่อ

admin

* นามสกุล

admin

* อีเมล

admin@email.com

✓

อีเมลพร้อมใช้งาน

* ชื่อผู้ใช้

admin01

✗

ชื่อผู้ใช้นี้ถูกใช้งานแล้ว

* รหัสผ่าน

.....

🗑

* ยืนยันรหัสผ่าน

.....

🗑

เบอร์โทรศัพท์ (ไม่บังคับ)

08X-XXX-XXXX

วันเกิด (ไม่บังคับ)

เลือกวันเกิด

📅

✓

ฉันยอมรับ

ข้อตกลงการใช้งาน และ นโยบายความเป็นส่วนตัว

ภาพที่ จ-13 หน้าจอแจ้งเตือนเมื่อมีข้อมูลซ้ำกับฐานข้อมูล (Database)

ง.1.1.4.2 ผลการทดสอบ Test Case ID 4.2 ตรวจสอบการใส่ข้อมูลไม่ตรงกับรูปแบบที่กำหนด ดังภาพที่ ง-14

The screenshot shows the FaceSocial registration page. At the top, there are three steps: 'ข้อมูลส่วนตัว' (Personal Information), 'สแกนใบหน้า' (Face Scan), and 'เสร็จสิ้น' (Completed). The 'ข้อมูลส่วนตัว' step is active, showing a progress bar and a message: 'กรอกข้อมูลพื้นฐานของคุณเพื่อสร้างบัญชี' (Fill in your basic information to create an account).

Below the progress bar, there is a yellow warning box with an exclamation mark icon and the text: 'พบข้อมูลที่ไม่พร้อมใช้งาน กรุณาแก้ไขข้อมูลที่ทำเครื่องหมายสีแดงก่อนดำเนินการต่อ' (Found unusable information. Please fix the information marked in red before proceeding).

The form fields and their validation errors are as follows:

- * ชื่อ (First Name):** Field contains 'ด' (Da). Error: 'ชื่อต้องมีอย่างน้อย 2 ตัวอักษร' (Name must be at least 2 characters).
- * นามสกุล (Last Name):** Field contains 'ด' (Da). Error: 'นามสกุลต้องมีอย่างน้อย 2 ตัวอักษร' (Last name must be at least 2 characters).
- * อีเมล (Email):** Field contains 'ฟัดกดฟแพ่หเก' (faddkdfpaehge). Error: 'รูปแบบอีเมลไม่ถูกต้อง' (Email format is incorrect).
- * ชื่อผู้ใช้ (Username):** Field contains 'ฟกเฟกเกฟ' (fkgfekgef). Error: 'ชื่อผู้ใช้ต้องมีอย่างน้อย 3 ตัวอักษร และใช้ได้เฉพาะ a-z, A-Z, 0-9, _' (Username must be at least 3 characters and only a-z, A-Z, 0-9, _ are allowed).
- * รหัสผ่าน (Password):** Field contains '.....'. Error: 'รหัสผ่านไม่ตรงกัน' (Passwords do not match).
- * ยืนยันรหัสผ่าน (Confirm Password):** Field contains '.....'.
- เบอร์โทรศัพท์ (ไม่บังคับ) (Phone Number):** Field contains '08X-XXX-XXXX'.
- วันเกิด (ไม่บังคับ) (Date of Birth):** Field contains 'เลือกวันเกิด' (Select date of birth).
- ฉันยอมรับ ข้อตกลงการใช้งาน และ นโยบายความเป็นส่วนตัว (I agree to the Terms of Service and Privacy Policy):** Checked.

At the bottom, there is a blue button labeled '> ดำเนินการต่อ' (Next) and a link 'มีบัญชีอยู่แล้ว? เข้าสู่ระบบ' (Already have an account? Log in).

ภาพที่ ง-14 หน้าจอแจ้งเตือนเมื่อกรอกข้อมูลไม่ตรงกับรูปแบบที่กำหนดไว้

ง.1.1.4.3 ผลการทดสอบ Test Case ID 4.3 ตรวจสอบเมื่อไม่กรอกข้อมูล ดังภาพที่ ง-15

The screenshot shows the FaceSocial registration page. At the top, there are three progress indicators: 'ข้อมูลส่วนตัว' (Personal Information), 'สแกนใบหน้า' (Face Scan), and 'เสร็จสิ้น' (Completed). The 'ข้อมูลส่วนตัว' indicator is active, and the form below it contains several fields with red error messages indicating missing information.

ข้อมูลส่วนตัว
กรอกข้อมูลพื้นฐานของคุณเพื่อสร้างบัญชี

ชื่อ * ชื่อ กรุณากรอกชื่อ

นามสกุล * นามสกุล กรุณากรอกนามสกุล

อีเมล * กรุณากรอกอีเมล

ชื่อผู้ใช้ * กรุณากรอกชื่อผู้ใช้

รหัสผ่าน * รหัสผ่าน กรุณากรอกรหัสผ่าน

ยืนยันรหัสผ่าน * ยืนยันรหัสผ่าน กรุณายืนยันรหัสผ่าน

เบอร์โทรศัพท์ (ไม่บังคับ)

วันเกิด (ไม่บังคับ)

☒ ฉันยอมรับ [ข้อตกลงการใช้งาน](#) และ [นโยบายความเป็นส่วนตัว](#)

[→ ดำเนินการต่อ](#)

มีปัญหาอยู่แล้ว? [เข้าสู่ระบบ](#)

ภาพที่ ง-15 หน้าจอแจ้งเตือนเมื่อไม่กรอกข้อมูล

ง.1.1.4.4 ผลการทดสอบ Test Case ID 4.4 ตรวจสอบเมื่อกรอกข้อมูลรูปแบบที่ถูกต้องและไม่ซ้ำกับฐานข้อมูล ดังภาพที่ ง-16

The screenshot displays the 'FaceSocial' web application interface. At the top, there's a navigation bar with the logo and links for 'หน้าแรก' (Home) and 'เข้าสู่ระบบ' (Login). Below this, a progress bar shows three steps: 'ข้อมูลส่วนตัว' (Personal Information), 'สแกนใบหน้า' (Face Scanning), and 'เสร็จสิ้น' (Completed). The current step is 'สแกนใบหน้า', which includes the subtext 'ลงทะเบียนใบหน้าเพื่อลงทะเบียนบัญชี' (Register face for account registration).

The main content area is titled 'สแกนใบหน้าเพื่อความปลอดภัย' (Face Scanning for Security) with a subtitle 'กรุณาให้ใบหน้าอยู่ในกรอบและมองตรงเข้ากล้อง' (Please keep your face in the frame and look straight at the camera). It features a 'Face Recognition API Status' section showing 'Face Detection', 'Face Recognition', 'Face Analysis', and 'GPU Manager' all as active (green dots). Below this is a camera icon and the same title 'สแกนใบหน้าเพื่อความปลอดภัย' with the subtitle 'ระบบจะใช้ AI เพื่อตรวจจับใบหน้าจริง ตรวจสอบคุณภาพ และสร้าง Face Embedding เพื่อความปลอดภัย' (The system will use AI to detect real faces, check quality, and create Face Embedding for security).

A table-like section details the scanning process:

ขั้นตอนการสแกน (Scanning Steps)	เคล็ดลับ (Tips)
Face Detection: ตรวจสอบใบหน้าและตัดภาพ (Check face and crop image) Anti-Spoofing: ตรวจสอบใบหน้าจริง (Check real face) Quality Check: ประเมินคุณภาพภาพ (Evaluate image quality) Embedding: สร้าง Face Embedding (Create Face Embedding)	- ใบหน้าควรอยู่ในแสงสว่างเพียงพอ (Face should be in sufficient light) - มองตรงเข้ากล้อง หลีกเลี่ยงการเอียง >10° (Look straight at camera, avoid tilting >10°) - ไม่สวมแว่นกันแดดหรือหน้ากาก (Don't wear sunglasses or masks) - ระบบตรวจสอบคุณภาพ ≥80% เท่านั้น (System only checks quality ≥80%) - เก็บภาพสดในไดคัท 0.5 วินาที (Capture live image in 0.5s dcut) - ต้องการ 15 ภาพคุณภาพสูง (Need 15 high-quality images)

Below the table is a large purple button labeled 'เริ่มสแกนใบหน้า' (Start Face Scanning). At the bottom, there are links for 'วิธีการสแกนใบหน้าที่ถูกต้อง' (Correct Scanning Method) and a button to 'กลับไปแก้ไขข้อมูล' (Go back to edit information).

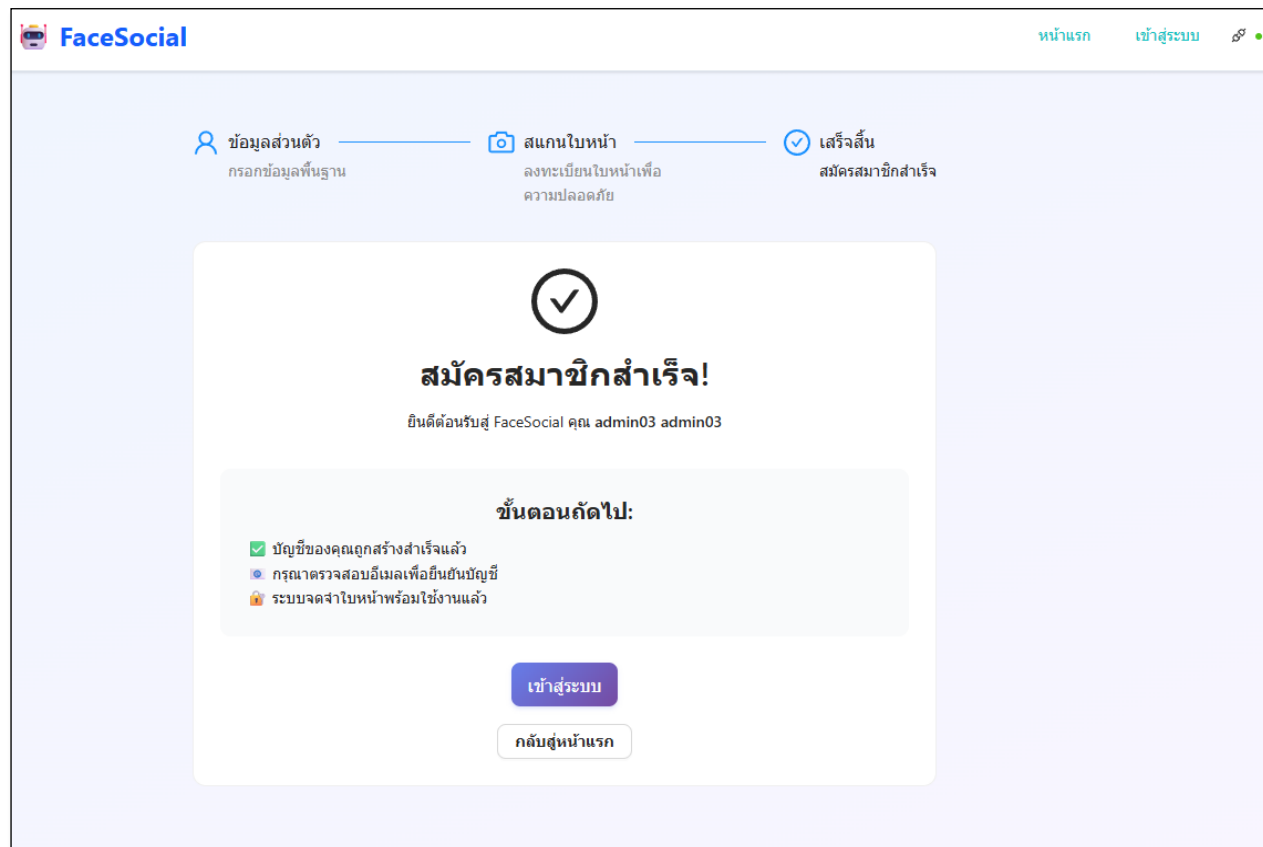
ภาพที่ ง-16 หน้าสแกนใบหน้าเพื่อความปลอดภัย

ง.1.1.5 การทำ UAT ในส่วนของหน้าสแกนหน้าเพื่อความปลอดภัย

ตารางที่ ง-5 หน้าสแกนหน้าเพื่อความปลอดภัย

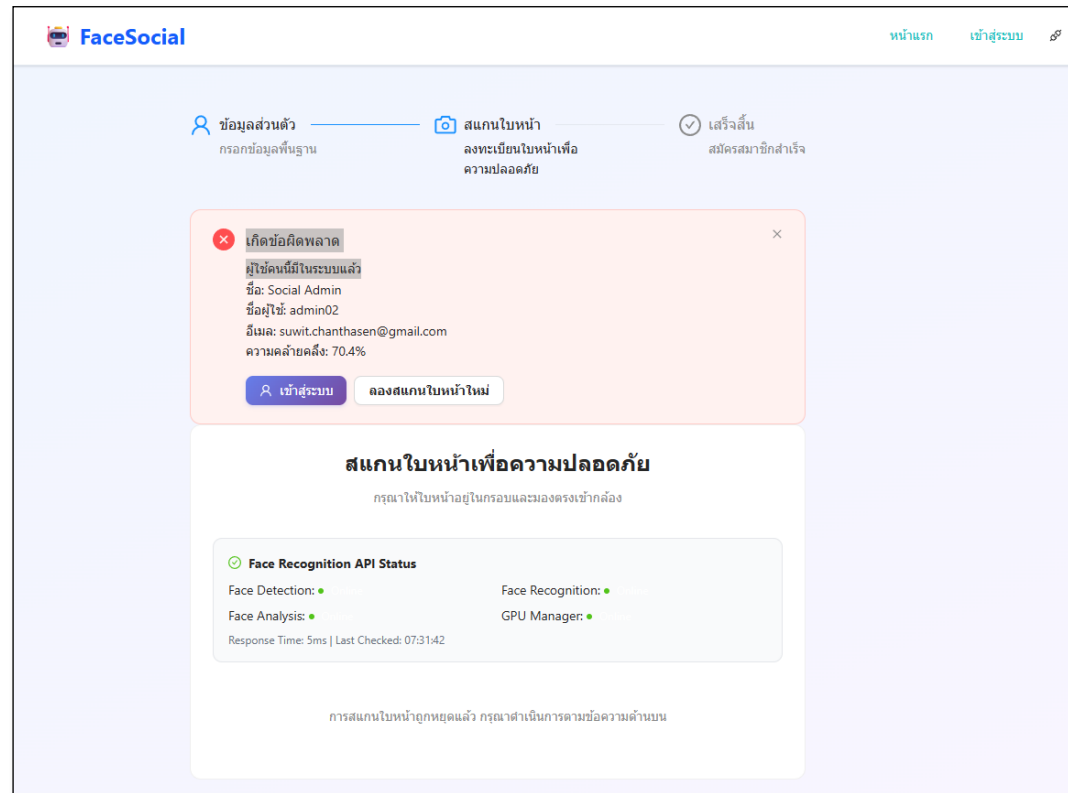
No.	Feature	Test case ID	Scenario	Test step	Expected Result	Test Result	Tesg Staus	Test
5	หน้าสแกนหน้าเพื่อความปลอดภัย	5.1	ตรวจสอบระบบสแกนใบหน้าที่ไม่มีในฐานข้อมูล	สแกนใบหน้าแล้วคลิกปุ่ม "เริ่มสแกนหน้า"	แจ้งเตือน "สมัครสมาชิกสำเร็จ"	สามารถแจ้งเตือน "สมัครสมาชิกสำเร็จ" ได้ดังภาพที่ ง-17	Pass	Suwit
		5.2	ตรวจสอบระบบสแกนใบหน้าที่มีในฐานข้อมูล	สแกนใบหน้าแล้วคลิกปุ่ม "เริ่มสแกนหน้า"	แจ้งเตือน "เกิดข้อผิดพลาด ผู้ใช้คนนี้มีในระบบแล้ว"	สามารถแจ้งเตือน "เกิดข้อผิดพลาด ผู้ใช้คนนี้มีในระบบแล้ว" ได้ดังภาพที่ ง-18	Pass	Suwit

ง.1.1.5.1 ผลการทดสอบ Test Case ID 5.1 ตรวจสอบระบบสแกนใบหน้าที่ไม่มีในฐานข้อมูล ดังภาพที่ ง-17



ภาพที่ ง-17 หน้าจอแจ้งเตือนการลงทะเบียนสมาชิกสำเร็จ

ง.1.1.5.2 ผลการทดสอบ Test Case ID 5.2 ตรวจสอบระบบสแกนใบหน้าที่มีพื้นฐานข้อมูล ดังภาพที่ ง-18



ภาพที่ ง-18 หน้าจอแจ้งเตือนผู้ใช้คนนี้มีในระบบแล้ว

ง.1.1.6 การทำ UAT ในส่วนของหน้า AI Testing Lab

ตารางที่ ง-6 หน้า AI Testing Lab

No.	Feature	Test case ID	Scenario	Test step	Expected Result	Test Result	Test Status	Test
6	หน้า AI Testing Lab	6.1	ตรวจสอบการทำงานของ การตรวจจับ ใบหน้า (Face Detection) ด้วยไฟล์ ภาพที่มีใบหน้า	วางไฟล์ภาพแล้ว คลิกปุ่ม "เริ่มทำ การตรวจจับ ใบหน้า"	แจ้งเตือน "สำเร็จ ตรวจ พบใบหน้า"	สามารถตรวจจับใบหน้า ได้ ดังภาพที่ ง-19	Pass	Suwit
		6.2	ตรวจสอบการทำงานของ การตรวจจับ ใบหน้า (Face Detection) ด้วยไฟล์ ภาพที่ไม่มีใบหน้า	วางไฟล์ภาพแล้ว คลิกปุ่ม "เริ่มทำ การตรวจจับ ใบหน้า"	แจ้งเตือน "ไม่ พบใบหน้า ไม่ สามารถ ตรวจจับใบหน้า ในรูปนี้ได้"	สามารถแจ้งเตือน "ไม่ พบใบหน้า" ได้ ดังภาพที่ ง-20	Pass	Suwit
		6.3	ตรวจสอบการทำงานของ การตรวจจับ ใบหน้า (Face Detection) แบบ เร็วไหม	คลิกปุ่ม "Real-time Face Detection (Working)"	แจ้งเตือน ใบหน้าที่พบ	สามารถตรวจจับใบหน้า แบบเร็วไหมได้ ดังภาพที่ ง-21	Pass	Suwit

ตารางที่ ง-6 หน้า AI Testing Lab (ต่อ)

No.	Feature	Test case ID	Scenario	Test step	Expected Result	Test Result	Test Status	Test
6	หน้า AI Testing Lab	6.4	ตรวจสอบการทำงานของ การตรวจจับการปลอมแปลง (Anti-Spoofing) ด้วยไฟล์ภาพที่มีใบหน้า	คลิกแท็บ "Anti-Spoofing" วางไฟล์ภาพแล้วคลิกปุ่ม "ตรวจสอบการปลอมแปลง"	แจ้งเตือน "สำเร็จ ตรวจสอบเสร็จสมบูรณ์"	สามารถตรวจจับการปลอมแปลงได้ ดังภาพที่ ง-22	Pass	Suwit
		6.5	ตรวจสอบการทำงานของ การตรวจจับการปลอมแปลง (Anti-Spoofing) ด้วยไฟล์ภาพที่เป็นภาพปลอมแปลง	คลิกแท็บ "Anti-Spoofing" วางไฟล์ภาพแล้วคลิกปุ่ม "ตรวจสอบการปลอมแปลง"	แจ้งเตือน "สำเร็จ ตรวจสอบเสร็จสมบูรณ์"	สามารถตรวจจับการปลอมแปลงได้ ดังภาพที่ ง-23	Pass	Suwit
		6.6	ตรวจสอบการทำงานของ การตรวจจับการปลอมแปลง (Anti-Spoofing) แบบเรียลไทม์	คลิกแท็บ "Anti-Spoofing" แล้วคลิกปุ่ม "ตรวจจับแบบ Real-time"	แจ้งเตือนการตรวจสอบทั้งหมด	สามารถตรวจจับการปลอมแปลงแบบเรียลไทม์ได้ ดังภาพที่ ง-24	Pass	Suwit

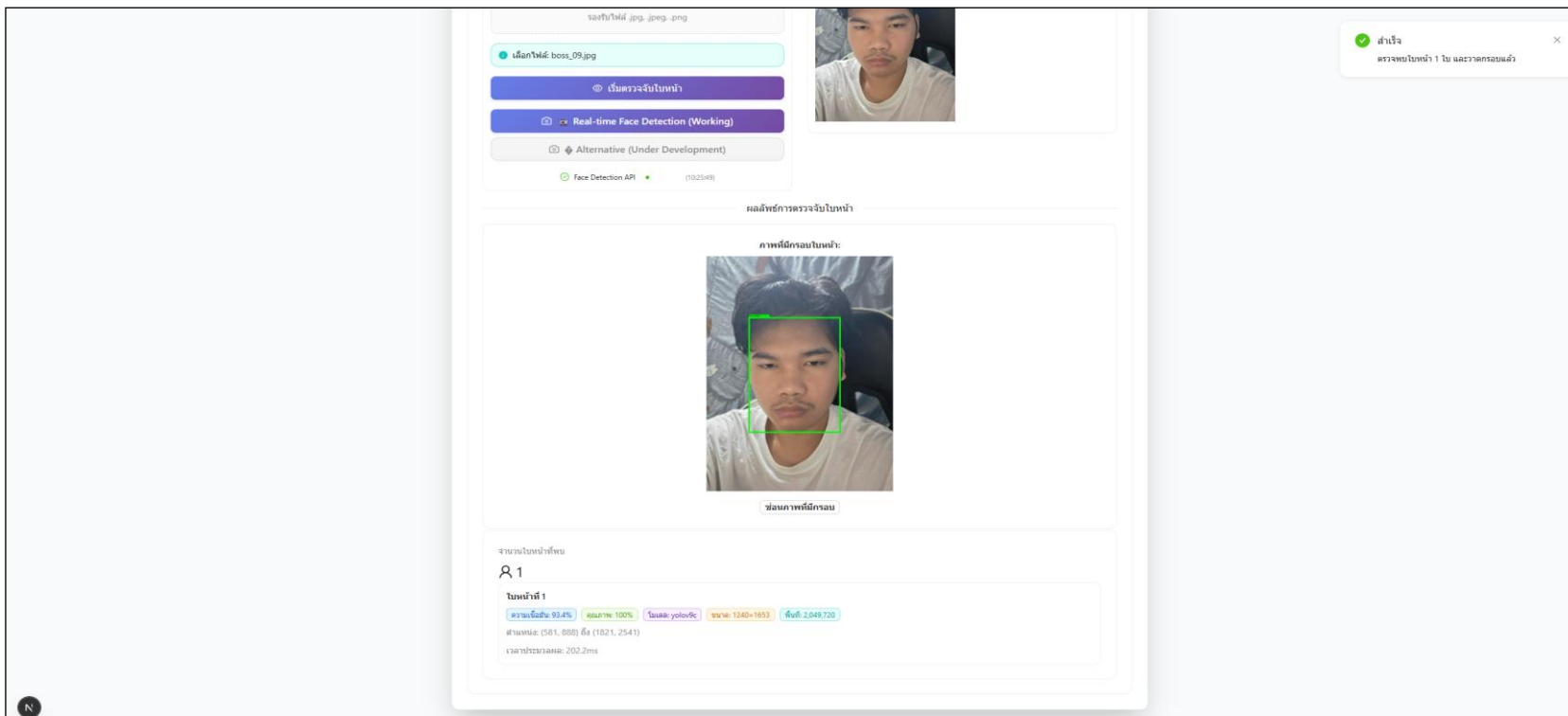
ตารางที่ ง-6 หน้า AI Testing Lab (ต่อ)

No.	Feature	Test case ID	Scenario	Test step	Expected Result	Test Result	Tesg Staus	Test
6	หน้า AI Testing Lab	6.7	ตรวจสอบการทำงานของ การวิเคราะห์ อายุและเพศ (Age and Gender Analysis) ที่มีใบหน้า	คลิกแท็บ "Age and Gender" วางไฟล์ภาพแล้ว คลิกปุ่ม "วิเคราะห์อายุ และเพศ"	แจ้งเตือน "สำเร็จ วิเคราะห์เสร็จ สมบูรณ์"	สามารถตรวจจับอายุ และเพศได้ ดังภาพที่ ง-25	Pass	Suwit
		6.8	ตรวจสอบการทำงานของ การวิเคราะห์ อายุและเพศ (Age and Gender Analysis) ที่ไม่มี ใบหน้า	คลิกแท็บ "Age and Gender" วางไฟล์ภาพแล้ว คลิกปุ่ม "วิเคราะห์อายุ และเพศ"	แจ้งเตือน "ไม่ พบใบหน้า ไม่ สามารถตรวจ พบใบหน้าใน รูปภาพ กรุณา ลองใช้รูปภาพที่ มีใบหน้าชัดเจน กว่า"	สามารถแจ้งเตือน "ไม่ พบใบหน้า" ได้ ดังภาพที่ ง-26	Pass	Suwit

ตารางที่ ง-6 หน้า AI Testing Lab (ต่อ)

No.	Feature	Test case ID	Scenario	Test step	Expected Result	Test Result	Tesg Staus	Test
	หน้า AI Testing Lab	6.9	ตรวจสอบการทำงานของกล้องวงจรปิดแบบเรียลไทม์ (CCTV Face Recognition)	คลิกแท็บ "Real-time CCTV" แล้วคลิกปุ่ม "เริ่มต้น CCTV"	จับภาพใบหน้าและบอกได้ว่าคนในภาพทั้งหมดเป็นใคร	สามารถจับภาพใบหน้าและบอกได้ว่าคนในภาพทั้งหมดดังภาพที่ ง-27	Pass	Suwit

ง.1.1.6.1 ผลการทดสอบ Test Case ID 6.1 ตรวจสอบการทำงานของการทำงานของการตรวจจับใบหน้า (Face Detection) ด้วยไฟล์ภาพที่มีใบหน้า
ดังภาพที่ ง-19

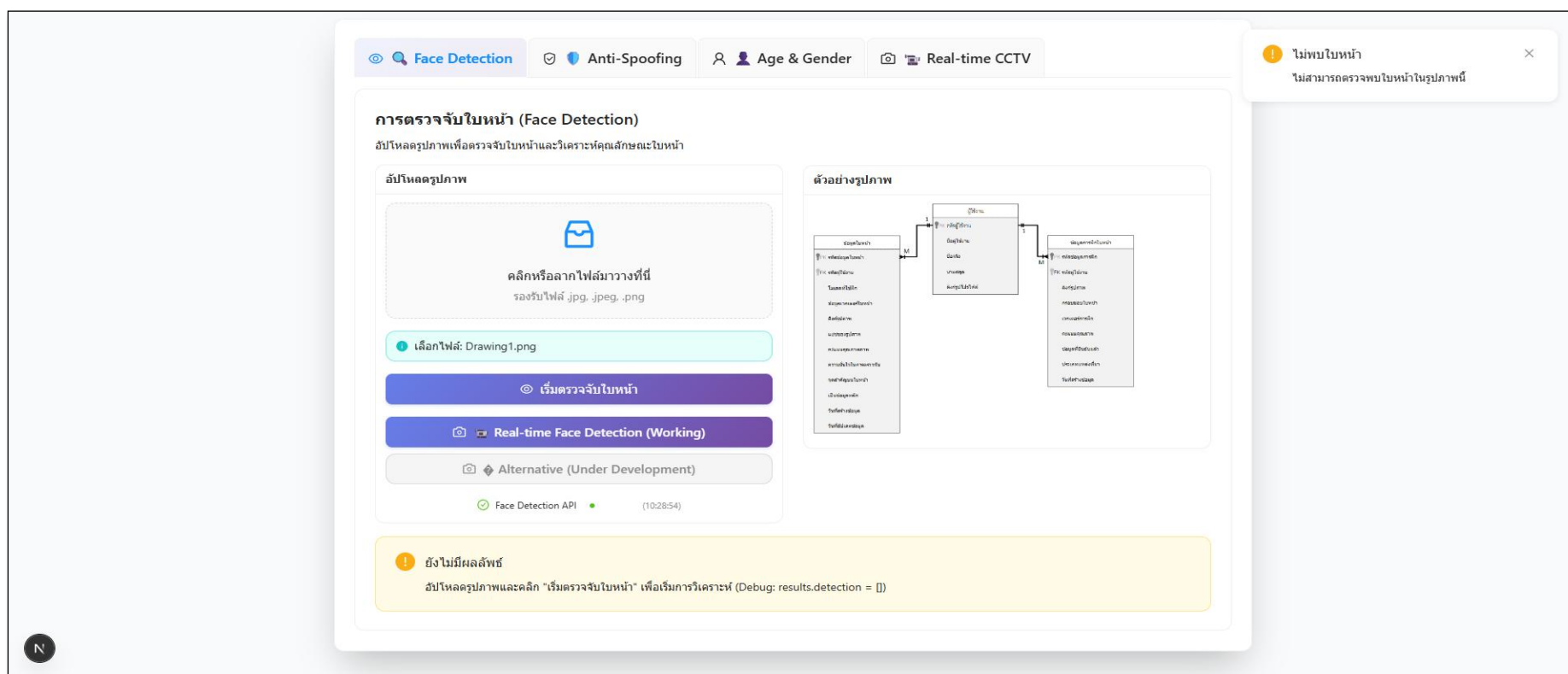


ภาพที่ ง-19 หน้าจอผลการตรวจจับใบหน้า

ง.1.1.6.2 ผลการทดสอบ Test Case ID 6.2 ตรวจสอบการทำงานของการทำงานการตรวจจับใบหน้า (Face Detection) ด้วยไฟล์ภาพที่ไม่มีใบหน้า

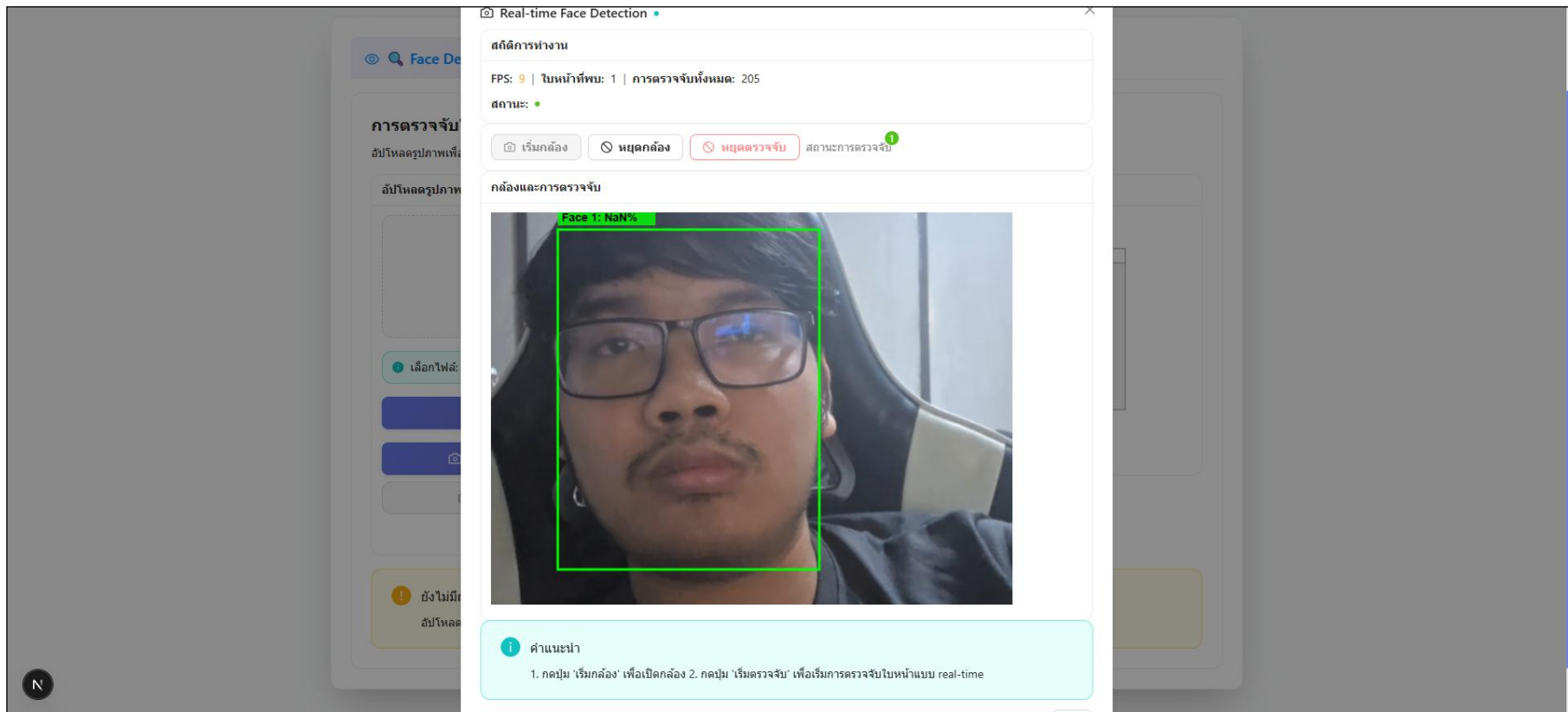
ดังภาพที่

ง-20



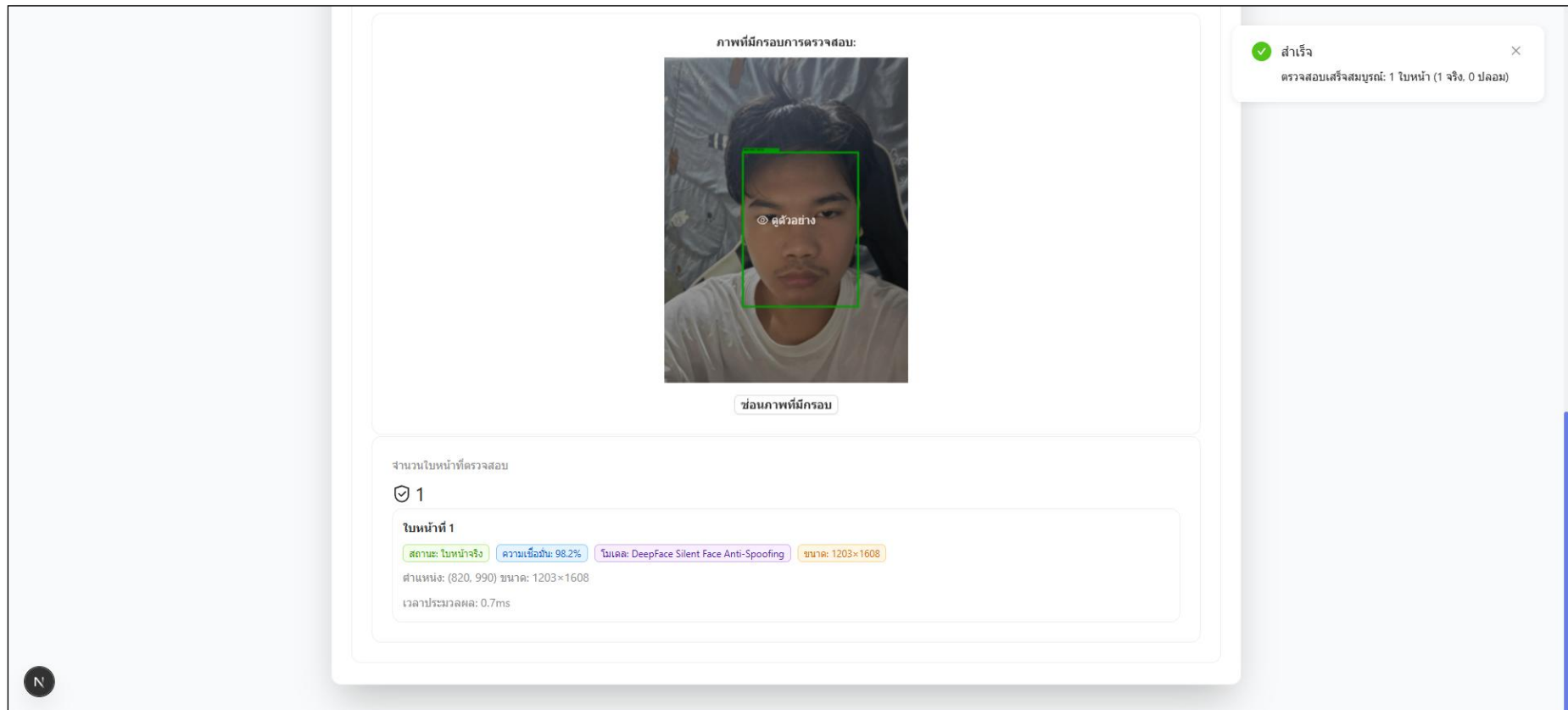
ภาพที่ ง-20 หน้าจอแจ้งเตือนไม่พบใบหน้า

ง.1.1.6.3 ผลการทดสอบ Test Case ID 6.3 ตรวจสอบการทำงานของการทำงานของการตรวจจับใบหน้า (Face Detection) แบบเรียลไทม์ ดังภาพที่ ง-21



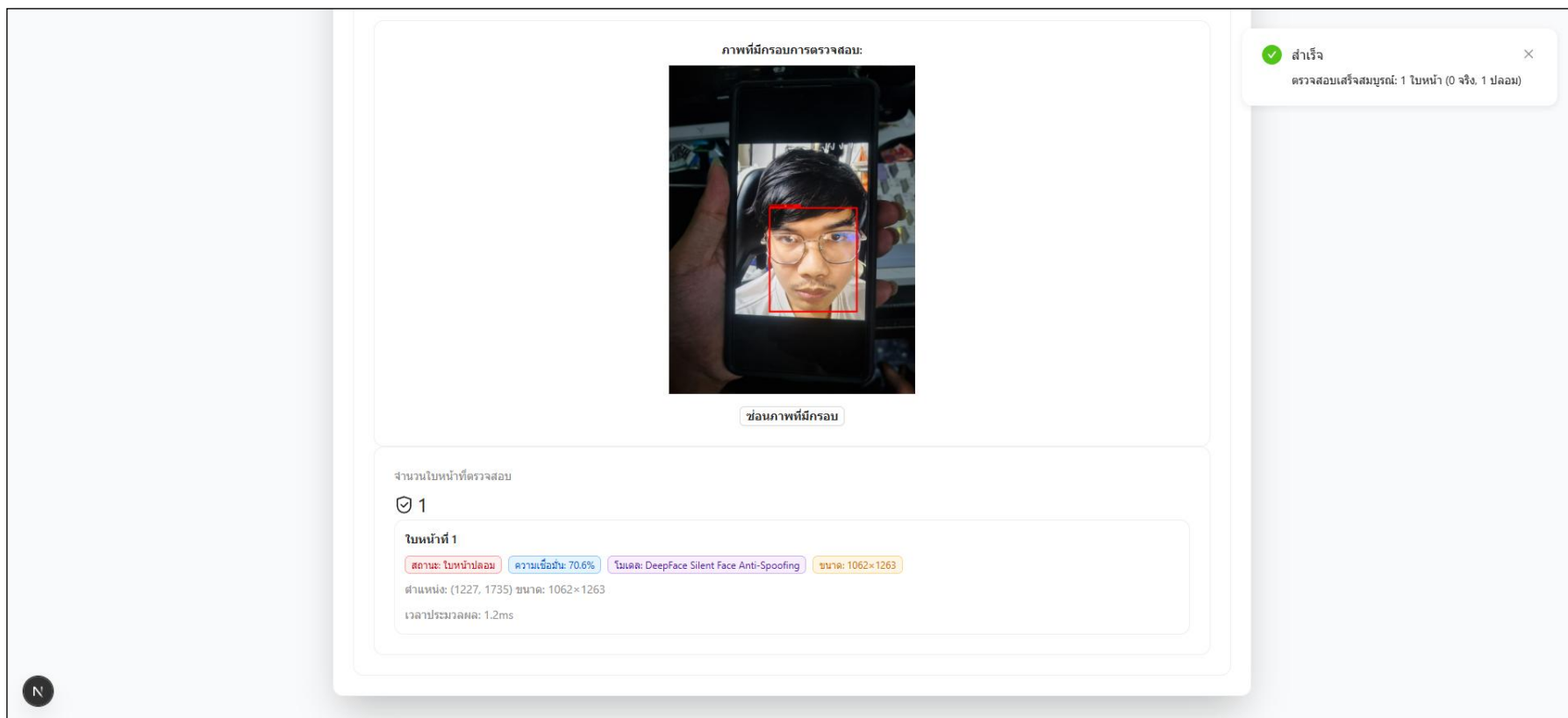
ภาพที่ ง-21 หน้าจอการตรวจจับใบหน้าแบบเรียลไทม์

ง.1.1.6.4 ผลการทดสอบ Test Case ID 6.4 ตรวจสอบการทำงานของการทำงานของการตรวจจับการปลอมแปลง (Anti-Spoofing) ด้วยไฟล์ภาพที่มีใบหน้า ดังภาพที่ ง-22



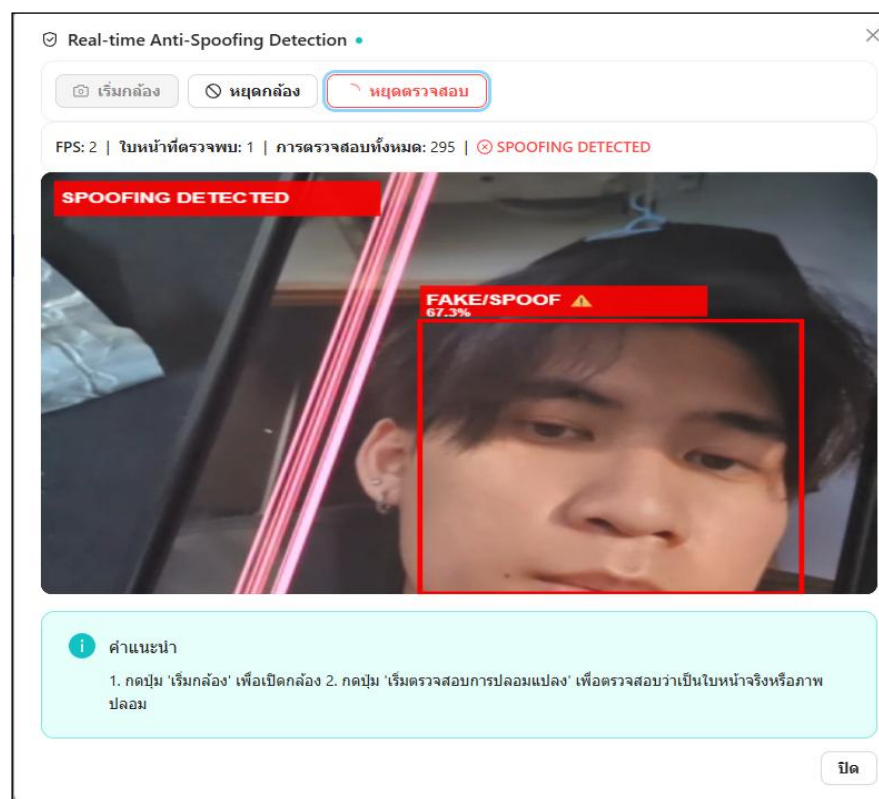
ภาพที่ ง-22 หน้าจอผลการตรวจสอบภาพจริง

ง.1.1.6.5 ผลการทดสอบ Test Case ID 6.5 ตรวจสอบการทำงานของการทำงานการตรวจจับการปลอมแปลง (Anti-Spoofing) ด้วยไฟล์ภาพที่เป็นภาพปลอมแปลง ดังภาพที่ ง-23



ภาพที่ ง-23 หน้าจอผลการตรวจสอบภาพปลอมแปลง

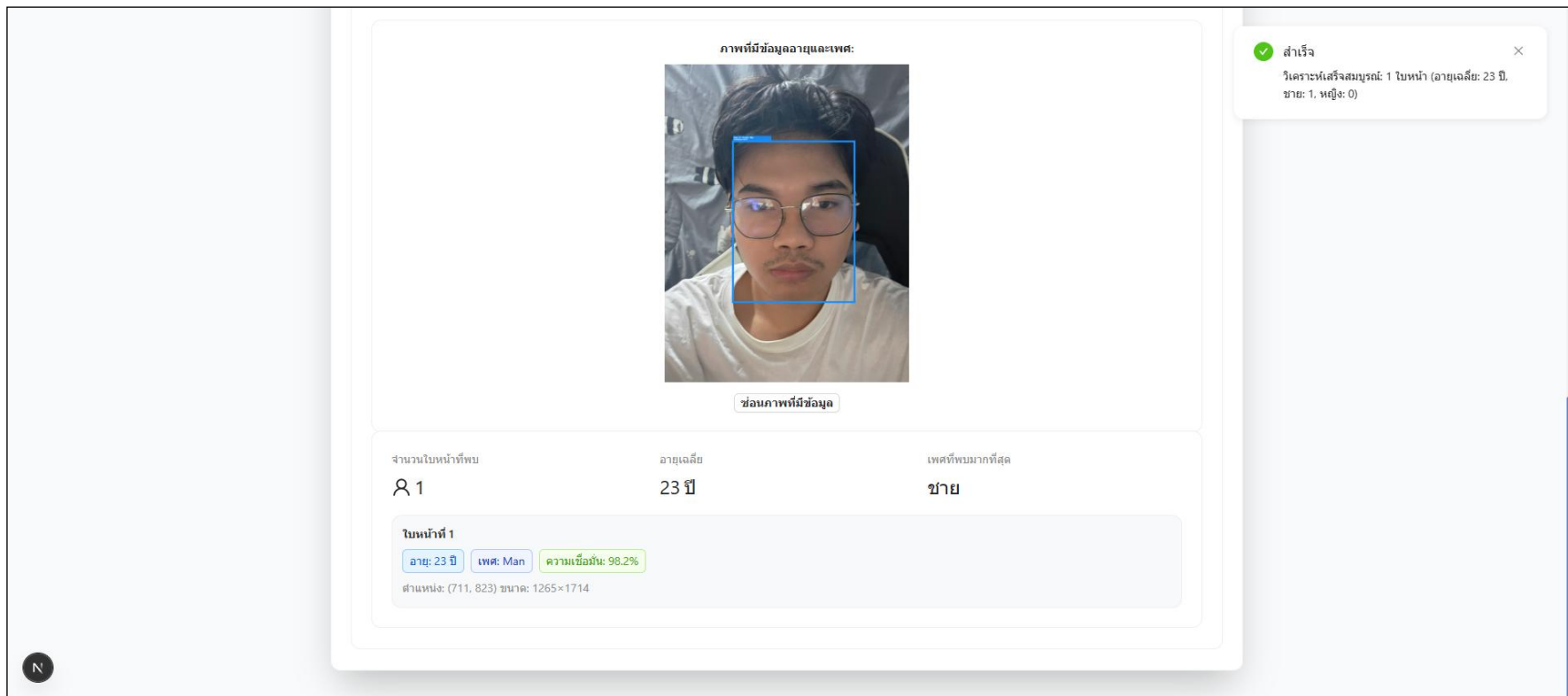
ง.1.1.6.6 ผลการทดสอบ Test Case ID 6.6 ตรวจสอบการทำงานของการทำงานของการตรวจจับการปลอมแปลง (Anti-Spoofing) แบบเรียลไทม์ที่เป็นภาพปลอมแปลง ดังภาพที่ ง-24



ภาพที่ ง-24 หน้าจอการตรวจจับภาพปลอมแบบเรียลไทม์

ง.1.1.6.7 ผลการทดสอบ Test Case ID 6.7 ตรวจสอบการทำงานของการทำงานของการวิเคราะห์อายุและเพศ (Age and Gender Analysis) ที่มีใบหน้า
ภาพที่ ง-25

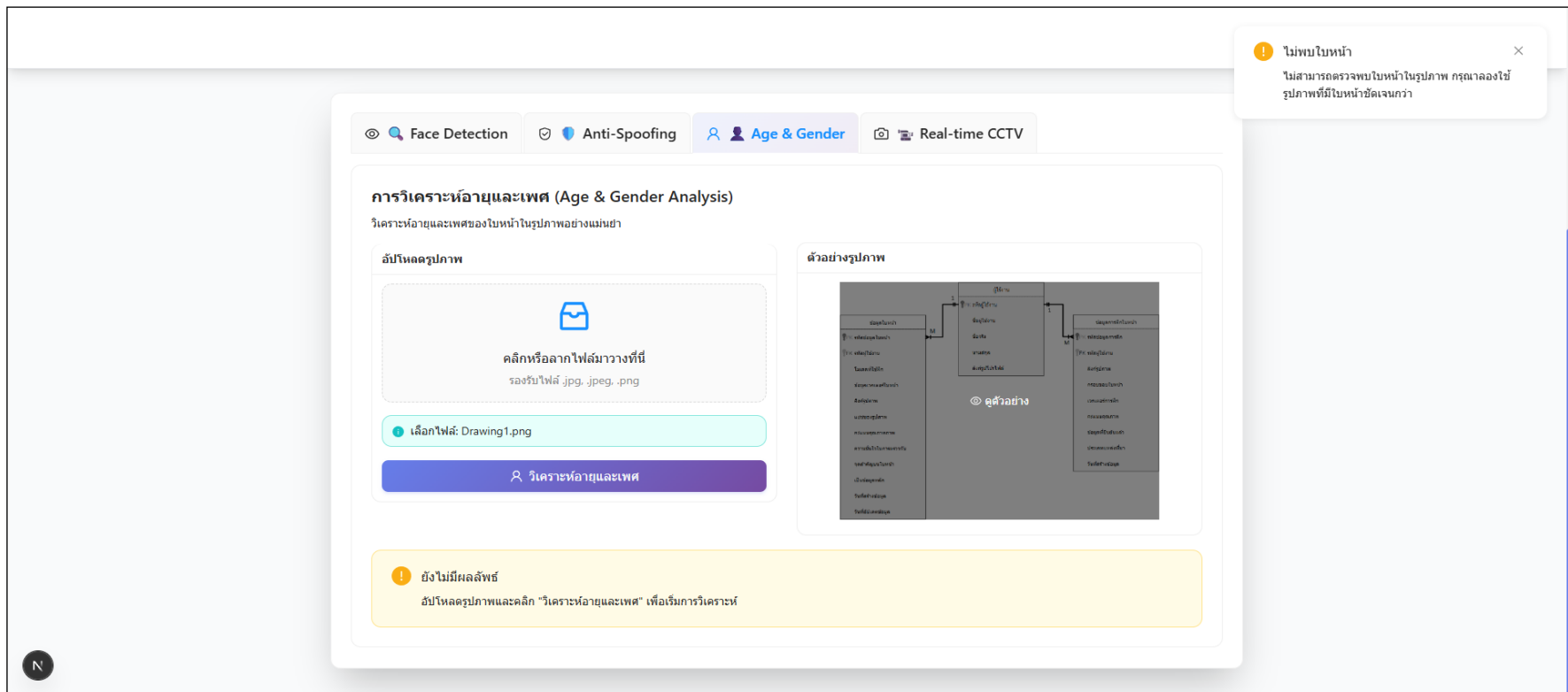
ดัง



ภาพที่ ง-25 หน้าจอผลการวิเคราะห์อายุและเพศ

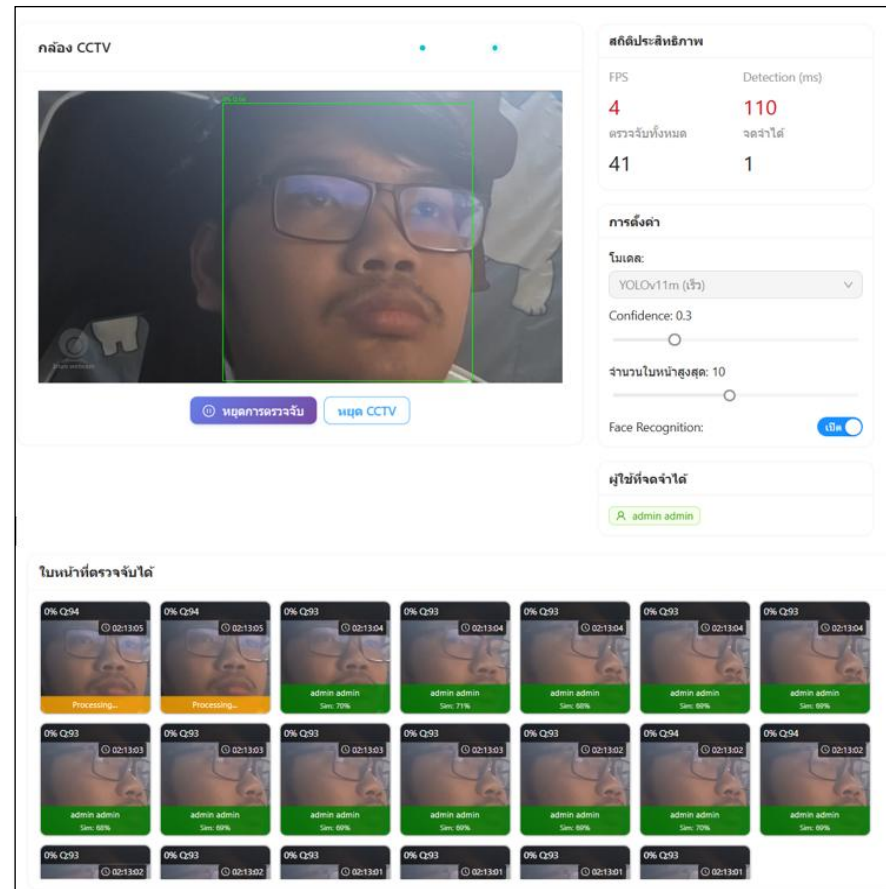
ง.1.1.6.8 ผลการทดสอบ Test Case ID 6.8 ตรวจสอบการทำงานของการทำงานการวิเคราะห์อายุและเพศ (Age and Gender Analysis) ที่ไม่มีใบหน้า
ภาพที่ ง-26

ดัง



ภาพที่ ง-26 หน้าจอแจ้งเตือนในภาพที่ไม่มีใบหน้า

ง.1.1.6.9 ผลการทดสอบ Test Case ID 6.9 ตรวจสอบการทำงานของกล้องวงจรปิดแบบเรียลไทม์ (CCTV Face Recognition) ดังภาพที่ ง-27



ภาพที่ ง-27 หน้าจอกล้องวงจรปิดที่ตรวจจับและระบุตัวตนใบหน้า

ง.1.1.7 การทำ UAT ในส่วนของหน้าNavigation Menu

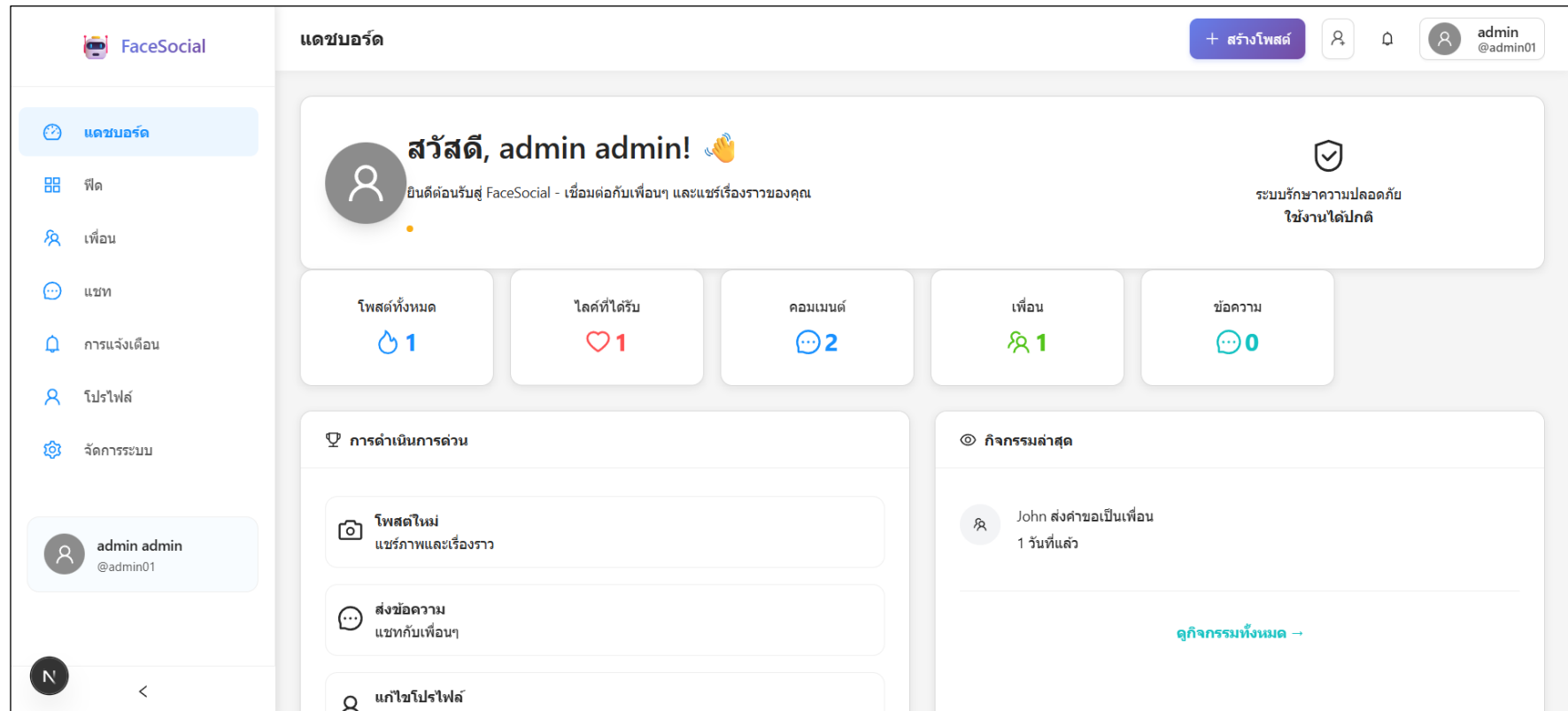
ตารางที่ ง-7 Navigation Menu

No.	Feature	Test case ID	Scenario	Test step	Expected Result	Test Result	Tesg Staus	Test
7	Navigation Menu	7.1	ไปยังหน้า Dashboard	คลิกปุ่ม "Dashboard"	ไปยังหน้า Dashboard	สามารถไปยังหน้า Dashboard ได้ ดังภาพที่ ง-28	Pass	Suwit
		7.2	ไปยังหน้า Feed	คลิกปุ่ม " Feed "	ไปยังหน้า Feed	สามารถไปยังหน้า Feed ได้ ดังภาพที่ ง-29	Pass	Suwit
		7.3	ไปยังหน้าเพื่อน	คลิกปุ่ม "เพื่อน"	ไปยังหน้าเพื่อน	สามารถไปยังหน้าเพื่อน ได้ ดังภาพที่ ง-30	Pass	Suwit
		7.4	ไปยังหน้า Chat	คลิกปุ่ม " Chat "	ไปยังหน้า Chat	สามารถไปยังหน้า Chat ได้ ดังภาพที่ ง-31	Pass	Suwit

ตารางที่ ง-7 Navigation Menu (ต่อ)

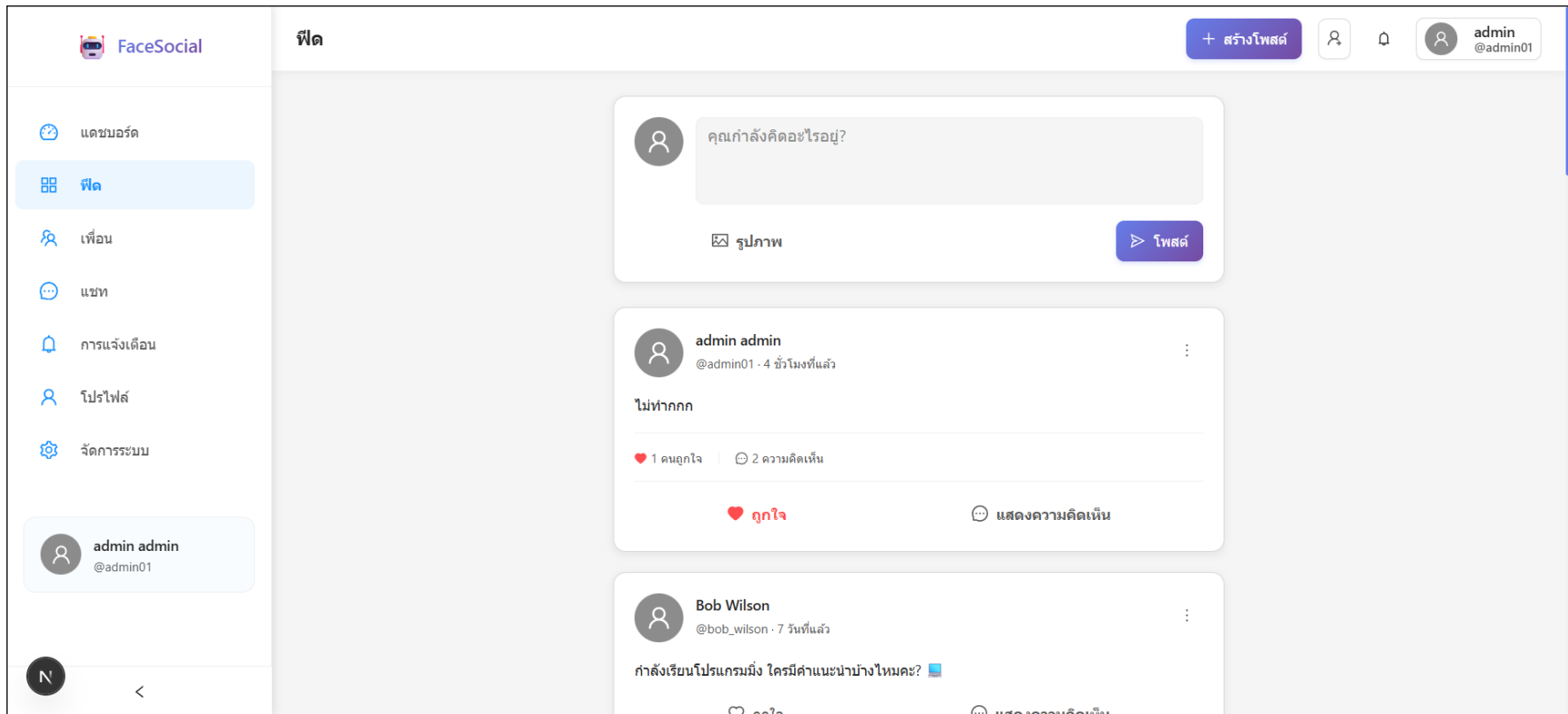
No.	Feature	Test case ID	Scenario	Test step	Expected Result	Test Result	Tesg Staus	Test
		7.5	ไปยังหน้าการแจ้งเตือน	คลิกปุ่ม "การแจ้งเตือน"	ไปยังหน้าการแจ้งเตือน	สามารถไปยังหน้าการแจ้งเตือนได้ ดังภาพที่ ง-32	Pass	Suwit
		7.6	ไปยังหน้าประวัติโดยย่อ	คลิกปุ่ม "ประวัติโดยย่อ"	ไปยังหน้าประวัติโดยย่อ	สามารถไปยังหน้าประวัติโดยย่อได้ดังภาพที่ ง-33	Pass	Suwit

ง.1.1.7.1 ผลการทดสอบ Test Case ID 7.1 กดปุ่มDashboardแล้วจะแสดงหน้า Dashboard ดังภาพที่ ง-28



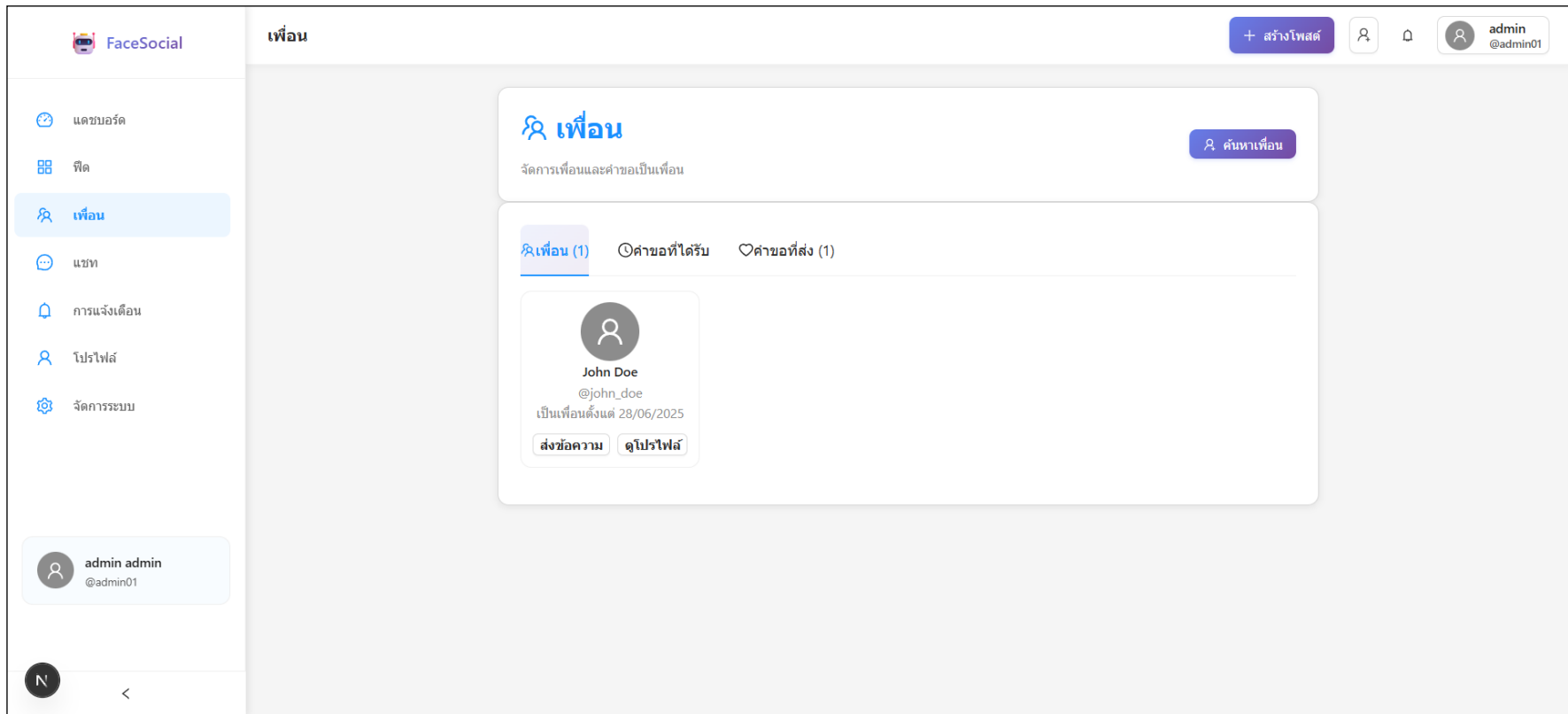
ภาพที่ ง-28 หน้า Dashboard

ง.1.1.7.2 ผลการทดสอบ Test Case ID 7.2 กดปุ่ม Feed แล้วจะแสดงหน้า Feed ดังภาพที่ ง-29



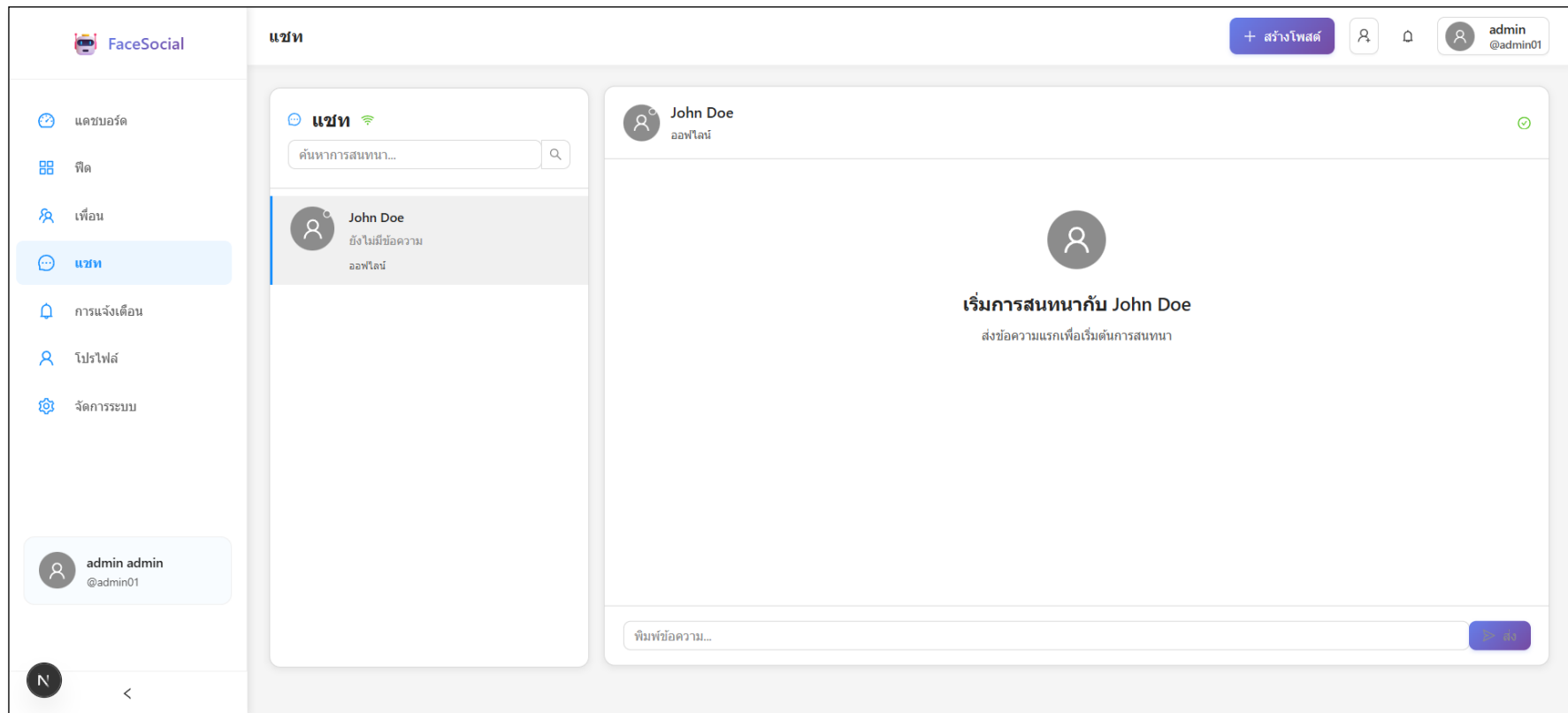
ภาพที่ ง-29 หน้า Feed

ง.1.1.7.3 ผลการทดสอบ Test Case ID 7.3 กดปุ่มเพื่อนแล้วจะแสดงหน้าเพื่อน ดังภาพที่ ง-30



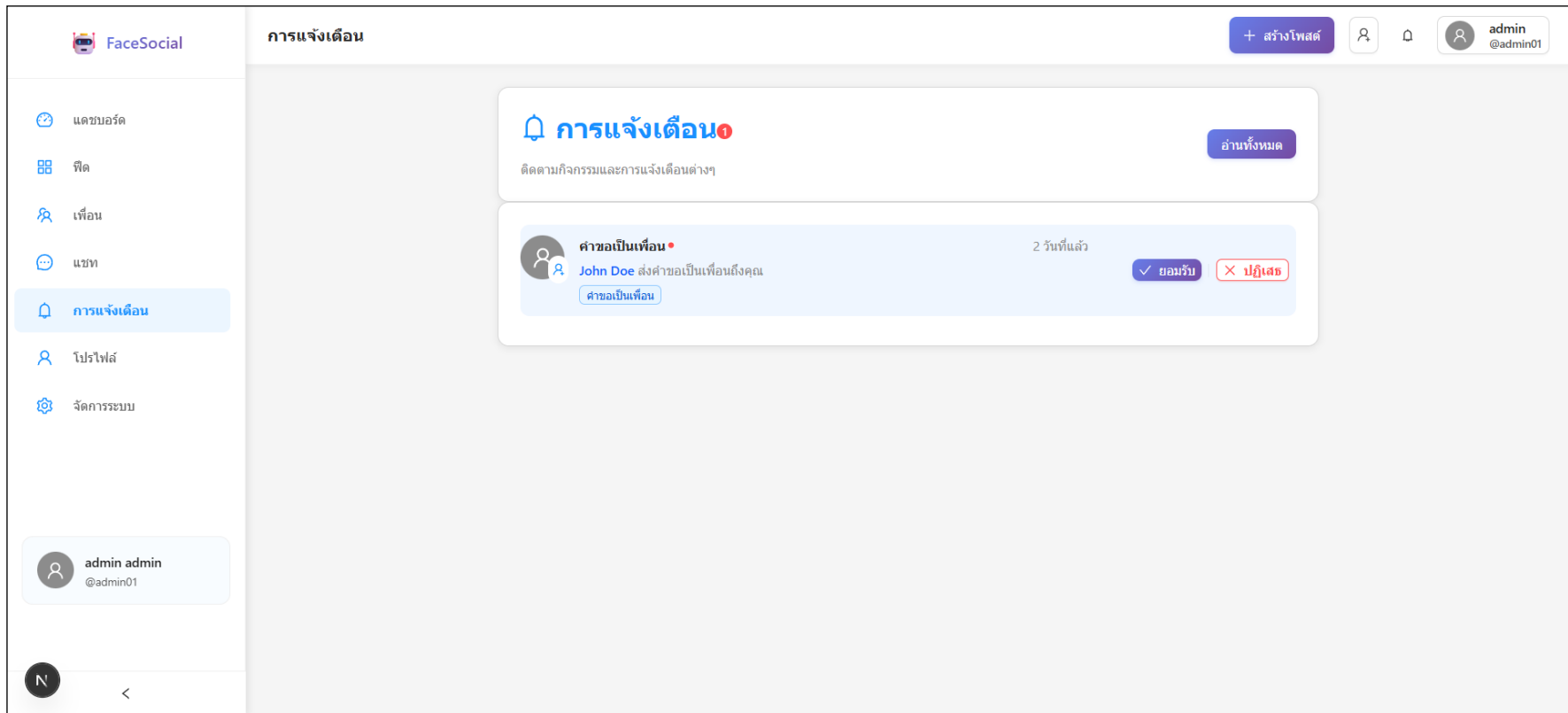
ภาพที่ ง-30 หน้าเพื่อน

ง.1.1.7.4 ผลการทดสอบ Test Case ID 7.4 กดปุ่ม Chat แล้วจะแสดงหน้า Chat ดังภาพที่ ง-31



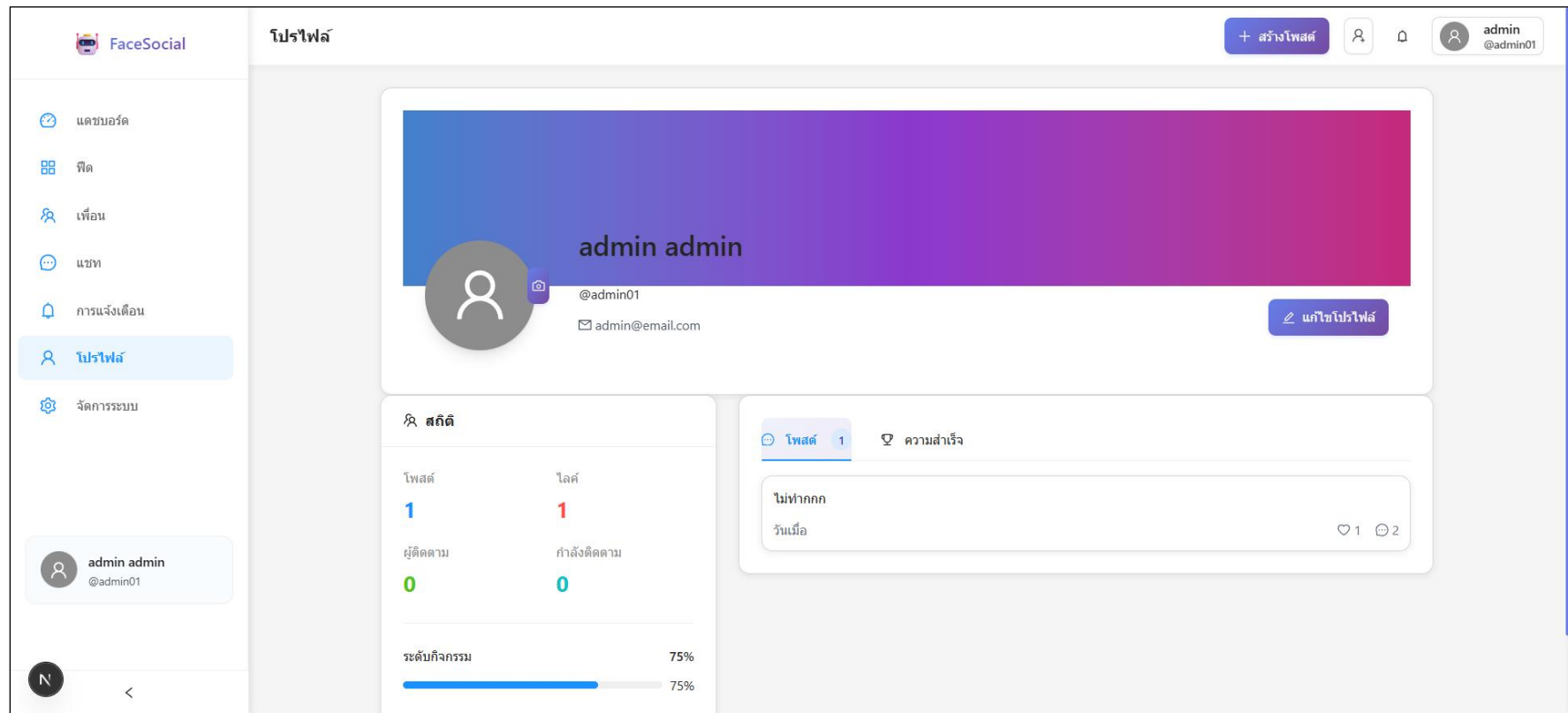
ภาพที่ ง-31 หน้า Chat

ง.1.1.7.5 ผลการทดสอบ Test Case ID 7.5 กดปุ่มการแจ้งเตือนแล้วจะแสดงหน้าการแจ้งเตือน ดังภาพที่ ง-32



ภาพที่ ง-32 หน้าการแจ้งเตือน

ง.1.1.7.6 ผลการทดสอบ Test Case ID 7.6 กดปุ่มประวัติโดยย่อแล้วจะแสดงหน้าประวัติโดยย่อ ดังภาพที่ ง-33



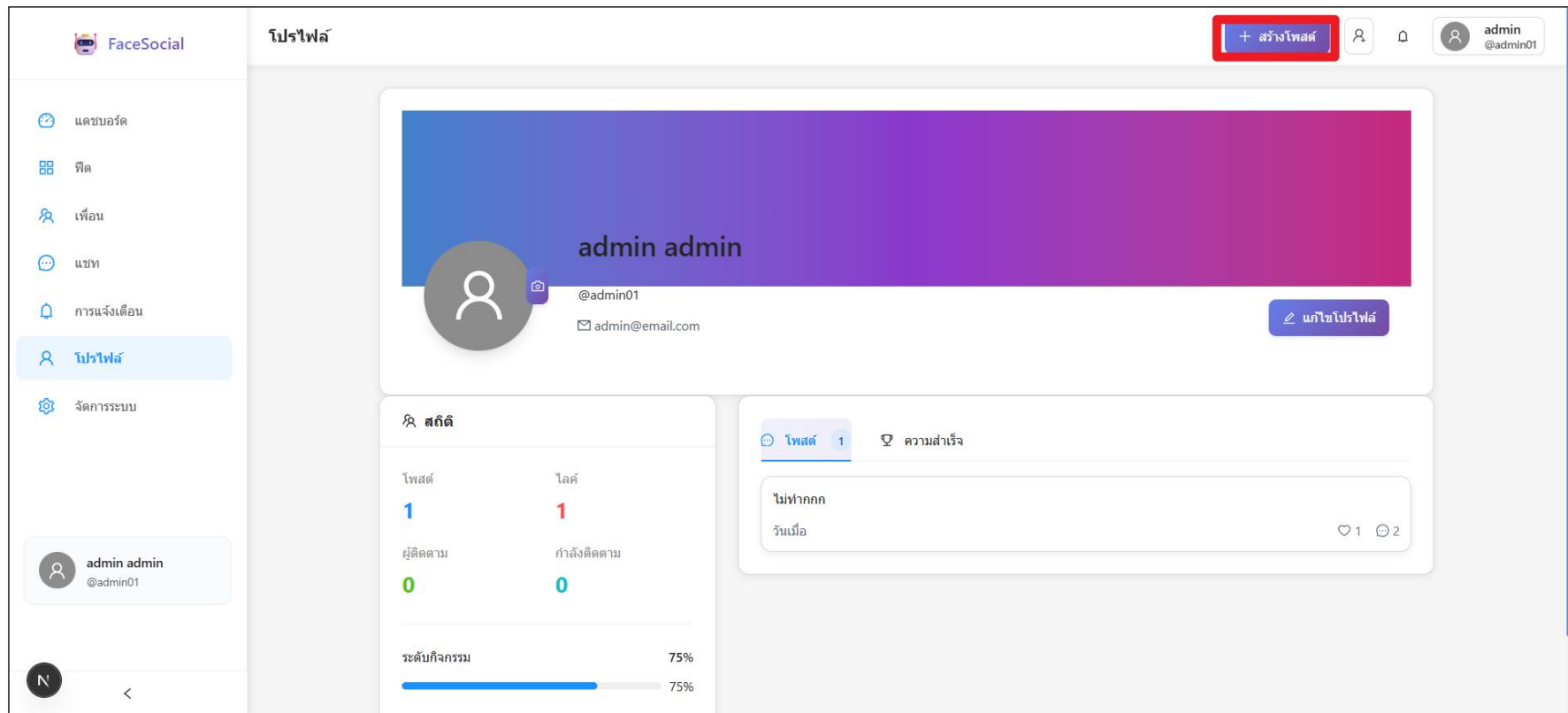
ภาพที่ ง-33 หน้าประวัติโดยย่อ

ง.1.1.8 การทำ UAT ในส่วนของหน้าสร้างโพสต์

ตารางที่ ง-8 สร้างโพสต์

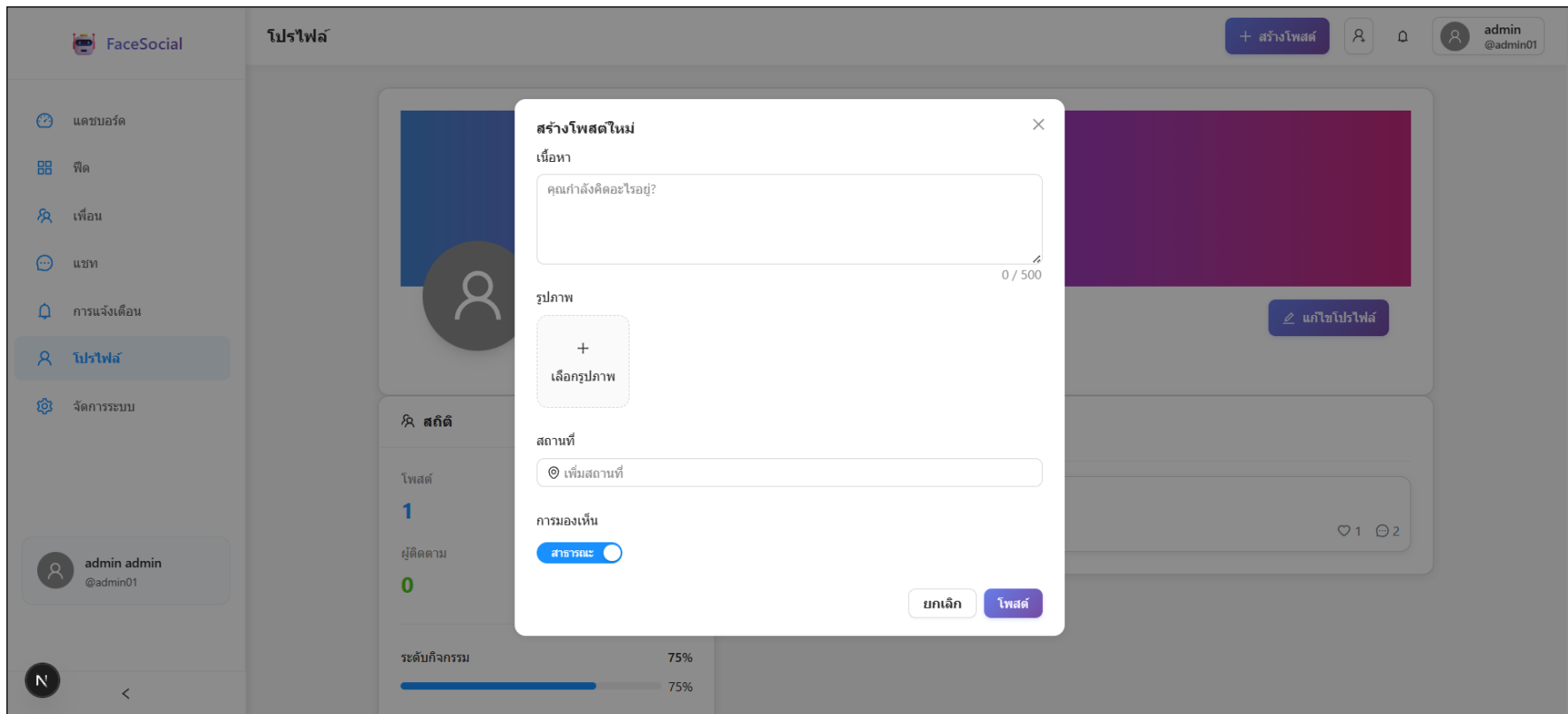
No.	Feature	Test case ID	Scenario	Test step	Expected Result	Test Result	Tesg Staus	Test
8	สร้างโพสต์	8.1	ไปยังหน้าสร้างโพสต์	คลิกปุ่ม "สร้างโพสต์"	ไปยังหน้าสร้างโพสต์	สามารถไปยังหน้าสร้างโพสต์ได้ ดังภาพที่ ง-34-35	Pass	Suwit
		8.2	ตรวจสอบการโพสต์เนื้อหา	กรอกเนื้อหาแล้วคลิกปุ่ม "โพสต์"	สร้างโพสต์สำเร็จ	สามารถสร้างโพสต์ได้ ดังภาพที่ ง-36-37-38	Pass	Suwit
		8.3	ตรวจสอบการโพสต์เนื้อหาและแท็กผู้ใช้อัตโนมัติ	ใส่รูปภาพ คลิกปุ่ม "แท็กเพื่อนอัตโนมัติ (AI ตรวจสอบจับใบหน้า)" เลือกผู้ใช้ที่ตรงกับภาพ แล้วคลิกปุ่ม "โพสต์"	สร้างโพสต์ที่แท็กผู้ใช้อัตโนมัติได้	สามารถสร้างโพสต์ที่แท็กผู้ใช้อัตโนมัติได้ ดังภาพที่ ง-39-40-41	Pass	Suwit

ง.1.1.8.1 ผลการทดสอบ Test Case ID 8.1 กดปุ่มสร้างโพสต์แล้วจะแสดงหน้าสร้างโพสต์ Step 1 ดังภาพที่ ง-34



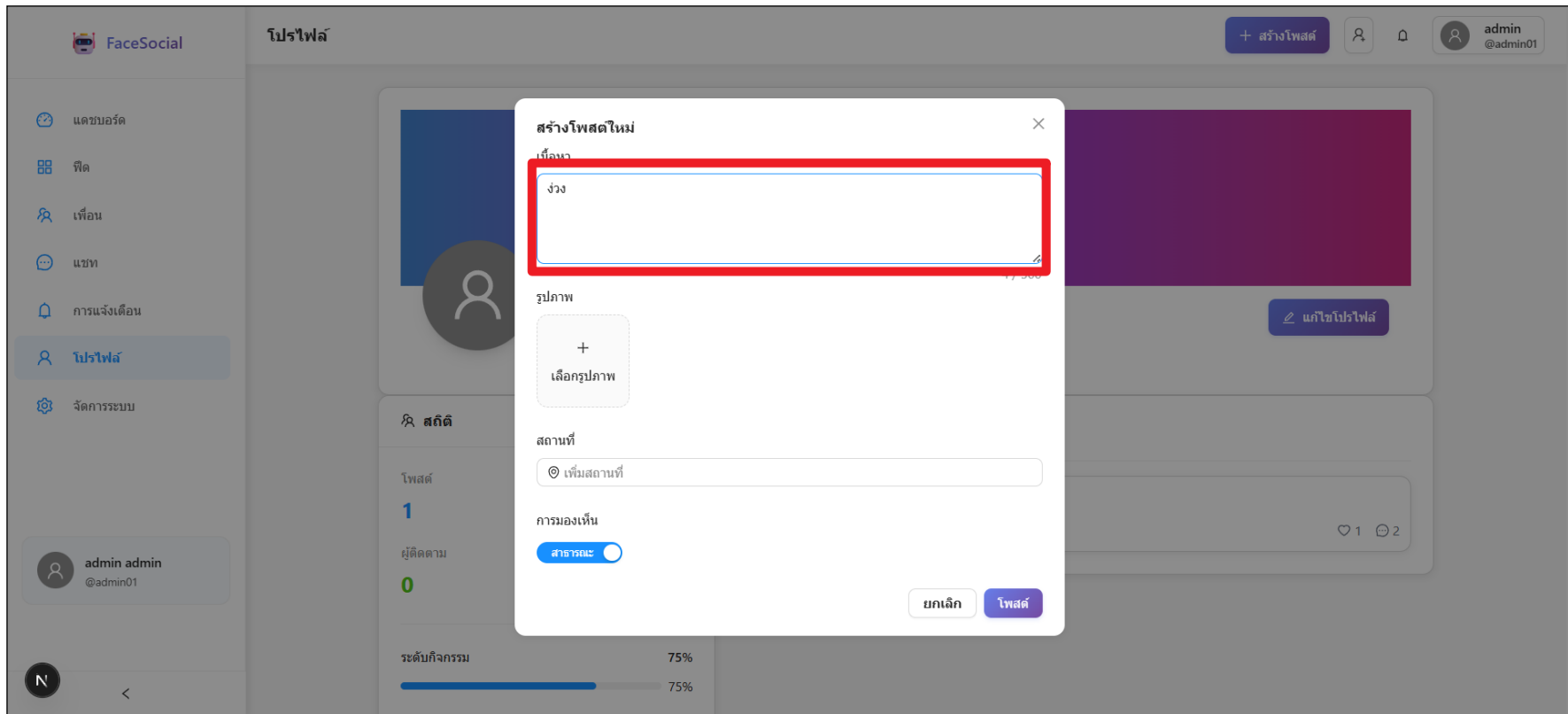
ภาพที่ ง-34 หน้าจอปุ่มสร้างโพสต์

ง.1.1.8.2 ผลการทดสอบ Test Case ID 8.1 กดปุ่มสร้างโพสต์แล้วจะแสดงหน้าสร้างโพสต์ Step 2 ดังภาพที่ ง-35



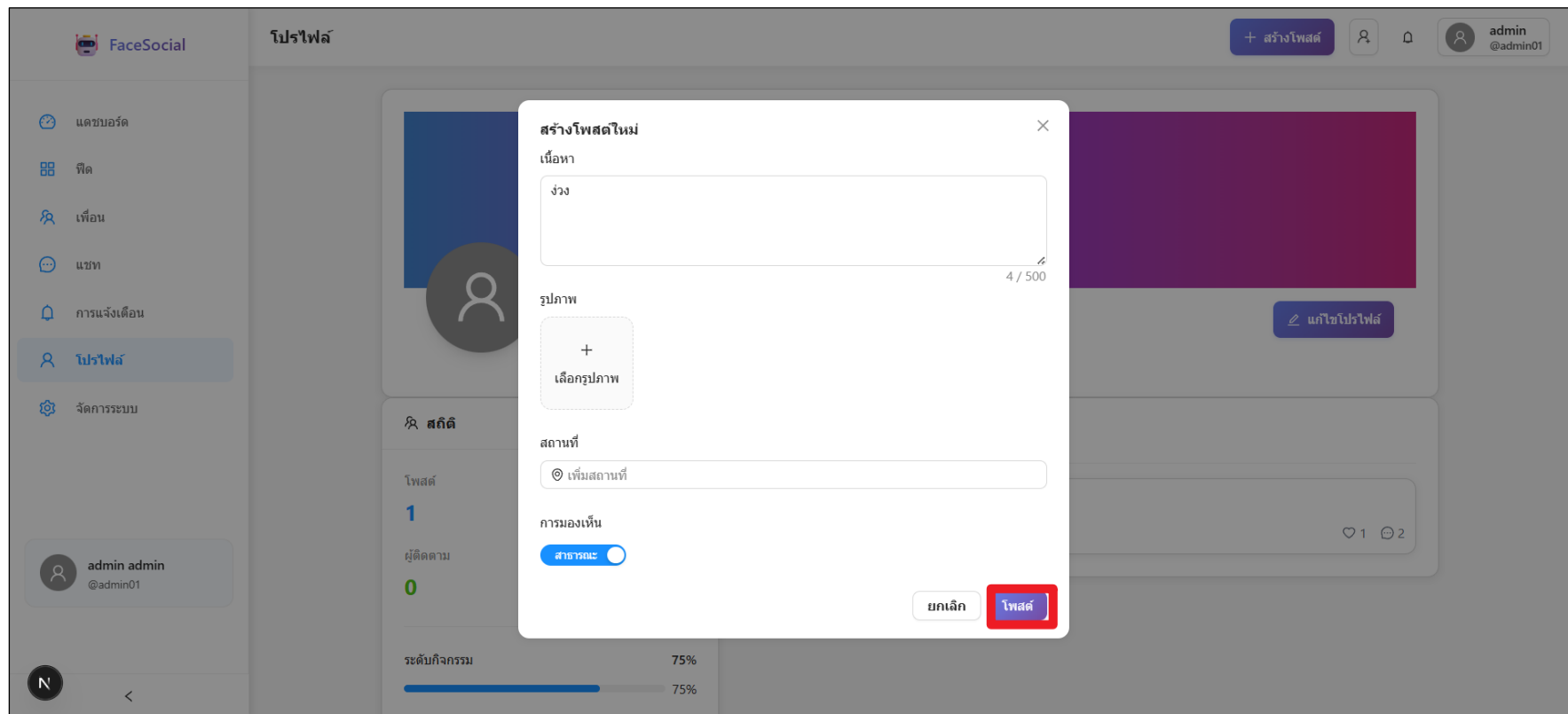
ภาพที่ ง-35 หน้าสร้างโพสต์

ง.1.1.8.3 ผลการทดสอบ Test Case ID 8.2 ตรวจสอบการโพสต์เนื้อหา Step 1 กรอกเนื้อหา ดังภาพที่ ง-36



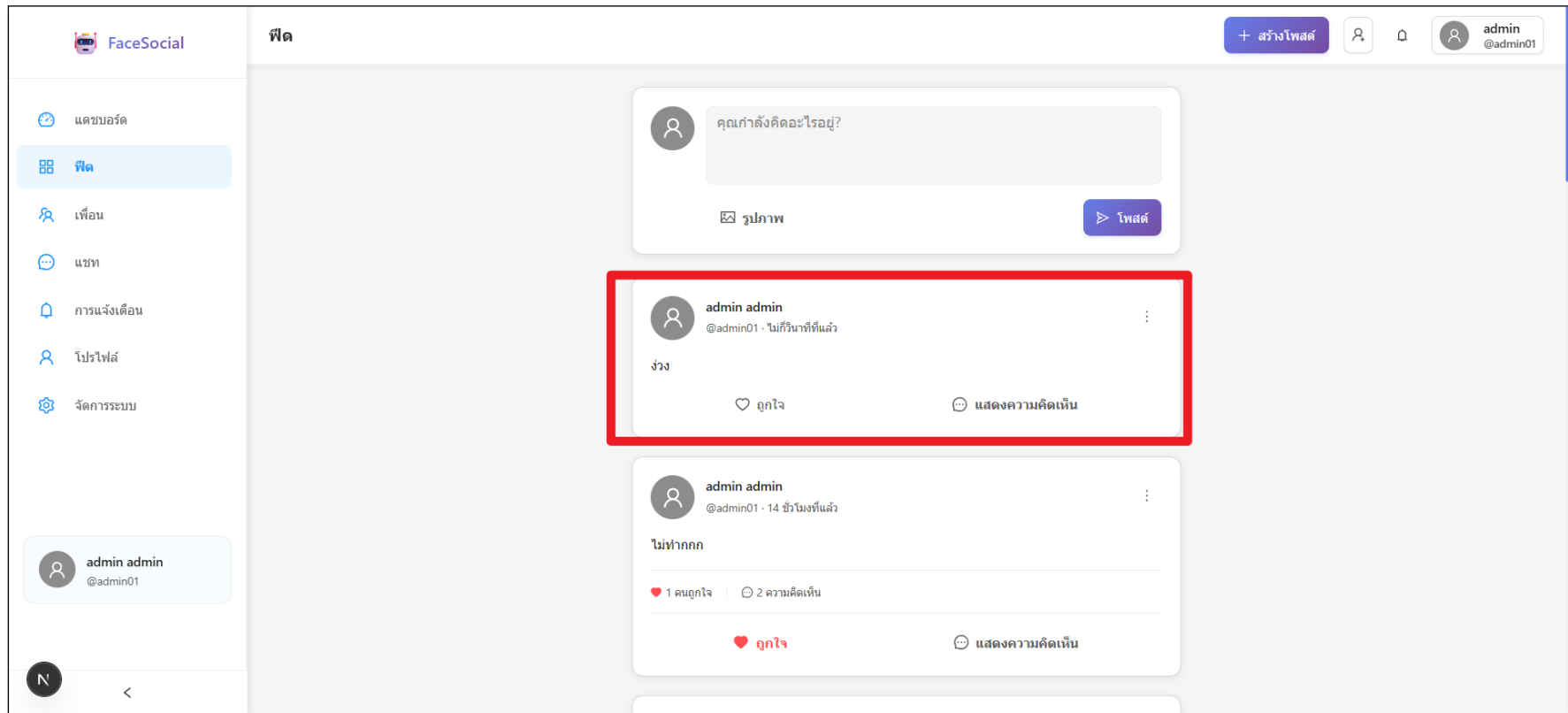
ภาพที่ ง-36 หน้าจอการกรอกเนื้อหาเพื่อทดสอบสร้างโพสต์

ง.1.1.8.4 ผลการทดสอบ Test Case ID 8.2 ตรวจสอบการโพสต์เนื้อหา Step 2 กดปุ่มโพสต์ ดังภาพที่ ง-37



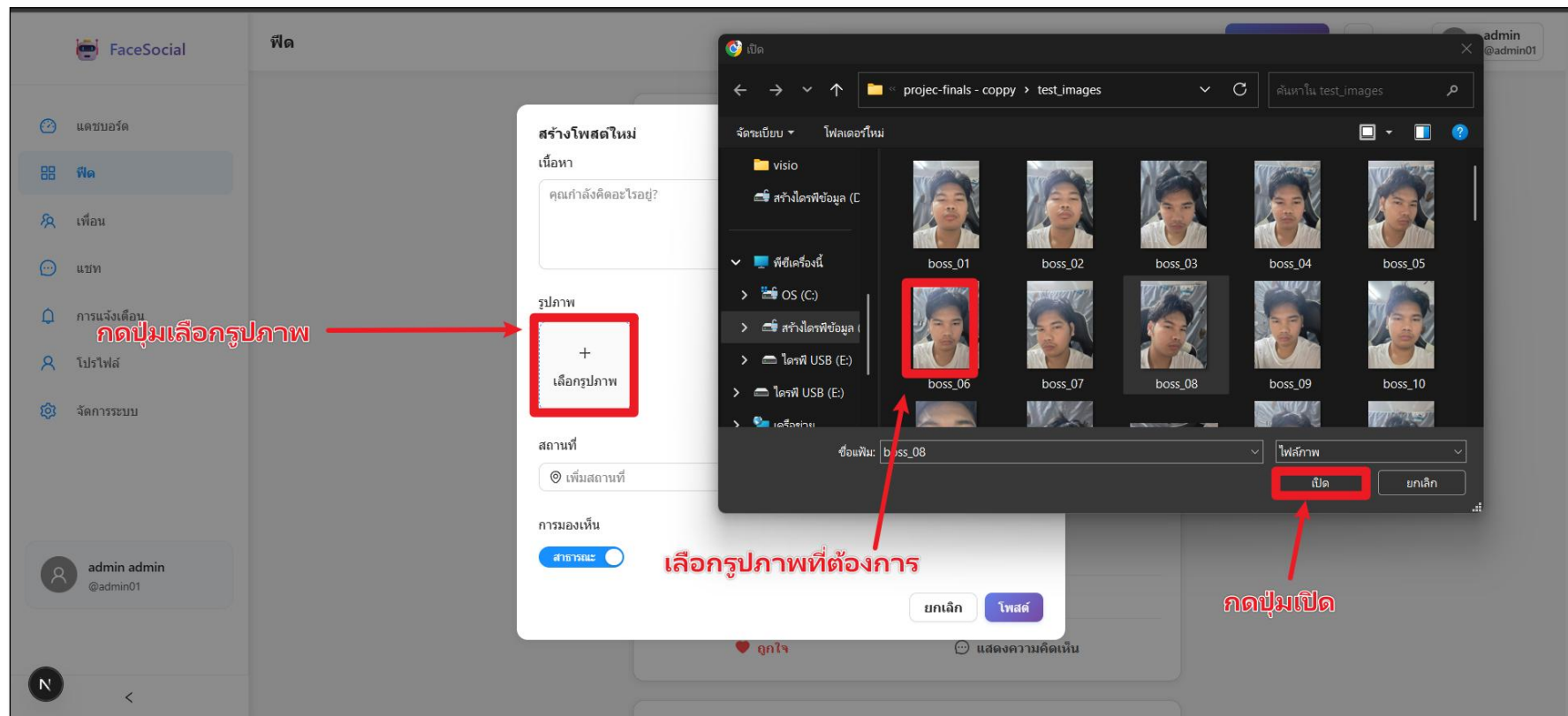
ภาพที่ ง-37 หน้าจอการกดปุ่มโพสต์

ง.1.1.8.5 ผลการทดสอบ Test Case ID 8.2 ตรวจสอบการโพสต์เนื้อหา ดังภาพที่ ง-38



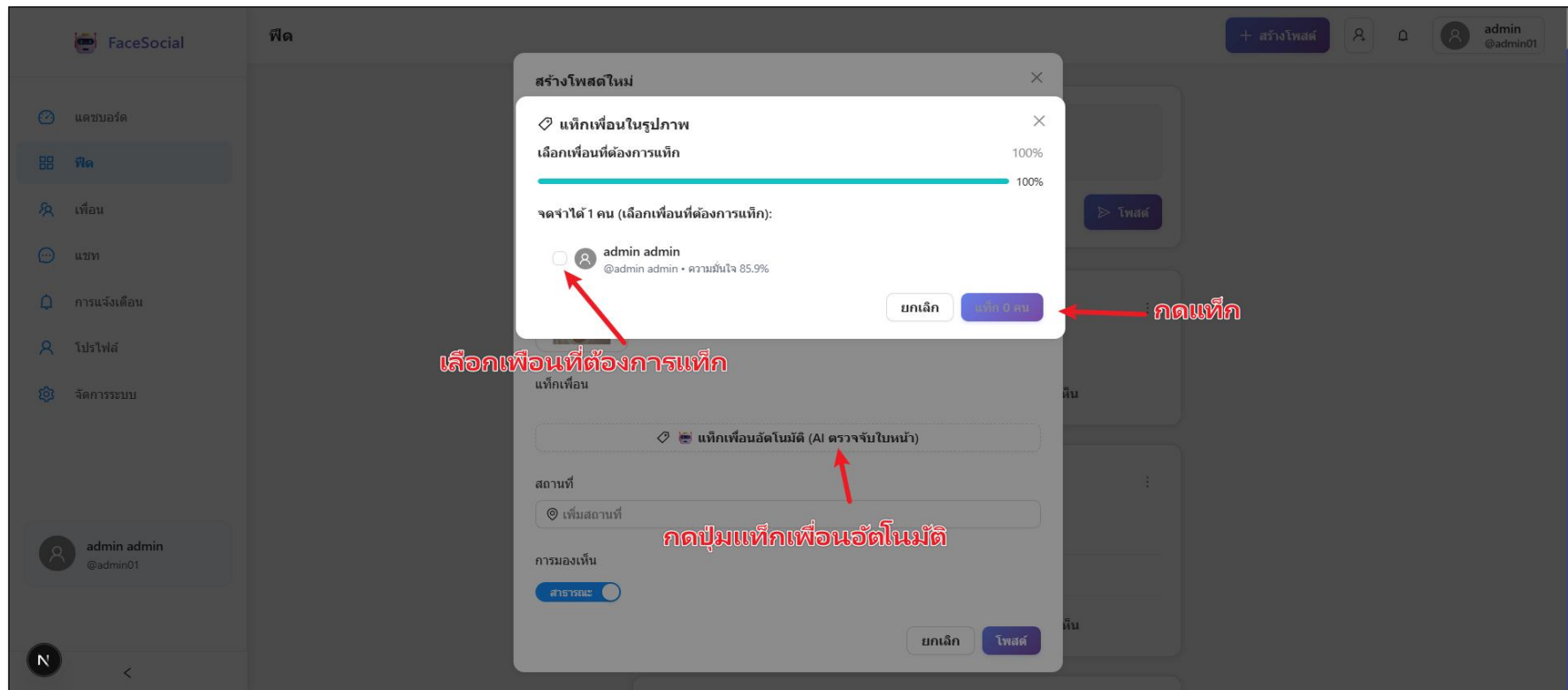
ภาพที่ ง-38 หน้าจอการสร้างโพสต์สำเร็จ

ง.1.1.8.6 ผลการทดสอบ Test Case ID 8.3 ตรวจสอบการโพสต์เนื้อหาและแท็กผู้ใช้ Step 1 เลือกรูปภาพ ดังภาพที่ ง-39



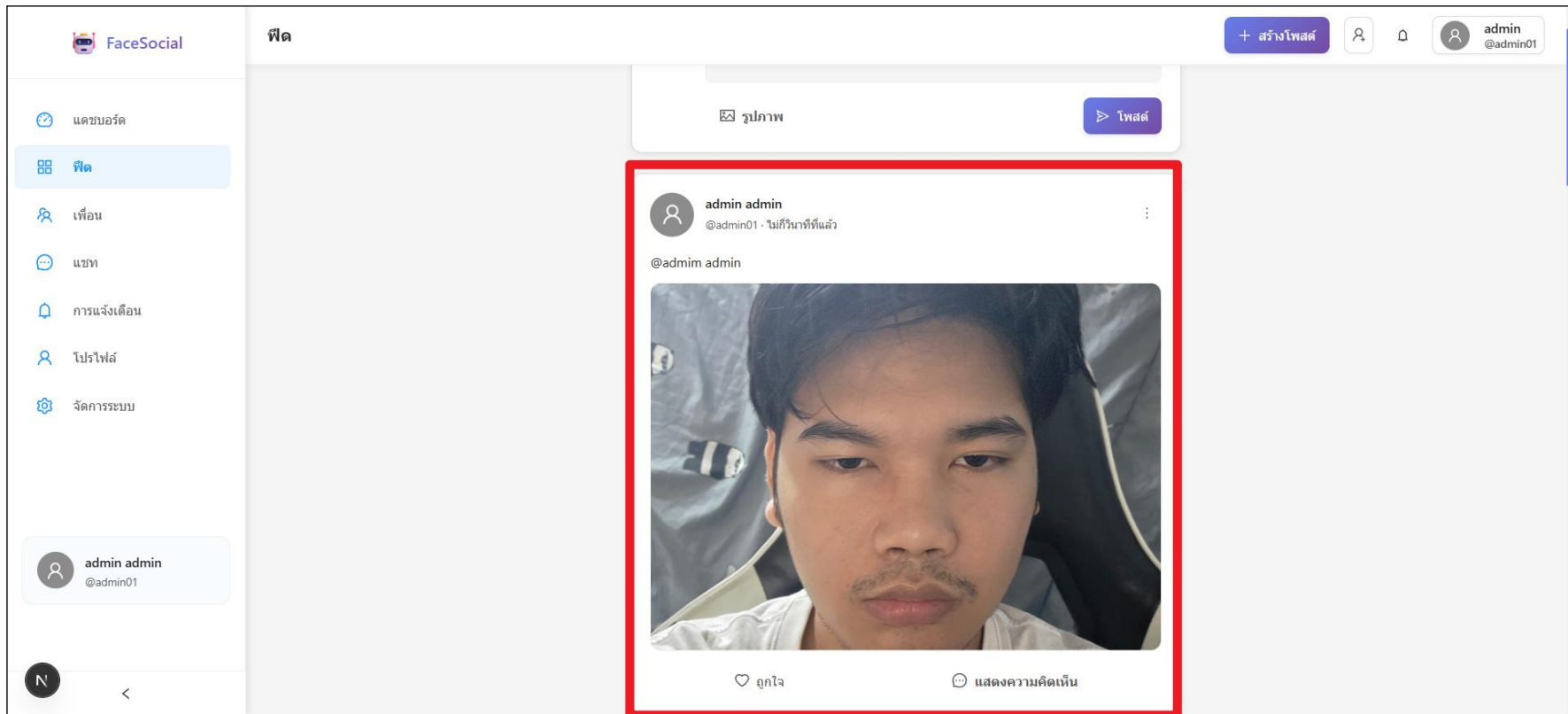
ภาพที่ ง-39 หน้าจอการเลือกรูปภาพ

ง.1.1.8.7 ผลการทดสอบ Test Case ID 8.3 ตรวจสอบการโพสต์เนื้อหาและแท็กผู้ใช้ Step 2 เลือกเพื่อนที่ต้องการแท็กจากการค้นหาอัตโนมัติ ดังภาพที่ ง-40



ภาพที่ ง-40 หน้าจอการแท็กเพื่อน

ง.1.1.8.8 ผลการทดสอบ Test Case ID 8.3 ตรวจสอบการโพสต์เนื้อหาและแท็กผู้ใช้ ดังภาพที่ ง-41



ภาพที่ ง-41 หน้าจอระบบแท็กผู้ใช้ที่มีในระบบอัตโนมัติ

ภาพผนวก จ

แบบฟอร์มประเมินผลการใช้งานแพลตฟอร์มสื่อสังคมออนไลน์และการจัดการ
รูปภาพบุคคลด้วยการจดจำใบหน้า

**แบบประเมินผลการทำงานของระบบ
แพลตฟอร์มสื่อสังคมออนไลน์ และการจัดการรูปภาพบุคคลด้วยการจดจำใบหน้า**

คำชี้แจง: โปรดทำเครื่องหมาย ✓ ลงในช่องว่างที่ตรงมากที่สุด

ขอให้พิจารณาข้อความต่อไปนี้ แล้วทำเครื่องหมาย ✓ ลงในช่องที่ตรงกับความคิดเห็นมากที่สุด

รายการประเมิน	ระดับคุณภาพในการทดสอบความ สมบูรณ์ของระบบที่ใช้งาน (ร้อยละ)				
	มาก ที่สุด	มาก	ปาน กลาง	น้อย	น้อย ที่สุด
1. ฟังก์ชันการทำงาน					
1.1 ความพึงพอใจในส่วนของหน้าหลัก					
1.2 ความพึงพอใจของส่วนสแกนหน้าเพื่อสมัคร					
1.3 ความพึงพอใจของส่วนสแกนหน้าเพื่อเข้าสู่ระบบ					
1.4 ความพึงพอใจของส่วนทำนายเพศ และอายุ					
1.5 ความพึงพอใจในระบบการตรวจจับใบหน้า					
1.6 ความพึงพอใจในระบบการตรวจสอบการปลอมแปลง					
1.7 ความพึงพอใจในระบบ กล้องวงจรปิดแบบ Real-time (CCTV Face Recognition)					
1.8 ความพึงพอใจในระบบ Chat					
1.9 ความพึงพอใจในระบบแท็กใบหน้าเพื่อน					
1.10 ความพึงพอใจในหน้า Dashboard					
1.11 ความพึงพอใจใช้ในระบบจดจำใบหน้า					
2. ประสิทธิภาพการใช้งานระบบ					
2.1 ความเหมาะสมในการใช้งานของระบบ					
2.2 ความเหมาะสมของขนาดตัวอักษร					
2.3 ความเหมาะสมของตำแหน่งในการใช้งานฟังก์ชัน					
2.4 ความง่ายต่อการใช้งานระบบ					

ข้อเสนอแนะอื่น ๆ :

.....

ประวัติผู้จัดทำโครงการ

ชื่อ-นามสกุล	: นายสุวิทย์ จันทเสน
รหัสนักศึกษา	: 6404022630308
อีเมล	: Suwit.chanthasen@gmail.com
ประวัติการศึกษา	: ระดับมัธยม โรงเรียนพรตพิทยพยัต ปัจจุบันศึกษาอยู่ที่ มหาวิทยาลัยเทคโนโลยี พระจอมเกล้าพระนครเหนือ
ผลงาน	: นำเสนอผลงานในงาน Science exhibition day : 2025 เข้าร่วมประกวดโครงการ The 13 th undergraduate in applied mathematics conference (UAMC2025)
ชื่อ-นามสกุล	: นายณัฐชนน ภูเวียง
รหัสนักศึกษา	: 6404022630103
อีเมล	: s6404022630103@email.kmutnb.ac.th
ประวัติการศึกษา	: ดั้มมัธยม โรงเรียนหอวัง ปทุมธานี ปัจจุบันศึกษาอยู่ที่มหาวิทยาลัยเทคโนโลยี พระจอมเกล้าพระนครเหนือ
ผลงาน	: นำเสนอผลงานในงาน Science exhibition day : 2025 เข้าร่วมประกวดโครงการ The 13 th undergraduate in applied mathematics conference (UAMC2025)